Laborator 01: Divide et Impera

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Radu Vișan (2018) [mailto:visanr95@gmail.com]
- Cristian Banu (2018) [mailto:cristb@gmail.com]
- Darius Neațu (2018) [mailto:neatudarius@gmail.com]

Obiective laborator

- Înțelegerea conceptului teoretic din spatele descompunerii unei probleme
- Rezolvarea de probleme abordabile folosind conceptul de Divide et Impera

Precizari initiale

Toate exemplele de cod se gasesc in demo-lab01.zip.

Exemplele de cod apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a acestuia.

- Toate bucatile de cod prezentate in partea introductiva a laboratorului (inainte de exercitii) au fost testate. Cu toate acestea, este posibil ca din cauza mai multor factori (formatare, caractere invizibile puse de browser etc) un simplu copy-paste sa nu fie de ajuns pentru a compila codul.
- Va rugam sa incercati si codul din arhiva **demo-lab01.zip**, inainte de a raporta ca ceva nu merge. :D
- Pentru orice problema legata de continutul acestei pagini, va rugam sa dati email unuia dintre responsabili.

Importanţă – aplicaţii practice

Paradigma Divide et Impera stă la baza construirii de algoritmi eficienți pentru diverse probleme:

- Sortări (ex: MergeSort [1] [http://www.sorting-algorithms.com/merge-sort], QuickSort [2] [http://www.sorting-algorithms.com/quick-sort])
- Înmulțirea numerelor mari (ex: Karatsuba [3] [http://en.wikipedia.org/wiki/Karatsuba_algorithm])
- Analiza sintactică (ex: parsere top-down [4] [http://en.wikipedia.org/wiki/Top-down_parser])
- Calcularea transformatei Fourier discretă (ex: FFT [5] [http://en.wikipedia.org/wiki/Fast_Fourier_transform])

Un alt domeniu de utilizare a tehnicii divide et impera este programarea paralelă pe mai multe procesoare, sub-problemele fiind executate pe mașini diferite.

Prezentarea generală a problemei

O descriere a tehnicii D&I: "Divide and Conquer algorithms break the problem into several sub-problems that are similar to the original problem but smaller in size, solve the sub-problems recursively, and then combine these solutions to create a solution to the original problem." [7]

Deci un algoritm D&I **împarte problema** în mai multe subprobleme similare cu problema inițială şi de dimensiuni mai mici, **rezolva subproblemele** recursiv şi apoi **combina soluțiile** obţinute pentru a obține

soluția problemei inițiale.

Sunt trei pași pentru aplicarea algoritmului D&I:

- **Divide**: împarte problema în una sau mai multe *probleme similare de dimensiuni mai mici*.
- **Impera** (stăpânește): rezolva subprobleme recursiv; dacă dimensiunea sub-problemelor este mica se rezolva iterativ.
- **Combină**: combină soluțiile subproblemelor pentru a obține soluția problemei inițiale.

Complexitatea algoritmilor D&I se calculează după formula:

$T(n) = D(n) + S(n) + C(n)$,

unde $D(n)$, $S(n)$ şi $C(n)$ reprezintă complexitățile celor 3 pași descriși mai sus: divide, stăpânește, respectiv combină.

Probleme clasice

MergeSort

Enunt

Sortarea prin interclasare (MergeSort [1] [http://www.sorting-algorithms.com/merge-sort]) este un algoritm de sortare de vectori ce folosește paradigma D&I:

- **Divide**: împarte vectorul inițial în doi sub-vectori de dimensiune n/2.
- **Stăpânește**: sortează cei doi sub-vectori recursiv folosind sortarea prin interclasare; recursivitatea se oprește când dimensiunea unui sub-vector este 1 (deja sortat).
- **Combina**: Interclasează cei doi sub-vectori sortați pentru a obține vectorul inițial sortat.

Pseudocod

Mai jos gasiti algoritmul MergeSort scris in pseudocod.

```
MergeSort(v, start, end)                // v – vector, start – limită inferioră, end – limită superioară
        if (start == end) return;       // condiția de oprire
        mid = (start + end) / 2;        // etapa divide
        MergeSort(v, start, mid);       // etapa stăpânește
        MergeSort(v, mid + 1, end);
        Merge(v, start, end);           // etapa combină

Merge(v, start, end)                    // interclasare sub-vectori
        // indecsi
        mid = (start + end) / 2;
        i = start;
        j = mid + 1;
        k = 1;

        tmp = buffer temporar in care incap (end - start + 1) numere
        while (i <= mid && j <= end)
                if (v[i] <= v[j]) tmp[k++] = v[i++];
                             else tmp[k++] = v[j++];

        while (i <= mid)
                tmp[k++] = v[i++];

        while (j <= end)
                tmp[k++] = v[j++];

        copy(v[start..end], tmp[1..k-1]);
```

Mai jos puteti gasi o implementare in C++.

```
#include <bits/stdc++.h>
using namespace std;

vector<int> v;

void merge_halves(int left, int right) {
        int mid = (left + right) / 2;
        vector<int> aux;
        int i = left, j = mid + 1;

        while (i <= mid && j <= right) {
                if (v[i] <= v[j]) {
                        aux.push_back(v[i]);
                        i++;
                } else {
                        aux.push_back(v[j]);
                        j++;
                }
        }

        while (i <= mid) {
                aux.push_back(v[i]);
                i++;
        }

        while (j <= right) {
                aux.push_back(v[j]);
                j++;
        }

        for (int k = left; k <= right; k++) {
                v[k] = aux[k - left];
        }
}


void merge_sort(int left, int right) {
        if (left >= right) return ;
        int mid = (left + right) / 2;

        merge_sort(left, mid);
        merge_sort(mid + 1, right);
        merge_halves(left, right);
}

int main() {
        random_device rd;
        for (int i = 0; i < 10; i++) {
                v.push_back(rd() % 100);
        }

        cout << "Vectorul initial: ";
        for (int i = 0; i < v.size(); i++) {
                cout << v[i] << " ";
        }
        cout << "\n";

        merge_sort(0, v.size() - 1);

        cout << "Vectorul sortat: ";
        for (int i = 0; i < v.size(); i++) {
                cout << v[i] << " ";
        }
        cout << "\n";

        return 0;
}
```

Complexitate

Complexitatea algoritmului este dată de formula: $T(n) = D(n) + S(n) + C(n)$, unde $D(n) = O(1)$, $S(n) = 2 * T(n/2)$ și $C(n) = O(n)$, rezulta $T(n) = 2 * T(n/2) + O(n)$.

Folosind teorema Master [8] [http://people.csail.mit.edu/thies/6.046-web/master.pdf] găsim complexitatea algoritmului: $T(n) = O(n * log(n))$.

- **complexitate temporala** : $T = O(n * log(n))$
  - Ce inseamna aceasta metrica? Masoara efectiv **timpul de executie** al algoritmului (nu include citiri, afisari etc).
- **complexitate spatiala** : $S = O(n)$
  - Ce inseamna aceasta metrica? Masoara efectiv **memoria suplimentara** folosita de algoritm (in acest caz ne referim strict la buffer-ul temporar).

Retineti cele doua conventii despre complexitati de mai sus. Le vom folosi pentru restul semestrului.

Binary Search

Enunt

Se dă un **vector sortat crescător** ($v[1]$, $v[2]$, ..., $v[n]$) ce conține valori reale distincte și o valoare x.

Sa se găsească la ce **poziție** apare x în vectorul dat.

Rezolvare

BinarySearch (Cautare Binara), se rezolva cu un algoritm D&I:

- **Divide**: împărțim vectorul în doi sub-vectori de dimensiune n/2.
- **Stăpânește**: aplicăm algoritmul de căutare binară pe sub-vectorul care conține valoarea căutată.
- **Combină**: soluția sub-problemei devine soluția problemei inițiale, motiv pentru care nu mai este nevoie de etapa de combinare.

Pseudocod

```
BinarySearch(v, left, right, x) {             // functia returneaza pozitia x pe care se afla numarul x (oricare pozitie) sa
       // vom cauta cat timp intervalul de cautare nu a fost inca epuizat (are cel putin un element)
       while (left <= right) {
            mid = (left + right) / 2          // mijlocul intervalului de cautare

            if (x == v[mid]) return mid;      // elementul cautat este cel din mijloc
            if (x < v[mid])  right = mid - 1; // elementul cautat este mai mic decat cel din mijloc, ne mutam in prima jumatate
            if (x > v[mid])  left  = mid + 1; // elementul cautat este mai mare decat cel din mijloc, ne mutam in a doua jumatate
        }

        return -1;                            // in acest punct ajungem daca si numai daca x nu a fost gasit
}
```

Complexitate

- **complexitate temporala** : $T = O(log(n))$
  - se deduce din recurenta $T(n) = T(n/2) + O(1)$
- * **complexitate spatiala** : $S = O(1)$
  - nu avem structuri de date complexe auxiliare
  - atragem atentia ca acest algoritm se poate implementa si **recursiv**, caz in care complexitatea spatiala devine $O(log(n))$, intrucat salvam pe stiva $O(log(n))$ parametri (intregi, referinte)

Turnurile din Hanoi

Enunt

Se considera 3 tije S (**sursa**), D (**destinatie**), aux (**auxiliar**) şi n discuri de dimensiuni distincte ($1, 2, \ldots, n$ - ordinea crescătoare a dimensiunilor) situate iniţial toate pe tija S în ordinea $1, 2, \ldots, n$ (de la vârf către baza).

Singura operaţie care se poate efectua este de a selecta un disc ce se află în vârful unei tije şi plasarea lui în vârful altei tije astfel încât să fie aşezat deasupra unui disc de dimensiune mai mare decât a sa.

Sa se găsească un algoritm prin care se mută toate discurile de pe tija S pe tija D (problema turnurilor din Hanoi).

Solutie

Pentru rezolvarea problemei folosim următoarea strategie [9] [http://www.mathcs.org/java/programs/Hanoi/index.html]:

- mutam primele $n-1$ discuri de pe tija S pe tija aux folosindu-ne de tija D.
- mutam discul n pe tija D.
- mutam apoi cele $n-1$ discuri de pe tija aux pe tija D folosindu-ne de tija S.

Ideea din spate este ca avem mereu o singura sursa si o singura destinatie sa atingem un scop. Intotdeauna a 3-a tija va fi considerata auxiliara si poate fi folosita pentru a atinge scopul propus.

Algoritm

```
// muta n discuri de pe tija S pe tija D folosind tija aux
Hanoi(n, S, D, aux)  {
    if (n >= 1) {
        Hanoi(n - 1, S, aux, D);   // mut n-1 discuri de pe sursa (S) pe auxiliar (aux)
                                   // in aceasta subproblema sursa este S, destinatia este aux, intermediarul este D

        Muta_disc(S, D);           // acum pot muta direct discul n de pe sursa (S) pe destinatie (D)

        Hanoi(n - 1, aux, D, S);   // mut n-1 discuri de pe sursa (aux, aici sunt ele momentan) pe destinatie (D - scop final)
                                   // in aceasta subproblema, S este auxiliar, intrucat este tija libera
    }
}
```

Complexitate

- **complexitate temporala** : $T(n) = O(2^n)$
  - se deduce din recurenta $T(n) = 2 * T(n-1) + O(1)$
- * **complexitate spatiala** : $S(n) = O(n)$
  - la un moment dat, nivelul maxim de recursivitate este n

ZParcurgere

Enunt

Gigel are o tabla patratica de dimensiuni $2^n * 2^n$. Ar vrea sa scrie pe patratelele tablei numere naturale cuprinse intre 1 si $2^n * 2^n$ conform unei parcurgeri mai deosebite pe care o numeste **Z-parcurgere**.

O Z-parcurgere viziteaza recursiv cele patru cadrane ale tablei in ordinea: **stanga-sus**, **dreapta-sus**, **stanga-jos**, **dreapta-jos**.

La un moment dat Gigel ar vrea sa stie ce **numar de ordine** trebuie sa scrie conform Z-parcurgerii pe anumite patratele date prin coordonatele lor (x, y). Gigel incepe umplerea tablei **intotdeauna** din coltul din stanga-sus.

$n = 1$ si $(x, y) = (2, 2)$

Raspuns: 4

Explicatie: Matricea arata ca in exemplul urmator.

1	2
3	4

$n = 2$ si $(x, y) = (3, 3)$

Raspuns: 13

Explicatie: Matricea arata ca in exemplul urmator.

1	2	5	6
3	4	7	8
9	10	13	14
11	12	15	16

Solutie

Analizand modul in care se **completeaza** tabloul/matricea din enunt, observam ca la fiecare etapa impartim matricea (**problema**) in 4 submatrici (**4 subprobleme**). De asemenea, sirul de numere pe care dorim sa il punem in matrice se imparte in 4 secvente, fiecare corespunzand unei submatrici.

Observam astfel ca problema suporta **descompunerea** in **subprobleme disjuncte** si cu **structura similara**, ceea ce ne face sa ne gandim la o solutie cu Divide et Impera.

Complexitate

- **complexitate temporala** : $T = O(n)$
  - $\log_4(2^n) = \frac{1}{2}\log_2(2^n) = \frac{1}{2}n$
- **complexitate spatiala** : $S = O(n)$
  - stocam parametri pentru recursivitate
  - solutia se poate implementa si iterativ, caz in care $S = O(1)$; deoarece dimensinile spatiului de cautare sunt $2^n * 2^n$, n este foarte mic ($n <= 15$), de aceea o solutie iterativa nu aduce nici un castig **efectiv** in aceasta situatie

Concluzii

Divide et impera este o tehnică folosită pentru a realiza solutii pentru o anumita clasa de probleme: acestea contin **subprobleme disjuncte** si cu **structura similara**. În cadrul acestei tehnici se disting trei etape: divide, stăpânește și combină.

Mai multe exemple de algoritmi care folosesc tehnica divide et impera puteți găsi la [11] [http://www.cs.berkeley.edu/~vazirani/algorithms/chap2.pdf].

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab01.zip.

Count occurrences

Se da un sir sortat **v** cu **n** elemente. Gasiti numarul de elemente egale cu **x** din sir.

$n = 6$ si $x = 10$

i	1	2	3	4	5	6
v	1	2	4	10	10	20

Raspuns: 2

Explicatie: 10 apare de 2 ori in sir.

Task-uri:

- Aceasta problema este deja rezolvata. Pentru a va acomoda cu scheletul, va trebui sa faceti cativa pasi:
  - Rulati comanda $./check.sh$ si cititi cum se foloseste checker-ul.
  - Rulati comanda necesara pentru a rula task-ul 1. Sursa nu implementeaza corect algoritmul si returneaza valori default. Din acest motiv primiti mesajul **WRONG ANSWER**.
  - Copiati urmatoarea sursa in folderul corespunzator. Rulati comanda anterioara. Observati mesajele afisate cand ati rezolvat corect un task.

Sursa **main.cpp** asociata cu task 1 este mai jos.

```
#include <bits/stdc++.h>
using namespace std;

class Task {
public:
        void solve() {
                read_input();
                print_output(get_result());
        }

private:
        void read_input() {
                ifstream fin("in");
                fin >> n;
                for (int i = 0, e; i < n; i++) {
                        fin >> e;
                        v.push_back(e);
                }
                fin >> x;
                fin.close();
        }

        int find_first() {
                int left = 0, right = n - 1, mid, res = -1;
                while (left <= right) {
                        mid = (left + right) / 2;
                        if (v[mid] >= x) {
                                res = mid;
                                right = mid - 1;
                        } else {
                                left = mid + 1;
                        }
                }
                return res;
        }

        int find_last() {
                int left = 0, right = n - 1, mid, res = -1;
                while (left <= right) {
                        mid = (left + right) / 2;
                        if (v[mid] <= x) {
                                res = mid;
                                left = mid + 1;
                        } else {
                                right = mid - 1;
                        }
                }
                return res;
        }

        int get_result() {
                int first = find_first();
                int last = find_last();
                if (first == -1 || last == -1) {
                        return 0;
                }
```

```
                return last - first + 1;
        }

        void print_output(int result) {
                ofstream fout("out");
                fout << result;
                fout.close();
        }

        int n, x;
        vector<int> v;
};

int main() {
        Task task;
        task.solve();
        return 0;
}
```

Sursa **Main.java** asociata cu task 1 este mai jos.

```
import java.io.File;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;

public class Main {
        static class Task {
                public final static String INPUT_FILE = "in";
                public final static String OUTPUT_FILE = "out";

                int n;
                int[] v;
                int x;

                private void readInput() {
                        try {
                                Scanner sc = new Scanner(new File(INPUT_FILE));
                                n = sc.nextInt();
                                v = new int[n];
                                for (int i = 0; i < n; i++) {
                                        v[i] = sc.nextInt();
                                }
                                x = sc.nextInt();
                                sc.close();
                        } catch (IOException e) {
                                throw new RuntimeException(e);
                        }
                }

                private void writeOutput(int count) {
                        try {
                                PrintWriter pw = new PrintWriter(new File(OUTPUT_FILE));
                                pw.printf("%d\n", count);
                                pw.close();
                        } catch (IOException e) {
                                throw new RuntimeException(e);
                        }
                }

                private int findFirst() {
                        int left = 0, right = n - 1, mid, res = -1;
                        while (left <= right) {
                                mid = (left + right) / 2;
                                if (v[mid] >= x) {
                                        res = mid;
                                        right = mid - 1;
                                } else {
                                        left = mid + 1;
                                }
                        }
                        return res;
                }

                private int findLast() {
```

```
                        int left = 0, right = n - 1, mid, res = -1;
                        while (left <= right) {
                                mid = (left + right) / 2;
                                if (v[mid] <= x) {
                                        res = mid;
                                        left = mid + 1;
                                } else {
                                        right = mid - 1;
                                }
                        }
                        return res;
                }

                private int getAnswer() {
                        int first = findFirst();
                        int last = findLast();
                        if (first == -1 || last == -1) {
                                return 0;
                        }
                        return last - first + 1;
                }

                public void solve() {
                        readInput();
                        writeOutput(getAnswer());
                }
        }

        public static void main(String[] args) {
                new Task().solve();
        }
}
```

- Intelegeti solutia oferita impreuna cu asistentul vostru.
- Care este complexitatea solutiei (timp + spatiu)? De ce?

SQRT

Se da un numar real **n**. Scrieti un algoritm de complexitate **O(log n)** care sa calculeze $sqrt(n)$ cu o precizie de 0.001.

$n = 0.25$

Raspuns: orice valoare intre 0.499 si 0.501 (inclusiv)

Pentru a putea trece testele, trebuie sa afisati rezultatul cu **cel putin** 4 zecimale.

ZParcurgere

Rezolvati problema ZParcurgere folosind scheletul pus la dispozitie. Enuntul si explicatiile le gasiti in partea de seminar.

Exponentiere logaritmica

Se dau doua numere naturale **base** si **exponent**. Scrieti un algoritm de complexitate $O(log(exponent))$ care sa calculeze $base^{exponent} \ \% \ MOD$.

Intrucat expresia $base^{exponent}$ este foarte mare, dorim sa aflam doar **restul** impartirii lui la un numar **MOD**.

Proprietati matematice necesare:

- $(a + b) \ \% \ MOD = ((a \ \% \ MOD) + (b \ \% \ MOD)) \ \% \ MOD$
- $(a \ * b) \ \% \ MOD = ((a \ \% \ MOD) \ * (b \ \% \ MOD)) \ \% \ MOD$

Atentie la inmultire! Rezultatul **temporar** poate provoca un overflow. Solutii:

- **C++**: $a * b \Rightarrow 1LL * a * b$
- **Java**: $a * b \Rightarrow 1L * a * b$

$base = 2$, $exponent = 10$, $MOD = 5$

Raspuns: 4

Explicatie: $2^{10} \ \% \ 5 = 4$

Bonus

Se da un sir S de n numere intregi. Sa se detemine cate inversiuni sunt in sirul dat. Numim inversiune o pereche de indici $1 <= i < j <= n$ astfel incat $S[i] > S[j]$

Exemplu: in sirul `{0 1 9 4 5 7 6 8 2}` sunt 12 inversiuni.

Aceasta problema nu are schelet.

Testați soluția voastră de la **Exponentiere logaritmica** pe infoarena, la problema ClassicTask [https://infoarena.ro/problema/classictask] (trebuie să modificați numele fișierelor).

Identificați problema și modificați sursa astfel încât să luați punctaj maxim.

Extra

Se dă un vector de numere întregi neordonate. Scriind o funcție de partitionare, folosiți **Divide et Impera** pentru

- a determina a k-lea element ca mărime din vector
- a sorta vectorii prin QuickSort

Puteti testa problema partition aici [https://leetcode.com/problems/kth-largest-element-in-an-array/description/]. Problema QuickSort [https://en.wikipedia.org/wiki/Quicksort] (chiar si MergeSort) poate fi testata aici [https://infoarena.ro/problema/algsort].

Exemplu: pentru vectorul `{0 1 2 4 5 7 6 8 9}`, al 3-lea element ca ordine este 2, iar vectorul sortat este {0 1 2 4 5 6 7 8 9}

Se dau $n - 1$ numere naturale distincte intre 0 si $n - 1$. Scriind o functie de partitionare, determinati numarul lipsa.

Exemplu: pentru $n = 9$ si vectorul 019457682, numarul lipsa este 3.

Puteti rezolva aceasta problema pe infoarena [https://infoarena.ro/problema/fractal].

Puteti rezolva aceasta problema de aici [https://ocw.cs.pub.ro/courses/pa/laboratoare/laborator-03#ssm].

Pentru problema SSM vom studia o solutie potrivita in lab03. Puteti sa incercati sa o rezolvati cu Divide et Impera pentru a exersa cele invatate astazi.

Dându-se N numere întregi sub forma unei secvenţe de numere strict crescătoare, care se continuă cu o secvenţă de întregi strict descrescătoare, se doreşte determinarea punctului din întregul şir înaintea căruia toate elementele sunt strict crescătoare, şi dupa care, toate elementele sunt strict descrescătoare. Considerăm

evident faptul că acest punct nu există dacă cele N numere sunt dispuse într-un şir fie doar strict crescător, fie doar strict descrescător.

pa/laboratoare/laborator-01.txt · Last modified: 2020/03/03 21:52 by darius.neatu

Laborator 02 : Greedy

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Alex Rotaru (2018) [mailto:rotarualexandruandrei94@gmail.com]
- Darius Neațu (2018) [mailto:neatudarius@gmail.com]
- Radu Vișan (2018) [mailto:visanr95@gmail.com]
- Cristian Banu (2018) [mailto:cristb@gmail.com]

Obiective laborator

- Înțelegerea noțiunilor de bază legate de tehnica greedy
- Însușirea abilităților de implementare a algoritmilor bazați pe greedy

Precizari initiale

Toate exemplele de cod se gasesc in arhiva demo-lab02.zip.

Acestea apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a laboratorului.

- Toate bucatile de cod prezentate in partea introductiva a laboratorului (inainte de exercitii) au fost testate. Cu toate acestea, este posibil ca din cauza mai multor factori (formatare, caractere invizibile puse de browser etc) un simplu copy-paste sa nu fie de ajuns pentru a compila codul.
- Va rugam sa incercati si codul din arhiva **demo-lab02.zip**, inainte de a raporta ca ceva nu merge. :D
- Pentru orice problema legata de continutul acestei pagini, va rugam sa dati email unuia dintre responsabili.

Importanță – aplicații practice

În general tehnicile de tip Greedy sau Programare Dinamică (lab04) sunt folosite pentru rezolvarea problemelor de optimizare. Acestea pot adresa probleme în sine sau pot fi subprobleme dintr-un algoritm mai mare. De exemplu, algoritmul Dijkstra pentru determinarea drumului minim pe un graf alege la fiecare pas un nod nou urmărind algoritmul greedy.

Exista însă probleme care ne pot induce în eroare. Astfel, există probleme în care urmărind criteriul Greedy nu ajungem la soluția optimă. Este foarte important să identificăm cazurile când se poate aplica Greedy și cazurile când este nevoie de altceva. Alteori această soluție neoptimă este o aproximare suficientă pentru ce avem nevoie. Problemele NP-complete necesita multă putere de calcul pentru a găsi optimul absolut. Pentru a optimiza aceste calcule mulți algoritmi folosesc decizii Greedy și găsesc un optim foarte aproape de cel absolut.

Greedy

"greedy" = "lacom". Algoritmii de tip greedy vor să construiască într-un mod cât mai rapid soluția unei probleme. Ei se caracterizează prin luarea unor decizii rapide care duc la găsirea unei soluții potențiale a problemei. Nu întotdeauna asemenea decizii rapide duc la o soluție optimă; astfel ne vom concentra atenția pe identificarea acelor anumite tipuri de probleme pentru care se pot obține soluții optime. În general exista mai multe soluții posibile ale problemei. Dintre acestea se pot selecta doar anumite soluții optime, conform unor anumite criterii. Algoritmii greedy se numără printre cei mai direcți algoritmi posibili. Ideea de bază este simplă: având o problema de optimizare, de calcul al unui cost minim sau maxim, se va alege la fiecare pas decizia cea mai favorabilă, fără a evalua global eficiența soluţiei. Scopul este de a găsi una dintre acestea sau dacă nu este posibil, atunci o soluție cât mai apropiată, conform criteriului optimal impus.

Trebuie înțeles faptul ca rezultatul obținut este optim doar dacă un optim local conduce la un optim global. În cazul în care deciziile de la un pas influențează lista de decizii de la pasul următor, este posibila obținerea unei valori neoptimale. În astfel de cazuri, pentru găsirea unui optim absolut se ajunge la soluții supra-polinomiale. De aceea, dacă se optează pentru o astfel de soluție, algoritmul trebuie însoțit de o demonstrație de corectitudine. Descrierea formală a unui algoritm greedy este următoarea:

```
// C este mulțimea candidaților
function greedy(C) {
    S ← ∅ // în S construim soluția

    while !solutie(C) and C ≠ ∅
        x ← un element din C care minimizează/maximizează select(x)
        C ← C \ {x}
        if fezabil( S ∪ {x}) then S ← S∪{x}

    return S
}
```

Este ușor de înțeles acum de ce acest algoritm se numește "greedy": la fiecare pas se alege cel mai bun candidat de la momentul respectiv, fără a studia alternativele disponibile în moment respectiv şi viabilitatea acestora în timp.

Dacă un candidat este inclus în soluție, rămâne acolo, fără a putea fi modificat, iar dacă este exclus din soluție, nu va mai putea fi niciodată selectat drept un potențial candidat.

Exemple

Simple task

Enunt

Fie un șir de N numere pentru care se cere determinarea unui subșir de numere cu suma maximă. Un subșir al unui șir este format din elemente (nu neapărat consecutive) ale șirului respectiv, în ordinea în care acestea apar în șir.

Pentru numerele $1, -5, 6, 2, -2, 4$ răspunsul este $1, 6, 2, 4$ (suma 13).

Solutie

Se observa ca tot ce avem de făcut este sa verificam fiecare număr dacă este pozitiv sau nu. În cazul pozitiv, îl introducem în subșirul soluție.

Daca toate numerele sunt negative, solutia este data de cel mai mare numar negativ (cel mai mic in modul).

Problema spectacolelor

Enunt

Se dau mai multe spectacole, prin timpii de start și timpii de final. Se cere o planificare astfel încât o persoană să poată vedea cât mai multe spectacole.

Solutie

Rezolvarea constă în **sortarea** spectacolelor crescător după timpii de final, apoi la fiecare pas se alege **primul spectacol** care are timpul de start mai mare decât ultimul timp de final. Timpul inițial de final este inițializat la $-\inf$ (spectacolul care se termină cel mai devreme va fi mereu selectat, având timp de start mai mare decât timpul inițial).

Implementare

```
bool end_hour_comp (pair<int, int>& e1, pair<int, int>& e2) {
    // comparam doar dupa ora de sfarsit
    return (e1.second < e2.second);
}

vector<pair<int, int>> plan(vector<pair<int, int> >& intervals) {
    vector<pair<int, int>> plan;
    // se sorteaza intervalele pe baza orei de sfarsit a spectacolelor
    sort(intervals.begin(), intervals.end(), end_hour_comp);

    // se ia ultimul spectacol ca terminat la -oo pt a putea incepe cu
    // cel mai devreme
    int last_end = INT_MIN; // -oo a.k.a -infinit
    for (auto interval : intervals) {
        // daca inceputul intervalului curent este dupa sfarsitul ultimului
        // spectacol (last_end) il adaugam in lista de spectacole la care
        // se participa
        if (interval.first >= last_end)
        {
            plan.push_back(interval);
            // dupa ce am adaugat un spectacol, updatam ultimul sfarsit de spectacol
            last_end = interval.second;
        }
    }
    return plan;
}
```

Complexitate

Solutia va avea urmatoarele complexitati:

- **complexitate temporala** : $T(n) = O(n * log(n))$
  - explicatie
    - sortarea are $O(n * log(n))$
    - facem inca o parcurgere in $O(n)$
- **complexitate spatiala** : depinde de algoritmul de sortare folosit.

Problema florarului

Enunt

Se da un grup de k oameni care vor sa cumpere impreuna n flori. Fiecare floare are un pret de baza, insa pretul cu care este cumparata variaza in functie de numarul de flori cumparate anterior de persoana respectiva. De exemplu daca George a cumparat 3 flori (diferite) si vrea sa cumpere o floare cu pretul 2, el

va plati $(3 + 1) * 2 = 8$. Practic el va plati un pret proportional cu numarul flori cumparate pana atunci tot de el.

Cerinta: Se cere pentru un numar k de oameni si n flori sa se determine care este costul minim cu care grupul poate sa achizitioneze toate cele n flori o singura data.

Observatie: Un tip de floare se cumpara o singura data. O persoana poate cumpara mai multe tipuri de flori. In final in grup va exista un singur exemplar din fiecare tip de floare.

Formal avem k numar de oameni, n numar de flori, $c[i]$ = pretul florii de tip i, costul de cumparare i va fi $(x + 1) * c[i]$, unde x este numarul de flori cumparate anterior de persoana respectiva.

n=3 k=3 c=[2 5 6]

Cost minim = 13

Explicatie: Fiecare individ cumpara cate o floare si deci acestea se cumpara la pretul nominal.

Solutie

Se observa ca pretul efectiv de cumpare va fi mai mare cu cat cumparam acea floare mai tarziu. Daca consideram cazul in care avem o singura persoana in grup observam ca are sens sa cumparam obiectele in ordine descrescatoare(deoarece vrem sa minimizam costul fiecarui tip de flori si aceste creste cu cat cumparam floarea mai tarziu).

De aici, gandindu-ne la versiunea cu k persoane, observam ca ar fi mai ieftin daca am repartiza urmatoarea cea mai scumpa floare la alt individ. Deci impartim florile sortate descrescator dupa pret in grupuri de cate k, fiecare individ luand o floare din acest grup si ne asiguram ca pretul va creste doar in functie de numarul de grupuri anterioare.

Implementare

```
struct greater_comparator
{
    template<class T>
    bool operator()(T const &a, T const &b) const { return a > b; }
};

int minimum_cost(int k, vector<int>& costs) {
    // sortam vectorul de preturi in ordine descrescatoare
    sort(costs.begin(), costs.end(), greater_comparator());

    // numarul de flori cumparate de fiecare individ din grup la un moment dat
    int x = 0;
    // o varianta mai putin eficienta spatial ar fi fost sa retinem pt fiecare
    // individ din grup numarul de flori cumparate intr-un hashmap
    // costul total
    int total_cost = 0;

    // parcurgem fiecare pret de floare si o "asignam" unui individ din grup
    // pretul acesteia fiind proportional cu numarul de achizitii facut pana acum
    // de acesta (x)
    for (int idx = 0; idx < costs.size(); idx++) {
        int customer_idx = idx % k;
        total_cost += (x + 1) * costs[idx];
        // in momentul in care ultimul individ a cumparat o floare din grupul curent
        // incrementam numarul de flori achizitionate de fiecare
        if (customer_idx == k - 1) {
            x += 1;
        }
    }
    return total_cost;
}
```

Complexitate

Solutia va avea urmatoarele complexitati:

- **complexitate temporala** : $T(n) = O(n * log(n))$
  - explicatie
    - sortarea are $O(n * log(n))$
    - facem inca o parcurgere in $O(n)$
- **complexitate spatiala** : depinde de algoritmul de sortare folosit. Fara partea de sortare, spatiul este constant (nu se ia in considerare vectorul de elemente).

Problema cuielor

Enunt

Fie N scânduri de lemn, descrise ca niște **intervale închise** cu capete reale. Găsiți o mulțime **minimă** de cuie astfel încât fiecare scândură să fie bătută de cel puțin un cui. Se cere poziția cuielor.

Formulat matematic: găsiți o mulțime de puncte de cardinal **minim** M astfel încât pentru orice interval [a_i, b_i] din cele N, să existe un punct x din M care să aparțină intervalului [a_i, b_i].

- intrare: N = 5, intervalele: [0, 2], [1, 7], [2, 6], [5, 14], [8, 16]
- ieșire: M = {2, 14}
- explicație: punctul 2 se afla în primele 3 intervale, iar punctul 14 în ultimele 2

Solutie

Se observa că dacă x este un punct din M care nu este capăt dreapta al nici unui interval, o translație a lui x la dreapta care îl duce în capătul dreapta cel mai apropiat nu va schimba intervalele care conțin punctul. Prin urmare, exista o mulțime de cardinal minim M pentru care toate punctele x sunt capete dreapta.

Astfel, vom crea mulțimea M folosind numai capete dreapta în felul următor:

- sortăm intervalele dupa capatul dreapta
- iterăm prin fiecare interval și dacă intervalul curent nu conține ultimul punct introdus în mulțime atunci îl adăugam pe acesta la mulțime

Implementare

```
bool point_in_interval(const pair<int, int>&interval, int point) {
    return point >= interval.first && point <= interval.second;
}

bool right_edge_comparator (pair<int, int>& e1, pair<int, int>& e2) {
    // comparam scandurile dupa capatul drepata
    return (e1.second < e2.second);
}

vector<int> cover_intervals_greedy(vector<pair<int, int>>& intervals) {
    vector<int> nails; // pozitiile cuielor, a.k.a multimea M
    // ultimul punct inserat
    int last_point = INT_MIN;

    //sortam invervalele dupa capatul drepata
    sort(intervals.begin(), intervals.end(), right_edge_comparator);
```

```
    for (auto interval : intervals) {
        // daca intervalul nu contine ultimul punct adaugat
        if (!point_in_interval(interval, last_point)) {
            // il adaugam in multimea M
            nails.push_back(interval.second);
            // updatam ultimul punt inserat
            last_point = interval.second;
        }
    }

    return nails;
}
```

Complexitate

Soluția va avea următoarele complexități:

- **complexitate temporala** : $T(n) = O(n * log(n))$
  - explicație
    - sortare: $O(n * logn)$
    - parcurgerea intervalelor: $O(n)$
- **complexitate spațială** : depinde de algoritmul de sortare folosit.

Concluzii şi observații

Aspectul cel mai important de reținut este că soluțiile găsite trebuie să reprezinte optimul global și nu doar local. Se pot confunda ușor problemele care se rezolvă cu Greedy cu cele care se rezolvă prin Programare Dinamică (vom vedea saptamana viitoare).

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab02.zip.

Rucsac

Fie un set cu n obiecte (care pot fi **taiate** - varianta continua a problemei). Fiecare obiect i are asociata o pereche (w_i, p_i) cu semnificatia:

- w_i = $weight_i$ = greutatea obiectului cu numarul i
- p_i = $price_i$ = pretul obiectului cu numarul i
  - $w_i >= 0$ si $p_i > 0$

Gigel are la dispozitie un rucsac de **volum infinit**, dar care suporta o **greutate maxima** (notata cu W - weight knapsack).

El vrea sa gaseasca **o submultime de obiecte** (nu neaparat intregi) pe care sa le bage in rucsac, astfel incat suma profiturilor sa fie **maxima**.

Daca Gigel baga in rucsac obiectul i, caracterizat de (w_i, p_i), atunci profitul adus de obiect este p_i (presupunem ca il vinde cu cat valoareaza).

In aceasta varianta a problemei, Gigel poate taia oricare dintre obiecte, obtinand o proportie din acesta. Daca Gigel alege alege doar x din greutatea w_i a obiectului i, atunci el castiga doar $\frac{x}{w_i} * p_i$.

Task-uri:

- Sa se determine profitul maxim pentru Gigel.

- Care este complexitatea solutiei (timp + spatiu)? De ce?

obiecte:

index	0	1	2
greutate	60	100	120
valoare	10	20	30

greutate = 50

Output: 12.5 Explicatie: avem 50 capacitate si toate obiectele au o greutatate mai mare, decidem sa luam cat putem din produsul cu raportul valoare / greutate cel mai mare. profitu = 30 / 120 * 50 = 12.5

obiecte:

index	0	1	2
greutate	20	50	30
valoare	60	100	120

greutate = 50

Output: 180 Explicatie: Sortam obiectele dupa raportul valoare profit si avem in ordine: {30, 120}, {20, 60}, {50, 100} Introducem obiecte pana cand umplem sacul ⇒ intra primele 2 obiecte. Calculam profitul 120 + 60 = 180

Distante

Consideram 2 localitati A si B aflate la distanta D. Intre cele 2 localitati avem un numar de n benzinarii, date prin distanta fata de localitatea A. Masina cu care se efectueaza deplasarea intre cele 2 localitati poate parcurge maxim m kilometri avand rezervorul plin la inceput. Se doreste parcurgerea drumului cu un numar minim de opriri la benzinarii pentru realimentare (dupa fiecare oprire la o benzinarie, masina pleaca cu rezervorul plin).

Distantele catre benzinarii se reprezinta printr-o lista de forma $0 < d1 < d2 < \ldots < dn$, unde di ($1 <= i <= n$) reprezinta distanta de la A la benzinaria i. Pentru simplitate, se considera ca localitatea A se afla la 0, iar $dn = D$ (localitatea B se afla in acelasi loc cu ultima benzinarie).

Se garanteaza ca exista o planificare valida a opririlor astfel incat sa se poata ajunge la localitatea B.

$n = 5$

$m = 10$

$d = (2, 8, 15, 25, 30)$

Raspunsul este 3, efectuand 3 opriri la a 2-a, a 3-a, respectiv a 4-a benzinarie.

Teme la ACS

Pe parcursul unui semestru, un student are de rezolvat n teme (nimic nou pana aici…). Se cunosc enunțurile tuturor celor n teme de la **începutul semestrului**.

Timpul de rezolvare pentru oricare dintre teme este de **o săptămână** și **nu** se poate lucra la mai multe teme în același timp. Pentru fiecare tema se cunoaște un termen limita $d[i]$ (exprimat în săptămâni - deadline pentru tema i) și un punctaj $p[i]$.

Nicio fracțiune din punctaj nu se mai poate obține după expirarea termenului limită.

Task-uri:

- Să se definească o planificare de realizare a temelor, în așa fel încât punctajul obținut să fie **maxim**.
- Care este complexitatea solutiei (timp + spatiu)? De ce?

index	0	1	2	3	4
deadline	6	6	2	7	7
punctaj	5	4	1	5	8

Output: $1 + 4 + 5 + 5 + 8 = 23$

Explicatie: Putem face toate temele deoarece pana ajungem la deadline-urile lor avem suficiente unitati de timp.

index	0	1	2	3	4	5	6	7
deadline	3	3	3	3	9	11	11	11
punctaj	4	9	6	5	10	4	2	6

Output: $5 + 6 + 9 + 10 + 2 + 4 + 6 = 42$

Explicatie: Pana in deadline 3 avem la dispozitie 3 unitati de timp si 4 teme. Deci sortam dupa punctaj si le includem pe cele mai valoroase: 5, 6 ,9. Pana la deadline 9 avem la dispoztie 6 unitati de timp si 4 teme. Le includem pe toate.

BONUS

Rezolvati problema Dishonest Sellers [http://codeforces.com/problemset/problem/779/C].

Hint: aici [http://codeforces.com/blog/entry/50724].

Extra

Incercati problema MaxSum [https://www.hackerrank.com/contests/test-practic-pa-2017-v1-plumbus/challenges/1-1-usoare] de la test PA 2017.

Problema 1 de la tema PA 2017. Puteti descarca enuntul si checkerul de aici [https://ocw.cs.pub.ro/courses/_media/pa/teme/pa2017_tema1.zip].

Problema 3 de la tema PA 2017. Puteti descarca enuntul si checkerul de aici [https://ocw.cs.pub.ro/courses/_media/pa/teme/pa2017_tema1.zip].

Referințe

[0] Capitolul **Greedy Algorithms** din **Introductions to Algorithms** de către T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein

[1] http://en.wikipedia.org/wiki/Greedy_algorithm [http://en.wikipedia.org/wiki/Greedy_algorithm]

[2] http://ww3.algorithmdesign.net/handouts/Greedy.pdf [http://ww3.algorithmdesign.net/handouts/Greedy.pdf]

Laborator 3: Programare Dinamică

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Darius Neațu (2018) [mailto:neatudarius@gmail.com]
- Radu Vișan (2018) [mailto:visanr95@gmail.com]
- Cristian Banu (2018) [mailto:cristb@gmail.com]
- Răzvan Chițu (2018) [mailto:razvan.ch95@gmail.com]

Obiective laborator

- Înțelegerea noțiunilor de bază despre programare dinamică
- Insușirea abilităților de implementare a algoritmilor bazați programare dinamică.

Precizari initiale

Toate exemplele de cod se gasesc in demo-lab03.zip.

Acestea apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a laboratorului.

- Toate bucatile de cod prezentate in partea introductiva a laboratorului (inainte de exercitii) au fost testate. Cu toate acestea, este posibil ca din cauza mai multor factori (formatare, caractere invizibile puse de browser etc) un simplu copy-paste sa nu fie de ajuns pentru a compila codul.
- Va rugam sa incercati si codul din arhiva **demo-lab03.zip**, inainte de a raporta ca ceva nu merge.
- Pentru orice problema legata de continutul acestei pagini, va rugam sa dati email unuia dintre responsabili.

Ce este DP?

Similar cu greedy, tehnica de programare dinamica este folosită in general pentru rezolvarea **problemelor de optimizare**. In continuare vom folosi acronimul **DP (dynamic programming)**.

De asemenea, DP se poate folosi si pentru probleme in care nu cautam un optim, cum ar fi **problemele de numarare** (vom exemplifica in lab04).

Aplicatii DP

Programarea dinamică are un **câmp larg de aplicare**, insa la PA ne vom rezuma la cateva aplicatii care vor fi mentionate pe parcursul laboratoarelor 3 si 4. De asemenea, aceasta tehnica va fi folosita si in laboratoarele de grafuri (ex. algoritmul Floyd-Warshall - pe care il veti implementa si la PA; algoritmi pe arbori etc).

Programare dinamică presupune rezolvarea unei probleme prin **descompunerea ei în subprobleme** şi rezolvarea acestora. Spre deosebire de divide et impera, subproblemele nu sunt disjuncte, ci **se suprapun**.

Pentru a evita recalcularea porțiunilor care se suprapun, rezolvarea se face pornind de la cele mai mici subprobleme şi folosindu-ne de rezultatul acestora calculăm subproblema imediat mai mare. Cele mai mici subprobleme sunt numite subprobleme unitare, acestea putând fi rezolvate într-o complexitate constantă, ex: cea mai mare subsecvență dintr-o mulțime de un singur element.

Pentru a nu recalcula soluțiile subproblemelor ce ar trebui rezolvate de mai multe ori, pe ramuri diferite, se reține soluția subproblemelor folosind o tabelă (matrice uni, bi sau multi-dimensională în funcție de problemă) cu rezultatul fiecărei subprobleme. Aceasta tehnica se numește **memoizare**.

Această tehnică determina ”valoarea” soluției pentru fiecare din subprobleme. Mergând de la subprobleme mici la subprobleme din ce în ce mai mari ajungem la soluția optimă, la nivelul întregii probleme. ”valoarea” is schimba înțelesul logic de la o problema la alta. În probleme de minimizarea costului, ”valoarea” este reprezentata de costul minim. In probleme care presupun identificarea unei componente maxime, ”valoarea” este caracterizată de dimensiunea componentei.

După calcularea valorii pentru toate subproblemele se poate determina efectiv mulțimea de elemente care compun soluția. „Reconstrucția” soluţiei se face mergând din subproblemă în subproblemă, începând de la problema cu valoarea optimă și ajungând în subprobleme unitare. Metoda și recurența variază de la problemă la problemă, dar în urma unor exerciții practice va deveni din ce în ce mai facil să le identificați.

Cei curiosi pot citi aici [https://en.wikipedia.org/wiki/Dynamic_programming#History] adevarul despre numele acestei tehnici. 😎

Ce determina DP?

Aplicând aceasta tehnică determinăm **una din soluțiile optime**, problema putând avea mai multe soluții optime. În cazul în care se dorește determinarea tuturor soluțiilor optime, algoritmul trebuie combinat cu unul de backtracking în vederea construcției soluțiilor.

In cazul problemelor de numarare, aceasta tehnica ne va furniza **numarul cautat**.

Tipar general DP

Aplicarea acestei tehnici de programare poate fi descompusă în următoarea secvență de pași:

1. Identificarea structurii și a metricilor utilizate în caracterizarea soluției optime;
2. Determinarea unei metode de calcul recursiv pentru a afla valoarea fiecărei subprobleme;
3. Calcularea “bottom-up” a acestei valori (de la subproblemele cele mai mici la cele mai mari);
4. Reconstrucția soluției optime pornind de la rezultatele obținute anterior.

Exemple clasice

Programarea Dinamică este cea mai flexibilă tehnica din programare. Cel mai ușor mod de a o înțelege presupune parcurgerea cât mai multor exemple.

Propunem cateva categorii de recurente, pe care le vom grupa astfel:

- recurente de tip **SSM** (Subsecventa de Suma Maxima)
- recurente de tip **RUCSAC**
- recurente de tip **PODM** (Parantezare Optima de Matrici)
- recurente de tip **numărat**
- recurente pe **grafuri**

Pentru o problema data, este **posibil** sa gasim **mai multe recurente corecte** (mai multe solutii posibile). Evident, criteriul de alegere intre acestea va fi cel bazat pe complexitate.

Categoria 1: SSM

Aceste recurente au o oarecare asemanare cu problema SSM (enunt + solutie).

SSM

Enunt

Fie un vector v cu n elemente intregi. O subsecventa de numere din sir este de forma: $v_i, v_{i+1}, \ldots, v_j$ ($i <= j$), avand suma asociata $s_{ij} = v_i + v_{i+1} + \ldots + v_j$. O subsecventa **nu** poate fi vida.

Cerinta

Sa se determine subsecventa de suma maxima (notata **SSM**).

Exemple

n = 6

i	1	2	3	4	5	6
v[i]	-10	2	3	-1	2	-3

Raspuns: SSM este intre 2 si 5 (pozitii). Are suma +6. ($SSM = 2, 3, -1, 2$)

Explicatie: avem numere pozitive, deci exista o solutie simpla in care putem sa alegem doar un numar pozitiv/mai multe numere pozitive de pe pozitii alaturate (adica incercam sa evitam numere negative). Cele mai lungi subsecvente cu numere pozitive sunt 2,3 si 2. Observam ca daca extindem $2, 3$ la $2, 3, -1, 2$, desi am inclus un numar negativ, suma secventei creste.

n = 4

i	1	2	3	4
v[i]	10	20	30	40

Raspuns: SSM este intre 1 si 4 (pozitii). Are suma 100. ($SSM = 10, 20, 30, 40$)

Explicatie: deoarece toate numerele sunt **pozitive**, SSM cuprinde toate numerele.

n = 4

i	1	2	3	4
v[i]	-10	-20	-30	-40

Raspuns: SSM este intre 1 si 1 (pozitii). Are suma -10. ($SSM = -10$)

Explicatie: deoarece toate numerele sunt **negative**, SSM cuprinde doar cel mai mare numar.

Rezolvare

Tipar

Tiparul acestei probleme ne sugereaza ca o solutie este obtinuta incremental, in sensul ca **putem** privi problema astfel: gasim cea mai buna solutie folosind **primele** $i-1$ elemente din sir, apoi incercam sa o **extindem** folosind elementul **i** (adica ne extindem la dreapta ~CU~ $v[i]$).

Numire recurenta

Intrucat la fiecare pas trebuie sa retinem **cea mai buna solutie** folosind un **prefix** din vectorul v, solutia va fi salvata intr-un tablou auxiliar definit astfel:

$dp[i]$ = suma subsecventei de suma maxima (**suma SSM**) folosind **doar primele i** elemente din vectorul v si care se termina pe pozitia i

Mentiuni

- Pentru a mentine o conventie, toate tablourile de acest tip din laborator vor fi notate cu **dp** (dynamic programming).
- Ca sa rezolvam problema data, trebuie sa rezolvam o multime de subprobleme
  - $dp[i]$ reprezinta **solutia** pentru problema $v[1], \ldots, v[i]$ si care se termina cu $v[i]$
- Solutia pentru problema initiala este maximul din vectorul $dp[i]$.

Gasire recurenta

Intrucat dorim ca aceasta problema sa fie rezolvabila printr-un algoritm/bucata de cod, trebuie sa descriem o metoda concreta prin care vom calcula $dp[i]$.

- **Cazul de baza**
  - In general in probleme putem avea mai multe cazuri de baza, care in principiu se leaga de valori extreme are dimensiunilor subproblemelor.
  - In cazul SSM, avem un singur caz de baza, cand avem un singur element in prefix: $dp[1] = v[1]$.
  - Explicatie: daca avem un singur element, atunci acesta formeaza singura subsecventa posibila, deci $SSM = v[1]$

- **Cazul general**
- presupune inductiv ca avem rezolvate toate subproblemele mai mici
- in cazul SSM, presupunem ca avem calculat $dp[i-1]$ si dorim sa calculam $dp[i]$ (cunoastem cea mai buna solutie folosind primele i-1 elememente si vedem daca elementul de pe pozitia i o poate imbunatati)
- la fiecare pas avem de ales daca $v[i]$ extinde cea mai buna solutie care se termina pe $v[i-1]$ sau se incepe o noua secventa cu $v[i]$
- decidem in functie de $dp[i-1]$ si $v[i]$
  - **daca** $dp[i-1] >= 0$ (cea mai buna solutie care se termina pe i - 1 are cost nenegativ)
    - extindem secventa care se termina cu v[i-1] folosind elementul v[i]: $dp[i] = dp[i-1] + v[i]$
    - Explicatie: $dp[i-1] + v[i] >= v[i]$ (inca are rost sa extind)
  - **daca** $dp[i-1] < 0$ (cea mai buna solutie care se termina pe i - 1 are cost negativ)
    - vom incepe o noua secventa cu $v[i]$, adica $dp[i] = v[i]$
    - Explicatie: $v[i] > dp[i-1] + v[i]$, deci prin extindere nu obtin solutie maxima!

Implementare recurenta

In majoritatea problemelor de DP, gasirea recurentei ocupa cea mai mare parte a timpului de rezolvare (lucru adevarat si in cazul problemelor de la PA). De aceea, faptul ca ati reusit sa scrieti pe foaie lucruri foarte complicate poate fi un indiciu ca ati pornit pe o cale gresita.

Mai jos se afla un exemplu simplu de implementare a recurentei gasite in C++.

```
// gaseste SSM pentru vectorul v cu n elemente
// pentru a mentine conventia din explicatii:
//      - elementele sunt indexate de la 0, dar le folosesc doar pe cele care incep de la 1
//                                          => v[1], ..., v[n]
int SSM(int n, vector<int> &v) {
        vector<int> dp(n + 1);    // vector cu n + 1 elemente (indexarea incepe de la 0)
                                  // am nevoie de dp[1], ..., dp[n]

        // caz de baza
        dp[1] = v[1];

        // caz general
        for (int i = 2; i <= n; ++i) {
                if (dp[i - 1] >= 0) {
                        // extinde la dreapta cu v[i]
                        dp[i] = dp[i - 1] + v[i];
                } else {
                        // incep o noua secventa
                        dp[i] = v[i];
                }
        }

        // solutia e maximul din vectorul dp
        int sol = dp[1];
        for (int i = 2; i <= n; ++i) {
                if (dp[i] > sol) {
                        sol = dp[i];
                }
        }

        return sol; // aceasta este suma asociata cu SSM
}
```

Daca dorim sa afisam si indicii intre care apare SSM, putem sa stocam si pozitia de start pentru fiecare solutie intermediara. Gasiti aceasta solutie in **demo-lab03.zip**. Hint: definiti **start[i]** = pozitia pe care a inceput subsecventa care da solutia cu cost dp[i].

Mentiuni

Intrucat aceasta solutie presupune calculul iterativ (coloana cu coloana) a matricei dp, complexitatea este liniara. De asemenea, se mai parcurge o data dp pentru a gasi maximul.

- **complexitate temporala** : $T = O(n)$
- **complexitate spatiala** : $S = O(n)$
  - desigur ca pentru problema SSM, nu era nevoie sa retinem, tablourile dp/start in memorie.
  - puteam sa construim element cu element si maximul din dp in aceleasi timp (intrucat ne trebuie ultima valoare la fiecare pas si maximul global).
  - in acest caz complexitatea spatiala devine $S = O(1)$

Pentru a ilustra toti pasii posibili intr-o astfel de problema, totul a fost prezentat cat mai simplu (NU in toate problemele putem facem simplificari de tipul "NU am nevoie sa stochez tabloul dp").

SCMAX

Enunt

Fie un vector v cu n elemente intregi. Un subsir de numere din sir este de forma: $v_{i_1}, v_{i_2}, \ldots, v_{i_k}$. Un subsir **nu** poate fi vid ($k >= 1$).

Cerinta

Sa se determine subsirul crescator maximal (notat **SCMAX**) - un subsir ordonat strict crescator si are lungime maxima (daca sunt mai multe solutii, sa se gaseasca una oarecare).

Exemple

n = 6

i	1	2	3	4	5	6
v[i]	100	12	13	-1	15	-30

Raspuns: $SCMAX = 12, 13, 15$ ($SCMAX = v[2], v[3], v[5]$).

Explicatie: Toate subsirurile ordonate strict crescator sunt:

- 100
- 12
- $12, 13$
- $12, 13, 15$
- $12, 15$
- 13
- $13, 15$
- -1
- $-1, 15$
- 15
- -30

Cel mentionat este singurul de lungime 3.

n = 6

i	1	2	3	4	5	6
v[i]	100	12	13	-1	15	14

Raspuns:

- $SCMAX = 12, 13, 15$ ($SCMAX = v[2], v[3], v[5]$).
- $SCMAX = 12, 13, 14$ ($SCMAX = v[2], v[3], v[6]$).

Explicatie: Toate subsirurile ordonate strict crescator sunt:

- 100
- 12
- $12, 13$
- $12, 13, 15$
- $12, 13, 14$
- 13
- $13, 15$
- $13, 14$
- -1
- $-1, 15$

- $-1, 14$
- 15
- 14

Cele 2 solutii indicate au ambele lungime maxima.

Rezolvare

Tipar

Verificam daca se aplica tiparul de la SSM: **gasim cea mai buna solutie folosind primele $i-1$ elemente din sir, apoi incercam sa o extindem folosind elementul i (adica ne extindem la dreapta ~CU~ $v[i]$)**.

- Daca avem cea mai buna solutie pentru intervalul $1, 2, . . , i-1$ si care se termina cu $v[i-1]$, atunci incercam sa extindem solutia cu $v[i]$ (putem daca $v[i-1] < v[i]$)
- Altfel.. Unde am putea sa il punem pe $v[i]$?
  - Pai am putea sa incercam sa il punem la finalul solutiei care se termina pe $v[i-2]$, $v[i-3]$, … sau $v[1]$

Numire recurenta

$dp[i]$ = lungimea celui mai lung subsir(**lungime SCMAX**) folosind (doar o parte) din primele i elemente din vectorul v si care se termina pe pozitia i

Mentiuni

- Ca sa rezolvam problema data, trebuie sa rezolvam o multime de subprobleme
  - $dp[i]$ reprezinta **solutia** pentru problema $v[1], \ldots, v[i]$ si care se termina cu $v[i]$
- Solutia pentru problema initiala este maximul din vectorul $dp[i]$.

Gasire recurenta

- **Cazul de baza**
  - Si in problema SCMAX, cazul pentru $i = 1$ este caz de baza.
    - daca avem un singur element, atunci avem o singura subsecventa de lungime 1, ea este solutia
    - $dp[1] = 1$
- **Cazul general**
  - presupune inductiv ca avem rezolvate toate subproblemele mai mici
  - in cazul SCMAX, presupunem ca avem calculate $dp[1], dp[2], \ldots, dp[i-1]$ si dorim sa calculam $dp[i]$ (cunoastem cea mai buna solutie folosind primele j elemente si vedem daca elementul de pe pozitia i o poate imbunatati - $j = 1 : i-1$)
  - deoarece nu stim unde e cel mai bine sa il pune pe $v[i]$ (dupa care v[j]?), incercam pentru toate valorile posibile ale lui j (unde $j = 1 : n-1$)
    - **daca** $v[j] < v[i]$, atunci subsirul crescator care se termina pe pozitia j, poate fi extins la dreapta cu elementul v[i], generand lungimea **dp[j] + 1**
      - deci dp[i] = max(dp[j] + 1), $j = 1 : i-1$ (daca nu exista un astfel de j, valoarea lui max(…) este 0)
    - Ce se intampla totusi daca nu exista un j care sa indeplineasca conditia de mai sus? Atunci $v[i]$ va forma singur un subsir crescator de lungime 1 (care poate fi la un pas

ulterior)

Reunind cele spuse mai sus:

- $dp[1] = 1$
- $dp[i] = 1 + max(dp[j])$, unde $j = 1 : i - 1$ **și** $v[j] < v[i]$

Implementare recurenta

Mai jos se afla un exemplu simplu de implementare a recurentei gasite in C++.

```
// n   = numarul de elemente din vector
// v   = vectorul dat (v[1], v[2], ..., v[n] - indexare de la 1 ca in explicatii)

void scmax(int n, vector<int> &v) {
        vector<int> dp(n + 1);   // in explicatii indexarea incepe de la 1

        // caz de baza
        dp[1] = 1;   // [ v[1] ] este singurul subsir (crescator) care se termina pe 1

        // caz general
        for (int i = 2; i <= n; ++i) {
                dp[i] = 1;   // [ v[i] ] - este un subsir (crescator) care se termina pe i

                // incerc sa il pun pe v[i] la finalul tuturor solutiilor disponibile
                // o solutie se termina cu un element v[j]
                for (int j = 1; j < i; ++j) {
                        // solutia triviala: v[i]
                        if (v[j] < v[i]) {
                                // din (..., v[j]) pot obtine (..., v[j], v[i])
                                // (caz in care prec[i] = j)

                                // voi alege j-ul curent, cand alegerea imi gaseste o solutie mai buna decat ce am deja
                                if (dp[j] + 1 > dp[i]) {
                                        dp[i] = dp[j] + 1;
                                }
                        }
                }
        }

        // solutia e maximul din vectorul dp
        int sol = dp[1], pos = 1;
        for (int i = 2; i <= n; ++i) {
                if (dp[i] > sol) {
                        sol = dp[i];
                        pos = i;
                }
        }

        return sol;
}
```

In **demo-lab03.zip** gasiti un exemplu de implementare care arata si cum puteti reconstitui SCMAX. Fata de implementarea anterioara, in aceasta versiune se foloseste un tablou auxiliar prec.

$prec[i]$ = indicele j al elementului v[j], pentru care $dp[j] + 1 == dp[i]$ (adica acel j pentru care subsirul crescator maximal care se termina cu $v[i]$ este extinderea cu un element a celui care se termina cu $v[j]$.

- daca nu exista un astfel de j, atunci $prec[i] = 0$ (prin conventie)

Mentiuni

Intrucat aceasta solutie presupune calculul iterativ (coloana cu coloana) a matricei dp, complexitatea este polinomiala (patratica - pentru fiecare element din tabloul, facem o trecere prin elementele deja calculate).

- **complexitate temporala** : $T = O(n^2)$
  - se poate obtine o solutie in complexitate $T = O(nlogn)$ daca se foloseste o cautare binara pentru a gasi elementul j dorit (ex. implemetare [https://www.infoarena.ro/job_detail/1248867?action=view-source]).
- **complexitate spatiala** : $S = O(n)$
  - NU putem obtine o complexitate spatiala mai buna, intrucat avem nevoie sa stocam cel putin vectorul dp (stocam si vectorul prec daca avem nevoie sa reconstituim SCMAX)

Categoria 2: RUCSAC

Aceste recurente au o oarecare asemanare cu problema RUCSAC - varianta discreta (enunt + solutie).

RUCSAC

Enunt

Fie un set (vector) cu n obiecte (care nu pot fi taiate - varianta discreta a problemei). Fiecare obiect i are asociata o pereche (w_i, p_i) cu semnificatia:

- w_i = $weight_i$ = greutatea obiectului cu numarul i
- p_i = $price_i$ = pretul obiectului cu numarul i
  - $w_i >= 0$ si $p_i > 0$

Gigel are la dispozitie un rucsac de **volum infinit**, dar care suporta o **greutate maxima** (notata cu W - weight knapsack).

El vrea sa gaseasca **o submultime de obiecte** pe care sa le bage in rucsac, astfel incat **suma profiturilor sa fie maxima**.

Daca Gigel baga in rucsac obiectul i, caracterizat de (w_i, p_i), atunci profitul adus de obiect este p_i (presupunem ca il vinde cu cat valoreaza obiectul).

Cerinta

Sa se determine **profitul maxim** pentru Gigel.

Exemple

$n = 5$ si $W = 10$

	1	2	3	4	5
w	3	3	1	1	2
p	6	3	2	8	5

Raspuns: **24** (profitul maxim)

Explicatie: va alege toate obiectele :D.

$n = 5$ si $W = 3$

	1	2	3	4	5
w	3	3	1	1	2

p	6	3	2	8	5

Raspuns: **13** (profitul maxim)

Explicatie: va alege obiectele cu indicii 4 si 5 (profit: 8 + 5)

Rezolvare

Tipar

Cum am transpune tiparul de la SSM/SCMAX in problema RUCSAC?

- stim care este profitul maxim pe care il obtine daca folosim
  - doar primul element
  - doar primele 2 elemente
  - …
  - doar primele $i-1$ elemente
- ajung sa ma gandesc la obiectul (elementul) i
  - este posibil ca acesta sa nu apara neaparat in solutia cea mai buna, caz in care nu il folosesc, deci solutia maxima se gaseste intre cele mentionate mai sus
  - daca folosesc elementul i caracterizat de (w_i, p_i), in primul rand acesta trebuie sa incapa in ghiozdan…
    - cum verific acest lucru?
    - o recurenta de tipul $dp[i] = \ldots$ nu va fi suficienta, pentru ca in aceasta problema am 2 dimensiuni: **obiectele** (submultimile de indici) si **greutatile** (asociate cu obiectele / submultimile de obiecte).

Numire recurenta

Intrucat la fiecare pas trebuie sa retinem **cea mai buna solutie** folosind un **prefix** din vectorul de obiecte, dar pentru ca trebuie sa punem si **o restrictie de greutate** necesara (ocupata in rucsac), solutia va fi salvata intr-un tablou auxiliar definit astfel:

$dp[i][cap]$ = profitul maxim (**profit RUCSAC**) obtinut folosind (doar o parte) din primele i obiecte si avand un rucsac de **capacitate maxima cap**

Observatii:

- NU exista restrictie daca in solutia mentionata de $dp[i][cap]$ este folosit OBLIGATORIU elementul i
- Solutia problemei se gaseste in $dp[n][W]$ (profitul maxim folosind (doar o parte) din primele n elemente - adica toate; capacitatea maxima folosita este W - adica capacitatea maxima a rucsacului).

Gasire recurenta

- **Cazul de baza**
  - Daca avem o submultime vida de obiecte selectate.
    - $dp[0][cap] = 0$
    - Explicatie: Daca nu alegem obiecte, atunci profitul este 0 indiferent de capacitate.
- **Cazul general**
  - $dp[i][cap] = ?$
  - presupune inductiv ca avem rezolvate toate subproblemele mai mici

- subprobleme mai mici inseamna sa foloseasca mai putine obiecte sau un rucsac cu capacitatea mai mica
- vedem daca prin folosirea obiectului i, obtinem cea mai buna solutie in $dp[i][cap]$
  - **NU folosesc obiectul i**
    - in acest caz, o sa alegem cea mai buna solutie formata cu celelalte $i-1$ elemente si aceeasi capacitate a rucsacului
    - solutia generata de acest caz: $dp[i][cap] = dp[i-1][cap]$
  - **folosesc obiectul i**
    - daca il folosesc, inseamna ca pentru el trebuie sa am rezervata in rucsac o capacitate egala cu w_i
      - adica cand am selectat dintre primele $i-1$ elemente, nu trebuia sa ocup mai mult de $cap - w_i$ din capacitatea rucsacului
      - fata de subproblema mentionata, castig in plus p_i (profitul pe care il aduce acest obiect
    - solutia generata de acest caz: $dp[i][cap] = dp[i-1][cap - w_i] + p_i$

Reunind cele spuse mai sus, obtinem:

- $dp[0][cap] = 0$, pentru $cap = 0 : G$
- $dp[i][cap] = max(dp[i-1], cap], dp[i-1][cap - w_i] + p_i)$
  - pentru $i = 1 : n$, $cap = 0 : W$

Implementare recurenta

Mai jos se afla un exemplu simplu de implementare a recurentei gasite in C++.

```
// n   = numarul de obiecte din colectie
// W   = capacitatea maxima a rucsacului
// (w[i], p[i]) = caracteristicile obiectului i ($i = 1 : n)

int rucsac(int n, int W, vector<int> &w, vector<int> &p) {
    // dp este o matrice de dimensiune (n + 1) x (W + 1)
    // pentru ca folosim dp[0][*] pentru multimea vida
    //                   dp[*][0] pentru situatia in care ghiozdanul are capacitate 0
    vector< vector<int> > dp(n + 1, vector<int>(W + 1, 0));

    // cazul de baza
    for (int cap = 0; cap <= W; ++cap) {
        dp[0][cap] = 0;
    }

    // cazul general
    for (int i = 1; i <= n; ++i) {
        for (int cap = 0; cap <= W; ++cap) {
            // nu folosesc obiectu i => e solutia de la pasul i - 1
            dp[i][cap] = dp[i-1][cap];

            // folosesc obiectul i, deci trebuie sa rezerv w[i] unitati in rucsac
            // inseamna ca inainte trebuie sa ocup maxim cap - w[i] unitati
            if (cap - w[i] >= 0) {
                int sol_aux = dp[i-1][cap - w[i]] + p[i];

                dp[i][cap] = max(dp[i][cap], sol_aux);
            }
        }
    }

    return dp[n][W];
}
```

Mentiuni

Intrucat aceasta solutie presupune calculul iterativ (linie cu linie) a matricei dp, complexitatea este polinomiala.

- **complexitate temporala** : $T = O(n * W)$
- **complexitate spatiala** : $S = O(n * W)$
  - daca nu ne intereseaza sa reconstituim solutia (sa afisam submultimea efectiv), atunci putem sa NU stocam toata matricea dp
  - ca sa calculam o linie, avem nevoie doar de ultima linie
  - putem sa stocam la orice moment de timp doar ultima linie si linia curenta
  - complexitatea spatiala se reduce astfel la $S = O(W)$

Exercitii

In laboratorul de astazi, implementarea exercitiilor nu va fi punctata. Cu toate acestea, daca doriti sa implementati problemele propuse spre rezolvare, puteti folosi scheletul de laborator din arhiva skel-lab03.zip.

1. Not again!

Gigel are o colectie impresionanta de monede. El ne spune ca are **n tipuri** de monede, avand un numar nelimitat de monede din fiecare tip. Cunoscand aceasta informatie (data sub forma unui vector **v** cu **n** elemente), el se intreaba care este numarul minim de monede cu care poate plati o **suma S**.

Task-uri:

- 1.1 **Determinati numarul minim de monede** (din cele pe care le are) cu care Gigel poate forma suma S.
- 1.2 Care este complexitatea solutiei (timp + spatiu)? De ce?

Este posibil ca pentru anumite valori ale lui S si v, aceasta problema sa nu aiba solutie. In acest caz raspunsul este **-1**.

$n = 4$ si $S = 12$

i	1	2	3	4
v	1	2	3	6

Raspuns: 2

Explicatie: Avem 4 tipuri de monede: 1 euro, 2 euro, 3 euro si 6 euro (lui Gigel nu ii mai place sa foloseasca RON). Avem la dispozitie oricate monede din fiecare tip. Suma 12 poate fi obtinuta in urmatoarele moduri:

- $12 = 6 + 6$
- $12 = 6 + 3 + 3$
- $12 = 6 + 3 + 2 + 1$
- $12 = 6 + 2 + 2 + 2$
- $12 = 6 + 3 + 3$
- $12 = 3 + 3 + 3 + 3$
- $12 = 3 + 3 + 3 + 2 + 1$
- $12 = 3 + 3 + 2 + 2 + 2$
- $12 = 3 + 2 + 2 + 2 + 2 + 1$
- $12 = 2 + 2 + 2 + 2 + 2 + 2$
- … (ati inteles ideea :D)

Solutia cu numar minim de monede se obtine pentru modul $6 + 6$.

$n = 3$ si $S = 11$

i	1	2	3
v	1	2	5

Raspuns: 3

Explicatie: Avem 3 tipuri de monede: 1 euro, 2 euro si 5 euro (lui Gigel nu ii mai place sa foloseasca RON). Avem la dispozitie oricate monede din fiecare tip. Suma 11 poate fi obtinuta in urmatoarele moduri:

- $11 = 5 + 5 + 1$
- $11 = 5 + 2 + 2 + 2$
- $11 = 5 + 2 + 2 + 1 + 1$
- $11 = 5 + 2 + 1 + 1 + 1 + 1$
- $11 = 5 + 1 + 1 + 1 + 1 + 1 + 1$
- $11 = 2 + 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1$
- $11 = 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$
- $11 = 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$

Solutia cu numar minim de monede se obtine pentru modul $5 + 5 + 1$.

$n = 3$ si $S = 11$

i	1	2	3
v	2	4	6

Raspuns: -1

Explicatie: Nu putem forma suma 11 folosind tipurile (valorile) 2, 4, 6.

2. CMLSC

Fie doi vectori cu numere intregi: **v** cu **n** elemente si **w** cu **m** elemente. Sa se gaseasca **cel mai lung subsir comun** (notat **CMLSC**) care apare in cei doi vectori. Se cere o solutie de complexitate optima. Daca exista mai multe solutii, se poate gasi oricare.

Task-uri:

- 2.1 **Determinare lungime** CMLSC. (Hint: DP)
- 2.2 **Reconstituire** CMLSC (afisati si care sunt termenii CMLSC).
- 2.3 Care este **complexitatea** solutiei (timp + spatiu)? De ce?

Rezolvati in ordine task-urile.

subsir (**subsequence** in engleza) pentru un vector **v** inseamna un alt vector $u = [v[i_1], v[i_2], \ldots, v[i_k]]]$ unde $i_1 < i_2 < \ldots < i_k$.

$n = 3$ si $m = 5$

i	1	2	3
v	6	-1	9

j	1	2	3	4	5

w	0	6	2	9	8

Raspuns: $lungime = 2$, $CMLSC = [6, 9]$

Explicatie: Toate subsirurile comune posibile sunt:

- $[6]$
- $[6, 9]$
- $[9]$

Solutia mentionata are lungime maxima.

$n = 8$ si $m = 5$

i	1	2	3	4	5	6	7	8
v	2	1	5	3	4	5	2	7

j	1	2	3	4	5
w	1	5	5	7	4

Raspuns: $lungime = 4$, $CMLSC = [1, 5, 5, 7]$

Explicatie: Toate subsirurile comune posibile sunt (duplicatele vor fi mentionate o singura data):

- $[1]$
- $[1, 5]$
- $[1, 7]$
- $[1, 4]$
- $[1, 5, 5]$
- $[1, 5, 7]$
- $[1, 5, 4]$
- $[1, 5, 5, 7]$
- $[5]$
- $[5, 5]$
- $[5, 7]$
- $[5, 4]$
- $[4]$

Solutia mentionata are lungime maxima.

$n = 8$ si $m = 5$

i	1	2	3	4	5	6	7	8
v	2	1	5	3	4	5	2	7

j	1	2	3	4	5
w	1	5	7	-5	4

Raspuns: $lungime = 3$, $CMLSC = [1, 5, 7]$ (exemplu de solutie)

Explicatie: Toate subsirurile comune posibile sunt (duplicatele vor fi mentionate o singura data):

- $[1]$
- $[1, 5]$
- $[1, 7]$

- $[1, 4]$
- $[1, 5, 7]$
- $[1, 5, 4]$
- $[5]$
- $[5, 7]$
- $[5, 4]$
- $[4]$

Solutii pot fi: $[1, 5, 7]$ si $[1, 5, 4]$. Pentru $[1, 5, 7]$, se observa ca sunt 2 astfel de subsiruri in vectorul v. Oricare este bun.

BONUS

Rezolvati pe infoarena problema custi [https://infoarena.ro/problema/custi].

Extra

Modificati solutia de la Rucsac prezentata in laborator pentru a obtine o complexitate spatiala mai buna (se va retine un numar minim de linii din matrice). Puteti testa solutia voastra pe infoarena la problema rucsac [https://infoarena.ro/problema/rucsac].

Se da o matrice de dimensiuni $n * m$ si Q intrebari de forma: "Care este suma din submatricea care are coltul stanga-sus (x, y) si coltul dreapta-jos (p,q)?"

Se considera proprietatea: Q este mult mai mare decat dimensiunile matricei.

Sa se raspunda in mod eficient la cele Q intrebari.

Rezolvatie pe infoarena problema joctv [https://infoarena.ro/problema/joctv].

Solutie: Se fixeaza 2 linii pentru zona dreptunghiulara. Se reduce problema la SSM in O(n). Complexitate: $O(n^3)$.

Rezolvati pe leetcode problema Interleaving String [https://leetcode.com/problems/interleaving-string/description/].

https://www.geeksforgeeks.org/cutting-a-rod-dp-13/ [https://www.geeksforgeeks.org/cutting-a-rod-dp-13/] Cutting rod [https://www.geeksforgeeks.org/cutting-a-rod-dp-13/] [0]

Partition Problem [https://www.geeksforgeeks.org/partition-problem-dp-18/]

Referințe

[0] Capitolul **Dynamic Programming** din **Introductions to Algorithms** de către T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein

[1] http://infoarena.ro/problema/ssm [http://infoarena.ro/problema/ssm]

[2] http://infoarena.ro/problema/scmax [http://infoarena.ro/problema/scmax]

[3] http://infoarena.ro/problema/rucsac [http://infoarena.ro/problema/rucsac]

[4] Colecție de probleme de programare dinamică (geeksforgeeks.com) [https://www.geeksforgeeks.org/dynamic-programming/]

[5] Probleme de interviu care se rezolvă cu PD [https://practice.geeksforgeeks.org/explore/?category%5B%5D=Dynamic%20Programming&page=1&sortBy=accuracy]

[6] Sniedovich, Moshe. Dynamic programming: foundations and principles. CRC press, 2010.

pa/laboratoare/laborator-03.txt · Last modified: 2020/03/10 00:12 by darius.neatu

Laborator 4: Programare Dinamică (continuare)

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Darius Neațu (2018) [mailto:neatudarius@gmail.com]
- Radu Vișan (2018) [mailto:visanr95@gmail.com]
- Cristian Banu (2018) [mailto:cristb@gmail.com]
- Răzvan Chițu (2018) [mailto:razvan.ch95@gmail.com]

Obiective laborator

- Înțelegerea noțiunilor de bază despre programare dinamică
- Insușirea abilităților de implementare a algoritmilor bazați programare dinamică.

Precizari initiale

Toate exemplele de cod se gasesc in demo-lab04.zip.

Acestea apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a laboratorului.

- Toate bucatile de cod prezentate in partea introductiva a laboratorului (inainte de exercitii) au fost testate. Cu toate acestea, este posibil ca din cauza mai multor factori (formatare, caractere invizibile puse de browser etc) un simplu copy-paste sa nu fie de ajuns pentru a compila codul.
- Va rugam sa incercati si codul din arhiva **demo-lab04.zip**, inainte de a raporta ca ceva nu merge. :D
- Pentru orice problema legata de continutul acestei pagini, va rugam sa dati email unuia dintre responsabili.

Ce este DP?

Similar cu greedy, tehnica de programare dinamica este folosită pentru rezolvarea **problemelor de optimizare**. In continuare vom folosi acronimul **DP (dynamic programming)**.

De asemenea, DP se poate folosi si pentru probleme in care nu cautam un optim, cum ar fi **problemele de numarare**.

Pentru restul notiunilor prezentate pana acum despre DP, va rugam sa consulati pagina laboratorului 3.

Exemple clasice

Programarea Dinamică este cea mai flexibilă tehnica din programare. Cel mai ușor mod de a o înțelege presupune parcurgerea cât mai multor exemple.

Propunem cateva categorii de recurente, pe care le vom grupa astfel:

- recurente de tip **SSM** (Subsecventa de Suma Maxima)
- recurente de tip **RUCSAC**
- recurente de tip **PODM** (Parantezare Optima de Matrici)
- recurente de tip **numărat**
- recurente pe **grafuri**

Pentru o problema data, este **posibil** sa gasim **mai multe recurente corecte** (mai multe solutii posibile). Evident, criteriul de alegere intre acestea va fi cel bazat pe complexitate.

Categoria 3: PODM

Aceste recurente au o oarecare asemanare cu problema PODM (enunt + solutie).

ATENTIE! Acest tip de recurente poate fi mai greu (decat celelalte). Puteti consulta **acasa** materialele puse la dispozitie pentru intelege mai bine aceasta categorie.

Caracteristici:

- Acest tip de problema presupune ca o putem formula ca pe o problema de tip **subinterval** $[i,j]$.
- Daca dorim sa gasim optimul pentru acest interval, va trebuie sa luam in calcul toate combinatiile de 2 subproblemele care ar fi putut genera solutie pentru probleme $[i,j]$.
- Se considera fiecare divizare in 2 subprobleme, data de intermediarul k
  - $[i,k]$ si $[k+1,j]$ sunt cele 2 subprobleme pentru care cunoastem solutiile
  - atunci o solutie pentru $[i,j]$ se poate obtine imbinandu-le pe cele doua
  - ca sa gasim solutia cea mai buna
    - vom itera prin toate valorile k posibile
    - vom alege pe cea care maximizeaza solutia problemei $[i,j]$
- Calculul se face de la intervale mici (probleme usoare - $[i,i]$ sau $[i,i+1]$) spre probleme generale (dimensiune generala - $[i,j]$). In final se ajunge si la dimensiunile initiale ($[1,n]$).
  - Privind imaginea de ansamblu, adica cum se completeaza matricea, obervam ca matricea dp se completeaza **diagonala cu diagonala**.

Exemple clasice

PODM

Enunt

Fie un produs matricial $M = M_1 M_2 \dots M_n$. Putem pune paranteze in mai multe moduri si vom obtine acelasi rezultat (inmultire asociativa), dar este posibil sa obtinem numar diferit de **inmultiri scalare** .

Matricea M_i are (prin conventie), dimensiunile $d_{i-1} d_i$.

Cerinta

Se cere sa se gaseasca o **parantezare optima de matrice** (PODM), adica sa se gaseasca o parantezare care sa minimizeze numarul de inmultiri scalare.

Exemple

$n = 3$

i	0	1	2	3
d	2	3	4	5

Raspuns: **64** (inmultiri scalare)

Explicatie: Avem 3 matrici:

- A de dimensiuni (2, 3)
- B de (3, 4)
- C de (4, 5)

In functie de ordinea efectuarii inmultirilor matriciale , numarul total de inmultiri scalare poate sa fie foarte diferit:

- $(AB)C \Rightarrow 24 + 40 = 64$ de inmultiri
  - explicatie: $X = (AB)$ genereaza $2 * 3 * 4 = 24$ inmultiri, (XC) genereaza $2 * 4 * 5 = 40$ de inmultiri
- $A(BC) \Rightarrow 60 + 30 = 90$ de inmultiri
  - explicatie: $X = (BC)$ genereaza $3 * 4 * 5 = 60$ inmultiri, (AX) genereaza $2 * 3 * 5 = 30$ de inmultiri

Rezultatul optim se obtine pentru cea de a treia parantezare: $(AB)C$.

$n = 4$

i	0	1	2	3	4
d	2	3	4	2	3

Raspuns: **48** (inmultiri scalare)

Explicatie: Avem 4 matrici:

- A de dimensiuni (2, 3)
- B de (3, 4)
- C de (4, 2)
- D de (2, 3)

In functie de ordinea efectuarii inmultirilor matriciale, numarul total de inmultiri scalare poate sa fie foarte diferit:

- $(AB)C)D \Rightarrow 24 + 16 + 12 = 52$ inmultiri
  - explicatie: $X = (AB)$ genereaza $2 * 3 * 4 = 24$ inmultiri scalare, $Y = (XC)$ genereaza $2 * 4 * 2 = 16$ inmultiri scalare, $Z = YD$ genereaza $2 * 2 * 3 = 12$ inmultiri scalare
- $(A(BC))D \Rightarrow 24 + 12 + 12 = 48$ inmultiri
  - explicatie: $X = (BC)$ genereaza $3 * 4 * 2 = 24$ inmultiri scalare, $Y = (AX)$ genereaza $2 * 3 * 2 = 12$ inmultiri scalare, $Z = YD$ genereaza 2 * 2 * 3 $= 12$ inmultiri scalare
- $(AB)(CD) \Rightarrow$ = inmultiri
  - explicatie: $X = (AB)$ genereaza $2 * 3 * 4 = 24$ inmultiri scalare, $Y = (CD)$ genereaza $4 * 2 * 3 =$ 24 inmultiri scalare, $Z = XY$ genereaza $2 * 4 * 3 = 24$ inmultiri scalare
- $A((BC)D) \Rightarrow 24 + 18 + 27 = 69$ inmultiri
  - explicatie: $X = (BC)$ genereaza $3 * 4 * 2 = 24$ inmultiri scalare, $Y = (XD)$ genereaza $3 * 2 * 3 = 18$ inmultiri scalare, $Z = AY$ genereaza $3 * 3 * 3 = 27$ inmultiri scalare
- $A(B(CD)) \Rightarrow 24 + 36 + 18 = 78$ inmultiri
  - explicatie: $X = (CD)$ genereaza $4 * 2 * 3 = 24$ inmultiri scalare, $Y = (BX)$ genereaza $3 * 4 * 3 = 36$ inmultiri scalare, $Z = AY$ genereaza $2 * 3 * 3 = 18$ inmultiri scalare

Rezultatul optim se obtine pentru cea de a treia parantezare: $((A(BC))D)$.

$n = 4$

i	0	1	2	3	4
d	13	5	89	3	34

Raspuns: **2856** (inmultiri scalare)

Explicatie: Avem 4 matrici:

- A de dimensiuni (13, 5)
- B de (5, 89)
- C de (89, 3)
- D de (3, 34)

In functie de ordinea efectuarii inmultirilor matriciale , numarul total de inmultiri scalare poate sa fie foarte diferit:

- $((AB)C)D \Rightarrow$ 10582 inmultiri
- $(AB)(CD) \Rightarrow$ 54201 inmultiri
- $(A(BC))D \Rightarrow$ 2856 inmultiri
- $A((BC)D) \Rightarrow$ 4055 inmultiri
- ...

Rezultatul optim se obtine pentru cea de a treia parantezare: $(A(BC))D$.

TIPAR

A fost descris in detaliu mai sus (cand s-a vorbit de categorie).

Numire recurenta

$dp[i][j]$ = **numarul minim de inmultiri scalare** cu care se poate obtine produsul $M_i * M_{i+1} * \ldots * M_j$

Gasire recurenta

- **Cazul de baza** :
  - $dp[i][i] = 0$
    - NU avem nici un efort daca nu avem ce inmulti.
  - $dp[i][i+1] = d_{i-1}d_id_{i+1}$
    - Daca avem doua matrice, putem doar sa le inmultim. Nu are sens sa folosim paranteze.
    - Daca inmultim 2 matrice de dimensiuni $d_{i-1} * d_i$ si $d_i * d_{i+1}$, avem costul $d_{i-1}d_id_{i+1}$
- **Cazul general**: $dp[i][j] = min(dp[i][k] + dp[k+1][j] + d_{i-1}d_kd_j)$, unde $k = i : j-1$
  - daca avem de efectuat sirul de inmultiri $M_i \ldots M_j$, atunci putem pune paranteze oriunde si sa facem inmultirile astfel $(M_i \ldots M_k)(M_{k+1} \ldots M_j)$
    - costul minim pentru $(M_i \ldots M_k)$ este $dp[i][k]$
    - costul minim pentru $(M_{k+1} \ldots M_j)$ este $dp[k+1][j]$
    - vom avea in final de inmultit 2 matrice de dimensiune $d_{i-1} * d_k$ si $d_k * d_j$, operatie care are costul $d_{i-1}d_kd_j$
  - insumam cele 3 costuri intermediare

Implementare

Puteti rezolva si testa problema PODM pe infoarena aici [https://infoarena.ro/problema/podm].

Un exemplu de implementare in C++ se gaseste mai jos.

```
// kInf este valoarea maxima - "infinitul" nostru
const unsigned long long kInf = std::numeric_limits<unsigned long long>::max();

// T = O(n ^ 3)
// S = O(n ^ 2) - stocam n x n intregi in tabloul dp
 unsigned long long solve_podm(int n, const vector<int> &d) {
    // dp[i][j] = numarul MINIM inmultiri scalare cu codare poate fi calculat produsul
    //            matricial M_i * M_i+1 * ... * M_j
    vector<vector<unsigned long long>>  dp(n + 1, vector<unsigned long long> (n + 1, kInf));

    // Cazul de baza 1: nu am ce inmulti
    for (int i = 1; i <= n; ++i) {
        dp[i][i] = 0ULL;  // 0 pe unsigned long long (voi folosi mai incolo si 1ULL)
    }

    // Cazul de baza 2: matrice d[i - 1] x d[i] inmultia cu matrice d[i] x d[i + 1]
    // (matrice pe pozitii consecutive)
    for (int i = 1; i < n; ++i) {
        dp[i][i + 1] =  1ULL * d[i - 1] * d[i] * d[i + 1];
    }

    // Cazul general:
    // dp[i][j] = min(dp[i][k] + dp[k + 1][j] + d[i - 1] * d[k] * d[j]), k = i : j - 1
    for (int len = 2; len <= n; ++len) {            // fixam lungimea intervalului (2, 3, 4, ...)
        for (int i = 1; i + len - 1 <= n; ++i) {    // fixam capatul din stanga: i
            int j = i + len - 1;                    // capatul din dreapta se deduce: j

            // Iteram prin indicii dintre capete, spargand sirul de inmultiri in doua (paranteze).
            for (int k = i; k < j; ++k) {
                // M_i * ... M_j  = (M_i * .. * M_k) * (M_k+1 *... * M_j)
                unsigned long long new_sol = dp[i][k] + dp[k + 1][j] + 1ULL * d[i - 1] * d[k] * d[j];

                // actualizam solutia daca este mai buna
                dp[i][j] = min(dp[i][j], new_sol);
            }
        }
    }

    // Rezultatul se afla in dp[1][n]: Numarul MINIM de inmultiri scalare
    // pe care trebuie sa le facem pentru a obtine produsul M_1 * ... * M_n
    return dp[1][n];
```

```
}
```

Sursa a fost scrisa pentru a fi testata pe infoarena. In cazul problemei PODM [https://infoarena.ro/problema/podm], deoarece avem o suma de foarte multe produse, rezultatul este foarte mare. Pe infoarena se cerea ca rezultatul sa fie afisat asa cum e, garantandu-se ca incape pe 64 biti.

Reamintim ca prin inmultirea/adunarea a doua variabile de tipul **int**, rezultatul poate sa nu incapa pe 32 biti. De aceea, in solutia prezentata, s-a facut cast pe 64biti.

ATENTIE! La PA, in general, vom folosi conventia $expresie \% kMod$, care va fi detaliata in capitolul urmatorul din acest laborator.

Complexitate

Intrucat solutia presupune fixarea capetelor unui subinterval (i, j), apoi alegerea unui intermediar (k), complexitatea este data de aceste 3 cicluri.

- **complexitate temporala**: $T(n) = O(n^3)$
- **complexitate spatiala**: $S(n) = O(n^2)$

Categoria 4: NUMARAT

Aceste recurente au o oarecare asemanare:

- toate numara lucruri! :p
- interesante sunt cazurile cand numarul cautat este foarte mare (altfel am putea apela la alte metode - ex. generarea tuturor candidatilor posibili cu backtracking)
  - in acest caz, deoarece numarul poate sa nu incapa pe un tip reprezentabil standard (ex. int pe 32/64 de biti), se cere (de obicei) restul impartirii numarului cautat la un numar **MOD** (vom folosi in continuare aceasta notatie).

Sfaturi / Reguli

- cand cautati o recurenta pentru o problema de numarare trebuie sa aveti grija la doua aspecte:
  - 1) sa **NU** numarati acelasi obiect de doua ori.
  - 2) sa numarati toate obiectele in cauza.
- de multe ori o problema de numarare implica o partitionare a **tuturor** posibilelor solutii dupa un anumit criteriu (relevant). Gasirea criteriului este partea esentiala pentru gasirea recurentei.

Regulile de lucru cu clase de resturi

Reamintim cateva proprietati matematice pe care ar trebui sa le aveti in vedere atunci in implementati pentru obtine corect resturile pentru anumite expresii. (corect poate sa insemne, de exemplu, sa evitati overflow :D - lucru neintuitiv cateodata).

- proprietati de de baza
  - $(a + b) \% MOD = ((a \% MOD) + (b \% MOD)) \% MOD$
  - $(a * b) \% MOD = ((a \% MOD) * (b \% MOD)) \% MOD$
  - $(a - b) \% MOD = ((a \% MOD) - (b \% MOD) + MOD) \% MOD$ (restul nu poate fi ceva negativ; in C++ **%** nu functioneaza pe numere negative)
- invers modular
  - $\frac{a}{b} \% MOD = ((a \% MOD) * (b^{MOD-2} \% MOD)) \% MOD)$
    - **DACA** MOD este prim; **DACA** a si b nu sunt multipli ai lui MOD

- **definitie** : **b** este inversul modular a lui **a** in raport cu **MOD** daca $a * b = 1 (modulo\ MOD)$
- **utilizare** : $\frac{a}{b} \% MOD = ((a \% MOD) * (invers(b) \% MOD)) \% MOD$
- **calculare** : deoarece la PA aceasta discutie are sens doar in contextul posibilitatii implementarii unei recurente DP in care folosim resturile doar pentru a evita overflow/imposibilitatea de a retine rezultatul pe tipurile standard de tip int (adica nu ne intereseaza sa dam o metoda generala pentru invers modular), vom simplifica problema - **MOD este prim!!!**
- **Mica teorema a lui Fermat**: Daca p este un numar prim si a este un număr intreg care nu este multiplu al lui p, atunci $a^{p-1} = 1 (modulo\ p)$.
  - din definitia inversului modular, reiese ca **a** si **b** nu sunt multipli ai lui **MOD**
  - introducand notatiile noastre in teorema si prelucrand obtinem
    - $a^{MOD-1} = 1 (modulo\ MOD) <=> a * (a^{MOD-2}) = 1 (modulo\ MOD)$
    - deci inversul modular al lui a (in aceste conditii specifice) este $b = a^{MOD-2}$

Reamintim ca prin inmultirea/adunarea a doua variabile de tipul int, rezultatul poate sa nu incapa pe 32 biti. E posibil sa trebuiasca sa combinam regulile cu resturi cu urmatoarele:

- C++
  - **1LL / 1ULL** - constanta 1 pe 64 biti cu semn / fara semn
    - **1LL * a * b** - am grija ca rezultatul sa nu dea overflow si sa se stocheze direct pe 64biti (cu semn)
- Java
  - **1L** - constanta 1 pe 64biti cu semn (in Java nu exista unsigned types)
    - **1L * a * b** - am grija ca rezultatul sa nu dea overflow si sa se stocheze direct pe 64biti (cu semn)

Gardurile lui Gigel

Enunt

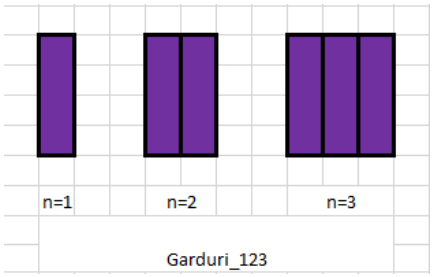
Gigel trece de la furat obiecte cu un rucsac la numarat garduri (fiecare are micile lui placeri :D). El doreste sa construiasca un gard folosind in mod repetat **un singur tip de piesa**.

O piesa are dimensiunile **4 x 1** (o unitate = 1m). Din motive irelevante pentru aceasta problema, orice gard construit trebuie sa aiba **inaltime 4m** in orice punct.

O piesa poate fi pusa in pozitie **orizontala** sau in pozitie **verticala**.

Cerinta

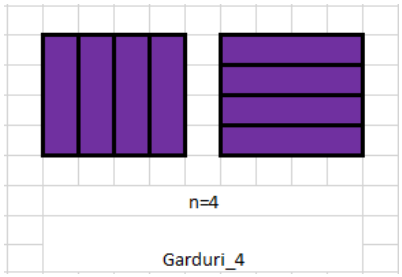
Gigel se intreaba **cate garduri de lungime n** si inaltime 4 exista? Deoarece celalalt prenume al lui este Bulănel, el intuieste ca acest numar este foarte mare, de aceea va cere **restul impartirii** acestui numar la **1009**.



$n = 1$ sau $n = 2$ sau $n = 3$

Raspuns: **1** (un singur gard)

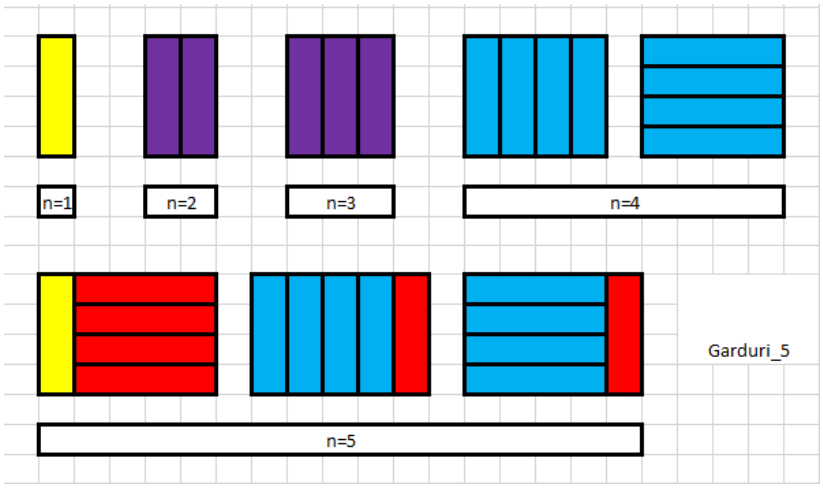
Explicatie: Se poate forma un singur gard in fiecare caz, dupa cum este ilustrat si in figura **Garduri_123**.



$n = 4$

Raspuns: **2**

Explicatie: Se pot forma 2 garduri, in functie de cum asezam piesele, dupa cum este ilustrat si in figura **Garduri_4**. Observam ca de fiecare daca cand punem o piesa in pozitie orizontala, de fapt suntem obligati sa punem 4 piese, una peste alta!

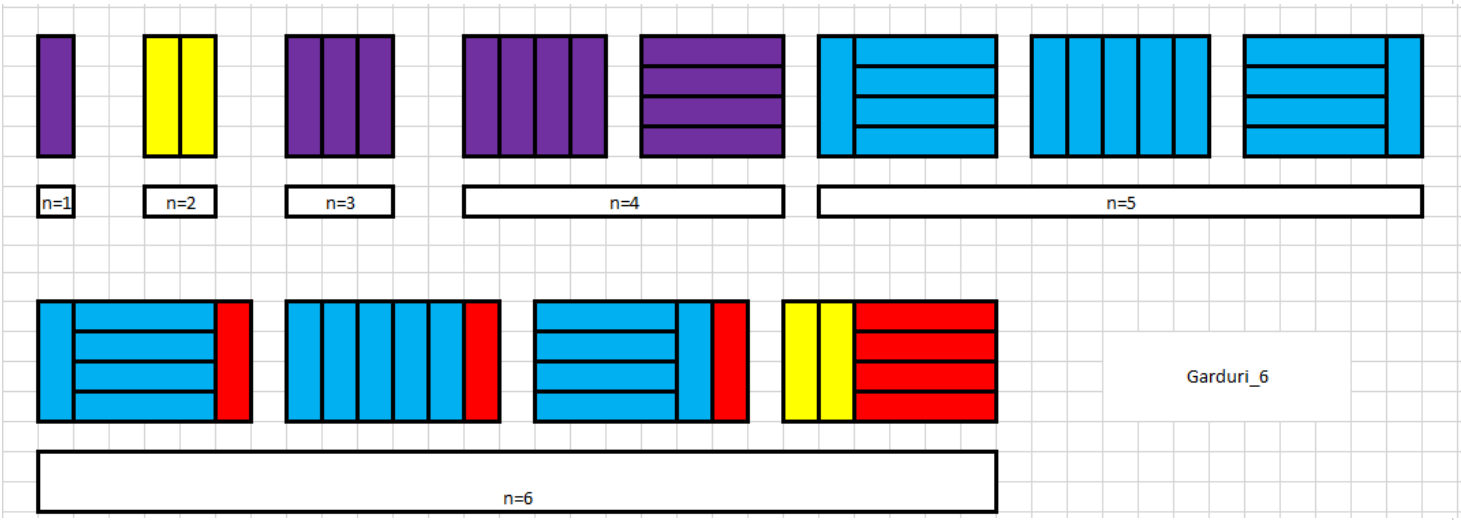


$n = 5$

Raspuns: **3**

Explicatie: Se pot forma 3 garduri, in functie de cum asezam piesele, dupa cum este ilustrat si in figura **Garduri_5**.

- daca dorim ca acest gard sa se termine cu 4 piese in pozitie **orizontala** (una peste alta - marcat cu rosu), atunci la stanga mai ramane de completat **un subgard de lungime 1**, in toate modurile posibile
- daca dorim ca acest gard sa se termine cu o piesa in pozitie **verticala** (marcat cu rosu), atunci la stanga mai ramane de completat **un subgard de lungime 4**, in toate modurile posibile



$n = 6$

Raspuns: **4**

Explicatie: Se pot forma 4 garduri, in functie de cum asezam piesele, dupa cum este ilustrat si in figura **Garduri_6**.

- daca dorim ca acest gard sa se termine cu o piesa in pozitie **verticala** (marcat cu rosu), atunci la stanga mai ramane de completat **un subgard de lungime 5**, in toate modurile posibile
- daca dorim ca acest gard sa se termine cu 4 piese in pozitie **orizontala** (una peste alta - marcat cu rosu), atunci la stanga mai ramane de completat **un subgard de lungime 2**, in toate modurile posibile

Recurenta

Numire recurenta

$dp[i]$ = numarul de garduri de lungime i si inaltime 4 (nimic special - exact ceeea ce se cere in enunt)

Raspunsul la problema este $dp[n]$.

Gasire recurenta

- **Caz de baza**
  - $dp[1] = dp[2] = dp[3] = 1$; $dp[4] = 2$
- **Caz general**
  - atunci dorim sa formam un gard de lungime i ($i >= 5$) am vazut ca putem alege cum sa punem ultima/ultimele piese
    - **DACA** alegem ca ultima piesa sa fie pusa in pozitie verticala, atunci la stanga mai ramane de completat **un subgard de lungime** $i - 1$
      - numarul de moduri in care putem face acest subgard este $dp[i - 1]$
    - **DACA** alegem ca ultima piesa sa fie in pozitie orizontala (de fapt punem 4 piese in pozitie orizontala), atunci la stanga mai ramane de completat **un subgard de lungime** $i - 4$
      - numarul de moduri in care putem face acest subgard este $dp[i - 4]$
  - $dp[i] = (dp[i - 1] + dp[i - 4]) \% MOD$
  -

Asa cum am zis in sectiunea de sfaturi si reguli [http://ocw.cs.pub.ro/courses/pa/laboratoare/laborator-04?&#sfaturireguli] vrem sa facem o **partionare** dupa un anumit **criteriu**, in cazul problemei de fata criteriul de partionare este daca gardul se termina cu o scandura verticala sau orizontala.

De asemenea tot in sectiunea sfaturi si reguli [http://ocw.cs.pub.ro/courses/pa/laboratoare/laborator-04?&#sfaturireguli] am precizat ca nu vrem **sa numaram un obiect** (un mod de a construi gardul) **de doua ori**. Recurenta noastra (dp[i] = dp[i-1] + dp[i-4]) nu ia un obiect de doua ori pentru ca orice solutie care vine din dp[i-4] e diferita de alta care vine din dp[i-1] pentru ca difera in cel putin ultima scandura asezata)

Implementare recurenta

Aici puteti vedea un exemplu simplu de implementare in C++.

```
#define MOD 1009
int gardurile_lui_Gigel(int n) {
    // cazurile de baza
    if (n <= 3) return 1;
    if (n == 4) return 2;

    vector<int> dp(n + 1); // pastrez indexarea de la 1 ca in explicatii

    // cazurile de baza
    dp[1] = dp[2] = dp[3] = 1;
    dp[4] = 2;

    // cazul general
    for (int i = 5; i <= n; ++i) {
        dp[i] = (dp[i - 1] + dp[i - 4]) % MOD;
    }
```

```
    return dp[n];
}
```

Mentionez ca am folosit expresia $dp[i] = (dp[i-1] + dp[i-4]) \% MOD$ in loc de $dp[i] = ((dp[i-1] \% MOD) + (dp[i-4] \% MOD)) \% MOD$, deoarece pe valorile anterior calculate in dp a fost deja aplicata operatia .

Am plecat cu numerele $1, 1, 1, 2$ si la fiecare pas rezultatul stocat este $\% MOD$, deci tot ce este stocat **deja** in dp este un rest in raport cu MOD. NU mai era nevoie deci sa aplica **%** si pe termenii din paranteza.

Complexitate

- **complexitate temporala**: $T = O(n)$
  - explicatie: avem o singura parcurgere in care construim tabloul dp
  - se poate obtine $T = O(logn)$ folosind exponentiere pe matrice!
- **complexitate spatiala**: $S = O(n)$
  - explicatie: stocam tabloul dp
  - se poate ontine $S = O(1)$ folosind exponentiere pe matrice!

Tehnici folosite in DP

De multe ori este nevoie sa folosim cateva tehnici pentru a obtine performanta maxima cu recurenta gasita.

In laboratorul 3 se mentiona tehnica de memoizare (in prima parte a laboratorului). In acesta ne vom rezuma la cum putem folosi cunostintele de lucru matricial pentru a favoriza implementarea unor anumite tipuri de recurente.

Exponentiere pe matrice pentru recurente liniare

Recurente liniare

O recurenta liniara in contextul laboratorului de DP este de forma:

- $dp[i] = \sum_{k=1}^{KMAX} c_k * dp[i-k]$
  - pentru **KMAX o constanta**
  - de obicei, KMAX este foarte mica comparativ cu dimensinea n a problemei
  - c_k constante reale (unele pot fi nule)

O astfel de recurenta ar insemna ca pentru a calcula **costul problemei i** , imbinam costurile problemelor $i-1, i-2, \ldots, i-k$, fiecare contribuind cu un anumit coeficient $c_1, c_2, \ldots, c_k$.

Presupunand ca nu mai exista alte specificatii ale problemei si ca avand cele KMAX cazuri de baza (primele KMAX valori ar trebui stiute/deduse prin alte reguli), atunci un algoritm poate implementa recurenta de mai sus folosind 2 cicluri de tip for (for i = 1 : n, for k = 1 : KMAX …).

- **complexitatea temporala** : $T = O(n * KMAX) = O(n)$
  - reamintim ca acea valoarea KMAX este o constanta foarte mica in compartie cu n (ex. KMAX < 100)
- **complexitatea spatiala** : $S = O(n)$
- am presupus ca avem nevoie sa retinem doar tabloul dp

Exponentiere pe matrice

Facem urmatoarele notatii:

- S_i = starea la pasul i
  - $S_i = (dp[i-k+1], dp[i-k+2], \ldots, dp[i-1], dp[i])$
- S_k = starea initiala (in care cunoaste cele k cazuri de baza)
  - $S_k = (dp[1], dp[2], \ldots, dp[k-1], dp[k])$
- C = matrice ce coeficienti constanti
  - are dimensiune $KMAX * KMAX$
  - putem pune constante in clar
  - putem pune constantele c_k care tin de problema curenta

Algoritm naiv

Putem formula problema astfel:

- S_k = este starea initiala
- pentru a obtine starea urmatoare, aplicam algoritmul urmator
  - $S_i = S_{i-1} C$

Determinare C

Pentru a determina elementele matricei C, trebuie sa ne uitam la inmultirea matriceala de mai sus si sa alegem elementele lui C astfel incat prin inmultirealui S_{i-1} cu C sa obtinem elementele din S_i.

$$\begin{bmatrix} dp[i-k+1] & \dots & dp[i-1] & dp[i] \end{bmatrix} = \begin{bmatrix} dp[i-k] & \dots & dp[i-2] & dp[i-1] \end{bmatrix} \begin{bmatrix} 0 & 0 & \dots & 0 & 0 & c_k \\ 1 & 0 & \dots & 0 & 0 & c_{k-1} \\ 0 & 1 & \dots & 0 & 0 & c_{k-2} \\ \dots & \dots & \dots & \dots & \dots & \\ 0 & 0 & \dots & 1 & 0 & c_2 \\ 0 & 0 & \dots & 0 & 1 & c_1 \end{bmatrix}$$

- ultima coloana contine toti coeficientii c_k intrucat $dp[i] = \sum_{k=1}^{KMAX} c_k * dp[i-k]$
- celelalte coloane contin doar cate o valoare nenula
  - pe coloana j vom avea valoarea 1 pe linia $j+1$ ($j = 1 : KMAX-1$)
    - cum obtinem, de exemplu, $dp[i-1]$?
    - pai avem $dp[i-1]$ chiar si in starea S_{i-1}, deci trebuie sa il copiam in starea S_i
      - copierea se realizeaza prin inmultirea cu 1
      - daca $dp[i-1]$ era pe ultima pozitiei (pozitia k) in starea S_{i-1}, in noua stare S_i este pe penultima pozitie (pozitia $k-1$)
        - deci s-a deplasat la stanga cu o pozitie!
  - in noua stare, noua pozitie este deplasata cu o unitate la stanga fata de starea precedenta
    - de aceea pe coloana j, vrem sa avem elementul 1 pe linia $j+1$ ($j = 1 : KMAX-1$)
    - cand inmultim S_{i-1} cu coloana C_j **dorim sa**
      - ce copiam?
        - valoarea $dp[i-KMAX+j]$ din S_{i-1} in S_i
        - adica sa copiam a j-a valoare de pe linie
      - unde copiam?
        - de pe pozitia $j+1$ pe pozitia j

Exponentiere logaritmica pe matrice

Algoritmul naiv de mai sus are dezavantajul ca are tot o complexitate temporala $O(n)$.

Sa executam cativa pasi de inductie pentru a vedea cum este determinat S_i.

$$S_i = S_{i-1}C$$

$$S_i = S_{i-2}C^2$$

$$S_i = S_{i-3}C^3$$

$$\dots$$

$$S_i = S_kC^{i-k}$$

In laboratorul 2 (Divide et Impera) am invatat ca putem calcula x^n in timp logaritmic. Deoarece si inmultirea matricilor este asociativa, putem calcula C^n in timp logaritmic.

Obtinem astfel o solutie cu urmatoarele complexitati:

- **complexitate temporala** : $T = O(KMAX^3 * log(n))$
  - explicatie
    - facem doar $O(logn)$ pasi, dar un pas implica inmultire de matrice
    - o inmultire de matrice patratica de dimensiune KMAX are $KMAX^3$ operatii
  - aceasta metoda este eficienta cand $KMAX << n$ (KMAX este mult mai mic decat n)
- **complexitatea spatiala** : $S = O(KMAX^3)$
  - explicatie
    - este nevoie sa stocam cateva matrici

Observatie! In ultimele calcule nu am sters contanta KMAX, intrucat apare la puterea a 3-a! $KMAX = 100$ implica $KMAX^3 = 10^6$, valoare care nu mai poate fi ignorata in practica ($KMAX^3$ poate fi comparabil cu n).

Gardurile lui Gigel (optimizare)

Dupa cum am vazut mai sus, in problema cu garduri data de Gigel solutia este o recurenta liniara:

- $dp[1] = dp[2] = dp[3] = 1$; $d[4] = 2$;
- $dp[i] = dp[i-1] + dp[i-4]$, pentru $i > 4$

Exponentiere rapida

- $k = 4$
- $S_4 = (dp[1], dp[2], dp[3], dp[4]) = (1, 1, 1, 4)$
- $S_i = (dp[i-3], dp[i-2], dp[i-1], dp[i])$
- Raspunsul se afla efectuand operatia $S_n = S_4 * C^{n-4}$, unde C are urmatorul continut:

$$C = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Mai jos se afla o implementare simplista in C++ care cuprinde toate etapele pe care trebuie sa le realizati in cod, dupa ce stiti cum arata recurenta sub forma matriceala.

```
#define MOD  1009
#define KMAX 4

// C = A * B
void multiply_matrix(int A[KMAX][KMAX], int B[KMAX][KMAX], int C[KMAX][KMAX]) {
    int tmp[KMAX][KMAX];

    // tmp = A * B
    for (int i = 0; i < KMAX; ++i) {
        for (int j = 0; j < KMAX; ++j) {
            unsigned long long sum = 0; // presupun ca suma  incape pe 64 de biti

            for (int k = 0; k < KMAX; ++k) {
                sum += 1LL * A[i][k] * B[k][j];
            }

            tmp[i][j] = sum % MOD;
        }
    }

    //  C = tmp
    memcpy(C, tmp, sizeof(tmp));
}

// R = C^p
void power_matrix(int C[KMAX][KMAX], int p, int R[KMAX][KMAX]) {
    // tmp = I (matricea identitate)
    int tmp[KMAX][KMAX];
    for (int i = 0; i < KMAX; ++i) {
        for (int j = 0; j < KMAX; ++j) {
            tmp[i][j] = (i == j) ? 1 : 0;
        }
    }

    while (p != 1) {
        if  (p % 2 == 0) {
            multiply_matrix(C, C, C);     // C = C*C
            p /= 2;                       // ramane de calculat C^(p/2)
        } else {
            // reduc la cazul anterior:
            multiply_matrix(tmp, C, tmp); // tmp = tmp*C
            --p;                          // ramane de calculat C^(p-1)
        }
    }

    // avem o parte din rezultat in C si o parte in tmp
    multiply_matrix(C, tmp, R);           // rezultat = tmp * C
}

int garduri_rapide(int n) {
    // cazurile de baza
    if (n <= 3) return 1;
    if (n == 4) return 2;

    // construiesc matricea C
    int C[KMAX][KMAX] = { {0, 0, 0, 1},
                          {1, 0, 0, 0},
                          {0, 1, 0, 0},
                          {0, 0, 1, 1}};
   // vreau sa aplic formula S_n = S_4 * C^(n-4)

   // C = C^(n-4)
   power_matrix(C, n - 4, C);

   // sol = S_4 * C = dp[n] (se afla pe ultima pozitie din S_n,
   // deci voi folosi ultima coloana din C)
   int sol = 1 * C[0][3] + 1 * C[1][3] + 1 * C[2][3] + 2 * C[3][3];
   return sol % MOD;
}
```

Remarcati faptul ca in functia de inmultire se foloseste o matrice temporara tmp. Motivul este ca vrem sa apelam functia $multiply(C, C, C)$, unde C joaca atat rol de intrare cat si de iesire. Daca am pune rezultatele direct in C, atunci am strica inputul inainte sa obtinem rezultatul.

Putem spune ca acea functie este **matrix_multiply_safe**, in sensul ca pentru orice A,B,C care respecta dimensiunile impuse, functia va calcula corect produsul.

In arhiva **demo-lab04.zip** gasiti o sursa completa in care se realizeaza:

- o verificare a faptului ca cele 2 implementari (**gardurile_lui_Gigel** si **garduri_rapide**) produc aceleasi rezultate
- un benchmark in care cele 2 implementari sunt comparate
    - pe sistem uzual (laptop) s-au obtinut urmatoarele rezulate:

```
test case: varianta simpla
n = 100000000 sol = 119; time = 0.984545 s
test case: varianta rapida
n = 100000000 sol = 119; time = 0.000021 s


test case: varianta simpla
n = 1000000000 sol = 812; time = 9.662377 s
```

```
test case: varianta rapida
n = 1000000000 sol = 812; time = 0.000022 s
```

- se observa clar diferenta intre cele 2 solutii (am confirmat ceea ce spunea si teoria: $O(n)$ vs $O(log(n))$); aceasta tehnica imbunatateste drastic o solutie gasita relativ usor.

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab04.zip.

DP or math?

Fie un sir de **numere naturale strict pozitive**. Cate **subsiruri** (submultimi nevide) au suma numerelor **para**?

subsir (**subsequence** in engleza) pentru un vector **v** inseamna un alt vector $u = [v[i_1], v[i_2], \ldots, v[i_k]]$ unde $i_1 < i_2 < \ldots < i_k$.

Task-uri:

- Se cere o **solutie folosind DP**.
- Inspectand recurenta gasita la punctul precedent, incercati sa o inlocuiti cu o **formula matematica**.
- Care este **complexitatea** pentru fiecare solutie (timp + spatiu)? Care este mai buna? De ce? :D

Deoarece rezultatul poate fi prea mare, se cere **restul impartirii** lui la 1000000007 ($10^9 + 7$).

Pentru punctaj maxim pentru aceasta problema, este necesar sa rezolvati toate subpunctele (ex. nu folositi direct formula, gasiti mai intai recurenta DP). Trebuie sa implementati **cel putin** solutia cu DP.

$n = 3$

i	1	2	3
v	2	6	4

Raspuns: 7

Explicatie: Toate subsirurile posibile sunt

- $[2]$
- $[2, 6]$
- $[2, 6, 4]$
- $[2, 4]$
- $[6]$
- $[6, 4]$
- $[4]$

Toate subsirurile de mai sus au suma para.

$n = 3$

i	1	2	3
v	2	1	3

Raspuns: 3

Explicatie: Toate subsirurile posibile sunt

- $[2]$
- $[2, 1]$
- $[2, 1, 3]$
- $[2, 3]$
- $[1]$
- $[1, 3]$
- $[3]$

Subsirurile cu suma para sunt: $[2]$, $[2, 1, 3]$, $[1, 3]$.

$n = 3$

i	1	2	3
v	3	2	1

Raspuns: 3

Explicatie: Toate subsirurile posibile sunt

- $[3]$
- $[3, 2]$
- $[3, 2, 1]$
- $[3, 1]$
- $[2]$
- $[2, 1]$

- [1]

Subsirurile cu suma para sunt: [3, 2, 1], [3, 1], [2].

Morala: există probleme pentru care găsim o soluție cu DP, dar pentru care pot exista și alte soluții mai bune (am ignorat citirea/afișarea).

In problemele de numarat, exista o **sansa** buna sa putem gasi (si) o formula matematica, care poate fi implementata intr-un mod mai eficient decat o recurenta DP.

Cate subsiruri au suma **impara**?

Expresie booleana

Se da o expresie booleana corecta cu n termeni. Fiecare din termeni poate fi unul din stringurile **true**, **false**, **and**, **or**, **xor**.

Numarati modurile in care se pot aseza paranteze astfel incat rezultatul sa fie **true**. Se respecta regulile de la logica (tabelele de adevar pentru operatiile **and**, **or**, **xor**).

Deoarece rezultatul poate fi prea mare, se cere **restul impartirii** lui la 1000000007 ($10^9 + 7$).

In schelet vom codifica cu valori de tip char cele 5 stringuri:

- **false**: 'F'
- **true**: 'T'
- **and**: '&'
- **or**: '|'
- **xor**: '^'

Functia pe care va trebui sa o implementati voi va folosi variabilele **n** (numarul de termeni) si **expr** (vectorul cu termenii expresiei).

$n = 5$ si `expr = ['T', '&', 'F', '^', 'T']` (expr = [**true and false xor true**])

Raspuns: 2

Explicatie: Exista 2 moduri corecte de a paranteza expresia astfel incat sa obtinem rezultatul **true** (1).

- `T&(F^T)`
- `(T&F)^T`

```
Complexitate temporală dorita este $O(n ^ 3)$.

Optional, se pot defini functii ajutatoare precum **is_operand**, **is_operator**, **evaluate**.
```

Pentru rezolvarea celor doua probleme ganditi-va la ce scrie in sectiunea Sfaturi / Reguli [http://ocw.cs.pub.ro/courses/pa/laboratoare/laborator-04?&#sfaturireguli].
Pentru fiecare dintre cele doua probleme facem o **partitionare dupa un anumit criteriu**.

Pentru problema **DP or math?** partitionam toate subsirurile dupa critieriul **paritatii sumei subsirului** (cate sunt pare/impare).
Pentru problema **expresie booleana** partitionam **toate parantezarile posibile dupa rezultatul lor** (cate dau true/false).

Bonus

Asistentul va alege una dintre problemele din sectiunea Extra.

Recomandam sa **NU** fie una din cele 3 probleme de la Test PA 2017. Recomandam sa le incercati dupa ce recapitulati acasa DP1 si DP2, pentru a verifica daca cunostintele acumulate sunt la nivelul asteptat.

Extra

Rezolvati problema extraterestrii [https://www.hackerrank.com/contests/test-practic-pa-2017-v1-plumbus/challenges/test-1-extraterestrii] de la Test PA 2017.

Rezolvati problema Secvente [https://www.hackerrank.com/contests/test-practic-pa-2017-v1-plumbus/challenges/test-1-secvente] de la Test PA 2017.

Rezolvati problema PA Country [https://www.hackerrank.com/contests/test-practic-pa-2017-v2-meeseeks/challenges/test-2-pa-country-medie] de la Test PA 2017.

Rezolvati pe infoarena problema iepuri [http://infoarena.ro/problema/iepuri].

Hint: Exponentiere logaritmica pe matrice

Solutie:

- $dp[0] = X; dp[1] = Y; dp[0] = Z;$
- $dp[i] = (A * dp[i-1] + B * dp[i-2] + C * dp[i-3]) \% 666013$

Pentru punctaj maxim, pentru fiecare test se foloseste ecuatia matriceala atasata. Complexitate: $O(T * log(n))$.

Rezolvati pe leetcode problema Minimum Path Sum [https://leetcode.com/problems/minimum-path-sum/description/#].

Rezolvati pe infoarena problema Lacusta [http://infoarena.ro/problema/Lacusta].

Rezolvati pe infoarena problema Suma4 [http://infoarena.ro/problema/Suma4].

Rezolvati pe infoarena problema subsir [https://www.infoarena.ro/problema/subsir].

Rezolvati pe infoarena problema 2sah [https://infoarena.ro/problema/2sah].

Hint: Exponentiere logaritmica pe matrice

O descrie detaliata se afla in arhiva OJI 2015 [http://olimpiada.info/oji2015/index.php?cid=arhiva].

Referințe

[0] Capitolul **Dynamic Programming** din **Introductions to Algorithms** de către T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein

[1] http://infoarena.ro/problema/podm [http://infoarena.ro/problema/podm]

[2] http://infoarena.ro/problema/kfib [http://infoarena.ro/problema/kfib]

pa/laboratoare/laborator-04.txt · Last modified: 2020/03/17 16:20 by radu.iacob

Laborator 5: Backtracking

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Autori:

- Radu Stochitoiu (2018) [mailto:radu.stochitoiu@gmail.com]
- Razvan Chitu (2018) [mailto:razvan.m.chitu@gmail.com]

Obiective laborator

- Intelegerea notiunilor de baza despre backtracking;
- Insusirea abilitatilor de implementare a algoritmilor bazati backtracking;
- Rezolvarea unor probleme NP-complete în timp exponential.

Precizari initiale

Toate exemplele de cod se gasesc in demo-lab05.zip. Acestea apar incorporate si in textul laboratorului pentru a facilita parcurgerea cursiva a laboratorului.

Ce este Backtracking?

Backtracking este un algoritm care cauta **una sau mai multe solutii** pentru o problema, printr-o cautare exhaustiva, mai eficienta insa in general decat o abordare „genereaza si testeaza", de tip „forta bruta", deoarece un candidat partial care nu duce la o solutie este abandonat. Poate fi folosit pentru orice problema care presupune o cautare in **spatiul starilor**. In general, in timp ce cautam o solutie e posibil sa dam de o infundatura in urma unei alegeri gresite sau sa gasim o solutie, dar sa dorim sa cautam in continuare alte solutii. In acel moment trebuie sa ne intoarcem pe pasii facuti (**backtrack**) si la un moment dat sa luam alta decizie. Este relativ simplu din punct de vedere conceptual, dar complexitatea algoritmului este exponentiala.

Importanţa – aplicaţii practice

Exista foarte multe probleme (de exemplu, probleme NP-complete sau NP-dificile) care pot fi rezolvate prin algoritmi de tip backtracking mai eficient decat prin „forta bruta" (adica generarea tuturor alternativelor si selectarea solutiilor). Atentie insa, complexitatea computationala este de cele mai multe ori exponentiala. O eficientizare se poate face prin combinarea cu tehnici de propagare a restrictiilor. Orice problema care are nevoie de parcurgerea spatiului de stari se poate rezolva cu backtracking.

Descrierea problemei si a rezolvarilor

Pornind de la strategiile clasice de parcurgere a **spatiului de stari**, algoritmii de tip backtracking practic enumera un set de candidati partiali, care, dupa completarea definitiva, pot deveni solutii potentiale ale problemei initiale. Exact ca strategiile de **parcurgere în latime/adancime** si backtracking-ul are la baza expandarea unui nod curent, iar determinarea solutiei se face într-o maniera incrementala. Prin natura sa, backtracking-ul este recursiv, iar în arborele expandat top-down se aplica operatii de tipul pruning (taiere) daca solutia partiala nu este valida.

Algoritm de baza

```
/* Domain - domeniul curent (cu un cardinal mai mic decat la pasul trecut)
Solution - solutia curenta pe care o extindem catre cea finala */
back(Domain, Solution):
    if check(Solution):
        print(Solution)
        return

    for value in Domain:
        NextSolution = Solution.push(value)
        NextDomain = Domain.erase(value)
        back(NextDomain, NextSolution)
```

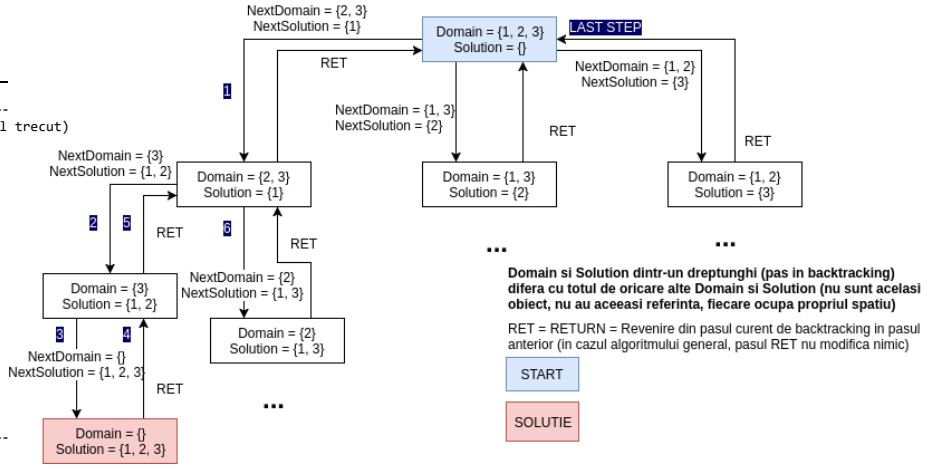
Algoritm de baza (modificat pentru transmitere prin referinta)

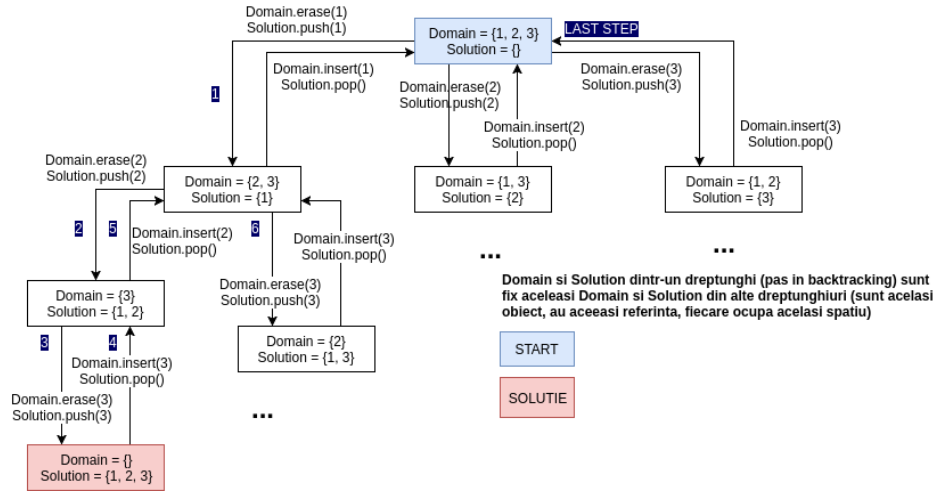
```
/* Domain - domeniul curent (cu un cardinal mai mic decat la pasul trecut)
Solution - solutia curenta pe care o extindem catre cea finala */
back(Domain, Solution):
    if check(Solution):
        print(Solution)
        return

    for value in Domain:
        /* DO */
        Solution = Solution.push(value)
        Domain = Domain.erase(value)

        /* RECURSION */
        back(Domain, Solution)

        /* UNDO */
        Solution = Solution.pop()
        Domain = Domain.insert(value)
```





Exemple clasice

Ne vom ocupa in continuare de urmatoarele probleme:

- Permutari
- Combinari
- Aranjamente
- Submultimi
- Genererare de siruri
- Problema damelor
- Problema soricelului
- Tic-Tac-Toe
- Sudoku
- Ultimate Tic-Tac-Toe

Sudoku si Ultimate Tic-Tac-Toe sunt probleme foarte grele. In general nu putem explora tot spatiul starilor pentru un input arbitrar dat.

Permutari

Enunt

Se da un numar N. Sa se genereze toate permutarile multimii formate din toate numerele de la 1 la N.

Exemple

N = 3 ⇒ M = {1, 2, 3}

Solutie:

- {1, 2, 3}
- {1, 3, 2}
- {2, 1, 3}
- {2, 3, 1}
- {3, 1, 2}
- {3, 2, 1}

Solutii

Backtracking (algoritmul in cazul general)

```
/* deoarece numerele sunt sterse din domeniu odata ce sunt folosite, solutia generata este garantata
sa nu contina duplicate. Astfel, atunci cand domeniul ajunge vid, solutia este intotdeauna corecta */
bool check(std::vector<int> solution) {
    return true;
}

void printSolution(std::vector<int> solution) {
    for (int s : solution) {
        std::cout << s << " ";
    }
    std::cout << "\n";
}

void back(std::vector<int> domain, std::vector<int> solution) {
    /* dupa ce am folosit toate elementele din domeniu putem verifica daca
    am gasit o solutie */
    if (domain.size() == 0) {
        if(check(solution)) {
            printSolution(solution);
        }
        return;
```

```
    }

    /* incercam sa adaugam in solutie toate valorile din domeniu, pe rand */
    for (unsigned int i = 0; i < domain.size(); ++i) {
        /* cream o solutie noua si un domeniu nou care sunt identice cu cele
        de la pasul curent */
        std::vector<int> newSolution(solution), newDomain(domain);

        /* adaugam in noua solutie elementul ales din domeniu */
        newSolution.push_back(domain[i]);
        /* stergem elementul ales din noul domeniu */
        newDomain.erase(newDomain.begin() + i);

        /* apelam recursiv backtracking pe noul domeniu si noua solutie */
        back(newDomain, newSolution);
    }
}

int main() {
    /* dupa ce am citit n initializam domeniul cu n elemente, numerele de la 1 la n,
    iar solutia este vida initial */
    std::vector<int> domain(n), solution;
    for (int i = 0; i < n; ++i) {
        domain[i] = i + 1;
    }

    /* apelam backtracking pe domeniul nostru, cautand solutia in vectorul solution */
    back(domain, solution);
}
```

Apelarea initiala (din “int main”) se face astfel: “back(domain, solution);”, unde domain reprezinta un vector cu elementele de la 1 la N, iar solution este un vector gol.

Nu este indicata implementarea backtracking-ului astfel deoarece este foarte costisitor din punct de vedere al memoriei, deoarece cream noi domenii si solutii la fiecare pas.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(n * n!) = O(n!)$
  - explicatie : Complexitate generarii permutarilor, $O(n!)$, se inmulteste cu complexitatea copierii vectorilor solutie si domeniu si a stergerii elementelor din domeniu, $O(n)$
- complexitate spatiala : $S(n) = O(n^2)$
  - explicatie : Fiecare nivel de recursivitate are propria lui copie a solutiei si a domeniului. Sunt n nivele de recursivitate, deci complexitatea spatiala este $O(n * n) = O(n^2)$

Backtracking (date transmise prin referinta)

```
/* deoarece numerele sunt sterse din domeniu odata ce sunt folosite, solutia generata este garantata sa nu contina duplicate. Astfel, atunci cand domeniul ajunge vid, solutia este intotde
bool check(std::vector<int> solution) {
    return true;
}

void printSolution(std::vector<int> &solution) {
    for (int s : solution) {
        std::cout << s << " ";
    }
    std::cout << "\n";
}

void back(std::vector<int> &domain, std::vector<int> &solution) {
    /* dupa ce am folosit toate elementele din domeniu putem verifica daca
    am gasit o solutie */
    if (domain.size() == 0) {
        if(check(solution)) {
            printSolution(solution);
        }
        return;
    }

    /* incercam sa adaugam in solutie toate valorile din domeniu, pe rand */
    for (unsigned int i = 0; i < domain.size(); ++i) {
        /* retinem valoarea pe care o scoatem din domeniu ca sa o readaugam dupa
        apelarea recursiva a backtracking-ului */
        int tmp = domain[i];

        /* adaug elementul curent la potentiala solutie */
        solution.push_back(domain[i]);
        /* sterg elementul curent din domeniu ca sa il pot pasa prin referinta
        si sa nu fie nevoie sa creez alt domeniu */
        domain.erase(domain.begin() + i);

        /* apelez recursiv backtracking pe domeniul si solutia modificate */
        back(domain, solution);

        /* refac domeniul si solutia la modul in care aratau inainte de apelarea
        recursiva a backtracking-ului, adica readaug elementul eliminat in
        domeniu si il sterg din solutie */
        domain.insert(domain.begin() + i, tmp);
        solution.pop_back();
    }
}

int main() {
    /* dupa ce am citit n initializam domeniul cu n elemente, numerele de la 1 la n,
    iar solutia este vida initial */
    std::vector<int> domain(n), solution;
    for (int i = 0; i < n; ++i) {
        domain[i] = i + 1;
    }

    /* apelam backtracking pe domeniul nostru, cautand solutia in vectorul solution */
```

```
    back(domain, solution);
}
```

Apelarea initiala (din “int main”) se face astfel: “back(domain, solution);”, unde domain reprezinta un vector cu elementele de la 1 la N, iar solution este un vector gol.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(n * n!)$
  - explicatie : Complexitate generarii permutarilor, $O(n!)$, se inmulteste cu complexitatea stergerii elementelor din domeniu, $O(n)$
- complexitate spatiala : $S(n) = O(n)$
  - explicatie : Spre deosebire de solutia anterioara, toate nivelele de recursivitate folosesc aceeasi solutie si acelasi domeniu. Complexitatea spatiala este astfel redusa la $O(n)$

Abordarea aceasta este mai eficienta decat cea generala prin evitarea folosirii memoriei auxiliare.

Backtracking (taierea ramurilor nefolositoare)

```
bool check(std::vector<int> &solution) {
    return true;
}

void printSolution(std::vector<int> &solution) {
    for (auto s : solution) {
        std::cout << s << " ";
    }
    std::cout << "\n";
}

void back(int step, int stop, std::vector<int> &domain,
        std::vector<int> &solution, std::unordered_set<int> &visited) {

    /* vom verifica o solutie atunci cand am adaugat deja N elemente in solutie,
    adica step == stop */
    if (step == stop) {
        /* deoarece am avut grija sa nu se adauge duplicate, "check()" va returna
        intotdeauna "true" */
        if(check(solution)) {
            printSolution(solution);
        }
        return;
    }

    /* Adaugam in solutie fiecare element din domeniu care *NU* a fost vizitat
    deja renuntand astfel la nevoia de a verifica duplicatele la final prin
    functia "check()" */
    for (unsigned int i = 0; i < domain.size(); ++i) {
        /* folosim elementul doar daca nu e vizitat inca */
        if (visited.find(domain[i]) == visited.end()) {
            /* il marcam ca vizitat si taiem eventuale expansiuni nefolositoare
            viitoare (ex: daca il adaug in solutie pe 3 nu voi mai avea
            niciodata nevoie sa il mai adaug pe 3 in continuare) */
            visited.insert(domain[i]);

            /* adaugam elementul curent in solutie pe pozitia pasului curent
            (step) */
            solution[step] = domain[i];

            /* apelam recursiv backtracking pentru pasul urmator */
            back(step + 1, stop, domain, solution, visited);

            /* stergem vizitarea elementului curent (ex: pentru N = 3, dupa ce
            la pasul "step = 0" l-am pus pe 1 pe prima pozitie in solutie si
            am continuat recursiv pana am ajuns la solutiile {1, 2, 3} si
            {1, 3, 2}, ne dorim sa il punem pe 2 pe prima pozitie in solutie si
            sa continuam recursiv pentru a ajunge la solutiile {2, 1, 3} etc.) */
            visited.erase(domain[i]);
        }
    }
}

int main() {
    /* dupa ce am citit n initializam domeniul cu n elemente, numerele de la 1 la n,
    iar solutia este initializata cu un vector de n elemente (deoarece o permutare
    contine n elemente) */
    std::vector<int> domain(n), solution(n);
    std::unordered_set<int> visited;
    for (int i = 0; i < n; ++i) {
        domain[i] = i + 1;
    }

    /* apelam back cu step = 0 (atatea elemente avem adaugate in solutie),
    stop = n (stim ca vrem sa adaugam n elemente in solutie pentru ca o
    permutare e alcatuita din n elemente), domain este vectorul de valori
    posibile, solution este vectorul care simuleaza stiva pe care o vom
    umple, visited este un unordered_set (initial gol) in care retinem daca
    un element din domeniu se afla deja in solutia curenta la un anumit pas */
    back(0, n, domain, solution, visited);
}
```

Apelarea initiala (din “int main”) se face astfel: “back(0, n, domain, solution, visited);”, unde domain reprezinta un vector cu elementele de la 1 la N, iar solution este un vector de n elemente, 0 este pasul curent, n este pasul la care dorim sa ne oprim, iar visited este map-ul care ne permite sa tinem cont de ce elemente au fost vizitate sau nu.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(n * n!) = O(n!)$

- explicatie : Complexitate generarii permutarilor, O(n!), se inmulteste cu complexitatea

iterarii prin domeniu, $O(n)$

- complexitate spatiala : $S(n) = O(n)$
  - explicatie : Toate nivelele de recursivitate folosesc aceeasi solutie si acelasi domeniu.

Aceasta solutie este optima si are complexitate temporala $T(n) = O(n!)$. Nu putem sa obtinem o solutie mai buna, intrucat trebuie sa generam n! permutari.

De asemenea, este optima si din punct de vedere spatial, intrucat trebuie sa avem $S(n) = O(n)$, din cauza stocarii permutarii generate.

Combinari

Enunt

Se dau numerele N si K. Sa se genereze toate combinarile multimii formate din toate numerele de la 1 la N, luate cate K.

Exemple

N = 4, K = 2 ⇒ M = {1, 2, 3, 4}

Solutie:

- {1, 2}
- {1, 3}
- {1, 4}
- {2, 3}
- {2, 4}
- {3, 4}

Solutii

Backtracking (taierea ramurilor nefolositoare)

```
bool check(std::vector<int> &solution) {
    return true;
}

void printSolution(std::vector<int> &solution, std::vector<int> &domain, int stop) {
    for (unsigned i = 0; i < stop; ++i) {
        std::cout << domain[solution[i]] << " ";
    }
    std::cout << "\n";
}

void back(int step, int stop, std::vector<int> &domain,
        std::vector<int> &solution) {
    /* vom verifica o solutie atunci cand am adaugat deja K elemente in solutie,
    adica step == stop */
    if (step == stop) {
        /* deoarece am avut grija sa se adauge elementele doar in ordine
        crescatoare, "check()" va returna intotdeauna "true" */
        if(check(solution)) {
            printSolution(solution, domain, stop);
        }
        return;
    }

    /* daca este primul pas, alegem fiecare element din domeniu ca potential
    candidat pentru prima pozitie in solutie; altfel, pentru a elimina ramurile
    in care de exemplu {2, 1} se va genera dupa ce s-a generat {1, 2} (adica
    ar fi duplicat), vom folosi doar elementele din domeniu care sunt mai mari
    decat ultimul element adaugat in solutie (solution[step - 1]) */
    unsigned i = step > 0 ? solution[step - 1] + 1 : 0;
    for (; i < domain.size(); ++i) {
        solution[step] = i;
        back(step + 1, stop, domain, solution);
    }
}

int main() {
    /* dupa ce citim n si k initializam domeniul cu valorile de la 1 la n,
    iar solutia este initializata cu un vector de k elemente (fiindca o
    combinare de "n luate cate k" are k elemente) */
    std::vector<int> domain(n), solution(k);
    for (int i = 0; i < n; ++i) {
        domain[i] = i + 1;
    }

    back(0, k, domain, solution);
}
```

In aceasta solutie ne bazam pe faptul ca toate combinarile pot fi generate in ordine crescatoare, adica solutia {1, 3, 4} e echivalenta cu {4, 1, 3}.

Aceasta solutie este optima intrucat toate solutiile generate sunt corecte (de aceea functia check intoarce true). Deoarece problema cere obtinerea tuturor combinarilor, aceasta complexitate nu poate fi mai mica de Combinari(n, k).

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(Combinari(n, k))$

- complexitate spatiala : $S(n) = O(n + k) = O(n)$
  - explicatie : $k <= n$, deci $O(n + k) = O(n)$

Problema soricelului

Enunt

Se da un numar N si o matrice patratica de dimensiuni N x N in care elementele egale cu 1 reprezinta ziduri (locuri prin care nu se poate trece), iar cele egale cu 0 reprezinta spatii goale. Aceasta matrice are un soricel in celula (0, 0) si o bucata de branza in celula (N - 1, N - 1). Scopul soricelului e sa ajunga la bucata de branza. Afisati toate modurile in care poate face asta stiind ca acesta poate merge doar in dreapta sau in jos cu cate o celula la fiecare pas.

Exemple

- 2
- 0 1
- 0 0

Exista 1 drum posibil:

- (0, 0)→(1, 0)→(1, 1)

- 3
- 0 0 0
- 0 1 0
- 0 0 0

Exista 2 drumuri posibile:

- (0,0)→(0,1)→(0,2)→(1,2)→(2,2)
- (0,0)→(1,0)→(2,0)→(2,1)→(2,2)

- 4
- 0 0 0 1
- 0 1 1 0
- 0 0 0 0
- 0 0 0 0

Exista 4 drumuri posibile:

- (0,0)→(1,0)→(2,0)→(2,1)→(2,2)→(2,3)→(3,3)
- (0,0)→(1,0)→(2,0)→(2,1)→(2,2)→(3,2)→(3,3)
- (0,0)→(1,0)→(2,0)→(2,1)→(3,1)→(3,2)→(3,3)
- (0,0)→(1,0)→(2,0)→(3,0)→(3,1)→(3,2)→(3,3)

Solutii

Backtracking (transmitere prin referinta)

```
bool check(std::vector<std::pair<int, int> > &solution, int walls[100][100]) {
    for (unsigned i = 0; i < solution.size() - 1; ++i) {
        /* line_prev si col_prev reprezinta celula in care se afla soricelul la
        pasul i; line_next si col_next reprezinta celula in care se afla
        la pasul i + 1; trebuie sa fim siguri ca soricelul nu a ajuns pe zid
        si ca urmatoarea celula este sub sau in dreapta celulei curente */
        int line_prev = solution[i].first;
        int line_next = solution[i + 1].first;
        int col_prev = solution[i].second;
        int col_next = solution[i + 1].second;

        /* walls[x][y] == 1 inseamna ca este zid pe linia x, coloana y */
        if (walls[line_prev][col_prev] == 1 ||
                !((line_next == line_prev + 1 && col_next == col_prev) ||
                (line_next == line_prev && col_next == col_prev + 1))) {
            return false;
        }
    }

    return true;
}

void printSolution(std::vector<std::pair<int, int> > &solution) {
    for (std::pair<int, int> s : solution) {
        std::cout << "(" << s.first << "," << s.second << ")->";
    }
    std::cout << "\n";
}

void back(std::vector<std::pair<int, int> > &domain, int walls[100][100],
        std::vector<std::pair<int, int> > &solution, int max_iter) {
    /* daca am facut "max_iter" pasi ma opresc si verific daca este corecta
    solutia */
    if (solution.size() == max_iter) {
        if(check(solution, walls)) {
            printSolution(solution);
        }
        return;
    }

    /* avand domeniul initializat cu toate celulele din matrice, incercam sa
    adaugam oricare dintre aceste celule la solutie, verificand la final daca
```

```
    solutia este buna */
    for (unsigned int i = 0; i < domain.size(); ++i) {
        /* pastram elementul curent pentru a-l readauga in domeniu dupa
        apelarea recursiva */
        std::pair<int, int> tmp = domain[i];

        /* adaugam elementul curent la solutia candidat */
        solution.push_back(domain[i]);
        /* stergem elementul curent din domeniu */
        domain.erase(domain.begin() + i);

        /* apelam recursiv backtracking */
        back(domain, walls, solution, max_iter);

        /* adaugam elementul sters din domeniu inapoi */
        domain.insert(domain.begin() + i, tmp);
        /* stergem elementul curent din solutia candidat pentru a o forma pe
        urmatoarea */
        solution.pop_back();
    }
}

int main() {
    /* initializam domeniul si solutia ca vectori de perechi de int-uri;
    domeniul va contine initial toate perechile de indici posibile din
    matrice ((0, 0), (0, 1) ... (n - 1, n - 1)), iar solutia va fi initial
    vida */
    std::vector<std::pair<int, int> > domain, solution;

    fin >> n;
    for (i = 0; i < n; ++i) {
        for (j = 0; j < n; ++j) {
            /* walls[i][j] == 1 daca pe pozitia (i, j) este zid; 0 altfel */
            fin >> walls[i][j];
            domain.push_back({i, j});
        }
    }

    /* apelam back cu domeniul format initial, cu matricea de ziduri, cu
    solutia vida si cu numarul maxim de iteratii = 2 * n - 1 pentru ca
    mergand doar in dreapta si in jos, in 2 * n - 1 pasi va ajunge din
    (0, 0) in (n - 1, n - 1) */
    back(domain, walls, solution, 2 * n - 1);
}
```

Apelarea initiala (din “int main”) se face astfel: “back(domain, walls, solution, 2 * n - 1);”, unde domain reprezinta un vector cu perechi in care sunt toate celulele matricii, solution este un vector gol, walls este matricea care ne arata daca este zid sau nu pe o anumita pozitie, iar 2 * n - 1 e numarul e pasi in care ar trebui ca soricelul sa ajunga la branza pe oriunde ar merge.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(Aranjamente(n^2, 2n - 1))$
  - explicatie: Initial in domeniu avem n^2 valori. Noi dorim sa generam toate submultimile ordonate de cate $2n - 1$ elemente. Acestea sunt tocmai aranjamentele de n^2 luate cate $2n - 1$.
- complexitate spatiala : $S(n) = O(n^2)$
  - explicatie: Trebuie sa stocam informatie despre drum, care are $2n - 1$ celule; stocam domeniul care are n^2 elemente

Backtracking (taierea ramurilor nefolositoare)

```
bool check(std::vector<std::pair<int, int> > &solution) {
    return true;
}

void printSolution(std::vector<std::pair<int, int> > &solution) {
    for (std::pair<int, int> s : solution) {
        std::cout << "(" << s.first << "," << s.second << ")->";
    }
    std::cout << "\n";
}

void back(int step, int stop, int walls[100][100],
        std::vector<std::pair<int, int> > &solution, int line_moves[2],
        int col_moves[2]) {
    /* ne oprim dupa ce am ajuns la pasul "stop" si verificam daca solutia este
    corecta */
    if (step == stop) {
        /* deoarece am eliminat ramurile nefolositoare am ajuns la o solutie care
        sigur este corecta */
        if(check(solution)) {
            printSolution(solution);
        }
        return;
    }

    /* daca este primul pas stiu ca soricelul este in pozitia (0, 0) */
    if (step == 0) {
        /* adaugam (0, 0) la solutia candidat */
        solution.push_back({0, 0});

        /* apelam backtracking recursiv la pasul urmator */
        back(step + 1, stop, walls, solution, line_moves, col_moves);

        /* scoatem (0, 0) din solutie */
        solution.pop_back();
        return;
    }

    /* sunt doar doua mutari pe care le pot face intr-un pas: dreapta si jos;
    acestea sunt encodate prin vectorii de directii line_moves[2] = {0, 1} si
    col_moves[2] = {1, 0} care reprezinta la indicele 0 miscarea in dreapta, iar
    la indicele 1 miscarea in jos */
    for (unsigned int i = 0; i < 2; ++i) {
```

```
        /* cream noua linie si noua coloana cu ajutorul vectorilor de directii */
        int new_line = solution.back().first + line_moves[i];
        int new_col = solution.back().second + col_moves[i];
        int n = (stop + 1) / 2;

        /* daca linia si coloana sunt valide (nu ies din matrice) si nu este
        zid pe pozitia lor, putem continua pe acea celula */
        if (new_line < n && new_col < n && walls[new_line][new_col] == 0) {
            /* adaugam noua celula in solutia candidat;
            NOTE: {new_line, new_col} este echivalent cu
            std::pair<int, int>(new_line, new_col) si se numeste "initializer
            list", feature in C++11 */
            solution.push_back({new_line, new_col});

            /* apelam backtracking recursiv la pasul urmator */
            back(step + 1, stop, walls, solution, line_moves, col_moves);

            /* scoatem celula adaugata din solutie */
            solution.pop_back();
        }
    }
}

int main() {
    /* initializam solutia ca vector de perechi de int-uri */
    std::vector<std::pair<int, int> > solution;

    fin >> n;
    for (i = 0; i < n; ++i) {
        for (j = 0; j < n; ++j) {
            /* citim matricea zidurilor; 1 pentru zid, 0 altfel */
            fin >> walls[i][j];
        }
    }

    /* apelam back cu step = 0, stop = 2 * n - 1 deoarece in 2 * n - 1
    pasi soricelul va ajunge la branza, vectorul de ziduri, vectorul in
    care vom stoca solutia, vectorii de directii line_moves[2] = {0, 1} si
    col_moves[2] = {1, 0}; nu avem nevoie de domeniu deoarece folosind
    vectorii de directii vom sti din ultima pozitie pusa in solutie cele
    doua solutii in care putem merge, astfel domeniul nostru va fi alcatuit
    din doua solutii la fiecare pas (daca ultima pozitie din solutie a fost
    (5, 7) => domeniul pasului curent = {(5 + 0, 7 + 1) si (5 + 1, 7 + 0)}
    care este egal cu {(5, 8), (6, 7)}. */
    back(0, 2 * n - 1, walls, solution, line_moves, col_moves);
}
```

Pentru aceasta abordare la fiecare pas de backtracking vom merge doar in doua directii, in loc sa mergem in oricare celula din matrice, lucru care imbunatateste semnificativ complexitatea temporala.

Complexitate

Solutia va avea urmatoarele complexitati:

- complexitate temporala : $T(n) = O(2^{2n})$
  - explicatie: avem de urmat un sir de $2n - 1$ mutari, iar la fiecare pas avem 2 variante posibile
- complexitate spatiala : $S(n) = O(n)$
  - explicatie: stocam maximum $2n - 1$ casute

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab05.zip.

Aranjamente

Fie N si K doua **numere naturale strict pozitive**. Se cere afisarea tuturor aranjamentelor de N elemente luate cate K din multimea {1, 2, …, N}.

Fie N = 3, K = 2 ⇒ M = {1, 2, 3}

Solutie:

- {1, 2}
- {1, 3}
- {2, 1}
- {2, 3}
- {3, 1}
- {3, 2}

Se doreste o complexitate $T(n, k) = A(n, k)$.

Folositi-va de problema **Permutari**.

Solutiile se vor genera in ordine lexico-grafica!

Checkerul asteapta sa le stocati in aceasta ordine.

Submultimi

Fie N un **numar natural strict pozitiv**. Se cere afisarea tuturor submultimilor multimii {1, 2, …, N}.

Fie N = 4 ⇒ M = {1, 2, 3, 4}

Solutie: {} - multimea vida {1} {1, 2} {1, 2, 3} {1, 2, 3, 4} {1, 2, 4} {1, 3} {1, 3, 4} {1, 4} {2} {2, 3} {2, 3, 4} {2, 4} {3} {3, 4} {4}

Se doreste o complexitate $T(n) = O(2^n)$.

Folositi-va de problema **Combinari**.

Solutiile se vor genera in ordine lexico-grafica!

Checkerul asteapta sa le stocati in aceasta ordine.

Problema damelor

Problema damelor (sau problema reginelor) trateaza plasarea a 8 regine de sah pe o tablă de șah de dimensiuni 8 x 8 astfel incat sa nu existe doua regine care se ameninta reciproc. Astfel, se cauta **o solutie** astfel incat nicio pereche de doua regine sa nu fie pe acelasi rand, pe aceeasi coloana, sau pe aceeasi diagonala. Problema cu opt regine este doar un caz particular pentru problema generala, care presupune plasarea a N regine pe o tablă de sah N x N în aceleasi conditii. Pentru aceasta problema, există solutii pentru toate numerele naturale N cu excepția lui N = 2 si N = 3.

Fie N = 5

Solutie:

X	-	-	-	-
-	-	X	-	-
-	-	-	-	X
-	X	-	-	-
-	-	-	X	-

X reprezinta o dama, - reprezinta spatiu gol.

E nevoie sa facem backtracking pe matrice sau e suficient pe vector?

Se va cauta o singura solutie (**oricare** solutie corecta), care va fi returnata sub format unui vector cu $n + 1$ elemente.

Solutia este $sol[0], sol[1], \ldots, sol[n]$, unde $sol[i]$ = coloana unde vom plasa regina de pe linia i.

Elementul 0 este nefolosit, dorim sa pastram conventia cu indexare de la 1.

Generare de siruri

Vi se da o lista de caractere si o lista de frecvente (pentru caracterul de pe pozitia i, frecventa de pe pozitia i). Vi se cere sa generati toate sirurile care se pot forma cu aceste caractere si aceste frecvente stiind ca nu pot fi mai mult de K aparitii consecutive ale aceluiasi caracter.

Fie caractere[] = {'a', 'b', 'c'}, freq[] = {1, 1, 2}, K = 5

Solutie:

- abcc
- acbc
- accb
- bacc
- bcac
- bcca
- cabc
- cacb
- cbac
- cbca
- ccab
- ccba

Fie caractere[] = {'b', 'c'}, freq[] = {3, 2}, K = 2

Solutie:

- bbcbc
- bbccb
- bcbbc
- bcbcb
- bccbb
- cbbcb
- cbcbb

Solutiile se vor genera in ordine lexico-grafica!

Checkerul asteapta sa le stocati in aceasta ordine.

Bonus

Problema damelor (AC3)

Aplicati AC3 pe problema damelor.

Algoritmul AC-3 (Arc Consistency Algorithm) este de obicei folosit in probleme de satisfacere a constrangerilor (CSP). Acesta sterge arcele din arborele de stari care sigur nu se vor folosi niciodata.

AC-3 lucreaza cu:

- constrangeri
- variabile
- domenii de variabile

O variabila poate lua orice valoare din domeniul sau la orice pas. O constrangere este o relatie sau o limitare a unor variabile.

Exemplu AC-3

Consideram A, o variabila ce are domeniul D(A) = {0, 1, 2, 3, 4, 5, 6} si B o variabila ce are domeniul D(B) = {0, 1, 2, 3, 4}. Cunoastem constrangerile: C1 = "A trebuie sa fie impar" si C2 = "A + B trebuie sa fie egal cu 5".

Algoritmul AC-3 va elimina in primul rand toate valorile pare ale lui A pentru a respecta C1 ⇒ D(A) = {1, 3, 5}. Apoi, va incerca sa satisfaca C2, asa ca va pastra in domeniul lui B toate valorile care adunate cu valori din D(A) pot da 5 ⇒ D(B) = {0, 2, 4}.

AC-3 a redus astfel domeniile lui A si B, reducand semnificativ timpul folosit de algoritmul backtracking.

Extra

Immortal

Enunt [https://www.infoarena.ro/problema/immortal]

OJI 2010 - clasele 11-12 [http://olimpiada.info/oji2010/index.php?cid=arhiva]

Ultimate Tic Tac Toe

The AI Games - Ultimate Tic Tac Toe [http://theaigames.com/competitions/ultimate-tic-tac-toe]

pa/laboratoare/laborator-05.txt · Last modified: 2020/03/21 19:44 by darius.neatu

Laborator 06: Minimax

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- Insusirea unor cunostinte de baza despre **teoria jocurilor** precum si despre **jocurile de tip joc de suma zero (suma nula, zero-sum games)**
- Insusirea abilitatii de rezolvare a problemelor ce presupun cunoasterea si exploatarea conceptului de joc de suma zero (zero-sum game);
- Insusirea unor cunostinte elementare despre algoritmii necesari rezolvarii unor probleme de joc de suma zero (zero-sum game).

Precizari initiale

Un curs foarte bine explicat este pe canalul de YouTube de la MIT. Va sfatuim sa vizionati integral Search: Games, Minimax, and Alpha-Beta [https://www.youtube.com/watch?v=STjW3eH0Cik&t=2480s&list=PLqUJoA5RTkI7UbJXJaHX80FotR1QJWHJt&index=1] inainte sa parcurgeti materialul de pe ocw.

Importanţă – aplicaţii practice

Algoritmul **Minimax** si variantele sale imbunatatite (**Negamax**, **Alpha-Beta** etc.) sunt folosite in diverse domenii precum teoria jocurilor (Game Theory), teoria jocurilor combinatorice (Combinatorial Game Theory – CGT), teoria deciziei (Decision Theory) si statistica.

Astfel, diferite variante ale algoritmului sunt necesare in proiectarea si implementarea de aplicatii legate de inteligenta artificiala, economie, dar si in domenii precum stiinte politice sau biologie.

Descrierea problemei și a rezolvărilor

Algoritmii Minimax permit abordarea unor probleme ce tin de teoria jocurilor combinatorice. CGT este o ramura a matematicii ce se ocupa cu studierea jocurilor in doi (two-player games), in care participantii isi modifica rand pe rand pozitiile in diferite moduri, prestabilite de regulile jocului, pentru a indeplini una sau mai multe conditii de castig.

Exemple de astfel de jocuri sunt: sah, go, dame (checkers), X si O (tic-tac-toe) etc.

CGT nu studiaza jocuri ce presupun implicarea unui element aleator (sansa) in derularea jocului precum poker, blackjack, zaruri etc. Astfel decizia abordarii unor probleme rezolvabile prin metode de tip Minimax se datoreaza in principal simplitatii atat conceptuale, cat si raportat la implementarea propriu-zisa.

Minimax

Strategia pe care se bazeaza ideea algoritmului este ca jucatorii implicati adopta urmatoarele strategii:

- Jucatorul 1 (**maxi**) va incerca mereu sa-si **maximizeze** propriul castig prin mutarea pe care o are de facut;

- Jucatorul 2 (**mini**) va incerca mereu sa **minimizeze** castigul jucatorului 1 la fiecare mutare.

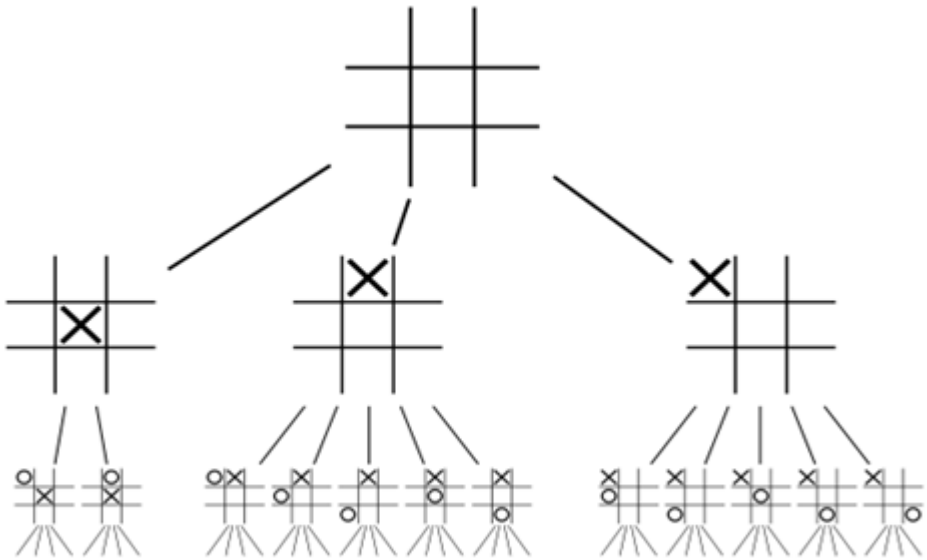
De ce merge o astfel de abordare? Dupa cum se preciza la inceput, discutia se axeaza pe jocuri de suma zero (zero-sum game). Acest lucru garanteaza, printre altele, ca orice castig al Jucatorului 1 este egal cu modulul sumei pierdute de Jucatorul 2. Cu alte cuvinte cat pierde Jucator 2, atat castiga Jucator 1. Invers, cat pierde Jucator 1, atat castiga Jucator 2. Sau

$$Win_{Player_1} = |Loss_{Player_2}|$$

$$|Loss_{Player_1}| = Win_{Player_2}$$

Reprezentarea spatiului solutiilor

In general spatiul solutiilor pentru un joc in doi de tip zero-sum se reprezinta ca un **arbore**, fiecarui nod fiindu-i asociata o stare a jocului in desfasurare (game state). Pentru exemplul nostru de X si O putem considera urmatorul arbore (partial) de solutii, ce corespunde primelor mutari ale lui X, respectiv O:



Metodele de reprezentare a arborelui variaza in functie de paradigma de programare aleasa, de limbaj, precum si de gradul de optimizare avut in vedere.

Avand notiunile de baza asupra strategiei celor doi jucatori, precum si a reprezentarii spatiului solutiilor problemei, putem formula o prima varianta a algoritmului Minimax:

```
int evaluate(stare);   // functia returneaza un scor asociat cu starea
                       // functia returneaza mereu scorul din perspectiva lui maxi


void apply_move(move); // functia modifica starea curenta: executa miscarea move
void undo_move(move);  // functia restaureaza starea anterioara (de dinainte de executa lui move)

// alege cea mai buna mutare pentru jucatorul maxi
// functia returneaza best_score pentru maxi
int maxi(int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
        return evaluate();            // ne oprim si evaluam starea curenta
    }

    int max = -oo;                    // jucatorul maxi doreste sa-si maximizeze

    // incercam pe rand fiecare care miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move
```

```
        // incercam sa simulam jocul mai departe:
        // daca maxi face move, ce ar face mini?
        int score = mini(depth - 1);

        // dintre toate variantele pe care mini le permite (!!!),
        // maxi o va alege pe cea cu scor maxim
        if (score > max) {
            max = score;
        }

        undo_move(move);         // restaureaza starea de dinainte de move
    }

    // cel mai bun scor pe care il poate obtine maxi este max
    // (s-a tinut cont si de ce i-ar permite mini, avand in vedere ca si el joaca optim)
    return max;
}

// alege cea mai buna mutare pentru jucatorul mini
// functia returneaza best_score pentru mini
int mini(int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
        return -evaluate();            // ne oprim si evaluam starea curenta
    }

    int min = +oo;                    // jucatorul mini doreste sa minimizeze scorul lui maxi

    // incercam pe rand fiecare care miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca mini face move, ce ar face maxi?
        int score = maxi(depth - 1);

        // dintre toate variantele pe care maxi le permite (!!!),
        // mini o va alege pe cea cu scor minim
        if (score < min) {
            min = score;
        }

        undo_move(move);         // este o functie care incearca sa refaca
                                 // starea de dinainte de aplicarea lui move
    }

    // cel mai bun scor pe care il poate obtine mini este min
    // (s-a tinut cont si de ce i-ar permite maxi, avand in vedere ca si el joaca optim)
    return min;
}
```

Argumentarea utilizarii unei adancimi maxime

Datorita **spatiului de solutii mare**, de multe ori coplesitor ca volum, o inspectare completa a acestuia nu este fezabila si devine impracticabila din punctul de vedere al timpului consumat sau chiar a memoriei alocate (se vor discuta aceste aspecte in paragraful legat de complexitate).

Astfel, de cele mai multe ori este preferata o abordare care parcurge arborele numai pana la o anumita **adancime maxima („depth”)**. Aceasta abordare permite examinarea arborelui destul de mult pentru a putea lua decizii minimalist coerente in desfasurarea jocului.

Totusi, **dezavantajul major** este ca pe termen lung se poate dovedi ca decizia luata la adancimea depth nu este global favorabila jucatorului in cauza.

De asemenea, se observa recursivitatea indirecta. Prin conventie acceptam ca **inceputul algoritmului** sa fie cu functia maxi. Astfel, se analizeaza succesiv diferite stari ale jocului din punctul de vedere al celor doi jucatori pana la adancimea depth. Rezultatul intors este scorul final al miscarii celei mai bune.

Negamax

Negamax este o varianta a minimax, ce se bazeaza pe urmatoarea observatie: intr-un joc cu suma zero castigul unui jucator este egal cu modulul sumei pierdute de celalalt jucator si invers.

Intr-adevar putem spune ca jucatorul mini incearca de fapt sa maximizeze in modul suma pierduta de maxi. Astfel putem formula urmatoarea implementare ce profita de observatia de mai sus.

Nota: putem exprima aceasta observatie si pe baza formulei **max(a, b) = -min(-a, -b)**.

```
int evaluate(stare);   // functia returneaza un scor asociat cu starea
                       // functia returneaza mereu scorul din perspectiva jucatorului CURENT!


void apply_move(move); // functia modifica starea curenta: executa miscarea move
void undo_move(move);  // functia restaureaza starea anterioara (de dinainte de executa lui move)

// alege cea mai buna mutare pentru jucatorul CURENT (EU)
// functia returneaza best_score
int negamax(int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
       return evaluate();            // ne oprim si evaluam starea curenta
    }

    int max = -oo;                    // jucatorul CURENT doreste sa-si  maximizeze scorul

    // incercam pe rand fiecare care miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca jucatorul CURENT face face move, ce ar face ADVERSARUL?
        int score = -negamax(depth - 1);  // cel mai bine pentru el,
                                          // este cel mai rau pentru mine si invers

        // dintre toate variantele pe care ADVERSARUL le permite (!!!),
        // EU (jucatorul CURENT) o voi alege pe cea cu scor maxim
        if (score > max) {
            max = score;
        }

        undo_move(move);         // restaureaza starea de dinainte de move
    }

    // cel mai bun scor pe care il pot obtine EU este max
    // (s-a tinut cont si de ce mi-ar permite ADVERSARUL, avand in vedere ca si el joaca optim)
    return max;
}
```

Se observa direct avantajele acestei formulari fata de Minimax-ul standard prezentat anterior:

- **claritatea** si **eleganta** sporita a codului
- usurinta in **intretinerea** si **extinderea** functionalitatii

Din punctul de vedere al complexitatii temporale, Negamax nu difera absolut deloc de Minimax (ambele examineaza acelasi numar de stari in arborele de solutii).

Putem concluziona ca este de **preferat** o implementare ce foloseste **negamax** fata de una bazata pe minimax in rezolvarea unor probleme ce tin de aceasta tehnica.

Alpha-beta pruning

Pana acum s-a discutat despre algoritmii Minimax / Negamax. Acestia sunt algoritmi exhaustivi (**exhausting search algorithms**). Cu alte cuvinte, ei gasesc solutia optima examinand intreg spatiul de solutii al problemei. Acest mod de abordare este extrem de ineficient in ceea ce priveste efortul de calcul necesar, mai ales considerand ca extrem de multe stari de joc inutile sunt explorate (este vorba de acele stari care nu pot fi atinse datorita incalcarii principiului de maximizare a castigului la fiecare runda).

O imbunatatire substantiala a minimax/negamax este **Alpha-beta pruning** (**taiere alfa-beta**). Acest algoritm incearca sa optimizeze mini/negamax profitand de o observatie importanta: **pe parcursul examinarii arborelui de solutii se pot elimina intregi subarbori, corespunzatori unei miscari m, daca pe parcursul analizei gasim ca miscarea m este mai slaba calitativ decat cea mai buna miscare curenta.**

Astfel, consideram ca pornim cu o prima miscare M1. Dupa ce analizam aceasta miscare in totalitate si ii atribuim un scor, continuam sa analizam miscarea M2. Daca in analiza ulterioara gasim ca adversarul are cel putin o miscare care transforma M2 intr-o miscare mai slaba decat M1 atunci orice alte variante ce corespund miscarii M2 (subarbori) nu mai trebuie analizate.

De ce? Pentru ca stim ca exista **cel putin** o varianta in care adversarul obtine un castig mai bun decat daca am fi jucat miscarea M1.

Nu conteaza exact cat de slaba poate fi miscarea M2 fata de M1. O analiza amanuntita ar putea releva ca poate fi si mai slaba decat am constatat initial, insa acest lucru este irelevant.

De ce insa ignoram intregi subarbori si miscari potential bune numai pentru o miscare slaba gasita? Pentru ca, in conformitate cu **principiul de maximizare al castigului** folosit de fiecare jucator, adversarul va alege exact acea miscare ce ii va da un castig maximal. Daca exista o varianta si mai buna pentru el este irelevant, deoarece noi suntem interesati daca cea mai slaba miscare buna a lui este mai buna decat miscarea noastra curent analizata.

O observatie foarte importanta se poate face analizand **modul de functionare** al acestui algoritm: este extrem de importanta **ordonarea miscarilor dupa valoarea castigului**.

In **cazul ideal** in care cea mai buna miscare a jucatorului curent este analizata prima, toate celelalte miscari, fiind mai slabe, vor fi eliminate din cautare timpuriu.

In **cazul cel mai defavorabil** insa, in care miscarile sunt ordonate crescator dupa castigul furnizat, Alpha-beta are aceeasi compelxitate cu Mini/Nega-max, neobtinandu-se nicio imbunatatire.

In **medie** se constata o imbunatatire vizibila a algoritmului Alpha-beta fata de Mini/Nega-max.

Rolul miscarilor analizate la inceput presupune stabilirea unor **plafoane de minim si maxim** legate de cat de bune/slabe pot fi miscarile.

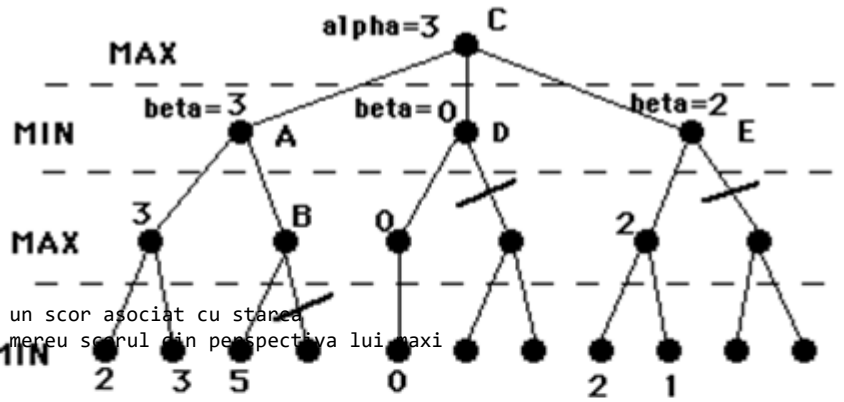
Astfel, plafonul de minim (Lower Bound), numit **alpha** stabileste ca o miscare nu poate fi mai slaba decat valoarea acestui plafon. Plafonul de maxim (Upper Bound), numit **beta**, este important doarece el foloseste la a stabili daca o miscare este prea buna pentru a fi luata in considerare. Depasirea plafonului de maxim inseamna ca o miscare este atat de buna incat adversarul nu ar fi permis-o, adica mai sus in arbore exista o miscare pe care ar fi putut s-o joace pentru a nu ajunge in situatia curent analizata.

Astfel alpha si beta furnizeaza o fereastra folosita pentru a filtra miscarile posibile pentru cei doi jucatori. Evident aceasta fereastra se poate actualiza pe masura ce se analizeaza mai multe miscari. De exemplul plafonul minim alpha se mareste pe masura ce gasim anumite tipuri de miscari mai bune (**better worst best moves**). Asadar, in implementare tinem seama si de aceste doua plafoane. In conformitate cu principiul Minimax, plafonul de minim al unui jocator (alpha-ul) este plafonul de maxim al celuilalt (beta-ul) si invers. Prezentam in continuare o descriere grafica a algoritmului Alpha-beta:

Un video cu un exemplu detaliat si foarte bine explicat se gaseste in tutorialul recomandat de pe YouTube (de la minutul 21:30 la 30:30).

Implementare

In continuare prezentam o implementare conceptuala a Alpha-beta, atat pentru Minimax, cat si pentru Negamax:



1. Start at C. Descend to full-ply depth and assign the heuristic to a state and all siblings (MIN 2, 3). Back up these values to their parent node (MAX 3).
2. Offer this value to the grandparent (A), as its beta value. So **A has beta=3**. A will be no larger than 3
3. Descend to A's other grandchildren. Terminate the search of their parent if any grandchildren is >= A's beta. **Node B is beta-pruned**, as shown, because its value must be at least 5.
4. Once A's value is known, offer it to its parent (C) as its alpha value. So **C has alpha=3**. C will be no smaller than 3.
5. Repeat this process, descending to C's great grandchildren (0) in a depth-first fashion. **D is alpha-pruned**, because no matter what happens on its right branch, it cannot be greater than 0.
6. Repeating on E, **E is alpha-pruned** because its beta value (2) is less than its parent's alpha value (3). So no matter what happens on its right branch, E cannot have a value greater than 2.
7. Therefore **C is 3.**

```
-----------------------------------------------
int evaluate(stare);   // functia returneaza un scor asociat cu starea
                       // functia returneaza mereu scorul din perspectiva lui maxi


void apply_move(move); // functia modifica starea curenta: executa miscarea move
void undo_move(move);  // functia restaureaza starea anterioara (de dinainte de executa lui move)

// alege cea mai buna mutare pentru jucatorul maxi
// functia returneaza best_score pentru maxi, dar tine cont si de ce ii permite mini
int aplhabeta_maxi(int alpha, int beta, int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
        return evaluate();            // ne oprim si evaluam starea curenta
    }

    // incercam pe rand fiecare miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca maxi face move, ce ar face mini?
        int score = alphabeta_mini(alpha, beta, depth - 1);

        if (score >= beta) {
            return beta;    // beta cut-off
        }

        // dintre toate variantele pe care mini le permite (!!!),
        // maxi o va alege pe cea cu scor maxim
        if (score > alpha) {
            alpha = score;
        }

        undo_move(move);         // restaureaza starea de dinainte de move
   }

    // cel mai bun scor pe care il poate obtine maxi este alpha
    // (s-a tinut cont si de ce i-ar permite mini, avand in vedere ca si el joaca optim)
    return alpha;
}


// alege cea mai buna mutare pentru jucatorul mini
// functia returneaza best_score pentru mini, dar tine cont si de ce ii permite maxi
int aplhabeta_mini(int alpha, int beta, int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (gameOver() || depth == 0 ) {
        return -evaluate();            // ne oprim si evaluam starea curenta
    }

    // incercam pe rand fiecare miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca mini face move, ce ar face maxi?
        int score = alphabeta_maxi(alpha, beta, depth - 1);

        if (score <= alpha) {
            return alpha;    // alpha cut-off
        }

        // dintre toate variantele pe care maxi le permite (!!!),
```

```
        // mini o va alege pe cea cu scor minim
        if (score < beta) {
            beta = score;
        }

        undo_move(move);         // restaureaza starea de dinainte de move
    }

    // cel mai bun scor pe care il poate obtine mini este beta
    // (s-a tinut cont si de ce i-ar permite maxi, avand in vedere ca si el joaca optim)
    return beta;
}
```

```
int evaluate(stare);   // functia returneaza un scor asociat cu starea
                       // functia returneaza mereu scorul din perspectiva jucatorului CURENT!


void apply_move(move); // functia modifica starea curenta: executa miscarea move
void undo_move(move);  // functia restaureaza starea anterioara (de dinainte de executa lui move)

// alege cea mai buna mutare pentru jucatorul CURENT (EU)
// functia returneaza best_score
int alphabeta_negamax(int aplha, int beta, int depth) {
    // daca jocul s-a terminat sau am atins nivelul maxim de recursivitate ales
    if (depth == 0 || gameOver()) {
       return evaluate();            // ne oprim si evaluam starea curenta
    }

    int max = -oo;

    // incercam pe rand fiecare care miscare posibila move
    for (move : all_moves) {
        apply_move(move);         // executa move

        // incercam sa simulam jocul mai departe:
        // daca jucatorul CURENT face face move, ce ar face ADVERSARUL?
        int score = -alphabeta_negamax(-beta, -alpha, depth - 1);  // cel mai bine pentru el,
                                                                   // este cel mai rau pentru mine
                                                                   // si invers

        // dintre toate variantele pe care ADVERSARUL le permite (!!!),
        // EU (jucatorul CURENT) o voi alege pe cea cu scor maxim
        if (score >= max) {
            max = score;
        }

        if (max > alpha) {
            alpha = max;
        }

        if (alpha >= beta) {
            break;               // cut-off
        }

        undo_move(move);         // restaureaza starea de dinainte de move
    }

    // cel mai bun scor pe care il pot obtine EU este alpha
    // (s-a tinut cont si de ce mi-ar permite ADVERSARUL, avand in vedere ca si el joaca optim)
    return alpha;
}
```

Din nou remarcam claritatea si coerenta sporita a variantei negamax!

Complexitate

In continuare prezentam complexitatile asociate algoritmilor prezentati anterior. Pentru aceasta vom introduce cateva notiuni:

- **branch factor** : **b** = **numarul mediu de ramificari** pe care le are un nod neterminal (care nu e frunza) din **arborele de solutii**
- **depth** : **d** = **adancimea maxima** pana la care se face cautarea in arborele de solutii
    - orice nod de adancime d va fi considerat terminal

Un arbore cu un branching factor **b**, care va fi examinat pana la un nivel **d** va furniza b^d noduri frunze ce vor trebui procesate (ex. calculam scorul pentru acele noduri).

Nivelurile sunt notate cu $0, 1, 2, \ldots, d$

- nivel 0: 1 nod (radacina)
- nivel 1: b noduri
- nivel 2: b^2 noduri
- nivel 3: b^3 noduri
- …
- nivel d: b^d noduri

- **minimax/negamax**
    - Un algoritm **mini/negamax** clasic, care analizeaza toate starile posibile, va avea complexitatea $O(b^d)$ - deci exponentiala.

- **alpha-beta**
    - Cat de bun este insa alpha-beta fata de un mini/nega-max naiv? Dupa cum s-a mentionat anterior, in functie de ordonarea miscarilor ce vor fi evaluate putem avea un caz cel mai favorabil si un caz cel mai defavorabil.
    - **best case** : miscarile sunt ordonate descrescator dupa castig (deci ordonate optim), rezulta o complexitate
        - $O(b * 1 * b * 1 * b * 1 ... de\ d\ ori \ldots b * 1)$ pentru d par
        - $O(b * 1 * b * 1 * b * 1 ... de\ d\ ori \ldots b)$ pentru d impar
        - restrangand ambele expresii rezulta o complexitate $O(b^{\frac{d}{2}}) = O(\sqrt{b^d})$
        - prin urmare, intr-un caz ideal, algoritmul alpha-beta poate explora de 2 ori mai putine nivele in arborele de solutii fata de un algoritm mini/nega-max naiv.
    - **worst case**: miscarile sunt ordonate crescator dupa castigul furnizat unui jucator, astfel fiind necesara o examinare a tuturor nodurilor pentru gasirea celei mai bune miscari.
        - in consecinta complexitatea devine egala cu cea a unui algoritm mini/negamax naiv.

Concluzii si observatii

Alpha-beta NU ofera o alta solutie fata de Minimax! Este doar o optimizare pusa deasupra algoritmului Minimax care ne permite sa exploram mai multe stari in acelasi timp sau pentru acelasi numar de stari sa optinem un timp de doua ori mai mic.

Exemple

Dintre cele mai importante jocuri in care putem aplica direct strategia minimax, mentionam:

- X și 0 [https://en.wikipedia.org/wiki/Tic-tac-toe]
    - joc foarte simplu/usor (spatiul starilor este mic).
    - Prin urmare tot arborele de solutii poate fi generat si explorat intr-un timp foarte scurt.

- sah [https://en.wikipedia.org/wiki/Chess]
  - joc foarte greu (spatiul starilor este foarte mare)
  - minimax/negamax simplu poate merge pana la $d = 7$ (nu reusea da bata campionul mondial la sah - campion uman)
  - alpha-beta poate merge pana la $d = 14$
  - **Deep Blue** a fost implementarea unui bot cu minimax si alpha-beta care a batut in 1997 campionul mondial la sah (Gary Kasparov).
- Ultimate tic-tac-toe [https://en.wikipedia.org/wiki/Ultimate_tic-tac-toe]
  - varianta mult mai grea de X si 0 (spatiul starilor foarte mare)
  - s-a dat la proiect PA 2016 :D
  - implemantarile se pot testa aici [http://theaigames.com/competitions/ultimate-tic-tac-toe/rules]
- Nim [https://en.wikipedia.org/wiki/Nim]
- Reversi [https://en.wikipedia.org/wiki/Reversi]

Alte exempe de jocuri interesante:

- Go [https://en.wikipedia.org/wiki/Go_(game)]
  - solutiile se bazeaza pe Monte Carlo Tree Search (nu pe minimax)
  - AlphaGo [https://en.wikipedia.org/wiki/AlphaGo] este botul cel mai bun pe tabla de 19×19

Nim

Fiind date 3 multimi de bile, fiecare jucator trebuie sa extraga la fiecare mutare 1, 2 sau 3 bile din oricare multime.

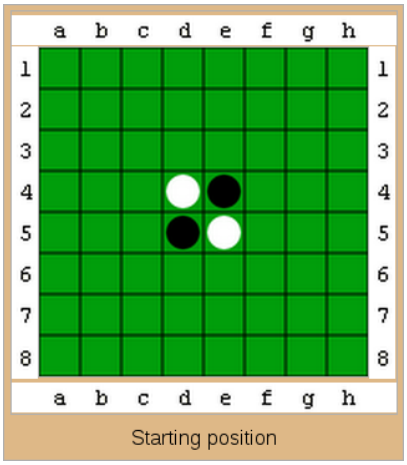
Cel care este fortat sa aleaga ultima bila, pierde.

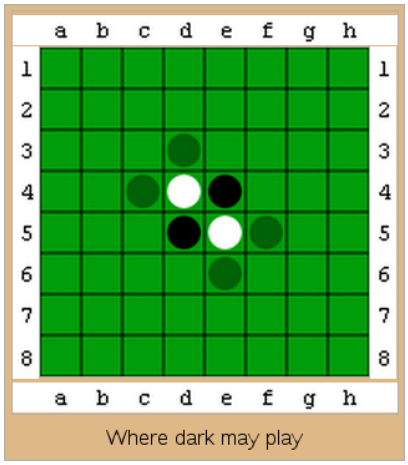
Reversi game

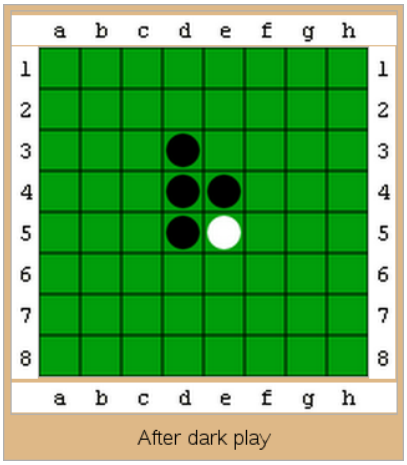
Tabla de joc consista dintr-un grid 6×6. Piesele pot fi reprezentate de monede, fiecarui jucator fiindu-i asociata o fata diferita a monezii. Jucatorii muta alternativ, dupa regula urmatoare:

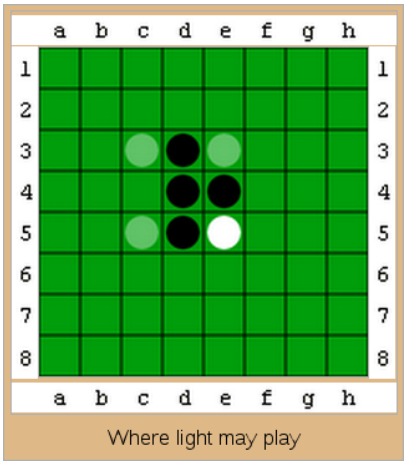
- Pozitia (x, y) in care este plasata piesa trebuie sa fie libera
- Trebuie sa existe o alta pozitie (x', y'), pe aceeasi linie, coloana sau diagonala a jucatorului aflat la mutare si toate pozitiile dintre (x, y) si (x', y') trebuie sa fie ocupate de piese ale adversarului
- Piesele adversarului dintre (x, y) si (x', y') vor fi capturate, intorcandu-se monedele pe cealalta fata.

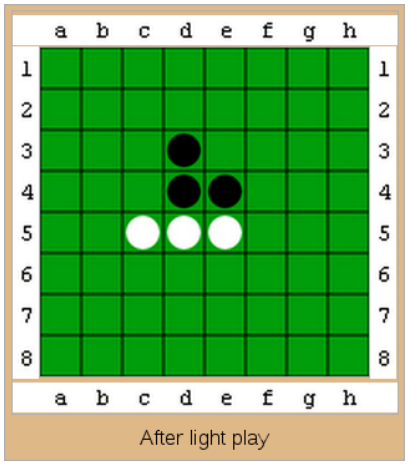
Mai jos, de la stanga spre dreapta: pozitia initiala, posibilitatile de mutare ale primului jucator, tabla dupa prima mutare, posibilitatile de mutare ale celuilalt jucator, tabla dupa a doua mutare.











Observatii:

- Jucatorul poate acapara piese ale adversarului in mai multe directii simultan
- Daca un jucator nu are unde muta, acesta cedeaza randul, adversarul efectuand a doua mutare la rand
- Jocul se incheie cand nimeni nu mai poate muta, invingatorul fiind acela care detine cele mai multe piese proprii

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab06.zip.

Fiecare problemă are surse și Makefile (ex. “cd cpp/p1/; make; make run”).

Vom implementa algoritmi pentru jocurile **Nim** si **Reversi**.

Minimax Nim

Se doreste implementarea algoritmulului minimax sau negamax pentru Nim.

Recomandam implementarea variantei Negamax.

Minimax Reversi

Se doreste implementarea algoritmulului minimax sau negamax pentru Reversi.

Recomandam implementarea variantei Negamax.

Bonus

Extindeti algoritmul implementat anterior pentru jocul **Reversi** intr-un algoritm de tip alpha-beta pruning. Cum puteti sa comparati cei doi algoritmi implementati pentru Reversi?

Referinte

[1] http://en.wikipedia.org/wiki/Minimax [http://en.wikipedia.org/wiki/Minimax]

[2] http://en.wikipedia.org/wiki/Negamax [http://en.wikipedia.org/wiki/Negamax]

[3] http://en.wikipedia.org/wiki/Alpha-beta_pruning [http://en.wikipedia.org/wiki/Alpha-beta_pruning]

[4] http://inst.eecs.berkeley.edu/~cs61b/fa14/ta-materials/apps/ab_tree_practice/ [http://inst.eecs.berkeley.edu/~cs61b/fa14/ta-materials/apps/ab_tree_practice/]

pa/laboratoare/laborator-06.txt · Last modified: 2020/04/22 18:52 by darius.neatu

Laborator 07: Parcurgerea Grafurilor. Sortare Topologica

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- Intelegerea conceptului de graf si a modurilor de parcurgere aferente
- Utilitatea si aplicabilitatea sortarii topologice

Importanţă – aplicaţii practice

Grafurile sunt utile pentru a modela diverse probleme si se regasesc implementati in multiple aplicatii practice:

- Retele de calculatoare (ex: stabilirea unei topologii fara bucle)
- Pagini Web (ex: Google PageRank [1])
- Retele sociale (ex: calcul centralitate [2])
- Harti cu drumuri (ex: drum minim)
- Modelare grafica (ex: prefuse [3], graph-cut [4])

Descrierea problemei și a rezolvărilor

Graful poate fi modelat drept o pereche de multimi G = (V, E). Multimea V contine nodurile grafului (vertices), iar multimea E contine muchiile (edges), fiecare muchie stabilind o relatie de vecinatate intre doua noduri. O mare varietate de probleme se modeleaza folosind grafuri, iar rezolvarea acestora presupune explorarea spatiului. O parcurgere isi propune sa ia in discutie fiecare nod al grafului, exact o singura data, pornind de la un nod ales, numit in continuare nod sursa.

Reprezentarea in memorie a grafurilor se face, de obicei, cu liste de adiacenta sau cu matrice de adiacenta. Se pot folosi insa si alte structuri de date, de exemplu un map de perechi < <sursa,destinatie>,cost> .

Pe parcursul rularii algoritmilor de parcurgere, un nod poate avea 3 culori:

- Alb = nedescoperit
- Gri = a fost descoperit si este in curs de prcesare
- Negru = a fost procesat

Se poate face o analogie cu o pata neagra care se extinde pe un spatiu alb. Nodurile gri se afla pe frontiera petei negre. Algoritmii de parcurgere pot fi caracterizati prin completitudine si optimalitate. Un algoritm de explorare complet va descoperi intotdeauna o solutie, daca problema accepta solutie. Un algoritm de explorare optimal va descoperi solutia optima a problemei din perspectiva numarului de pasi care trebuie efectuati.

Parcurgerea in lățime - BFS

Parcurgerea in latime **(Breadth-first Search - BFS)** este un algoritm de cautare in graf, in care, atunci cand se ajunge intr-un nod oarecare v, nevizitat, se viziteaza toate nodurile nevizitate adiacente lui v, apoi toate varfurile nevizitate adiacente varfurilor adiacente lui v, etc. Atentie! BFS depinde de nodul de start.

Plecand dintr-un nod se va parcurge doar componenta conexa din care acesta face parte. Pentru grafuri cu mai multe componente conexe se vor obtine mai multi arbori de acoperire.

In urma aplicarii algoritmului BFS asupra fiecarei componente conexe a grafului, se obtine un arbore de acoperire (prin eliminarea muchiilor pe care nu le folosim la parcurgere). Pentru a putea reconstitui acest arbore, se pastreaza pentru fiecare nod dat identitatea parintelui sau. In cazul in care nu exista o functie de cost asociata muchiilor, BFS va determina si drumurile minime de la radacina la oricare nod.

Pentru implementarea BFS se foloseste o coada. In momentul adaugarii in coada, un nod trebuie colorat gri (a fost descoperit si urmeaza sa fie prelucrat).

Algoritmul de explorare BFS este complet si optimal.

Algoritm:

```
BFS(s, G) {
    foreach (u ∈ V) {
        p(u) = null; // initializari
        dist(s,u) = inf;
        c(u) = alb;
    }
    dist(s) = 0; // distanta pana la sursa este 0
    c(s) = gri; //incepem prelucrarea nodului, deci culoarea devine gri
    Q = (); //se foloseste o coada cu nodurile de prelucrat
    Q = Q + s; // adaugam sursa in coada
    while (!empty(Q)) { // cat timp mai am noduri de prelucrat
        u = top(Q); // se determina nodul din varful cozii
        foreach v ∈ succs(u) { // pentru toti vecinii
            if (c(v) = alb) {// nodul nu a fost gasit, nu e in coada
                // actualizam structura date
                dist(v) = dist(u) + 1;
                p(v) = u;
                c(v) = gri;
                Q = Q + v;
            } // close if
        } // close foreach
        c(u) = negru; //am terminat de prelucrat nodul curent
        Q = Q - u; //nodul este eliminat din coada
    } //close while
}
```

Complexitate:

- cu lista: O(|E|+|V|)
- cu matrice: O(|V|^2)

Parcurgerea in adancime – DFS

Parcurgerea in adancime **(Depth-First Search - DFS)** porneste de la un nod dat (nod de start), care este marcat ca fiind in curs de procesare. Se alege primul vecin nevizitat al acestui nod, se marcheaza si acesta ca fiind in curs de procesare, apoi si pentru acest vecin se cauta primul vecin nevizitat, si asa mai departe. In momentul in care nodul curent nu mai are vecini nevizitati, se marcheaza ca fiind deja procesat si se revine la nodul anterior. Pentru acest nod se cauta primul vecin nevizitat. Algoritmul se repeta pana cand toate nodurile grafului au fost procesate.

In urma aplicarii algoritmului DFS asupra fiecarei componente conexe a grafului, se obtine pentru fiecare dintre acestea cate un arbore de acoperire (prin eliminarea muchiilor pe care nu le folosim la parcurgere). Pentru a putea reconstitui acest arbore, pastram pentru fiecare nod dat identitatea parintelui sau.

Pentru fiecare nod se vor retine:

- timpul descoperirii
- timpul finalizarii
- parintele

- culoarea

Algoritmul de explorare DFS nu este nici complet (in cazul unei cautari pe un subarbore infinit), nici optimal (nu gaseste nodul cu adancimea minima).

Spre deosebire de BFS, pentru implementarea DFS se foloseste o stiva (abordare **LIFO** in loc de **FIFO**). Desi se poate face aceasta inlocuire in algoritmul de mai sus, de cele mai multe ori este mai intuitiva folosirea recusivitatii.

Algoritm:

```
DFS(G) {
    V = noduri(G)
    foreach (u ∈ V) {
        // initializare structura date
        c(u) = alb;
        p(u)=null;
    }
    timp = 0; // retine distanta de la radacina pana la nodul curent
    foreach (u ∈ V)
        if (c(u) = alb) explorare(u); // explorez nodul
}

explorare(u) {
    d(u) = timp++; // timpul de descoperire al nodului u
    c(u) = gri; // nod in curs de explorare
    foreach (v ∈ succes(u)) // incerc sa prelucrez vecinii
        if (c(v) = alb) { // daca nu au fost prelucrati deja
            p(v) = u;
            explorare(v);
        }
    c(u) = negru; // am terminat de prelucrat nodul curent
    f(u) = timp++; // timpul de finalizare al nodului u
}
```

Complexitate:

- cu lista: O(|E|+|V|)
- cu matrice: O(|V|^2)

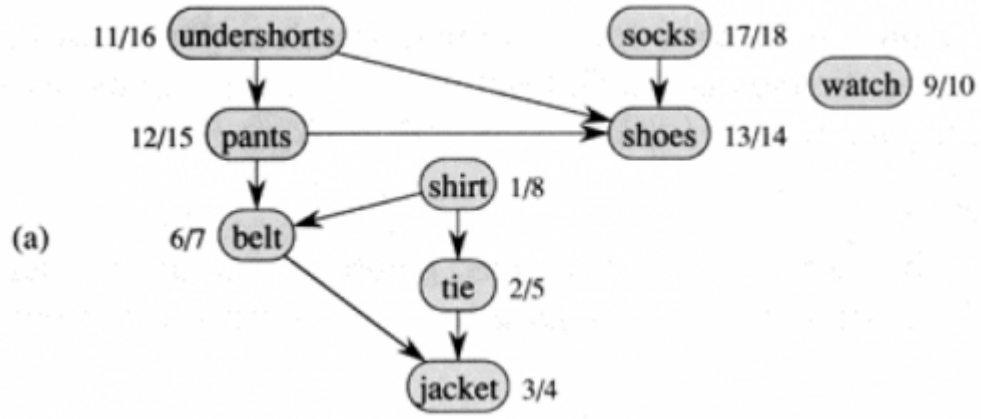
Sortarea Topologica

Dandu-se un graf orientat aciclic, sortarea topologica realizeaza o aranjare liniara a nodurilor in functie de muchiile dintre ele. Orientarea muchiilor corespunde unei relatii de ordine de la nodul sursa catre cel destinatie. Astfel, daca (u,v) este una dintre muchiile grafului, u trebuie sa apara inaintea lui v in insiruire. Daca graful ar fi ciclic, nu ar putea exista o astfel de insiruire (nu se poate stabili o ordine intre nodurile care alcatuiesc un ciclu).

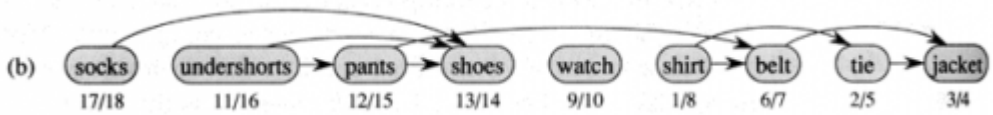
Sortarea topologica poate fi vazuta si ca plasarea nodurilor de-a lungul unei linii orizontale astfel incat toate muchiile sa fie directionate de la stanga la dreapta.

Exemplu:

Profesorul Bumstead isi sorteaza topologic hainele inainte de a se imbraca.



(a) Fiecare muchie (u,v) inseamna ca obiectul de imbracaminte u trebuie imbracat inaintea obiectului de imbracaminte v. Timpii de descoperire d(u) si de finalizare f(u) obtinuti in urma parcurgerii DFS sunt notati langa noduri.



(b) Acelasi graf, sortat topologic. Nodurile lui sunt aranjate de la stanga la dreapta in ordinea **descrescatoare** a **timpului de finalizare** f(u). Observati ca toate muchiile sunt orientate de la stanga la dreapta. Acum profesorul Bumstead se poate imbraca linistit.

Algoritm:

Sunt doi algoritmi cunoscuti pentru sortarea topologina.

Algoritmul bazat pe DFS:

- parcurgere DFS pentru determinarea timpilor
- sortare descrescatoare in functie de timpul de finalizare

Pentru a evita sortarea nodurilor in functie de timpul de finalizare, se poate folosi o stiva ce retine aceste noduri in ordinea terminarii parcurgerii.

Un alt algoritm este cel descris de Kahn:

```
TopSort(G) {
    V = noduri(G)
    L = vida;// lista care va contine elementele sortate
    // initializare S cu nodurile care nu au in-muchii
    foreach (u ∈ V) {
        if (u nu are in-muchii)
            S = S + u;
    }
    while (!empty(S)) { // cat timp mai am noduri de prelucrat
        u = random(S); // se scoate un nod din multimea S
        L = L + u;  // adaug U la lista finala
        foreach v ∈ succs(u) { // pentru toti vecinii
            sterge u-v; // sterge muchia u-v
            if (v nu are in-muchii)
                S = S + v;  // adauga v la multimea S
        } // close foreach
    } //close while
    if (G are muchii)
        print(eroare);  // graf ciclic
    else
        print(L); // ordinea topologica
}
```

Complexitate optima: O(|E|+|V|)

Concluzii si observatii

Grafurile sunt foarte importante pentru reprezentarea si rezolvarea unei multitudini de probleme. Cele mai uzuale moduri de reprezentare a unui graf sunt:

- liste de adiacenta
- matrice de adiacenta

Cele doua moduri uzuale de parcurgere neinformata a unui graf sunt:

- BFS – parcurgere in latime
- DFS – parcurgere in adancime

Sortarea topologica este o modalitate de aranjare a nodurilor in functie de muchiile dintre ele. In functie de nodul de start al DFS, se pot obtine sortari diferite, pastrand insa proprietatile generale ale sortarii topologice.

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab07.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

BFS

Se da un graf **neorientat** cu **n** noduri si **m** muchii. Se mai da un nod special **source**, pe care il vom numi sursa.

Se cere sa se gaseasca **numarul minim de muchii** ce trebuie parcurse de la **source** la **toate** celelalte noduri.

Restrictii si precizari:

- $n, m <= 10^5$
- timp de executie
  - C++: 1s
  - Java: 1s

Rezultatul se va returna sub forma unui vector **d** cu **n** elemente.

Convention:

- **d[node]** = numarul minim de muchii ce trebuie parcurse de la **source** la nodul **node**
- **d[source] = 0**
- **d[node] = -1**, daca nu se poate ajunge de la **source** la **node**

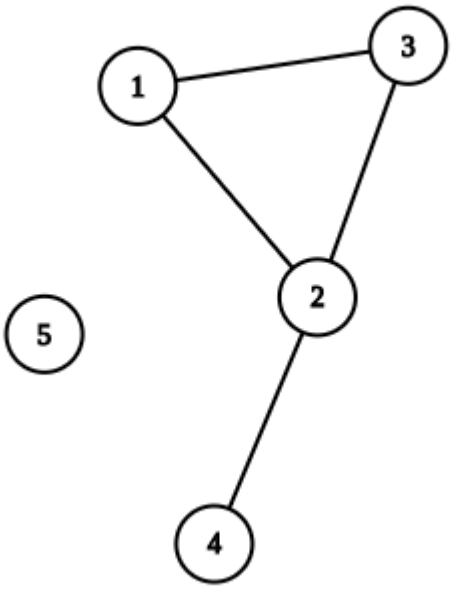
$n = 5\ m = 4\ source = 3$

$muchii : (1, 2); (1, 3); (2, 3); (2, 4);$

Raspuns:

node	1	2	3	4	5
d	1	1	0	2	-1

Explicatie: Graful dat este cel din figura urmatoare.



- **d[3] = 0** pentru ca 1 este sursa
- **d[1] = d[2] = 1** pentru ca exista muchie directa de la 2 la fiecare nod
- **d[4] = 2** pentru ca trebuie sa parcurgem 2 muchii $(3 - 2 - 4)$
- **d[5] = -1** pentru ca nu se poate ajunge de la 3 la 5

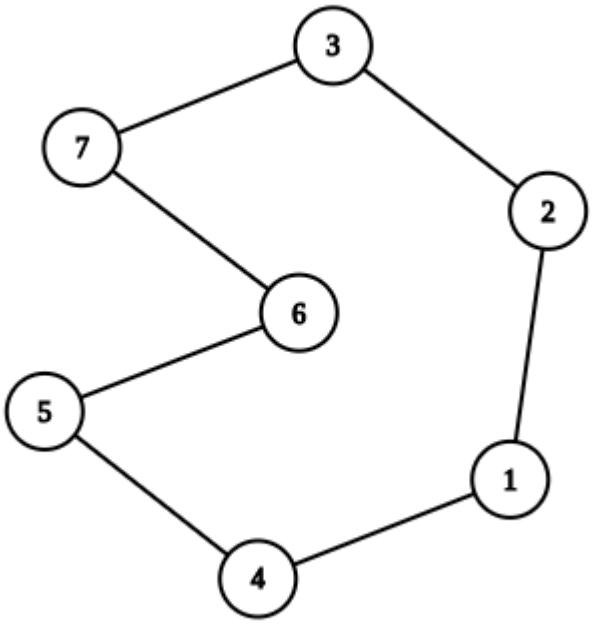
$n = 7\ m = 7\ source = 1$

$muchii : (1, 2); (1, 4); (2, 3); (4, 5); (5, 6); (3, 7); (7, 6)$

Raspuns:

node	1	2	3	4	5	6	7
d	0	1	2	1	2	3	3

Explicatie: Graful dat este cel din figura urmatoare.



- **d[1] = 0** pentru ca 1 este sursa
- **d[2] = d[4] = 1** pentru ca exista muchie directa de la 2 la fiecare nod
- **d[3] = d[5] = 2** pentru ca trebuie sa parcurgem 2 muchii ($1-2-3$, $1-4-5$)
- **d[6] = d[7] = 3** pentru ca trebuie sa parcurgem 3 muchii ($1-2-3-7$ sau $1-4-5-6$)

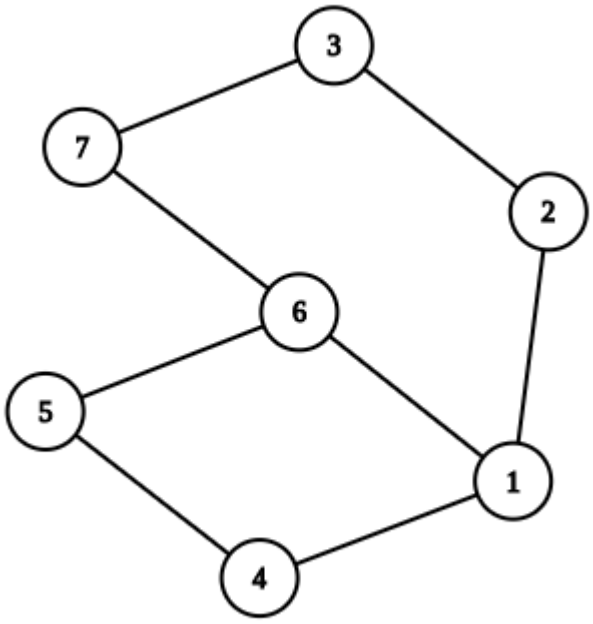
$n = 7$ $m = 8$ $source = 1$

$muchii$: $(1,2); (1,4); (2,3); (4,5); (5,6); (3,7); (7,6); (1,6)$

Raspuns:

node	1	2	3	4	5	6	7
d	0	1	2	1	2	1	2

Explicatie: Graful dat este cel din figura urmatoare.



- **d[1] = 0** pentru ca 1 este sursa
- **d[2] = d[4] = d[6] = 1** pentru ca exista muchie directa de la 2 la fiecare nod
- **d[3] = d[5] = d[7] = 2** pentru ca trebuie sa parcurgem 2 muchii ($1-2-3$, $1-4-5$, $1-6-7$)

TOPOLOGICAL SORT

Se da un graf **orientat** aciclic cu **n** noduri si **m** arce. Se cere sa se gaseaca **o sortare topologica** valida.

Restrictii si precizari:

- $n, m <= 10^5$
- timp de executie
  - C++: 1s
  - Java: 1s

Rezultatul se va returna sub forma unui vector **topsort** cu **n** elemente.

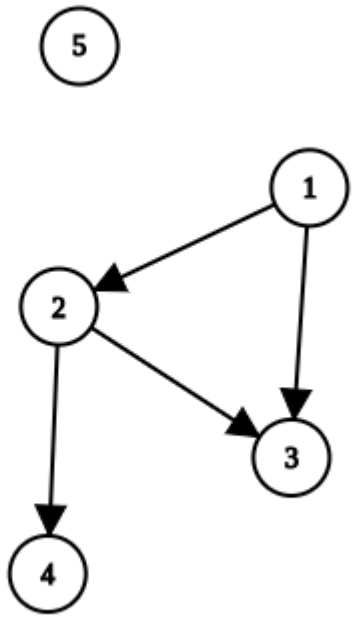
Vectorul **topsort** va reprezenta o permutare a multimii $1, 2, 3, \ldots, n$ reprezentand sortarea topologica gasita.

$n = 5$ $m = 4$

$arce: (1,2); (1,3); (2,3); (2,4);$

Raspuns: $topsort = [1, 2, 3, 4, 5]$

Explicatie: Graful dat este cel din figura urmatoare.



- deoarece avem arcele $1 \rightarrow 3$ si $1 \rightarrow 2$, 1 trebuie sa apara inainte lui 2 si 3
- deoarece avem arcul $2 \rightarrow 3$ si $2 \rightarrow 4$, 2 trebuie sa apara inainte lui 3 si 4
- 5 nu depinde de nimeni, poate sa apara oriunde

Toate sortarile topologice valide sunt:

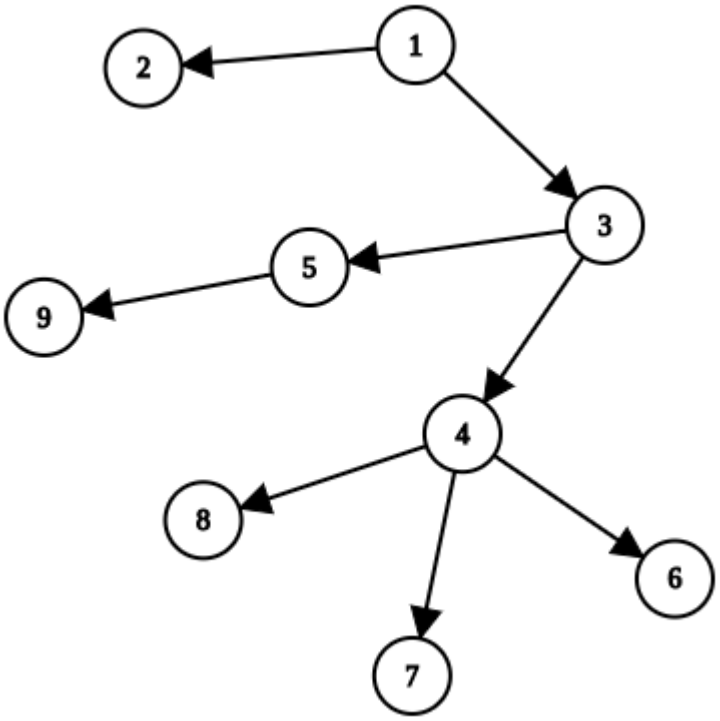
- cele date de ordinea relativa a primelor 4 noduri: 1,2,3,4)
  - $topsort = [1, 2, 3, 4, 5]$
  - $topsort = [1, 2, 3, 5, 4]$
  - $topsort = [1, 2, 5, 3, 4]$
  - $topsort = [1, 5, 2, 3, 4]$
  - $topsort = [5, 1, 2, 3, 4]$
- cele date de ordinea relativa a primelor 4 noduri: (1,2,4,3)
  - $topsort = [1, 2, 4, 3, 5]$
  - $topsort = [1, 2, 4, 5, 3]$
  - $topsort = [1, 2, 5, 4, 3]$
  - $topsort = [1, 5, 2, 4, 3]$
  - $topsort = [5, 1, 2, 4, 3]$

$n = 9$ $m = 8$

$arce : (1, 2); (1, 3); (3, 4); (3, 5); (5, 9); (4, 6); (4, 7); (4, 8);$

Raspuns: $topsort = [1, 2, 3, 4, 6, 7, 8, 5, 9]$

Explicatie: Graful dat este cel din figura urmatoare.



Se observa din desen ca solutia mentionata este valida.

BONUS

Se da un tablou bidimensional de dimensiune **n x n**. Fiecare celula este o camera.

O valoare din matrice poate sa fie **0**, **1** sau **2**.

- 0 = celula libera
- 1 = celula blocata (contine un obstacol - nu se poate intra intr-o astfel de camera)
- 2 = celula contine o poarta de teleportare (ex. iesire catre exterior - cine stie ce o fi afara)

Un student de la Poli se afla intr-un punct interior dat prin coodonatele **start_row** si **start_col**. Deoarece studentul este dornit sa termine facultatea cat mai repede, se intreaba care este **lungimea celui mai scurt drum** de la pozitia sa catre o poarta de teleportare.

Studentul se poate deplasa doar pe verticala sau pe orizontala.

Task-uri:

- gasiti **lungimea** ceruta
- reconstituiti **drumul** de lungime minima (daca sunt mai multe, se poate afisa oricare)

Restrictii si precizari:

- $n <= 10^3$
- timp de executie
  - C++: 1s
  - Java: 1s

$n = 5$

1	0	2	1	0

0	0	0	1	1
2	1	0	0	1
1	1	0	1	0
0	1	2	1	0

Raspuns: $lungime = 3$

$drum = (3,4) \rightarrow (3,3) \rightarrow (2,3) \rightarrow (1,3)$

Explicatie: Exista doua posibilitati de a porni din punctul de coordonate $(3,4)$ si a ajunge printr-un drum de lungime minima la o poarta. Unul dintre acestea urmeaza traseul mentionat.

Extra

Rezolvati problema muzeu [https://infoarena.ro/problema/muzeu] pe infoarena.

Rezolvati problema arbore3 [https://infoarena.ro/problema/arbore3] pe infoarena.

Rezolvati problema Pokemon GO AWAY [https://www.hackerrank.com/contests/test-practic-pa-2017-v2-meeseeks/challenges/test-2-pokemon-go-away-grea] de la test PA 2017.

Cu ce problema seamana?

Rezolvati problema insule [https://infoarena.ro/problema/insule] pe infoarena.

Rezolvati problema tsunami [https://infoarena.ro/problema/tsunami] pe infoarena.

Rezolvati problema berarii2 [https://infoarena.ro/problema/berarii2] pe infoarena.

Referinte

[1] http://en.wikipedia.org/wiki/PageRank [http://en.wikipedia.org/wiki/PageRank]

[2] http://en.wikipedia.org/wiki/Social_network#Social_network_analysis [http://en.wikipedia.org/wiki/Social_network#Social_network_analysis]

[3] http://prefuse.org/ [http://prefuse.org/]

[4] http://classes.engr.oregonstate.edu/eecs/spring2008/cs419/Lectures/jun_graphcut.pdf [http://classes.engr.oregonstate.edu/eecs/spring2008/cs419/Lectures/jun_graphcut.pdf]

[5] http://en.wikipedia.org/wiki/Breadth-first_search [http://en.wikipedia.org/wiki/Breadth-first_search]

[6] http://en.wikipedia.org/wiki/Depth-first_search [http://en.wikipedia.org/wiki/Depth-first_search]

[7] http://en.wikipedia.org/wiki/Topological_sorting [http://en.wikipedia.org/wiki/Topological_sorting]

[8] Introducere in Algoritmi, T. Cormen s.a., pag 403-419

[9] http://ww3.algorithmdesign.net/handouts/DFS.pdf [http://ww3.algorithmdesign.net/handouts/DFS.pdf]

[10] http://ww3.algorithmdesign.net/handouts/BFS.pdf [http://ww3.algorithmdesign.net/handouts/BFS.pdf]

Laborator 08: Aplicatii DFS

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

Întelegerea noţiunilor teoretice:

- tare conexitate, componente tare conexe (pentru grafuri orientate)
- punct de articulaţie (pentru grafuri neorientate)
- punţi (pentru grafuri neorientate)
- componente biconexe (în general, pentru grafuri neorientate)

Înţelegerea algoritmilor ce rezolvă aceste probleme şi implementarea acestor algoritmi.

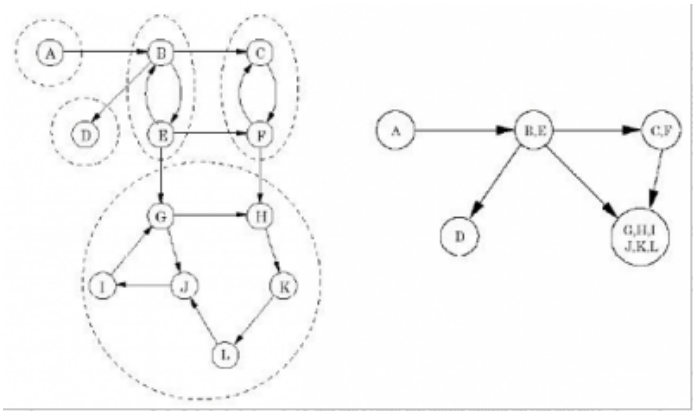
Importanţă – aplicaţii practice

- Componentele biconexe au aplicaţii importante în reţelistică, deoarece o componentă biconexă asigură redundanţa.
- Descompunerea în componente tare conexe: data mining, compilatoare, calcul ştiinţific, 2SAT

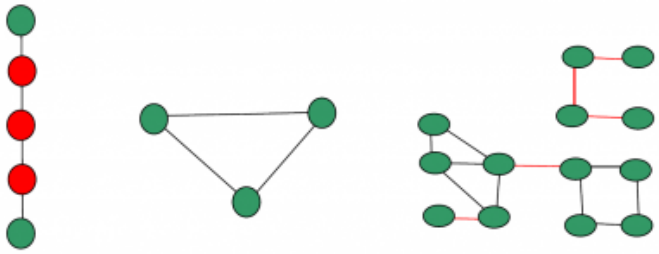
Notiuni teoretice

- **Tare conexitate.** Un graf orientat este tare conex, dacă oricare ar fi două vârfuri u şi v, ele sunt tare conectate (strongly connected) - există drum atât de la u la v, cât şi de la v la u.
- O componentă tare conexă este un subgraf maximal tare conex al unui graf orientat, adică o submulţime de vârfuri U din V, astfel încât pentru orice u şi v din U ele sunt tare conectate. Dacă fiecare componentă tare conexă este redusă într-un singur nod, se va obţine **un graf orientat aciclic**.

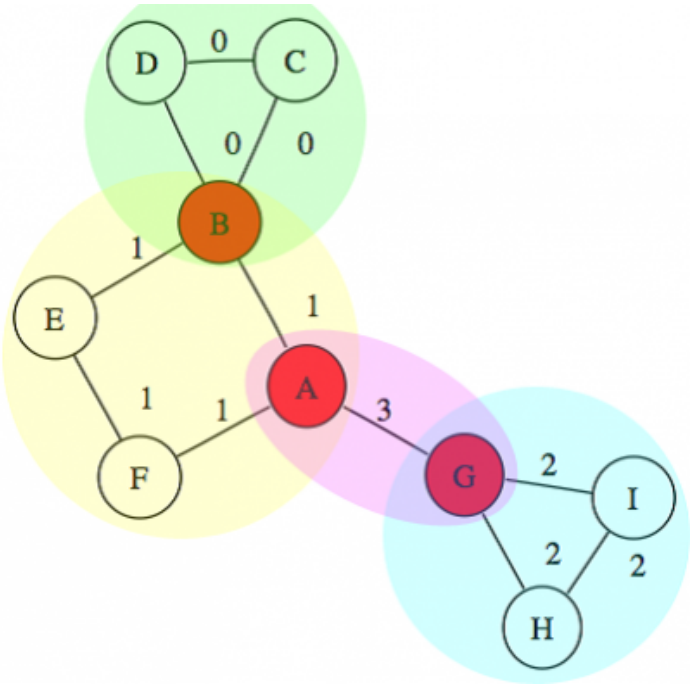
De exemplu:



- **Un punct de articulaţie (cut vertex)** este un nod al unui graf a cărui eliminare duce la creşterea numărului de componente conexe ale acelui graf.
- **O punte (bridge)** este o muchie a unui graf (se mai numeşte şi **muchie critică**) a cărei eliminare duce la creşterea numărului de componente conexe ale acelui graf.



- **Biconexitate**. Un graf biconex este un graf conex cu proprietatea că eliminând oricare nod al acestuia, graful rămâne conex.
- O componentă biconexă a unui graf este o mulţime **maximală** de noduri care respectă proprietatea de biconexitate.



Componente tare conexe

Vom porni de la definiţie pentru a afla componenta tare conexă din care face parte un nod v. Vom parcurge graful (DFS sau BFS) pentru a găsi o mulţime de noduri S ce sunt accesibile din v. Vom parcurge apoi graful transpus (obţinut prin inversarea arcelor din graful iniţial), determinând o nouă mulţime de noduri T ce sunt accesibile din v în graful transpus. Intersecţia dintre S şi T va reprezenta componenta tare conexa. Graful iniţial şi cel transpus au aceleaşi componente conexe.

Algoritmul lui Kosaraju

Algoritmul foloseşte două DFS (una pe graful iniţial şi una pe graful transpus) şi o stivă pentru a reţine ordinea terminării parcurgerii nodurilor grafului original (evitând astfel o sortare a nodurilor după acest timp la terminarea parcurgerii).

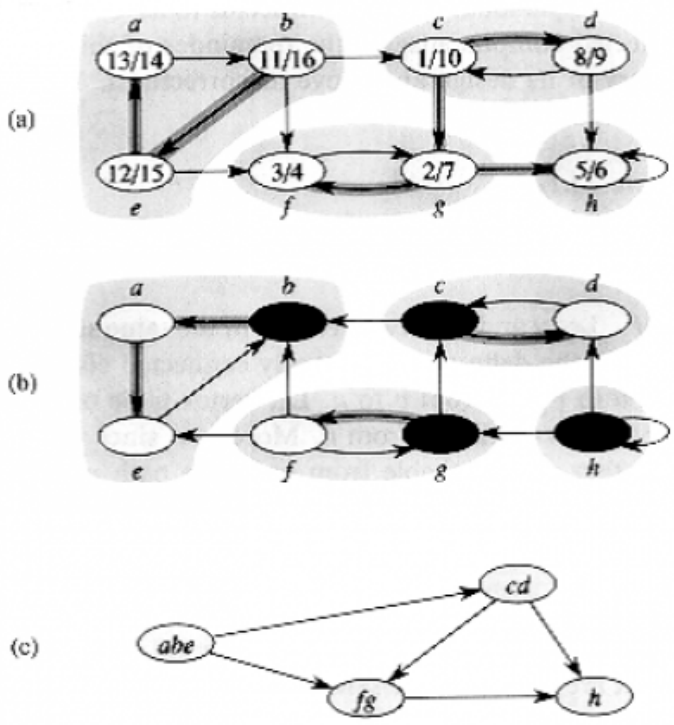
```
ctc(G = (V, E))
        S <- stiva vida
        culoare[1..n] = alb
        cat timp exista un nod v din V care nu e pe stiva
                dfs(G, v)
        culoare[1..n] = alb
        cat timp S != stiva vida
                v = pop(S)
                dfsT(GT, v) /* toate nodurile ce pot fi vizitate din v fac  parte din ctc; dupa vizitare, aceastea sunt scoase din S si din G */

dfs(G, v)
        culoare[v] = gri
        pentru fiecare (v, u) din E
                daca culoare[u] == alb
                        dfs(u)
        push(S, v) // nodul este terminat de expandat, este pus pe stiva
```

```
        culoare[v] = negru

dfsT(G, v) – similar cu dfs(G, v): fara stiva, dar cu retinerea solutiei
```

Complexitate: O(|V| + |E|)



Algoritmul lui Tarjan

Algoritmul foloseşte o singură parcurgere DFS şi o stivă. Ideea de bază a algoritmului este că o parcurgere în adâncime porneşte dintr-un nod de start. Componentele tare conexe formează subarborii arborelui de căutare, rădăcinile cărora sunt de asemenea rădăcini pentru componentele tare conexe.

Nodurile sunt puse pe o stivă, în ordinea vizitării. Când parcurgerea termină de vizitat un subarbore, nodurile sunt scoase din stivă şi se determină pentru fiecare nod dacă este rădăcina unei component tare conexe. Dacă un nod este rădăcina unei componente, atunci el şi toate nodurile scoase din stivă înaintea lui formează acea componenta tare conexă.

Pentru a determina daca un nod este radacina unei componente conexe, calculam pentru fiecare nod:

```
idx[v]     -> nivelul / ordinea de vizitare
lowlink[v] -> min { idx[u] | u este accesibil din v, folosind doar noduri din subarborele DFS al lui v (inclusiv v) }
```

v este rădăcina unei componente tare conexe ⇔ lowlink[v] = idx[v].

```
ctc_tarjan(G = (V, E))
        index = 0
        S = stiva vida
        pentru fiecare v din V
                daca (idx[v] nu e definit) // nu a fost vizitat
                        tarjan(G, v)

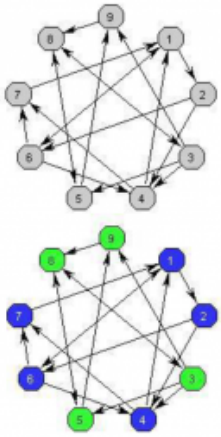
tarjan(G, v)
        idx[v] = index
        lowlink[v] = index
        index = index + 1
        push(S, v)

        pentru (v, u) din E
                daca (idx[u] nu e definit)
                        tarjan(G, u)
                        lowlink[v] = min(lowlink[v], lowlink[u])
                altfel
                        daca (u e in S)
                                lowlink[v] = min(lowlink[v], idx[u])

        daca (lowlink[v] == idx[v])
                // este v radacina unei CTC?
                print "O noua CTC: "
```

```
                repeat
                        u = pop(S)
                        print u
                until (u == v)
```

Complexitate: O(|V| + |E|)



	id	pre	low
1	1	0	10
2	1	1	10
3	2	5	10
4	1	2	10
5	2	6	10
6	1	4	10
7	1	3	10
8	2	7	10
9	2	8	10

Puncte de articulatie

Pentru determinarea punctelor de articulaţie într-un graf neorientat se foloseşte o parcurgere în adâncime modificată, reţinându-se informaţii suplimentare pentru fiecare nod. Acest algoritm a fost identificat tot de către Tarjan si este foarte similar cu algoritmul pentru determinarea CTC în grafuri orientate prezentat anterior. Fie T un arbore de adâncime descoperit de parcurgerea grafului. Atunci, un nod v este punct de articulaţie dacă:

- v este rădăcina lui T şi v are doi sau mai mulţi copii

sau

- v nu este rădăcina lui T şi are un copil u în T, astfel încât nici un nod din subarborele dominat de u nu este conectat cu un strămoş al lui v printr-o muchie înapoi (copii lui nu pot ajunge pe altă cale pe un nivel superior în arborele de adâncime).

Găsirea punctelor care se încadrează în primul caz este uşor de realizat.

Notăm:

```
idx[u] = timpul de descoperire a nodului u
low[u] = min( {idx[u]} U { idx[v] : (u, v) este o muchie înapoi } U
        { low[vi] : vi copil al lui u în arborele de adâncime} )
```

v este punct de articulaţie ⇔ low[u] ≥ idx[v], pentru cel puțin un copil u al lui v în T.

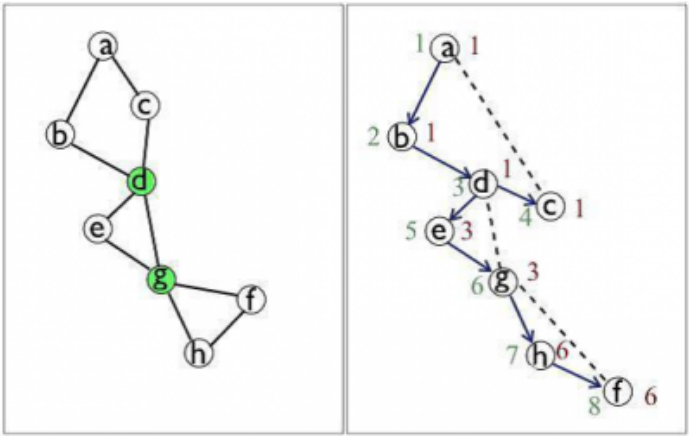
```
puncte_articulatie(G = (V, E))
        timp = 0
        pentru fiecare v din V
        daca (idx[v] nu e definit)
                dfsCV(G, v)

dfsCV(G, v)
        idx[v] = timp
        low[v] = timp
        timp = timp + 1
        copii = { } // multime vida
        pentru fiecare (v, u) din E
                daca u nu este parintele lui v
                        daca (idx[u] nu e definit)
                                // inseamna ca nodul u este nedescoperit, deci alb
                                copii = copii U {u}
                                dfsCV(G, u)
                                low[v] = min(low[v], low[u])
                        altfel
                                // inseamna ca nodul u este descoperit, deci gri, iar muchia v->u este muchie inapoi
                                low[v] = min(low[v], idx[u])

        daca v radacina arborelui
                daca |copii| >= 2
                        v este punct de articulatie
```

```
        altfel
                daca (∃u ∈ copii) astfel incat (low[u] >= idx[v])
                        v este punct de articulatie
```

Complexitate: O(|V| + |E|)



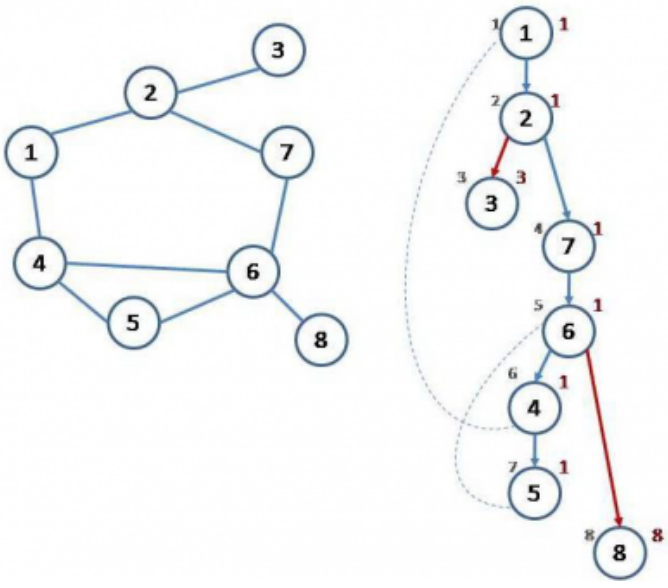
Punti

Pentru a determina muchiile critice se foloseşte tot o parcurgere în adâncime modificată, pornind de la următoarea observaţie: **muchiile critice sunt muchiile care nu apar în niciun ciclu.** Prin urmare, o muchie de întoarcere nu poate fi critică, deoarece o astfel de muchie închide întotdeauna un ciclu. Trebuie să verificăm pentru muchiile de avansare (în număr de |V| - 1) dacă fac parte dintr-un ciclu. Să considerăm că dorim să verificăm muchia de avansare (v, u).

Ne vom folosi de low[v] (definit la punctul anterior): dacă din nodul u putem să ajungem pe un nivel mai mic sau egal cu nivelul lui v, atunci muchia nu este critică, în caz contrar ea este critică.

```
dfsB(G, v, parinte)
        idx[v] = timp
        low[v] = timp
        timp = timp + 1
        pentru fiecare (v, u) din E
                daca u nu este parintele lui v
                        daca idx[u] nu este definit
                                dfsB(G, u, v)
                                low[v] = min(low[v], low[u])
                                daca (low[u] > idx[v])
                                        (v, u) este muchie critica
                        altfel
                                low[v] = min(low[v], idx[u])
```

Complexitate: O(|V| + |E|)



Componente biconexe

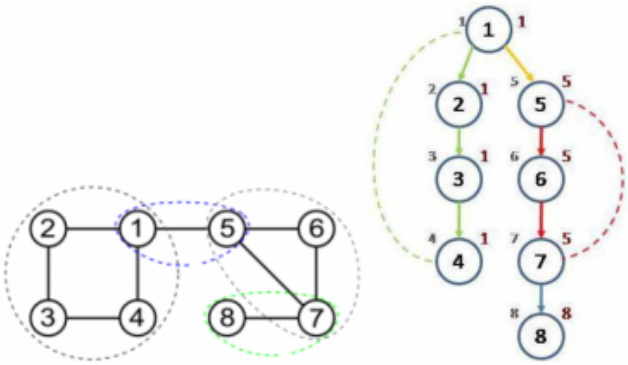
O componenta biconexă (sau biconectată) este o componentă a grafului care nu conţine puncte de articulatie. Astfel după eliminarea oricărui vârf din componenta curentă, restul vârfurilor vor rămâne conectate întrucât între oricare două vărfuri din aceeași componentă biconexă există cel puțin două căi disjuncte.

Astfel, pentru a determina componentele biconexe ale unui graf, vom adapta algoritmul de aflare a punctelor critice, reţinând şi o stivă cu toate muchiile de avansare şi de întoarcere parcurse până la un moment dat. La întâlnirea unui nod critic v se formează o nouă componentă biconexă pe care o vom determina extrăgând din stivă muchiile corespunzătoare. Nodul **v** este critic dacă am găsit un copil **u** din care nu se poate ajunge pe un nivel mai mic în arborele de adâncime pe un alt drum care foloseşte muchii de întoarcere **(low[u] >= idx[v])**. Atunci când găsim un astfel de nod **u**, toate muchiile aflate în stivă până la muchia (v, u) inclusiv formează o nouă componentă biconexă.

Nodul rădăcină trebuie tratat separat. Conform regulilor acestuia pentru a fi marcat nod critic (numărul de copii >= 2), este suficient să considerăm că fiecare copil face parte dintr-o componentă biconexă separată.

Atenție! Împărţirea în componente biconexe a unui graf neorientat reprezintă o partiție disjunctă a muchiilor grafului (împreună cu vârfurile adiacente muchiilor corespunzătoare fiecărei componente în parte). Acest lucru implică că unele vârfuri pot face parte din mai multe componente biconexe diferite. Care sunt acestea?

Complexitate: O(|V| + |E|)



Concluzii

Algoritmul de parcurgere în adâncime poate fi modificat pentru calculul componentelor tare conexe, a punctelor de articulaţie, a punţilor şi a componentelor biconexe. Complexitatea acestor algoritmi va fi cea a parcurgerii: O(|V| + |E|).

Referinte

[1] Introducere in Algoritmi, Thomas H. Cormen, Charles E. Leiserson, Ronald L. – Capitolul 23 Algoritmi elementari pe grafuri http://net.pku.edu.cn/~course/cs101/resource/Intro2Algorithm/book6/chap23.htm
[http://net.pku.edu.cn/~course/cs101/resource/Intro2Algorithm/book6/chap23.htm]

[2] Wikipedia – Algoritmul lui Tarjan http://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm
[http://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm]

[3] Wikipedia – Algoritmul lui Kosaraju http://en.wikipedia.org/wiki/Kosaraju%27s_algorithm
[http://en.wikipedia.org/wiki/Kosaraju%27s_algorithm]

[4] Wikipedia - Componente biconexe http://en.wikipedia.org/wiki/Biconnected_component
[http://en.wikipedia.org/wiki/Biconnected_component]

[5] Infoarena - Arhiva educationala - Componente tari conexe http://infoarena.ro/problema/ctc
[http://infoarena.ro/problema/ctc]

[6] Infoarena - Arhiva educationala - Componente biconexe http://infoarena.ro/problema/biconex
[http://infoarena.ro/problema/biconex]

[7] Infoarena - Arhiva educationala - 2SAT http://infoarena.ro/problema/2sat [http://infoarena.ro/problema/2sat]

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab08.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

CTC

Se da un graf **orientat** cu **n** noduri si **m** arce. Sa se gaseasca **componentele tare-conexe**.

Restrictii si precizari:

- $n <= 10^5$
- $m <= 2 * 10^5$
- timp de executie
  - C++: 1s
  - Java: 2s

Rezultatul se va returna sub forma unui vector, unde fiecare element este un vector (o CTC).

CUT VERTEX

Se da un graf **neorientat conex** cu **n** noduri si **m** muchii. Se cere sa se gaseaca toate **punctele critice**.

Restrictii si precizari:

- $n <= 10^5$
- $m <= 2 * 10^5$
- timp de executie
  - C++: 1s
  - Java: 2s

Rezultatul se va returna sub forma unui vector cu **X** elemente, unde X este numarul de puncte critice din graf.

CRITICAL EDGE

Se da un graf **neorientat conex** cu **n** noduri si **m** muchii. Se cere sa se gaseaca toate **muchiile critice**.

Restrictii si precizari:

- $n <= 10^5$
- $m <= 2 * 10^5$
- timp de executie
  - C++: 1s
  - Java: 2s

Rezultatul se va returna sub forma unui vector cu **X** elemente, unde X este numarul de muchii critice.

BONUS

Se da un graf **neorientat conex** cu **n** noduri si **m** muchii. Se cere sa se gaseaca toate **componentele biconexe**.

Pentru testare **trebuie** sa folositi problema biconex [https://infoarena.ro/problema/biconex] de pe infoarena.

Extra

Rezolvati problema retele [https://infoarena.ro/problema/retele] pe infoarena.
Rezolvati **ACASA** problema clepsidra [https://infoarena.ro/problema/clepsidra] pe infoarena.

Rezolvati **ACASA** problema course-schedule [https://leetcode.com/problems/course-schedule/description/] pe leetcode. (aplicatie tipuri de muchii)

pa/laboratoare/laborator-08.txt · Last modified: 2020/04/22 18:53 by darius.neatu

Laborator 09: Drumuri minime

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- Înțelegerea conceptelor de cost, relaxare a unei muchii, drum minim
- Prezentarea si asimilarea algoritmilor pentru calculul drumurilor minime

Importanţă – aplicaţii practice

Algoritmii pentru determinarea drumurilor minime au multiple aplicații practice si reprezintă clasa de algoritmi pe grafuri cel mai des utilizata:

- Rutare in cadrul unei rețele (telefonice, de calculatoare etc.)
- Găsirea drumului minim dintre doua locații (Google Maps, GPS etc.)
- Stabilirea unei agende de zbor in vederea asigurării unor conexiuni optime
- Asignarea unui peer / server de fișiere in funcție de metricile definite pe fiecare linie de comunicație

Concepte

Costul unei muchii si al unui drum

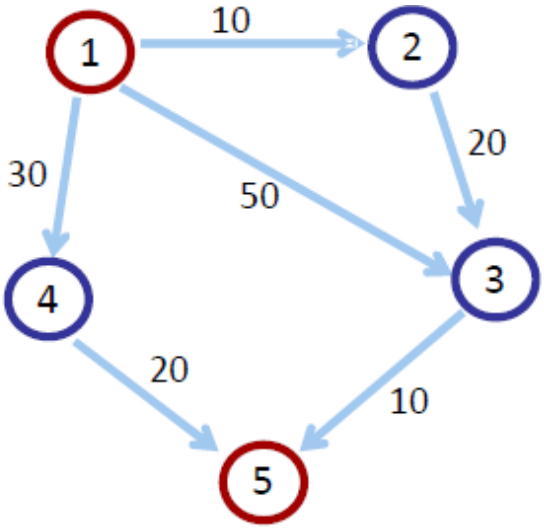
Fiind dat un graf orientat G = (V, E), se considera funcția w: E → W, numita funcție de cost, care asociază fiecărei muchii o valoare numerica. Domeniul funcției poate fi extins, pentru a include si perechile de noduri intre care nu exista muchie directa, caz in care valoarea este +∞ . Costul unui drum format din muchiile p12 p23 … p(n-1)n, având costurile w12, w23, …, w(n-1)n, este suma w = w12 + w23 + … + w(n-1)n.

In exemplul alăturat, costul drumului de la nodul 1 la 5 este:

drumul 1: w14 + w45 = 30 + 20 = 50

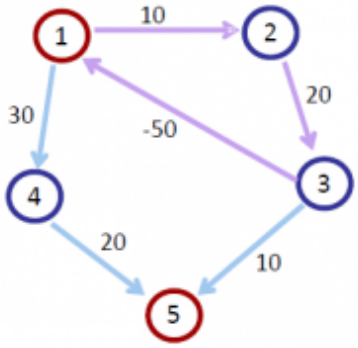
drumul 2: w12 + w23 + w35 = 10 + 20 + 10 = 40

drumul 3: w13 + w35 = 50 + 10 = 60



Drumul de cost minim

Costul minim al drumului dintre doua noduri este minimul dintre costurile drumurilor existente intre cele doua noduri. In exemplul de mai sus, drumul de cost minim de la nodul 1 la 5 este prin nodurile 2 si 3. Deși, in cele mai multe cazuri, costul este o funcție cu valori nenegative, exista situații in care un graf cu muchii de cost negativ are relevanta practica. O parte din algoritmi pot determina drumul corect de cost minim inclusiv pe astfel de grafuri. Totuși, nu are sens căutarea drumului minim in cazurile in care graful conține cicluri de cost negativ – un drum minim ar avea lungimea infinita, intrucat costul sau s-ar reduce la fiecare reparcurgere a ciclului:



In exemplul alăturat, ciclul 1 → 2 → 3 → 1 are costul -20.

drumul 1: w12 + w23 + w35 = 10 + 20 + 10 = 40

drumul 2: (w12 + w23 + w31) + w12 + w23 + w35 = -20 + 10 + 20 + 10 = 20

drumul 3: (w12 + w23 + w31) + (w12 + w23 + w31) + w12 + w23 + w35 = -20 + (-20) + 10 + 20 + 10 = 0

Relaxarea unei muchii

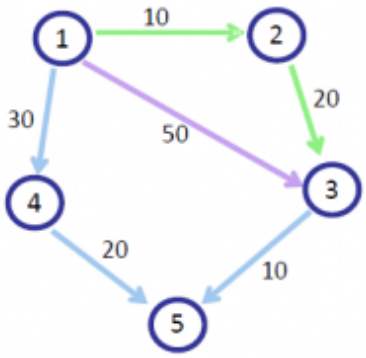
Relaxarea unei muchii v1 - v2 consta in a testa daca se poate **reduce distanta** de la **sursă** până la v2, trecând prin nodul intermediar v1 si apoi circuland pe muchia (v1,v2).

Fie w12 costul inițial al **muchiei directe** de la v1 la v2, w1u costul **drumului** de la v1 la u, si wu2 costul **drumului** de la u la v2. Daca w12 > w1u + wu2, **muchia directa** este înlocuita cu succesiunea de muchii v1 - u, u - v2. Dacă muchia nu există, se consideră **w12 = infinit**.

În acest exemplu, **muchia relaxată** este **u - v2** , întrucât aceasta reduce distanta pentru drumul de la **sursa** la v2, trecând prin v1.

In exemplul alăturat, muchia de la 1 la 3, de cost w13 = 50, poate fi relaxata la costul 30, prin nodul intermediar u = 2, fiind înlocuita cu succesiunea w12, w23.

Toți algoritmii prezentați in continuare se bazează pe relaxare pentru a determina drumul minim.



Drumuri minime de sursa unica

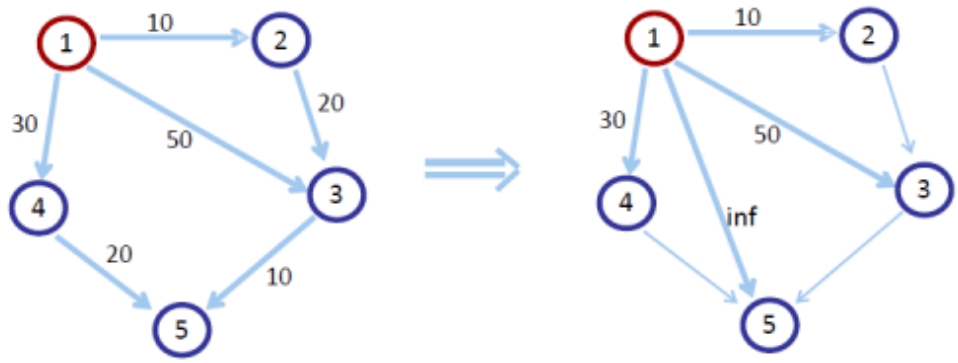
Algoritmii din aceasta secțiune determina drumul de cost minim de la un nod sursa, la restul nodurilor din graf, pe baza de relaxări repetate.

Algoritmul lui Dijkstra

Dijkstra poate fi folosit doar in grafuri care au toate muchiile nenegative.

Algoritmul este de tip Greedy:
optimul local căutat este reprezentat de costul drumului dintre nodul sursa s si un nod v. Pentru fiecare nod se retine un cost estimat d[v], inițializat la început cu costul muchiei s → v, sau cu +∞, daca nu exista muchie.

In exemplul următor, sursa s este nodul 1. Inițializarea va fi:



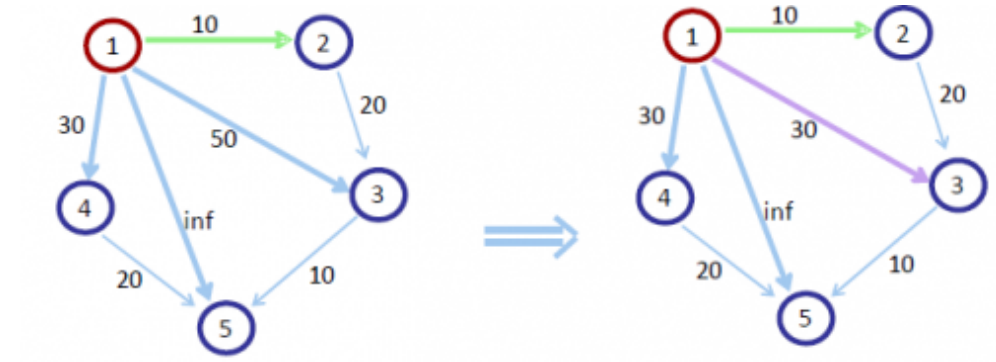
Aceste drumuri sunt îmbunătățite la fiecare pas, pe baza celorlalte costuri estimate.

Algoritmul selectează, in mod repetat, nodul u care are, la momentul respectiv, costul estimat minim (fata de nodul sursa). In continuare, se încearcă sa se relaxeze restul costurilor d[v]. Daca d[v] >= d[u] + wuv , d[v] ia valoarea d[u] + wuv.

Pentru a tine evidenta muchiilor care trebuie relaxate, se folosesc doua structuri: S (mulțimea de vârfuri deja vizitate) si Q (o coada cu priorități, in care nodurile se afla ordonate după distanta fata de sursa) din care este mereu extras nodul aflat la distanta minima. In S se afla inițial doar sursa, iar in Q doar nodurile spre care exista muchie directa de la sursa, deci care au d[nod] < +∞.

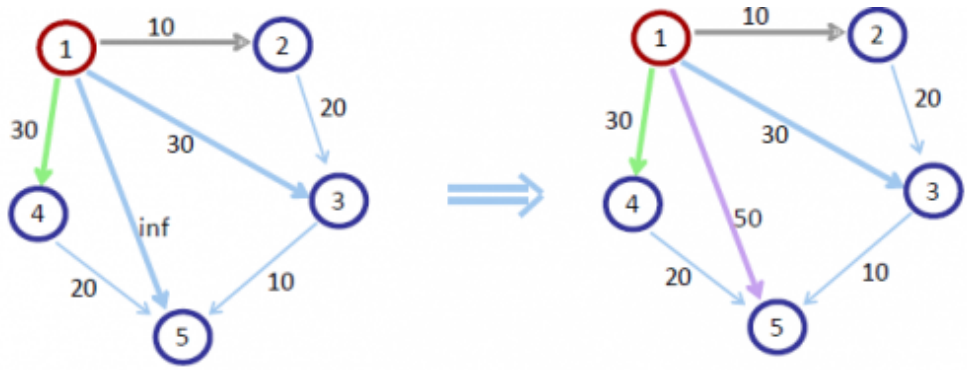
In exemplul de mai sus, vom inițializa S = {1} si Q = {2, 4, 3}.

La primul pas este selectat nodul 2, care are d[2] = 10.
Singurul nod pentru care d[nod] poate fi relaxat este 3 : d[3] = 50 > d[2] + w23 = 10 + 20 = 30



După primul pas, S = {1, 2} si Q = {4, 3}.

La următorul pas este selectat nodul 4, care are d[4] = 30.
Pe baza lui, se poate modifica d[5] : d[5] = +∞ > d[4] + w45 = 30 + 20 = 50



După al doilea pas, S = {1, 2, 4} si Q = {3, 5}.

La următorul pas este selectat nodul 3, care are d[3] = 30, si se modifica din nou d[5]: d[5] = 50 > d[3] + w35 = 30 + 10 = 40.

Algoritmul se încheie când coada Q devine vida, sau când S conține toate nodurile. Pentru a putea determina si muchiile din care este alcătuit drumul minim căutat, nu doar costul sau final, este necesar sa reținem un vector de părinți P. Pentru nodurile care au muchie directa de la sursa, P[nod] este inițializat cu sursa, pentru restul cu null.

Pseudocodul pentru determinarea drumului minim de la o sursa către celelalte noduri utilizând algoritmul lui Dijkstra este:

```
Dijkstra(sursa, dest):
introdu sursa in Q
d[sursa] = 0
d[nod] = +∞ // pentru orice nod != sursa
P[nod] = null // pentru orice nod din V

// relaxari succesive
cat timp Q nu e vida
    u = extrage_min (Q)
    selectat(u) = true
    foreach nod in vecini[u] // (*)
        /* daca drumul de la sursa la nod prin u este mai mic decat cel curent */
        daca not selectat(nod) si d[nod] > d[u] + w[u, nod]
            // actualizeaza distanta si parinte
            d[nod] = d[u] + w[u, nod]
            P[nod] = u
            /* actualizeaza pozitia nodului in coada prioritara */
            actualizeaza (Q,nod)

// gasirea drumului efectiv
Initializeaza Drum = {}
```

```
nod = P[dest]
cat timp nod != null
    insereaza nod la inceputul lui Drum
    nod = P[nod]
```

Reprezentarea grafului ca matrice de adiacenta duce la o implementare ineficienta pentru orice graf care nu este complet, datorita parcurgerii vecinilor nodului u, din linia (*), care se va executa în |V| pași pentru fiecare extragere din Q, iar pe întreg algoritmul vor rezulta |V|^2 pași. Este preferata reprezentarea grafului cu liste de adiacenta, pentru care numărul total de operații cauzate de linia (*) va fi egal cu |E|. Complexitatea algoritmului este O(|V|^2+|E|) în cazul în care coada cu priorități este implementata ca o căutare liniara. În acest caz funcția extrage_min se executa în timp O(|V|), iar actualizează(Q) in timp O(1).

O varianta mai eficienta este implementarea cozii ca heap binar. Funcția extrage_min se va executa în timp O(lg|V|); funcția actualizează(Q) se va executa tot în timp O(lg|V|), dar trebuie cunoscuta poziția cheii nod în heap, adică heapul trebuie sa fie indexat. Complexitatea obținută este O(|E|lg|V|) pentru un graf conex.

Cea mai eficienta implementare se obține folosind un heap Fibonacci pentru coada cu priorități:

Aceasta este o structura de date complexa, dezvoltata în mod special pentru optimizarea algoritmului Dijkstra, caracterizata de un timp amortizat de O(lg|V|) pentru operația extrage_min si numai O(1) pentru actualizeaza(Q). Complexitatea obținută este O(|V|lg|V| + |E|), foarte bună pentru grafuri rare.

Algoritmul Bellman – Ford

Algoritmul Bellman Ford poate fi folosit si pentru grafuri ce conțin muchii de cost negativ, dar nu poate fi folosit pentru grafuri ce conțin cicluri de cost negativ (când căutarea unui drum minim nu are sens).
Cu ajutorul sau putem afla daca un graf conține cicluri. Algoritmul folosește același mecanism de relaxare ca si Dijkstra, dar, spre deosebire de acesta, nu optimizează o soluție folosind un criteriu de optim local, ci parcurge fiecare muchie de un număr de ori egal cu numărul de noduri si încearcă sa o relaxeze de fiecare data, pentru a îmbunătăți distanta până la nodul destinație al muchiei curente.

Motivul pentru care se face acest lucru este ca drumul minim dintre sursa si orice nod destinație poate sa treacă prin maximum |V| noduri (adică toate nodurile grafului), respectiv |V|-1 muchii; prin urmare, relaxarea tuturor muchiilor de |V|-1 ori este suficienta pentru a propaga până la toate nodurile informația despre distanta minima de la sursa.

Daca, la sfârșitul acestor |E|*(|V|-1) relaxări, mai poate fi îmbunătățită o distanță, atunci graful are un ciclu de cost negativ si problema nu are soluție.

Menținând notațiile anterioare, pseudocodul algoritmului este:

```
BellmanFord(sursa):

d[sursa] = 0
d[nod] = +∞ // pentru orice nod != sursa
p[nod] = null // pentru orice nod din V

// relaxari succesive
// cum in initializare se face o relaxare (daca exista drum direct de la sursa la nod =>
// d[nod] = w[sursa, nod]) mai sunt necesare |V-1| relaxari
for i = 1 to |V|-1
    foreach (u, v) in E  // E = multimea muchiilor
        daca d[v] > d[u] + w(u,v)
            d[v] = d[u] + w(u,v)
            p[v] = u;

// daca se mai pot relaxa muchii
foreach (u, v) in E
    daca d[v] > d[u] + w(u,v)
        fail (”exista cicluri negativ”)
```

Complexitatea algoritmului este O(|E|*|V|).

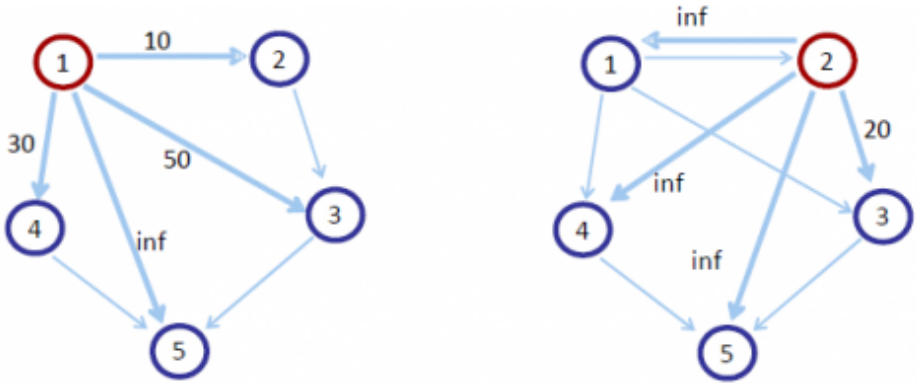
Drumuri minime intre oricare doua noduri

Floyd-Warshall

Algoritmii din aceasta secțiune determina drumul de cost minim dintre oricare doua noduri dintr-un graf. Pentru a rezolva aceasta problema s-ar putea aplica unul din algoritmii de mai sus, considerând ca sursa fiecare nod, pe rând, dar o astfel de abordare ar fi ineficienta.

Algoritmul Floyd-Warshall(intalnit si sub numele de Roy-Floyd) compara toate drumurile posibile din graf dintre fiecare 2 noduri, si poate fi utilizat si in grafuri cu muchii de cost negativ.

Estimarea drumului optim poate fi reținut intr-o structura tridimensionala d[v1, v2, k], cu semnificația – costul minim al drumului de la v1 la v2, folosind ca noduri intermediare doar noduri pana la nodul k. Daca nodurile sunt numerotate de la 1, atunci d[v1, v2, 0] reprezintă costul muchiei directe de la v1 la v2, considerând +∞ daca aceasta nu exista. Exemplu, pentru v1 = 1, respectiv 2:



Pornind cu valori ale lui k de la 1 la |V|, ne interesează să găsim cea mai scurta cale de la fiecare v1 la fiecare v2 folosind doar noduri intermedire din mulțimea {1, …, k}. De fiecare data, comparam costul deja estimat al drumului de la v1 la v2, deci d[v1, v2, k-1] obținut la pasul anterior, cu costul drumurilor de la v1 la k si de la k la v2, adică d[v1, k, k-1] + d[k, v2, k-1], obținutae la pasul anterior. Atunci, d[v1, v2, |V|] va conține costul drumului minim de la v1 la v2.

Pseudocodul acestui algoritm este:

```
FloydWarshall(G):
n = |V|
int d[n, n, n]
foreach (i, j) in (1..n,1..n)
    d[i, j, 0] = w[i,j] // costul muchiei, sau infinit
for k = 1 to n
    foreach (i,j) in (1..n,1..n)
        d[i, j, k] = min(d[i, j, k-1], d[i, k, k-1] + d[k, j, k-1])
```

Complexitatea temporala este O(|V|^3), iar cea spațială este tot O(|V|^3). O complexitate spațială cu un ordin mai mic se obține observând ca la un pas nu este nevoie decât de matricea de la pasul precedent d[i, j, k-1] si cea de la pasul curent d[i, j, k]. O observație și mai bună este că, de la un pas k-1 la k, estimările lungimilor nu pot decât sa scadă, deci putem sa lucram pe o singura matrice. Deci, spațiul de memorie necesar este de dimensiune |V|^2.

Rescris, pseudocodul algoritmului arata astfel:

```
FloydWarshall(G):
n = |V|
int d[n, n]
foreach (i, j) in (1..n,1..n)
    d[i, j] = w[i,j] // costul muchiei, sau infinit
for k = 1 to n
    foreach (i,j) in (1..n,1..n)
        d[i, j] = min(d[i, j], d[i, k] + d[k, j])
```

Pentru a determina drumul efectiv, nu doar costul acestuia, avem doua variante:

1. Se retine o structura de părinți, similara cu cea de la Dijkstra, dar, bineînțeles, bidimensionala.
2. Se folosește divide et impera astfel:

- se caută un pivot k astfel încât cost[i][j] = cost[i][k] + cost[j][k]
- se apelează funcția recursiv pentru ambele drumuri → (i,k),(k,j)
- dacă pivotul nu poate fi găsit, afișăm i
- după terminarea funcției recursie afișăm extremitatea dreapta a drumului

Cazuri speciale

1. Daca avem un graf **neorientat**, **fara cicluri** (un arbore), exista un singur drum intre oricare doua noduri, care poate fi aflat printr-o simpla parcurgere DFS. Folosind diferite preprocesari [8],[9], putem calcula distanta intre oricare doua noduri in timp constant, O(1).

2. Daca avem un graf **orientat**, **fara cicluri** (un DAG [10]), putem sa sa relaxam muchiile nodurilor, parcurgandu-le pe acestea in ordinea data de sortarea topologica. O(|V|+|E|)

3. Daca avem un graf unde toate muchiile au **cost egal**, putem afla distanta minima de la un nod sursa la orice alt nod printr-o parcurgere BFS. (de asemenea, tinand cont de faptul ca pot exista mai multe drumuri pana la un anumit nod). O(|V|+|E|)

4. Pentru grafuri orientate, **rare** (relativ putine muchii), putem folosi algoritmul lui Johnson([11]) pentru calcularea distantei minime de la un nod, la oricare alt nod. O(|V|^2log|V| + |V||E|)

Concluzii

- **Dijkstra***

– calculează drumurile minime de la o sursa către celelalte noduri
– nu poate fi folosit daca exista muchii de cost negativ
– complexitate minima O(|V|lg|V| + |E|) utilizând heapuri Fibonacci;

- **Bellman – Ford**

– calculează drumurile minime de la o sursă către celelalte noduri
– detectează existența ciclurilor de cost negativ
– complexitate O(|V| * |E|)

- **Floyd – Warshall**

– calculează drumurile minime intre oricare doua noduri din graf
– poate fi folosit in grafuri cu cicluri de cost negativ, dar nu le detectează
– complexitate O(|V|^3)

Referinţe:

[1] http://en.wikipedia.org/wiki/Dijkstra's_algorithm [http://en.wikipedia.org/wiki/Dijkstra's_algorithm]

[2] http://en.wikipedia.org/wiki/Bellman-Ford_algorithm [http://en.wikipedia.org/wiki/Bellman-Ford_algorithm]

[3] http://www.algorithmist.com/index.php/Floyd-Warshall's_Algorithm
[http://www.algorithmist.com/index.php/Floyd-Warshall's_Algorithm]

[4] http://en.wikipedia.org/wiki/Binary_heap [http://en.wikipedia.org/wiki/Binary_heap]

[5] http://en.wikipedia.org/wiki/Fibonacci_heap [http://en.wikipedia.org/wiki/Fibonacci_heap]

[6] T. Cormen, C. Leiserson, R. Rivest, C. Stein – Introducere în Algoritmi

[7] C. Giumale – Introducere în analiza algoritmilor

[8] http://en.wikipedia.org/wiki/Range_Minimum_Query [http://en.wikipedia.org/wiki/Range_Minimum_Query]

[9] http://en.wikipedia.org/wiki/Lowest_common_ancestor
[http://en.wikipedia.org/wiki/Lowest_common_ancestor]

[10] http://en.wikipedia.org/wiki/Directed_acyclic_graph [http://en.wikipedia.org/wiki/Directed_acyclic_graph]

[11] http://en.wikipedia.org/wiki/Johnson%27s_algorithm [http://en.wikipedia.org/wiki/Johnson%27s_algorithm]

Resurse

- Varianta PDF a laboratorului
  [http://cs.curs.pub.ro/2015/pluginfile.php/32415/mod_resource/content/1/Lab%208%20Drumuri%20minime.pdf]

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab09.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

Dijkstra

Se da un graf **orientat** cu **n** noduri si **m** arce. Graful are pe arce **costuri pozitive**.

Folositi **Dijkstra** pentru a gasi **costul minim** (**lungimea minima**) a unui drum de la o sursa data (**source**) la toate celelalte n - 1 noduri din graf.

Costul / lungimea unui drum este suma costurilor/lungimilor arcelor care compun drumul.

Restrictii si precizari:

- $n <= 50.000$
- $m <= 2.5 * 10^5$
- $0 <= c <= 20.000$, unde c este costul/lungimea unui arc
- timp de executie
  - C++: 1s
  - Java: 2s

Rezultatul se va returna sub forma unui vector **d** cu **n + 1** elemente.

Conventie:

- **d[node]** = costul minim / lungimea minima a unui drum de la **source** la nodul **node**
- **d[source] = 0**
- **d[node] = -1** , daca nu se poate ajunge de la **source** la **node**

d[0] nu este folosit, deci ca fi initializat cu 0! (am pastrat indexarea nodurilor de la 1)

Bellman-Ford

Se da un graf **orientat conex** cu **n** noduri si **m** arce. Graful are pe arce **costuri pozitive sau negative**.

Folositi **Bellman-Ford** pentru a gasi **costul minim** (**lungimea minima**) a unui drum de la o sursa data (**source**) la toate celelalte n - 1 noduri din graf. In caz ca se va detecta un ciclu de cost negativ, se va semnala acest lucru.

Costul / lungimea unui drum este suma costurilor/lungimilor arcelor care compun drumul.

Restrictii si precizari:

- $n <= 50.000$
- $m <= 2.5 * 10^5$
- $-1.000 <= c <= +1.000$, unde c este costul/lungimea unui arc
- timp de executie
    - C++: 1s
    - Java: 2s
- Pentru punctaj maxim, implementarea din laborator trebuie sa treaca toate testele, cu exceptia testelor 8 si 9, pe care va lua TLE. Pentru cei curiosi, exista si o implementare mai eficienta a algoritmului, oarecum similara cu cea de la Dijkstra (pentru mai multe detalii: https://infoarena.ro/problema/bellmanford [https://infoarena.ro/problema/bellmanford])

Rezultatul se va returna sub forma unui vector **d** cu **n + 1** elemente.

Conventie:

- **d[node]** = costul minim / lungimea minima a unui drum de la **source** la nodul **node**
- **d[source] = 0**
- **d[node] = -1** , daca nu se poate ajunge de la **source** la **node**

d[0] nu este folosit, deci ca fi initializat cu 0! (am pastrat indexarea nodurilor de la 1)

ATENTIE!!! Este posibil ca un astfel de graf sa aiba ciclu de cost negativ. In cazul detectarii unui ciclu de cost negativ, functia voastra va returna un vector gol! (**std::vector<int>() / {}** sau **new ArrayList<Integer>()**).

RoyFloyd

Se da un graf **orientat** cu **n** noduri. Graful are **costuri pozitive** pe arce.

Se da **matricea ponderilor** , se cere **matricea drumurilor minime**.

Restrictii si precizari:

- $n <= 100$
- $0 <= c <= 1.000$, unde c este costul unui arc
- daca **nu exista muchie** intre o pereche de noduri x si y, distanta de la nodul x la nodul y din **matricea ponderilor** va fi 0
- daca dupa aplicarea algoritmului **nu se gaseste drum** pentru o pereche de noduri x si y, se va considera **distanta** dintre ele egala cu 0 (se stocheaza in **matricea distantelor** valoarea 0)
- drumul de la nodul i la nodul i are lungime 0 (prin conventie)
- timp de executie
    - C++: 1s
    - Java: 2s

Rezultatul se va **stoca** in matricea **d** declarata in schelet! Algoritmul vostru trebuie doar sa o populeze corect, tinand cont ca nodurile sunt indexate de la 1.

BONUS

Pentru exercitiul cu Dijkstra, **reconstituiti** drumul de lungime minima **source** la celelalte noduri din graf.

Extra

Rezolvati problema rfinv [https://infoarena.ro/problema/rfinv] pe infoarena.
Rezolvati problema coach [https://infoarena.ro/problema/coach] pe infoarena.
Rezolvati problema rf [https://infoarena.ro/problema/rf] pe infoarena.
Rezolvati problema TODO [https://infoarena.ro/problema/TODO] pe infoarena.

pa/laboratoare/laborator-09.txt · Last modified: 2020/05/05 19:12 by darius.neatu

Laborator 10 - Arbori Minimi de Acoperire

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- Însuşirea conceptului de arbore minim de acoperire;
- Înţelegerea modului de funcţionare a algoritmilor de determinare a unui arbore minim de acoperire prezentaţi;
- Aplicarea algoritmilor în rezolvarea problemelor;

Importanţă – aplicaţii practice

Găsirea unui arbore minim de acoperire pentru un graf are aplicaţii în domenii cât se poate de variate:

- Reţele (de calculatoare, telefonie, cablu TV, electricitate, drumuri): se doreşte interconectarea mai multor puncte, cu un cost redus şi atunci este utilă cunoaşterea arborelui care conectează toate punctele, cu cel mai mic cost posibil. STP(Spanning Tree Protocol) este un protocol de rutare care previne apariţia buclelor într-un LAN, şi se bazează pe crearea unui arbore de acoperire. Singurele legături active sunt cele care apar în acest arbore, iar astfel se evită buclele.
- Segmentarea imaginilor: împărţirea unei imagini în regiuni de pixeli cu proprietăţi asemănătoare. E utilă mai apoi în analiza medicală a unei zone afectate de o tumoare de exemplu.
- Algoritmi de aproximare pt probleme NP-dure: problema comis-voiajorului, arbori Steiner.
- Clustering: pentru detectarea de clustere cu forme neregulate [8], [9].

Descrierea problemei şi a rezolvărilor

Dându-se un graf conex neorientat G =(V, E), se numeşte arbore de acoperire al lui G un subgraf G'=(V, E') care conţine toate vârfurile grafului G şi o submulţime minimă de muchii E'⊆ E cu proprietatea că uneşte toate vârfurile şi nu conţine cicluri. Cum G' este conex şi aciclic, el este arbore. Pentru un graf oarecare, există mai mulţi arbori de acoperire.

Dacă asociem o matrice de costuri, w, pentru muchiile din G, fiecare arbore de acoperire va avea asociat un cost egal cu suma costurilor muchiilor conţinute. Un arbore care are costul asociat mai mic sau egal cu costul oricărui alt arbore de acoperire se numeşte arbore minim de acoperire (minimum spanning tree) al grafului G. Un graf poate avea mai mulţi arbori minimi de acoperire. Dacă toate costurile muchiilor sunt diferite, există un singur AMA. Primul algoritm pentru determinarea unui arbore minim de acoperire a fost scris în 1926 de Otakar Boruvka. În prezent, cei mai folosiţi algoritmi sunt Prim şi Kruskal. Toţi trei sunt algoritmi greedy, şi rulează în timp polinomial. La fiecare pas, pentru a construi arborele se alege cea mai bună variantă posibilă la momentul respectiv. Generic, algoritmul de determinare a unui AMA se poate scrie astfel:

```
ArboreMinimDeAcoperire(G(V, E), c)
        MuchiiAMA = ∅;
        while (MuchiiAMA nu reprezintă muchiile unui arbore minim de acoperire)
                găsește o muchie (u, v) care este sigură pentru MuchiiAMA;
                MuchiiAMA = MuchiiAMA ∪ {(u, v)};
        return MuchiiAMA;
```

O muchie sigură este o muchie care se poate adăuga unei submulţimi de muchii ale unui arbore minim de acoperire, astfel încât noua mulţime obţinută să aparţină tot unui arbore minim de acoperire. Iniţial, MuchiiAMA este o mulţime vidă. La fiecare pas, se adaugă câte o muchie sigură, deci MuchiiAMA rămâne o submulţime a unui AMA. În consecinţă, la sfarşitul rulării algoritmului (când muchiile din mulţime unesc toate nodurile din graf), MuchiiAMA va conţine de fapt arborele minim de acoperire dorit.

Algoritmul Kruskal

Algoritmul a fost dezvoltat în 1956 de Joseph Kruskal. Determinarea arborelui minim de acoperire se face prin reuniuni de subarbori minimi de acoperire. Iniţial, se consideră că fiecare nod din graf este un arbore. Apoi, la fiecare pas se selectează muchia de cost minim care uneşte doi subarbori disjuncţi, şi se realizează unirea celor doi subarbori. Muchia respectivă se adaugă la mulţimea MuchiiAMA, care la sfârşit va conţine chiar muchiile din arborele minim de acoperire.

Pseudocod

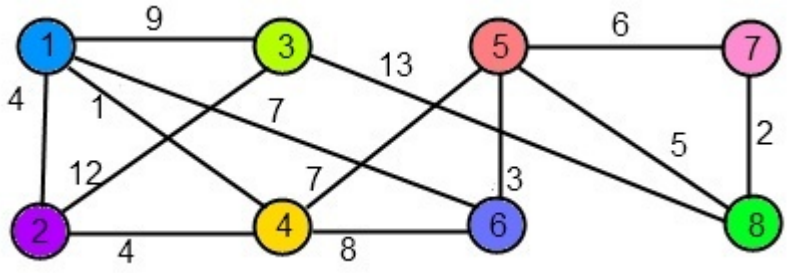
```
Kruskal(G(V, E), w)
MuchiiAMA <- Ø;
for each v in V do
        MakeSet(v);     //fiecare nod e un arbore diferit
sort(E);                //sortează muchiile în ordine crescătoare a costului
for each (u,v) in E do
        if (FindSet(u) != FindSet(v)) then      //capetele muchiei fac parte //din subarbori disjuncţi
                MuchiiAMA = MuchiiAMA ∪ {(u, v)};       //adaugă muchia la arbore
                Union(u, v);    //unește subarborii corespunzători lui u și v
return MuchiiAMA;
```

Bucla principală for poate fi înlocuită cu o buclă while, în care se verifică dacă în MuchiiAMA există mai puţin de |V| - 1 muchii, pentru că orice arbore de acoperire are |V| - 1 muchii, iar la fiecare pas se adaugă o muchie sigură.

Exemplu de rulare

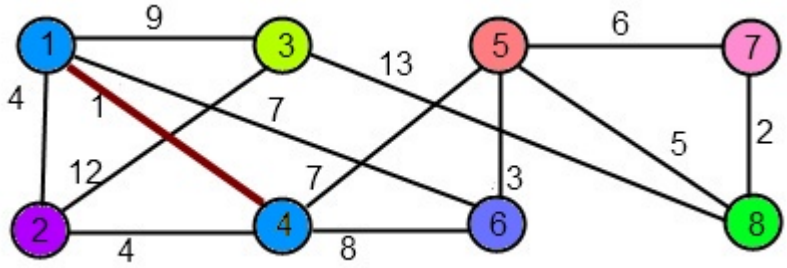
Se consideră graful din figura următoare:

Fiecare subarbore va fi colorat diferit. Cum iniţial fiecare nod reprezintă un subarbore, nodurile au culori diferite. Pe măsură ce subarborii sunt uniţi, nodurile aparţinând aceluiaşi subarbore vor fi coloraţi identic. Costurile muchiilor sunt sortate în ordine crescătoare.



Pas 1

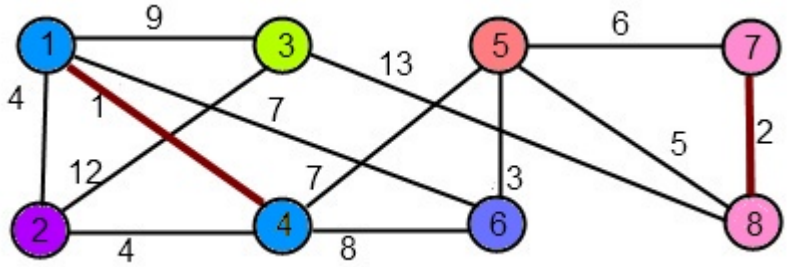
Se alege prima muchie, (1,4). Se observă că uneşte subarborii {1} şi {4}, deci muchia e adăugată la MuchiiAMA, iar cei doi subarbori se unesc.



MuchiiAMA = {(1,4)}.

Pas 2

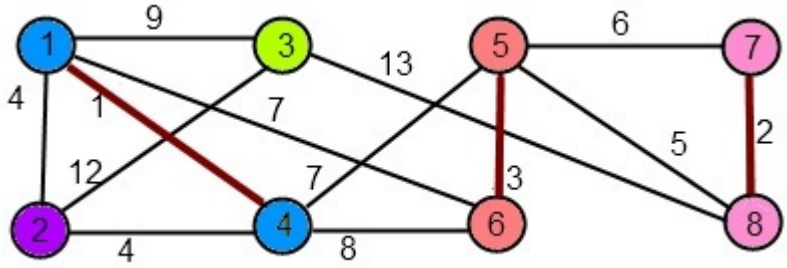
Următoarea muchie este (7,8), care uneşte {7} şi {8}. Se adaugă la MuchiiAMA şi se unesc cei doi subarbori.



MuchiiAMA = {(1,4),(7,8)}.

Pas 3

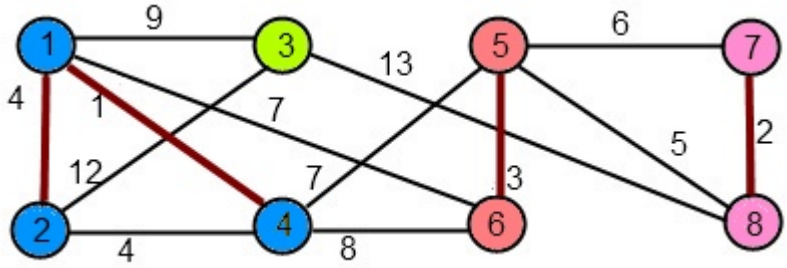
Următoarea muchie este (5,6), care uneşte {5} şi {6}. Se adaugă la MuchiiAMA şi se unesc cei doi subarbori.



MuchiiAMA = {(1,4),(7,8),(5,6)}.

Pas 4

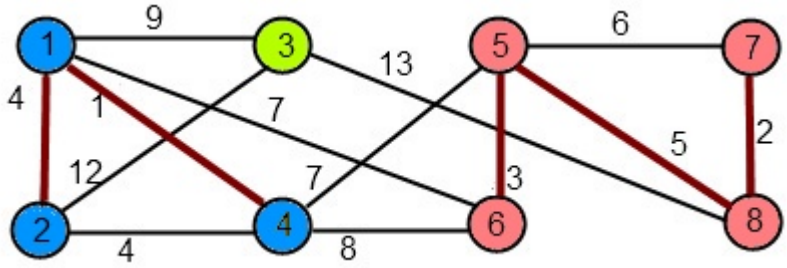
Următorul cost este 4. Se observă că muchiile (1,2) şi (2,4) au costul 4 şi unesc {2} cu {1,4}. Se adaugă la MuchiiAMA una dintre cele două muchii, fie ea (1,2), şi se unesc cei doi subarbori. Alegerea muchiei (2,4) va duce la găsirea unui alt AMA. [Am spus anterior că un graf poate avea mai mulţi arbori minimi de acoperire, cu acelaşi cost, dacă există muchii diferite cu acelaşi cost.]



MuchiiAMA = {(1,4),(7,8),(5,6),(1,2)}.

Pas 5

Următoarea muchie de cost minim este (5,8), care uneşte {5,6} şi {7,8}. Se adaugă la MuchiiAMA şi se unesc cei doi subarbori, rezultând {5,6,7,8}.



MuchiiAMA = {(1,4),(7,8),(5,6),(1,2), (5,8)}.

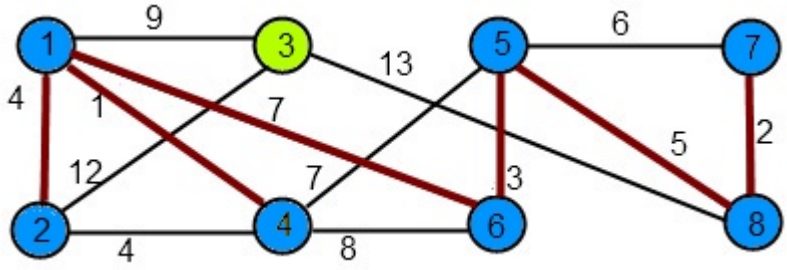
Pas 6

Muchia (5,7), care are cel mai mic cost actual, are ambele extremităţi în subarborele {5,6,7,8}. În consecinţă, nu se efectuează nicio schimbare.

MuchiiAMA = {(1,4),(7,8),(5,6),(1,2),(5,8)}.

Pas 7

Următorul cost este 7. Se observă că muchiile (1,6) şi (4,5) au costul 7 şi unesc subarborii {1,2,4} şi {5,6,7,8}. Se adaugă la MuchiiAMA (1,6), şi se unesc cei doi subarbori. Alegerea muchiei (4,5) va duce la găsirea unui alt AMA.



MuchiiAMA = {(1,4),(7,8),(5,6),(1,2),(5,8),(1,6)}.

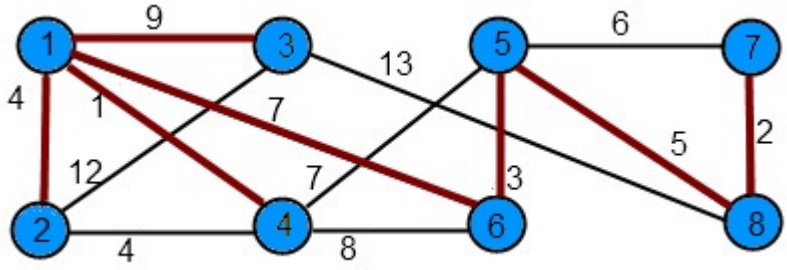
Pas 8

Muchia (4,6) de cost 8 are capetele în acelaşi subarbore, deci nu se produc schimări.

MuchiiAMA = {(1,4),(7,8),(5,6),(1,2),(5,8),(1,6)}.

Pas 9

Muchia (1,3) de cost 9 uneşte cei doi subarbori rămaşi, {1,2,4,5,6,7,8} şi {3}. Deci după unire obţinem un singur arbore. (1,3) se adaugă la MuchiiAMA, care va conţine acum 7 muchii, iar algoritmul se opreşte.



Arborele minim de acoperire obţinut este {(1,4),(7,8),(5,6),(1,2),(5,8),(1,6), (1,3)}.Costul său se calculează însumând costurile tuturor muchiilor:

Cost(MuchiiAMA) = 1 + 2 + 3 + 4 + 5 + 7 + 9 = 31

Alţi arbori minimi de acoperire pentru exemplul propus sunt: * {(1,4),(7,8),(5,6),(1,2),(5,8),(4,5), (1,3)} * {(1,4),(7,8),(5,6),(2,4),(5,8),(1,6), (1,3)} * {(1,4),(7,8),(5,6),(2,4),(5,8),(4,5), (1,3)}.

Pentru alte exemple explicate consultaţi [2], [3] şi [5].

Complexitate

Timpul de execuţie depinde de implementarea structurilor de date pentru mulţimi disjuncte. Vom presupune că se foloseşte o pădure cu mulţimi disjuncte [2][11][12]. Iniţializarea se face într-un timp O(|V|). Sortarea muchiilor în funcţie de cost se face în O(|E|log|E|). În bucla principală se execută |E| operaţii care presupun două operaţii de găsire a subarborilor din care fac parte extremităţile muchiilor şi eventual o reuniune a acestor arbori, într-un timp O(|E|log|E|).

Deci, complexitatea totală este: O(|V|) + O(|E|log|E|) + O(|E|log|E|) = O(|E|log|E|).

Cum |E| ⇐ |V|^2, şi O(log(|V|^2)) = O(2*log(|V|)) = O(log(|V|)), rezultă o complexitate O(|E|log|V|).

Algoritmul Prim

Algoritmul a fost prima oară dezvoltat în 1930 de matematicianul ceh Vojtěch Jarnik, şi independent în 1957 de informaticianul Robert Prim, al cărui nume l-a luat. Algoritmul consideră iniţial că fiecare nod este un subarbore independent, ca şi Kruskal. Însă spre deosebire de acesta, nu se construiesc mai mulţi subarbori care se unesc şi în final ajung să formeze AMA, ci există un arbore principal, iar la fiecare pas se adaugă acestuia muchia cu cel mai mic cost care uneşte un nod din arbore cu un nod din afara sa. Nodul rădăcină al arborelui principal se alege arbitrar. Când s-au adăugat muchii care ajung în toate nodurile grafului, s-a obţinut AMA dorit. Abordarea seamănă cu algoritmul Dijkstra de găsire a drumului minim între două noduri ale unui graf.

Pentru o implementare eficientă, următoarea muchie de adăugat la arbore trebuie să fie uşor de selectat. Vârfurile care nu sunt în arbore trebuie sortate în funcţie de distanţa până la acesta (de fapt costul minim al unei muchii care leagă nodul dat de un nod din interiorul arborelui). Se poate folosi pentru aceasta o structură de heap. Presupunând că (u, v) este muchia de cost minim care uneşte nodul u cu un nod v din arbore, se vor reţine două informaţii:

- d[u] = w[u,v] distanţa de la u la arbore
- p[u] = v predecesorul lui u în drumul minim de la arbore la u.

La fiecare pas se va selecta nodul u cel mai apropiat de arborele principal, reunind apoi arborele principal cu subarborele corespunzător nodului selectat. Se verifică apoi dacă există noduri mai apropiate de u decât de nodurile care erau anterior în arbore, caz în care trebuie modificate distanţele dar şi predecesorul. Modificarea unei distanţe impune şi refacerea structurii de heap.

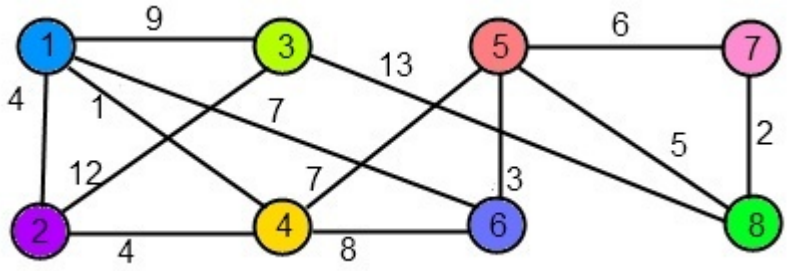
Pseudocod

```
Prim(G(V,E), w, root)
1.      MuchiiAMA <- Ø;
2.      for each u in V do
3.              d[u] = INF;     //inițial distanțele sunt infinit
4.              p[u] = NIL;     //și nu există predecesori
5.      d[root] = 0;    //distanța de la rădăcină la arbore e 0
6.      H = Heap(V,d);  //se construiește heap-ul
7.      while (H not empty) do  //cât timp mai sunt noduri neadăugate
8.              u = GetMin(H);  //se selectează cel mai apropiat nod u
9.              MuchiiAMA = MuchiiAMA ∪ {(u, p[u])};//se adaugă muchia care unește u cu un nod din arborele principal
10.             for each v in Adj(u) do
                //pentru toate nodurile adiacente lui u se verifică dacă
                //trebuie făcute modificări
11.                     if w[u][v] < d[v] then
12.                             d[v] = w[u][v];
13.                             p[v] = u;
14.                             Heapify(v, H);  //refacerea structurii de heap
15.     MuchiiAMA = MuchiiAMA \ {(root, p[root])};
16.     return MuchiiAMA;
```

Exemplu de rulare

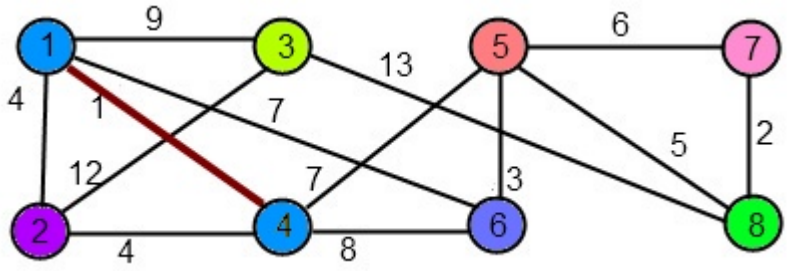
Se consideră graful folosit pentru exemplificarea algoritmului Kruskal.

Iniţial fiecare nod reprezintă un arbore independent, şi are o culoare unică. Se alege ca rădăcină a arborelui principal nodul 1.



Pas 1

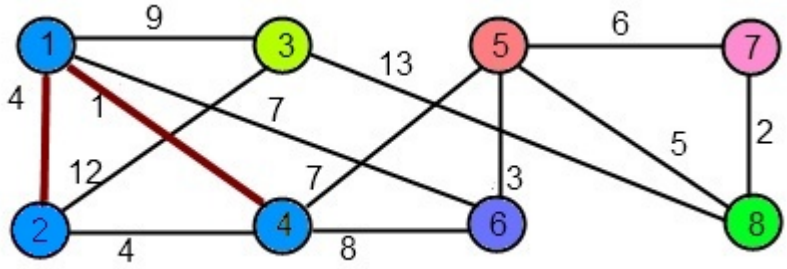
Se alege muchia de cost minim care uneşte rădăcina cu un alt nod: (1,4) de cost 1 si se adaugă arborelui principal.



MuchiiAMA = {(1,4)}.

Pas 2

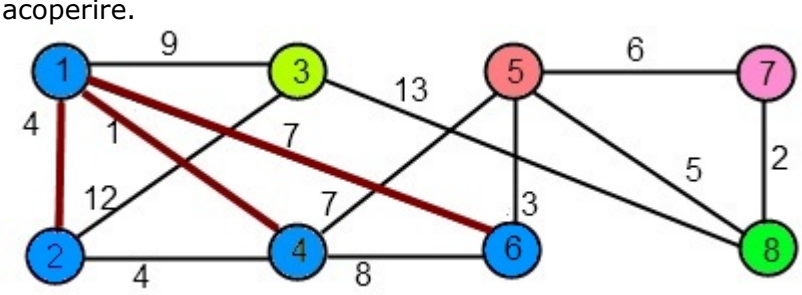
Se alege muchia de cost minim care uneşte 1 sau 4 cu un alt nod. (1,2) şi (2,4) au ambele costul 4. Se alege una dintre ele, fie ea (1,2).



MuchiiAMA = {(1,4),(1,2)}.

Pas 3

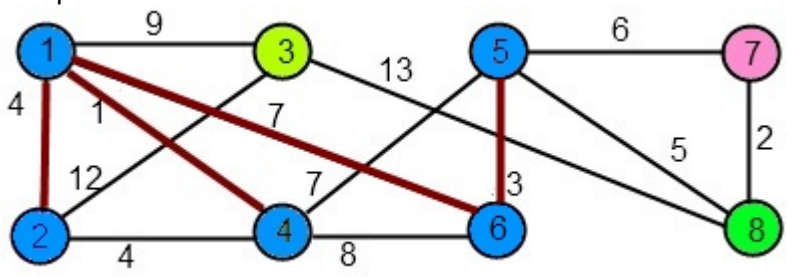
Se alege următoarea muchie care uneşte {1,4,2} cu alt nod. (1,6) şi (4,5) au acelaşi cost, dar adaugă arborelui noduri diferite. La acest pas se selectează de exemplu (1,6). Selectând la acest pas (4,5) s-ar obţine un alt arbore de acoperire.



MuchiiAMA = {(1,4),(1,2),(1,6)}.

Pas 4

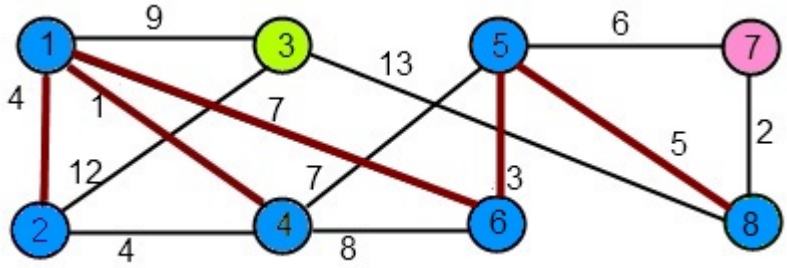
Se caută muchia de cost minim care unește {1,4,2,6} cu un alt nod, și se găsește (6,5) de cost 3. Nodul 5 va fi adăugat arborelui principal.



MuchiiAMA = {(1,4),(1,2),(1,6),(6,5)}.

Pas 5

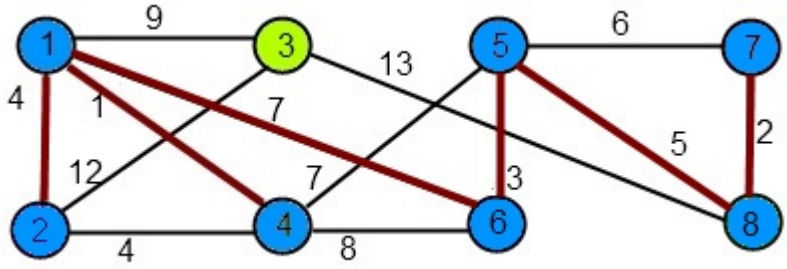
Se alege muchia de cost minim care unește {1,4,2,6,5} cu un alt nod. (5,8) de cost 5 se adaugă listei de muchii.



MuchiiAMA = {(1,4),(1,2),(1,6),(5,6),(5,8)}

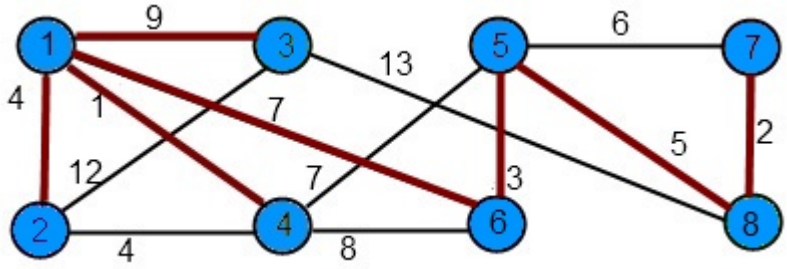
Pas 6

Se alege muchia de cost minim (8,7) care unește {1,4,2,6,5,8} cu nodul 7.



MuchiiAMA = {(1,4),(1,2),(1,6),(5,6),(5,8),(8,7)}.

Pas 7



Se alege muchia (1,3) de cost 9, care unește arborele principal cu ultimul nod ramas, și se adaugă mulțimii de muchii. Cum toate nodurile sunt acoperite, am obținut un arbore minim de acoperire.

MuchiiAMA = {(1,4),(1,2),(1,6),(5,6),(5,8),(8,7),(1,3)}.

Cost(MuchiiAMA) = 1 + 4 + 7 + 3 + 5 + 2 + 9 = 31

Alți arbori minimi de acoperire pentru exemplul propus se pot obține alegând diferit muchiile cu același cost (vezi pașii 2 și 3) .

Pentru alte exemple explicate consultaţi [2], [4] şi [6].

Complexitate

Iniţializările se fac în O(|V|). Bucla principală while se execută de |V| ori. Procedura GetMin() are nevoie de un timp de ordinul O(lg|V|), deci toate apelurile vor dura O(|V|lg|V|). Bucla for este executată în total de O(|E|) ori, deoarece suma tuturor listelor de adiacenţă este 2|E|. Modificarea distanţei, a predecesorului, şi refacerea heapului se execută într-un timp de O(1), O(1) şi respectiv O(lg|V|). Deci în total bucla interioară for durează O(|E|lg|V|).

În consecinţă, timpul total de rulare este O(|V|lg|V|+|E|lg|V|), adică O(|E|lg|V|). Aceeaşi complexitate s-a obţinut şi pentru algoritmul Kruskal. Totuşi, timpul de execuţie al algoritmului Prin se poate îmbunătăţi până la O(|E|+|V|lg|V|), folosind heap-uri Fibonacci.

Concluzii

Un arbore minim de acoperire al unui graf este un arbore care conţine toate nodurile, şi în plus acestea sunt conectate prin muchii care asigură un cost total minim. Determinarea unui arbore minim de acoperire pentru un graf este o problemă cu aplicaţii în foarte multe domenii: reţele, clustering, prelucrare de imagini. Cei mai cunoscuţi algoritmi, Prim şi Kruskal, rezolvă problema în timp polinomial. Performanţa algoritmilor depinde de modul de reprezentare a structurilor de date folosite.

Referinţe

[1] – http://en.wikipedia.org/wiki/Minimum_spanning_tree
[http://en.wikipedia.org/wiki/Minimum_spanning_tree]

[2] – T. Cormen, C. Leiserson, R. Rivest, C. Stein – Introducere în Algoritmi, cap. 24

[3] – http://en.wikipedia.org/wiki/Kruskal%27s_algorithm
[http://en.wikipedia.org/wiki/Kruskal%27s_algorithm]

[4] – http://en.wikipedia.org/wiki/Prim%27s_algorithm [http://en.wikipedia.org/wiki/Prim%27s_algorithm]

[5] – http://w3.cs.upt.ro/~calin/resources/sdaa/kruskal.ppt
[http://w3.cs.upt.ro/~calin/resources/sdaa/kruskal.ppt]

[6] – http://www.cs.upt.ro/~calin/resources/sdaa/prim.ppt
[http://www.cs.upt.ro/~calin/resources/sdaa/prim.ppt]

[7] – http://www.cs.princeton.edu/~wayne/kleinberg-tardos/04mst.pdf
[http://www.cs.princeton.edu/~wayne/kleinberg-tardos/04mst.pdf]

[8] – http://hc.ims.u-tokyo.ac.jp/JSBi/journal/GIW01/GIW01F03.pdf [http://hc.ims.u-tokyo.ac.jp/JSBi/journal/GIW01/GIW01F03.pdf]

[9] – http://www4.ncsu.edu/~zjorgen/ictai06.pdf [http://www4.ncsu.edu/~zjorgen/ictai06.pdf]

[10] – C. Giumale – Introducere în Analiza Algoritmilor, cap.5.5

[11] - Păduri de mulțimi disjuncte - Infoarena [https://www.infoarena.ro/problema/disjoint]

[12] - Păduri de mulțimi disjuncte - Wikipedia [https://en.wikipedia.org/wiki/Disjoint-set_data_structure]

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab10.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

Kruskal

Se da un graf **neorientat conex** cu **n** noduri si **m** muchii. Folositi algoritmul lui Kruskal pentru a calcula costul minim al unui arbore de acoperire.

Restrictii si precizari:

- $n <= 2 * 10^5$
- $m <= 4 * 10^5$
- $-10^3 <= c <= 10^3$, unde c este costul unei muchii

Prim

Aceeasi cerinta si aceleasi restrictii ca mai sus, doar ca se cere gasirea costului minim al unui arbore de acoperire folosind algoritmul lui Prim.

pa/laboratoare/laborator-10.txt · Last modified: 2020/04/24 10:12 by darius.neatu

Laborator 11 - Flux Maxim

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

- formalizarea noțiunilor de rețea de transport și flux în rețea
- prezentarea unei metode de rezolvare a problemei de flux maxim
- analiza unei implementări a metodei oferite

Importanţă – aplicaţii practice

Un graf orientat poate fi utilizat pentru modelarea unui proces de transport într-o rețea între un producător *s* și un consumator *t*. Destinația nu poate consuma mai mult decât se produce, iar cantitatea trimisă pe o cale nu poate depăși capacitatea sa de transport.

Rețelele de transport pot modela curgerea lichidului în sisteme cu țevi, deplasarea pieselor pe benzi rulante, deplasarea curentului prin rețele electrice, transmiterea informațiilor prin rețele de comunicare etc.

Descrierea problemei şi a rezolvărilor

O problemă des întâlnită într-o rețea de transport este cea a găsirii fluxului maxim posibil prin arcele rețelei astfel încât:

1. să nu fie depășite capacitățile arcelor

2. fluxul să se conserve în drumul său de la *s* la *t*

Definiție 1

O rețea de transport este un graf orientat $G = (V,E)$ cu proprietățile:

1. există două noduri speciale în V: *s* este nodul sursă (sau producătorul) și *t* este nodul terminal (sau consumatorul).

2. este definită o funcție totală de capacitate $c: V \times V \rightarrow R+$ astfel încât:

- $c(u,v) = 0$ dacă $(u,v) \notin E$
- $c(u,v) \geq 0$ daca $(u,v) \in E$

3. pentru orice nod $v \in V \setminus \{s,t\}$ există cel puțin o cale $s \rightarrow v \rightarrow t$.

Definiție 2

Numim flux în rețeaua $G = (V,E)$ o funcție totală $f: V \times V \rightarrow R$ cu proprietățile:

1. Restricție de capacitate:

- $f(u,v) \leq c(u,v), \forall (u,v) \in V$ - fluxul printr-un arc nu poate depăși capacitatea acestuia

2. Antisimetrie:

- $f(u,v) = -f(v,u)$, $\forall u \in V, \forall v \in V$

3. Conservarea fluxului:

- $\Sigma f(u,v) = 0, \forall u \in V \setminus \{s,t\}, v \in V$

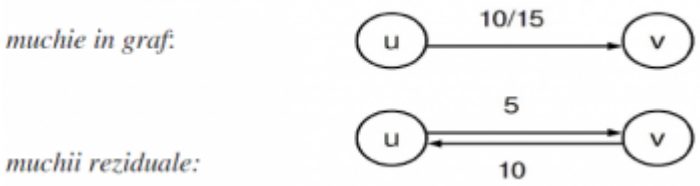
Un flux negativ de la u la v este unul virtual, el nu reprezintă un transport efectiv, ci doar sugerează că există un transport fizic de la v la u (este o convenție asemănătoare cu cea făcută pentru intensitățile curenților într-o rețea electrică). Ultima proprietate ne spune că la trecerea printr-un nod fluxul se conservă: suma fluxurilor ce intră într-un nod este 0 (ținând cont de convenția de semn stabilită).

Numim **capacitate reziduală** a unui arc

$c_f(u,v) = c(u,v) - f(u,v)$

și o interpretăm ca fiind cantitatea de flux adițional care poate fi transportat de la *u* la *v*, fără a depăși capacitatea $c(u,v)$.

Exemplu:



Daca avem arcul $(u,v) \in V$ cu $c(u,v) = 15$ și $f(u,v) = 10$, se pot transporta $c_f(u,v) = 5$ unități suplimentare fără a încălca restricția de capacitate. Dar, conform definiției, deși arcul $(v,u) \notin V$ vom avea totuși o capacitate reziduală $c_f(v,u) = c(v,u) - f(v,u) = 0 - (-10) = 10$: as putea transporta 10 unități în sens opus care să le anuleze pe cele 10 ale fluxului direct pe muchia (u,v).

Definiție 3

Fie o rețea de flux $G = (V,E)$, iar *f* fluxul prin G. Numim rețea reziduală a lui G, indusă de *f*, o rețea de flux notată cu $G_f = (V, E_f)$, astfel încât

- $E_f = \{(u,v) \in V \mid c_f(u,v) = c(u,v) - f(u,v) > 0\}$

Este important de observat că E_f și E pot fi disjuncte: un arc rezidual (u,v) apare în rețeaua reziduală doar dacă capacitatea sa este strict pozitivă (ceea ce nu implică existența arcului în rețeaua originală).

Un drum de ameliorare este o cale $(u1, u2, \ldots, uk)$, unde $u1 = s$ și $uk = t$, în graful rezidual cu $c_f(ui, ui+1) > 0, \forall i=1,2,\ldots,k-1$. Practic, un drum de ameliorare va reprezenta o cale în graf prin care se mai poate pompa flux adițional de la sursa la destinație.

Așa cum era de intuit, capacitatea reziduală a unui drum de ameliorare p este cantitatea maximă de flux ce se poate transporta de-a lungul lui:

- $c_f(p) = \min\{c_f(u,v) \mid (u,v) \in p\}$

Acum că am introdus noțiunile necesare pentru formalizarea problemei de flux maxim într-un graf, putem să prezentăm și cea mai utilizată metodă de rezolvare.

Algoritmul Ford-Fulkerson

Aceasta este o metodă iterativă de găsire a fluxului maxim într-un graf care pleacă de la ideea: cât timp mai există un drum de ameliorare (o cale de la sursă la destinație) pot pompa pe această cale un flux suplimentar egal cu capacitatea reziduală a căii.

Acest algoritm reprezintă mai mult un șablon de rezolvare pentru că nu detaliază modul în care se alege drumul de ameliorare din rețeaua reziduală.

```
Ford_Fulkerson
    Input G(V,E), s, t
    Output |fmax|
    |fmax| ← 0
    f{u,v) ← 0, ∀(u,v) ∈ V×V
    while ∃a path p(s ---> t) in Gf such that cf(u,v) > 0, ∀(u,v) ∈ p
        find cf(p) = min{cf(u,v) | cf(u,v) ∈ p}
        |fmax| += cf(p)
        for-each (u,v) ∈ p
            f(u,v) ← f(u,v) + cf(p)
            f(v,u) ← –f(u,v)
    return |fmax|
```

Complexitatea va fi $O(E * f max)$ pentru că în ciclul while putem găsi, în cel mai rău caz, doar cai care duc la creșterea fluxului cu doar o unitate la fiecare pas.

Ne punem acum problema dacă acest algoritm este corect, dacă la final vom avea cu adevărat fluxul maxim posibil prin graf. Corectitudinea algoritmului derivă imediat din teorema **Flux maxim - tăietura minimă**.

Numim o **tăietură a unui graf** o partiție (S,T) a nodurilor sale cu proprietatea $s \in S$ si $t \in T$.

Teorema flux maxim – tăietura minimă:

Pentru o rețea de flux $G(V,E)$ următoarele afirmații sunt echivalente:

1. f este fluxul maxim în G

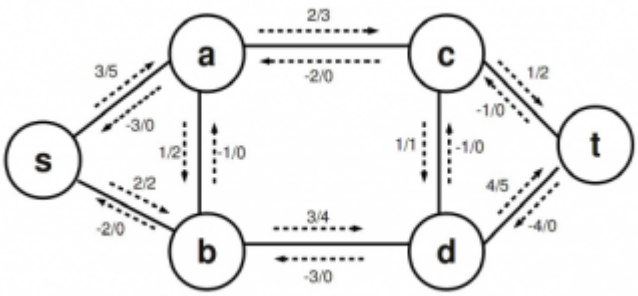
2. Rețeaua reziduală G_f nu conține drumuri de ameliorare

3. Există o tăietură (S,T) a lui G astfel încât fluxul net prin tăietură este egal cu capacitatea acelei tăieturi.

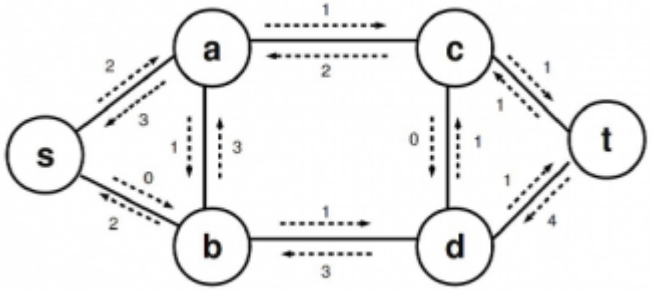
Obs: Prin orice tăietură fluxul este egal cu cel maxim pentru că nu există o altă cale pe care ar putea ajunge flux de la sursă la destinație și care să nu treacă prin tăietură (ar încălca tocmai definiția ei); sau, altfel spus, valoarea unui flux într-o rețea este dată de fluxul oricărei tăieturi. Astfel, fluxul total va fi mărginit de cea mai mică capacitate a unei tăieturi. Dacă este îndeplinit punctul 3. al teoremei atunci știm că acea tăietură nu poate fi decât una de capacitate minimă. Ultima incercare de a gasi o cale de la sursa la drena va rezulta in gasirea doar a elementelor marginite de o astfel de taietura.

Exemplu:

Rețeaua inițială:



Rețeaua reziduală:



Observăm că deși muchia `(d,c)` are capacitate 0 în rețeaua originală (ea nu ar putea transporta flux) în rețeaua reziduală avem `c(d,c) = 1` ceea ce îi permite să facă parte din drumul de ameliorare `p1 = (s, a, b, d, c, t)` de capacitate c_f`(p1) = 1`, astfel că adăugarea acestui flux suplimentar pe muchia `(d,c)` nu va duce la încălcarea restricției de capacitate; sau am fi putut alege drumul `p2=(s, a, c, t)` cu c_f`(p2) = 1` sau `p3 = (s,a,b,d,t)` cu c_f`(p3) = 1`. Performanța algoritmului ține de modul în care va fi ales drumul de ameliorare, se poate întâmpla ca pentru `|fmax|` de valoarea mare o alegere nepotrivită să ducă la timpi de execuție foarte mari.

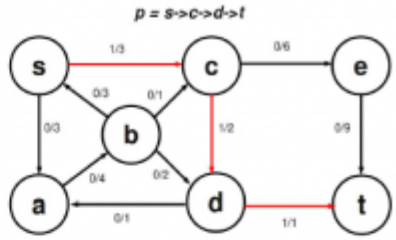
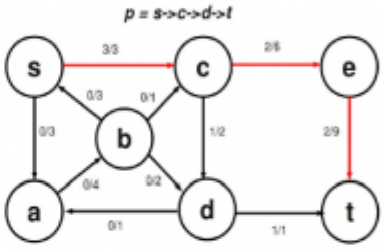
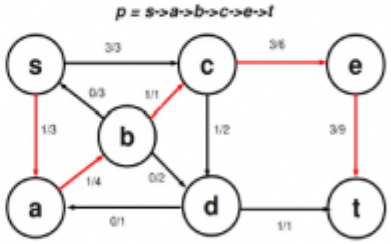
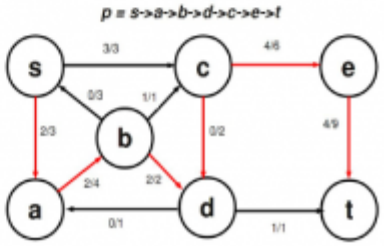
Implementarea Edmonds-Karp

Așa cum am văzut, algoritmul Ford-Fulkerson nu definește o metodă de alegere a drumului de ameliorare pe baza căruia se modifică fluxul în graf. Implementarea Edmonds-Karp alege întotdeauna cea mai scurtă cale folosind o căutare în lățime în graful rezidual unde fiecare arc are ponderea 1. Se poate demonstra că lungimea căilor găsite astfel crește monoton cu fiecare nouă ameliorare.

```
Edmonds-Karp
    Input G(V,E), s, t
    Output |fmax|
    |fmax| ← 0
    f(u,v) ← 0, ∀(u,v) ∈ V×V
    while true
        p(s ⇢ t) ← BFS(Gf,s,t)
        if not ∃p(s ⇢ t)
            break;
        find cf(p) = min{cf(u,v) | cf(u,v) ∈ p}
        |fmax| += cf(p)
        for-each (u,v) ∈ p
            f(u,v) ← f(u,v) + cf(p)
            f(v,u) ← –f(u,v)
    return |fmax|
```

Plecând de la ideea că drumurile de ameliorare găsite au lungimi din ce în ce mai mari se poate arăta că în această implementare fluxul se mărește de cel mult `O(V*E)` ori ([1], pag.513).

Complexitatea algoritmului va fi $O(V*E^2)$. Să luăm un exemplu de rulare al acestui algoritm. Vom considera starea rețelei după ce a fost găsita prima cale de pompare flux(inițial toate arcele sunt etichetate cu 0/0 conform notației stabilite):

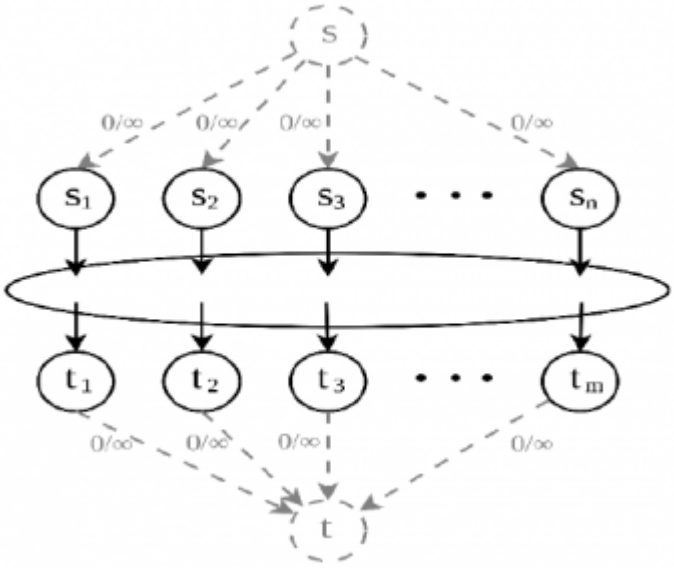
Capacitatea reziduala a caii de ameliorare	Graful si calea de ameliorare gasita
$c_f(p)=min(c_f(s,c),c_f(c,d),c_f(d,t))=$ $=min(3-0,2-0,1-0)=min(3,2,1)=1$	
$c_f(p)=min(c_f(s,c),c_f(c,e),c_f(e,t))=$ $=min(3-1,6-0,9-0)=min(2,6,9)=2$	
$c_f(p)=min(c_f(s,a),c_f(a,b),c_f(b,c),$ $c_f(c,e),c_f(e,t))=$ $=min(3-0,4-0,1-0,6-2,9-2)=$ $=min(3,4,1,4,7)=1$	
$c_f(p)=min(c_f(s,a),c_f(a,b),c_f(b,d),$ $c_f(d,c),c_f(c,e),c_f(e,t))=$ $=min(3-1,4-1,2-0,0-(-1),6-3,9-3)=$ $=min(2,3,2,1,3,6)=1$	

Obs: In cazul ultimului drum de ameliorare găsit in exemplul dat, se observa ca deși nu exista muchia `(d,c)` se simulează un flux pe aceasta printr-unul negativ in sens opus.

Variații ale problemei clasice

In rețelele clasice studiate pana acum aveam o sursa unica care putea produce oricât, destinația unica consuma oricât, orice nod intermediar conserva fluxul, iar singura constrângere a muchiilor era limitarea superioara a fluxului prin capacitate. Sa vedem cum putem generaliza aceste condiții si daca rețelele obținute ar putea fi reduse la una clasica.

Surse si destinații multiple



Se observă ca putem reduce acest graf la cel cunoscut prin adăugarea unei meta-surse legată de sursele mici prin muchii de capacitate nelimitata si analog un meta-terminal cu aceleași proprietăți. Global comportamentul rețelei de flux nu se va schimba.

Cuplaj bipartit maxim

Fiind dat un graf neorientat `G = (V,E)`, un cuplaj este o submultime de muchii `M` inclusa in `E` astfel incat pentru toate varfurile `v ∈ V`, exista cel mult o muchie in `M` incidenta in `v`. Spunem ca un varf `v ∈ M` este cuplat de cuplajul `M` daca exista o muchie in `M` incidenta in `v`. Un cuplaj maxim este un cuplaj de cardinalitate maxima. In cazul grafurilor bipartite, multimea de varfuri poate fi partitionata in `V = L U R`, unde `L` si `R` sunt disjuncte si toate muchiile din `E` sunt intre `L` si `R`. Problema poate fi rezolvata cu ajutorul notiunii de flux, construind reteaua de transport `G' = (V',E')` pentru graful bipartit `G`. Vom alege sursa *s* si destinatia *t* ca fiind noi varfuri care nu sunt in `V` si vom construi `V' = V U {s,t}`. Arcele orientate ale lui `G'` sunt date de:

- `E' = { (s, u) : u ∈ L } U { (u, v) : u ∈ L, v ∈ R si (u, v) ∈ E} U { (v, t) : v ∈ R}`

Pentru a completa constructia, vom atribui fiecarei muchii din `E'` capacitatea unitate.

Rețea cu noduri ce nu conservă fluxul

Spre deosebire de *s* si *t* care produc/consumă oricât, un nod intermediar ar putea produce sau consuma o cantitate constanta de flux la trecerea prin el. In acest caz vom avea un invariant la nivel de nod care ia forma:

- $f_{in} - f_{out} = di$, unde `di` este cantitatea produsă suplimentar (`> 0`) sau solicitată (`< 0`) de un nod.

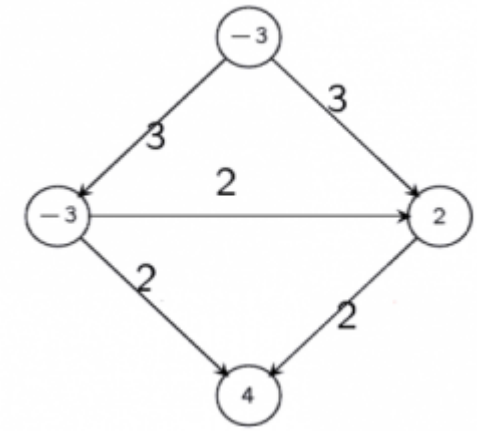
Am putea transforma egalitatea in:

- $(f_{in} + |di|) - f_{out} = 0$, dacă `di < 0`
- $f_{in} - (f_{out} + |di|) = 0$, daca `di > 0`

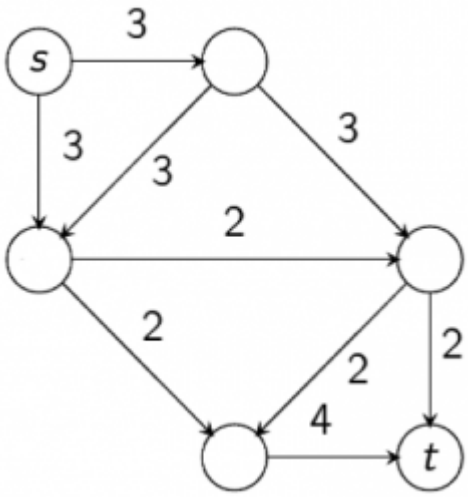
Altfel spus, un nod ce consumă flux poate fi transformat intr-unul ce conservă fluxul și are un in-arc adițional de capacitate `|di|`, iar unul ce produce flux va avea un out-arc de aceeași capacitate.

La nivelul unei rețele întregi se adaugă un nod sursă cu muchii către toate nodurile ce consumau flux si un nod destinație dinspre toate nodurile ce produceau flux.

Exemplu:



se va transforma in:

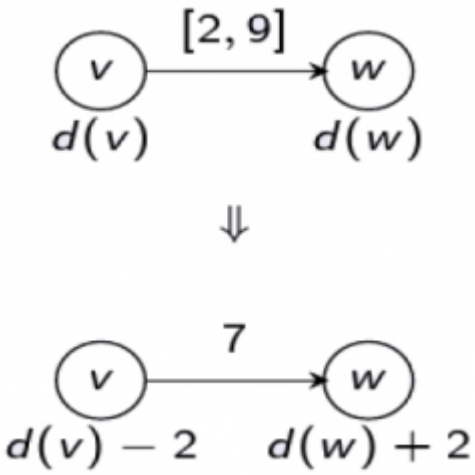


Rețea cu limite inferioare de capacitate

Există cazuri în care aș vrea ca datele de pe muchiile rețelei sa facă parte dintr-un anumit interval `[inf, sup]`. Pe o astfel de muchie fluxul trebuie să respecte inegalitatea `inf ≤ f ≤ sup`

Plecând tot de la condițiile de conservare la nivel de nod putem translata intervalul `[inf, sup]` în `[0, sup-inf]` și să considerăm că nodul sursă a consumat `inf` unități de flux iar nodul destinație a produs `inf` unități. Din exterior entitatea alcătuită din doua noduri și o muchie este văzută ca acționând în același fel asupra fluxului ce o traversează.

Iată un exemplu de transformare:



Bineînțeles că toate aceste transformări pot fi combinate pentru a se ajunge la rețeaua de flux clasică.

Concluzii și observații

Laboratorul de față s-a vrut a fi doar o introducere în domeniul fluxurilor într-un graf – modalitate de a reprezenta probleme de circulație a materialelor atât de frecvent întâlnite. In [1] și [2] găsiți și alți algoritmi interesanți împreună cu studiul complexității lor (algoritmul de pompare preflux, algoritmul „mutare-in-fata"). Spre exemplu, cel mai bun algoritm în prezent pentru cuplajul bipartit maxim se executa in $O(\sqrt{V}*E)$.

Referinte

[1] Introducere in algoritmi, Thomas H. Cormen, Charles E. Leiserson, Ronald R. Rivest – Capitolul VI Algoritmi pe grafuri: Flux maxim

[2] Introducere in analiza algoritmilor, Cristian A. Giumale – Cap. V Algoritmi pr grafuri: Fluxuri maxime intr-un graf

[3] Un articol de la MIT: A Labeling Algorithm for the maximum flow network problem [http://web.mit.edu/15.053/www/AMP-Appendix-C.pdf]

[4] Resurse wiki – Edmonds Karp [http://en.wikipedia.org/wiki/Edmonds%E2%80%93Karp_algorithm] Max Flow - Min Cut Theorem [http://en.wikipedia.org/wiki/Max-flow_min-cut_theorem]

[5] Aplicatii flux maxim [http://www.cs.princeton.edu/~wayne/cs423/lectures/max-flow-applications-4up.pdf]

Exercitii

In acest laborator vom folosi scheletul de laborator din arhiva skel-lab11.zip.

Inainte de a rezolva exercitiile, asigurati-va ca ati citit si inteles toate precizarile din sectiunea Precizari laboratoare 07-12 [https://ocw.cs.pub.ro/courses/pa/skel_graph].

Prin citirea acestor precizari va asigurati ca:

- cunoasteti **conventiile** folosite
- evitati **buguri**
- evitati **depunctari** la lab/teme/test

Flux maxim

Se da un graf **orientat** cu **n** noduri si **m** arce. Graful are pe arce **capacitati pozitive**.

Folositi **Edmonds-Karp** pentru a gasi **fluxul maxim** intre nodul sursa **1** si nodul destinatie **n** in graful dat.

Restrictii si precizari:

- $n <= 1.000$
- $m <= 5.000$
- $1 <= c <= 110.000$, unde c este capacitatea unui arc
- Nu exista arc de la **n** la **1**.
- timp de executie
    - C++: 1s
    - Java: 2s

Rezultatul se va returna sub forma unui singur numar **F**, reprezentand fluxul maxim ce poate fi trimis prin retea.

BONUS

Rezolvati problema harta [https://infoarena.ro/problema/harta] pe infoarena.

pa/laboratoare/laborator-11.txt · Last modified: 2020/05/30 19:30 by darius.neatu

Laborator 12 : Algoritmi euristici de explorare a grafurilor. A*

Responsabili:

- Darius Neațu [mailto:neatudarius@gmail.com]
- Ștefan Popa [mailto:stefanpopa2209@gmail.com]

Obiective laborator

In cadrul acestui laborator se va discuta despre modelarea problemelor sub forma grafurilor de stari si despre algoritmi specializati in gasirea solutiilor pentru acest tip de grafuri. De asemenea, se vor discuta modalitatile care pot fi folosite in analiza complexitatii si pe baza acestor metode se vor prezenta avantajele si limitarile acestei clase de algoritmi.

Importanţă – aplicaţii practice

Algoritmii de cautare euristica sunt folositi in cazurile care implica gasirea unor solutii pentru probleme pentru care fie nu exista un model matematic de rezolvare directa, fie acest model este prea complicat pentru a fi implementat. In acest caz e necesara o explorare a spatiului starilor problemei pentru gasirea unui raspuns. Intrucat o mare parte dintre problemele din viata reala pornesc de la aceste premise, gama de aplicatii a algoritmilor euristici este destul de larga. Proiectarea agentilor inteligenti [1], probleme de planificare, proiectare circuitelor VLSI [3], robotica, cautare web, algoritmi de aproximare pentru probleme NP-Complete [10], teoria jocurilor sunt doar cateva dintre domeniile in care cautarea informata este utilizata.

Descrierea problemei şi a rezolvărilor

Prezentare generală a problemei

Primul pas in rezolvarea unei probleme folosind algoritmi euristici de explorare este definirea exacta a problemei, prin tuplul (Si, O, Sf) – stare initiala, operatori, stari finale. Incercam sa cunoastem urmatorii parametri:

- **Starea initiala a problemei** – reprezinta configuratia de plecare.
- **Functie de expandare a nodurilor** – in cazul general este o lista de perechi (actiune, stare_rezultat). Astfel, pentru fiecare stare se enumera toate actiunile posibile precum si starea care va rezulta in urma aplicarii respectivei actiuni.
- **Predicat pentru starea finala** – functie care intoarce adevarat daca o stare este stare scop si fals altfel
- **Functie de cost** – atribuie o valoare numerica fiecarei cai generate in procesul de explorare. De obicei se foloseste o functie de cost pentru fiecare actiune/tranzitie, atribuind, astfel, o valoare fiecarui arc din graful starilor.

In functie de reprezentarea problemei, sarcina algoritmilor de cautare este de a gasi o cale din starea initiala intr-o stare scop. Daca algoritmul gaseste o solutie atunci cand multimea solutiilor este nevida spunem ca algoritmul este complet. Daca algoritmul gaseste si calea de cost minim catre starea finala spunem ca algoritmul este optim.

In principiu, orice algoritm pe grafuri discutat in laboratoarele si cursurile anterioare poate fi utilizat pentru gasirea solutiei unei probleme astfel definite. In practica, insa, multi dintre acesti algoritmi nu sunt utilizati

in acest context, fie pentru ca exploreaza mult prea multe noduri, fie pentru ca nu garanteaza o solutie pentru grafuri definite implicit (prin stare initiala si functie de expandare).

Algoritmii euristici de cautare sunt algoritmi care lucreaza pe grafuri definite ca mai sus si care folosesc o informatie suplimentara, necontinuta in definirea problemei, prin care se accelereaza procesul de gasirea a unei solutii. In cadrul explorarii starilor fiecare algoritm genereaza un arbore, care in radacina va contine starea initiala. Fiecare nod al arborelui va contine urmatorele informatii:

1. **Starea continuta** - stare(nod)
2. **Parintele nodului** – ɲ(nod)
3. **Cost cale** – costul drumului de la starea initiala pana la nod – g(nod)

De asemenea, pentru fiecare nod definim si o functie de evaluare **f** care indica cat de promitator este un nod in perspectiva gasirii unui drum catre solutie. (De obicei, cu cat **f** este mai mic, cu atat nodul este mai promitator). Am mentionat mai sus ca algoritmii de cautare euristica utilizeaza o informatie suplimentara referitoare la gasirea solutiei problemei. Aceasta informatie este reprezentata de o functie **h**, unde h(nod) reprezinta drumul estimat de la nod la cea mai apropiata stare solutie. Functia h poate fi definita in orice mod, existand o singura constrangere:

```
h(n)=0 pt ∀n,solutie(n)=adevarat
```

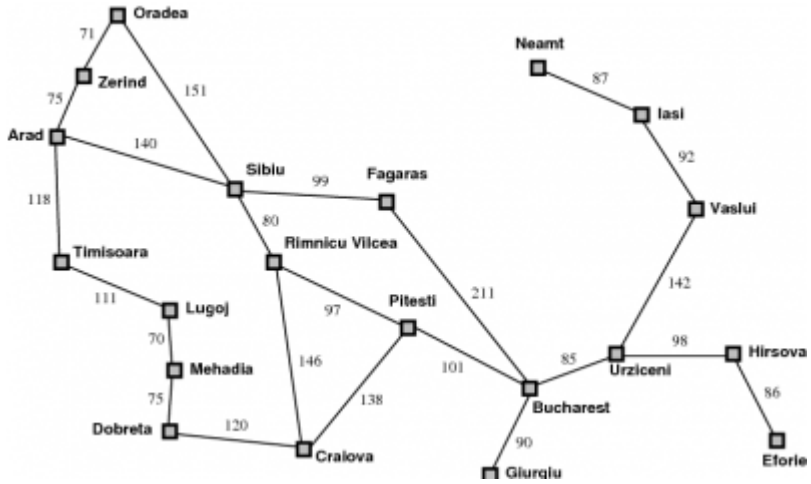
Algoritmi de cautare informata

Intrucat functioneaza prin alegerea, la fiecare pas, a nodului pentru care f(nod) este minim, algoritmii prezentati mai jos fac parte din clasa algoritmilor de cautare informata (nodul cel mai promitator este explorat mereu primul). Best-First tine cont doar de istoric (informatii sigure), pe cand A* estimeaza costul pana la gasirea unei solutii. De notat ca BFS si DFS (desi sunt algoritmi de cautare neinformata) sunt particularizari ale Best-First:

- pentru BFS: f = adancime (S)
- pentru DFS: f = - adancime (S)

Vom prezenta in continuare cativa dintre cei mai importanti algoritmi de cautare euristica. Vom folosi pentru exemplificare, urmatoarea problema: data fiind o harta rutiera a Romaniei, sa se gaseasca o cale (de preferinta de cost minim) intre Arad si Bucuresti.

Pentru aceasta problema starea initiala indica faptul ca ne aflam in orasul Arad, starea finala este data de predicatul Oras_curent == Bucuresti, functia de expandare intoarce toate orasele in care putem ajunge dintr-un oras dat, iar functia de cost indica numarul de km al fiecarui drum intre doua orase, presupunand ca viteza de deplasare este constanta. Ca euristica vom utiliza pentru fiecare oras distanta geometrica(in linie dreapta) pana la Bucuresti.



Greedy Best-First

In cazul acestui algoritm se considera ca nodul care merita sa fie expandat in pasul urmator este cel mai apropiat de solutie. Deci, in acest caz, avem:

f(n) = h(n), pt. oricare n

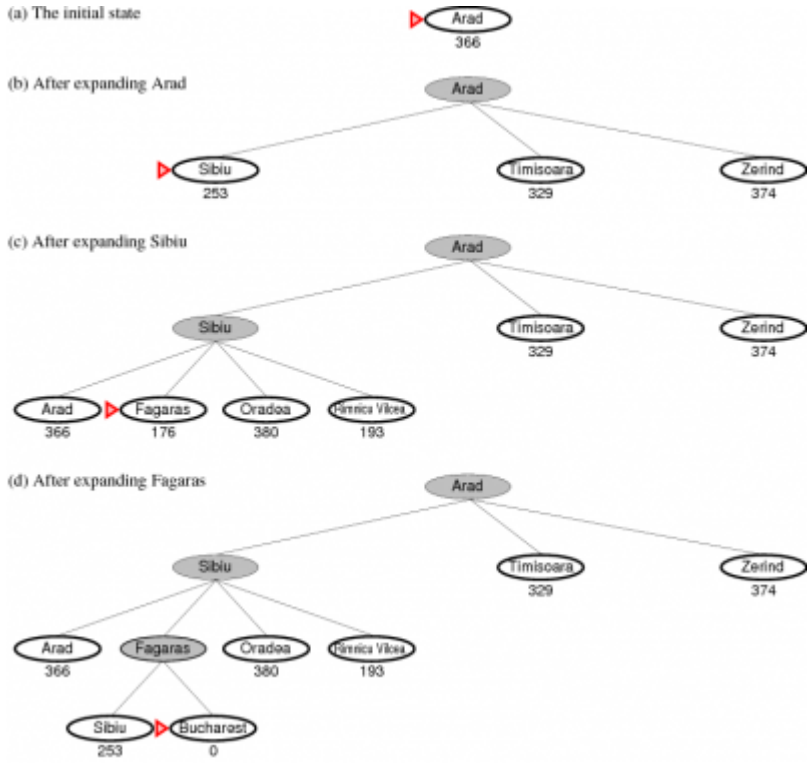
Pseudocodul acestui algoritm:

```
Greedy Best-First(sinitial , expand, h, solution)
    closed ← {}
    n ← new-node()
    state(n) ← sinitial
    π(n) ← nil
    open ← { n }
    // Bucla principala
    repeat
        if open = ∅ then return failure
        n ← get_best(open) with f(n) = h(n) = min
        open ← open - {n}
        if solution(state(n)) then return build-path(n)
        else if n not in closed then
            closed ← closed U {n}
            for each s in expand(n)
                n' ← new-node()
                state(n') ← s
                π(n') ← n
                open ← open U {n'}
            end-for
    end-repeat
```

In cadrul algoritmului se folosesc doua multimi: closed – indica nodurile deja explorate si expandate si open – nodurile descoperite dar neexpandate. Open este intializata cu nodul corespunzator starii intiale. La fiecare pas al algoritmului este ales din open nodul cu valoarea f(n) = h(n) cea mai mica (din acest motiv e de preferat ca open sa fie implementata ca o coada cu prioritati). Daca nodul se dovedeste a fi o solutie a problemei atunci este intoarsa ca rezultat calea de la starea initiala pana la nod (mergand recursiv din parinte in parinte). Daca nodul nu a fost deja explorat atunci se expandeaza iar nodurile corespunzatoare starilor rezultate sunt introduse in multimea open. Daca multimea open ramane fara elemente atunci nu exista niciun drum catre solutie si algoritmul intoarce esec.

Greedy Best-First urmareste mereu solutia care pare cea mai aproape de destinatie. Din acest motiv nu se vor analiza stari care desi par mai departate de solutie produc o cale catre solutie mai scurta (vezi exemplul de rulare). De asemenea, intrucat nodurile din closed nu sunt niciodata reexplorate se va gasi calea cea mai scurta catre scop doar daca se intampla ca aceasta cale sa fie analizata inaintea altor cai catre aceiasi stare scop. Din acest motiv, algoritmul nu este optim. De asemenea, pentru grafuri infinite e posibil ca algoritmul sa ruleze la infinit chiar daca exista o solutie. Rezulta ca algoritmul nu indeplineste nici conditia de completitudine.

In figura de mai jos se prezinta rularea algoritmului Greedy Best-First pe exemplul dat mai sus. In primul pas algoritmul expandeaza nodul Arad, iar ca nod urmator de explorat se alege Sibiu, intrucat are valoarea h(n) minima. Se alege in contiuare Fagaras dupa care urmeaza Bucuresti, care este un nod final. Se observa insa ca acest drum nu este minimal. Desi Fagaras este mai aproape ca distanta geometrica de Bucuresti, in momentul in care starea curenta este Sibiu alegerea optimala este Ramnicu-Valcea. In continuare ar fi urmat Pitesti si apoi Bucuresti obtinandu-se un drum cu 32 km mai scurt.



A* (A star)

A* reprezinta cel mai cunoscut algoritm de cautare euristica. El foloseste, de asemenea o politica Best-First, insa nu sufera de aceleasi defecte pe care le are Greedy Best-First definit mai sus. Acest lucru este realizat prin definirea functiei de evaluare astfel:

```
f(n) = g(n) + h(n) , pt. oricare n ∈ Noduri
```

A* evalueaza nodurile combinand distanta deja parcursa pana la nod cu distanta estimata pana la cea mai apropiata stare scop. Cu alte cuvinte, pentru un nod n oarecare, **f(n) reprezinta costul estimat al celei mai bune solutii care trece prin n**. Aceasta strategie se dovedeste a fi completa si optimala daca euristica h(n) este admisibila:

```
0 ≤ h(n) ≤ h*(n) , pt. oricare n ∈ Noduri
```

unde h*(n) este distanta exacta de la nodul n la cea mai apropiata solutie. Cu alte cuvinte A* intoarce mereu solutia optima daca o solutie exista atat timp cat ramanem optimisti si nu supraestimam distanta pana la solutie. Daca h(n) nu este admisibila o solutie va fi in continuare gasita, dar nu se garanteaza optimalitatea. De asemenea, pentru a ne asigura ca vom gasi drumul optim catre o solutie chiar daca acest drum nu este analizat primul, A* permite scoaterea nodurilor din closed si reintroducerea lor in open daca o cale mai buna pentru un nod din closed (g(n) mai mic) a fost gasita.

Algoritmul evolueaza in felul urmator: initial se introduce in multimea open (organizata ca o coada de prioritati dupa f(n)) nodul corespunzator starii initiale. **La fiecare pas se extrage din open nodul cu f(n) minim**. Daca se dovedeste ca nodul n contine chiar o stare scop atunci se intoarce calea de la starea initiala pana la nodul n. Altfel, daca nodul nu a fost explorat deja se expandeaza. Pentru fiecare stare rezultata, daca nu a fost generata de alt nod inca (nu este nici in open nici in closed) atunci se introduce in open. Daca exista un nod corespunzator starii generate in open sau closed se verifica daca nu cumva nodul curent produce o cale mai scurta catre s. Daca acest lucru se intampla se seteaza nodul curent ca parinte al nodului starii s si **se corecteaza distanta g**. Aceasta corectare implica reevaluarea tuturor cailor care trec prin nodul lui s, deci acest nod va trebui reintrodus in open in cazul in care era inclus in closed.

Pseudocodul pentru A* este prezentat in continuare:

```
// s_i      = stare initiala
// expand   = functie de expandare care returneaza toti succesorii unei stari
// h        = functie euristica (h(n) = aproximarea drumului minim de la n la o destinatie)
// is_goal  = is_goal(n) returneaza true daca n este o destinatie (stare goal/finala)
// w        = w[x][y] reprezinta costul muchiei directe intre starile x si y
A*(s_i, expand, h, is_goal)
        //initializari
    closed ← {}     // multimea nodurilor expandate
    open   ← { }    // multimea nodurilor in curs de explorare

    // pentru fiecare stare s
    for each s
        g(s) ← Infinity // initializare distante
        π(s) ← Infinity // initializare parinti

    // pentru s_i avem distanta 0 si parinte NULL
    g(s_i) ← 0
    π(s_i) ← NULL
    open   ← open U { s_i }

    // parcurgerea continua cat timp avem noduri in open
    while !open.empty()
        // extrage nodul s cu f(s) minim
        s ← get_best(open) with minimum f(s) = g(s) + h(s)
        open ← open - { s }

        // daca s este stare finala
        if is_goal(s)
            // reconstituie drumul de la s la s_i
            return build_path(s, s_i)

        // daca nodul nu este deja expandat
        if s not in closed
            // il adaug in closed
            closed ← closed U { s }

            // pentru fiecare copil c al starii s
            for each c in expand(s)
                if g(s) + w[s][c] < g(c)
                    // actualizez distanta si parintele
                    g(c) ← g(s) + w[s][c]
                    π(c) ← s

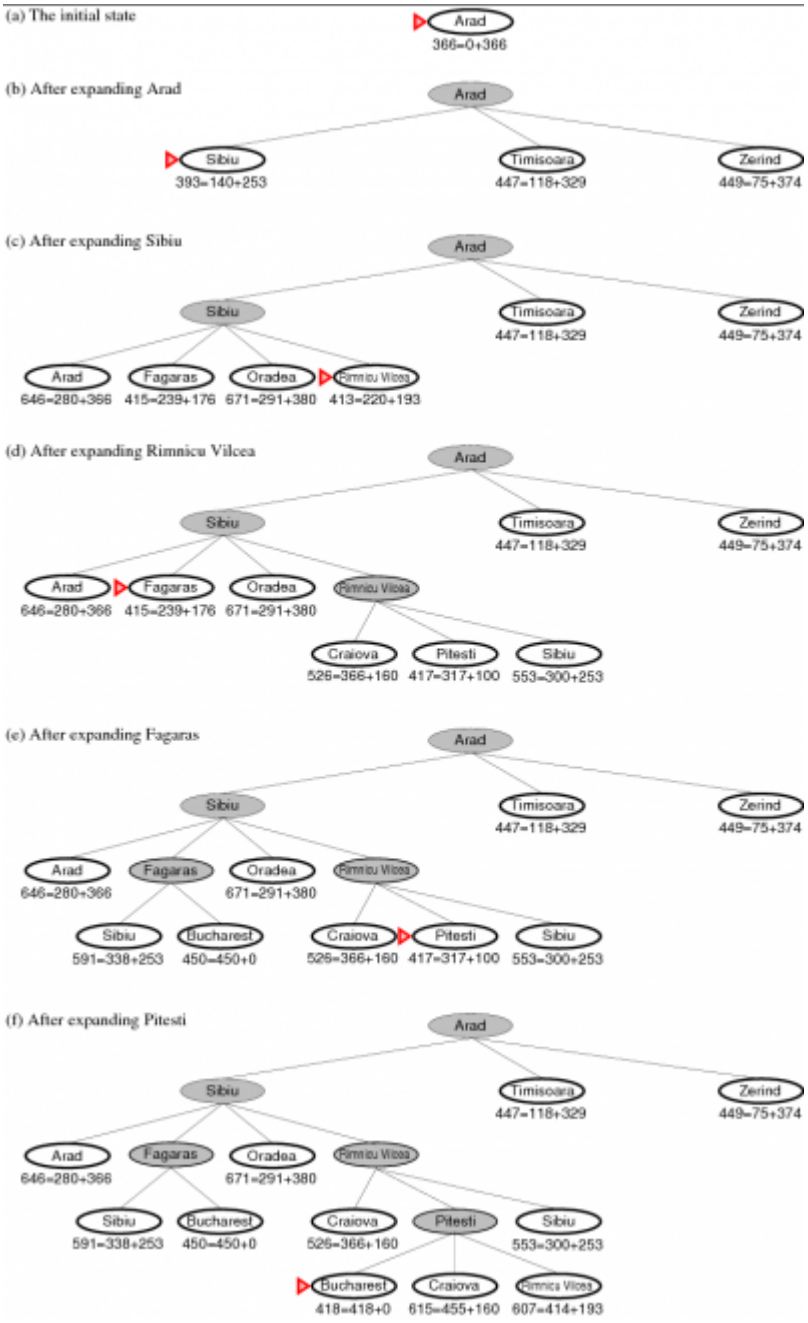
                    if not (c in closed U open) // daca nodul c nu a mai fost intalnit
                        open   ← open U { c }   // c intra in open
                    else if c in closed         // daca nodul c a mai fost expandat
                        closed ← closed – { c } // c trece din closed in open
                        open   ← open U { c }

    // nu a fost gasita o cale de la s_i la o stare goal s
    return failure
```

Algoritmul prezentat mai sus va intoarce calea optima catre solutie, daca o solutie exista. Singurul incovenient fata de Greedy Best-First este ca sunt necesare reevaluarile nodurilor din closed. Si aceasta problema poate fi rezolvata daca impunem o conditie mai tare asupra euristicii h, si anume ca euristica sa fie **consistenta** (sau **monotona**):

```
h(n) ≤ h(n') + cost(n,n') , pt. oricare n' ∈ expand(n)
```

Daca o functie este consistenta atunci ea este si admisibila. Daca euristica h indeplineste si aceasta conditie atunci algoritmul A* este asemanator cu Greedy Best-First cu modificarea ca functia de evaluare este f = g + h, in loc de f = h. In imaginea de mai jos se prezinta rularea algoritmului pe exemplul laboratorului. Se observa ca euristica aleasa (distanta in linie dreapta) este consistenta si deci admisibila. Se observa ca in pasul (e), dupa expandarea nodului Fagaras desi exista o solutie in multimea open aceasta nu este aleasa pentru explorare. Se va alege Pitesti, intrucat f(nod(Bucuresti)) = 450 > f(nod(Pitesti)) = 417, semnificatia acestei inegalitati fiind ca e posibil sa existe prin Pitesti un drum mai bun catre Bucuresti decat cel descoperit pana acum.



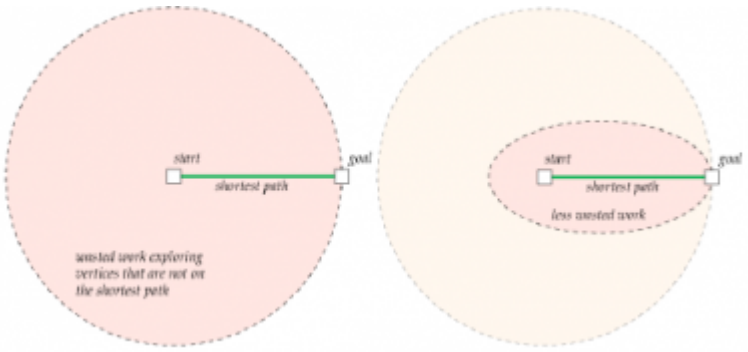
Complexitate algoritmului A*

Pe langa proprietatile de completitudine si optimalitate A* mai are o calitate care il face atragator: pentru o euristica data orice algoritm de cautare complet si optim va explora cel putin la fel de multe noduri ca A*, ceea ce inseamna ca A* este optimal din punctul de vedere al eficientei. In practica, insa, A* poate fi de multe ori imposibil de rulat datorita dimensiunii prea mari a spatiului de cautare. Singura modalitate prin care spatiul de cautare poate fi redus este prin gasirea unei euristici foarte bune – cu cat euristica este mai apropiata de distanta reala fata de stare solutie cu atat spatiul de cautare este mai strans (vezi figura de mai jos). S-a demonstrat ca spatiul de cautare incepe sa creasca exponential daca eroarea euristicii fata de distanta reala pana la solutie nu are o crestere subexponentiala:

```
|h(n) - h*(n)| ≤ O(logh*(n))
```

Din pacate, in majoritatea cazurilor, eroarea creste liniar cu distanta pana la solutie ceea ce face ca A* sa devina un algoritm mare consumator de timp, dar mai ales de memorie. Intrucat in procesul de cautare se retin toate nodurile deja explorate (multimea closed), in cazul unei dimensiuni mari a spatiului de cautare cantitatea de memorie alocata cautarii este in cele din urma epuizata. Pentru acest incovenient au fost

gasite mai multe solutii. Una dintre acestea este utilizarea unor euristici care sunt mai stranse de distanta reala pana la starea scop, desi nu sunt admisibile. Se obtin solutii mai rapid, dar nu se mai garanteaza optimalitatea acestora. Folosim aceasta metoda cand ne intereseaza mai mult sa gasim o solutie repede, indiferent de optimalitatea ei.



Volumul spatiului de cautare in functie de euristica aleasa

Alte abordari presupun sacrificarea timpului de executie pentru a margini spatiul de memorie utilizat [3]. Aceste alternative sunt prezentate in continuare.

IDA*, RBFS, MA*

IDA*

Iterative deepening A* [5] utilizeaza conceptul de adancire iterativa[1][8] in procesul de explorare a nodurilor – la un anumit pas se vor explora doar noduri pentru care functia de evaluare are o valoare mai mica decat o limita data, limita care este incrementata treptat pana se gaseste o solutie. IDA* nu mai necesita utilizarea unor multimi pentru retinerea nodurilor explorate si se comporta bine in cazul in care toate actiunile au acelasi cost. Din pacate, devine ineficient in momentul in care costurile sunt variabile.

RBFS

Recursive Best-First Search [6] functioneaza intr-un mod asemanator cu DFS, explorand la fiecare pas nodul cel mai promitator fara a retine informatii despre nodurile deja explorate. Spre deosebire de DFS, **se retine in orice moment cea mai buna alternativa sub forma unui pointer catre un nod neexplorat**. Daca valoarea functiei de evaluare (f = g + h) pentru nodul curent devine mai mare decat valoarea caii alternative, drumul curent este abandonat si se incepe explorarea nodului retinut ca alternativa. RBFS are avantajul ca spatiul de memorie creste doar liniar in raport cu lungimea caii analizate. Marele dezavantaj este ca se ajunge de cele mai multe ori la reexpandari si reexplorari repetate ale acelorasi noduri, lucru care poate fi dezavantajos mai ales daca exista multe cai prin care se ajunge la aceiasi stare sau functia expand este costisitoare computational (vezi problema deplasarii unui robot intr-un mediu real). RBFS este optimal daca euristica folosita este admisibila.

MA*

Memory-bounded A* este o varianta a A* in care se limiteaza cantitatea de memorie folosita pentru retinerea nodurilor. Exista doua versiuni – MA* [7]si SMA* [4] (Simple Memory-Bounded A*), ambele bazandu-se pe acelasi principiu. SMA* ruleaza similar cu A* pana in momentul in care cantitatea de memorie devine insuficienta. In acest moment spatiul de memorie necesar adaugarii unui nod nou este obtinut prin stergerea celui mai putin promitator nod deja explorat. In cazul in care exista o egalitate in privinta valorii functiei de evaluare se sterge nodul cel mai vechi. Pentru a evita posibilitatea in care nodul sters este totusi un nod care conduce la o cale optimala catre solutie valoarea f a nodului sters este retinuta la nodul parinte intr-un mod asemanator felului in care in RBFS se retine cea mai buna alternativa la nodul curent. Si in acest caz vor exista reexplorari de noduri, dar acest lucru se va intampla doar cand toate caile mai promitatoare vor esua. Desi exista probleme in care aceste regenerari de noduri au o frecventa care face ca algoritmul sa devina intractabil, MA* si SMA* asigura un compromis bun intre timpul de executie si limitarile de memorie.

Concluzii și observații

Deseori singura modalitate de rezolvare a problemelor dificile este de e explora graful starilor acelei probleme. Algoritmii clasici pe grafuri nu sunt intotdeauna potriviti pentru cautarea in aceste grafuri fie pentru ca nu garanteaza un rezultat fie pentru ca sunt ineficienti. Algoritmii euristici sunt algoritmi care exploreaza astfel de grafuri folosindu-se de o informatie suplimentara despre modul in care se poate ajunge la o stare scop mai rapid. A* este, teoretic, cel mai eficient algoritm de explorare euristica. In practica, insa, pentru probleme foarte dificile, A* implica un consum prea mare de memorie. In acest caz se folosesc variante de algoritmi care incearca sa minimizeze consumul de memorie in defavoarea timpului de executie.

Referinte

[1] S. Russel, P. Norvig - Artificial Intelligence: A Modern Approach - Prentice Hall, 2nd Edition – cap. 4

[2] C.A Giumale – Introducere in Analiza Algoritmilor, cap. 7

[3] Mehul Shah - Algorithms in the real world (Course Notes) - Introduction to VLSI routing [http://www.cs.cmu.edu/afs/cs/project/pscico-guyb/294/class-notes/all/18.ps]

[4] S. Russel - Efficient memory-bounded search methods [http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.54.6489&rep=rep1&type=pdf]

[5] R. Korf - Depth-first iterative-deepening: an optimal admissible tree search

[6] Recursive Best-First Search - Prezentare [http://www.ailab.si/gregorl/Students/mui/Recursive%20Best-First%20Search.ppt]

[7] P. Chakrabati, S. Ghosh, A. Acharya, S. DeSarkar – Heuristic search in restricted memory

[8] Wikipedia - Iterative deepening [http://en.wikipedia.org/wiki/Iterative_deepening_depth-first_search]

[9] A. E. Prieditis - Machine Discovery of Effective Admissible Heuristics [http://dli.iiit.ac.in/ijcai/IJCAI-91-VOL2/PDF/017.pdf]

[10] Wikipedia - Travelling salesman problem - Heuristic and approximation algorithms [http://en.wikipedia.org/wiki/Travelling_salesman_problem#Heuristic_and_approximation_algorithms]

[11] Wikipedia – A* [http://en.wikipedia.org/wiki/A*_search_algorithm]

[12] Tutorial A* [http://www.policyalmanac.org/games/aStarTutorial.htm]

[13] H. Kaindl, A. Khorsand - Memory Bounded Bidirectional Search, AAAI-94 Proceedings [http://aaaipress.org/Papers/AAAI/1994/AAAI94-209.pdf]

Probleme

În acest laborator vom rezolva o singură problemă: PACMAN [https://www.hackerrank.com/challenges/pacman-astar/problem] pe hackerrank. Puteți porni implementarea pe hackerrank de la următorul schelet: skel-lab12.zip.

Bonus

Problema misionarilor şi a canibalilor

Trei misionari şi trei canibali ajung la malul estic al unui râu. Aici se află o barcă cu două locuri cu care se poate traversa râul (râul nu se poate traversa înot). Dacă pe unul dintre maluri numărul de canibali este

mai mare decât numărul de misionari, atunci misionarii de pe acel mal vor fi mâncaţi de canibali. Se cere sa determinaţi cum pot trece toţi râul fără ca misionarii să fie mâncaţi de canibali.

Să se rezolve această problemă folosind algoritmul A* cu:

a) O euristica care nu este admisibilă

h(S) = nE(S)

b) Doua euristici admisibile

h1(S) = nE(S)/2

h2(S) =

* nE(S) + 1 daca barca pe malul vestic si nE(S) diferit de 0

* nE(S) - 1 daca barca pe malul estic si nE(S) diferit de 0

* 0 altfel

nE - numar oameni est

Comparati numarul de pasi necesar pentru a ajunge la solutie in functie de euristica

pa/laboratoare/laborator-12.txt · Last modified: 2020/05/31 20:27 by darius.neatu

Puteți propune probleme / rezolvări în modul "suggest" (apare cu verde). DN sau Radu Nichita dau ulterior approve dacă informația este corectă.

1. Cum arata graful pt problema cu fete si baieti, ce tip de graf e si cum se cuplajul

E un graf bipartit, care are o multime de noduri pentru 7, o multime de noduri pentru baieti, fetele in stanga, baietii dreapta. Exista muchie intre fata x si baiatul y daca 7. Y! JJj? , facem o retea de flux pe baza acestui graf bipartit, legand toate nodurile din stanga de o sursa, toate nodurile din dreapta de o destinatie, iar absolut toate.m. 76 h, hmmmuchiile din graf trebuie sa aiba capacitate 1

2. Cum ati folosi schema minimax pentru un joc de sah?

Aici ar merge eventual spus ca faci arborele starilor pana la o adancime anume, ca la un minimax obisnuit si ca dupa nu stiu cate mutari bagi o euristica pentru evaluarea starilor, idk. B 55

3. Enumerati utilizarile folosite in algoritmii de la curs ale timpilor de debut.. si de finish de la DFS

Apar pe la sortare topologica, la Kosaraju (CTC in general) si pe la componente biconexe (Tarjan).

4. Las Vegas vs Monte Carlo

Alea din lab

I a. Las Vegas are timp nedeterminat de rulare, Monte Carlo timp determinat (ex. il lasam sa ruleze X iteratii)

b. Las Vegas cand se opreste garanteaza solutie corecta/optima. La Monte Carlo cu cat il lasi sa ruleze mai mult cu atat solutia este mai aproape de cea corecta / cea optima.

5. ! I i. U.utilitatea algoritmului lui Johnson. De ce este mai bun decat alternativele.

Problema: drum de la n surse la m destinatii (in pricipiu la grafuri rare e mai bun comparativ cu Roy).

Solutii:

1. N x Dijkstra - O(M * N log N), dar cu costuri pozitive
2. g de la un nod fictiv - h[i] = distanta nod fictiv i
    a. La Johnson e ideea sa faci costurile pozitive ca sa poti aplica 1
    b. w[i][j] => w[i][j] + h[i] - h[j]
    c. Bagi 2 => d[i][j] = distanta calculata cu Dijkstra pe noul graf cu costuri pozitive
    d. d[i][j] => d[i][j] + h[j] - h[i] = distanta pe graful initial

13. Cum modifici Dijkstra daca graful este aciclic si orientat.

Cand graful este orientat si aciclic, putem sa sortam topologic graful si sa calculam acelasi vector de distante ca la Dijkstra parcurgand nodurile din sortarea topologica de la

stanga la dreapta si updatand d[vecin] = min(d[vecin], d[nod] + cost(nod, vecin))
d[sursa] = 0 initial, oo celelalte

14. Ce inseamna o euristica admisibila la A*? ca => 0 <= h(n) <= h*(n) oricare ar fi n
Ce s-ar intampla daca nu e admisibila? => A* ar gasi o solutie dar nu se garanteaza optimalitatea.

15. Evidentiati caracteristici DEI. Exemplificati pe x^n.

x ^ n = x ^ (n / 2) * x ^ (n / 2) daca n e par
x ^ n = x * x ^ (n - 1) daca n e impar
Complexitate: O(log n).
Caracteristicile DEI sunt mai sus, la comparatia cu DP

16. BF* vs A* (asta e foooarte prost explicata in curs, ma mai gandesc)

La BF*, g = 0, algoritm neoptim si complet, posibil sa ruleze la infinit pe grafuri infinite.
La A*, g = drumul de la sursa (ca la Dijkstra), algoritm complet si optim (depinde de euristica aleasa) .

17. Dati exemplu de un graf cu n noduri si m > n arce si o parcurgere in adancime care genereaza n arbori de parcurgere a grafului

Se poate desena o sortare topologica, iar dfs-ul se face de la dreapta la stanga. Se genereaza n arbori.

18. Cum se poate modifica numarul de CTCuri la adaugarea unei muchii? Justificati!

Numarul de CTCuri poate sa ramana la fel sau sa scada la adaugarea unei muchii, nu poate sa creasca. Numarul scade daca avem de exemplu un graf in forma de lant si muchia adaugata inchide ciclul (sau alte situatii similare), iar in celelalte situatii numarul ramane acelasi.

19. Transformand unul din algoritmii facuti la curs, scrieti in pseudocod un algoritm de detectare eficienta a existentei ciclurilor intr-un graf orientat. Justificati. Ce complexitate are?

Se poate face si cu DFS (se alege un nod random, se porneste DFS din el si se cauta cicluri asa, iar asta se face cat timp mai avem noduri nevizitate)
Data fiind exprimarea “transformand unul din algoritmii facuti la curs”, e posibil sa ceara o modificare a Bellman Ford, si anume: se pune cost -1 pe fie care muchie din graf si se porneste Bellman Ford, iar daca in urma rularii algoritmului gasim ca exista un ciclu de cost negativ, inseamna ca exista ciclu in graful nostru (automat orice ciclu din graf conduce la un ciclu de cost negativ)
Ca si complexitate, solutia cu DFS are O(N + M), iar cea cu Bellman Ford are O(N * M).

20. Cum se poate transforma o problema de determinare a fluxului maxim intr-o retea cu mai multe surse si mai multe destinatii intr-o problema de determinare a fluxului

maxim intr-o retea cu o sursa si o destinatie?

Se construieste o sursa noua, pe care o legam cu capacitate INF de toate sursele initiale,precum si o destinatie noua, pe care o legam cu capacitate INF de toate destinatiile initiale.

21. In ce cazuri complexitatea cautarii drumurilor de cost minim intr-un graf orientat G=(N,A) cu costuri pozitive este O(N + A)?

Cand graful este aciclic, deoarece avem cel mult un drum intre oricare 2 noduri si atunci costul e unic determinat.

22. Comparati A* si parcurgerea in latime.

Pe langa diferentele evidente, Trausanul s-a plans anul trecut ca nu a zis nimeni de Best F2 Best First, pe cand A* e un algoritm de cautare euristica.

23. Numarul maxim si minim de CTC-uri nebanale intr-un graf cu n noduri si a arce

Probs - minim 1

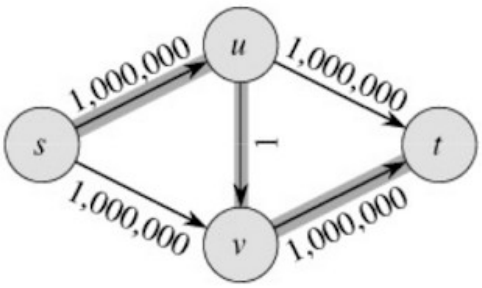
TODO - trebuie sa ma lamuresc ce ai voie sa faci si ce nu :)

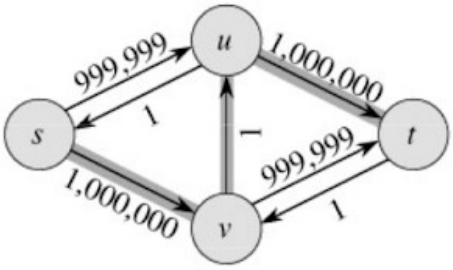
24. Pe ce caz merge Ford Fulkerson prost + exemplu

Chestia din curs cu 4 noduri si 5 muchii cu costurile de 1000 si 1.

Analiza

- Capacitati intregi – marire a |f| cu ≥ 1.
- Daca fluxul max e f*, atunci ≤ |f*| iteratii
  → O(a|f*|).
- Observati ca timpul de rulare este **nepolinomial** fata de marimea intrarii. Depinde de |f*|, care nu e o functie de |n| sau |a|.
- Daca capacitatile sunt rationale, se pot scala la intregi
- Daca sunt irationale, FORD-FULKERSON s-ar putea sa nu se termine!





Se alege drumul ala care contine 1 de fiecare data si se fac 1kk iteratii

25. ~~Greed~~y vs Divide et Impera = combinatie de DP vs DEI cu Greedy vs DP

Altele

1. **Ce este o cale reziduala si cum se poate obtine? Care este capacitatea reziduala a caii?**
   Exempificati.

Presupunem ca avem o retea de flux. Graful rezidual este un graf care contine doar muchiile nesaturate (muchie nesaturata = muchie cu capacitate != flux, adica muchie pe care se mai poate trimnuite flux). O cale reziduala este un drum care pleaca din sursa de la flux si merge pana in destinate (drena, whatever) doar pe muchii nesaturate (din graful rezidual).

Daca presupunem ca acea cale reziduala este v1 v2 … vk, unde v1 = sursa, vk = destinatia, atunci capacitatea reziduala a caii este min(capacitate[v[i + 1]][v[i]] - flux[v[i + 1]][v[i]]), i intre 1 si k - 1.

+ un exemplu random

2. **Ce tipuri de muchii se pot intalni intr-un graf neorientat?**

Fiind graf neorientat, se pot gasi intr-o parcurgere DFS doar 2 tipuri de muchii, **muchii de avans** (muchii de arbore), folosite pentru explorarea nodurilor; ele formează un graf parţial format din unul (graf conex) sau mai mulţi arbori ai parcurgerii **DF** si **muchii de intoarcere,** care unesc un nod cu un strămoş al său în arborele DF.

Pentru un graf orientat, sunt 4 tipuri de muchii:

• arce de arbore – care conduc la descoperirea de vârfuri noi
• arce de revenire – care leagă un vârf cu un strămoş în arborele DFS
• arce de înaintare – care leagă un vârf cu un descendent în arborele DFS
• arce de traversare – care leagă vârfuri din arbori DFS diferiţi (nu se încadrează în categoriile precedente).

3. **Care este numarul total de relaxari de muchii facute de catre Bellman Ford? Explicati din ce motiv algoritmul foloseste acest numar de relaxari de muchii (si nu mai multe sau mai putine)**

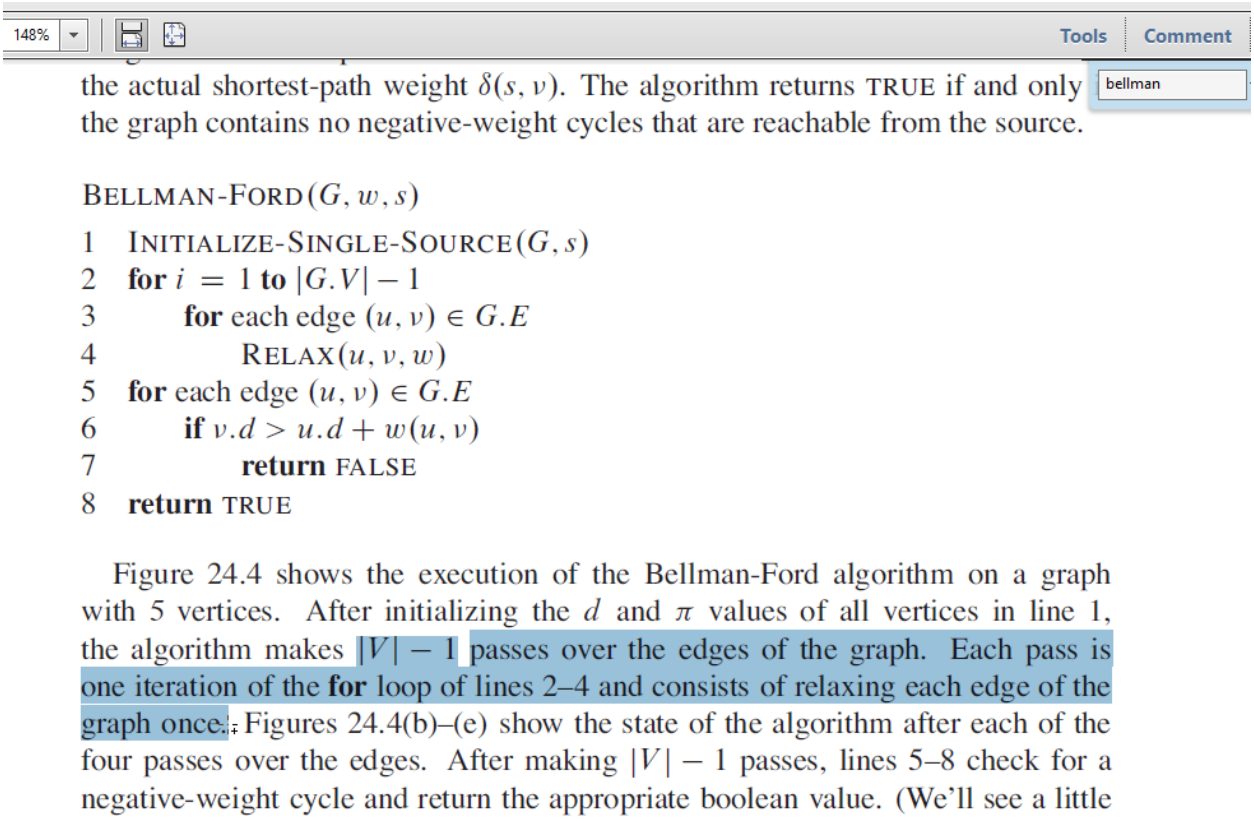
Numarul total de relaxari de muchii facute de Bellman Ford este (N - 2) * M (sau (N - 1) * M). Motivul pentru care algoritmul foloseste acest numar de relaxari de muchii este ca daca vrem sa detectam existenta unui ciclu negativ, atunci acest ciclu poate contine cel mult N noduri. Inainte sa facem Bellman Ford consideram cumva doar potentialele cicluri de lungime 2, apoi dupa k iteratii luam in calcul potentialele cicluri de lungime maxim k + 2, de aceea k trebuie sa fie N - 2. La final se mai face o iteratie prin care se incearca relaxarea muchiilor si in caz ca se gaseste una care inca mai updateaza o distanta, inseamna ca exista un ciclu negativ.

Motivul pentru care se face acest lucru este ca drumul minim dintre sursa si orice nod

destinație poate sa treacă prin maximum |V| noduri (adică toate nodurile grafului), respectiv |V|-1 muchii; prin urmare, relaxarea tuturor muchiilor de |V|-1 ori este suficienta pentru a propaga până la toate nodurile informația despre distanta minima de la sursa.

Daca, la sfârșitul acestor |E|*(|V|-1) relaxări, mai poate fi îmbunătățită o distanță, atunci graful are un ciclu de cost negativ si problema nu are soluție.

Cormen:



Deci ar trebui sa fie (V-1) * E ca sa afle distantele (dar asta ar trebui sa fie raspunsul uzual). Iar daca se cere cu tot cu verificarea ciclului V * E.

Adica cred ca sunt corecte amandoua abordarile, doar ca e vorba de o conventie: de ex pe wiki initial toate distantele sunt infinit, chiar si acolo unde are muchie. Si le numara inclusiv pe astea (cand foloseste muchia i->j sa spuna ca distanta de la i la j nu mai e infinit). Deci banuiesc ca daca le numeri si pe alea triviale sunt mai multe cu E…

P.S. pseudocodul din lab este acelasi cu cel de pe wiki, diferenta fiind fix asta (cu infinit pus degeaba in unele locuri).

4. **Cum se poate modifica alg lui Dijkstra daca graful este aciclic si orientat?**
   Cand graful este orientat si aciclic, putem sa sortam topologic graful si sa calculam

acelasi vector de distante ca la Dijkstra parcurgand nodurile din sortarea topologica de la stanga la dreapta si updatand d[vecin] = min(d[vecin], d[nod] + cost(nod, vecin))
d[sursa] = 0 initial, oo celelalte
al

5. **Daca facem +2 la costuri pe muchii / *2 la costuri, se modifica drumul minim gasit de Dijkstra?**

Daca facem *2 drumul ramane acelasi, doar i se dubleaza costul, iar daca facem +2 se modifica drumul. Spre exemplu:
1 -> 3 cu cost 4
1 -> 2 cu cost 2
2 -> 3 cu cost 1

Initial Dijkstra gaseste drumul 1 -> 2 -> 3 cu cost 3, dar dupa ce crestem costurile, costul drumului 1 -> 3 ajunge 6, pe cand 1 -> 2 -> 3 ajunge 7, ceea ce inseamna ca gasim alt drum.

6. **Ce se întâmplă cu algoritmul minimax cu tăieturi alfa-beta dacă modificăm condiția de tăiere din (β<α) în (β-k<α), unde k este o constantă oarecare. Discuție după k.**

In cazul in care:
β-k=β (k=0) => algoritmul nu se schimba;
β-k<β (k>0) => riscam sa facem taieturi inutile ce va duce la rezultate gresite in urma rularii algoritmului;
β-k>β (k<0) => riscam sa omitem taituri ce va duce la crestea timpului de rularea al algoritmului insa functionalitatea acestuia va ramane;

CA + CB (se repeta cateva de mai sus)
.
Raspuns
Raspuns dat partial
Nu sunt sigur de raspuns

Subiecte examen

1. Sa se spuna ce algoritm facut la curs poate sa determine eficient care este drumul de lungime maxima intr-un graf aciclic orientat cu lungimi pe arce si sa se schiteze idea folosirii acestuia si modificarile necesare pentru rezolvare.

Sortare topologica pentru a gasi sursa + dfs modificat din sursa care intoarce distanta maxima calculata pe parcurs

2. Sa se spuna ce schema de algoritm facuta la curs poate sa determine, cat mai eficient posibil, daca un careu de rebus are o asignare posibila de cuvinte (nu se considera definitiile cuvintelor). Sa se schiteze idea rezolvarii. Se presupune ca exista un dictionar in care cuvintele sunt reprezentate prin vectori de caractere. (Un careu de rebus este o matrice m x n cu unele celule goale). Toate celelalte celule trebuie sa primeasca cate o litera astfel incat secventele maximale de celule pe orizontala si verticala sa fie cuvinte valide).
Ceva cu constrangeri?

4. Explicati pe un exemplu simplu de ce algoritmul Dijkstra merge doar pentru costuri pozitive pe arce.(sigur nu e intrebarea gresita? Dijkstra merge pe costuri negative, atat timp cat nu avem cicluri de cost negativ - https://stackoverflow.com/a/25158544)

Recall that in Dijkstra's algorithm, **once a vertex is marked as "closed" (and out of the open set) - the algorithm found the shortest path to it**, and will never have to develop this node again - it assumes the path developed to this path is the shortest.

But with negative weights - it might not be true. For example:

`V={A,B,C} ; E = {(A,C,2), (A,B,5), (B,C,-10)}`

Dijkstra from A will first develop C, and will later fail to find `A->B->C`

`https://stackoverflow.com/questions/13159337/why-doesnt-dijkstras-algorithm-work-for-negative-weight-edge`

5. Fie un graf neorientat cu n noduri si a arce. Care este numarul minim si maxim de punti si puncte de articulatie ale acestui graf? Justificati. Dar daca graful are puncte de articulatie dar nu are punti?
I.
Nr maxim puncte de articulatie: n - 2, graful este un lant * - * - * - … - *, iar stergerea nodurilor

de la capete nu influenteaza numarul de componente conexe
Nr maxim punti:n - 1, graful este un lant * - * - * - … - *
Nr minim puncte de articulatie: 0, graful nu are muchii (sau 2-nod conex)
Nr minim punti: 0, graful nu are muchii (sau 2-arc conex)
II. graful are puncte de articulatie dar nu are punti
Nr maxim puncte de articulatie: n - 2, graful este un lant cu cate doua muchii intre un nod si vecinul lui * = * = * = … = *, iar stergerea nodurilor de la capete nu influenteaza numarul de componente conexe
Nr minim puncte de articulatie: 0, graful nu are muchii

11.Care este diferenta intre alg Las Vegas si Monte Carlo?las vegas

a. Las Vegas are timp nedeterminat de rulare, Monte Carlo timp determinat (ex. il lasi sa ruleze X iteratii)

b. Las Vegas cand se opreste garanteaza solutie corecta/optima. La Monte Carlo cu cat il lasi sa ruleze mai mult cu atat solutia este mai aproape de cea corecta / cea optima.

12.Schitati reteaua de restrictii pentru problema unei asignari de cifre pentru litere astfel incat sa fie valabila adunarea: SEND + MORE = MONEY

14.Sa consideram multimea elevilor de la un bal de absolvire. Pentru ultimul dans al balului, fiecare fata a facut o lista cu colegii cu care i-ar placea sa danseze. Spuneti cum ati putea construi un graf pentru aceasta problema, ce tip de graf este acesta si cu ce algoritm ati putea rezolva eficient problema de a gasi o asignare a perechilor de dans astfel incat sa fie un numar maxim de perechi. Precizam ca asignarea perechilor se face astfel incat orice fata sa nu danseze cu un coleg care nu e pe lista de preferinte.

E un graf bipartit, care are o multime de noduri pentru fete, o multime de noduri pentru baieti, fetele in stanga, baietii in dreapta. Exista muchie intre fata x si baiatul y daca fata x vrea sa danseze cu baiatul y.

Pentru a afla numarul maxim de perechi pe care le putem forma, facem o retea de flux pe baza acestui graf bipartit, legand toate nodurile din stanga de o sursa, toate nodurile din dreapta de o destinatie, iar absolut toate muchiile din graf trebuie sa aiba capacitate 1

16.Enumerati utilizarile folosite in algoritmii de la curs ale timpilor de debut si de finis de la parcurgerea in adancime a unui graf.
Apar pe la sortare topologica, la Kosaraju (CTC in general) si pe la componente biconexe (Tarjan).

17.Dati exemplu de un graf cu n noduri si m > n arce si o parcurgere in adancime care genereaza n arbori de parcurgere a grafului.
Fiecare nod e o componenta conexa si am mai multe drumuri de la fiecare nod la el insusi?
Sau 1->2->3->... (+ muchii de la 1 la 3, etc) si se face dfs pe noduri in ordine descrescatoare a

valorii.

19.Transformand unul din algoritmii facuti la curs, scrieti in pseudocod un algoritm de detectare eficienta a existentei ciclurilor intr-un graf orientat. Justificati. Ce complexitate are?

Se poate face si cu DFS (se alege un nod random, se porneste DFS din el si se cauta cicluri asa, iar asta se face cat timp mai avem noduri nevizitate)

Data fiind exprimarea “transformand unul din algoritmii facuti la curs”, e posibil sa ceara o modificare a Bellman Ford, si anume: se pune cost -1 pe fiecare muchie din graf si se porneste Bellman Ford, iar daca in urma rularii algoritmului gasim ca exista un ciclu de cost negativ, inseamna ca exista ciclu in graful nostru (automat orice ciclu din graf conduce la un ciclu de cost negativ)

Ca si complexitate, solutia cu DFS are O(N + M), iar cea cu Bellman Ford are O(N * M).

20.Cum se transforma o problema de determinare a fluxului maxim intr-o retea cu mai multe surse si mai multe destinatii intr-o problema de determinare a fluxului maxim intr-o retea cu o sursa si o destinatie.

Se construieste o sursa noua, pe care o legam cu capacitate INF de toate sursele initiale, precum si o destinatie noua, pe care o legam cu capacitate INF de toate destinatiile initiale.

21.Modificati un algortim de la curs astfel incat sa detecteze daca un graf e conex sau nu.
Nu am fost eu la curs, dar DFS/BFS sigur s-au facut. Faci BFS/DFS dintr-un nod oarecare. Daca la final toate nodurile sunt visited (colorate cu negru), atunci inseamna ca graful e conex. Daca nu, nu e conex.

22.Mentionati de ce e mai bun AC3 fata de AC1.
AC1 = O(n*r*a^3)
AC3 = O(r*a^3)
AC3 face mult mai puţine apeluri ale funcţiei REVISE decat AC1. Dacă o constrângere aduce modificări în reţea AC3 adauga pentru reverificare nodurile care punctează către nodul de plecare al restricţiei verificate, iar la AC1 se aplică algoritmul de consistenţa nodurilor şi apoi se aplică REVISE până nu se mai realizează nici o schimbare.

24.Care sunt pasii principali care trebuie urmati pentru a rezolva o problema folosind programarea dinamica? Exemplificati pe scurt aplicarea acestor pasi pe una dintre problemele discutate la curs./ Exemplificare pe problema parantezarii matricilor.

Algoritm generic

Soluții-parțiale^0 ← {Soluție-parțială-elementară$_1$, Soluție-parțială-elementară$_2$,...}

Pentru i=1 ***la*** n ***repetă***

Soluții-parțiale^i =combină(Soluții-parțiale$^{j<i}$,Criteriu-de-optim)

Întoarce Soluții-parțiale^n

Descompunere în subprobleme

- Încercăm să definim subprobleme identice cu problema originală, dar de dimensiune mai mică.

- $\forall$ $1 \le i \le j \le n$:
  - Notăm $A_{i,j} = A_i \times \ldots \times A_j$. $A_{i,j}$ are p_{i-1} linii si p_j coloane: $A_{i,j}(p_{i-1}, p_j)$
  - m[i, j] = numărul optim de înmulţiri pentru a rezolva subproblema $A_{i,j}$
  - s[i, j] = poziţia primei paranteze pentru subproblema $A_{i,j}$
  - Care e parantezarea optimă pentru $A_{i,j}$?

- Problema iniţială: $A_{1,n}$

Combinarea subproblemelor

- Pentru a rezolva $A_{i,j}$
  - trebuie găsit acel indice $i \le k < j$ care asigură parantezarea optimă:

$A_{i,j} = (A_i \times \ldots \times A_k) \times (A_{k+1} \times \ldots \times A_j)$

$A_{i,j} = A_{i,k} \times A_{k+1,j}$

Alegerea optimală

- Căutăm optimul dintre toate variantele posibile de alegere ($i \le k < j$)

- Pentru aceasta, trebuie însă ca şi subproblemele folosite să aibă soluţie optimală

(adică $A_{i,k}$ şi $A_{k+1,j}$)

26.Ce este o muchie inapoi (ce culori au varfurile ce determina muchia respectiva)? Demonstrati ca un graf aciclic nu are voie sa contina muchii inapoi in parcurgera in adancime.
O muchie de la un nod in curs de evaluare la un alt nod in curs de evaluare. Nu are voie sa contina muchii inapoi pentru ca inseamna ca in cursul algoritmului dfs se gasesc muchii catre noduri care au fost deja evaluate, deci ca exista un ciclu. (Merge desen sau Metoda Reducerii la Absurd)

27.Schitati pe scurt pseudocodul pentru algorimtul minimax. Spuneti apoi minim 2 posibiliati prin care acesta poate fi imbunatatit pentru a reduce timpul de rezolvare.

```
01  function minimax(node, depth, maximizingPlayer)
02      if depth = 0 or node is a terminal node
03          return the heuristic value of node

04      if maximizingPlayer
05          bestValue := −∞
06          for each child of node
07              v := minimax(child, depth − 1, FALSE)
08              bestValue := max(bestValue, v)
09          return bestValue

10      else                (* minimizing player *)
11          bestValue := +∞
12          for each child of node
13              v := minimax(child, depth − 1, TRUE)
14              bestValue := min(bestValue, v)
15      return bestValue
```

Doua posibilitati pentru a reduce timpul de rezolvare:
Conteaza foarte mult ordinea in care analizam mutarile posibile => Sortarea mutarilor ce vor fi analizate
Limitarea adancirii cautarii (atat in mod dinamic cat si in mod static)

28.In ce cazuri complexitatea cautarii drumurilor de cost minim intr-un graf orientat G=(N, A) cu costuri pozitive este O(#N + #A)?

https://www.geeksforgeeks.org/shortest-path-for-directed-acyclic-graphs/

Pentru un graf aciclic orientat (DAG) se poate face o sortare topologica si apoi distantele se pot calcula folosind DFS. Se ajunge la o complexitate de O(V + E).

Cand toate costurile dintr-un graf oarecare sunt egale (in acest caz putem aplica BFS ca sa aflam costul minim).

29.Ce este o cale reziduala intr-o retea de conducte si cum se poate obtine? Care este

capacitatea reziduala a caii si cum se poate determina? Exemplificati.

30.Comparati A* si cautarea in latime.
Cautarea in latime pur si simplu se uita pas cu pas, mai departe la care e urmatorul “nod” din graf si verifica daca a ajuns unde trebuie. La a* se foloseste o euristica care daca este aleasa corect duce mult mai rapid la rezolvarea problemei. Merge un desen aici in care sa arati ca la BFS se cauta in jurul punctului de start (Faci un punct cu cercuri in jurul lui), iar la un A* ok se duce efectiv in directia generala a destinatiei.

31.Ce tip de algoritmi ati folosi daca ar trebui sa calculati care este unul dintre cele mai mari numere prime?
Abordare de tipul Las Vegas. (poate MonteCarlo, cf curs?)

Mica teorema a lui Fermat: Daca n este prim -> oricare ar fi X, 0 < X < n, X^(n - 1)%n=1.
Luam aleator un numar de la 0 la n si verificam relatia lui Fermat. Daca da false, numarul nu este prim. Continuam sa luam aleator cate-un numar pana cand demosntram ca numarul nu este prim, sau pana epuizam toate numerele, adica am descoperit un numar prim.
Sau Miller-Rabin, dar e mizerie ala.

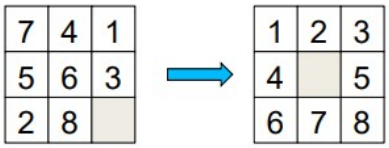
32.Evidentiati etapele cele mai importante ale strategiei de rezolvare divide et impera, dupa care explicati cum poate fi folosita aceasta pentru a calcula eficient x pow n (x ^ n), pentru x – numar real, iar n >= 0.
E explicat mai sus

33.Explicati utilitatea algoritmilor de satisfacere/propagare a constrangerilor, pornind de la functionalitatea oferita de REVISE. Trecand apoi in revista algoritmul AC1, explicati de ce acesta este considerat ineficient.
Functia revise sterge acele valori din domeniul de definiţie al unei variabile pentru care nu este satisfăcută restricţia pentru nici o valoare corespunzătoare celeilalte variabile a restricţiei. Astfel acele ramuri nu mai sunt parcurse.
AC1 este ineficient deoarece se aplică REVISE până nu se mai realizează nici o schimbare, nu se face reverificarea nodurilor direct implicate de o constrângere din reţea.

34.Definiti notiunea de dominanta pentru algoritmul A*, explicand de ce aceasta este importanta in alegerea euristicii. Dati exemplu de o problema unde puteti defini (cel putin) doua euristici diferite si discutati care dintre ele este dominanta.
Un algoritm A1 * domină un algoritm A2 * dacă orice nod expandat de A1 * este expandat si de A2 *. (eventual, A2 * expandează noduri suplimentare față de A1 *, deci A1 * poate fi mai rapid DECAT A2 *.)

Dominanța - Exemplu

- Considerăm jocul 8-pătrățele care trebuie aranjat pornind de la forma inițială prin mutarea locului 'liber' astfel incat să ajungem la forma finală:



- Două euristici posibile:
  - h_1 = numărul pătrățelelor a căror poziție curentă diferă de poziția finală;
  - $h_1 = \Sigma_{p \in piese}(\delta_p)$, unde $\delta_p = 0$ dacă poziția curenta coincide cu cea finala si $\delta_p = 1$, altfel
  - h_2 = distanța Manhattan = suma distanțelor pe verticală si orizontală intre pozițiile curente ale pătrățelelor si pozițiile lor finale
  - $h_2 = \Sigma_{p \in piese}(dist_h_p + dist_v_p)$

h2 e mai informata decat h1

35.Dati exemplu de un algoritm studiat la curs care poate fi utilizat pentru determinarea ciclurilor de cost negativ intr-un graf orientat. Dupa aceea, scrieti pseudocodul pentru determinarea tuturor nodurilor care alcatuiesc un astfel de ciclu SAU demonstrati corectitudinea determinarii ciclului negativ de catre acest algoritm.
Floyd Warshall si daca pe diagonala principala apar valori negative inseamna ca avem ciclu de cost negativ, cred?

37.Care este inaltimea maxima la care poate ajunge un nod in cadrul algoritmilor de tip preflux (push-relabel) folositi pentru determinarea fluxului maxim. Argumentati pe scurt./ + De ce este utilă/necesară această limită?
Inaltimea maxima este 2 * |V| - 1 deoarece aceasta este lungimea maxima posibila a unui drum rezidual.

38.Dandu-se matricea finala de distante obtinunta folosind algoritimul Floyd-Warshall, se poate stabili daca graful respectiv are un ciclu de cost negativ? Argumentati.
In mod normal, matricea ar trebui sa aiba 0 pe diagonala principala. In cazul unui ciclu negativ, pe diagonala pot aparea valori diferite de 0 deoarece daca plec dintr-un nod si urmez ciclul, la final voi ajunge tot in el, si voi avea un cost < 0, fiind ciclu negativ, deci imi va pune noua valoare in matricea finala (ceva negativ < 0).

39.Demonstrati ca un graf are cicluri daca si numai daca in timpul unei parcurgeri DFS sunt

descoperite arce inapoi.
Dfs => cycle : in timpul parcurgerii dfs se gaseste o muchie noua catre un nod “gri” => exista o muchie care duce care a fost deja marcat ca fiind in curs de vizitare => exista ciclu (merge desen aici)
Cycle => dfs : exista ciclu => exista macar o muchie care se “intoarce” la un nod care va fi marcat ca “gri” deci care a fost marcat ca in curs de vizitare => parcurgerea dfs va gasi ac Vea muchie. (merge desen, dar ar fi cam acelasi lucru)

40.Explicati utilitatea taieturilor alfa-beta pentru arborii de joc si dati un exemplu de arbore de joc in care sa evidentiati cel putin 2 taieturi:una pe nivelul MIN, a doua pe nivelul MAX./ Explicați ce reprezintă o tăietura alfa-beta într-un arbore de joc și dați un exemplu de astfel de arbore, cu maxim 3 nivele, în care să scoateți în evidență minim două tăieturi alfa-beta.

44.Pornind de la definiția funcției REVISE, arătați de ce algoritmul AC3 este mai perfomant decât algoritmul AC1.
AC1 = O(n*r*a3)
AC3 = O(r*a3)
AC3 face mult mai puţine apeluri ale funcţiei REVISE decat AC1. Dacă o constrângere aduce modificări în reţea AC3 adauga pentru reverificare nodurile care punctează către nodul de plecare al restricţiei verificate, iar la AC1 se aplică algoritmul de consistenţa nodurilor şi apoi se aplică REVISE până nu se mai realizează nici o schimbare.

46.Ce se întâmplă cu algoritmul minimax cu tăieturi alfa-beta dacă modificăm condiția de tăiere din (β<α) în (β-k<α), unde k este o constantă oarecare. Discuție după k.

In cazul in care:

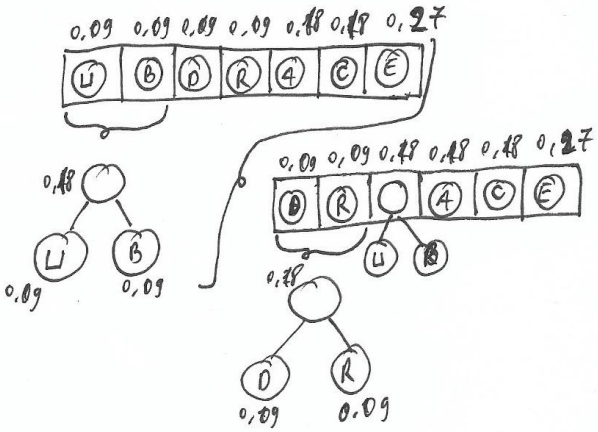
1. k=0 => algoritmul nu se schimba;
2. β-k<β (k>0) => riscam sa facem taieturi inutile ce va duce la rezultate gresite in urma rularii algoritmului;
3. β-k>β (k<0) => riscam sa omitem taituri ce va duce la crestea timpului de rularea al algoritmului insa functionalitatea acestuia va ramane;

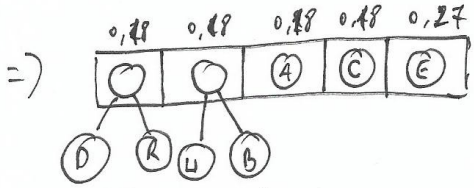
47.Construiti arborele Huffman de compresie pentru urmatorul text “ce abecedar” si calculati care este costul acestuia (cu 2 zecimale).

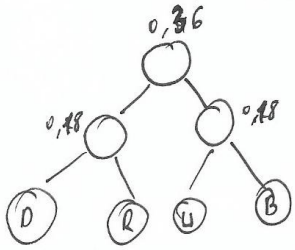
+

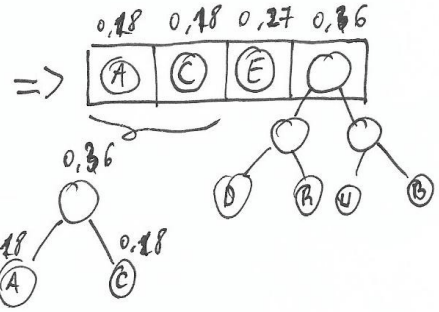
CE␣ABECEDAR

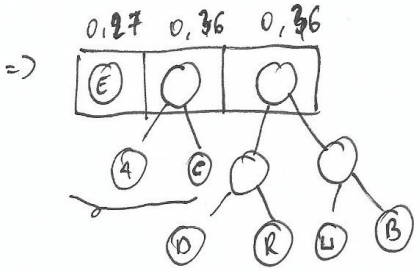
E – 3/11 = 0,27
C – 2/11 = 0,18
A – 2/11 = 0,18
␣ – 1/11 = 0,09
B – 1/11 = 0,09
D – 1/11 = 0,09
R – 1/11 = 0,09

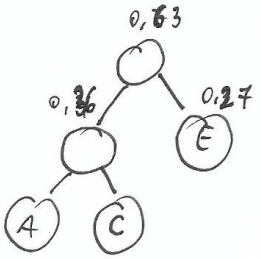


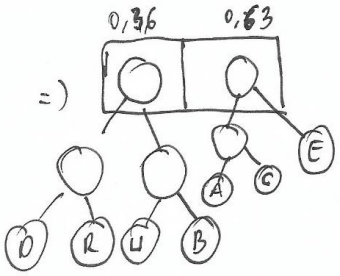


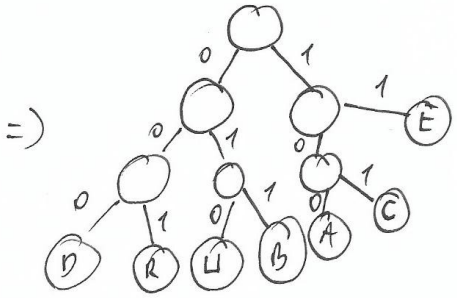










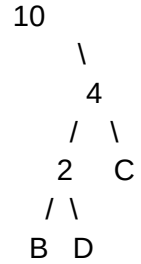


aSortam literele dupa numarul de aparitii:
E - 3
C - 2
A - 2
Spatiu - 1
B - 1
D - 1
R - 1

A aparut si R, acum cum faci arborele? Ca initial l-ati uitat.

Initial fiecare litera face parte dintr-un subarbore cu un nod.
Cat timp avem cel putin 2 subarbori, luam subarborii cu cele mai putin aparitii si ii unim intr-un arbore cu numar de aparitii egal cu suma nr celor doi subarbori

Pentru exemplul dat, in final arborele Huffman va arata asa:



Importante sunt doar literele si caracterul spatiu.
Pentru a determina incodarea huffman pentru fiecare caracter, procedam in felul urmator: pornim din radacina, si continuam in jos pentru a ajunge la caracterul respectiv. Daca am luat-o prin fiul stang al nodului curent, adaugam un 0 la encodare; altfel, adaugam un 1

Ex: vrem sa encodam pe A. pentru a ajunge la el va trebuie sa mergem: stanga -> stanga -> dreapta, deci incodarea Huffman este 001

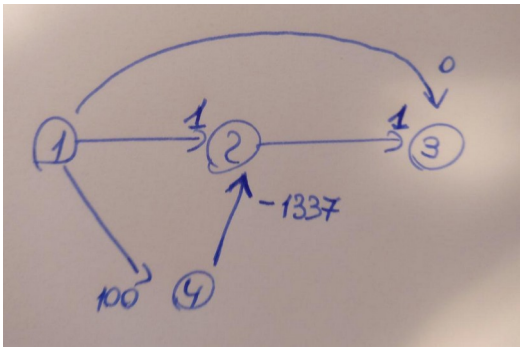
Pentru restul caracterelor, incodarea Huffman este:
E - 01
C - 11
‘ ‘ - 000
B - 100
D - 101

Se observa ca aceasta incodare asigneaza coduri mai scurte caracterelor care apar mai des in mesaj si, in plus, niciun cod nu este prefixul altuia.

Legat de cost, aici (http://dept.cs.williams.edu/~tom/weavingCS/s07/homeworks/Hw7.pdf) zice

ca efectiv faci traducerea din caractere in coduri huffman si vezi care e lungimea. Dar in cazul asta nu stiu de ce zice Trausan ca vrea cu 2 zecimale avand in vedere ca facem doar adunari :| <- trausan vrea sa faci arborele nu dupa nr de aparitii ci dupa probabilitatea aparitiei, adica A - 2/10, B - 1/10 etc si costul arborelui e suma de probabilitate litera * nr de biti (3/10 * 2 pt E)

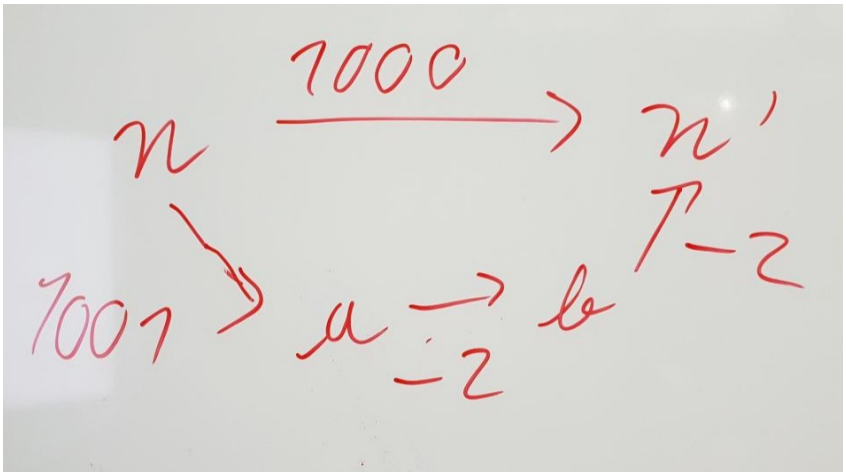
48.Dati un exemplu de graf cu maxim 5 noduri si cel putin o muchie de cost negativ pentru care algoritmul lui Dijkstra pentru drumuri minime de sursa unica calculeaza gresit costul drumurilor minime (este suficient sa scrieti pentru un nod costul corect si cel calculat de algoritm). Cum se poate modifica algoritmul pentru a rezolva aceasta problema?



Solutie: verificam si pentru nodurile din closed (facem Bellman Ford)

49.Taieturile alfa-beta reduc complexitatea unei rezolvari cu minimax pentru jocuri doar daca sunt folosite impreuna cu o alta tehnica. Care este aceasta si de ce este importanta? Dati un scurt exemplu care demonstreaza ca folosirea acesteia impreuna cu alfa-beta aduce imbunatatiri.
?Nu mai generam anumite ramuri cand stim ca min/max oricum nu o vor lua pe acolo. Astfel se reduce timpul de calcul.
?Folosirea unei euristici care sa parcurga subarbori care garanteaza o taiere alfa-beta, astfel restul subarborilor din la acel nivel nu mai trebuie parcursi.
?Limitarea adâncimii căutării = căutare în adâncime mărind iterativ adâncimea maximă până la care căutăm

50.Daca pentru fiecare nod n din cadrul unei explorari folosind algoritmul A* euristica este definita in felul urmator: h(n) = max{cost(n, n') | n' $\in$ succs(n)} (costul muchiei de cost maxim care pleaca din n catre un succesor al sau). Ce puteti spune despre aceasta euristica si despre rezultatul intors de catre algoritm?
h nu e admisibila, ex:



h*(n) = 1001 - 2 - 2 = 997
h(n) = max{1000, 1001} = 1001
Deci nu este garantat ca algoritmul va intoarce o solutie optima.

52.Dați exemplu de o problemă care se poate rezolva, eventual cu constrângeri diferite asupra datelor de intrare, atât folosind programare dinamică, cât și cu programare lacomă. Explicați diferențele și asemănările între cei doi algoritmi (inclusiv complexitatea lor).
Problema rucsacului

- Greedy: se pot lua doar bucati dintr-un anumit obiect
  alege in ordine descrescatoare in functie de pret/greutate unitati din obiecte
  T(nlogn), S(1)
- DP: obiectele nu se pot sectiona
  Construieste dp[i][j] unde dp[i][j] reprezinta valoarea maxima care se poate obtine umpland un ghiozdan de capacitate j folosind doar primele i obiecte
  T(n * capacitate), S(n * capacitate)

54.Fie un algoritm A* ghidat de următoarea euristică, h(n) = 0 ∀ n ∈ V, pentru problema 8-puzzle discutată la curs. Explicați care este numărul maxim de noduri care sunt scoase din mulțimea CLOSED pentru a fi reintroduse în mulțimea OPEN în timpul găsirii unei secvențe de pași de la o configurație inițială a problemei la o configurație finală. Asigură această euristică găsirea soluției optime ca număr de pași efectuați?
Euristica este admisibila, deci A*gaseste solutia optima ca numar de pasi efectuati.

55.Explicati utilitatea algoritmului lui Johnson. De ce este mai bun decat alternativele.
Problema: drum de la n surse la m destinatii.
Solutii:

1. RoyFloyd - O(n^3)
2. Nx Dijkstra - O(N * M log N), dar cu costuri pozitive
3. Johnson e smenul sa faci costurile pozitive ca sa poti aplica 2

e. BellmanFord de la un nod fictiv - h[i] = distanta nod fictiv i
f. w[i][j] => w[i][j] + h[i] - h[j]
g. Bagi 2 => d[i][j] = distanta calculata cu Dijkstra pe noul graf cu costuri pozitive
h. d[i][j] => d[i][j] + h[j] - h[i] = distanta pe graful initial

57.Daca facem +2 la costuri pe muchii / *2 la costuri, se modifica drumul minim gasit de Dijkstra?

Daca facem *2 drumul ramane acelasi, doar i se dubleaza costul, iar daca facem +2 se modifica drumul. Spre exemplu:
1 -> 3 cu cost 4
1 -> 2 cu cost 2
2 -> 3 cu cost 1

Initial Dijkstra gaseste drumul 1 -> 2 -> 3 cu cost 3, dar dupa ce crestem costurile, costul drumului 1 -> 3 ajunge 6, pe cand 1 -> 2 -> 3 ajunge 7, ceea ce inseamna ca gasim alt drum.

Cursul 1 – Divide et Impera

Scheme de algoritmi = tipare commune pe care le putem aplica in rezolvarea unor probleme similar

- ⇨ Cunoașterea schemelor determină o rezolvare mai rapidă și mai eficientă a problemelor.

Divide et Impera:

- ⇨ o clasă de algoritmi care au ca principale caracteristici faptul că împart problema în subprobleme similare cu problema inițială dar mai mici ca dimensiune, rezolvă problemele recursiv și apoi combină soluțiile pentru a crea o soluție pentru problema originală.
- ⇨ Schema Divide et impera consta in 3 pasi la fiecare nivel al recurentei:
  - Divide problema dată într-un număr de subprobleme
  - Impera (cucerește) – subproblemele sunt rezolvate recursiv. Dacă subproblemele sunt suficient de mici ca date de intrare se rezolvă direct (ieșirea din recurență)
  - Combină – soluțiile subproblemelor sunt combinate pentru a obține soluția problemei inițiale.
- ⇨ Avantaje:
  - Produce algoritmi eficienti
  - Descompunerea problemei în subprobleme facilitează paralelizarea algoritmului în vederea execuției sale pe mai multe procesoare.
- ⇨ Dezavantaje:
  - Se adaugă un overhead datorat recursivității (reținerea pe stivă a apelurilor funcțiilor).

- ⇨ Exemple: **Merge Sort** – exemplu clasic de rezolvare cu D&I
  - **Divide**: Divide cele n elemente ce trebuie sortate în 2 secvențe de lungime n/2.
  - **Impera**: Sortează secvențele recursiv folosind merge sort.
  - **Combina**: Secvențele sortate sunt asamblate pentru a obține vectorul sortat.
  - Recurența se oprește când secvența ce trebuie sortată are lungimea 1 (un vector cu un singur element este întotdeauna sortat ☺) .

```
MERGE(A, p, q, r) // p si r sunt capetele intervalului, q este "mijlocul"
1       n1 ← q - p + 1 // numărul de elemente din partea stânga
2       n2 ← r – q  // numărul de elemente din partea dreapta
3       creează vectorii S[1 -> n1 + 1] și D[1 -> n2 + 1]
4       Pentru i de la 1 la n1
5               S[i] ← A[p + i - 1] // se copiază partea stânga în S
6       Pentru j de la 1 la n2
7               D[j] ← A[q + j] // si partea dreapta în D
8       S[n1 + 1] ← ∞
9       D[n2 + 1] ← ∞
10      i ← 1
11      j ← 1
12      Pentru k de la p la r  // se copiază înapoi în vectorul de
13              Dacă S[i] ≤ D[j]        // sortat elementul mai mic din cei
14                      Atunci A[k] ← S[i]   // doi vectori sortați deja
15                              i ← i + 1
16                      Altfel A[k] ← D[j]
17                              j ← j + 1
```

  - **Complexitate**: T(n) = 2T(n/2) + O(n) => T(n) = O(n*log n).

- ⇨ Exemplu 2: **Calculul puterii unui numar**:
  - Dacă n este par:
    Atunci intoarce x^(n/2) * x^(n/2);
    Altfel (n este impar) intoarce x * x^((n-1)/2) * x^((n-1)/2)
  - **Complexitate**: T(n) = T(n/2) + O(1) = O(log n)

- ⇨ Exemplu 3: **Calcului celei mai scurte distante intre 2 puncte din plan**:
  - Sortează punctele în ordinea crescătoare a coordonatei x (Θ(nlog n));
  - Împărțim setul de puncte în 2 seturi de dimensiune egală și calculăm recursiv distanța minimă în fiecare set (l = linia ce împarte cele 2 seturi, d = distanța minimă calculată în cele 2 seturi);
  - Elimină punctele care sunt plasate la distanță de l > d;
  - Sortează punctele rămase după coordonata y;
  - Calculează distanțele de la fiecare punct rămas la cei 6 vecini (nu pot fi mai mulți);
  - Dacă găsește o distantă < d, atunci actualizează d.

Cursul 2 – Greedy

- ⇨ Metodă de rezolvare eficientă a unor probleme de optimizare.
- ⇨ Soluția trebuie să satisfacă un criteriu de optim global (greu de verificat) -> optim local mai ușor de verificat.
- ⇨ Se aleg soluții parțiale ce sunt îmbunătățite repetat pe baza criteriului de optim local până ce se obțin soluții finale.
- ⇨ Soluțiile parțiale ce nu pot fi îmbunătățite sunt abandonate proces de rezolvare irevocabil (fără reveniri)!

```
1. sol_parțiale = sol_inițiale(Problemă); // determinarea soluțiilor parțiale
2. sol_fin = Φ;
3. Cât timp (sol_parțiale ≠ Φ)
4.         Pentru fiecare (s în sol_parțiale)
5.                 Dacă (s este o soluție a problemei) { // dacă e soluție
6.                         sol_fin = sol_fin U {s}; // finală se salvează
7.                         sol_parțiale = sol_parțiale \ {s};
8.                 } Altfel // se poate optimiza?
9.                 Dacă (optimizare_posibilă (s, Crit_optim, Problemă))
10.                        sol_parțiale = sol_parțiale \ {s} U      // da
                                  optimizare(s, Crit_optim, Problemă)
11.                Altfel sol_parțiale = sol_parțiale \ {s}; // nu
12. Întoarce sol_fin;
```

- ⇨ **Comparatie D&I si Greedy**
  - Tip de abordare: D&I: topdown ; Greedy: top-down
  - Criteriu de optim: D&I: nu; Greedy: da
- ⇨ Cand functioneaza algoritmii Greedy?
  - Problema are proprietatea de substructură optima: Soluţia problemei conţine soluţiile subproblemelor.
  - Problema are proprietatea alegerii locale: Alegând soluţia optimă local se ajunge la soluţia optimă global.
  - Teoremă. Dacă (E, I) este matroid atunci există un algoritm Greedy care rezolvă problema de optimizare. Un sistem accesibil este un matroid dacă satisface

proprietatea de interschimbare: oricare X, Y apartine I şi |X| < |Y| => exista e apartine Y \ X a. î. X U {e} apartine I.

-

⇨ Exemplu 1: **Arbori Huffman**

- Construcția unui astfel de arbore se realizează printrun algoritm Greedy.
- 1. Pentru fiecare k din K se construiește un arbore cu un singur nod care conține cheia k si este caracterizat de ponderea w = p(k). Subarborii construiți formează o mulțime numită Arb.
- 2. Se aleg doi subarbori a şi b din Arb astfel încât a şi b au pondere minimă.
- 3. Se construiește un arbore binar cu o rădăcina r care nu conține nici o cheie și cu descendenții a și b. Ponderea arborelui este definită ca w(r) = w(a) + w(b).
- 4. Arborii a și b sunt eliminați din Arb iar r este inserat în Arb.
- 5. Se repetă procesul de construcție descris de pașii 2-4 până când mulțimea Arb conține un singur arbore – Arborele Huffman pentru cheile K.

```
Huffman(K,p){
1. Arb = {frunză(k, p(k)) | k ∈ K};
2. Cât timp (card (Arb) > 1) // am mai mulți subarbori
3.          Fie a1 și a2 arbori din Arb a.i. ∀a ∈ Arb a ≠ a1 si a ≠ a2, avem
                w(a1) ≤ w(a) și w(a2) ≤ w(a));     // practic se extrage
                // de două ori minimul și se salvează în a1 și a2
4.          Arb = Arb \ {a1, a2} U nod_intern(a1, a2, w(a1) + w(a2));
5. Dacă (Arb = Φ)
6.          Întoarce arb_vid;
6. Altfel
7.          fie A singurul arbore din mulțimea Arb;
8.          Întoarce A;
```

```
Decodificare (in, out)
   A = restaurare_arbore (in)       // reconstruiesc arborele
  Cât timp (! terminare_cod(in)) // mai am caractere de citit
      nod = A                       // pornesc din rădăcină
      Cât timp (! frunză(nod))     // cât timp nu am determinat caracterul
          Dacă (bit(in) = 1) nod = dreapta(nod)      // avansez în arbore
              Altfel nod = stânga(nod)
      Scrie (out, cheie(nod))       // am determinat caracterul și îl scriu la
                                    // ieșire
```

⇨ Exemplu 2: **Problema rucsacului**

Varianta 1: Algoritm Greedy

- sortăm obiectele după raportul v_i/m_i;
- adăugăm fracţiuni din obiectul cu cea mai mare valoare per kg până epuizăm stocul şi apoi adăugăm fracţiuni din obiectul cu valoarea următoare.
- Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10
- Soluție: (m_2 , v_2) 8kg și 2kg din (m_1,v_1) sau din (m_4,v_4) – valoarea totală: 19 + 2 * 10 / 5 = 23

Varianta 2: Algoritmul Greedy nu funcţionează => contraexemplu

- Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10
- Rezultat corect – 2 obiecte: (m_1,v_1) și (m_4,v_4) – valoarea totală: 20
- Rezultat algoritm Greedy – 1 obiect (m_2,v_2) – valoarea totală: 19

Fie I = mulțimea subsoluțiilor și X = $\{m_2\}$ și Y = $\{m_1, m_4\}$ → $m_1 \in Y \setminus X$ dar $X \cup \{m_1\} \notin I$ → problema nu respectă proprietatea de matroid → problema nu se poate rezolva folosind tehnica Greedy!

Cursul 3 – Programare dinamica

- ⇨ Descriere generala: Soluții optime construite iterativ asamblând soluții optime ale unor probleme similare de dimensiuni mai mici.
- ⇨ Algortmi clasici:
  - Înmulţirea unui şir de matrici
  - AOC
  - Algoritmul Floyd-Warshall care determină drumurile de cost minim dintre toate perechile de noduri ale unui graf.
  - Numere catalane
  - Viterbi
- ⇨ Algoritm generic:

Programare dinamică (crit_optim, problema)

- // fie $problema_0$ $problema_1$ … $problema_n$ astfel încât
- // $problema_n$ = problema; $problema_i$ mai simplă decât $problema_{i+1}$
- 1. Sol = soluții_inițiale(crit_optim, $problema_0$);
- 2. **Pentru** *i* **de la** 1 **la** *n* // construcție soluții pentru // $problema_i$ folosind soluțiile problemelor precedente
- 3. Sol_i = calcul_soluții(Sol, crit_optim, $Problema_i$); // determin soluția $problemei_i$
- 4. Sol = Sol U Sol_i; // noile soluții se adaugă pentru a fi refolosite pe viitor
- 5. s = soluție_pentru_$problema_n$(Sol); // selecție / construcție soluție finală
- 6. **Întoarce** s;

⇨ Caracteritici:
- O soluție optimă a unei probleme conține soluții optime ale subproblemelor.
- Decompozabilitatea recursivă a problemei P în subprobleme similare P = Pn, Pn-1, … P0 care acceptă soluții din ce în ce mai simple.
- Suprapunerea problemelor (soluția unei probleme Pi participă în procesul de construcție a soluțiilor mai multor probleme Pk de talie mai mare k > i) – memoizare (se foloseşte un tablou pentru salvarea soluţiilor subproblemelor cu scopul de a nu le recalcula)
- În general se folosește o abordare bottom-up, de la subprobleme la probleme.

⇨ Diferente: **Greedy – Programare dinamica**

Greedy	**Programare dinamica**
Sunt mentinute doar solutiile partiale curente din care evolueaza solutiile partiale urmatoare	La constructia unei solutii noi poate contribui orice alta solutie partial generata anterior
Solutiile partiale anterioare sunt eliminate	Se pastreaza toate solutiile partiale
Se poate obtine o solutie neoptima	Se obtine solutia optima

⇨ Diferente: **D&I – Programare dinamica**

Divide et impera	**Programare dinamica**
Abordare top-down – problema este descompusă în subprobleme care sunt rezolvate independent	In general abordare bottom-up - se porneşte de la sub-soluţii elementare şi se combină sub-soluţiile mai simple în sub-soluţii mai complicate, pe baza criteriului de optim
Putem rezolva aceeaşi problemă de mai multe ori (dezavantaj potenţial foarte mare)	Se evită calculul repetat al aceleiaşi subprobleme prin memorarea rezultatelor intermediare (memoizare)

⇨ Exemplu 1: **Parantezarea matricilor**
- Se dă un şir de matrice: A1, A2, ..., An.
- Care este numărul minim de înmulţiri de scalari pentru a calcula produsul: A1 x A2 x ... x An?
- Să se determine una dintre parantezările care minimizează numărul de înmulţiri de scalari.
- Descompunerea in subprobleme:
  - Încercăm să definim subprobleme identice cu problema originală, dar de dimensiune mai mică.
  - Oricare $1 \le i \le j \le n$:
    - Notăm Ai, j = Ai x … x Aj. Ai,j are pi-1 linii şi pj coloane: Ai,j(pi-1, pj)
    - m[i, j] = numărul optim de înmulţiri pentru a rezolva subproblema Ai,j
    - s[i, j] = poziţia primei paranteze pentru subproblema Ai,j.
  - Problema iniţială: A1,n
  - Pentru a rezolva Ai,j
    - Trebuie găsit acel indice $i \le k < j$ care asigură parantezarea optimă: Ai, j = (Ai x…x Ak) x (Ak+1 x…x Aj)

Ai, j = Ai, k x Ak+1, j

- Alegerea optimala:
  - Căutăm optimul dintre toate variantele posibile de alegere (i ≤ k < j)
  - Pentru aceasta, trebuie însă ca şi subproblemele folosite să aibă soluţie optimală (adică Ai, k şi Ak+1, j să aibă soluție optimă).
- Substructura optimala:
  - Dacă ştim că alegerea optimală a soluţiei pentru problema Ai, j implică folosirea subproblemelor (Ai, k şi Ak+1, j) şi soluţia pentru Ai, j este optimală, atunci şi soluţiile subproblemelor Ai, k şi Ak+1, j trebuie să fie optimale!
- Definirea recursiva
  - $$m[i,j] = \begin{cases} 0 & i = j, \\ \min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j) & i < j \end{cases}$$
  - Cazurile de bază sunt m[i, i]
  - Noi vrem să calculăm m[1, n]
  - Cum alegem s[i, j] ?: Bottom-up de la cele mai mici subprobleme la cea inițială.

$$m[1,2], m[2,3], m[3,4], \ldots, m[n-3,n-2], m[n-2,n-1], m[n-1,n]$$
$$m[1,3], m[2,4], m[3,5], \ldots, m[n-3,n-1], m[n-2,n]$$
$$m[1,4], m[2,5], m[3,6], \ldots, m[n-3,n]$$
$$\vdots$$
$$m[1,n-1], m[2,n]$$
$$m[1,n]$$

- Pseudocod:

```
Înmulțire_matrici (p, n)
  Pentru i de la 1 la n // inițializare
    m[i, i] = s[i, i] = 0
  Pentru l de la 1 la n - 1 // dimensiune problema
    Pentru i de la 1 la n – l  // indice stânga
      j = i + l // indice dreapta
      m[i, j] = ∞ // pentru determinare minim
      Pentru k de la i la j – 1
        q = m[i, k] + m[k + 1, j] + p[i - 1] * p[k] * p[j]
        Dacă q < m[i, j]
              m[i, j] = q
              s[i, j] = k
  Întoarce m și s
```

- Complexitate:
  - Spatiala: O(n^2)
  - Temporala: O(n^3) (O(n^2) = numarul total de subprobleme; O(n) = numarul total de alegeri la fiecare pas)

- ⇨ Exemplu 2: **Arbori binari de cautare**
  - Probleme:
    - Costul căutării depinde de frecvenţa cu care este căutat fiecare termen. -> Ne dorim ca termenii cei mai des căutaţi să fie cât mai aproape de vârful arborelui pentru a micşora numărul de apeluri recursive.
    - Dacă arborele nu este construit prin sosirea aleatorie a cheilor putem ajunge la o simplă listă cu n elemente.
  - Un arbore de căutare probabilistică având cost minim este un arbore optim la căutare (AOC).
  - AOC(x, p, q, n)
    Pentru *i* **de la** 0 **la** *n*
    $\{C_{i,i} = 0, R_{i,i} = 0, w_{i,i} = q_i\}$ // iniţializare costuri AOC vid $A_{i,i}$
    Pentru *d* **de la** 1 **la** *n*
    Pentru *i* **de la** 0 **la** *n-d* // calcul indice rădăcină şi cost pentru $A_{i,i+d}$
    $j = i + d,\ C_{i,j} = \infty,\ w_{i,j} = w_{i,j-1} + p_j + q_j$
    Pentru *α* **de la** *i + 1* **la** *j* // ciclul critic – operaţii intensive
    Dacă $(C_{i,\alpha-1} + C_{\alpha,j} < C_{i,j})$ // cost mai mic?
    $\{ C_{i,j} = C_{i,\alpha-1} + C_{\alpha,j} ; R_{i,j} = \alpha\}$ // update
    $C_{i,j} = C_{i,j} + w_{i,j}$ // update
    Întoarce gen_AOC(C, R, x, 0, n) // construcţie efectivă arbore $A_{0,n}$
    // cunoscând indicii
  - Complexitate: O(n^3) ; optimizare-knuth: O(n^2)

Curs 4 – Backtracking

- ⇨ Cronologic – orb:
  - Construieşte soluţiile iterativ.
  - Menţine evidenţa alegerilor făcute.
  - În momentul în care se ajunge la o contradicţie se revine la ultima decizie luată şi se încearcă alegerea unei alte variante.
  - **Schema Backtracking**
    - Soluție-parțială ← INIT // inițializez
    - EȘEC-DEFINITIV ← fals // nu am ajuns (încă) la eșec
    - **Cât timp** Soluție-parțială nu este soluție finală **și nu avem** EȘEC-DEFINITIV
      - Soluție-parțială ← AVANS (Soluție-parțială) // avansez
      - **Dacă** EȘEC (Soluție-parțială) // nu mai pot avansa
        - **Atunci** REVENIRE (Soluție-parțială) // mă întorc
    - **Dacă** EȘEC-DEFINITIV
      - **Atunci** Întoarce EȘEC // nu s-a găsit nicio soluție
      - **Altfel** Întoarce SUCCES // am ajuns la soluția problemei
    - **Sfârșit.**

    Procedura AVANS (Soluție-parțială)
    - **Dacă** există alternativă de extindere // pot avansa?
      - **Atunci** Soluție-parțială ← Soluție-parțială ∪ alternativă de extindere // avansez
      - **Altfel Dacă** Soluție-parțială este INIT
        - **Atunci** EȘEC-DEFINITIV ← adevărat // nu s-au găsit soluții pentru problemă
        - **Altfel** EȘEC (Soluție-parțială) // ramura curentă a dus la eșec

- Optimizari posibile:
  - Alegerea variabilelor în altă ordine.
  - Îmbunătăţirea revenirilor. -> Necesită detectarea cauzei producerii erorii.
  - Evitarea redundanţelor în spaţiul de căutare (îmbunătăţirea avansului). -> Evitarea repetării unei căutări care ştim că va duce la un rezultat greşit.

- O restricţie c este o relaţie între una sau mai multe variabile v1, …, vm, (denumite nodurile sau celulele restricţiei). Fiecare variabila vi poate lua valori într-o anumită mulţime Di, denumită domeniul ei (ce poate fi finit sau nu, numeric sau nu).
  - Unare: specificarea domeniului variabile
  - Binare: intre 2 variabile $V1j \neq V1k$ orice $j \neq k$, $0 < j, k < 10$
  - N-are: intre n variabile, Regula triunghiului: $L1 + L2 > L3$
  - O problemă de satisfacere a restricţiilor (PSR) este un triplet <V,D,C>, format din:
    - o mulţime V formată din n variabile V = {v1, …, vn};
    - mulţimea D a domeniilor de valori corespunzătoare acestor variabile: D = {D1, … Dn} ;
    - o mulţime C de restricţii C = {c1, …, cp} peste submulţimi ale V (ci (vi1, . . ., vij) ⊆ Di1 x Di2 x ... x Dij).
  - O restricţie ci (vi1, ..., vij), este o submulţime a produsului cartezian Di1 x Di2 x ... x Dij, constând din toate tuplurile de valori considerate că satisfac restricţia pentru (vi1, . . ., vij).
  - O soluţie a unei PSR <V,D,C> este un tuplu de valori <x1, … , xn> care conține toate variabilele V, din domeniile corespunzătoare D, astfel încât toate restricţiile din C să fie satisfăcute.
  - PSR binară este o PSR ce conţine doar restricţii unare şi binare.
  - Reprezentarea folosită: graf
    - Nodurile: variabilele restricţiei
    - Arcele: restricţiile problemei
    - Valabilă doar pentru PSR binare
  - Altă reprezentare posibilă: graf
    - Nodurile: restricţiile problemei
    - Arcele: variabilele restricţiei
    - Valabilă pentru orice tip de PSR

- Algoritmi de rezolvare a PSR: - Propagarea restrictiilor
  - Rezolvă PSR binare;
  - Variabilele au domenii finite de valori;
  - Prin propagarea restricţiilor se filtrează mulţimile de valori (se elimină elementele din domeniu conform unui criteriu dat);
  - Procesul de propagare se opreşte când:
    - O mulţime de valori este vidă -> EȘEC;
    - Nu se mai modifică domeniul vreunei mulţimi

- Algoritm de consitenta nodurilor – restrictii unare

Procedura NC(i) este:
- **Pentru fiecare** x ∈ D_i
  - **Dacă** not Q_i (x) // nu este satisfăcută restricția unară
    - **Atunci** șterge x din D_i
- **Sfârșit**.

Algoritm NC-1 este:
- **Pentru** i **de la** 1 **la** *n* **Execută** NC(i) // pentru fiecare var
- **Sfârșit**.

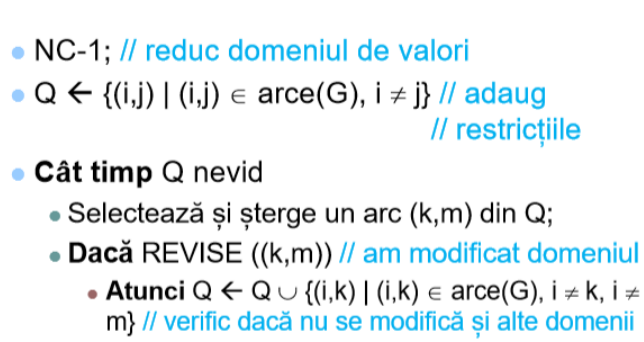
- Algoritm de consistenta a arcelor
  - Inlatura toate inconsistentele submultimilor de 2 elemente ale retelei de restrictii

Funcţia REVISE ((i,j)) este:
- ȘTERS ← fals // nu am modificat domeniul de valori
- **Pentru fiecare** x ∈ D_i
  - **Dacă** nu există y ∈ D_j a.î. P_{ij}(x,y) // nu se respectă restricţia
    - **Atunci**
      - Șterge x din D_i;
      - ȘTERS ← adevărat; // am făcut modificări
- **Întoarce** ȘTERS.

  - Funcţia Revise este apelată pentru un arc al grafului de restricţii (binare) și șterge acele valori din domeniul de definiţie al unei variabile pentru care nu este satisfăcută restricţia pentru nici o valoare corespunzătoare celeilalte variabile a restricţiei.
  - Compexitate: O(a^2)
- AC-1 (Arc Consistency - 1)
  -

- NC-1; // reduc domeniul de valori
- Q ← {(i,j) | (i,j) ∈ arce(G), i ≠ j} // adaug restricțiile
- **Repetă**
  - SCHIMBAT ← fals // nu am modificat niciun // domeniu
  - **Pentru fiecare** (i,j) ∈ Q // pentru fiecare restricție
    - SCHIMBAT ← (REVISE ((i,j)) sau SCHIMBAT)
- **Până când *non*** SCHIMBAT // nu am mai făcut // modificări
- **Sfârșit.**

  - Se aplică algoritmul de consistenţa nodurilor şi apoi se aplică REVISE până nu se mai realizează nici o schimbare
  - Complexitate: O(na * 2r * a^2) (na = La fiecare iteraţie (din Repetă) eliminăm o singură valoare (şi avem maxim na valori posibile) ; 2r =Numărul de apelări al Revise într-o iteraţie (din Repetă); a^2 = Complexitate Revise).

- AC-3 (Arc Consistency - 3)
  - 

  - Se elimină pe rând arcele (constrângerile).
  - Dacă o constrângere aduce modificări în reţea adăugăm pentru reverificare nodurile care punctează către nodul de plecare al restricţiei verificate. -> Scopul: Reverificarea nodurilor direct implicate de o constrângere din reţea.
  - Avantaj: Se fac mult mai puţine apeluri ale funcţiei REVISE.
  - Complexitate: O(a^3*r)

⇨ Backtracking+ Propagarea restrictiilor
- În general, propagarea restricţiilor nu poate rezolva complet problema dată.
- Metoda ajută la limitarea spaţiului de căutare (foarte importantă în condiţiile în care backtracking-ul are complexitate exponenţială!).
- În cazul în care propagarea restricţiilor nu rezolvă problema se foloseşte:
  - Backtracking pentru a genera soluţii parţiale;
  - Propagarea restricţiilor după fiecare pas de backtracking pentru a limita spaţiul de căutare (şi eventual a găsi că soluţia nu este validă).

Curs 5- Minimax

⇨ Metoda Minimax:
- 2 jucători: Max și Min care mută pe rând (Max mută primul).
- Max urmărește să-și maximizeze câștigul.
- Min urmărește să-și minimizeze pierderea.
- Se construiește un arbore de tip AND-OR:
- Nivelurile pare -> mutările jucătorului Max.
- Nivelurile impare -> mutările jucătorului Min.
- Frunzele desemnează câștigul/pierderea lui Max.
- Arcele reprezintă mutările propriu-zise.

⇨ Functionare Minimax:
- 1) Se generează întregul arbore;
- 2) Se evaluează frunzele și li se asociază valori;
- 3) Se propagă rezultatele dinspre frunze spre rădăcină astfel:
  - Nivelul MIN alege cea mai mică valoare dintre cele ale copiilor.
  - Nivelul MAX alege cea mai mare valoare dintre cele ale copiilor.

⇨ Probleme:
- Dimensiunea arborelui pentru “X și 0” e ≤ 9!
- Pentru Șah fiecare nod are în medie 35 copii!
- Pentru Go ramificarea este de cca. 150 – 250!
- Complexitatea arborelui: pentru Șah – 10123 noduri; pentru Go – 10360 noduri.

- Limitări:
  - Nu putem să construim întregul arbore
  - Nu putem ajunge de fiecare dată la stările finale pentru a le putea evalua.

⇨ Optimizari:

- Limitarea adâncimii căutării
  - Trebuie să construim o funcție euristică care să estimeze șansele de câștig pentru o poziție dată.
    - Ex. pentru șah: Regină:10p; Turn: 5p; Cal, Nebun: 3p; Pion: 1p;
    - Ex: Funcție de evaluare a poziției = suma pieselor proprii – suma pieselor adversarului.
  - Oprirea căutării:
    - Limitare statică: după un număr maxim de nivele/interval de timp.
    - Limitare dinamică: când profitul obținut din continuarea căutării devine foarte mic (scade sub o valoare fixată).
  - Se estimează valoarea funcției de evaluare la nivelul respectiv.
  - Apoi propagăm valorile conform principiului enunțat anterior.

⇨ Functii de evaluare:

- Funcția euristică trebuie să cuantifice ”poziția”. -> Chiar în dauna avantajului material.
- Trebuie să ia în calcul potențialele amenințări!

⇨ Algoritm:

MINIMAX_limitat (n, nivel_limită)
- **Pentru fiecare** n' ∈ succs(n) // pentru toate mutările
  - Fie m = mutarea corespunzătoare arcului (n, n')
  - VAL(m) = W(n', nivel_limită, 1) // determin valoarea mutării
- **Întoarce** m a.î. VAL(m) = max {VAL(x)| x ∈ mutări(n)}

W(n, limită, nivel)
- **Dacă** n este frunză **Întoarce** cost(n)
- **Dacă** nivel ≥ limită **Întoarce** euristică(n)
- **Dacă** jucătorul MAX este la mutare **Întoarce**
  - max {w(n', limită, nivel + 1) | n' ∈ succs(n)}
- **Dacă** jucătorul MIN este la mutare **Întoarce**
  - min {w(n', limită, nivel + 1) | n' ∈ succs(n)}

⇨ Taieri α-β

- Încercăm să limităm spațiul de căutare prin eliminarea variantelor ce nu au cum să fie alese.
- Idee: Dacă V21<V1 toată ramura V2 poate fi Ignorată.
- α = max dintre valorile găsite pentru un nod MAX.
- β = min dintre valorile găsite pentru un nod MIN
- Tăiem o ramură dacă:
  - am găsit un nod pe nivelul MAX cu valoare β <= oricare din valorile α calculate anterior;
  - am găsit un nod pe nivelul MIN cu valoare α >= oricare din valorile β calculate anterior.
- Teorema α-β. Fie J un nod din arborele MINIMAX explorat. Daca α(J) ≥ β(J), atunci explorarea nodului J nu este necesară.

- Algoritm

α-β(n, limită)
- w = eval_max(n, -∞, ∞, 0, limită)
- **Întoarce** m ∈ mutări(n) a.i. VAL(m) = w

eval_max(n, α, β, nivel, limită)
- **Dacă** n este frunză **Întoarce** cost(n)
- **Dacă** (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat
- a = -∞ // valoarea curenta a nodului de tip Max
- **Pentru fiecare** (n' ∈ succs(n))
  - a = max(a, eval_min(n', max(α,a), β, nivel+1, limită)); // propag
  - **Dacă** (a ≥ β) **oprire**;
- **Întoarce** a

similar eval_min

  - Reduce complexitatea minimax în cazul ideal de la: Număr_ramificări^(număr_nivele) la Număr_ramificări^(număr_nivele/2)
  - Contează foarte mult ordinea în care analizăm mutările! -> Sortarea mutărilor după un criteriu dat nu este costisitoare comparativ cu costul exponențial al algoritmului.
  - Se folosesc euristici pentru a alege mutările examinate mai întâi.
- ⇨ Observatii :
  - Algoritmi de căutare în adâncime
  - Pot cauza probleme când avem un timp limită. -> Soluție posibilă IDDFS (căutare în adâncime mărind iterativ adâncimea maximă până la care căutăm).
  - Algoritmi cu complexitate foarte mare.
  - Soluții euristice pentru limitarea complexității.
  - Recomandabil să se combine cu alte strategii – baze de date cu poziții, pattern-matching.

Cursul 6 – Introducere in grafuri

- ⇨ Tipuri de grafuri:
  - Orientate: noduri (A-L) + arce (AB, BC, CD…).
  - Neorientate: noduri (A-L) + muchii (AB, BC, CD…).
- ⇨ Utilizari practice:
  - Hărți, rețele (calculatoare, instalații, etc.), rețele sociale, analiza fluxurilor (semaforizare, proiectarea dimensiunii țevilor de apă).
- ⇨ Culoarea nodului – specifică starea nodului la un anumit moment al parcurgerii:
  - Alb – nedescoperit;
  - Gri – descoperit, în curs de prelucrare;
  - Negru – descoperit și terminat (cu semnificații diferite pentru BFS si DFS).

⇨ BFS – Parcurgere in latime

```
BFS(s,G)
- Pentru fiecare nod u (u ∈ V)
    - p(u) = null; dist(s,u) = inf; c(u) = alb; // inițializări
- Q = (); // se folosește o coadă în care reținem nodurile de prelucrat

- dist(s,s) = 0; // actualizări: distanța de la sursă până la sursă este 0
- Q ← Q + s; // adăugăm sursa în coadă → începem prelucrarea lui s
- c(s) = gri; // și atunci culoarea lui devine gri

- Cât timp (!empty(Q)) // cât timp mai am noduri de prelucrat
    - u = top(Q); // se determină nodul din vârful cozii
    - Pentru fiecare nod v (v ∈ succs(u)) // pentru toți vecinii
        - Dacă c(v) este alb // nodul nu a mai fost găsit, nu e în coadă
            - Atunci { dist(s,v) = dist(s,u) + 1; p(v) = u; c(v) = gri; Q ← Q + v;}
                // actualizăm structura date
    - c(u) = negru; // am terminat de prelucrat nodul curent
    - Q ← Q - u; // nodul este eliminat din coadă
```

- Proprietati:
  - În cursul execuției BFS(s,G) v∈Q ⇔ v∈R(s). => ∈ Q => v ∈ R(s): Q conține exclusiv noduri din R(s) (BFS parcurge toate nodurile ce pot fi atinse din s); Toate nodurile ce pot fi atinse din s vor fi introduse cândva în coadă. (drum s = v0v1…vp = v) .
  - Oricare (u,v) ∈ E, δ(s,v) ≤ δ(s,u) + 1 : δ(s,v) ≤ δ(s,u) + 1 în general este egalitate când v este descoperit din u; < când v deja a fost descoperit înainte să se ajungă în u.
  - La terminarea BFS(s,G) există proprietatea dist(s,u) ≥ δ(s,u).
  - După orice execuție a ciclului principal al BFS, Q conține v1, v2, …, vp ai: Prop(Q) = dist(s,v1) ≤ dist(s,v2) ≤ ... ≤ dist(s,vp) ≤ dist(s,v1) + 1 => la un moment dat in coadă sunt elemente de pe același nivel din arborele generat de BFS (sau maxim 1 nivel diferență).
  - d(u) = momentul în care nodul u este inserat în coada Q. Atunci: d(u) < d(v) => dist(s,u) ≤ dist(s,v).
  - BFS este corect și după terminare δ(s,u) = dist(s,u), oricare u din V.

- Complexitate: O(n+m)
- Optimalitate: DA
- Parcurge tot graful? NU

⇨ DFS – parcurgerea in adancime

```
DFS(G)
- V = noduri(G)
- Pentru fiecare nod u (u ∈ V)
    - c(u) = alb; p(u) = null; // inițializare structură date
- timp = 0; // reține distanța de la rădăcina arborelui DFS până la nodul curent
- Pentru fiecare nod u (u ∈ V)
    - Dacă c(u) este alb
        - Atunci explorare(u); // explorez nodul

explorare(u)
- d(u) = ++ timp; // timpul de descoperire al nodului u
- c(u) = gri; // nod în curs de explorare
- Pentru fiecare nod v (v ∈ succs(u)) // încerc sa prelucrez vecinii
    - Dacă c(v) este alb
        - Atunci {p(v) = u; explorare(v);} // dacă nu au fost prelucrați deja
- c(u) = negru; // am terminat de explorat nodul u
- f(u) = ++ timp; // timpul de finalizare al nodului u
```

- Nu mai avem nod de start, nodurile fiind parcurse în ordine.
- Proprietati:
  - G = (V,E); u ∈ V; pentru fiecare v descoperit de DFS pornind din u este construită o cale v, p(v), p(p(v)),…, u.
  - G = (V,E); DFS(G) sparge graful G într-o pădure de arbori Arb(G) = { Arb(u); p(u) = null } unde Arb(u) = (V(u),E(u));
    - V(u) = { v | d(u) < d(v) < f(u) } + {u};
    - E(u) = { (v, z) | v, z ∈ V(u) && p(z) = v }.
  - Dacă DFS(G) generează 1 singur arbore => G este conex. Graf orientat conex înseamnă că prin transformarea arcelor în muchii se obține un graf neorientat conex.
  - Teorema parantezelor: Oricare u, v avem I(u) ∩ I(v) = 0 sau I(u) inclus I(v) sau I(v) inclus I(u).
  - Oricare u, v apartine V, atunci v apartine V(u) ⇔ I(v) inclus I(u).
  - Teorema drumurilor albe: G = (V,E); Arb(u); v este descendent al lui u in Arb(u) ⇔ la momentul d(u) există o cale numai cu noduri albe u..v.
  - Într-un graf neorientat, DFS poate descoperi doar muchii directe și inverse.
  - G = graf orientat; G ciclic ⇔ în timpul execuției DFS găsim arce inverse.

- Tipuri de arce:
  - Arc direct(de arbore) : intre nod gri si nod alb
  - Arc invers (de ciclu) : intre nod gri si nod gri
  - Arc inainte: nod gri si nod negru si d(u) < d(v)
  - Arc transversal: nod gri si nod negru si d(u) > d(v)

- Complexitate: O(n+m)
- Optimalitate: NU
- Parcurge tot graful? DA

Cursul 7 – Sortare Topologica, Componente Tare Conexe

⇨ Sortare Topologica:
- Se folosește la sortarea unei mulțimi parțial ordonate (nu orice pereche de elemente pot fi comparate).
- Fie A o mulțime parțial ordonată față de o relație de ordine ∝ (∝ ⊆ A*A) atunci ∃ e1 și e2 astfel încât e1, e2 nu pot fi comparate.
- O sortare topologică a lui A este o listă L = <e1,e2…en>, cu proprietatea că ⍰⍰ i, j, dacă ei ∝ ej, atunci i < j.
- G = (V,E) orientat, aciclic.
- VS – secvența de noduri a.î. oricare (u,v) apartine E, avem index(u) < index(v).
- Scop: Sortare_topologică(G) => VS.
- Idee bazată pe DFS: G = (V,E) orientat, aciclic; la sfârșitul DFS avem oricare(u,v) apartine E, f(v) < f(u) => colectăm în VS vârfurile în ordinea descrescătoare a timpilor f

- Sortare_topologică (G)
  - **Pentru fiecare** nod u (u ∈ V) {c(u) = alb;} // inițializări
  - $V_S = \varnothing$;
  - **Pentru fiecare** nod u (u ∈ V) // pentru fiecare componentă conexă
    - **Dacă** c(u) este alb
      - V_S = Explorează (u, V_S) // prelucrez componenta conexă
  - **Întoarce** V_S

  Explorează (u, V_S)
  - c(u) = gri // prelucrez nodul, deci îi actualizez culoarea
  - **Pentru fiecare** nod v (v ∈ succs(u))
    - **Dacă** c(v) este alb **atunci** V_S = Explorează (v, V_S) // recursivitate
    - **Dacă** c(v) este gri **atunci Întoarce** Eroare: graf ciclic
  - c(u) = negru // am terminat prelucrarea nodului
  - **Întoarce** cons(u, V_S) // inserează nodul u la începutul lui V_S

- Complexitate: O(n+m)

⇨ Componente tari conexe

- Fie G = (V,E) un graf orientat. G este tareconex ⇔ oricare u,v apartine V exista o cale u..v și o cale v..u (u apartine R(v) și v apartine R(u)).
- G = (V,E) graf orientat. G' = (V',E'), V'⊆V, E'⊆E. G' este o CTC a lui G ⇔ G' e tare-conex (oricare u,v apartine V', u apartine R(v) și v apartine R(u)) și G' este maximal (ca număr de noduri).
- Lema : G = (V,E) graf orientat, G' CTC => oricare u,v apartine V' avem oricare u..v din G are noduri exclusiv in V'
- G = (V,E) orientat, G' = (V',E') o CTC a lui G. Toate nodurile v apartine V' sunt grupate in același Arb(u) construit de DFS(G), unde u este primul nod descoperit al componentei.
- G = (V,E) orientat, u apartine V. Φ (u) = strămoș DFS al lui u determinat în cursul DFS(G) dacă: Φ (u) apartine R(u) ; f(Φ (u)) = max{f(v) | v apartine R(u)} ; Φ(u) = primul nod din CTC descoperit de DFS(G).
- Teorema: Φ (u) satisface următoarele proprietăți:
  - 1. f(u)≤f(Φ (u)) când e egalitate? U este primul nod din CTC
  - 2. Oricare v apartine R(u), f(Φ (v)) ≤ f(Φ (u)) ce înseamnă ca e egalitate? U si v sunt in aceeasi CTC
  - 3. Φ (Φ (u)) = Φ (u)
- Φ(u) apartine R(u) și f(Φ(u)) = max{f(v) | v apartine R(u)}.
- G = (V,E) orientat, oricare u apartine V, u este descendent al lui Φ(u) în Arb(Φ (u)) construit de DFS
- G = (V,E) orientat, oricare u,v apartine V; u și v aparțin aceleiași CTC ⇔ Φ(u) = Φ (v).
- Probleme:
  - vrem ca fiecare arbore construit să conțină o CTC.
  - trebuie să eliminăm nodurile care nu sunt în componenta conexă.
  - Idee: eliminăm nodurile ce nu aparțin CTC! Dacă aparțin Arb(u) și nu CTC => exista u..v și ∄ v..u. -> DFS pe graful transpus, în ordinea descrescătoare a timpilor de finalizare obținuți din DFS pe graful normal!
- Observatii:
  - Înlocuind componentele tare conexe cu noduri obținem un graf aciclic. (Pentru că altfel am avea o singură CTC!)

- Prima parcurgere DFS este o sortare topologică. (Pentru că sortează nodurile în ordinea inversă a timpilor de finalizare a strămoșilor fiecărei CTC!)

- Algoritmul lui Kosaraju:

  CTC(G)
  - DFS(G)
  - G^T = transpune(G)
  - DFS(G^T) (în bucla principală se tratează nodurile în ordinea descrescătoare a timpilor de finalizare de la primul DFS)

- Complexitate: O(n+m)
- Algoritmul CTC calculează corect componentele tare conexe ale unui graf G = (V,E).

Cursul 8 – Puncte de articulatie, Punti, Drumuri minime

⇨ Puncte de articulatie:

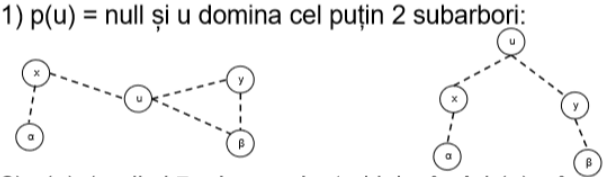
- Def: G = (V,E) graf neorientat, u apartine V. U este punct de articulație dacă oricare x,y apartine V, x ≠ y, x ≠ u, y ≠ u, a.î. oricare x..y în G trece prin u.
- Algoritm naiv:

  Elimină fiecare nod și verifică conectivitatea grafului rezultat:
  - Graf conex → nodul nu e punct de articulație.
  - Altfel → punct de articulație.

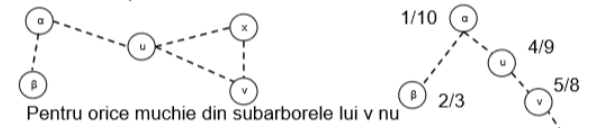
  Complexitate?
  O(V(V+E))

- Teorema: G = (V,E), graf neorientat și u apartine V. U este punct de articulație în G ⇔ în urma DFS în G una din proprietățile de mai jos este satisfăcută:
  - p(u) = null și u domină cel puțin 2 subarbori;
  - p(u) ≠ null și exista v descendent al lui u în Arb(u) a.î. oricare x apartine Arb(v) și oricare (x,z) parcursă de DFS(G) avem d(z) ≥ d(u).

  1) p(u) = null și u domina cel puțin 2 subarbori:

  2) p(u) ≠ null și ∃v descendent al lui u în Arb(u) a.î. ∀x∈Arb(v) și ∀(x,z) parcursă de DFS(G) d(z) ≥ d(u):

  Pentru orice muchie din subarborele lui v nu există nici o muchie înapoi spre u sau spre un nod descoperit înaintea lui u.

- Algoritm Tarjan:
  - Se aplică DFS și se salvează pentru fiecare nod până unde merge înapoi (low): low[u] = min {d(u), d(v) pentru toate muchiile înapoi (u,v), low(w) pentru toți fiii w ai lui u}.

- Pentru eficiență, trebuie ca fiii să se parcurgă înaintea părinților -> ordinea inversă a d(u)
-

```
Articulații (G)
- V = noduri(G) // inițializări
- Timp = 0;
- Pentru fiecare (u ∈ V)
  - c(u) = alb;
  - d(u) = 0;
  - p(u) = null;
  - low(u) = 0;
  - subarb(u) = 0; // reține numărul de subarbori dominați de u
  - art(u) = 0; // reține punctele de articulație
- Pentru fiecare (u∈V)
  - Dacă c(u) e alb
    - Exploreaza(u);
    - Dacă (subarb(u) > 1) // cazul în care u este rădăcina în arborele
      - art(u) = 1          // DFS și are mai mulți subarbori → cazul
                            // 1 al teoremei
```

```
Explorează(u)
- d(u) = low(u) = timp++; // inițializări
- c(u) = gri;
- Pentru fiecare nod v ∈ succs(u)
  - Dacă (c(v) e alb)
    - p(v) = u; subarb(u)++; // actualizare nr subarbori
                             // dominați de u
    - Explorează(v);
    - low(u) = min{low(u), low(v)} // actualizare low
    - Dacă (p(u) != null && low(v) ≥ d(u)) art(u) = 1;
                             // cazul 2 al teoremei
  - Altfel low(u) = min{low(u), d(v)} // actualizare low
```

- Algoritm Tarjan pentru CTC

```
index = 0  // nivelul pe care este nodul în arborele DFS
S = empty // se folosește o stivă care se inițializează cu Ø
Pentru fiecare v din V
  - Dacă (v.index e nedefinit) atunci // se pornește DFS din fiecare nod pe care
    - Tarjan(v)                       // nu l-am vizitat încă

Tarjan(v)
  - v.index = index // se setează nivelul nodului v
  - v.lowlink = index // reține strămoșul nodului v
  - index = index + 1  // incrementez nivelul
  - S.push(v) // introduc v în stivă
  - Pentru fiecare (v, v') din E // se prelucrează succesorii lui v
    - Dacă (v'.index e nedefinit sau v' e în S) atunci // CTC deja identificate sunt ignorate
      - Dacă (v'.index e nedefinit) atunci Tarjan(v') // dacă nu a fost vizitat v' intru în  recursivitate
      - v.lowlink = min(v.lowlink, v'.lowlink) //actualizez strămoșul
  - Dacă (v.lowlink == v.index) atunci // printez CTC începând de la coadă spre rădăcină
    - print "CTC:"
    - Repetă
      - v' = S.pop // extrag nodul din stiva și îl printez
      - print v'
    - Până când (v' == v) // până când extrag rădăcina
```

⇨ Punti

- Def: G = (V,E), graf neorientat și (u,v) ∈ E. (u,v) este punte în G ⇔ exista x,y ∈ V, x ≠ y, a.î. oricare x..y conține muchia (u,v).

- Algoritm:

```
Punți(G)
- V = noduri(G) // inițializări
- Timp = 0;
- Pentru fiecare nod u (u ∈ V)
  - c(u) = alb;
  - d(u) = 0;
  - p(u) = null;
  - low(u) = 0;
  - punte(u) =0; // înlocuiește: subarb(u) = 0; art(u) = 0;
- Pentru fiecare nod u (u ∈ V)
  - Dacă c(u) e alb
    - Explorează(u)
```

```
Explorează(u)
- d(u) = low(u) = timp++; // inițializări
- c(u) = gri;
- Pentru fiecare nod v (v ∈ succs(u))
  - Dacă c(v) e alb
    - p(v) = u; // se elimină: subarb(u)++;
    - Explorează(v);
    - low(u) = min{low(u), low(v)} // actualizare low
    - Dacă (low(v) > d(u)) punte(v) = 1;
    // în loc de: Dacă(p(u) != null && low(v) >= d(u))
  - Altfel
    - Dacă (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low
```

⇨ Drumuri de cost minim
- Variante de calcul:
  - Drumuri punct – multipunct: pentru un nod dat s ∈ V, să se găsească un drum de cost minim de la s la oricare u ∈ V; **Dijkstra, Bellman-Ford**
  - Drumuri multipunct – punct: pentru un nod dat e ∈ V, să se găsească un drum de cost minim de la oricare u ∈ V la e; **G^T si apoi 1**
  - Drumuri punct – punct: pentru două noduri date u și v ∈ V, să se găsească un drum u..v de cost minim; **Folosind 1**
  - Drumuri multipunct – multipunct: ∀u, v ∈ V, să se găsească un drum u..v de cost minim. **Floyd-Warshall**
- Subdrumurile unui drum minim sunt drumuri optimale: G = (V,E), w : E -> R funcție de cost asociată. Fie p = v1v2...vk un drum optim de la v1 la vk. Atunci pentru orice i și j cu 1≤ i ≤ j ≤ k, subdrumul lui p de la vi la vj este un drum minim.
- G = (V,E), w : E -> R funcție de cost asociată. Fie p = s..uv un drum optim de la s la v. Atunci costul optim al acestui drum poate fi scris ca δ(s,v) = δ(s,u) + w(u,v)
- G = (V,E), w : E -> R funcție de cost asociată. ∀ (u,v) ∈ E avem δ(s,v) ≤ δ(s,u) + w(u,v).
- Drumuri minime de sursa unica:
  - Pentru grafuri orientate
  - Bazati pe Greedy
  - Se pornește de la nodul de start și pe baza unui optim local, drumurile sunt extinse și optimizate până la soluția finală.
  - Relaxarea arcelor -> dacă d[v] > d[u] + w(u,v), atunci actualizează d[v].
  - **Dijkstra, Bellman-Ford**
  - G = (V,E), w : E -> R funcție de cost asociată. ∀ v ∈ V, d[v] obținut de algoritmul lui Dijkstra respectă d[v] ≥ δ(s,v). În plus, odată atinsă valoarea δ(s,v), ea nu se mai modifică.
  - G = (V,E), w : E -> R funcție de cost asociată. Fie p = s..uv un drum optim de la s la v. Dacă d[u] = δ(s,u) la un moment dat, atunci începând cu momentul imediat următor relaxării arcului (u,v) avem d[v] = δ(s,v)

⇨ **DIJKSTRA**
- Folosește o coadă de priorități în care se adaugă nodurile în funcție de distanța cunoscută în momentul respectiv de la s până la nod.
- Se folosește NUMAI pentru costuri pozitive (w(u,v) > 0, orice u,v apartine V).
-

```
Dijkstra_generic (G,s)
  V = nodurile lui G
  Cât timp (V != ∅)
    u = nod din V cu d[u] min
    V = V - {u}
    Pentru fiecare (v ∈ succesorii lui u) relaxare_arc(u,v)
  // optimizare drum s..v pentru v ∈ succesorilor lui u
```

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V) // inițializări
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele

- Complexitate:
  - Operații ce trebuie realizate pe coadă + frecvența lor:
    - insert – V;
    - delete – V;
    - conține? – E;
    - micșorează_val – E;
    - este_vidă? – V.
- Complexitate:
  - Vector:
    - insert – 1 * V = V;
    - delete – V * V = V2 (necesită căutarea minimului);
    - conține? – 1 * E = E;
    - micșorează_val – 1 * E = E;
    - este_vidă? – 1 * V = V;
  - Heap binar:
    - structură de date de tip arbore binar + 2 constrângeri:
    - Fiecare nivel este complet; ultimul se umple de la stânga la dreapta; oricare u ∈ Heap; u ≥ răd(st(u)) și u ≥ răd(dr(u)) (u este ≥ decât ambii copii ai săi) unde ≥ este o relație de ordine pe mulțimea pe care sunt definite elementele heapului.
    - insert – logV * V = VlogV;
    - delete – logV * V = VlogV;
    - conține? – 1 * E = E;
    - micșorează_val – logV * E = ElogV;
    - este_vidă? – 1 * V = V.
    - Eficient dacă graful are arce puține comparativ cu numărul de noduri.
  - Heap Fibonacci
    - Poate fi format din mai mulți arbori.
    - Cheia unui părinte ≤ cheia oricărui copil
    - Fiind dat un nod u și un heap H:
      - p(u) – părintele lui u;
      - copil(u) – legătura către unul din copiii lui u;
      - st(u), dr(u) – legătura la frații din stânga și din dreapta (cei de pe primul nivel sunt legați între ei astfel);

        - grad(u) – numărul de copii ai lui u;
        - min(H) – cel mai mic nod din H;
        - n(H) – numărul de noduri din H.
    - insert – 1 * V = V;
    - delete – logV * V = VlogV(amortizat!);
    - micșorează_val – 1 * E = E;
    - este_vidă? – 1 * V = V.
    - Cea mai rapidă structură dpdv teoretic.
- Implementarea trebuie realizată în funcție de tipul grafului pe care lucrăm:
    - vectori pentru grafuri “dese” - O(V2);
    - heap pentru grafuri “rare”: HB - O(E logV), HF - O(V log V+E)
- Heapul Fibonacci este mai eficient decât heapul binar dar mai dificil de implementat.

Curs 9 – Drumuri de cost minim

⇨ **DIJKSTRA**

- G = (V,E), w : E -> R funcție de cost asociată nenegativă. La terminarea aplicării algoritmului Dijkstra pe acest graf plecând din sursa s vom avea d[v] = δ(s,v) pentru oricare v apartine V.
- Algoritmul nu funcționează pentru grafuri ce conțin arce de cost negativ!

⇨ **BELLMAN-FORD**

- Cicluri de cost negativ
-

```
BellmanFord(G,s)   // G=(V,E),s=sursa
  Pentru fiecare nod v (v ∈ V) // inițializări
    d[v] = ∞;
    p[v] = null;
  d[s] = 0; // actualizare distanță de la s la s
  Pentru i de la 1 la |V| -1 // pentru fiecare pas pornind din s
  // spre restul nodurilor se încearcă construcția unor drumuri
  // optime de dimensiune i
    Pentru fiecare (u,v) din E
    // pentru arcele ce pleacă de la nodurile deja considerate
      Dacă d[v] > d[u] + w(u,v) atunci // se relaxează arcele corespunzătoare
        d[v] = d[u] + w(u,v);
        p[v] = u;
  Pentru fiecare (u,v) din E
    Dacă d[v] > d[u] + w(u,v) atunci
      Eroare ("ciclu negativ");
```

```
BellmanFordOpt2(G,s)
  Pentru fiecare nod v (v ∈ V)
    d[v] = ∞;
    p[v] = null;
    marcat[v] = false; // marcăm nodurile pentru care am făcut relaxare
  Q = ∅; // coadă cu priorități
  d[s] = 0; marcat[s] = true; Introdu(Q,s);
  Cât timp (Q != ∅)
    u = ExtrageMin(Q);   marcat[u] = false; // extrag minimul
    Pentru fiecare nod v (v ∈ succesorilor lui u)
      Dacă d[v] > d[u] + w(u,v) atunci // relaxez arcele ce pleacă din u
        d[v] = d[u] + w(u,v);
        p[v] = u;
        Dacă (marcat[v] == false) {marcat[v] = true; Introdu(Q,v);}
```

- Complexitate:
    - Caz defavorabil:

Pentru i de la 1 la |V| - 1 -> V *
Pentru fiecare (u,v) din E -> E
Dacă d[v] > d[u] + w(u,v) atunci
d[v] = d[u] + w(u,v);
p[v] = u; => **O(V*E)**

- G = (V,E), w : E -> R funcție de cost asociată; dacă G nu conține ciclu de cost negativ atunci după |V| - 1 iterații ale relaxării fiecărui arc avem d[v] = δ(s,v) pentru oricare v apartine R(s).
- G = (V,E), w : E -> R funcție de cost asociată. Algoritmul Bellman-Ford aplicat acestui graf plecând din sursa s nu returnează EROARE dacă G nu conține cicluri negative, iar la terminare d[v] = δ(s,v) pentru oricare v apartine V. Dacă G conține cel puțin un ciclu negativ accesibil din s, atunci algoritmul întoarce EROARE.
- Observatii:
  - Dacă d[v] nu se modifică la pasul i atunci nu trebuie sa relaxăm niciunul din arcele care pleacă din v la pasul i + 1. => păstrăm o coadă cu vârfurile modificate (o singură copie).

⇨ **FLOYD-WARSHALL**

- Algoritm prin care se calculează distanțele minime între oricare 2 noduri dintr-un graf (drumuri optime multipunct-multipunct).
- Exemplu clasic de programare dinamică.
- Idee: la pasul k se calculează cel mai bun cost între u și v folosind cel mai bun cost u..k și cel mai bun cost k..v calculat până în momentul respectiv.
- Se aplică pentru grafuri ce nu conțin cicluri de cost negativ.
- Fie formulele de mai jos pentru calculul valorii dk(i,j), $0 \le k \le n$:

  d0(i,j) = w(i,j);

  dk(i,j) = min{dk-1(i,j), dk-1(i,k) + dk-1(k,j)}, pentru $0 < k \le n$;

  Atunci dn(i,j) = δ(i,j), pentru ∀ i, j ∈ V

- Algoritm:

Floyd-Warshall2(G)
- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n // inițializări
    - d(i,j) = w(i,j)
    - **Dacă** (w(i,j) == ∞)
      - p(i,j) = null;
    - **Altfel** p(i,j) = i;
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - **Dacă** (d(i,j) > d(i,k) + d(k,j)) // determinăm minimul
        - d(i,j) = d(i,k) + d(k,j)
        - p(i,j) = p(k,j); // și actualizăm părintele

Complexitate? $O(V^3)$

Complexitate spațială? $O(V^2)$

⇨ Inchiderea tranzitiva

- Fie G = (V,E). Închiderea tranzitivă a lui E e un G* = (V,E*), unde

  E* (I,j) = 1, dacă ∃ i..j 0, dacă ∄ i..j
- Poate fi determinată prin modificarea algoritmului Floyd-Warshall:
  - min ⟹ operatorul boolean sau (ᵛ)
  - + ⟹ operatorul boolean și (^)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - $E^*(i,j) = (((i,j) \in E) \vee (i = j))$ // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - $E^*(i,j) = E^*(i,j) \vee (E^*(i,k) \wedge E^*(k,j))$

Complexitate? $O(V^3)$ Complexitate spațială? $O(V^2)$

⇨ **JOHNSON**

- Pentru grafuri rare.
- Folosește liste de adiacență.
- Bazat pe Dijkstra și Bellman-Ford.
- Complexitate: O(V^2*logV + VE) => Mai bună decât Floyd-Warshall pentru grafuri rare.
- Idee algoritm:
  - Dacă graful are numai arce pozitive: se aplică Dijkstra pentru fiecare nod ⟹ cost V^2*logV.
  - Altfel se calculează costuri pozitive pentru fiecare arc menținând proprietățile:
    - w1(u,v) ≥ 0, oricare (u,v) apartine E;
    - p este drum minim utilizând w ⟺ p este drum minim utilizând w1.
- Johnson(G)
  - // Construim G' = (V',E');
  - V' = V ∪ {s}; // adăugăm nodul s
  - E' = E ∪ (s,u), ∀u∈V; w(s,u) = 0; // și îl legăm de toate nodurile
  - **Dacă** BF(G',s) e fals // aplic BF pe G'
    - **Eroare** "ciclu negativ"
  - **Altfel**
    - **Pentru fiecare** v ∈ V
      - h(v) = δ(s,v); // calculat prin BF
    - **Pentru fiecare** (u,v) ∈ E
      - $w_1(u,v) = w(u,v) + h(u) - h(v)$ // calculez noile costuri pozitive
    - **Pentru fiecare** (u ∈ V)
      - Dijkstra(G,w_1,u) // aplic Dijkstra pentru fiecare nod
      - **Pentru fiecare** (v ∈ V)
        - $d(u,v) = \delta_1(u,v) + h(v) - h(u)$ // calculez costurile pe graful inițial

⇨ Concluzii **FLOYD-WARSHALL & JOHNSON**

- Algoritmi ce găsesc drumurile minime între oricare 2 noduri din graf.
- Funcționează pe grafuri cu arce ce au costuri negative (dar care nu au cicluri de cost negativ).
- Floyd-Warshall e optim pentru grafuri dese.
- Johnson e mai bun pentru grafuri rare.

Cursul 10 – Arbori minimi de acoperire

⇨ Definitii:

- Un arbore liber al lui G este un graf neorientat conex și aciclic Arb = (V',E'); V' ⊆ V, E' ⊆ E. Costul arborelui este: C(Arb) = Σ w(e), e apartine E'.
- Un arbore liber se numește arbore de acoperire dacă V' = V.
- Un arbore de acoperire (Arb) se numește arbore minim de acoperire (notăm AMA) dacă Arb apartine ARB(G) a.î. C(Arb) = min{C(Arb') | Arb' ∄ ARB(G)}.
- Fie A ⊆ E o mulțime de muchii ale unui graf G = (V,E) și (S,V-S) o partiționare a lui V. Partiționarea respectă mulțimea A dacă ∄ e apartine A care taie frontiera dintre S și V-S (∀ (u,v) apartine A → u,v apartine S sau u,v apartine V-S).
- Fie A ⊆ E' o mulțime de muchii ale unui AMA parțial Arb = (V,E') al grafului G = (V,E), iar e apartine E o muchie oarecare din G. Muchia e este sigură în raport cu A dacă mulțimea A U {e} face parte dintr-un AMA al lui G.
- Fie A o mulțime de muchii ale unui AMA parțial al grafului G = (V,E). Fie (S,V-S) o partiționare care respectă A, iar (u,v) apartine E o muchie care taie frontiera dintre S și V-S a.î. w(u,v) = min{w(x,y) | (x,y) apartine E și (x apartine S, y apartine V-S) sau (x apartine V-S, y apartine S)} Muchia (u,v) este sigură in raport cu A.

⇨ Utilizari:
- Proiectarea retelelor: electrice, calculatoare, drumuri
- Clustering
- Algoritmi de aproximare pentru probleme NP-complete.

⇨ Proprietati:
- G = (V,E), C = (V',E') – ciclu în G; e apartine E' a.î. w(e) = max {w(e') | e' apartine E'} => e !apartine Arb(G) unde Arb(G) = AMA în G.
- G = (V,E), S = (V',E') un AMA parțial al lui G, V' ⊆ V; e = (u,v) a.î. e !apartine E' și (u apartine V' și v !apartine V') sau (u !apartine V' și v apartine V') cu proprietatea că: w(u,v) = min{w(u',v')| (u' apartine V' și v' !apartine V') sau (u' !apartine V' și v apartine V')} => (u,v) apartine AMA.

⇨ Algoritmi de tip Greedy:
- Prim: se pornește cu un nod și se extinde pe rând cu muchiile cele mai ieftine care au un singur capăt în mulțimea de muchii deja formată (Proprietatea 2). Algoritmul este asemănător algoritmului Dijkstra.
- Kruskal: inițial toate nodurile formează câte o mulțime și la fiecare pas se reunesc 2 mulțimi printr-o muchie. Muchiile sunt considerate în ordinea costurilor și sunt adăugate în arbore doar dacă nu creează ciclu (Proprietatea 1).
- **Prim:**
  - Prim(G,w,s) **Complexitate?**
    - A = ∅ // AMA
    - **Pentru fiecare** (u ∈ V)
      - d[u] = ∞; p[u] = null // inițializăm distanța și părintele
    - d[s] = 0; // nodul de start are distanța 0
    - Q = constrQ(V, d); // ordonată după costul muchiei // care unește nodul de AMA deja creat
    - **Cât timp** (Q != ∅) // cât timp mai sunt noduri neadăugate
      - u = ExtrageMin(Q); // extrag nodul aflat cel mai aproape
      - A = A ∪ {(u,p[u])}; // adaug muchia în AMA
      - **Pentru fiecare** (v ∈ succs(u))
        - **Dacă** d[v] > w(u,v) **atunci**
          - d[v] = w(u,v); //+ d[u] // actualizăm distanțele și părinții nodurilor
          - p[v] = u; // adiacente care nu sunt în AMA încă
    - **Întoarce** A - {(s,p(s))} // prima muchie adăugată
  - Complexitate:

- Matrice de adiacență O(V^2)
- Heap binar O(E^logV)
- Heap Fibonacci O(V*logV + E)

- Observatii:
  - Grafuri dese -> Matrice de adiacență preferată
  - Grafuri rare -> Heap binar sau Fibonacci

- **Kruskal:**
  -

Kruskal(G,w)
- A = ∅; // AMA
- **Pentru fiecare** (v ∈ V)
  - Constr_Arb(v) // creează o mulțime formată din nodul respectiv // (un arbore cu un singur nod)
- Sortează_asc(E,w) // se sortează muchiile în funcție de // costul lor
- **Pentru fiecare** ((u,v) ∈ E) // muchiile se extrag în ordinea // costului
  - **Dacă** Arb(u) != Arb(v) **atunci** // verificăm dacă se creează ciclu
    - Arb(u) = Arb(u) ∪ Arb(v) // se reunesc mulțimile de noduri (arborii)
    - A = A ∪ {(u,v)} // se adaugă muchia sigură în AMA
- **Întoarce** A

Complexitate?

  - Complexitate:
    - Sortarea muchiilor: O(ElogE) ≈ O(ElogV)
    - Arb(u) = Arb(v) – compararea a 2 mulțimi disjuncte {1,2,3} {4,5,6} – mai precis trebuie identificat dacă 2 elemente sunt în aceeași mulțime
    - Arb(u) U Arb(v) – reuniunea a 2 mulțimi disjuncte într-una singură

  - Implementare rapida multimi disjuncte
    -

Comparare (u, v)
- Verifică dacă 2 noduri au aceeași rădăcină;
- Implică identificarea rădăcinii:

Arb(u) // identificarea rădăcinii unei componente
- **Cât timp** (i != id[i]) i = id[i];
- **Întoarce** i

Comparare (u, v)
- **Întoarce** Arb(u) != Arb(v)

Complexitate?

Reuniune (u,v) // implică identificarea rădăcinii
- v = Arb(v)
- id[v] = u;

    - Complexitate:
      - Compararea – O(V)
      - Reuniunea – O(V)

- Optimizare-reuniune rapida balansata
    - Se menține numărul de noduri din fiecare subarbore
    - Se adaugă arborele mic la cel mare pentru a face mai puține căutări → înălțimea arborelui e mai mică și numărul de căutări scade de la V la lg V.
    - Complexitate:
        - Compararea – O(lg V)
        - Reuniune – O(lg V)
- Complexitate dupa optimizare:
    - Orice secvență de E operații de căutare și reuniune asupra unui graf cu V noduri consumă O(V + E*α(V,E)).
    - α – de câte ori trebuie aplicat lg pentru a ajunge la 1. (in practica ≤ 5)
    - In practica: O(E)

▪ Complexitate dupa optimizari:
- Max (complexitate sortare, complexitate operații mulțimi) = max(O(ElogV), O(E)) = O(ElogV)
- Complexitatea algoritmului Kruskal este dată de complexitatea sortării costurilor muchiilor.

o Aplicatie practica- K-clustering
▪ Împărțirea unui set de obiecte în k grupuri astfel încât obiectele din cadrul unui grup să fie “apropiate” considerând o “distanță” dată.
▪ Utilizat în clasificare, căutare (web search de exemplu).
▪ Dându-se un întreg K și un grup de obiecte, se cere să se împartă grupul de obiecte în K grupuri astfel încât spațiul dintre grupuri să fie maximizat.
▪ Algoritm:
- Se formează V clustere (un cluster per obiect).
- Găsește cele mai apropiate 2 obiecte din clustere diferite și unește cele 2 clustere.
- Se oprește când au mai rămas k clustere.
- => Kruskal

Cursul 11 – Retele de flux. Flux maxim

⇨ Flux
o Proprietati:
▪ Oricare u, v apartine V
▪ $f(u,v) \leq c(u,v)$ (fluxul printr-un arc este mai mic sau egal cu capacitatea arcului) – respectarea capacității arcelor;
▪ Oricare u, v apartine V, $f(u,v) = -f(v,u)$ – simetria fluxului;
▪ $\Sigma f(u,v) = 0$ pentru oricare u apartine V \ {s,t} – conservarea fluxului.
o Notatii:
▪ $fi(u) = \Sigma f(v,u)$ – fluxul total care intra în nodul u;
▪ $fo(u) = \Sigma f(u,v)$ – fluxul total care iese din nodul u;
▪ Valoarea totală a fluxului:
- $|f| = \Sigma f(s,v) = fo(s)$;

- |f| = fluxul ce părăsește sursa;
- |f| = Σf(s,v) = Σf(v,t) = fi(t).

- Se adaugă o sursă unică cu arce de capacitate infinită spre sursele s1..sn și flux egal cu fluxul generat de sursele respective;
- Se adaugă o destinație unică t și arce de capacitate infinită între t1..tm și t și flux egal cu fluxul ce intră în destinațiile respective.

⇨ Arc rezidual
- Un arc (u,v) pentru care f(u,v) < c(u,v) se numește arc rezidual. => Fluxul pe acest arc se poate mări.

⇨ Capacitate reziduala
- Cantitatea cu care se poate mări fluxul pe arcul (u,v) se numește capacitatea reziduală a arcului (u,v) (cf(u,v)): cf(u,v) = c(u,v) - f(u,v)

⇨ Retea reziduala:
- Rețeaua reziduală (Gf = (V,Ef)) este o rețea de flux formată din arcele ce admit creșterea fluxului: Ef = {(u,v) ∈ V x V | cf(u,v) > 0}.
- Ef ⊄ E
- Fie G = (V,E) rețea de flux, f fluxul în G, Gf rețeaua reziduală a lui G. Fie f' un flux prin Gf și f+f' o funcție definită astfel: f+f' (u,v) = f(u,v) + f'(u,v). Atunci f+f' reprezintă un flux în G și |f+f'| = |f| + |f'| => cum putem mări fluxul printr-o rețea de flux.
- G – rețea de flux, f flux în G, p = s..t – cale reziduală în Gf, fp:V x V-> R se definește ca fiind:
  - fp(u,v) = cf(p), dacă (u,v) apartine p; -cf(p), dacă (v,u) apartine p 0; dacă (u,v) și (v,u) !apartine p

  fp = flux în Gf; |fp| = cf(p)
- f' = f + fp = flux în G, astfel încât |f'| = |f| + |fp| > |f|. => cum se defineste fluxul printr-o retea reziduala

⇨ Cale reziduala:
- Cale reziduală e un drum s..t ⊆ Gf, unde cf(u,v) este capacitatea reziduală a arcului (u,v).
- Capacitatea reziduală a căii = capacitatea reziduală minimă de pe calea s..t descoperită.

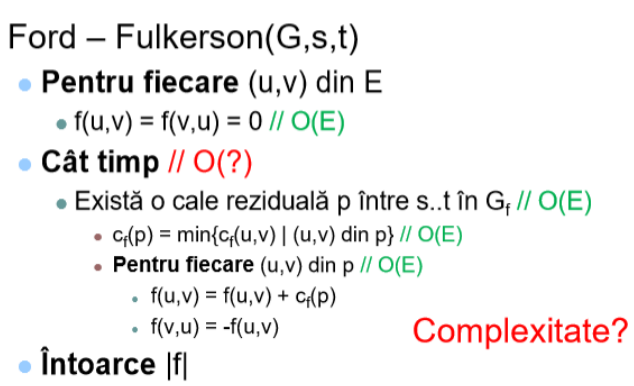
⇨ Calculul fluxului maxim: **FORD-FULKERSON**
-

Metoda Ford-Fulkerson
- f(u,v) = 0 ∀ (u,v) // inițializarea fluxului
- **Repetă** // creștere iterativă a fluxului
  - găsește un drum s..p..t pe care se poate mări fluxul (cale reziduală)
  - f = f + flux(s..p..t)
- **Până când** nu se mai poate găsi nici un drum s..p..t
- **Întoarce** f

⇨ Taieturi in retele de flux:
- O tăietură (S,T) a unei rețele de flux G = partiționare a nodurilor în 2 mulțimi disjuncte S și T = V \ S astfel încât s apartine S și t apartine T.
  - f(S,T) = ΣxΣy f(x,y) (x apartine S, y apartine T) – fluxul prin tăietura
  - c(S,T) = ΣxΣy c(x,y) (x apartine S, y apartine T) – capacitatea tăieturii
- Fluxul prin tăietură = fluxul prin rețea – f(S,T) = |f|

- S, T – tăietură oarecare – fluxul maxim este limitat superior de capacitatea tăieturii $|f| \leq c(S,T)$

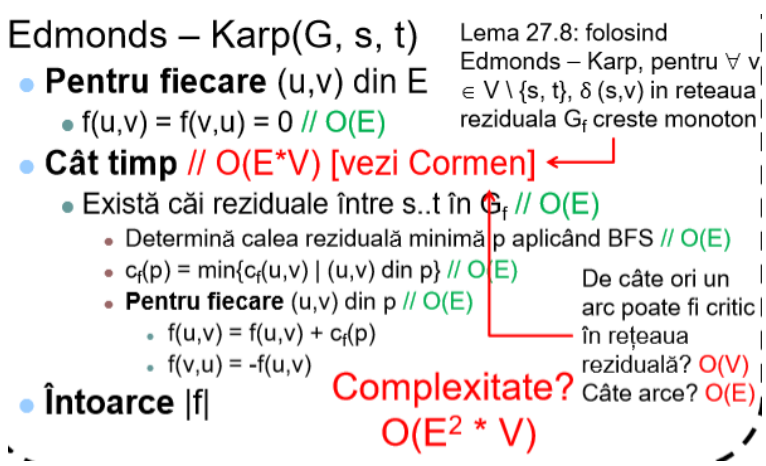
⇨ Flux maxim – taietura minima

- G = (V,E) rețea de flux – următoarele afirmații sunt echivalente:
  - f este o funcție de flux în G astfel încât |f| este flux maxim total în G;
  - rețeaua reziduală Gf nu are căi reziduale;
  - există o tăietură (S,T) astfel încât |f| = c(S,T).
- 

- Complexitate: O(E*fmax) ; fmax = fluxul maxim
- Probleme:
  - Se folosesc căi cu capacitate mică;
  - Se pun fluxuri pe mai multe arce decât este nevoie.

- Imbunatatiri:
  - Se aleg căile reziduale cu capacitate maximă – complexitatea va depinde în continuare de fmax și de valoarea capacităților;
  - Se aleg căile reziduale cele mai scurte → în acest caz complexitatea nu mai depinde de fmax ci numai de numărul de arce (ex. Edmonds-Karp: identificarea căilor reziduale minime prin aplicarea unui BFS)

⇨ **EDMONDS-KARP**

- 

⇨ Pompare preflux

- Simularea curgerii lichidelor într-un sistem de conducte ce leagă noduri aflate la diverse înălțimi;
- Sursa – înălțime |V|
- Inițial toate nodurile exceptând sursa sunt la înălțime 0

- Destinația rămâne în permanență la înălțimea 0
- Există un preflux inițial în rețea obținut prin încărcarea la capacitate maximă a tuturor conductelor ce pleacă din s;
- Excesul de flux dintr-un nod poate fi stocat întrun rezervor al nodului (Notat e(u));
- Când un nod u are flux disponibil în rezervor și o conductă spre un alt nod v nu este încărcată complet → înălțimea lui u este crescută pentru a permite curgerea din u în v.
- Definitii
  - Preflux = f: V x V → R astfel încât să fie satisfăcute restricțiile:
    - (u,v) ≤ c(u,v) (fluxul printr-un arc este mai mic sau egal cu capacitatea arcului) – respectarea capacității arcelor;
    - Oricare u, v apartine V, f(u,v) = -f(v,u) – simetria fluxului;
  - Supraîncărcare a unui nod: $e(u) = f(V,u) \geq 0$, oricare u apartine $V \setminus \{s\}$.
  - O funcție h: V → N este o funcție de înălțime dacă îndeplinește restricțiile:
    - h(s) = |V| – fixă;
    - h(t) = 0 – fixă;
    - h(u) ≤ h(v) + 1 pentru orice arc rezidual (u,v) apartine Gf – variabilă.
  - G – rețea de flux, h: V → N este o funcție de înălțime. Dacă oricare u, v apartine V, h(u) > h(v) + 1 atunci arcul (u,v) nu este arc rezidual.

- Algoritm:

Init_preflux(G, s, t)
- **Pentru fiecare** (u ∈ V)
  - e(u) = 0 // inițializare exces flux în nodul u
  - h(u) = 0 // inițializare înălțime nod u
- **Pentru fiecare** ((u,v) ∈ E) // inițializare fluxuri
  - f(u,v) = 0
  - f(v,u) = 0
- h(s) = |V| // inițializare înălțime sursă
- **Pentru fiecare** (u ∈ succs(s) \ {s})
  - f(s,u) = c(s,u); // actualizare flux
  - f(u,s) = -c(s,u);
  - e(u) = c(s,u) // actualizare exces

Complexitate?

Pompare_preflux(G, s, t)
- Init_preflux(G, s, t) // inițializarea prefluxului
- **Cât timp** (1) // cât timp pot face pompări sau înălțări
  - **Dacă** (∃u ∈ V \ {s, t}, v ∈ V | e(u) > 0 **și** c_f(u,v) > 0 **și** h(u) = h(v) + 1) // încerc să pompez
    - Pompare(u,v); **continuă;**
  - **Dacă** (∃u ∈ V \ {s, t}, v ∈ V | e(u) > 0 **și** ∀(u,v) ∈ E_f, h(u) ≤ h(v))
    - Înălțare(u); **continuă;** // încerc să înalț
  - **Întrerupe;** // nu mai pot face nimic → am ajuns la flux max
- **Întoarce** e(t) // e(t) = |f| = fluxul total în rețea

- Complexitate:
  - Init_preflux: O(E)
  - Pompare(u,v): O(1)
  - Înălțare(u): O(V) – implică găsirea minimului dintre nodurile succesoare
  - Cât timp: [vezi Cormen]
    - Câte înălțări?
      - Care e înălțimea maximă? 2 |V| - 1 – drum rezidual de lungime maximă
      - Care este numărul maxim total de înălțări? (2 |V| - 1) (|V| - 2)
    - Câte pompări?
      - Pompări saturate: 2 |V| |E| - de câte ori un arc poate fi saturat? (în funcție de suma h(u) + h(v))

        - Pompări nesaturate: 4 |V|2 (|V| + |E|) – sumă înălțimi noduri excedentare
    - Complexitate totală: O(V2 * E)

Cursul 12 - Algoritmi euristici de explorare

⇨ Explorarea spațiului stărilor problemei
- Stare a problemei = abstractizare a unei configurații valide a universului problemei, configurație ce determină univoc comportarea locală a fenomenului descris de problemă.
- Spațiul stărilor = graf în care nodurile corespund stărilor problemei, iar arcele desemnează tranzițiile valide între stări.
    - Caracteristică importantă: nu este cunoscut apriori, ci este descoperit pe măsura explorării!
    - Descriere:
        - Nodul de start (starea inițială);
        - Funcție de expandare a nodurilor (produce lista nodurilor asociate stărilor valide în care se poate ajunge din starea curentă);
        - Predicat de testare dacă un nod corespunde unei stări soluție.

- Obiectivele navigarii:
    - Cartografierea sistematică a spațiului stărilor.
    - Asamblarea soluțiilor parțiale care în final conduc la soluția finală. Această soluție finală poate fi:
        - Identificarea stărilor soluție (poziționarea a n regine pe tabla de șah fără să se atace);
        - Drumul străbătut de la starea inițială spre o stare soluție (acoperirea tablei de șah cu un cal);
        - Strategia de rezolvare = arbore multicăi în care rădăcina este starea inițială, iar frunzele sunt stări soluție. În acest arbore, unele noduri corespund unor evenimente neprevăzute care influențează calea de urmat în rezolvare (identificarea monedei false dintr-un grup de 3 monede).

- Algoritmi tentativi/irevocabili:
    - Dacă explorarea se bazează pe informația acumulată în cursul explorării, informație prelucrată euristic (costuri)→ algoritm informat.
    - Dacă explorarea este 'la întâmplare' → algoritm neinformat.
    - Dacă algoritmul de explorare are posibilitatea să abandoneze calea curentă de rezolvare și să revină la o cale anterioară → algoritmi tentativi.
    - Altfel (algoritmul avansează pe o singură direcție) → algoritmi irevocabili.

- Cautari informate vs neinformate
    - Căutările informate beneficiază de informații suplimentare pe care le colectează și le utilizează în încercarea de a ghici direcția în care trebuie explorat spațiul stărilor pentru a găsi soluția.
    - Aceste informații sunt stocate:

- În nodurile din spațiul stărilor:
  - Starea problemei reprezentată de nod;
  - Părintele nodului curent;
  - Copii nodului curent (obținuți prin expandarea acestuia);
  - Costul asociat nodului curent care estimează calitatea nodului f(n);
  - Adâncimea de explorare.
- În structuri auxiliare pentru diferențierea nodurilor în raport cu gradul de prelucrare:
  - Expandat (închis) – toți succesorii nodului sunt cunoscuți;
  - Explorat (deschis) – nodul e cunoscut, dar succesorii săi nu;
  - Neexplorat – nodul nu e cunoscut.

- Listele Closed si Open
  - OPEN = mulțimea (lista) nodurilor explorate (frontiera dintre zona cunoscută și cea necunoscută).
  - CLOSED = mulțimea (lista) nodurilor expandate (regiunea cunoscută în totalitate).
  - Explorarea zonelor necunoscute se face prin alegerea și expandarea unui nod din OPEN. După expandare, nodul respectiv e trecut în CLOSED.
  - Majoritatea algoritmilor tentativi folosesc lista OPEN, dar doar o parte folosesc lista CLOSED.

- Completitudine si optimalitate
  - Algoritm complet = algoritm de explorare care garantează descoperirea unei soluții, dacă problema acceptă soluție.
    - Algoritmii irevocabili sunt mai rapizi și consumă mai puține resurse decât cei tentativi, dar nu sunt compleți pentru că pierd informație.
  - Algoritm optimal = algoritm de explorare care descoperă soluția optimă a problemei.

- Algoritm generic de explorare:
  -
```
Explorare(StInit, test_sol)
  OPEN = {constr_nod(StInit)}; // starea inițială

  Cât timp (OPEN ≠ Ø)
                    // mai am noduri de prelucrat
    nod = selecție_nod(OPEN); // aleg un nod
    Dacă (test_sol(nod)) Întoarce nod;
                              // am găsit o soluție
    OPEN = OPEN \ {nod} U expandare{nod};
                              // extind căutarea
  Întoarce insucces; // nu s-a găsit nicio soluție
```
  - Dacă selecție_nod se realizează independent de costul nodurilor din graful stărilor → căutare neinformată:
    - Dacă e de tip “random” → algoritm aleator - ex: RandomBounce
    - Dacă e de tip “primul venit, primul servit” → OPEN e coadă → BFS – ex: Breadth-first

- Dacă e de tip “ultimul venit, primul servit” → OPEN e stivă → DFS – ex: Depth-first limitat / IDDFS

- Dacă selecție_nod se bazează pe un cost exact sau estimat (euristic) al stărilor problemei → căutare informată:
  - Estimarea costului și folosirea sa în procesul de selecție → esențiale pentru completitudinea, optimalitatea și complexitatea algoritmilor de explorare!

⇨ Explorare informata irevocabila

- Algoritmul alpinistului = algoritmul gradientului maxim
- Fiecărui nod i se asociază o valoare f(nod) ≥ 0 → calitatea soluției parțiale din care face parte nodul.
- Se păstrează doar cel cu valoare maximă → OPEN are un singur element!
- Gradientul maxim:
  -

```
Gradient_maxim(StInit, f, test_sol)
  nod = constr_nod(StInit); // starea inițial        Inițializări
  π(nod) = null;

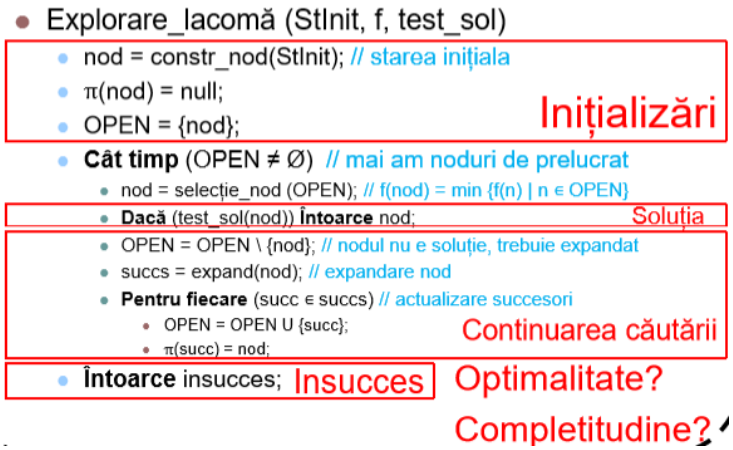
  Cât timp (!test_sol(nod))                          Testez soluția
    succs = expandare(nod); // nodurile au o valoare estimata
                            // prin f
    Dacă (succs = Ø) Întoarce insucces;              Insucces
                            // nu mai am noduri de prelucrat
    succ = selecție_nod(succs); // f(succs) = max {f(n) | n ∈ succs}
    π(succ) = nod;                                   Găsesc calea de continuat
    nod = succ;
  Întoarce nod; // am ajuns la soluție               Soluția
```

- Discutie
  - Algoritmul nu e complet și nu e optimal!
  - Complexitate scăzută: O(bd) - b = branching factor, iar d = depth!
  - Performanțele algoritmului depind foarte tare de forma teritoriului explorat și de euristica folosită (de dorit să existe puține optime locale și o euristică de evaluare cât mai bună).
  - Pseudo-soluție eliminare optim local: se lansează algoritmul de mai multe ori plecând din stări inițiale diferite și se alege cea mai bună soluție obținută.

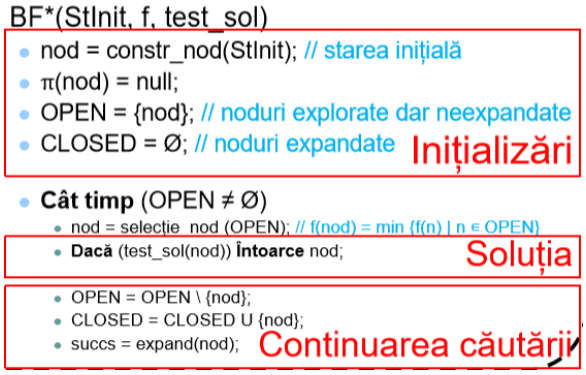
⇨ Explorări tentative informate

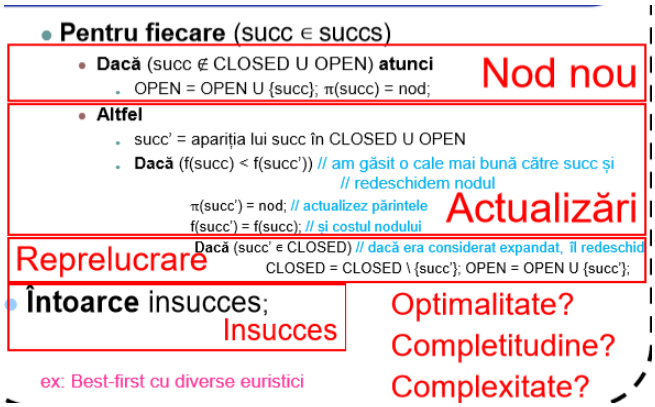
- Păstrează toate nodurile de pe frontieră (OPEN), unii păstrând și nodurile expandate (CLOSED).
- Fiecare nod are un cost asociat f(n) ≥ 0 care estimează calitatea nodului (distanța de la nodul respectiv până la un nod soluție).
- Cu cât f(n) este mai mic, cu atât nodul este mai bun.
- Greedy:

-



  - Explorarea lacomă nu e completă → trebuie să se rețină teritoriul deja parcurs ca să se evite ciclurile!
- Best first*
  -





  - Păstrează întreg teritoriul explorat:
    - OPEN – nodurile de pe frontieră;
    - CLOSED – nodurile expandate (unele noduri pot fi redeschise) → se evită ciclurile.
  - Algoritmul este complet dar nu este optim → optimalitatea depinde de euristica f!
  - Complexitate: O(b^(d+1))

- A*
  - Varianta a BF*
  - Nu poate fi aplicat mereu → trebuie demonstrat că păstrează ordinea soluțiilor unde soluțiile problemelor sunt drumuri în spațiul stărilor!
  - Costul unui drum este aditiv (= suma costurilor arcelor) și crescător în lungul drumului.
  - Folosește două funcții de cost:
    - h(n) - distanța estimată de la nodul curent până la nodul țintă;
    - g(n) - distanța parcursă de la nodul inițial până la nodul curent;
    - f(n) = g(n) + h(n).

- Notatii:

S = (V,E) – graful asociat spațiului stărilor problemei;

n_0 – nodul de start asociat stării inițiale a problemei;

$\Gamma \subseteq V$ – mulțimea nodurilor soluție. Un nod soluție se notează γ;

c(n,n') > 0 – costul arcului (n,n');

π(n) – părintele lui n;

g(n) – costul drumului $n_0..n$ descoperit de algoritm la momentul curent de timp;

$g_p(n)$ – costul exact al porțiunii $n_0..n$ din lungul unei căi date P;

$g^*(n)$ – costul exact al unui drum optim $n_0..n$;

h(n) ≥ 0 – costul estimat al drumului optim de la nodul n la cel mai favorabil nod soluție $\gamma \in \Gamma$. În plus $h(\gamma) = 0$, pentru orice $\gamma \in \Gamma$;

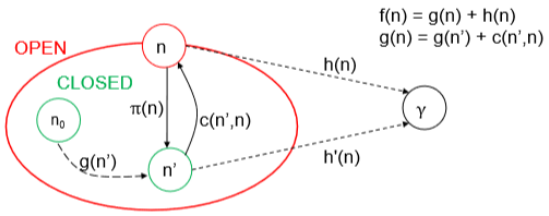
$h^*(n)$ – costul exact al porțiunii de drum $n..\gamma$, pentru cel mai favorabil nod $\gamma \in \Gamma$ ($h^*(n) = \min\{cost(n..\gamma) | \gamma \in \Gamma\}$);

f(n) = g(n) + h(n) – costul estimat al întregului drum $n_0..n..\gamma$, pentru cel mai favorabil nod $\gamma \in \Gamma$, unde porțiunea de drum $n_0..n$ este cea descoperită de algoritm la momentul curent de timp în cursul execuției;

$f^*(n) = g^*(n) + h^*(n)$ – costul exact al unui drum optim $n_0..n..\gamma$, pentru cel mai favorabil nod $\gamma \in \Gamma$;

$C = \min\{f^*(\gamma) | \gamma \in \Gamma\}$ – costul exact al unui drum optim $n_0..\gamma$, $\gamma \in \Gamma$. (C = costul soluției optime);

- Functia de evaluare A*



- Algoritm:

A*(StInit, h, test_sol)

- n_0 = constr_nod(StInit); // starea inițială

Inițializări

- f(n_0) = h(n_0); g(n_0) = 0; π(n_0) = null; // euristici
- OPEN = {n_0}; CLOSED = Ø; // și cozi
- **Cât timp** (OPEN ≠ Ø) // mai am noduri de prelucrat
  - nod = selecție nod (OPEN); // f(nod) = min {f(n) | n ∈ OPEN}
  - **Dacă** (test_sol(nod)) **întoarce** nod;

Soluția

Continuarea căutării

- OPEN = OPEN \ {nod}; // updatez OPEN
- CLOSED = CLOSED U {nod}; // și CLOSE
- succs = expand(nod); // determin nodurile succesoare

- **Pentru fiecare** (succ ∈ succs) { // prelucrare succs

Prelucrare succesori

  - g_succ = g(nod) + c(nod, succ); // calculez g
  - f_succ = g_succ + h(succ); // calculez f = g + h
  - **Dacă** (succ ∉ CLOSED U OPEN) **atunci** // nod nou descoperit →
    - OPEN = OPEN U {succ}; // îl bag în OPEN
      g(succ)= g_succ; f(succ)= f_succ; π(succ) = nod;

Nod nou

  - **altfel** // a mai fost prelucrat
    - **Dacă** (g_succ < g(succ)) // verific dacă noul g este mai mic decât // anteriorul
      g(succ)= g_succ; f(succ)= f_succ; π(succ) = nod; // cale mai bună

Actualizări

Reprelucrare **Dacă** (succ ∈ CLOSED) // dacă era considerat expandat, îl redeschid
CLOSED = CLOSED \ {succ'}; OPEN = OPEN U {succ'};

- **Întoarce** Insucces;

Insucces

ex: A* cu diverse euristici

- Completitudine si optimalitate:
  - Algoritmul A* este complet chiar dacă graful explorat nu este finit.
  - Fie P = n0,n1,...,nm un drum oarecare în graful explorat de A*, astfel încât la un moment T al explorării toate nodurile din P sunt în CLOSED. Atunci, la orice moment de timp egal sau superior lui T, există inegalitatea g(ni) ≤ gp(ni), i = 0,m:
    - Costul nodurilor din CLOSED poate să scadă, dar de fiecare dată când acest lucru se întâmplă, se pierde timp → scoaterea nodului din CLOSED, punerea în OPEN, prelucrarea acestuia încă o dată → trebuiesc evitate aceste

situații → alegerea unei euristici cât mai bune care să minimizeze numărul acestor actualizări!

- Funcția euristică h este admisibilă dacă pentru orice nod n din spațiul stărilor $h(n) \leq h^*(n)$. Cu alte cuvinte, o euristică admisibilă h este optimistă și $h(\gamma) = 0$ pentru orice nod $\gamma \in \Gamma$.
- Algoritmul A* ghidat printr-o euristică admisibilă descoperă soluția optimă dacă există soluții.
- Complexitate:
  - Liniară dacă $|h^*(n) - h(n)| \leq \delta$, unde $\delta \geq 0$ este o constantă.
  - Subexponențială, dacă $|h^*(n) - h(n)| \leq O(\log(h^*(n)))$.
  - Exponențială, altfel, (dar mult mai bună decât a căutărilor neinformate).

- Euristici: consistenta si monotonie
  - O euristică h este consistentă dacă pentru oricare două noduri n și n' ale grafului explorat, astfel încât n' este accesibil din n, există inegalitatea: $h(n) \leq h(n') + k(n,n')$, unde k(n,n') este costul unui drum optim de la n la n'.
  - O euristică h este monotonă dacă pentru oricare două noduri n și n' ale grafului explorat, astfel încât n' este succesorul lui n, există inegalitatea $h(n) \leq h(n') + c(n,n')$, unde c(n,n') este costul arcului (n,n').
  - O euristică este consistentă ⇔ este monotonă.
  - O euristică consistentă este admisibilă.
  - O euristică monotonă este admisibilă.

- Dominanta
  - Fie h1 și h2 două euristici admisibile. Spunem că h1 este mai informată decât h2 dacă $h2(n) < h1(n)$ pentru orice nod $n \notin \Gamma$ din graful spațiului de stare explorat.
  - Un algoritm A1* domină un algoritm A2* dacă orice nod expandat de A1* este expandat și de A2*. (eventual, A2* expandează noduri suplimentare față de A1*, deci A1* poate fi mai rapid ca A2*.)
  - Dacă o euristică monotonă h1 este mai informată decât o euristică monotonă h2, atunci un algoritm A1* condus de h1 domină un algoritm A2* condus de h2.

Cursul 13 – Algoritmi aleatori

⇨ Algorimi aleatori
- Micșorăm timpul de rezolvare a problemei relaxând restricțiile impuse soluțiilor.
- Determinarea soluției optime (Las Vegas): Renunțăm la optimalitate (soluția suboptimală are o marjă de eroare garantată prin calcul probabilistic).
- Găsirea unei singure soluții (Monte Carlo): Găsim o soluție ce se apropie cu o probabilitate măsurabilă de soluția exactă.

⇨ **LAS VEGAS**
- Găsesc soluția corectă a problemei, însă timpul de rezolvare nu poate fi determinat cu exactitate.

- Creșterea timpului de rezolvare -> creșterea probabilității de terminare a algoritmului (găsirea soluției optime).
- Timp = ∞ -> algoritmul se termină sigur (soluția e optimă).
- Probabilitatea de găsire a soluției crește extrem de repede astfel încât să se determine soluția optimă într-un timp suficient de scurt.
- Complexitate:
  - Algoritmii Las Vegas au complexitatea f(n) = O(g(n)) dacă exista c > 0 și n0 > 0 a.î. oricare n ≥ n0 avem 0 < f(n) < c α g(n) cu o probabilitate de cel puțin 1 – n^(-α) pentru α > 0 fixat și suficient de mare.

⇨ **MONTE CARLO**

- Găsesc o soluție a problemei care nu e garantat corectă (soluție aproximativă).
- Timp = ∞ -> soluția corectă a problemei.
- Probabilitatea ca soluția să fie corectă crește o dată cu timpul de rezolvare.
- Soluția găsită într-un timp acceptabil este aproape sigur corectă.
- Complexitate:
  - Algoritmii Monte Carlo au complexitatea f(n) = O(g(n)) dacă exist c > 0 și n0 > 0 astfel încât:
    - Oricare n ≥ n0, 0 < f(n) ≤ c α g(n) cu o probabilitate de cel puțin 1 – n^(-α) pentru α > 0 fixat si suficient de mare;
    - Probabilitatea ca soluția determinată de algoritm să fie corectă este cel puțin 1 - n^(-α).

Metoda Greedy

Pentru rezolvarea următoarelor probleme vom utiliza o metodă de programare importantă, denumită Greedy. În general, metoda Greedy se aplică problemelor de optimizare. Specificul acestei metode constă în faptul că se construieşte soluţia optimă pas cu pas, la fiecare pas fiind selectat (sau „înghiţit") în soluţie elementul care pare „cel mai bun" la momentul respectiv, în speranţa că această alegere locală va conduce la optimul global.

Algoritmii Greedy sunt foarte eficienţi, dar nu conduc în mod necesar la o soluţie optimă. Şi nici nu este posibilă formularea unui criteriu general conform căruia să putem stabili exact dacă metoda Greedy rezolvă sau nu o anumită problemă de optimizare. Din acest motiv, orice algoritm Greedy trebuie însoţit de o demonstraţie a corectitudinii sale[1]*. Demonstraţia faptului că o anumită problemă are proprietatea alegerii Greedy se face de obicei prin inducţie matematică.*

Problema spectacolelor

Managerul artistic al unui festival trebuie să selecteze o mulţime cât mai amplă de spectacole ce pot fi jucate în singura sală pe care o are la dispoziţie. Ştiind că i s-au propus $n \leq 100$ spectacole şi pentru fiecare spectacol i i-a fost anunţat intervalul în care se poate desfăşura $[s_i, f_i)$ (s_i reprezintă ora şi minutul de început, iar f_i ora şi minutul de final al spectacolului i) scrieţi un program care să permită spectatorilor vizionarea unui număr cât mai mare de spectacole. De exemplu, dacă vom citi $n=5$ şi următorii timpi:

```
12 30 16 30
15 0 18 0
10 0 18 30
18 0 20 45
12 15 13 0
```

Spectacolele selectate sunt: `5 2 4`.

Soluţie

Ordonăm spectacolele crescător după ora de final. Selectăm iniţial primul spectacol (deci cel care se termină cel mai devreme). La fiecare pas selectăm primul spectacol neselectat, care nu se suprapune cu cele deja selectate (deci care începe după ce se termină ultimul spectacol selectat).

```
#include <iostream.h>
int inceput[100], sfarsit[100], nr[100];
int main()
{int n, i, h, m, schimb, ultim, aux;
 cout << "n= "; cin >> n;                          //citire
 cout<<"Introduceti inceputul si sfarsitul spectacolelor";
 for (i=0; i<n; i++)
```

1. Demonstraţia depăşeşte nivelul de cunoştinţe al unui elev de clasa a IX-a.

```
      {nr[i]=i+1;           //transform timpul in minute
       cin>>h>>m; inceput[i]=h*60+m;
       cin>>h>>m; sfarsit[i]=h*60+m;}
//ordonez spectacolele crescator dupa ora de final
do
 {schimb=0;
  for (i=0; i<n-1; i++)
       if (sfarsit[nr[i]]>sfarsit[nr[i+1]])
          {aux=nr[i];nr[i]=nr[i+1];nr[i+1]=aux; schimb=1;}
 }
while (schimb);
cout << "Spectacolele selectate sunt:\n"<<nr[0]<<' ';
for (ultim=0, i=1; i<n; i++)
     if (inceput[nr[i]]>=sfarsit[nr[ultim]])
         {cout <<nr[i]<<' '; ultim=i;}
cout<<endl;
return 0;}
```

Problema rucsacului

Un hoț neprevăzător are la dispoziție un singur rucsac cu care poate transporta o greutate maximă Gmax. Hoțul are de ales din $n \leq 50$ obiecte şi, evident, intenționează să obțină un câştig maxim în urma singurului transport pe care îl poate face. Cunoscând pentru fiecare obiect i greutatea g_i şi câştigul c_i pe care hoțul l-ar obține transportând obiectul respectiv în întregime, scrieți un program care să determine o încărcare optimală a rucsacului, în ipoteza că hoțul poate încărca în rucsac orice parte dintr-un obiect. De exemplu, pentru n=5, GMax=100 şi câştigurile-greutățile următoare:

(1000 120)(500 20)(400 200)(1000 100)(25 1) se va afişa pe ecran:

```
 2              100.00%
 4               79.00%
 5              100.00%
```

Soluție

Vom reprezenta o soluție a problemei ca un vector x, cu n componente, în care reținem pentru fiecare obiect fracțiunea încărcată în rucsac de hoț. Deci vectorul x trebuie să îndeplinească următoarele condiții:

1. $x_i \in [0, 1], \forall i \in \{1, 2, \ldots, n\}$.
2. $g_1 \cdot x_1 + g_2 \cdot x_2 + \ldots + g_n \cdot x_n \leq Gmax$
3. $f(x) = c_1 \cdot x_1 + c_2 \cdot x_2 + \ldots + c_n \cdot x_n$ este maximă.

Ordonăm obiectele descrescător după câştigul pe unitatea de greutate (valoare care constituie o măsură a eficienței transportării obiectelor). Cât timp este posibil (încap în rucsac), selectăm obiectele în întregime. Completăm rucsacul cu un fragment din următorul obiect ce nu a fost selectat.

```
#include <iostream.h>
int o[50]; //ordinea obiectelor
float c[100], g[100], x[100], Gr, GMax;
```

```
int n;
int main()
{int i, schimb, aux;
 cout << "n= "; cin >> n;                    //citire
 cout<< "GMax= "; cin >> GMax;
 cout<<"Castigul si greutatea pt. fiecare obiect\n";
 for (i=0; i<n; i++)
     {o[i]=i; cin>>c[i]>>g[i];}
 //ordonez obiectele descrescator dupa castigul unitar
 do
  {schimb=0;
   for (i=0; i<n-1; i++)
       if (c[o[i]]/g[o[i]]<c[o[i+1]]/g[o[i+1]])
          {aux=o[i];o[i]=o[i+1]; o[i+1]=aux; schimb=1;}
  } while (schimb);
 for (i=0, Gr=GMax; i<n && Gr>g[o[i]]; i++)
     {x[o[i]]=1; Gr-=g[o[i]];}
 if (i<n) x[o[i]]= Gr/g[o[i]];
 cout << "Obiectele selectate sunt:\n";
 for (i=0; i<n; i++)
     if (x[i]) cout<<i+1<<' '<<x[i] * 100<<'%'<<endl;
 return 0;}
```

Observaţie

Am aplicat tot o strategie de tip *Greedy*. Se poate demonstra corectitudinea acestui algoritm în condiţiile în care putem încărca în rucsac orice parte a unui obiect. Pentru cazul în care hoţul poate încărca un obiect doar în întregime (varianta discretă a problemei rucsacului) algoritmul *Greedy* nu mai funcţionează. De exemplu, pentru `n=3`, `Gmax=10`, `g=(8, 6, 4)`, `c=(8, 6, 4)`, algoritmul *Greedy* va obţine soluţia `x=(1, 0, 0)` pentru care câştigul obţinut în urma transportului este `8`. Soluţia optimă este `x=(0, 1, 1)` pentru care se obţine câştigul `10`.

Reactivi

Într-un laborator de analize chimice se utilizează `N` reactivi. Se ştie că, pentru a evita accidentele sau deprecierea reactivilor, aceştia trebuie să fie stocaţi în condiţii de mediu speciale. Mai exact, pentru fiecare reactiv `x`, se precizează intervalul de temperatură $[min_x, max_x]$ în care trebuie să se încadreze temperatura de stocare a acestuia. Reactivii vor fi plasaţi în frigidere. Orice frigider are un dispozitiv cu ajutorul căruia putem stabili temperatura (constantă) care va fi in interiorul acelui frigider (exprimată într-un număr întreg de grade Celsius).

Scrieţi un program care să determine numărul minim de frigidere necesare pentru stocarea reactivilor chimici.

Date de intrare

Fişierul de intrare `react.in` conţine:

– pe prima linie numărul natural N, care reprezintă numărul de reactivi;
– pe fiecare dintre următoarele N linii se află min max (două numere întregi separate printr-un spațiu); numerele de pe linia x+1 reprezintă temperatura minimă, respectiv temperatura maximă de stocare a reactivului x.

Date de ieşire

Fişierul de ieşire react.out va conține o singură linie pe care este scris numărul minim de frigidere necesar.

Restricții

- $1 \leq N \leq 8000$
- $-100 \leq min_x \leq max_x \leq 100$ (numere întregi, reprezentând grade Celsius), pentru orice x de la 1 la N
- un frigider poate conține un număr nelimitat de reactivi

Exemple

react.in	react.out	react.in	react.out	react.in	react.out
3 -10 10 -2 5 20 50	2	4 2 5 5 7 10 20 30 40	3	5 -10 10 10 12 -20 10 7 10 7 8	2

(Olimpiada Județeană de Informatică 2004, clasa a IX-a)

Soluție

Intervalele de temperatură pot fi considerate segmente de dreaptă. Problema cere, de fapt, determinarea unui număr minim de puncte astfel încât orice segment să conțină cel puțin unul dintre punctele determinate.

Vom sorta în primul rând intervalele de temperatură crescător după temperatura minimă şi descrescător după temperatura maximă.

Deschidem un frigider şi plasăm primul reactiv în acest frigider. Pentru frigiderul curent reţin temperatura minimă şi temperatura maximă (intervalul de temperatură în care poate fi setat).

Parcurg succesiv reactivii (în ordine) şi pentru fiecare reactiv verific dacă el poate fi plasat în frigiderul curent (pentru aceasta, trebuie ca intersecția dintre intervalul de temperatură al frigiderului şi intervalul de temperatură al reactivului să fie nevidă). Dacă da, plasăm acest reactiv în frigiderul curent (actualizând corespunzător intervalul de temperatură al frigiderului). În caz contrar, deschidem un nou frigider (intervalul de temperatură al acestui frigider va fi intervalul reactivului plasat în el).

```
#include <fstream.h>
ifstream fin("react.in");
ofstream fout("react.out");
int N;
int ti[8000], tf[8000];
/*ti[i]=temperatura minima pentru reactivul i
```

```
  tf[i]=temperatura maxima pentru reactivul i */
int main ()
{int nf, poz, miny, maxx, max, i, j, icx, icy;
 //citire
 fin>>N;
 for (i=0;i<N;i++) fin>>ti[i]>>tf[i];
 fin.close();
 //sortare prin selectia maximului
 for (i=N-1; i>0; i--)
     {for (poz=i, j=0; j<i; j++)
          if (ti[j]>ti[poz] ||
              ti[j]==ti[poz] && tf[j]<tf[poz]) poz=j;
      max=ti[poz];ti[poz]=ti[i]; ti[i]=max;
      max=tf[poz];tf[poz]=tf[i]; tf[i]=max;  }
/* deschid un frigider si plasez primul reactiv
icx=temperatura minima de functionare a frigiderului curent
icy=temperatura maxima de functionare a frigiderului curent
nf=numarul de frigidere deschise */
 nf=1; icx=ti[0];icy=tf[0];
//parcurg ceilalti reactivi in ordine
 for (i=1; i<N; i++)
     {
/* verific daca intervalul curent icx, icy se intersecteaza
cu intervalul de temperatura al reactivului i */
      miny=icy;if (miny>tf[i]) miny=tf[i];
      maxx=icx;if (maxx<ti[i]) maxx=ti[i];
      if (maxx <= miny)
         //actualizez intervalul de temperatura
         {icx=maxx;icy=miny;}
         else //deschid un nou frigider
         {nf++;
          icx=ti[i];icy=tf[i]; }
     }
 fout<<nf<<'\n';
 fout.close(); return 0;}
```

Problema instructorului de schi

Un instructor de schi are la dispoziție `n` perechi de schiuri (`n≤50`) pe care trebuie să le distribuie la `n` elevi începători. Scrieți un program care să distribuie cele `n` perechi de schiuri astfel încât suma diferențelor absolute dintre înălțimea elevului şi lungimea schiurilor atribuite să fie minimă.

Zig-zag

Din fişierul `ZigZag.in` se citesc `N` numere întregi (`N≤1000`). Afişați pe prima linie a fişierului `ZigZag.out` cel mai lung MZigZag care se poate construi

din numerele citite. Numim MZigZag o secvență de numere a_1, a_2, ..., a_m astfel încât $a_1 \le a_2 \ge a_3 \le a_4 \ge a_5 \le ... \ a_{m-1} \ge a_m$. De exemplu:

ZigZag.in	ZigZag.out
7 7 5 0 1 4 9 3	0 9 3 7 1 5 4

Sortare

Fie n ($n \in \mathbf{N}^*$) elemente a_0, a_1, …, a_{n-1} dintr-o mulțime total ordonată. Ordonați crescător elementele a_0, a_1, …, a_{n-1}.

Problema ordonării unor elemente (cunoscută şi sub denumirea de sortare) este frecvent întâlnită în practică şi din acest motiv a fost studiată intens. Ca urmare, au fost elaboraţi numeroşi algoritmi de sortare. Cum de obicei numărul de elemente care trebuie să fie ordonate este mare, s-a studiat şi eficienţa acestor algoritmi, în scopul elaborării unor algoritmi de sortare performanţi. Pentru început, vom studia algoritmi de sortare simpli, nu performanţi, urmând ulterior să învăţăm să evaluăm eficienţa acestor algoritmi şi să elaborăm algoritmi eficienţi.

Sortare prin selecție

Sortarea prin selecție are două variante: sortarea prin selecția elementului maxim şi sortarea prin selecția elementului minim. În ambele variante, ideea de bază este aceeaşi: se selectează cel mai mare element din vector (sau cel mai mic) şi se plasează pe ultima poziție în vector (respectiv, pe prima poziție). Apoi se calculează cel mai mare dintre elementele rămase şi se plasează pe penultima poziție în vector (sau cel mai mic dintre elementele rămase şi se plasează pe a doua poziție), ş.a.m.d. Acest procedeu se repetă de n−1 ori.

```
for (dr=n-1; dr>0; dr--)   //calculez maximul de la 0 la dr
    {for (max=a[0],pozmax=0,i=1; i<=dr; i++)
          if (a[i] > max)  max=a[i], pozmax=i;
     a[pozmax]=a[dr];      //plasez maximul pe pozitia dr
     a[dr] = max; }
```

Sortare prin compararea vecinilor (bubblesort)

O altă metodă de sortare este de a parcurge vectorul, comparând fiecare două elemente vecine şi (dacă este cazul) interschimbându-le. Cum într-o singură trecere nu se poate realiza sortarea vectorului, acest procedeu se repetă până când vectorul devine sortat (la ultima trecere nu am mai efectuat nici o interschimbare).

```
do
  {schimb=0;         //initial nu am facut nici o schimbare
   for (i=0; i<n-1; i++)                //parcurg vectorul
        if (a[i]>a[i+1])     //compar doua elemente vecine
           { //nu sunt in ordine, le interschimb
             aux=a[i]; a[i]=a[i+1]; a[i+1]=aux;
             schimb=1;}      //retin ca am facut schimbari
  }    //procedeul se repeta cat timp se executa schimbari
while (schimb);
```

Sortare prin inserție

Metoda de sortare prin inserție, este de asemenea o metodă simplă, pe care o utilizăm adesea când ordonăm cărțile la jocuri de cărți: de fiecare dată când tragem o carte o plasăm pe poziția sa corectă, astfel încât în mână cărțile să fie ordonate.

Utilizând această idee, sortăm vectorul astfel: parcurgem vectorul, element cu element; la fiecare pas `i`, căutăm poziția corectă a elementului curent `a[i]`, astfel încât secvența `a[0]`, `a[1]`, ..., `a[i]` să fie ordonată:

```
for (i=1; i<n; i++)
    {v=a[i];                  //caut pozitia lui v
     for (poz=i; poz && a[poz-1]>v; poz--)
         a[poz]=a[poz-1];   //mut la dreapta elementele > v
     a[poz] = v;  }     //poz este pozitia corecta pentru v
```

Sortare prin numărarea aparițiilor

În anumite probleme, elementele vectorului au ca valori numere naturale dintr-un interval [`0`, `Max`) de dimensiune redusă (sau au valori care pot fi asociate numerelor naturale dintr-un astfel de interval). Prin dimensiune redusă înțelegem că se poate declara un vector `V` cu `Max` componente întregi.

În acest caz particular cea mai bună metodă de sortare este sortarea prin numărarea aparițiilor: se numără pentru fiecare valoare din intervalul `[0, Max)` de câte ori apare în vector.

```
#include <iostream.h>
#define Max 1000
#define NrMax 10000

int n, V[Max], a[NrMax];
int main()
{int x, i, j, nr;
 cout<<"n="; cin>>n;
 for (i=0; i<n; i++)
     {cin>>x;  //citesc o noua valoare
     V[x]++;} //marim numarul de aparitii ale valorii x
//plasam in ordine elementele in vectorul a
for (nr=i=0; i<Max; i++)
    for (j=0; j<V[i]; j++)
        a[nr++]=i;
for (i=0; i<n; i++) cout<<a[i]<<' ';
cout<<endl; return 0;}
```

Observație

Numărul de operații efectuate de acest algoritm de sortare este de ordinul lui `n+Max`.

Ciurul lui Eratostene

Fie n un număr natural (n≤10000). Să se genereze toate numerele prime mai mici decât n.

Soluție

O primă idee ar fi să parcurgem toate numerele naturale din intervalul [2, n], pentru fiecare număr să verificăm dacă este prim şi să afişăm numerele prime determinate.

O idee mai eficientă provine in antichitate şi poartă numele ciurul lui Eratostene[2]. Ideea este de a „pune în ciur" toate numerele mai mici decât n, apoi de a „cerne" aceste numere până rămân în ciur numai numerele prime. Mai întâi „cernem" (eliminăm din ciur) toți multiplii lui 2, apoi cernem multiplii lui 3, ş.a.m.d.

Vom reprezenta ciurul ca un vector cu 10000 de componente care pot fi 0 sau 1, cu semnificația ciur[i]=1 dacă numărul i este în ciur şi 0 altfel.

Vom parcurge vectorul ciur de la 2 (cel mai mic număr prim), până la $\sqrt{n}$).

Dacă ciur[i] este 1 deducem că i este prim, dar toți multiplii lui nu vor fi (deci eliminăm din ciur toți multiplii lui i).

În final în vectorul ciur vor avea valoarea 1 doar componentele de pe poziții numere prime.

```
#include <iostream.h>
#define NMax 10000
int main()
{int ciur[NMax], n, i, j;
 cout<<"n= "; cin>>n;
 //initial toate numerele sunt in ciur
 for (i=2; i<n; i++) ciur[i]=1;
 for (i=2; i*i<=n; i++)
     if (ciur[i])//i este prim
        //elimin toti multiplii lui i
        for (j=2; j*i<n; j++)
            ciur[i*j]=0;
 for (i=2; i<n; i++)
     if (ciur[i]) cout<<i<<' ';
 return 0; }
```

Subsecvență de sumă maximă

Fie $x=(x_0, x_1, ..., x_{n-1})$ o secvență de numere întregi, dintre care cel puțin un element pozitiv. Să se determine o subsecvență de elemente consecutive x_i, x_{i+1},

2. Eratostene (276–196 î.e.n) a fost un important matematician şi filosof al antichității. Deşi puține dintre lucrările lui s-au păstrat, a rămas celebru prin metoda rapidă de determinare a numerelor prime şi prin faptul că a fost primul care a estimat cu acuratețe diametrul Pământului.

..., x_j, astfel încât suma elementelor din subsecvență să fie maximă. De exemplu, pentru x=(1, 10, 3, −5, 4, 2, −100, 30, 15, 25, −10, 40, 1000) subsecvența de sumă maximă începe la poziția 7, are lungimea 6, iar suma elementelor este 1100.

Soluția 1

Considerăm toate subsecvențele de elemente consecutive, calculăm suma elementelor din subsecvență şi reținem poziția de început şi lungimea subsecvenței de sumă maximă.

```
#include <iostream.h>
int main ()
{int x[50], n, i, st, dr, poz, Lg, Sum, SMax;
 cout <<"n="; cin >> n;
 for (i=0; i<n; i++) {cout <<"x["<<i<<"]="; cin>>x[i];}
 for (SMax=x[0], st=0; st<n; st++)
     for (dr=st; dr<n; dr++)
         {for (Sum=0,i=st; i<=dr; i++) Sum += x[i];
              if (SMax<Sum) SMax=Sum, Lg=dr-st+1, poz=st; }
cout <<"Poz= "<<poz<<" ;Lg= "<<Lg<< " ; Smax= "<<SMax;
return 0;}
```

Datorită celor trei cicluri for imbricate, algoritmul realizează un număr de operații comparabil (ca ordin de mărime) cu n^3.

Soluția 2

Pentru a calcula suma elementelor din subsecvența de la st la dr, vom folosi suma deja calculată a elementelor de la st la dr−1. Secvența de instrucțiuni care rezolvă cerințele problemei devine:

```
for (SMax=x[0], st=0; st<n; st++)
     for (Sum=0,dr=st; dr<n; dr++)
          {Sum += x[dr];
           if (SMax < Sum) SMax=Sum, poz=st, Lg=dr-st+1;}
```

În acest caz, numărul de operații efectuate de algoritm este comparabil (ca ordin de mărime) cu n^2. Spunem că algoritmul este *pătratic*.

Soluția 3

O dată cu parcurgerea vectorului x de la stânga la dreapta, se va memora suma maximă calculată până la momentul curent, în variabila SMax. în variabila Sum este reținută suma elementelor subsecvenței curente. Elementul x[i] va fi înglobat în subsecvența curentă dacă Sum+x[i]≥0. Dacă Sum+x[i]<0, înglobarea lui x[i] în orice subsecvență ar conduce la o subsecvență cu suma elementelor mai mică decât cea maximă până la momentul curent, prin urmare x[i] trebuie ignorat, iar noua subsecvență curentă va începe la poziția următoare. Mai exact, la pasul i: Sum=max{Sum, 0}+x[i]; SMax=max{SMax, Sum}. Secvența de instrucțiuni care rezolvă cerințele problemei devine:

```
for (SMax=Sum=x[0],st=poz=0, Lg=i=1; i<n; i++)
     if (Sum < 0)    //noua subsecventa incepe pe pozitia i
         Sum = x[i], st=i;
         else        //inglobez x[i] in subsecventa curenta
         {Sum += x[i];
          if (SMax<Sum) //subsecventa curenta este maximala
             SMax=Sum, poz=st, Lg=i-st+1;}
```

În acest caz, numărul de operații efectuate de algoritm este comparabil (ca ordin de mărime) cu n. Spunem că algoritmul a devenit *liniar*.

Depozit

Considerăm un depozit cu n ($n \leq 1000$) camere, care conțin respectiv cantitățile de marfă C_1, C_2, …, C_n, (numere naturale). Scrieți un program care să determine un grup de camere cu proprietatea că suma cantităților de marfă pe care le conțin se poate împărți exact la cele n camioane care o transportă.

(Olimpiada Națională de Informatică, Iaşi, 1993)

Soluție

Problema este particulară: solicită determinarea unei submulțimi a unei mulțimi cu n elemente, divizibilă tot cu n. Vom construi sumele S1, S2, ..., Sn şi resturile pe care acestea le dau prin împărțire la n astfel:

```
S1=C1               ⇒ R1=S1 % n
S2=C1+C2            ⇒ R2=S2 % n
...
Si=C1+C2+...+Ci     ⇒ Ri=Si % n
...
Sn=C1+C2+...+Cn     ⇒ Rn=Sn % n
```

Cazul I: Există un rest Ri=0. În acest caz suma Si este divizibilă cu n, prin urmare camerele solicitate sunt 1, 2, ..., i.

Cazul II: Toate resturile sunt diferite de 0. Prin urmare R1, R2, ..., Rn sunt n resturi care iau valori în mulțimea {1, 2, ..., n−1}. În mod obligatoriu există cel puțin două resturi egale: **Ri=Rj** (i<j), adică Si şi Sj produc acelaşi rest la împărțirea cu n ⇒ suma Sj−Si este divizibilă cu n, deci camerele solicitate sunt i+1, i+2, ..., j.

Observații

1. Soluția nu este unică. Procedeul prezentat produce o soluție posibilă.
2. Rezolvarea se bazează pe *Principiul cutiei al lui Dirichlet*[3].

```
#include <iostream.h>
int main ()
{ int SR[100], n, i, j, k, c;
  cout <<"n="; cin >> n;
```

3. Matematicianul Peter Gustav Lejeune Dirichlet a enunțat următorul principiu: „*Se consideră n obiecte care trebuie plasate în p cutii, unde n>p*k, k∈N*. Oricum am plasa obiectele, există o cutie care va conține k+1 obiecte.*”

```
 SR[0]=0;                  //calculez sumele de la citire
 for (i=1; i<=n; i++)
     {cout<<"C"<<i<<"="; cin>>c;
      SR[i]=SR[i-1]+c;}
 for (i=1; i<=n; i++) SR[i]%=n;       //calculez resturile
 cout << "Solutia este " << endl;
 for (i=1; i<=n; i++) //caut un rest = 0
     if (!SR[i])
        {for (k=1;k<=i; k++) cout <<k<<' '; return 0;}
 //cazul II, caut doua resturi egale
 for (i=1; i<n; i++)
     for (j=i+1; j<=n; j++)
          if (SR[i]==SR[j])
             {for (k=i+1;k<=j;k++) cout<<k<<' ';return 0;}
return 0;}
```

Proiectarea Algoritmilor

II CA: Stefan Trausan-Matu stefan.trausan@cs.pub.ro

II CB: Traian Rebedea traian.rebedea@cs.pub.ro

II CC: Costin Chiru costin.chiru@cs.pub.ro

II CD: Andrei Mogos andrei.mogos@cs.pub.ro



Obiectivele cursului (I)

- Cunoașterea unui set important de algoritmi și metode de rezolvare a problemelor de algoritmică. → curs/laborator
- Dezvoltarea abilităților de adaptare a unui algoritm la o problemă din viața reală.→ laborator/teme/proiect/test practic
- Dezvoltarea abilităților de lucru în echipă.→ proiect

Obiectivele cursului (II)

- Utilizarea teoriei predate la curs pentru proiectarea algoritmilor de rezolvare pentru probleme tipice întâlnite în practica dezvoltării sistemelor de programe.
- Discutarea relaţiei dintre caracteristicile problemelor, modul de rezolvare şi calitatea soluţiilor.
- Compararea variantelor unor algoritmi pentru rezolvarea problemelor dificile.

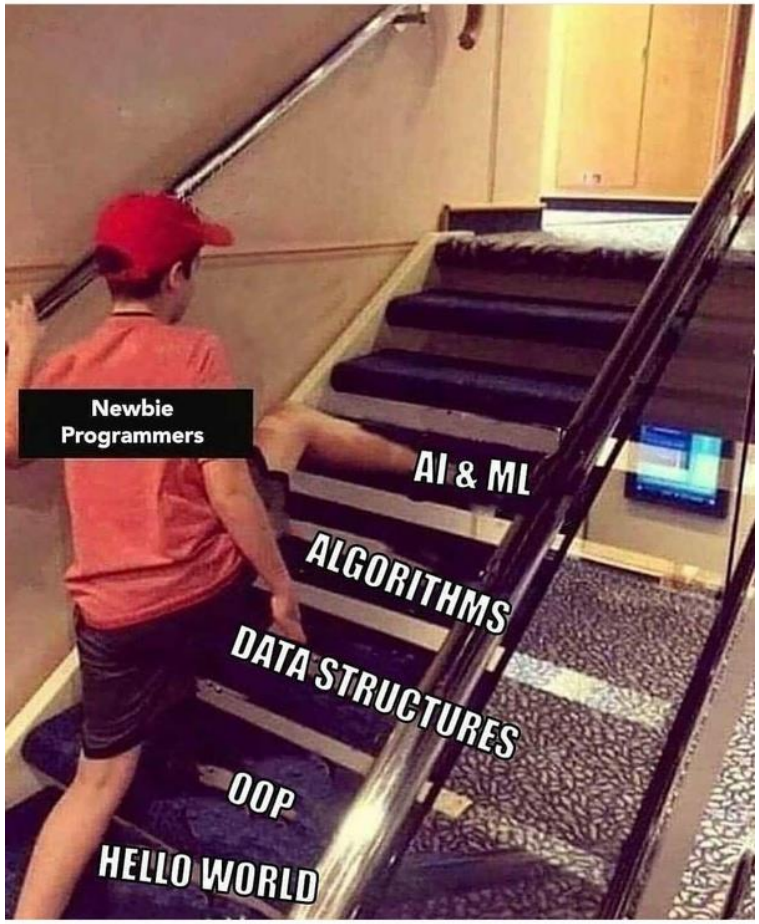


De ce să învăț PA?

- Exemple de utilizări ale PA-ului in diferite meserii:
  - web developer – web social, teoria grafurilor, data mining, clustering;
  - game dev – căutări, grafuri, inteligență artificială;
  - project manager – fluxuri, grafuri de activități;
  - dezvoltator de sisteme de operare – structuri de date avansate, scheme de algoritmi;
  - programator – tot ce tine de algoritmi, in special complexitate si eficiență;
  - tester – tot ce tine de algoritmi, in special complexitate, eficiență si debugging;



De ce să învăț PA?



Nu doar pentru AI / ML, dar pentru o gamă variată de subiecte din calculatoare / computer science (și câteodată și din alte domenii)

Image source: https://www.quora.com/How-do-I-start-learning-or-strengthen-my-knowledge-of-data-structures-and-algorithms

Planul cursului (I)

- Scheme de algoritmi
  - Caracteristici ale problemelor şi tehnici asociate de rezolvare: divide & impera (Merge Sort, puterea unui număr), rezolvare lacomă (arbori Hufmann, problema rucsacului continuă), programare dinamică (înmulțirea matricelor, AOC, problema rucsacului discretă). Backtracking cu optimizări. Propagarea restricţiilor.
- Algoritmi pentru jocuri – Minimax şi α-β.
- Algoritmi pentru grafuri
  - Algoritmi pentru grafuri: parcurgeri, sortare topologică, componente tare conexe, articulaţii, punţi, arbori minimi de acoperire, drumuri de cost minim, fluxuri.

Planul cursului (II)

- Rezolvarea problemelor prin căutare euristică
  - Rezolvarea problemelor prin căutarea euristică A*. Completitudine şi optimalitate, caracteristici ale euristicilor.
- Algoritmi aleatorii
  - Algoritmi aleatori. Las Vegas şi Monte Carlo, aproximare probabilistică.

Evaluarea

- Citiți documentul “Regulament Proiectarea Algoritmilor 2020” de pe site! http://ocw.cs.pub.ro/courses/pa/regulament-general
- Examen 4 p (minim 2 p pentru promovare, minim 3 p pentru a putea lua nota 10)
- Activitate semestru 6 p ☺ (minim 3 p pentru promovare)
  - 2.5 p teme (2 teme punctate egal) // am redus numărul de teme față de anii anteriori
  - 2.5 p teste practice (2 teste practice) // avem un test practice în plus față de anul trecut
  - 2 p proiect
  - 0 p laborator /* am eliminat punctajul ca sa fiți atenți la explicații, să lucrați individual, pentru a învăța, și nu pentru punctaj */
- Bonus maxim 0,5 – 1,5 p (în funcție de realizări): Activitate științifică / rezultate bune la concursuri de programare relevante
- Obs. 1 - Nu se copiază in facultate!
- Obs. 2 - Prima temă copiată se punctează cu minus valoarea maximă a temei. La a doua temă copiată, se repetă materia!



Proiect PA

- Tema proiectului: va fi anunțată în prima săptămână
  Șah, Halite, Generals

- Documentul aferent proiectului va fi disponibil la adresa:
- http://ocw.cs.pub.ro/courses/pa/regulament-proiect

- Obiective:
  - Rezolvarea unei probleme interesante din viaţa reală;
  - Găsirea si aplicarea unor euristici pentru această problemă;
  - Dezvoltarea abilităților de lucru în echipă;
  - Dezvoltarea spiritului competitiv.

Proiect PA (2)

- Etape:
  - 1. Formarea echipelor.
  - 2. Familiarizarea cu frameworkul cu care se va lucra. Lupta cu un bot random sau îndeplinirea anumitor obiective simple.
  - 3. Lupta cu un bot slab.
  - 4. Lupta cu un bot inteligent.
  - 5. Finalizare proiectului și concursul pentru stabilirea celui mai bun proiect din an.
- Mai multe informații în “Regulament Proiect PA 2021” (disponibil probabil la finalul săptămânii)



Feedback 2008, 2009

- Idei preluate din feedback 2008:
  - Schimbarea modului de organizare al proiectului (etape, mod de lucru in echipă).
  - Schimbarea orientării temelor (variante mai ușoare, notare parţială).
  - Subliniată importanţa bibliografiei.
- Idei preluate din feedback 2009:
  - Eliminare restricţii echipa proiect la nivel de grupă.
  - Publicare teme pe infoarena / checker teme.
  - Fără net în laboratoare.
  - Laboratoare mai bune, teme mai atent elaborate, organizare mai bună.
  - Evitată ora 8 dimineața pentru curs. ☺

Feedback 2010 (1)

- Curs:
  - Prea puține 2 ore. (nu avem ce face, așa e in programă)
  - Evitată ora 8 dimineața pentru curs. (Am reușit!!!)
  - Pseudocod uniform - română sau engleză. (Am trecut totul în română)
  - Evitarea greșelilor din cursuri. (îmi cer scuze de pe acum pentru eventualitatea in care mai apar)

- Laborator:
  - Prea grele. (am încercat să le ușurăm prin introducerea de schelete de cod)
  - Furnizare de rezolvări. (am început anul trecut – sperăm să fie ok anul acesta)
  - Laboratoare mai clare si mai uniforme din punctul de vedere al scheletului de cod. (două persoane vor scrie codul de la laboratoare – C++/Java)
  - Păreri împărțite referitoare la absenţa internetului din laborator. (niciodată nu vom putea să împăcăm pe toată lumea!)

Feedback 2010 (2)

- Teme:
  - Mai puține (3 in loc de 4) si mai utile. (vor fi 3 teme)
  - Corectate mai repede si uniform pe serii. (fiecare temă va fi corectată de către o singură persoană pe serie)
  - Temă de recuperare peste vară. (va fi disponibilă o astfel de temă)
- Proiect:
  - Schimbarea șahului. (F1- ants)
  - Interesantă competiția finală. (ne bucurăm!, am păstrat-o)
  - E bine că este pe grupe – abilități de lucru in echipă. (asta ne și dorim!)
  - Punctați si cei care nu intră în grupe. (încercăm să schimbăm punctarea – se va face un clasament general deoarece vom încerca să facem meciuri fiecare cu fiecare)
- Examen:
  - Timp prea scurt. (deja am dublat timpul fata de acum doi ani, mai mult de atât nu vi se va pune la dispoziție!)
  - Examen greu!

Feedback 2016 (1)

- Cadrul didactic a fost bine pregatit pentru sustinerea?
  Cursuri: 4.80 / 6
  Laboratoare:/Seminarii: 5.05 / 6
- Pot fi obtinute explicatii suplimentare in afara orelor de?
  Cursuri: 4.34 / 6
  Laboratoare:/Seminarii: 4.81 / 6
- Materialele puse la dispozitie sunt suficiente pentru intelegerea?
  Cursuri: 4.29 / 6
  Laboratoare:/Seminarii: 4.52 / 6
- Numarul si dificultatea temelor au fost adecvate? 4.41 / 6
  - Creștere semnificativă a scorului fața de anii anteriori!

Feedback 2019

- Aspecte pozitive:
  - Implicarea asistentilor
  - Proiectul a fost o modalitate buna de învățare
  - Predarea materiei curs (conținut, tabla + slideuri)
  - Organizarea laboratoarelor (materiale, schelet)
  - Problemele de la teme, enunțurile temelor
  - Testul practic a avut probleme relevante
  - Distribuție punctaj (inclusiv bonusuri)
- Aspecte negative:
  - Curs câteodată plictisitor sau dificil în unele momente, doar 2 ore
  - Demonstrațiile de corectitudine sunt dificile, nu ne interesează
  - Dificultate laborator, dificultate test practic, dificultate teme

  (unele lucruri sunt mai dificile în viață)
  - Checker copiat laboratoare (încercam să rezolvăm)
  - Nu se dă pauză la curs (încercam să rezolvăm)
  - Prea mult timp explicații laborator (încercam să rezolvăm)



Feedback 2020

- Aspecte pozitive:
  - Organizarea laboratoarelor (materiale, schelet, soluții oficiale)
  - Proiectul a fost o modalitate buna de învățare
  - Implicarea asistentilor
  - Explicațiile de la curs
  - Problemele de la teme, enunțurile temelor
  - Testul practic a avut probleme relevante, folosire hackerrank
  - Adaptare bună la predat online
- Aspecte negative:
  - Încărcare mare
  - Demonstrațiile de corectitudine sunt dificile, nu ne interesează
  - Dificultate laborator, dificultate test practic, dificultate teme

  (unele lucruri sunt mai dificile în viață)
  - Verificare copiat laboratoare și teme => devine stresant
  - Traian scrie urât

Bibliografie

- Introducere in Analiza Algoritmilor de *Cristian Giumale* – Ed. Polirom 2004
- Introducere in Algoritmi de *Thomas H. Cormen, Charles E. Leiserson, Ronald R. Rivest, Clifford Stein* – Ed. Agora (preferabil ediția a 3a) 2009
  - Varianta în engleză de la MIT Press
- Vedeţi şi recomandările din Regulament!
- Mențiune importanta – **slide-urile reprezintă doar un suport pentru prezentare!**

Proiectarea Algoritmilor

Curs 1 – Scheme de algoritmi – Divide et impera



Curs 1 – Cuprins

- Scheme de algoritmi
- Divide et impera
- Exemplificare folosind Merge sort
- Alte exemple de algoritmi divide et impera



Curs 1 – Bibliografie

- Giumale – Introducere in Analiza Algoritmilor cap 4.4
- Cormen – Introducere în Algoritmi cap. Algoritmi Greedy (17)
- http://www.cs.ucsb.edu/~suri/cs235/ClosestPair.pdf
- http://www.cs.rit.edu/~ib/Classes/CS515_Spring12-13/Slides/022-SelectMasterThm.pdf



Scheme de algoritmi

- Prin scheme de algoritmi înțelegem tipare comune pe care le putem aplica în rezolvarea unor probleme similare.
- O gamă largă de probleme se pot rezolva folosind un număr relativ mic de scheme.
- => Cunoașterea schemelor determină o rezolvare mai rapidă și mai eficientă a problemelor.



Divide et Impera (I)

- Ideea (divide si cucerește) este atribuită lui Filip al II-lea, regele Macedoniei (382-336 i.e.n.), tatăl lui Alexandru cel Mare și se referă la politica acestuia față de statele grecești.

- In CS – Divide et impera se referă la o clasă de algoritmi care au ca principale caracteristici faptul că împart problema în subprobleme similare cu problema inițială dar mai mici ca dimensiune, rezolvă problemele recursiv și apoi combină soluțiile pentru a crea o soluție pentru problema originală.

Divide et Impera (II)

- Schema Divide et impera constă în 3 pași la fiecare nivel al recurenței:
  - Divide problema dată într-un număr de subprobleme;
  - Impera (cucerește) – subproblemele sunt rezolvate recursiv. Dacă subproblemele sunt suficient de mici ca date de intrare se rezolvă direct (ieșirea din recurență);
  - Combină – soluțiile subproblemelor sunt combinate pentru a obține soluția problemei inițiale.

Divide et Impera – Avantaje și Dezavantaje

- Avantaje:
  - Produce algoritmi eficienți.
  - Descompunerea problemei în subprobleme facilitează paralelizarea algoritmului în vederea execuției sale pe mai multe procesoare.
- Dezavantaje:
  - Se adaugă un overhead datorat recursivității (reținerea pe stivă a apelurilor funcțiilor).



Merge Sort (I)

- Algoritmul Merge Sort este un exemplu clasic de rezolvare cu D&I.
- Divide: Divide cele n elemente ce trebuie sortate în 2 secvențe de lungime n/2.
- Impera: Sortează secvențele recursiv folosind *merge sort.*
- Combină: Secvențele sortate sunt asamblate pentru a obține vectorul sortat.
- Recurența se oprește când secvența ce trebuie sortată are lungimea 1 (un vector cu un singur element este întotdeauna sortat ☺) .
- Operația cheie este: asamblarea soluțiilor parțiale.

Merge Sort (II)

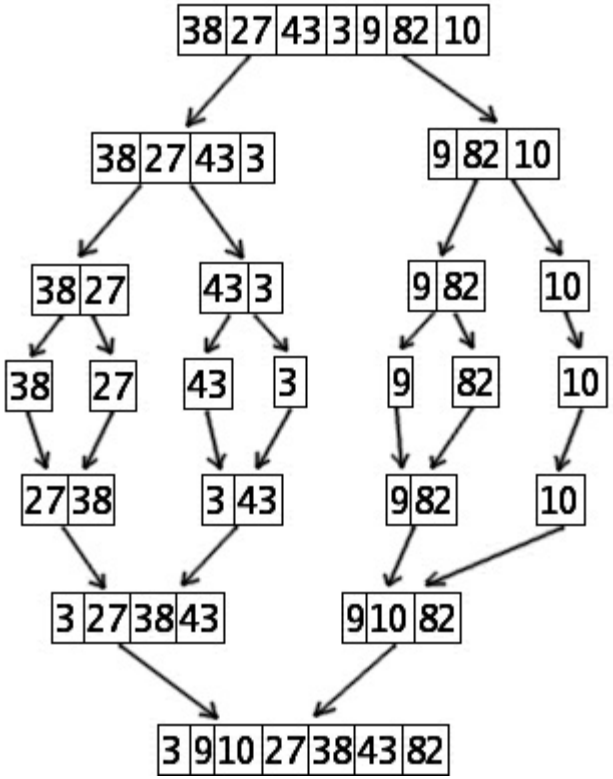
- Algoritm [Cormen]
  - MERGE-SORT(A, p, r)
  - 1 **Dacă** $p < r$
  - 2 **Atunci** $q \leftarrow [(p + r)/2]$ // divide
  - 3 MERGE-SORT(A, p, q) //impera
  - 4 MERGE-SORT(A, $q + 1$, r)
  - 5 MERGE(A, p, q, r) // combină
    // (interclasare)

Merge Sort (III) – Algoritmul de interclasare

- Algoritm [Cormen]
  - MERGE(A, p, q, r) // p si r sunt capetele intervalului, q este “mijlocul”
  - 1 $n_1 \leftarrow q - p + 1$ // numărul de elemente din partea stânga
  - 2 $n_2 \leftarrow r - q$ // numărul de elemente din partea dreapta
  - 3 **creează** vectorii $S[1 \rightarrow n_1 + 1]$ și $D[1 \rightarrow n_2 + 1]$
  - 4 **Pentru** i **de la** 1 **la** n_1
  - 5 $S[i] \leftarrow A[p + i - 1]$ // se copiază partea stânga în S
  - 6 **Pentru** j **de la** 1 **la** n_2
  - 7 $D[j] \leftarrow A[q + j]$ // si partea dreapta în D
  - 8 $S[n_1 + 1] \leftarrow \infty$
  - 9 $D[n_2 + 1] \leftarrow \infty$
  - 10 $i \leftarrow 1$
  - 11 $j \leftarrow 1$
  - 12 **Pentru** k **de la** p **la** r // se copiază înapoi în vectorul de
  - 13 **Dacă** $S[i] \leq D[j]$ // sortat elementul mai mic din cei
  - 14 **Atunci** $A[k] \leftarrow S[i]$ // doi vectori sortați deja
  - 15 $i \leftarrow i + 1$
  - 16 **Altfel** $A[k] \leftarrow D[j]$
  - 17 $j \leftarrow j + 1$

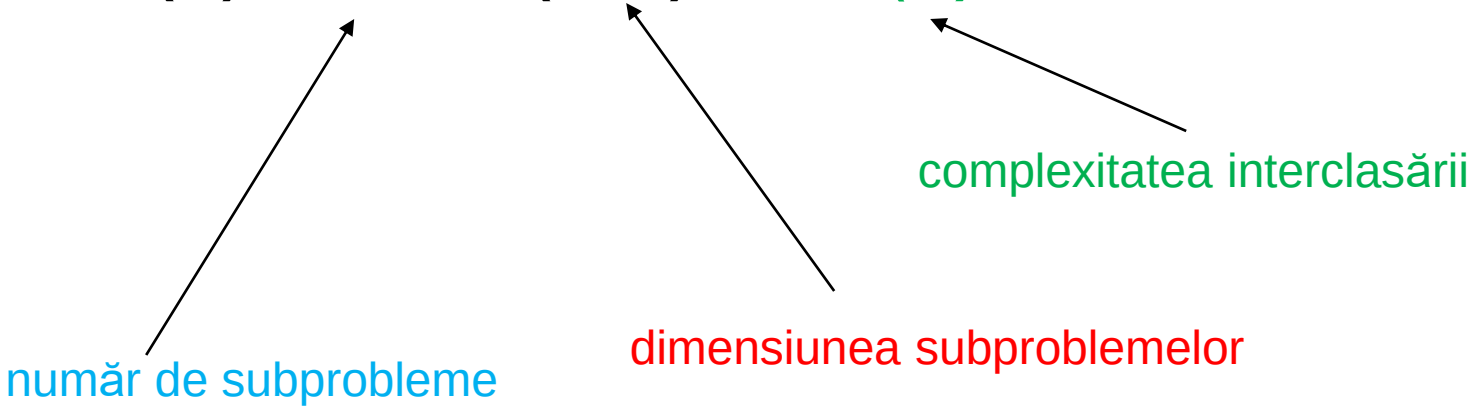
Exemplu funcționare Merge Sort

- Exemplu funcționare [Wikipedia]:



Merge Sort - Complexitate

- $T(n) = 2 * T(n/2) + \Theta(n)$



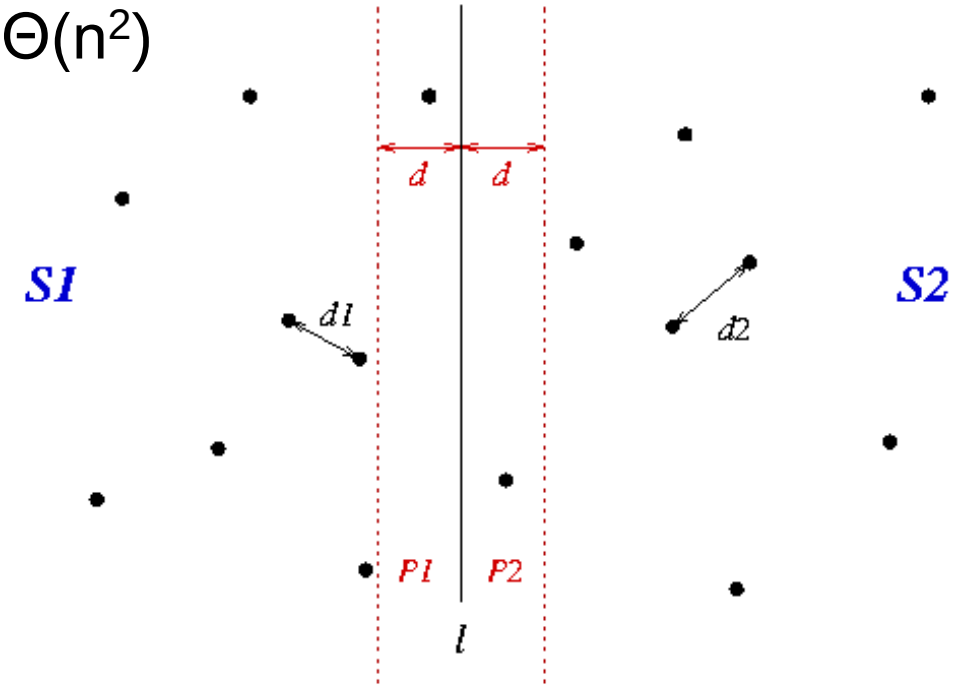
=> (din T. Master) $T(n) = \Theta(n * \log n)$

Divide et Impera – alte exemple (I)

- Calculul puterii unui număr: x^n
  - Algoritm "clasic":
    - **Pentru** *i* **de la** 1 **la** *n* rez = rez * x;
    - **Întoarce** rez.
  - Complexitate: $\Theta(n)$
  - Algoritm Divide et Impera:
    - **Dacă** n este par
      - **Atunci Întoarce** $x^{n/2} * x^{n/2}$
      - **Altfel** (n este impar) **Întoarce** $x * x^{(n-1)/2} * x^{(n-1)/2}$
  - Complexitate: $T(n) = T(n/2) + \Theta(1) = \Theta(\log n)$

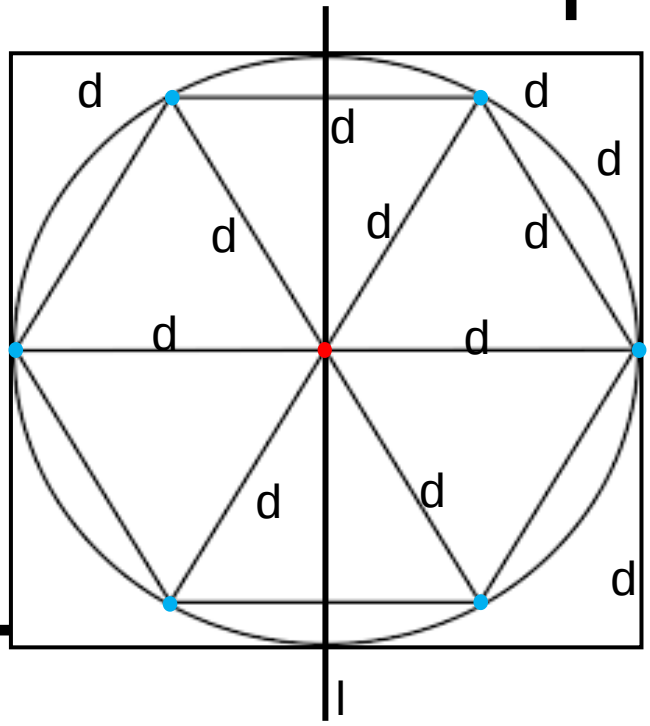
Divide et Impera – alte exemple (II)

- Calculul celei mai scurte distanțe între 2 puncte din plan (http://www.cs.ucsb.edu/~suri/cs235/ClosestPair.pdf)
  - algoritmul naiv – $\Theta(n^2)$



Divide et Impera – alte exemple (III)

- Sortează punctele în ordinea crescătoare a coordonatei x ($\Theta(n \log n)$);
- Împărțim setul de puncte în 2 seturi de dimensiune egală și calculăm recursiv distanța minimă în fiecare set (l = linia ce împarte cele 2 seturi, d = distanța minimă calculată în cele 2 seturi);
- Elimină punctele care sunt plasate la distanță de l > d;
- Sortează punctele rămase după coordonata y;
- Calculează distanțele de la fiecare punct rămas la cei 6 vecini (nu pot fi mai mulți);
- Dacă găsește o distantă < d, atunci actualizează d.



Divide et Impera – Temă de gândire (1)

- Se dă o mulțime M de numere întregi si un număr x. Se cere să se determine dacă există $a,b \in M$ a.î. $a + b = x$.
- Algoritmul propus trebuie să aibă complexitatea $\theta(n \log n)$.
- Temele de la curs sunt facultative!☺

Divide et Impera – Temă de gândire (2)

- Cum sortăm un vector fără interclasare sau alte operații complexe în etapele de împărțire a problemei și de combinare a soluțiilor?

- Hint: se folosesc trei apeluri recursive!
- Observație! Algoritmul este ineficient, dar este interesant!



Divide et Impera – Temă de gândire (3)

- Cum se găseste eficient elementul median al unui vector?
- Problema mai generală: cum se găsește al k-lea cel mai mic (sau cel mai mare) element dintr-un vector?
- Eficient înseamnă Θ(n)
- http://www.cs.rit.edu/~ib/Classes/CS515_Spring12-13/Slides/022-SelectMasterThm.pdf



ÎNTREBĂRI?



Proiectarea Algoritmilor

Curs 2 – Scheme de algoritmi – Greedy



Bibliografie

- Cormen – Introducere în Algoritmi cap. Algoritmi Greedy (17)
- Giumale – Introducere în Analiza Algoritmilor cap 4.4
- https://people.cs.umass.edu/~barring/cs611/lecture/4.pdf
- http://profs.info.uaic.ro/~dlucanu/cursuri/tpaa/resurse/Curs6.pps
- http://euler.math.fau.edu/locke/Greedy.htm
- http://en.wikipedia.org/wiki/Greedoid

Greedy (I)

- Metodă de rezolvare eficientă a unor probleme de optimizare.
- Soluția trebuie să satisfacă un criteriu de optim global (greu de verificat) → optim local mai ușor de verificat.
- Se aleg soluții parțiale ce sunt îmbunătățite repetat pe baza criteriului de optim local până ce se obțin soluții finale.
- Soluțiile parțiale ce nu pot fi îmbunătățite sunt abandonate → proces de rezolvare irevocabil (fără reveniri)!



Greedy (II)

- Schema generală de rezolvare a unei probleme folosind Greedy (programarea lacomă):

- Rezolvare_lacomă(Crit_optim, Problemă)
  - 1. sol_parțiale = sol_inițiale(Problemă); // determinarea soluțiilor parțiale
  - 2. sol_fin = Φ;
  - 3. **Cât timp** (sol_parțiale ≠ Φ)
  - 4. **Pentru fiecare** (s **în** sol_parțiale)
  - 5. **Dacă** (s este o soluție a problemei) { // dacă e soluție
  - 6. sol_fin = sol_fin U {s}; // finală se salvează
  - 7. sol_parțiale = sol_parțiale \ {s};
  - 8. } **Altfel** // se poate optimiza?
  - 9. **Dacă** (optimizare_posibilă (s, Crit_optim, Problemă))
  - 10. sol_parțiale = sol_parțiale \ {s} U // da optimizare(s, Crit_optim, Problemă)
  - 11. **Altfel** sol_parțiale = sol_parțiale \ {s}; // nu
  - 12. **Întoarce** sol_fin;

Comparație D&I și Greedy

- Tip abordare
  - D&I: top-down;
  - Greedy: top-down.
- Criteriu de optim
  - D&I: nu;
  - Greedy: da.

Arbori Huffman

- Metodă de codificare folosită la compresia fișierelor.
- Construcția unui astfel de arbore se realizează printr-un algoritm Greedy.
- Considerăm un text, de exemplu:
  - **ana are mere**
- Vom exemplifica pas cu pas construcția arborelui de codificare pentru acest text și vom defini pe parcurs conceptele de care avem nevoie.



Arbori Huffman – Definiții (I)

- K – mulțimea de simboluri ce vor fi codificate. (a, n, “ “, r, e, m)
- Arbore de codificare a cheilor K este un arbore binar ordonat (păstrează ordinea copiilor fiecărui nod) cu proprietățile:
  - Doar frunzele conțin cheile din K; nu există mai mult de o cheie într-o frunză;
  - Toate nodurile interne au exact 2 copii;
  - Arcele sunt codificate cu 0 și 1 (arcul din stânga unui nod – codificat cu 0).
- k = Codul unei chei – este șirul etichetelor de pe calea de la rădăcina arborelui la frunza care conține cheia k (k este din K).
- p(k) – frecvența de apariție a cheii k în textul ce trebuie comprimat.
- Ex pentru “ana are mere”:
  - p(a) = p(e) = 0.25; p(n) = p(m) = 0.083; p(r) = p( ) = 0.166

Arbori Huffman – Definiții (II)

- A – arborele de codificare a cheilor.
- lg_cod(k) – lungimea codului cheii k conform A.
- nivel(k,A) – nivelul pe care apare în A frunza ce conține cheia K.
- Costul unui arbore de codificare A al unor chei K relativ la o frecventa p este:

$$Cost\ (A) = \sum_{k \in K} lg_cod(k) * p(k) = \sum_{k \in K} nivel\ (k, A) * p(k)$$

- Un arbore de codificare cu cost minim al unor chei K, relativ la o frecvență p este un arbore Huffman, iar codurile cheilor sunt coduri Huffman.

Arbori Huffman – algoritm de construcție (I)

- 1. Pentru fiecare k din K se construiește un arbore cu un singur nod care conține cheia k si este caracterizat de ponderea w = p(k). Subarborii construiți formează o mulțime numită Arb.

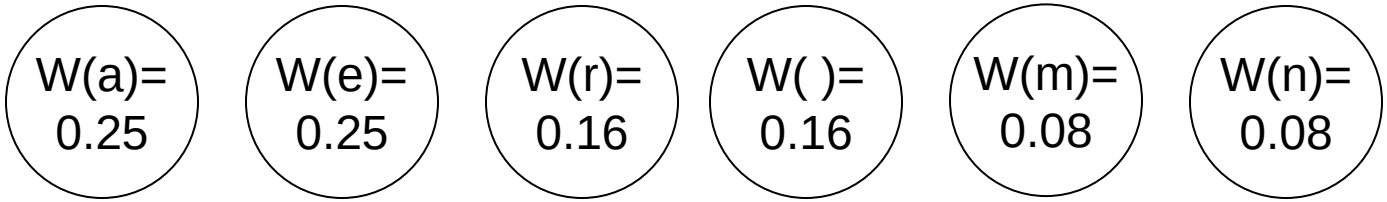
- 2. Se aleg doi subarbori a şi b din Arb astfel încât a şi b au pondere minimă.

Arbori Huffman – algoritm de construcție (II)

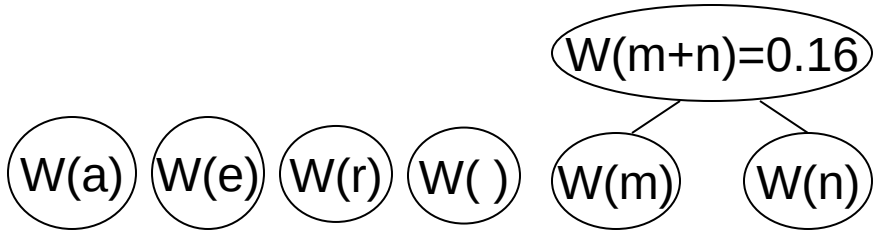
- 3. Se construiește un arbore binar cu o rădăcina *r* care nu conține nici o cheie și cu descendenții a și b. Ponderea arborelui este definită ca w(r) = w(a) + w(b).
- 4. Arborii a și b sunt eliminați din Arb iar r este inserat în Arb.
- 5. Se repetă procesul de construcție descris de pașii 2-4 până când mulțimea Arb conține un singur arbore – Arborele Huffman pentru cheile K.

Arbori Huffman – Exemplu (I)

- Text: ana are mere
- p(a) = p(e) = 0.25; p(n) = p(m) = 0.083; p(r) = p( ) = 0.166
- Pasul 1:

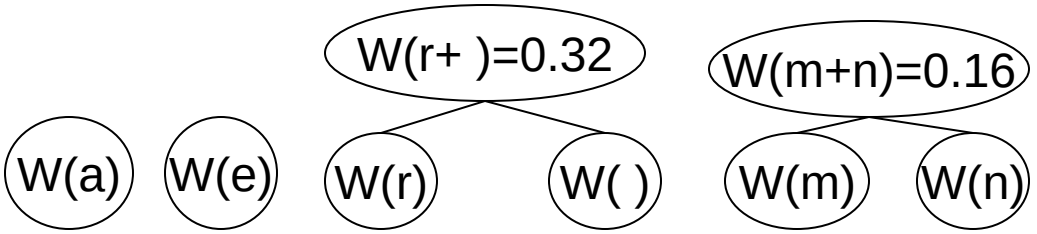


- Pasii 2-4:

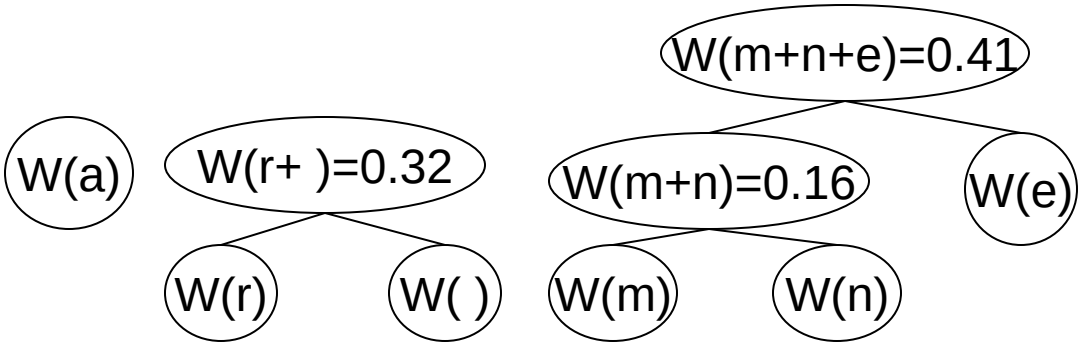


Arbori Huffman – Exemplu (II)

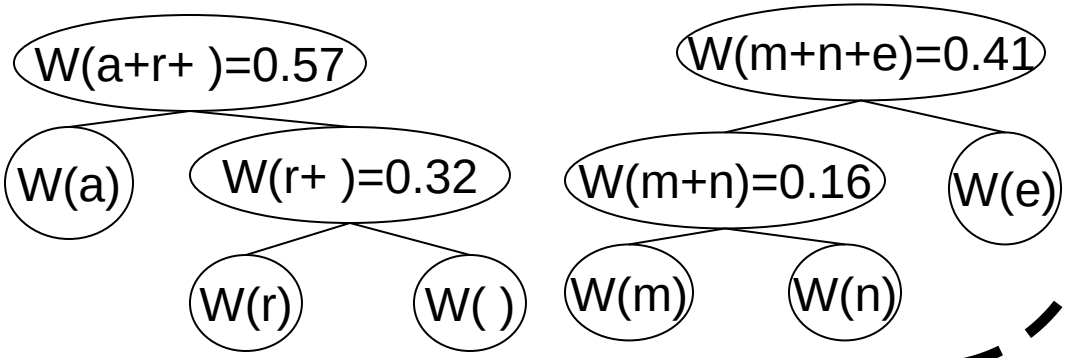
- Pasii 2-4 (II):



- Pasii 2-4 (III):

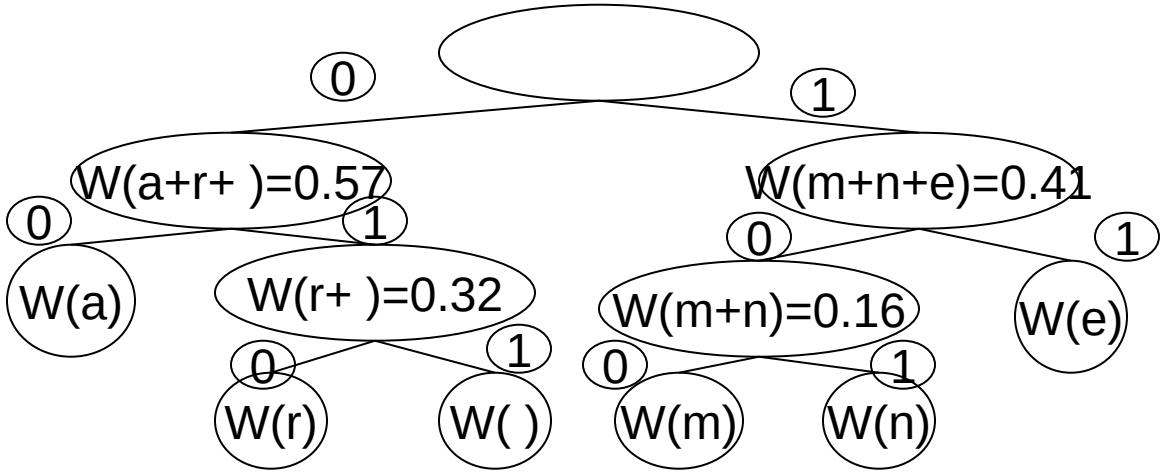


- Pasii 2-4 (IV):



Arbori Huffman – Exemplu (III)

- Pasii 2-4 (V):



- Codificare: a - 00; e -11; r - 010; ’ ‘ - 011; m - 100; n - 101;

$$Cost(A) = \sum_{k \in K} lg_cod(k) * p(k) = \sum_{k \in K} nivel(k, A) * p(k)$$

- p(a) = p(e) = 0.25; p(n) = p(m) = 0.083; p(r) = p( ) = 0.166
- Cost(A) = 2 * 0.25 + 2 * 0.25 + 3 * 0.083 + 3 * 0.083 + 3 * 0.166 + 3 * 0.166 = 1 + 1.2 = 2.2 biti.

Arbori Huffman - pseudocod

- Huffman(K,p){
  - 1. Arb = {frunză(k, p(k)) | k ∈ K};
  - 2. **Cât timp** (card (Arb) > 1) // am mai mulți subarbori
  - 3. Fie a_1 și a_2 arbori din Arb a.i. ∀a ∈ Arb a ≠ a_1 si a ≠ a_2, avem $w(a_1) \leq w(a)$ și $w(a_2) \leq w(a)$); // practic se extrage // de două ori minimul și se salvează în a_1 și a_2
  - 4. Arb = Arb \ {a_1, a_2} U nod_intern(a_1, a_2, $w(a_1) + w(a_2)$);
  - 5. **Dacă** (Arb = Φ)
  - 6. **Întoarce** arb_vid;
  - 6. **Altfel**
  - 7. fie A singurul arbore din mulțimea Arb;
  - 8. **Întoarce** A;

- Notații folosite:
  - a = frunză (k, p(k)) – subarbore cu un singur nod care conține cheia k, iar w(a) = p(k);
  - a = nod_intern(a_1, a_2, x) – subarbore format dintr-un nod intern cu descendenții a_1 și a_2 și w(a) = x.

Arbori Huffman - Decodificare

- Se încarcă arborele și se decodifică textul din fișier conform algoritmului:

- Decodificare (in, out)

```
A = restaurare_arbore (in)        // reconstruiesc arborele
Cât timp (! terminare_cod(in)) // mai am caractere de citit
    nod = A                              // pornesc din rădăcină
    Cât timp (! frunză(nod))      // cât timp nu am determinat caracterul
        Dacă (bit(in) = 1) nod = dreapta(nod)      // avansez în arbore
            Altfel nod = stânga(nod)
    Scrie (out, cheie(nod))       // am determinat caracterul și îl scriu la
                                                  // ieșire
```

Demonstrație (I)

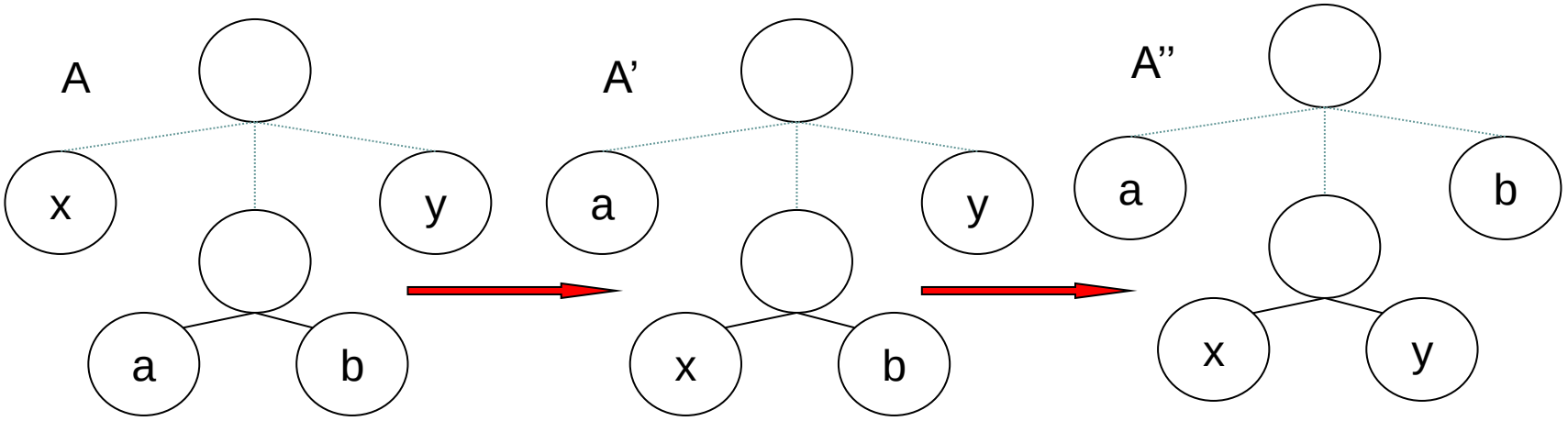
- Arborele de codificare construit trebuie să aibă cost minim pentru a fi arbore Huffman.

- Lema 1. Fie K mulțimea cheilor dintr-un arbore de codificare, card(K) ≥ 2, x, y două chei cu pondere minimă. ∃ un arbore Huffman de înălțime h în care cheile x şi y apar pe nivelul h fiind descendente ale aceluiași nod intern.

Demonstrație (II)

- Demonstrație Lema 1:

$$Cost(A) = \sum_{k \in K} lg_cod(k) * p(k) = \sum_{k \in K} nivel(k, A) * p(k)$$



- Se interschimbă a cu x şi b cu y şi din definiţia costului arborelui => cost(A'') ≤ cost(A') ≤ cost(A) => A'' arbore Huffman

Demonstrație (III)

- Lema 2. Fie A un arbore Huffman cu cheile K, iar x şi y două chei direct descendente ale aceluiaşi nod intern *a*. Fie K' = K \ {x,y} U {z} unde z este o cheie fictivă cu ponderea w(z) = w(x) + w(y). Atunci arborele A' rezultat din A prin înlocuirea subarborelui cu rădăcina *a* și frunzele x, y cu subarborele cu un singur nod care conţine frunza z, este un arbore Huffman cu cheile K'.

- Demonstrație:
  - 1) analog Cost(A') ≤ Cost(A) (Cost(A) = Cost(A') + w(x) + w(y))
  - 2) pp există A'' a.i. Cost(A'') < Cost(A') =>
    - Cost(A'') < Cost(A) - (w(x) + w(y));
    - Cost(A'') + w(x) + w(y) < Cost(A); => A nu este Huffman (contradicţie)

Demonstrație (IV)

- Teoremă – Algoritmul Huffman construiește un arbore Huffman.
- Demonstrație: prin inducție după numărul de chei din mulțimea K.
- $n \leq 2$ => evident
- $n > 2$
  - Ip. Inductivă: algoritmul Huffman construiește arbori Huffman pentru orice mulțime cu n-1 chei
  - Fie $K = \{k_1, k_2, \dots, k_n\}$ a.i. $w(k_1) \leq w(k_2) \leq \dots \leq w(k_n)$

Demonstrație (V)

- Cf. Lema 1, ∃ Un arbore Huffman unde cheile k_1, k_2 sunt pe același nivel şi descendente ale aceluiași nod.
- A_{n-1} – arborele cu n-1 chei K' = K - $\{k_1,k_2\} \cup z$ unde w(z) = w(k_1) + w(k_2).
- A_{n-1} rezultă din A_n prin modificările prezentate in Lema 2 => A_{n-1} este Huffman, şi cf. ipotezei inductive e construit prin algoritmul Huffman(K',p').
- => Algoritmul Huffman(K, p) construiește arborele format din k_1 si k_2 si apoi lucrează ca algoritmul Huffman(K', p') ce construiește A_{n-1} => construiește arborele Huffman(K, p).

Alt exemplu (I)

Problema rucsacului

Trebuie să umplem un rucsac de capacitate maximă M kg cu obiecte care au greutatea m_i şi valoarea v_i. Putem alege mai multe obiecte din fiecare tip cu scopul de a maximiza valoarea obiectelor din rucsac.

- Varianta 1: putem alege fracţiuni de obiect – "problema continuă"
- Varianta 2: nu putem alege decât obiecte întregi (număr natural de obiecte din fiecare tip) – "problema 0-1"



Alt exemplu (II)

- Varianta 1: Algoritm Greedy
  - sortăm obiectele după raportul v_i/m_i;
  - adăugăm fracţiuni din obiectul cu cea mai mare valoare per kg până epuizăm stocul şi apoi adăugăm fracţiuni din obiectul cu valoarea următoare.
  - Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10
  - Soluție: (m_2 , v_2) 8kg şi 2kg din (m_1,v_1) sau din (m_4,v_4) – valoarea totală: 19 + 2 * 10 / 5 = 23
- Varianta 2: Algoritmul Greedy nu funcţionează => contraexemplu
  - Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10
  - Rezultat corect – 2 obiecte: (m_1,v_1) și (m_4,v_4) – valoarea totală: 20
  - Rezultat algoritm Greedy – 1 obiect (m_2,v_2) – valoarea totală: 19

Când funcţionează algoritmii Greedy? (I)

- Problema are proprietatea de substructură optimă
  - Soluţia problemei conţine soluţiile subproblemelor.
- Problema are proprietatea alegerii locale
  - Alegând soluţia optimă local se ajunge la soluţia optimă global.

Când funcţionează algoritmii Greedy? (II)

- Fie E o mulţime finită nevidă şi I $\subset \mathcal{P}$(E) a.i. $\varnothing \in$ I, $\forall$X $\subseteq$ Y şi Y $\in$ I => X $\in$ I. Atunci spunem că (E,I) este un sistem accesibil.

- Submulţimile din I sunt numite submulţimi "independente".

- Exemple:
  - Ex1: E = {e_1, e_2, e_3} si I = {$\varnothing$, {e_1}, {e_2}, {e_3}, {e_1, e_2}, {e_2, e_3}} – mulțimile ce nu conțin e_1 si e_3.
  - Ex2: E – muchiile unui graf neorientat şi I mulţimea mulţimilor de muchii ce nu conţin un ciclu (mulțimea arborilor).
  - Ex3: E set de vectori dintr-un spaţiu vectorial, I mulţimea mulţimilor de vectori linear independenţi.
  - Ex4: E – muchiile unui graf neorientat şi I mulţimea mulţimilor de muchii în care oricare 2 muchii nu au un vârf comun.

Când funcţionează algoritmii Greedy? (III)

- Un sistem accesibil este un matroid dacă satisface proprietatea de interschimbare:

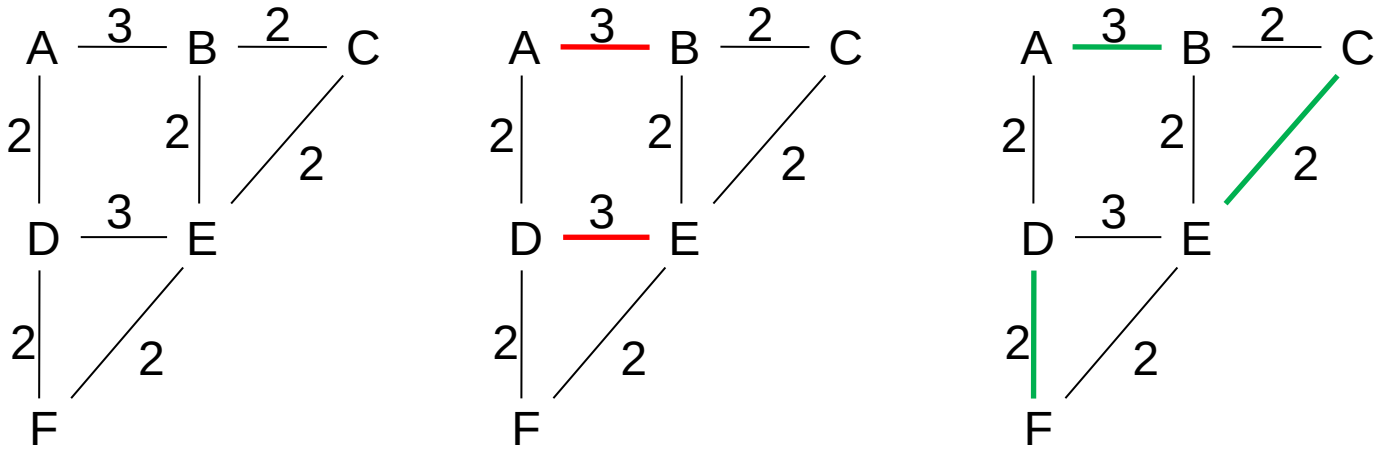
  $\forall X, Y \in I$ şi $|X| < |Y| \Rightarrow \exists e \in Y \setminus X$ a. î. $X \cup \{e\} \in I$

- Teoremă. Dacă (E, I) este matroid atunci există un algoritm Greedy care rezolvă problema de optimizare.

Verificăm exemplele

- Ex1: I = {∅, {e_1}, {e_2}, {e_3}, {e_1, e_2}, {e_2, e_3}}
- De exemplu, fie Y = {e_1, e_2} și X = {e_1}
- → Y \ X = {e_2} → X ∪ {e_2} ∈ I → dacă proprietatea e validă pentru toate perechile de mulțimi X, Y → matroid

- Ex4:



Problema rucsacului discretă (varianta 2)

- Exemplu: M = 10; m_1 = 5 kg, v_1 = 10, m_2 = 8 kg, v_2 = 19, m_3 = 4 kg, v_3 = 4, m_4 = 5 kg, v_4 = 10

- Fie I = mulțimea subsoluțiilor și X = $\{m_2\}$ și Y = $\{m_1, m_4\}$ → $m_1 \in Y \setminus X$ dar $X \cup \{m_1\} \notin I$ → problema nu respectă proprietatea de matroid → problema nu se poate rezolva folosind tehnica Greedy!

Greedy – tema de gândire

- Se dă un număr natural n. Să se găsească cel mai mare subset S din {1, 2, ..., n} astfel încât nici un element din S să nu fie divizibil cu nici un alt element din S.
- În plus, dacă există mai multe subseturi maximale ce respectă proprietatea de mai sus, să se determine întotdeauna cel mai mic din punct de vedere lexicografic.
- S este lexicografic mai mic decât T dacă cel mai mic element care este membru din S sau T, dar nu din amândouă, face parte din S.

ÎNTREBĂRI?



Proiectarea Algoritmilor

Curs 3 – Programare dinamică



BIbliografie

- Cormen – Introducere în Algoritmi - Programare dinamică (cap. 16)
- Giumale – Introducere în Analiza Algoritmilor cap 4.5
- http://www.cse.ust.hk/~dekai/271/notes/L12/L12.pdf

Programare dinamică

- Programare dinamică
  - Descriere generală
  - Algoritm generic
  - Caracteristici
- Exemplificare: Înmulțirea matricilor
- Exemplificare: Arbori optimi la căutare (AOC)
  - Definiții
  - Construcția AOC

Programare dinamică

- Descriere generală
  - Soluții optime construite iterativ asamblând soluții optime ale unor probleme similare de dimensiuni mai mici.
- Algoritmi “clasici”
  - Înmulţirea unui şir de matrici
  - AOC
  - Algoritmul Floyd-Warshall care determină drumurile de cost minim dintre toate perechile de noduri ale unui graf.
  - Numere catalane
  - Viterbi

Algoritm generic

- Programare dinamică (crit_optim, problema)
  - // fie $problema_0$ $problema_1$ … $problema_n$ astfel încât
  - // $problema_n$ = problema; $problema_i$ mai simplă decât $problema_{i+1}$
  - 1. Sol = soluții_inițiale(crit_optim, $problema_0$);
  - 2. **Pentru** *i* **de la** 1 **la** *n* // construcție soluții pentru
    // $problema_i$ folosind soluțiile problemelor precedente
  - 3. Sol_i = calcul_soluții(Sol, crit_optim, $Problema_i$);
    // determin soluția $problemei_i$
  - 4. Sol = Sol U Sol_i;
    // noile soluții se adaugă pentru a fi refolosite pe viitor
  - 5. s = soluție_pentru_$problema_n$(Sol);
    // selecție / construcție soluție finală
  - 6. **Întoarce** s;

Caracteristici

- O soluție optimă a unei probleme conține soluții optime ale subproblemelor.
- Decompozabilitatea recursivă a problemei P în subprobleme similare P = P_n, P_{n-1}, … P_0 care acceptă soluții din ce în ce mai simple.
- Suprapunerea problemelor (soluția unei probleme P_i participă în procesul de construcție a soluțiilor mai multor probleme P_k de talie mai mare k > i) – memoizare (se foloseşte un tablou pentru salvarea soluţiilor subproblemelor cu scopul de a nu le recalcula).
- În general se folosește o abordare bottom-up, de la subprobleme la probleme.

Diferențe Greedy – Programare dinamică

Programare lacomă	*Programare dinamică*
• Sunt menținute doar soluțiile parțiale curente din care evoluează soluțiile parțiale următoare	• La construcția unei soluții noi poate contribui orice altă soluție parțială generată anterior
• Soluțiile parțiale anterioare sunt eliminate	• Se păstrează toate soluțiile parțiale
• Se poate obține o soluție neoptimă. (trebuie demonstrat că se poate aplica).	• Se obține soluția optimă.

Diferenţe divide et impera – programare dinamică

Divide et impera

- abordare top-down – problema este descompusă în subprobleme care sunt rezolvate independent
- putem rezolva aceeaşi problemă de mai multe ori (dezavantaj potenţial foarte mare)

Programare dinamică

- în general abordare bottom-up - se porneşte de la sub-soluţii elementare şi se combină sub-soluţiile mai simple în sub-soluţii mai complicate, pe baza criteriului de optim
- se evită calculul repetat al aceleiaşi subprobleme prin memorarea rezultatelor intermediare (memoizare)

Parantezarea matricilor (Chain Matrix Multiplication)

- Se dă un şir de matrice: A_1, A_2, ..., A_n.
- Care este numărul minim de înmulţiri de scalari pentru a calcula produsul:

$$A_1 \times A_2 \times \ldots \times A_n \ ?$$

- Să se determine una dintre parantezările care minimizează numărul de înmulţiri de scalari.

Înmulţirea matricilor

- A(p, q) x B (q, r) => pqr înmulţiri de scalari.
- Dar înmulţirea matricilor este asociativă (deşi nu este comutativă).
- A(p, q) x B (q, r) x C(r, s)
  (AB)C => pqr + prs înmulţiri
  A(BC) => qrs + pqs înmulţiri
- Ex: p = 5, q = 4, r = 6, s = 2
  (AB)C => 180 înmulţiri
  A(BC) => 88 înmulţiri
- Concluzie: Parantezarea este foarte importantă!

Soluţia banală

- Matrici: $A_1, A_2, ..., A_n$.
- Vector de dimensiuni: $p_0, p_1, p_2, ... , p_n$.
- $A_i(p_{i-1}, p_i) \rightarrow A_1(p_0, p_1), A_2(p_1, p_2), \ldots$
- Dacă folosim căutare exhaustivă şi vrem să construim toate parantezările posibile pentru a determina minimul: $\Omega(4^n / n^{3/2})$.
- Vrem o soluţie polinomială folosind P. D.



Descompunere în subprobleme

- Încercăm să definim subprobleme identice cu problema originală, dar de dimensiune mai mică.

- $\forall\ 1 \le i \le j \le n$:
  - Notăm $A_{i,j} = A_i \times \ldots \times A_j$. $A_{i,j}$ are p_{i-1} linii și p_j coloane: $A_{i,j}(p_{i-1}, p_j)$
  - m[i, j] = numărul optim de înmulţiri pentru a rezolva subproblema $A_{i,j}$
  - s[i, j] = poziţia primei paranteze pentru subproblema $A_{i,j}$
  - Care e parantezarea optimă pentru $A_{i,j}$?

- Problema iniţială: $A_{1,n}$

Combinarea subproblemelor

- Pentru a rezolva $A_{i,j}$
  - Trebuie găsit acel indice $i \leq k < j$ care asigură parantezarea optimă:

$$A_{i,j} = (A_i \times \ldots \times A_k) \times (A_{k+1} \times \ldots \times A_j)$$

$$A_{i,j} = A_{i,k} \times A_{k+1,j}$$

Alegerea optimală

- Căutăm optimul dintre toate variantele posibile de alegere ($i \leq k < j$)
- Pentru aceasta, trebuie însă ca şi subproblemele folosite să aibă soluţie optimală (adică $A_{i, k}$ şi $A_{k+1, j}$ să aibă soluție optimă).

Definirea recursivă

- Folosind descompunerea în subprobleme, combinarea subproblemelor, alegerea optimală şi substructura optimală putem să rezolvăm problema prin programare dinamică.
- Următorul pas este să definim recursiv soluţia unei subprobleme.
- Vrem să găsim o formulă recursivă pentru m[i, j] şi s[i, j].



Definirea recursivă (II)

$$m[i,j] = \begin{cases} 0 & i = j, \\ \min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j) & i < j \end{cases}$$

- Cazurile de bază sunt m[i, i]
- Noi vrem să calculăm m[1, n]
- Cum alegem s[i, j] ?
- Bottom-up de la cele mai mici subprobleme la cea iniţială.

Substructura optimală

- Dacă ştim că alegerea optimală a soluţiei pentru problema $A_{i, j}$ implică folosirea subproblemelor ($A_{i, k}$ şi $A_{k+1, j}$) şi soluţia pentru $A_{i, j}$ este optimală, atunci şi soluţiile subproblemelor $A_{i, k}$ şi $A_{k+1, j}$ trebuie să fie optimale!

- Demonstraţie: Folosind metoda cut-and-paste (metodă standard de demonstrare a substructurii optimale pentru problemele de programare dinamică).

- Observație: Nu toate problemele de optim posedă această proprietate! Ex: drumul maxim dintr-un graf orientat.



Rezolvare bottom-up

$m[1,2], m[2,3], m[3,4], \ldots, m[n-3,n-2], m[n-2,n-1], m[n-1,n]$

$m[1,3], m[2,4], m[3,5], \ldots, m[n-3,n-1], m[n-2,n]$

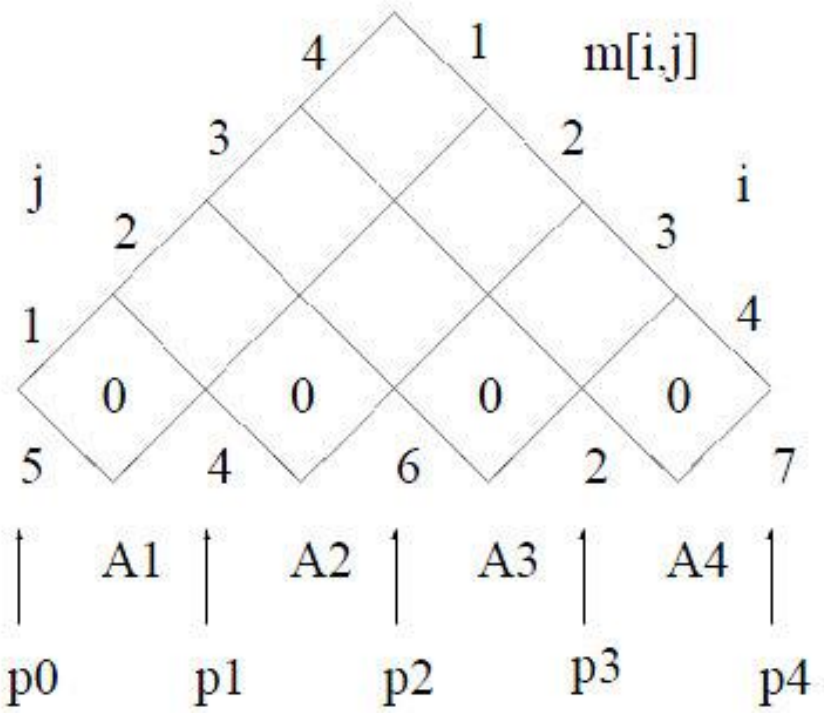
$m[1,4], m[2,5], m[3,6], \ldots, m[n-3,n]$

$\vdots$

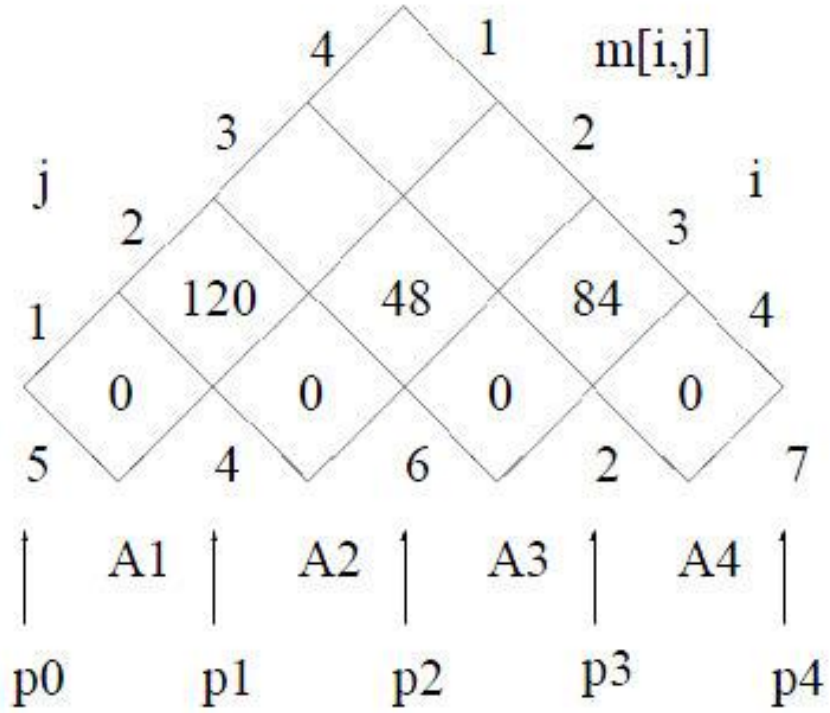
$m[1,n-1], m[2,n]$

$m[1,n]$

Rezolvare - iniţializare



Rezolvare – pas intermediar (I)

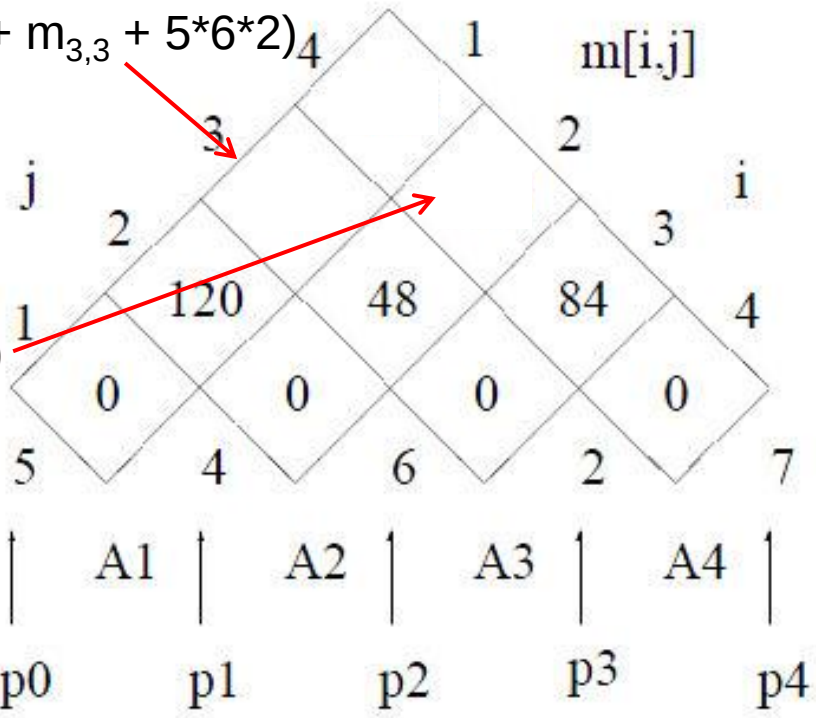


Rezolvare – pas intermediar (II)

$$m[i,j] = \begin{cases} 0 & i = j, \\ \min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j) & i < j \end{cases}$$

$A_{1,3} = \{A_1 * A_{2,3}, A_{1,2} * A_3\}$
$m_{1,3} = \min(m_{1,1} + m_{2,3} + 5*4*2, m_{1,2} + m_{3,3} + 5*6*2)$
$s_{1,3} = 1$

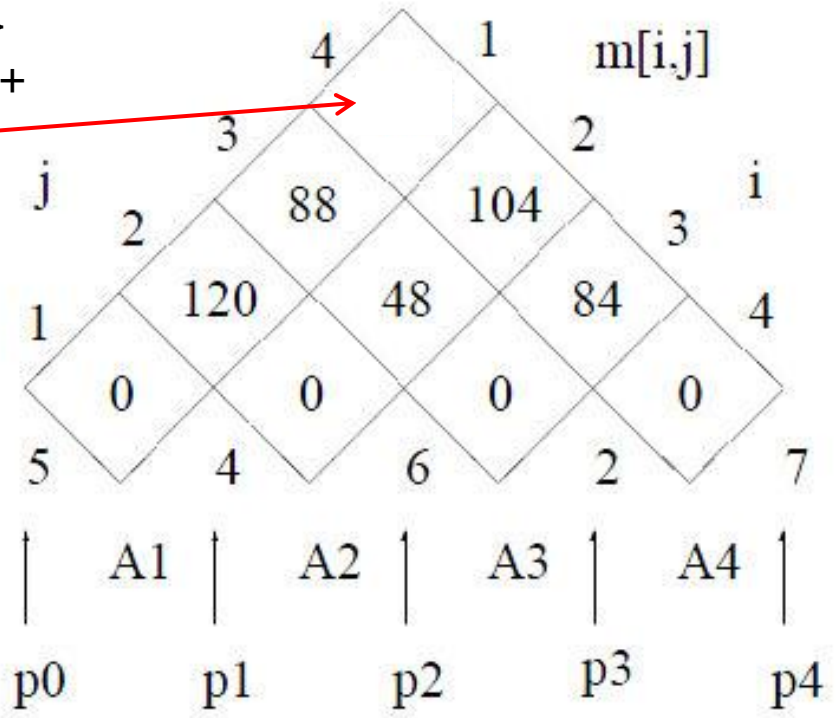
$A_{2,4} = \{A_2 * A_{3,4}, A_{2,3} * A_4\}$
$m_{2,4} = \min(m_{3,4} + 4*6*7, m_{2,3} + 4*2*7)$
$s_{2,4} = 3$



Rezolvare – final

$$m[i,j] = \begin{cases} 0 & i = j, \\ \min_{i \le k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j) & i < j \end{cases}$$

$A_{1,4} = \{A_1 * A_{2,4}, A_{1,2} * A_{3,4}, A_{1,3} * A_4\}$
$m_{1,4} = \min(m_{2,4} + 5*4*7, m_{1,2} + m_{3,4} + 5*6*7, m_{1,3} + 5*2*7) = 158$
$s_{1,4} = 3$



Parantezarea optimă este:

$(A_1(A_2A_3))A_4$

Numărul optim de operații

este: 158

Pseudocod

- Înmulțire_matrici (p, n)
  - **Pentru** *i* **de la** 1 **la** *n* // inițializare
    - m[i, i] = s[i, i] = 0
  - **Pentru** *l* **de la** 1 **la** *n - 1* // dimensiune problema
    - **Pentru** *i* **de la 1 la** *n – l* // indice stânga
      - j = i + l // indice dreapta
      - m[i, j] = ∞ // pentru determinare minim
      - **Pentru** *k* **de la** *i* **la** *j – 1*
        - q = m[i, k] + m[k + 1, j] + p[i - 1] * p[k] * p[j]
        - **Dacă** q < m[i, j]
        - m[i, j] = q
        - s[i, j] = k
  - **Întoarce** *m* și *s*

Complexitate

- Spaţială: $\Theta(n^2)$
  - Pentru memorarea soluţiilor subproblemelor
- Temporală: $O(n^3)$
  - Ns: Număr total de subprobleme: $O(n^2)$
  - Na: Număr total de alegeri la fiecare pas: $O(n)$
  - Complexitatea: $O(n^3)$ este de obicei egală cu Ns x Na

Arbori optimi la căutare (AOC)

- Def 2.1: Fie K o mulțime de chei. Un arbore binar cu cheile K este un graf orientat și aciclic A = (V,E) a.î.:
  - Fiecare nod $u \in V$ conține o singură cheie $k(u) \in K$ iar cheile din noduri sunt distincte.
  - Există un nod unic $r \in V$ a.î. i-grad(r) = 0 și $\forall u \neq r$, i-grad(u) = 1.
  - $\forall u \in V$, e-grad(u) ≤ 2; S(u) / D(u) = succesorul stânga / dreapta.

- Def 2.2: Fie K o mulțime de chei peste care există o relație de ordine $\prec$. Un arbore binar de căutare satisface:
  - $\forall u,v,w \in V$ avem $(v \in S(u) \Rightarrow cheie(v) \prec cheie(u)) \wedge (w \in D(u) \Rightarrow cheie(u) \prec cheie(w))$

Căutare într-un arbore de căutare

- Caută(elem, Arb)
  - **Dacă** Arb = null
    - **Întoarce** null
  - **Dacă** elem = Arb.val // valoarea din nodul crt.
    - **Întoarce** Arb
  - **Dacă** elem $\prec$ Arb.val
    - **Întoarce** Caută(elem, Arb.st)
  - **Întoarce** Caută(elem, Arb.dr)

Complexitate: $\Theta(\log n)$

Inserţie într-un arbore de căutare

- Inserare(elem, Arb)
  - **Dacă** Arb = vid // adaug cheia în arbore
    - nod_nou(elem, null, null)
  - **Dacă** elem = Arb.val // valoarea există deja
    - **Întoarce** Arb
  - **Dacă** elem $\prec$ Arb.val
    - **Întoarce** Inserare(elem, Arb.st) // adaugă în stânga
  - **Întoarce** Inserare(elem, Arb.dr) // sau în dreapta

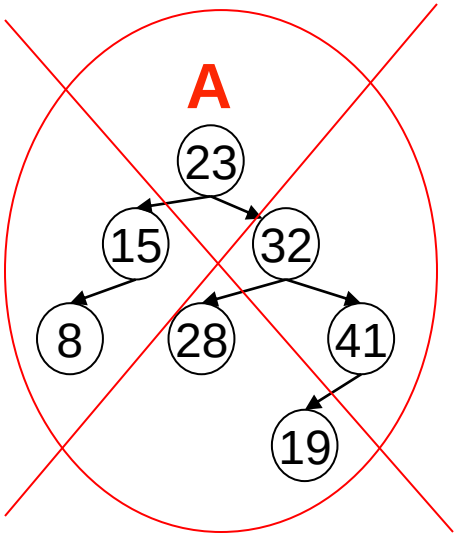
nod Stânga

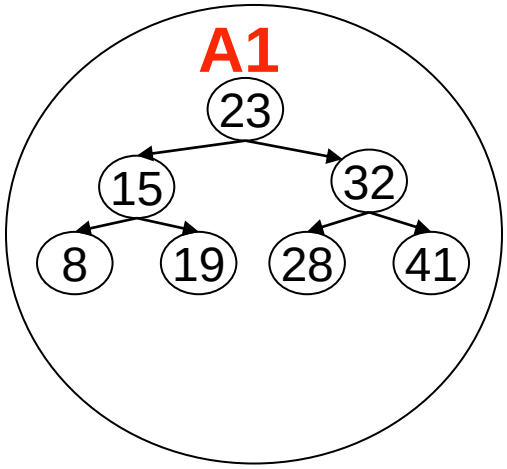
nod Dreapta

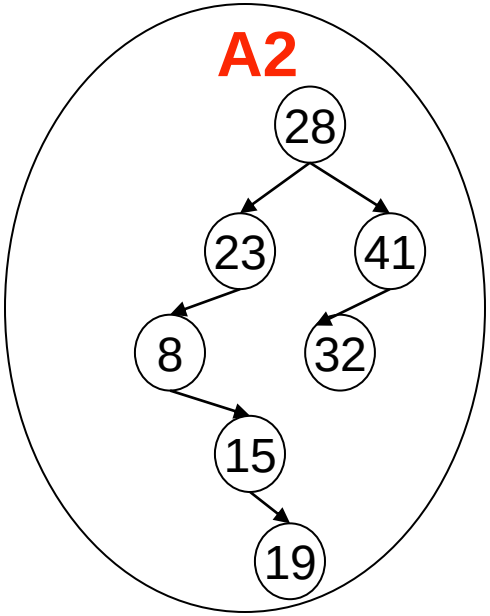
Complexitate: $\Theta(\log n)$

Exemplu de arbori de căutare

- Cu aceleaşi chei se pot construi arbori distincţi

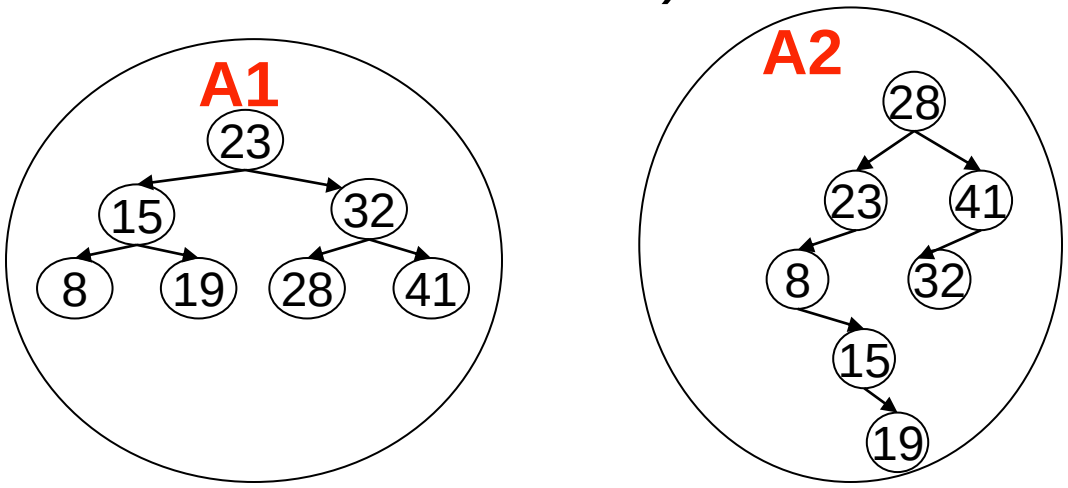






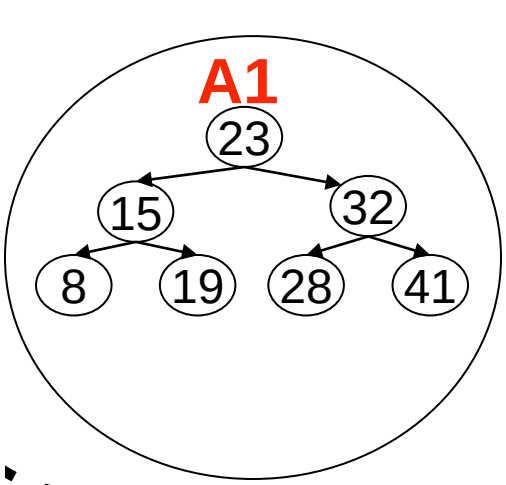
Exemplu (I)

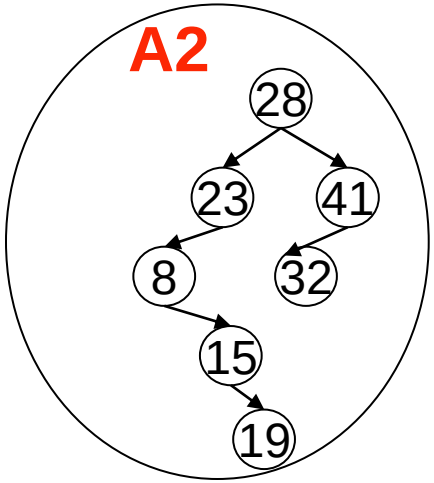
- Presupunem că elementele din A1 şi A2 au probabilităţi de căutare egale:
  - Numărul mediu de comparaţii pentru A1 va fi: (1 + 2 + 2 + 3 + 3 + 3 + 3) / 7 = 2.42
  - Numărul mediu de comparaţii pentru A2 va fi: (1 + 2 + 2 + 3 + 3 + 4 + 5) / 7 = 2.85

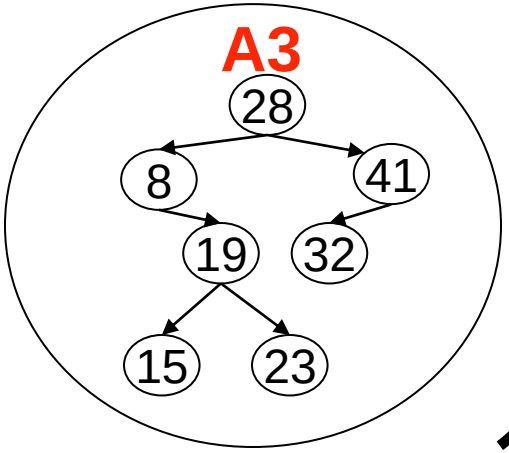


Exemplu (II)

- Presupunem că elementele au următoarele probabilităţi:
  - 8: 0.2; 15: 0.01; 19: 0.1; 23: 0.02; 28: 0.25; 32: 0.2; 41: 0.22;
  - Numărul mediu de comparaţii pentru A1:
    - 0.02*1+0.01*2+0.2*2+0.2*3+0.1*3+0.25*3+0.22*3=2.85
  - Numărul mediu de comparaţii pentru A2:
    - 0.25*1+0.02*2+0.22*2+0.2*3+0.2*3+0.01*4+0.1*5=2.47







Probleme

- Costul căutării depinde de frecvenţa cu care este căutat fiecare termen.
- → Ne dorim ca termenii cei mai des căutaţi să fie cât mai aproape de vârful arborelui pentru a micşora numărul de apeluri recursive.
- Dacă arborele nu este construit prin sosirea aleatorie a cheilor putem ajunge la o simplă listă cu n elemente.
  - Ce se întâmplă când cheile sunt deja sortate crescător / descrescător?

Definiţie AOC

- Definiție: Fie A un arbore binar de căutare cu chei într-o mulțime K; fie $\{x_1, x_2, \ldots x_n\}$ cheile conținute în A, iar $\{y_0, y_1, \ldots y_n\}$ chei reprezentante ale cheilor din K ce nu sunt în A astfel încât: $y_{i-1} \prec x_i \prec y_i, i = \overline{1,n}$. Fie p_i, i = 1,n probabilitatea de a căuta cheia x_i și q_j, j = 0,n probabilitatea de a căuta o cheie reprezentată de y_j. Vom avea relația: $\sum_{i=1}^{n} p_i + \sum_{j=0}^{n} q_j = 1$. A se numește arbore de căutare probabilistică, având costul:

$$Cost(A) = \sum_{i=1}^{n} (nivel(x_i, A) + 1) * p_i + \sum_{j=0}^{n} nivel(y_j, A) * q_j$$

- Definiție: Un arbore de căutare probabilistică având cost minim este un arbore optim la căutare (AOC).

Algoritm AOC naiv

- Generarea permutărilor x_1, ... x_n.
- Construcţia arborilor de căutare corespunzători.
- Calcularea costului pentru fiecare arbore.
- Alegerea arborelui de cost minim.
- Complexitate: Ω(n!) (deoarece sunt n! permutări).
- → căutăm altă variantă!!!

Construcţia AOC – Notaţii

- $A_{i,j}$ desemnează un AOC cu cheile $\{x_{i+1}, x_{i+2}, \ldots x_j\}$ în noduri și cu cheile $\{y_i, y_{i+1}, \ldots y_j\}$ în frunzele fictive.
- $C_{i,j}$ = Cost $(A_{i,j})$. $Cost(A_{ij}) = \sum_{k=i+1}^{j}(nivel(x_k, A_{ij})+1)* p_k + \sum_{k=i}^{j} nivel(y_k, A_{ij})* q_k$
- $R_{i,j}$ este indicele α al cheii x_α din rădăcina arborelui $A_{i,j}$.

$$w_{i,j} = \sum_{k=i+1}^{j} p_k + \sum_{k=i}^{j} q_k \quad w_{i,j} = \sum_{k=i+1}^{j-1} p_k + p_j + \sum_{k=i}^{j-1} q_k + q_j = w_{i,j-1} + p_j + q_j$$

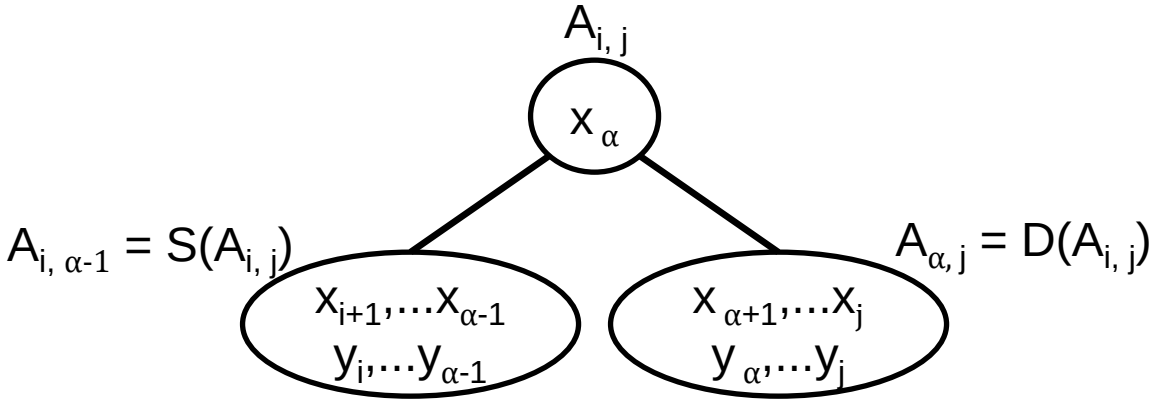
- Observație: $A_{0,n}$ este chiar arborele A, $C_{0,n}$ = Cost (A) iar $w_{0,n} = 1$.

Construcţia AOC - Demonstraţie

- Lemă: Fie $0 \le i < j \le n$ astfel încât există relaţia:

$$C_{i,j} = \min_{i<\alpha\le j}\{C_{i,\alpha-1} + C_{\alpha,j}\} + w_{i,j}$$

și pentru orice α, cu $i < \alpha \le j$, $A_{i,\alpha-1}$ şi $A_{\alpha,j}$ sunt AOC. Atunci arborele de mai jos este AOC ($A_{i,j}$).



Construcţia AOC - Demonstraţie

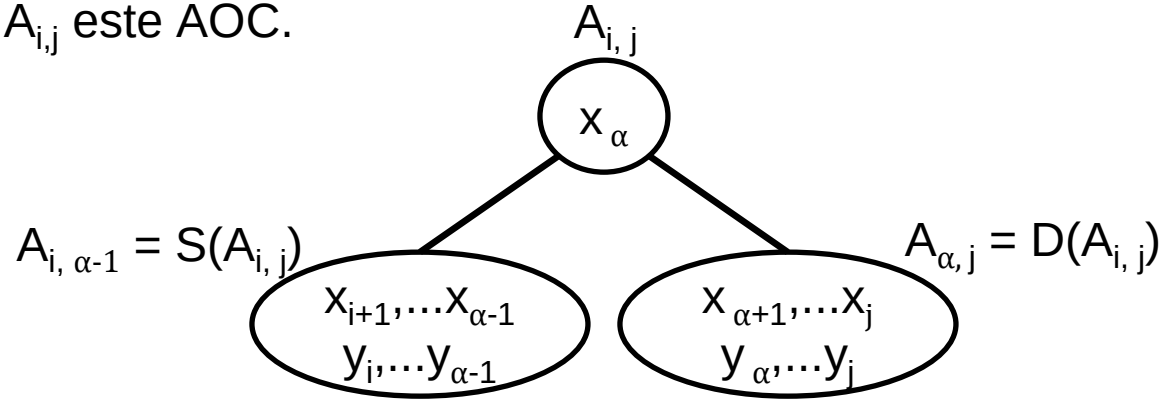
- Demonstrație:

$$Cost(A_{ij}) = \sum_{k=i+1}^{j} (nivel(x_k, A_{ij}) + 1) * p_k + \sum_{k=i}^{j} nivel(y_k, A_{ij}) * q_k$$

$$C_{i,j} = C_{i,\alpha-1} + C_{\alpha,j} + w_{i,j}$$

- $C_{i,j}$ depinde de indicele α al nodului rădăcină.

- Din ipoteză, $A_{i,\alpha-1}$ şi $A_{\alpha,j}$ sunt AOC → $C_{i,\alpha-1}$ şi $C_{\alpha,j}$ sunt minime → $C_{i,j}$ este minim → $A_{i,j}$ este AOC.



Construcţia AOC

- 1. În etapa d, d = 1, 2, … n se calculează costurile și indicele cheilor din rădăcina arborilor AOC $A_{i, i+d}$, i = 0, n-d cu d noduri și d + 1 frunze fictive.
- Arborele $A_{i, i+d}$ conține în noduri cheile $\{x_{i+1}, x_{i+2}, \dots x_{i+d}\}$, iar în frunzele fictive sunt cheile $\{y_i, y_{i+1}, \dots y_{i+d}\}$. Calculul este efectuat pe baza rezultatelor obținute în etapele anterioare.
- Conform lemei avem $$C_{i,i+d} = \min_{i<\alpha\le i+d}\{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$
- Rădăcina $A_{i, i+d}$ are indicele $R_{i, j} = \alpha$ care minimizează $C_{i, i+d}$.
- 2. Pentru d = n, $C_{0, n}$ corespunde arborelui AOC $A_{0, n}$ cu cheile $\{x_1, x_2, \dots x_n\}$ în noduri si cheile $\{y_0, y_1, \dots y_n\}$ în frunzele fictive.

Algoritm AOC

AOC(x, p, q, n)

Pentru *i* **de la** 0 **la** *n*

{$C_{i,i} = 0$, $R_{i,i} = 0$, $w_{i,i} = q_i$} // inițializare costuri AOC vid $A_{i,i}$

Pentru *d* **de la** 1 **la** *n*

Pentru *i* **de la** 0 **la** *n-d* // calcul indice rădăcină și cost pentru $A_{i,i+d}$

$j = i + d$, $C_{i,j} = \infty$, $w_{i,j} = w_{i,j-1} + p_j + q_j$

Pentru *α* **de la** *i + 1* **la** *j* // ciclul critic – operații intensive

Dacă ($C_{i,\alpha-1} + C_{\alpha,j} < C_{i,j}$) // cost mai mic?

{ $C_{i,j} = C_{i,\alpha-1} + C_{\alpha,j}$; $R_{i,j} = \alpha$} // update

$C_{i,j} = C_{i,j} + w_{i,j}$ // update

Întoarce gen_AOC(C, R, x, 0, n) // construcție efectivă arbore $A_{0,n}$

// cunoscând indicii

Complexitate???

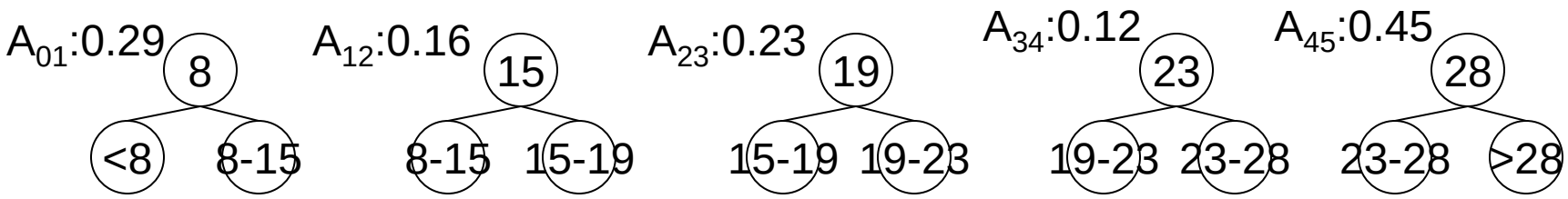
Exemplu constructie AOC (I)

- 8: 0.2; 15: 0.01; 19: 0.1; 23: 0.02; 28: 0.25; (58%)
- [0:8): 0.02; (8:15): 0.07; (15:19): 0.08; (19:23): 0.05; (23:28): 0.05; (28,∞): 0.15 (42%)
- $C_{01}=p_1+q_0+q_1=0.2+0.02+0.07=0.29$
- $C_{12}=p_2+q_1+q_2=0.01+0.07+0.08=0.16$
- $C_{23}=p_3+q_2+q_3=0.1+0.08+0.05=0.23$
- $C_{34}=p_4+q_3+q_4=0.02+0.05+0.05=0.12$
- $C_{45}=p_5+q_4+q_5=0.25+0.05+0.15=0.45$

$$w_{i,j} = \sum_{k=i+1}^{j} p_k + \sum_{k=i}^{j} q_k$$

$$w_{i,j} = w_{i,j-1} + p_j + q_j \qquad C_{i,i+d} = \min_{i<\alpha\le i+d}\{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$

Exemplu constructie AOC (II)

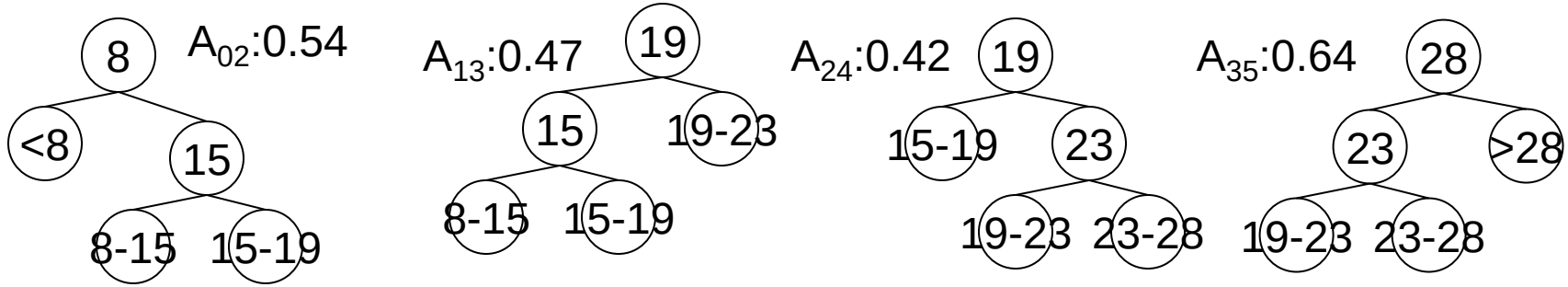


C_{02}= min {(C_{00}+C_{12}),(C_{01}+C_{22})} + w_{02} = min(0.16,0.29) + 0.38 = 0.54 R_{02}= 1 (α=1)

C_{13} = min{C_{11}+C_{23},C_{12}+C_{33}} + w_{13} = min(0.23,0.16) + 0.31 = 0.47 R_{13}= 3 (α=3)

C_{24} = min{C_{34},C_{23}} + w_{24} = min(0.12,0.23) + 0.3 = 0.42 R_{24}= 3

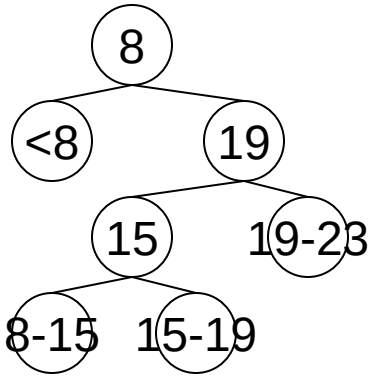
C_{35} = min{C_{45},C_{34}} + w_{35} = min(0.45,0.12) + 0.52 = 0.64 R_{35}= 5

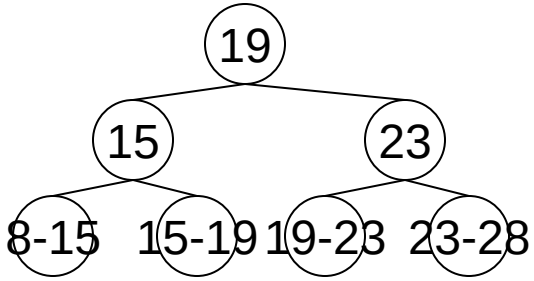


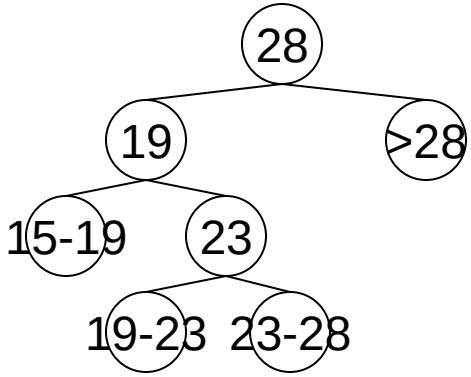
$$w_{i,j} = w_{i,j-1} + p_j + q_j \qquad C_{i,i+d} = \min_{i<\alpha\le i+d}\{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$

Exemplu constructie AOC (III)

- $C_{03} = \min(C_{00}+C_{13}, C_{01}+C_{23}, C_{02}+C_{33}) + w_{03} = \min(0.47, 0.52, 0.54) + w_{03} \Rightarrow R_{03} = 1$
- $C_{14} = \min(C_{11}+C_{24}, C_{12}+C_{34}, C_{13}+C_{44}) + w_{14} = \min(0.42, 0.28, 0.47) + w_{14} \Rightarrow R_{14} = 3$
- $C_{25} = \min(C_{22}+C_{35}, C_{23}+C_{45}, C_{24}+C_{55}) + w_{25} = \min(0.64, 0.67, 0.42) + w_{25} \Rightarrow R_{25} = 5$







$$w_{i,j} = \sum_{k=i+1}^{j} p_k + \sum_{k=i}^{j} q_k \qquad C_{i,i+d} = \min_{i<\alpha\le i+d} \{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$

AOC – Corectitudine (I)

- Teoremă: Algoritmul AOC construiește un arbore AOC A cu cheile x = $\{x_1, x_2, \ldots x_n\}$ conform probabilităților de căutare p_i, i = 1,n și q_j, j = 0,n.

- Demonstrație: prin inducție după etapa de calcul a costurilor arborilor cu d noduri.

- Caz de bază: d = 0. Costurile $C_{i, i}$ ale arborilor vizi $A_{i, i}$, i = 0, n sunt 0, așa cum sunt inițializate de algoritm.

- Pas de inducție: d ≥ 1. Ip. ind.: pentru orice d' < d, algoritmul AOC calculează costurile $C_{i, i+d'}$ și indicii $R_{i, i+d'}$, ai rădăcinilor unor AOC $A_{i, i+d'}$ cu i = 0, n-d' cu cheile $\{x_{i+1}, x_{i+2}, \ldots x_{i+d'}\}$. Trebuie să arătam că valorile $C_{i, i+d}$ și $R_{i, i+d}$ corespund unor AOC $A_{i, i+d}$ cu i = 0, n-d cu cheile $\{x_{i+1}, x_{i+2}, \ldots x_{i+d}\}$.

AOC – Corectitudine (II)

- Pentru d și i fixate, algoritmul calculează:

$$C_{i,i+d} = \min_{i<\alpha\leq i+d} \{C_{i,\alpha-1} + C_{\alpha,i+d}\} + w_{i,i+d}$$

- unde costurile $C_{i, \alpha-1}$ și $C_{\alpha, i+d}$ corespund unor arbori cu un număr de noduri d' = α – 1 – i în cazul $C_{i, \alpha-1}$ și d' = i + d – α în cazul $C_{\alpha, i+d}$.

- 0 ≤ d' ≤ d – 1 → aceste valori au fost deja calculate în etapele d' < d și conform ipotezei inductive → sunt costuri și indici ai rădăcinilor unor AOC.

- Conform Lemei anterioare, $C_{i, j}$ este costul unui AOC. Conform algoritmului → rădăcina acestui arbore are indicele r = $R_{i, j}$, iar cheile sunt $\{x_{i+1}, x_{i+2}, \ldots x_{r-1}\}$ $\{x_r\}$ $\{x_{r+1}, x_{r+2}, \ldots x_j\}$ = $\{x_{i+1}, x_{i+2}, \ldots x_j\}$

- Pentru d = n, costul $C_{0, n}$ corespunde unui AOC $A_{0, n}$ cu cheile x și cu rădăcina de indice $R_{0, n}$.

AOC – Concluzii

- Ns: Care este numărul total de subprobleme?

$O(n * (n – d + 1)) = O(n^2)$

- Na: Câte variante de ales avem de făcut pentru determinarea alegerii optimale într-un anumit pas?

$d = j – i = O(n)$ candidaţi pentru rădăcină

- Informal, complexitatea = Ns * Na (Ns = număr subprobleme; Na = număr alegeri)

Complexitate AOC: $O(n^2) * O(n) = O(n^3)$

Optimizare – Knuth: $O(n^2)$

ÎNTREBĂRI?



Proiectarea Algoritmilor

Curs 4 – Backtracking şi propagarea restricţiilor



Bibliografie

http://ktiml.mff.cuni.cz/~bartak/constraints/intro.html

Problema

	2		8	1		7	4	
7					3	1		
	9				2	8		5
		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	



SUDOKU

- Joc foarte la modă cu reguli foarte simple.
- Fiecare rând, coloană sau regiune nu trebuie să conţină decât o dată cifrele de la unu la nouă (Wikipedia).
- Prin trecerea în revistă a soluţiilor posibile pentru acest joc vom explora tehnicile de rezolvare backtracking şi propagarea restricţiilor.



Soluţia 1 – generează şi testează

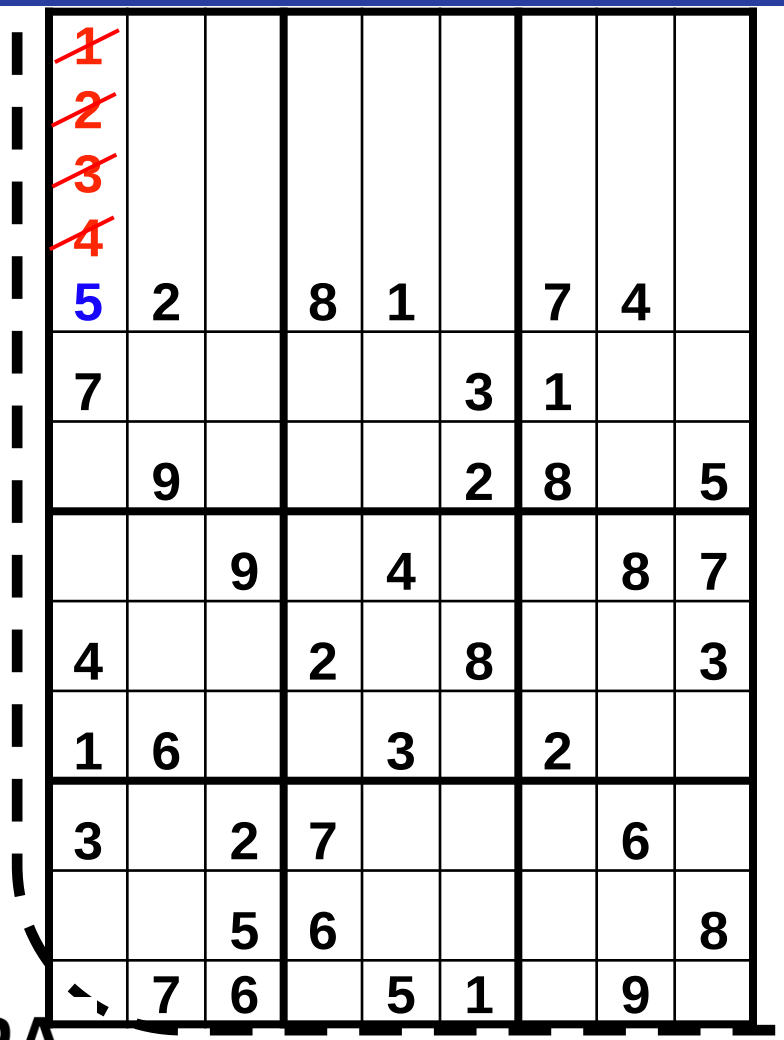
- Generăm toate soluţiile posibile şi le testăm.
- 45 spaţii de completat, 9 posibilităţi de completare pentru fiecare căsuţă => 9^{45} soluţii de testat.

1-9	2	1- 9	8	1	1-9	7	4	1-9
7	1-9	1-9	1-9	1-9	3	1	1-9	1-9
1-9	9	1-9	1-9	1-9	2	8	1-9	5
1-9	1-9	9	1-9	4	1-9	1-9	8	7
4	1-9	1-9	2	1-9	8	1-9	1-9	3
1	6	1-9	1-9	3	1-9	2	1-9	1-9
3	1-9	2	7	1-9	1-9	1-9	6	1-9
1-9	1-9	5	6	1-9	1-9	1-9	1-9	8
1-9	7	6	1-9	5	1	1-9	9	1-9

Soluţia 2 – Backtracking cronologic (orb) (I)

- Construieşte soluţiile iterativ.
- Menţine evidenţa alegerilor făcute.
- În momentul în care se ajunge la o contradicţie se revine la ultima decizie luată şi se încearcă alegerea unei alte variante.

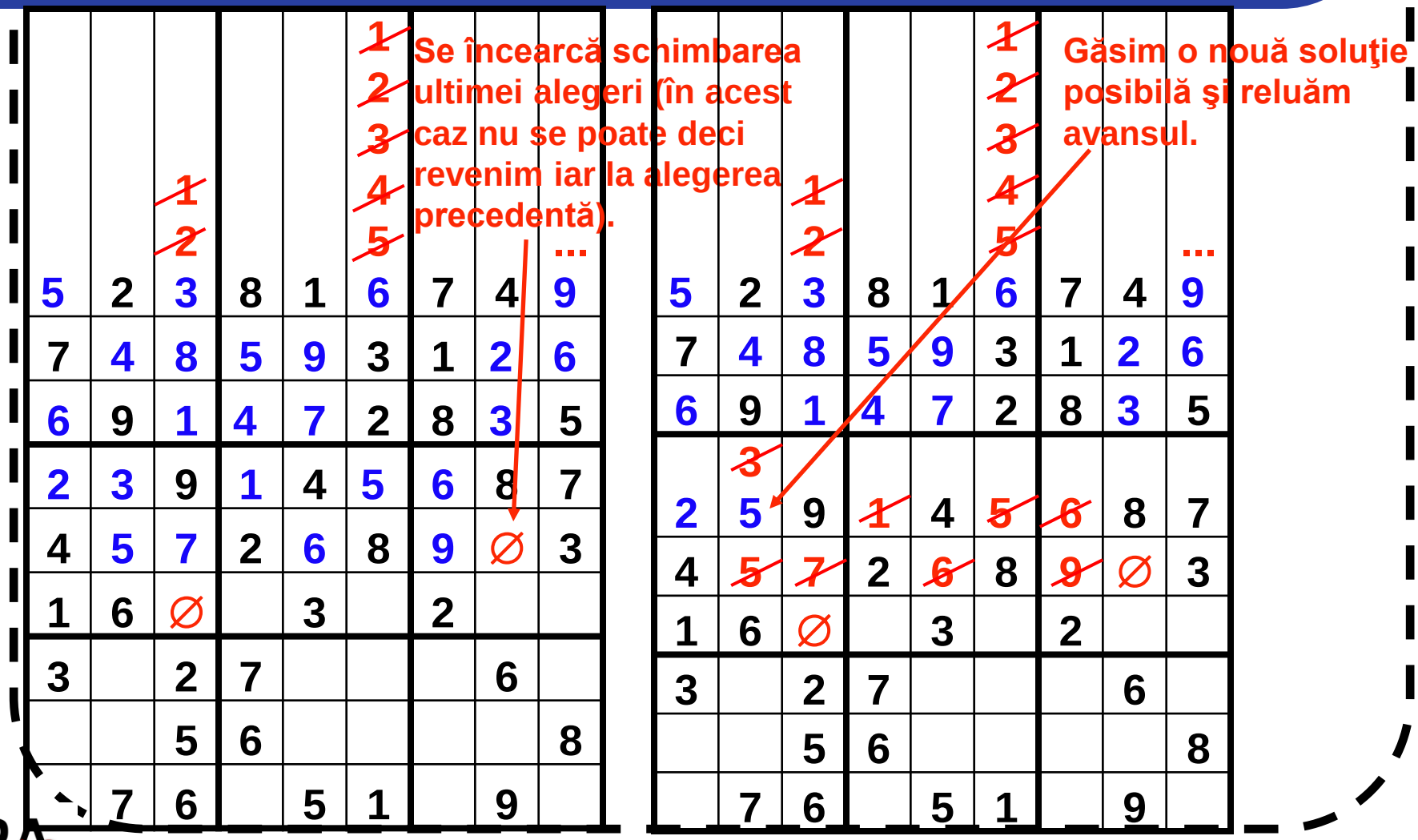
Soluţia 2 – Backtracking cronologic (orb) (II)





Nu se găseşte nici o soluţie pe această cale deci trebuie să revenim.

Soluţia 2 – Backtracking cronologic (orb) (III)



Soluţia 2 – Backtracking cronologic (orb) (IV)

Schema Backtracking

- Soluție-parţială ← INIT // inițializez
- EȘEC-DEFINITIV ← fals // nu am ajuns (încă) la eșec
- **Cât timp** Soluție-parţială nu este soluţie finală **și nu avem** EȘEC-DEFINITIV
  - Soluție-parţială ← AVANS (Soluție-parțială) // avansez
  - **Dacă** EȘEC (Soluție-parțială) // nu mai pot avansa
    - **Atunci** REVENIRE (Soluție-parțială) // mă întorc
- **Dacă** EȘEC-DEFINITIV
  - **Atunci** Întoarce EȘEC // nu s-a găsit nicio soluție
  - **Altfel** Întoarce SUCCES // am ajuns la soluția problemei
- **Sfârșit.**

Procedura AVANS (Soluție-parțială)

- **Dacă** există alternativă de extindere // pot avansa?
  - **Atunci** Soluție-parțială ← Soluție-parțială ∪ alternativă de extindere // avansez
  - **Altfel Dacă** Soluție-parțială este INIT
    - **Atunci** EȘEC-DEFINITIV ← adevărat // nu s-au găsit soluții pentru problemă
    - **Altfel** EȘEC (Soluție-parțială) // ramura curentă a dus la eșec

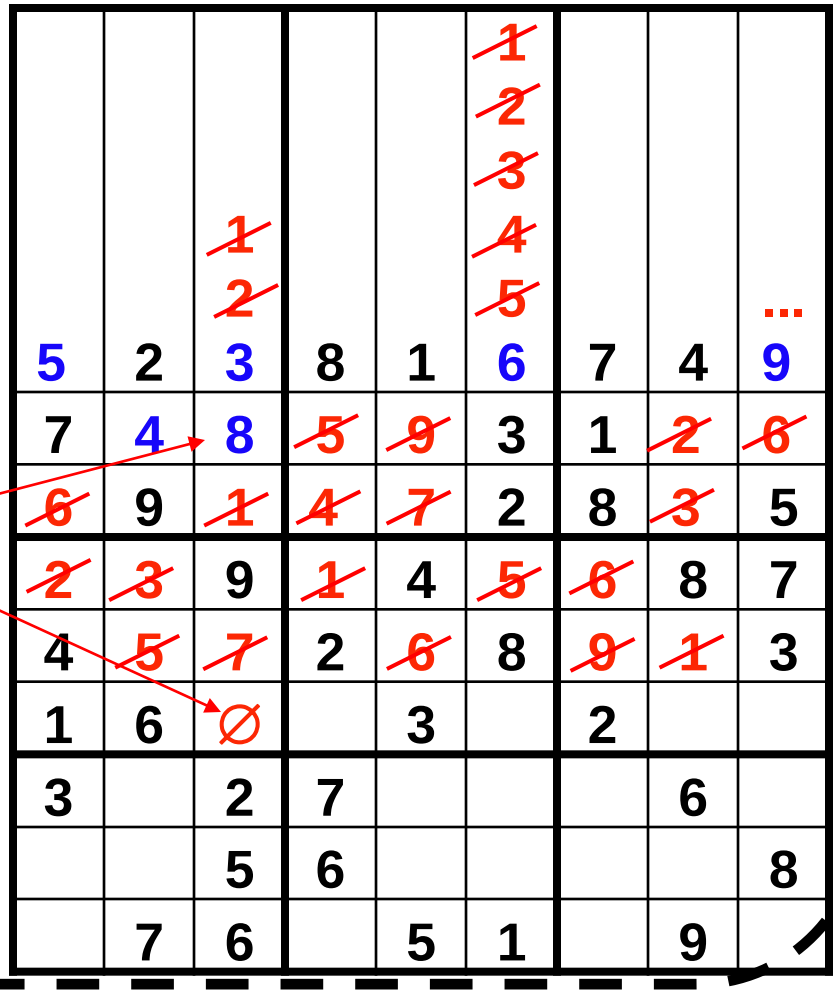
Backtracking – optimizări posibile (I)

- Alegerea variabilelor în altă ordine.
- Îmbunătăţirea revenirilor.
  - Necesită detectarea cauzei producerii erorii.
- Evitarea redundanţelor în spaţiul de căutare (îmbunătăţirea avansului).
  - Evitarea repetării unei căutări care ştim că va duce la un rezultat greşit.

Backtracking – optimizări posibile (II)

Îmbunătăţirea revenirilor

Revenire la alegerea variabilei care a cauzat eşecul (8 nu poate fi pus decât la poziţia indicată).



Backtracking – optimizări posibile (III)

Evitarea redundanţelor în spaţiul de căutare

Alegerea lui 8 pe această poziţie va produce un eşec în viitor indiferent de celelalte alegeri făcute deci în cazul revenirii în această poziţie nu are sens să mai facem această alegere.

					~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~			...
		~~1~~ ~~2~~						
5	2	3	8	1	6	7	4	9
7	4	~~8~~	~~5~~	~~9~~	3	1	~~2~~	~~6~~
~~6~~	9	~~1~~	~~4~~	~~7~~	2	8	~~3~~	5
~~2~~	~~3~~	9	~~1~~	4	~~5~~	~~6~~	8	7
4	~~5~~	~~7~~	2	~~6~~	8	~~9~~	~~1~~	3
1	6	∅		3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

Restricţii, reţele de restricţii, probleme de prelucrarea restricţiilor

- Definiție: O restricţie c este o relaţie între una sau mai multe variabile $v_1, \ldots, v_m$, (denumite nodurile sau celulele restricţiei). Fiecare variabila v_i poate lua valori într-o anumită mulţime D_i, denumită domeniul ei (ce poate fi finit sau nu, numeric sau nu).

- Definiție: Se spune că un tuplu (o atribuire) de valori $(x_1, \ldots, x_m)$ din domeniile corespunzătoare celor m variabile satisface restricţia $c(v_1, ..., v_m)$, dacă $(x_1, \ldots, x_m) \in c(v_1, ..., v_m)$.

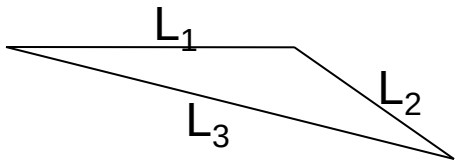
Exprimarea restricţiilor

- Enumerarea tuplurilor restricţiei.
  - **(5,2,3,8,1,6,7,4,9); (6,2,3,8,1,5,7,4,9); etc.**
- Formule matematice, cum ar fi ecuaţiile sau inecuaţiile.
  - $0 < V_{1j} < 10$; $V_{1j} \neq V_{1k}$; $\forall\ j \neq k,\ 0 < j, k < 10$
- Precizarea unei mulţimi de reguli.

	2		8	1		7	4	
7					3	1		
	9				2	8		5
		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

Tipuri de restricţii

- Unare
  - Specificarea domeniului variabilei.
  - $V_{11} \in \{1, 2, 3, 4, 5, 6, 7, 8, 9\} \setminus \{2, 8, 1, 7, 4, 3, 9\}$
- Binare
  - Între 2 variabile.
  - $V_{1j} \neq V_{1k}\ \forall j \neq k, 0 < j, k < 10$
- N-are
  - Între n variabile.
  - Regula triunghiului: $L_1 + L_2 > L_3$.



Probleme de satisfacerea restricţiilor

- Definiție: O problemă de satisfacere a restricţiilor (PSR) este un triplet <V,D,C>, format din:
  - o mulţime V formată din n variabile V = $\{v_1, \ldots, v_n\}$;
  - mulţimea D a domeniilor de valori corespunzătoare acestor variabile: D = $\{D_1, \ldots D_n\}$;
  - o mulţime C de restricţii C = $\{c_1, \ldots, c_p\}$ peste submulţimi ale V (c_i (v_{i1}, . . ., v_{ij}) $\subseteq$ D_{i1} x D_{i2} x ... x D_{ij}).

- Conform Definiției tuplului, o restricţie c_i (v_{i1}, ..., v_{ij}), este o submulţime a produsului cartezian D_{i1} x D_{i2} x ... x D_{ij}, constând din toate tuplurile de valori considerate că satisfac restricţia pentru (v_{i1}, . . ., v_{ij}).

Soluții ale PSR

- Definiție: O soluţie a unei PSR <V,D,C> este un tuplu de valori <x_1, … , x_n> care conține toate variabilele V, din domeniile corespunzătoare D, astfel încât toate restricţiile din C să fie satisfăcute.

- Definiție: PSR binară este o PSR ce conţine doar restricţii unare şi binare.

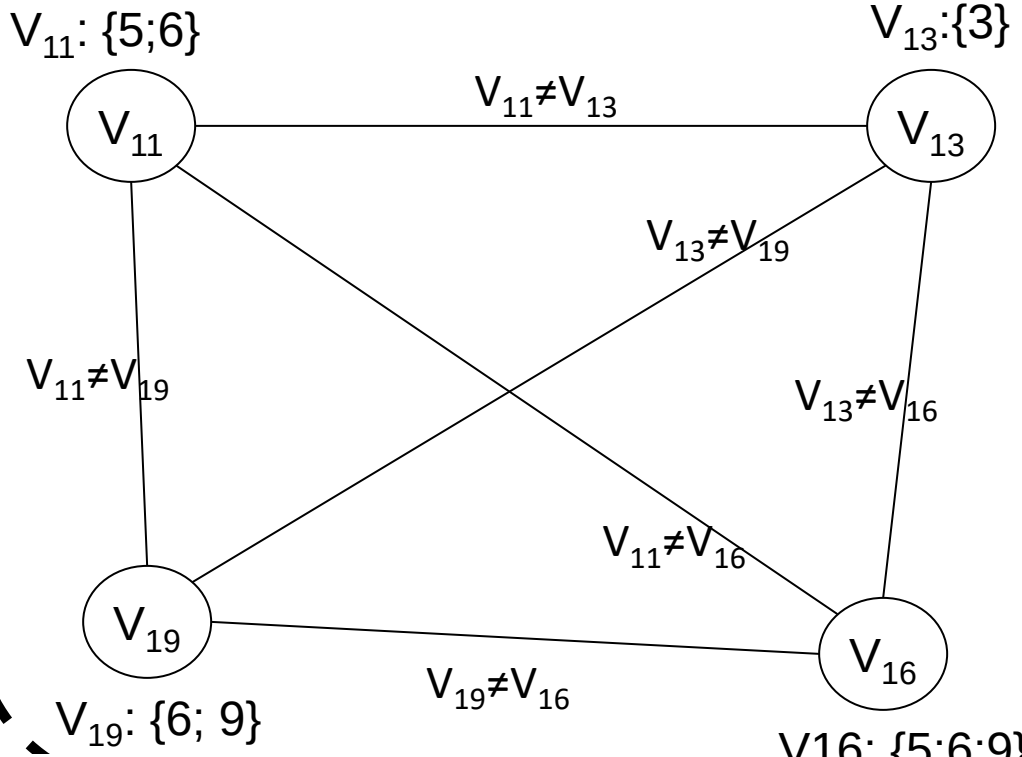
Probleme de satisfacere a restricţiilor - Reprezentare

- Reprezentare PSR prin reţele de restricţii.
- Reprezentarea folosită: graf
  - Nodurile: variabilele restricţiei
  - Arcele: restricţiile problemei
  - Valabilă doar pentru PSR binare
- Altă reprezentare posibilă: graf bipartit
  - Nodurile: restricţiile și variabilele problemei
  - Arcele: între restricţie și variabilele implicate
  - Valabilă pentru orice tip de PSR



Exemplu de reprezentare a PSR

- $0 < V_{1j} < 10$; $V_{1j} \neq V_{1k}$; $\forall j \neq k, 0 < j, k < 10$



5;6	2	3	8	1	5;6;9	7	4	6;9
7					3	1		
	9				2	8		5
		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

Algoritmi de rezolvare a PSR – Propagarea restricțiilor

- Caracteristici
  - Rezolvă PSR binare;
  - Variabilele au domenii finite de valori;
  - Prin propagarea restricţiilor se filtrează mulţimile de valori (se elimină elementele din domeniu conform unui criteriu dat);
  - Procesul de propagare se opreşte când:
    - O mulţime de valori este vidă → EȘEC;
    - Nu se mai modifică domeniul vreunei mulţimi.



Algoritmi de rezolvare a PSR – Propagarea restricțiilor

Notaţii:

- n = număr variabile = număr restricţii unare;
- r = număr restricţii binare;
- G = reţeaua de restricţii cu variabile drept noduri și restricţii drept arce;
- D_i = domeniul variabilei i;
- Q_i = predicat care verifică restricţia unară pe variabila i;
- P_{ij} = predicatul care reprezintă restricţia binară pe variabilele i și j (O muchie între i și j se înlocuiește cu arcele orientate de la i la j și de la j la i);
- a = max $|D_i|$.

NC-1 (Node Consistency -1)

- Algoritm de consistenţa nodurilor (pentru restricţii unare).

- Procedura NC(i) este:
  - **Pentru fiecare** $x \in D_i$
    - **Dacă** not $Q_i(x)$ // nu este satisfăcută restricția unară
      - **Atunci** șterge x din D_i
  - **Sfârșit**.

- Algoritm NC-1 este:
  - **Pentru** i **de la** 1 **la** *n* **Execută** NC(i) // pentru fiecare var
  - **Sfârșit**.

Complexitate NC-1? na

NC-1: Exemplu (I)

- Elimină din domeniul de valori al fiecărui nod valorile care nu satisfac restricţiile care au ca argument variabila din nodul respectiv.
- În cazul Sudoku variabilele iniţial iau valori între 1-9.
- Algoritmul NC-1 elimină pentru fiecare variabilă acele valori din domeniu care nu sunt consistente cu valorile fixe (celulele deja fixate).

NC-1: Exemplu (II)

5;6	2	3;	8	1	5;6;9	7	4	6;9;
7	4;5;8;	4;8;			3	1		
6;	9	1;3;4;			2	8		5
		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
		5	6					8
	7	6		5	1		9	

Algoritmi de consistenţă a arcelor

- Algoritmii de consistenţă a arcelor înlătură toate inconsistenţele submulţimilor de 2 elemente ale reţelei de restricţii.

- Funcţia REVISE ((i,j)) este:
  - ȘTERS ← fals // nu am modificat domeniul de valori
  - **Pentru fiecare** $x \in D_i$
    - **Dacă** nu există $y \in D_j$ a.î. $P_{ij}(x,y)$ // nu se respectă restricția
      - **Atunci**
        - Șterge x din D_i;
        - ȘTERS ← adevărat; // am făcut modificări
  - **Întoarce** ȘTERS.

Exemplu funcţionare REVISE

x

y

5;~~6~~	2	3;	8	1	5;6;9	7	4	6;9;
7	4;5;8;	4;8;			3	1		
6;	9	1;~~3~~;4;			2	8		5
5;6;2;		9		4			8	7
4			2		8			3
1	6			3		2		
3		2	7				6	
9;		5	6					8
8;	7	6		5	1		9	

Complexitate Revise ? a^2

REVISE - Concluzie

- Funcţia Revise este apelată pentru un arc al grafului de restricţii (binare) și șterge acele valori din domeniul de definiţie al unei variabile pentru care nu este satisfăcută restricţia pentru nici o valoare corespunzătoare celeilalte variabile a restricţiei.

- Complexitate Revise: $O(a^2)$



AC-1 (Arc Consistency -1)

- Algoritm AC-1 este:
  - NC-1; // reduc domeniul de valori
  - Q ← {(i,j) | (i,j) ∈ arce(G), i ≠ j} // adaug restricțiile
  - **Repetă**
    - SCHIMBAT ← fals // nu am modificat niciun // domeniu
    - **Pentru fiecare** (i,j) ∈ Q // pentru fiecare restricție
      - SCHIMBAT ← (REVISE ((i,j)) sau SCHIMBAT)
  - **Până când *non*** SCHIMBAT // nu am mai făcut // modificări
  - **Sfârșit.**

AC-1 Caracteristici & Complexitate

- Se aplică algoritmul de consistenţa nodurilor şi apoi se aplică REVISE până nu se mai realizează nici o schimbare.

- Complexitate: $O(na * 2r * a^2)$

La fiecare iteraţie (din Repetă) eliminăm o singură valoare (şi avem maxim na valori posibile).

Numărul de apelări al Revise într-o iterație (din Repetă).

Complexitate Revise.

Exemplu AC-1

5,~~6~~	2	3;	8	1	5;6;9	7	4	6;9;
7	4;5;8;	4;8;			3	1		
6;	9	1;~~3~~,4;			2	8		5
~~5;6;2;~~	~~3,5~~	9		4			8	7
4	5	7	2		8			3
1	6	8		3		2		
3		2	7				6	
9;		5	6					8
8;	7	6		5	1		9	

AC-3 (Arc Consistency -3)

- Algoritm AC-3 este:
  - NC-1; // reduc domeniul de valori
  - Q ← {(i,j) | (i,j) ∈ arce(G), i ≠ j} // adaug // restricțiile
  - **Cât timp** Q nevid
    - Selectează și șterge un arc (k,m) din Q;
    - **Dacă** REVISE ((k,m)) // am modificat domeniul
      - **Atunci** Q ← Q ∪ {(i,k) | (i,k) ∈ arce(G), i ≠ k, i ≠ m} // verific dacă nu se modifică și alte domenii

AC-3 Caracteristici

- Se elimină pe rând arcele (constrângerile).
- Dacă o constrângere aduce modificări în reţea adăugăm pentru reverificare nodurile care punctează către nodul de plecare al restricţiei verificate.
  - Scopul: Reverificarea nodurilor direct implicate de o constrângere din reţea.
- Avantaj: Se fac mult mai puţine apeluri ale funcţiei REVISE.
- Complexitate: $O(a^3r)$.

Backtracking + Propagarea restricţiilor

- În general, propagarea restricţiilor nu poate rezolva complet problema dată.
- Metoda ajută la limitarea spaţiului de căutare (foarte importantă în condiţiile în care backtracking-ul are complexitate exponenţială!).
- În cazul în care propagarea restricţiilor nu rezolvă problema se foloseşte:
  - Backtracking pentru a genera soluţii parţiale;
  - Propagarea restricţiilor după fiecare pas de backtracking pentru a limita spaţiul de căutare (şi eventual a găsi că soluţia nu este validă).

ÎNTREBĂRI?



Proiectarea Algoritmilor

Curs 5 – Algoritmi pentru jocuri

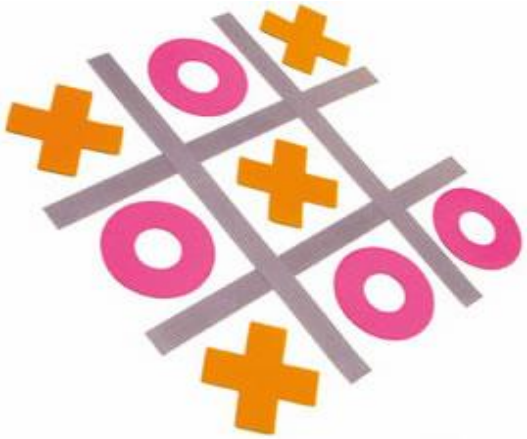
Minimax, α-β

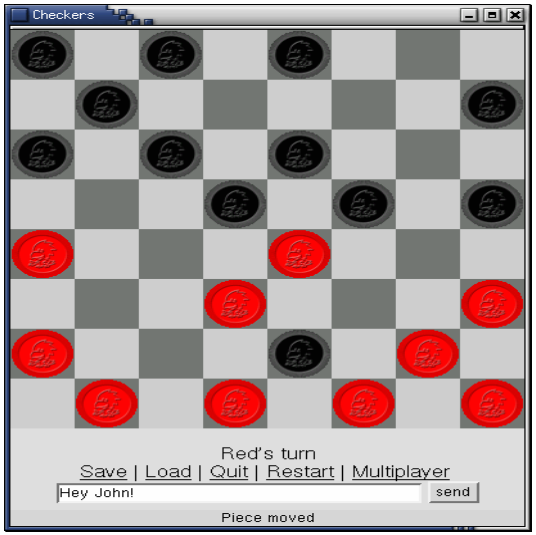


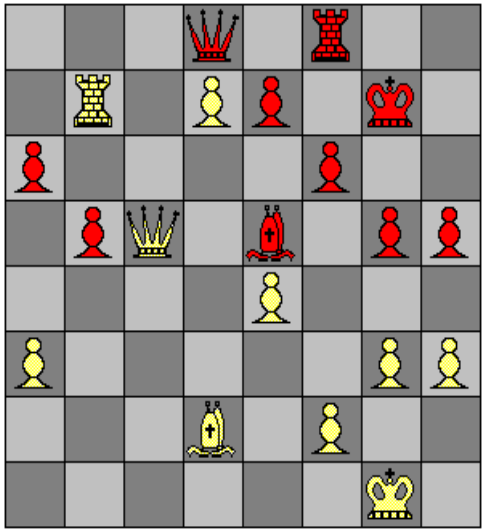
Bibliografie

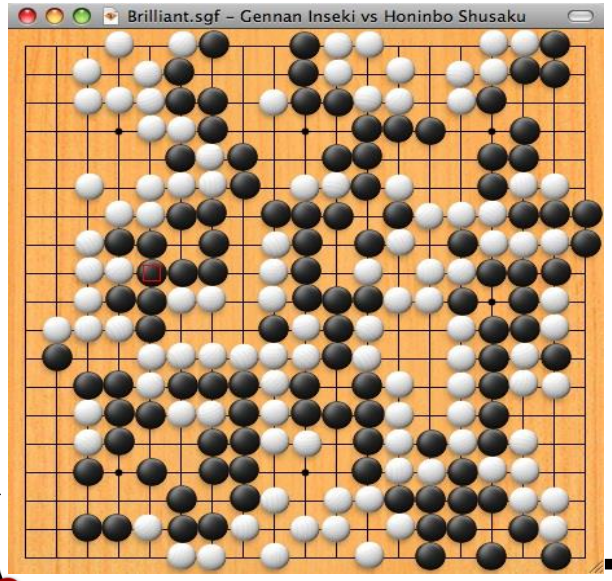
- Giumale – Introducere in Analiza Algoritmilor cap 7.6
- http://www.dwheeler.com/chess-openings/#Sicilian%20Defense
- http://mouserunner.com/MozllaTicTacToe/Mozilla_Tic_Tac_Toe.htm
- http://www.emunix.emich.edu/~evett/AI/AlphaBeta_movie/index_movie.htm

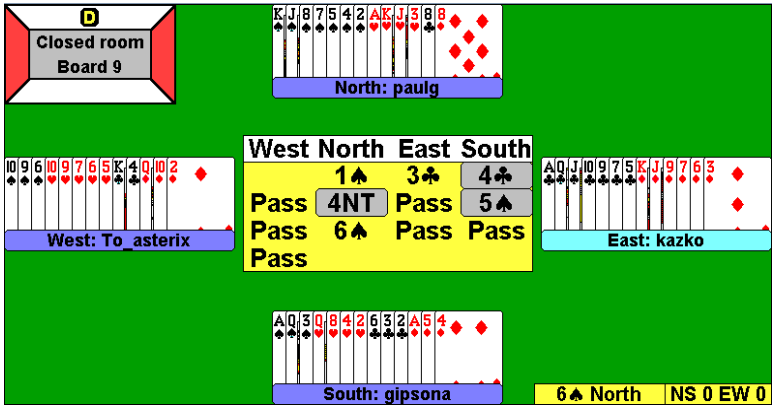
Problema



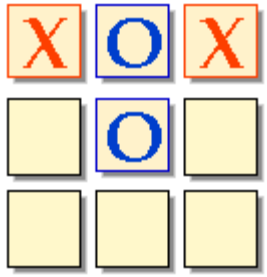

Checkers
Red's turn
Save | Load | Quit | Restart | Multiplayer
Hey John!
send
Piece moved



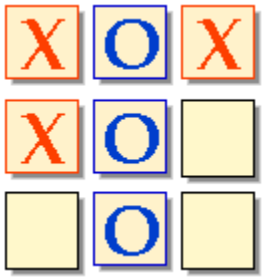

Brilliant.sgf – Gennan Inseki vs Honinbo Shusaku


Closed room
Board 9
North: paulg
West: To_asterix
East: kazko
South: gipsona
West North East South
1♠ 3♣ 4♣
Pass 4NT Pass 5♠
Pass 6♠ Pass Pass
Pass
6♠ North
NS 0 EW 0

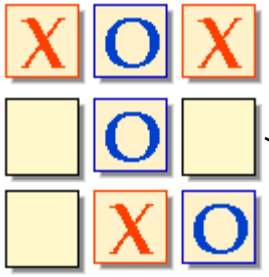
Cum gândim noi?



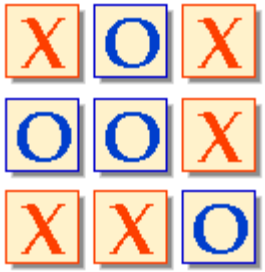
Analizăm posibilitățile și evaluăm
fiecare mutare în funcție de consecințe.



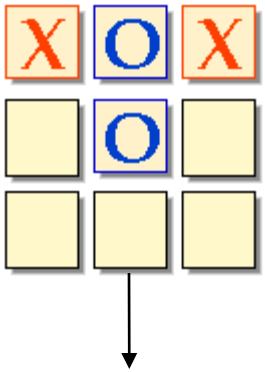
Am pierdut! :(



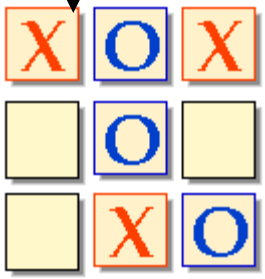
Egal! :(

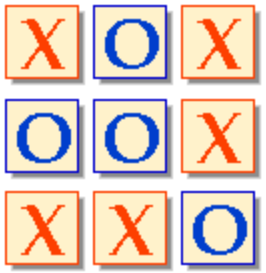


Cum gândim noi?



Ne dăm seama ”instinctiv” că avem o singură opțiune pentru a nu pierde partida și mutăm în consecință!





Egal! :(

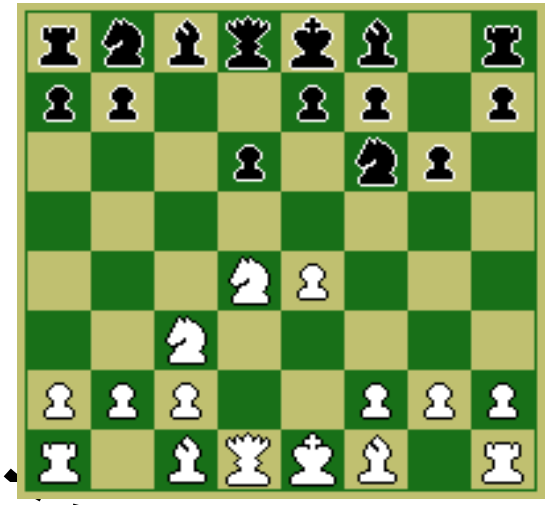
Cum gândim noi?

http://www.dwheeler.com/chess-openings/#Sicilian%20Defense



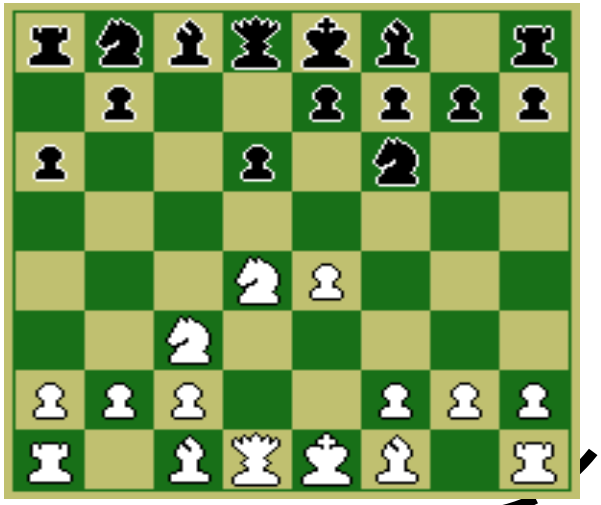
Apărarea siciliană!

Varianta Najdorf



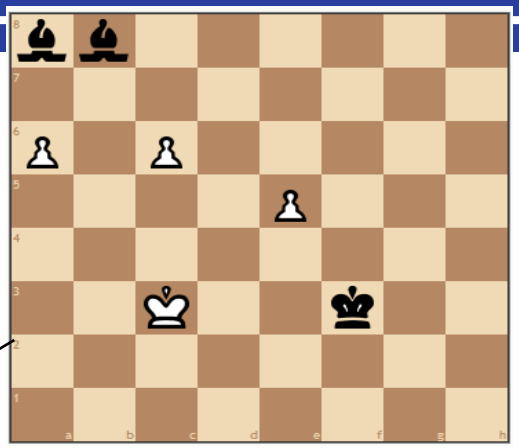
Când avem foarte multe posibilități la dispoziție încercăm să folosim poziții (pattern-uri) cunoscute.

Varianta Dragon

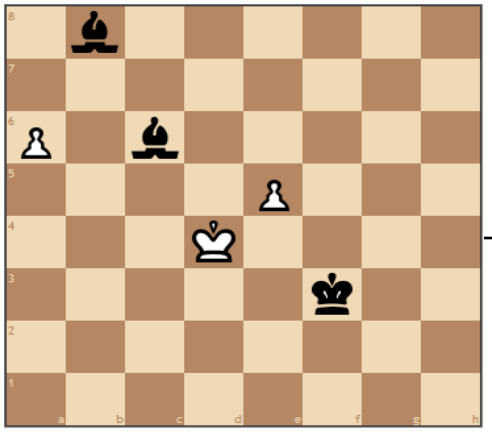


Cum gândim noi?

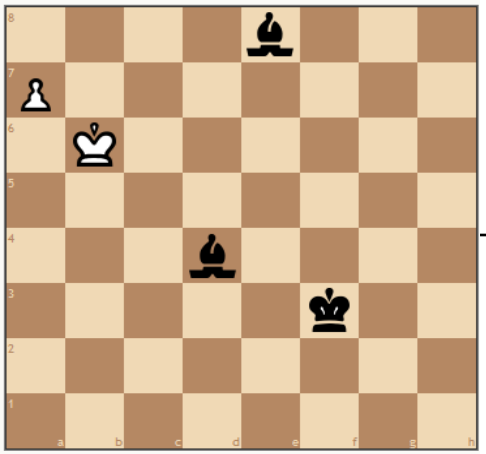
Albul la mutare
-11 posibilități de mutare;
- le putem încerca pe toate să vedem ce se întâmplă.



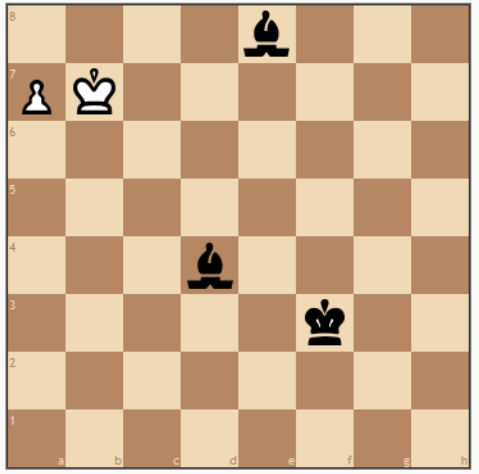
Circa 15.000 de mutări de analizat – ușor pentru calculator. Noi eliminăm mutările ce ni se par fără sens (mai mult de jumătate).



Numărul mutărilor posibile se reduce la 6.



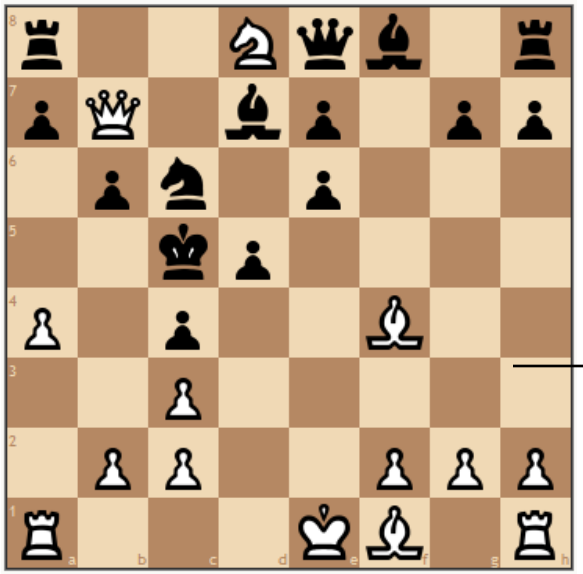
Doar 4 mutări posibile.



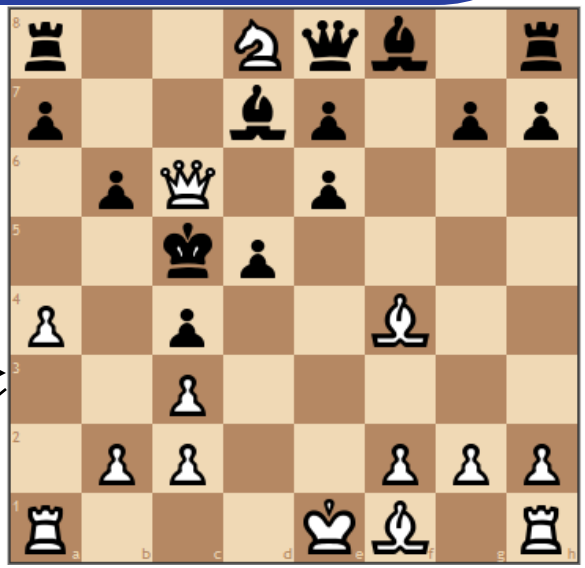
Remiză asigurată!

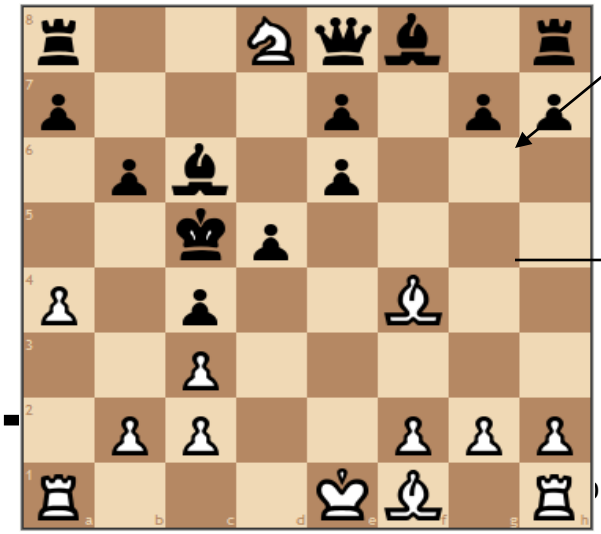
Cum gândim noi?

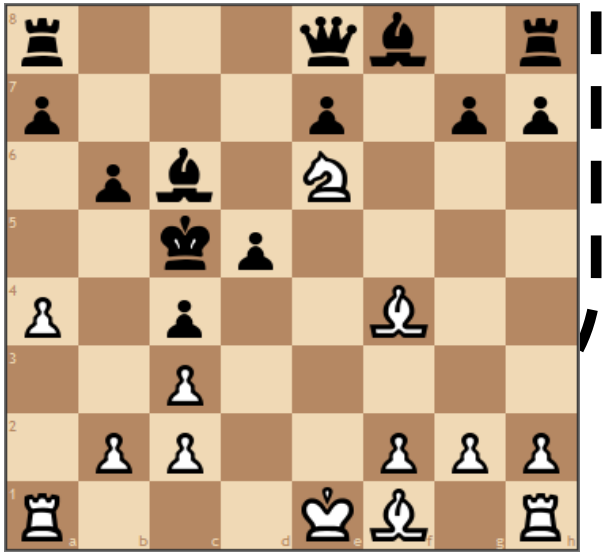
Cam 35 de mutări posibile



Varianta câștigătoare presupune sacrificarea unei piese valoroase:







Cum gândim noi?

- Evaluăm amenințările:
  - Căutăm mutări care să minimizeze pierderile;
  - Căutăm mutări care să maximizeze câștigul.
- Alegem mutările ce ni se ”par” cele mai bune pe moment:
  - Explorăm în adâncime graful mutărilor;
  - Numărul de niveluri = minim dintre:
    - Terminarea jocului;
    - Obținerea unui avantaj consistent fără pericol aparent de a-l pierde;
    - Nivelul maxim al capacității noastre de calcul.



Abordări posibile pentru calculator

- Șabloane pentru poziții standard.
- Căutare în spațiul de poziții.
- Utilizare euristici pentru evaluarea poziției curente.
- Ne vom concentra asupra căutărilor.

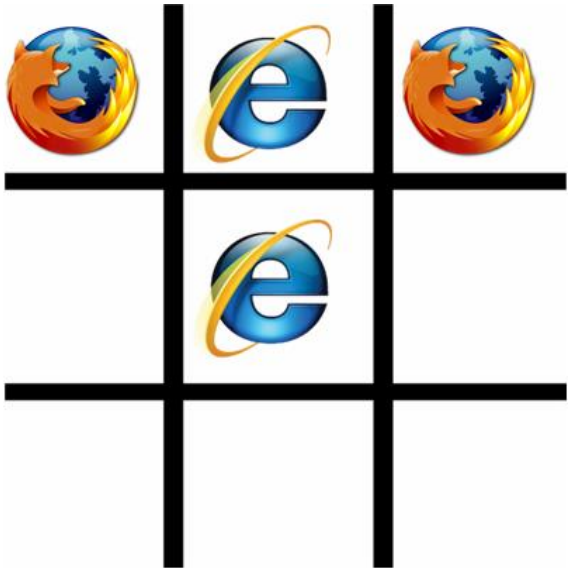


Metoda Minimax

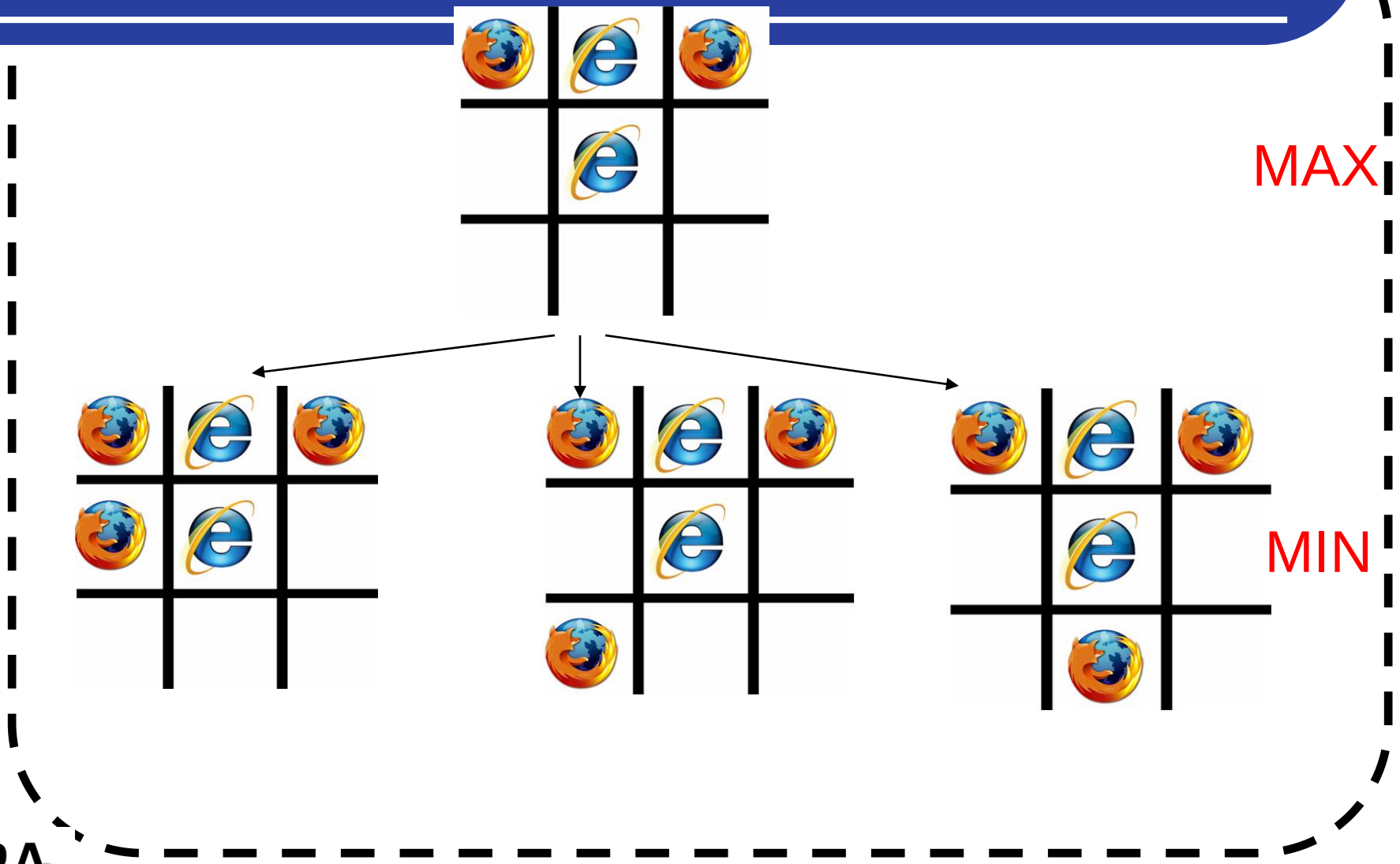
- 2 jucători: Max și Min care mută pe rând (Max mută primul).
- Max urmărește să-și maximizeze câștigul.
- Min urmărește să-și minimizeze pierderea.
- Se construiește un arbore de tip AND-OR:
- Nivelurile pare → mutările jucătorului Max.
- Nivelurile impare → mutările jucătorului Min.
- Frunzele desemnează câștigul/pierderea lui Max.
- Arcele reprezintă mutările propriu-zise.

Exemplu (I)

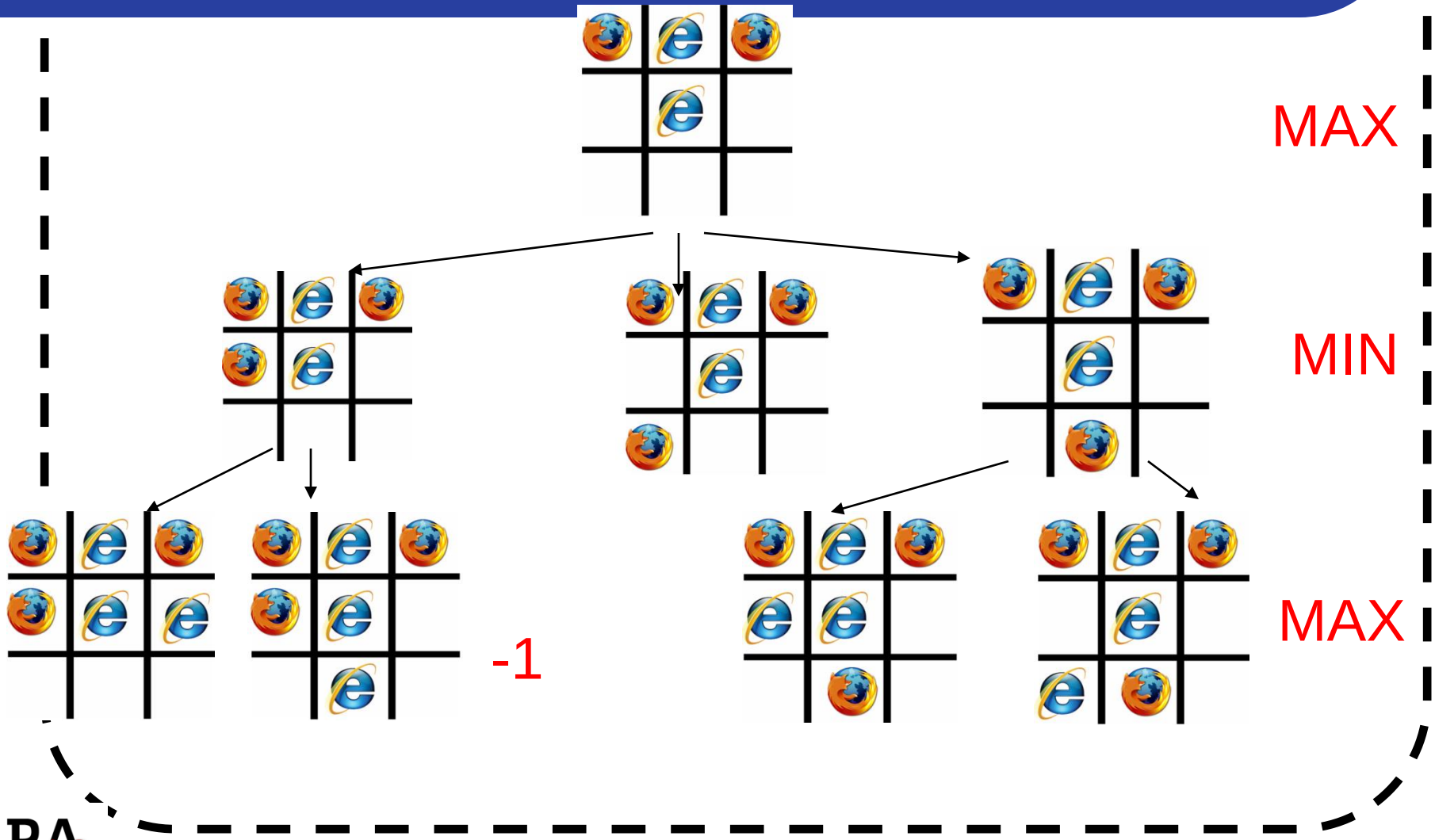
MAX (Firefox) trebuie să mute:



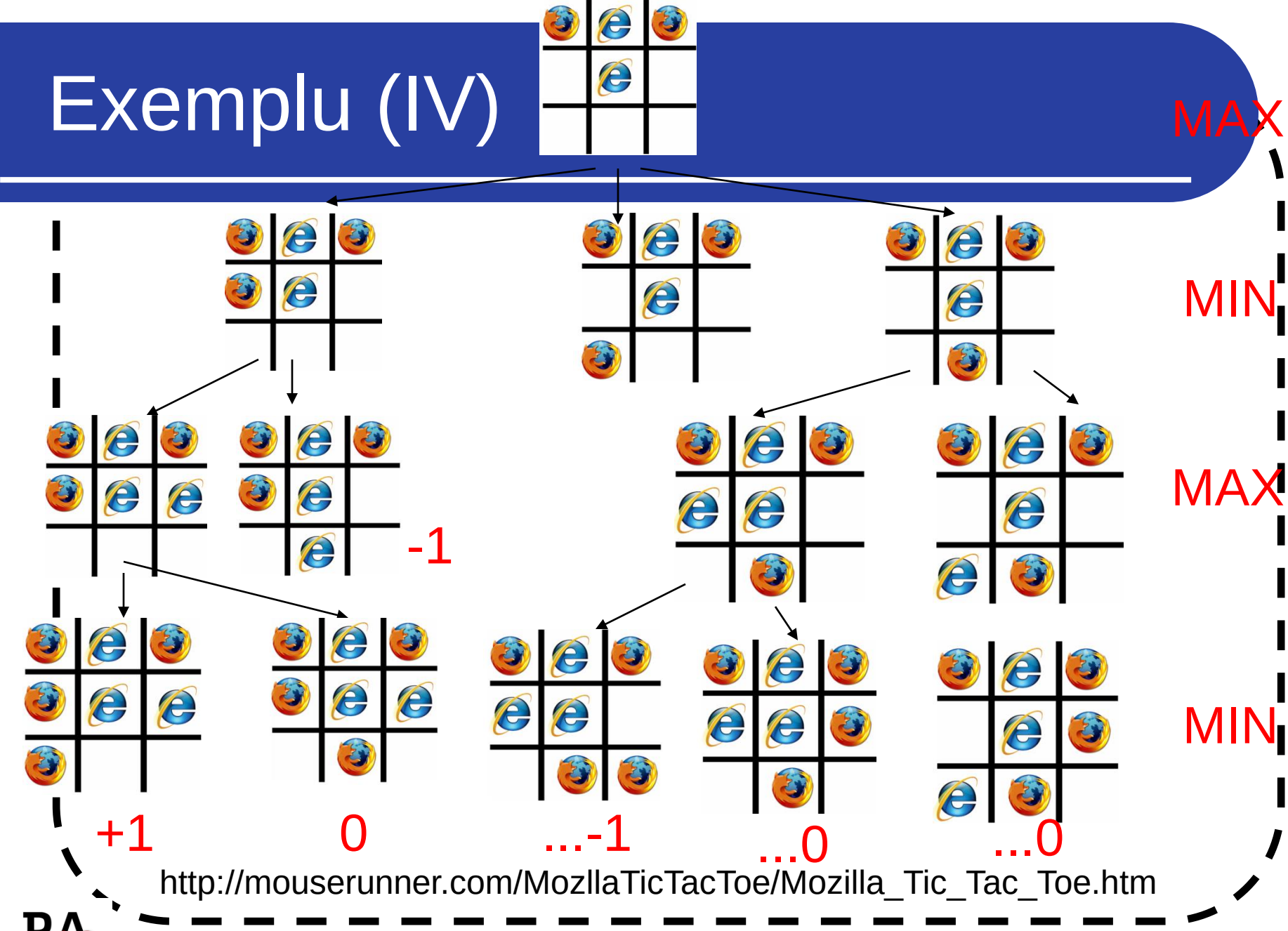
Exemplu (II)



Exemplu (III)



Exemplu (IV)



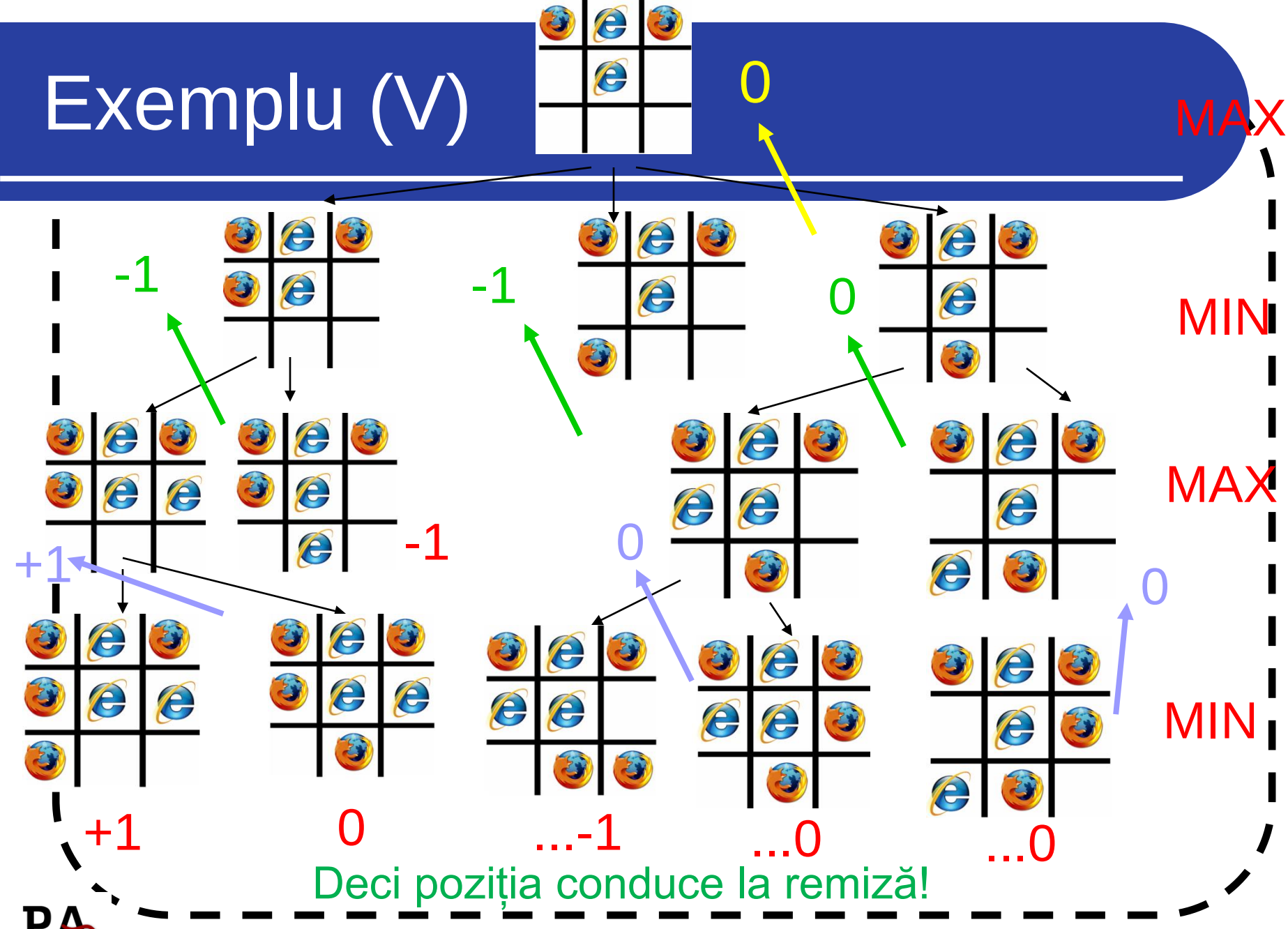
http://mouserunner.com/MozllaTicTacToe/Mozilla_Tic_Tac_Toe.htm

Funcționare Minimax

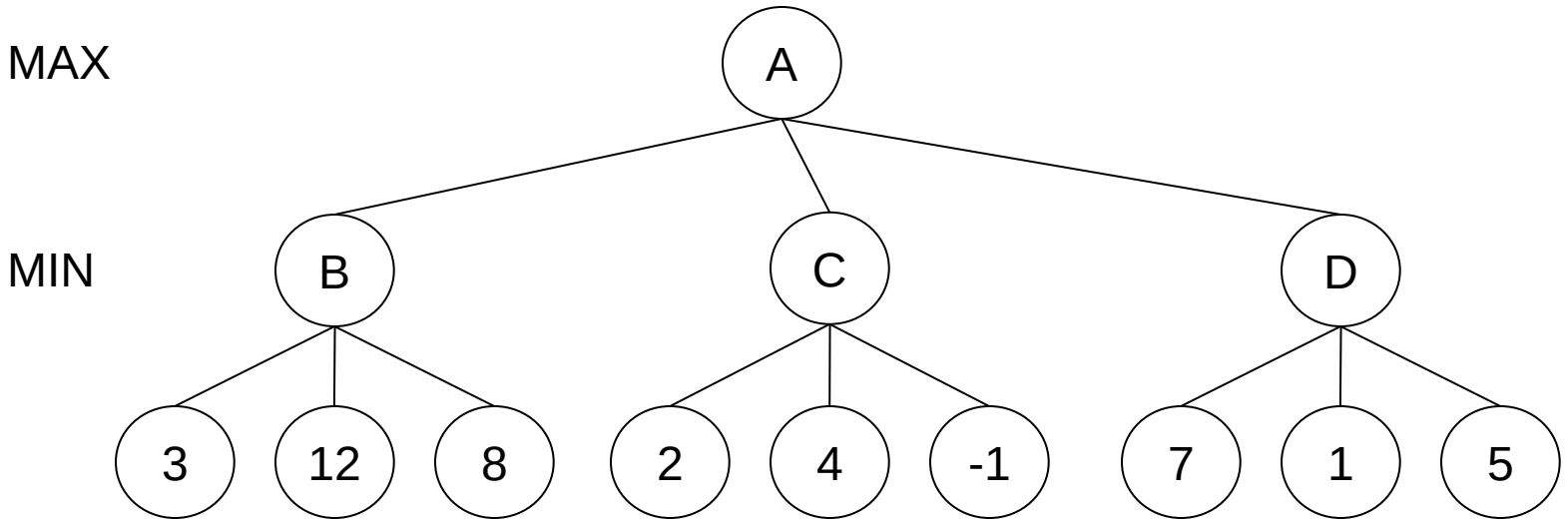
- 1) Se generează întregul arbore;
- 2) Se evaluează frunzele și li se asociază valori;
- 3) Se propagă rezultatele dinspre frunze spre rădăcină astfel:
  - Nivelul MIN alege cea mai mică valoare dintre cele ale copiilor.
  - Nivelul MAX alege cea mai mare valoare dintre cele ale copiilor.



Exemplu (V)


0
MAX
-1
-1
0
MIN
-1
MAX
+1
0
0
MIN
+1
0
...-1
...0
...0
Deci poziția conduce la remiză!

Alt exemplu (I)



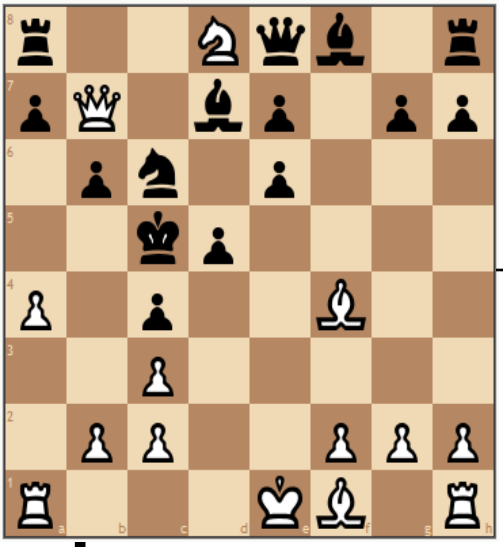
Probleme

- Dimensiunea arborelui pentru “X și 0” e ≤ 9!
- Pentru Șah fiecare nod are în medie 35 copii!
- Pentru Go ramificarea este de cca. 150 – 250!
- Complexitatea arborelui:
  - pentru Șah – 10^{123} noduri;
  - pentru Go – 10^{360} noduri.
- Limitări:→ Nu putem să construim întregul arbore → Nu putem ajunge de fiecare dată la stările finale pentru a le putea evalua.

Optimizări minimax

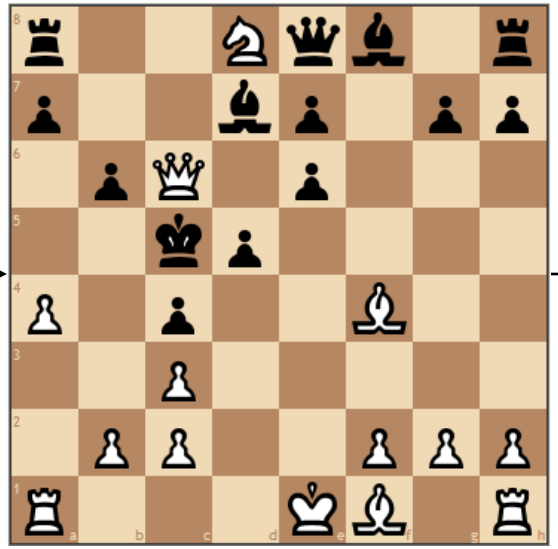
- Limitarea adâncimii căutării
  - Trebuie să construim o funcție euristică care să estimeze șansele de câștig pentru o poziție dată.
    - Ex. pentru șah:
      - Regină:10p; Turn: 5p; Cal, Nebun: 3p; Pion: 1p;
      - Ex: Funcție de evaluare a poziției = suma pieselor proprii – suma pieselor adversarului.
  - Oprirea căutării:
    - Limitare statică: după un număr maxim de nivele/interval de timp.
    - Limitare dinamică: când profitul obținut din continuarea căutării devine foarte mic (scade sub o valoare fixată).
  - Se estimează valoarea funcției de evaluare la nivelul respectiv.
  - Apoi propagăm valorile conform principiului enunțat anterior.

Exemplu și contraexemplu



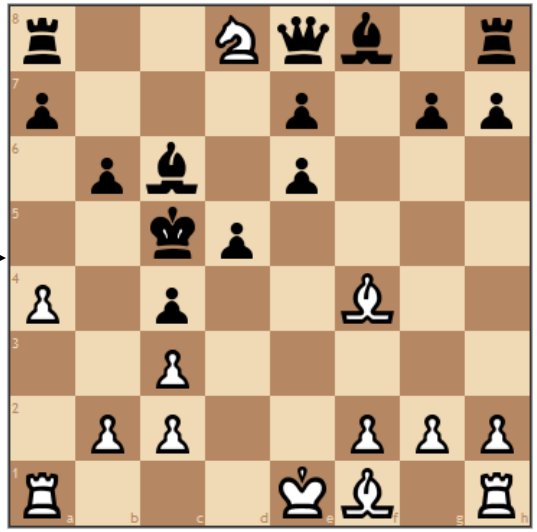
Eval: 36-37=-1

Funcția nu ține cont de poziție – albul are o poziție net superioară dar funcția de evaluare o ignoră



Eval: 36-34=2

Dacă căutarea se oprește la acest nivel atunci aparent albul iese în câștig material ignorându-se faptul că la mutarea următoare se pierde dama



Eval: 26-34=-8

În cazul în care căutarea se oprește la acest nivel aparent albul iese in dezavantaj deoarece a pierdut dama

Minimax – funcții de evaluare

- Funcția euristică trebuie să cuantifice ”poziția”.
  - Chiar în dauna avantajului material.
- Trebuie să ia în calcul potențialele amenințări!



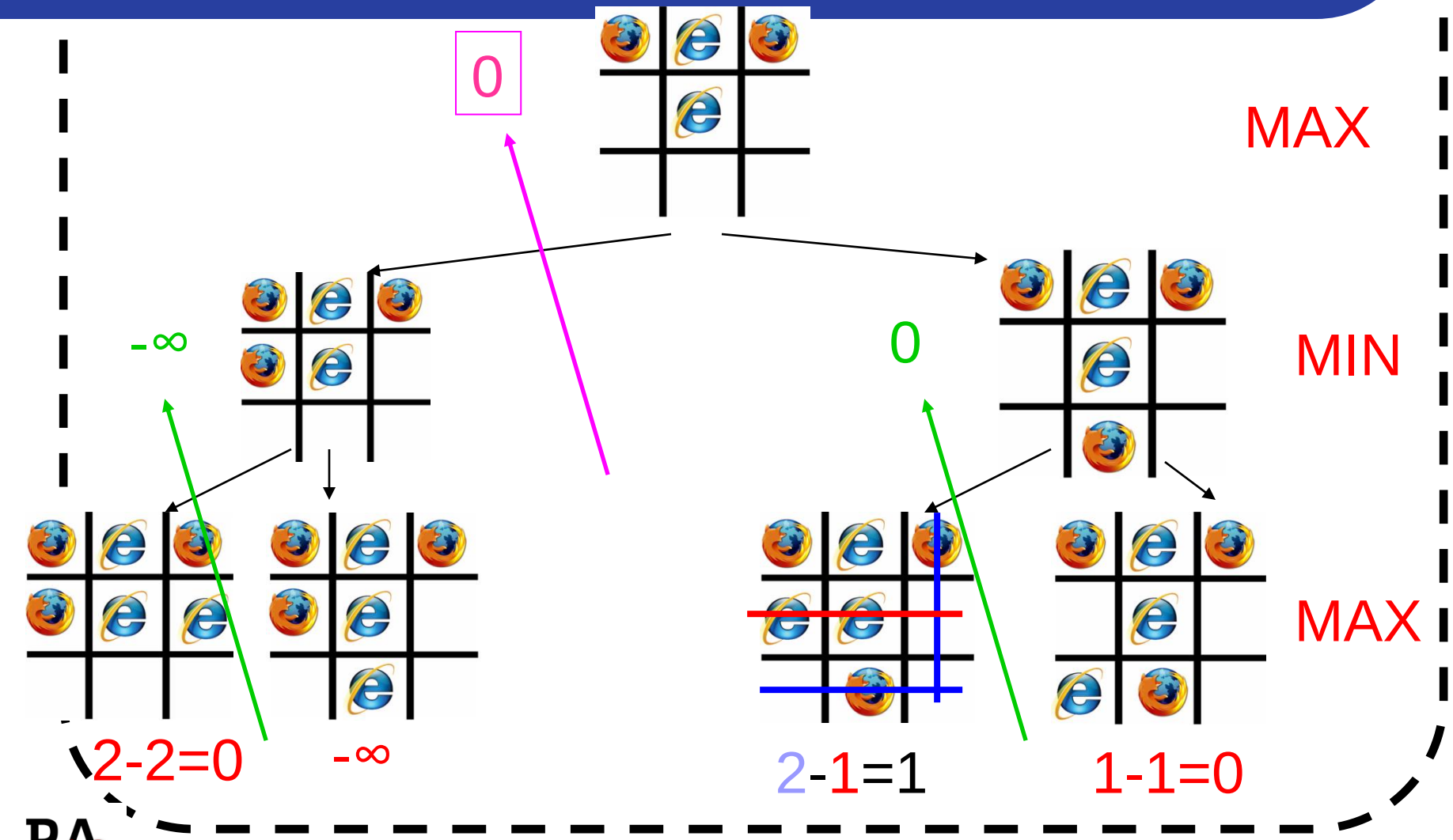
Exemplu funcție euristică X și 0

- F = numărul de linii/coloane/diagonale posibil câștigătoare pentru MAX – numărul de linii/coloane/diagonale posibil câștigătoare pentru MIN.

- Dacă MAX a câștigat, atunci F = +∞; dacă MIN a câștigat, atunci F = -∞.



Exemplu funcție euristică X și 0



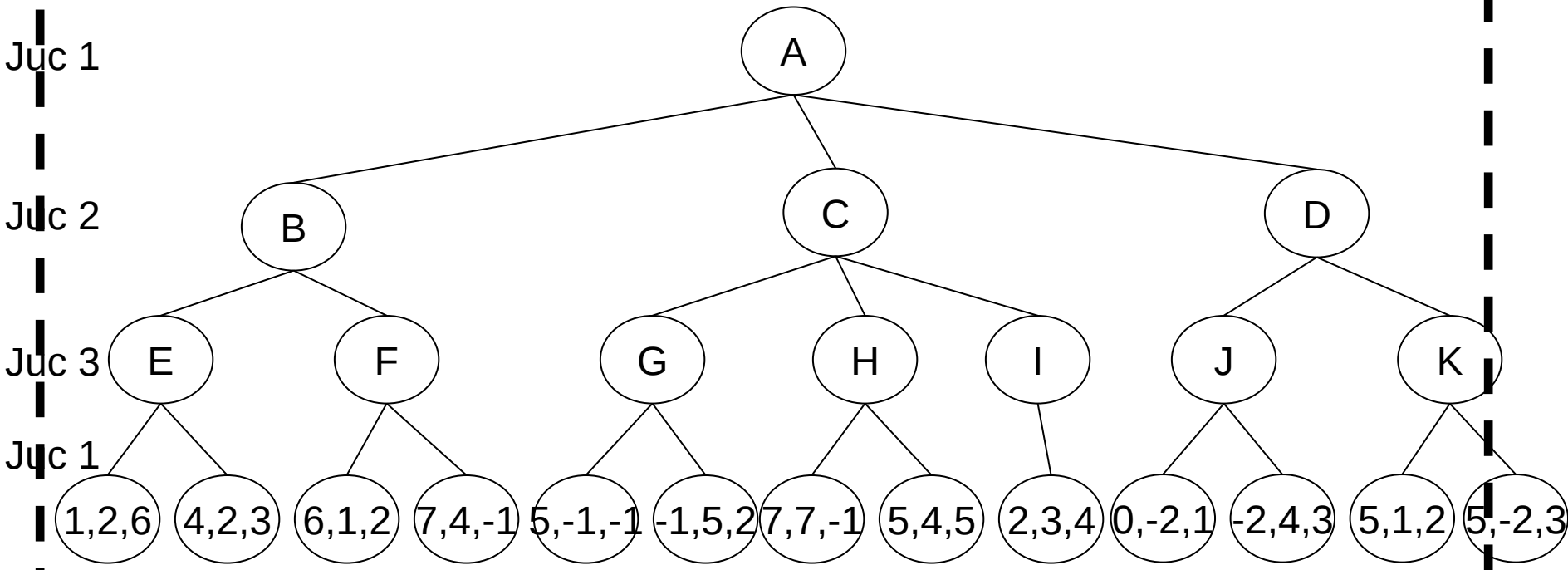
Algoritm MINIMAX

- MINIMAX_limitat (n, nivel_limită)
  - **Pentru fiecare** n' ∈ succs(n) // pentru toate mutările
    - Fie m = mutarea corespunzătoare arcului (n, n')
    - VAL(m) = w(n', nivel_limită, 1) // determin valoarea mutării
  - **Întoarce** m a.î. VAL(m) = max {VAL(x)| x ∈ mutări(n)}

- W(n, limită, nivel)
  - **Dacă** n este frunză **Întoarce** cost(n)
  - **Dacă** nivel ≥ limită **Întoarce** euristică(n)
  - **Dacă** jucătorul MAX este la mutare **Întoarce**
    - max {w(n', limită, nivel + 1) | n' ∈ succs(n)}
  - **Dacă** jucătorul MIN este la mutare **Întoarce**
    - min {w(n', limită, nivel + 1) | n' ∈ succs(n)}

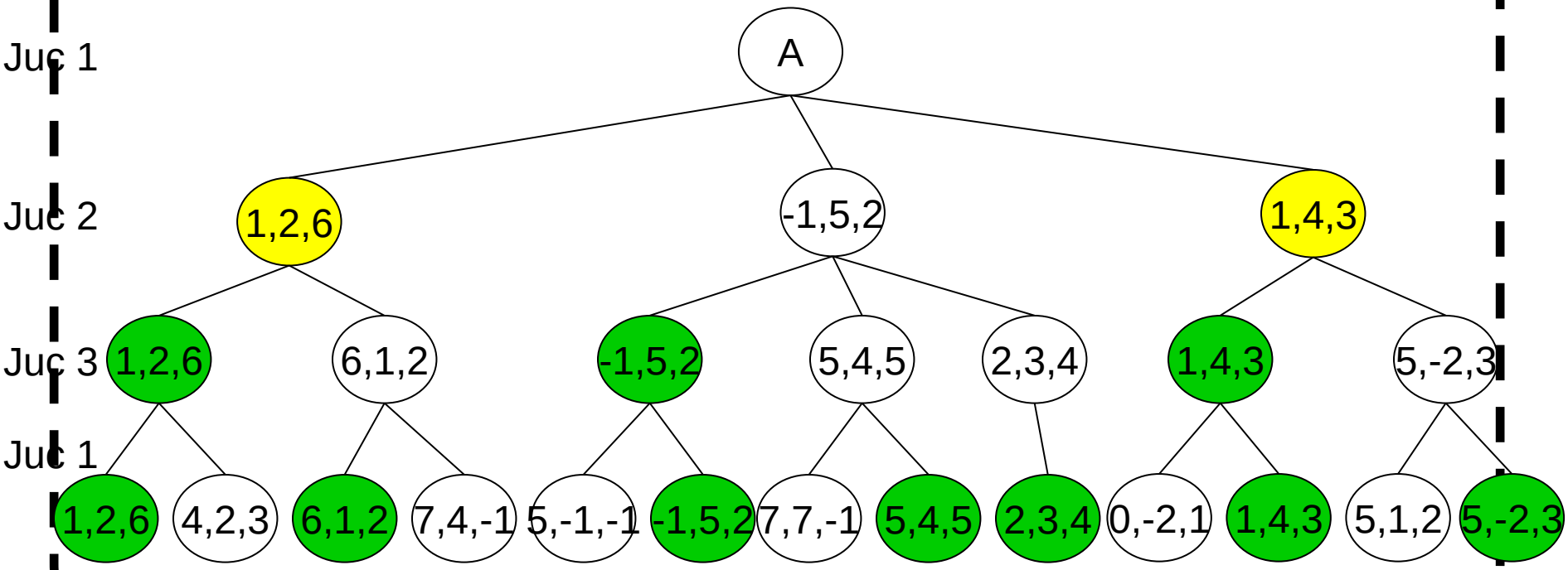
Caz special - Minimax 3 jucatori (1)

Jucătorii vor alege pe rând valoarea care le maximizează câștigul propriu



Caz special - Minimax 3 jucatori (2)

Jucătorii vor alege pe rând valoarea care le maximizează câștigul propriu



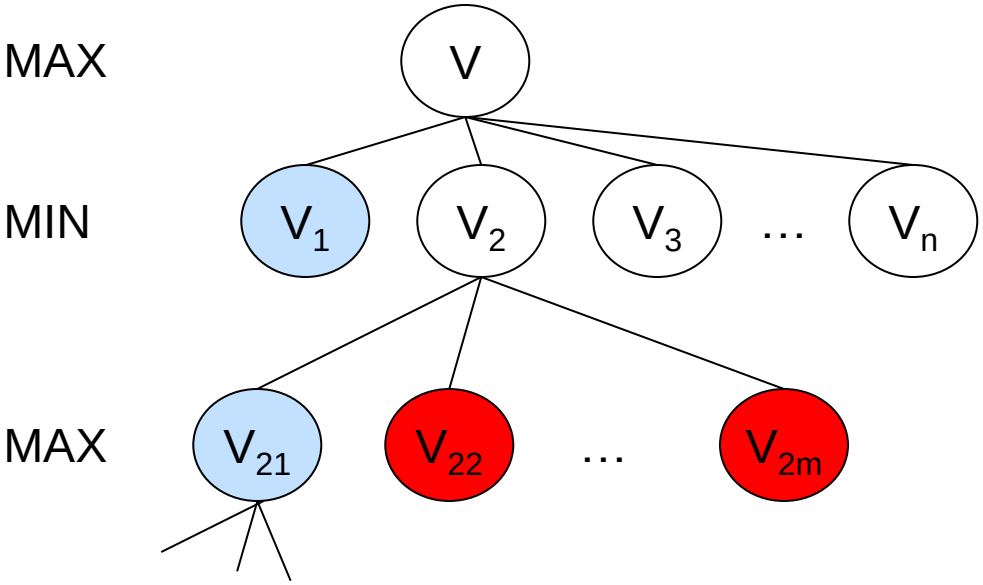
Caz special (2) – Minimax Probabilistic

- La unele jocuri, mutările sunt guvernate de șansă.
- Ex: Jocul de Table – mulțimea mutărilor este limitată de:
  - starea curentă a jocului;
  - combinația zarurilor în starea curentă.
- Arborele MINIMAX este completat cu noduri suplimentare (noduri șansă) plasate între nodurile MIN/MAX (MIN – șansă – MAX și MAX – șansă – MIN).
- Valorile se calculează ca sumă ponderată între probabilitatea nodului și evaluarea acestuia (prin cost sau euristică).

Tăiere α-β

- Încercăm să limităm spațiul de căutare prin eliminarea variantelor ce nu au cum să fie alese.

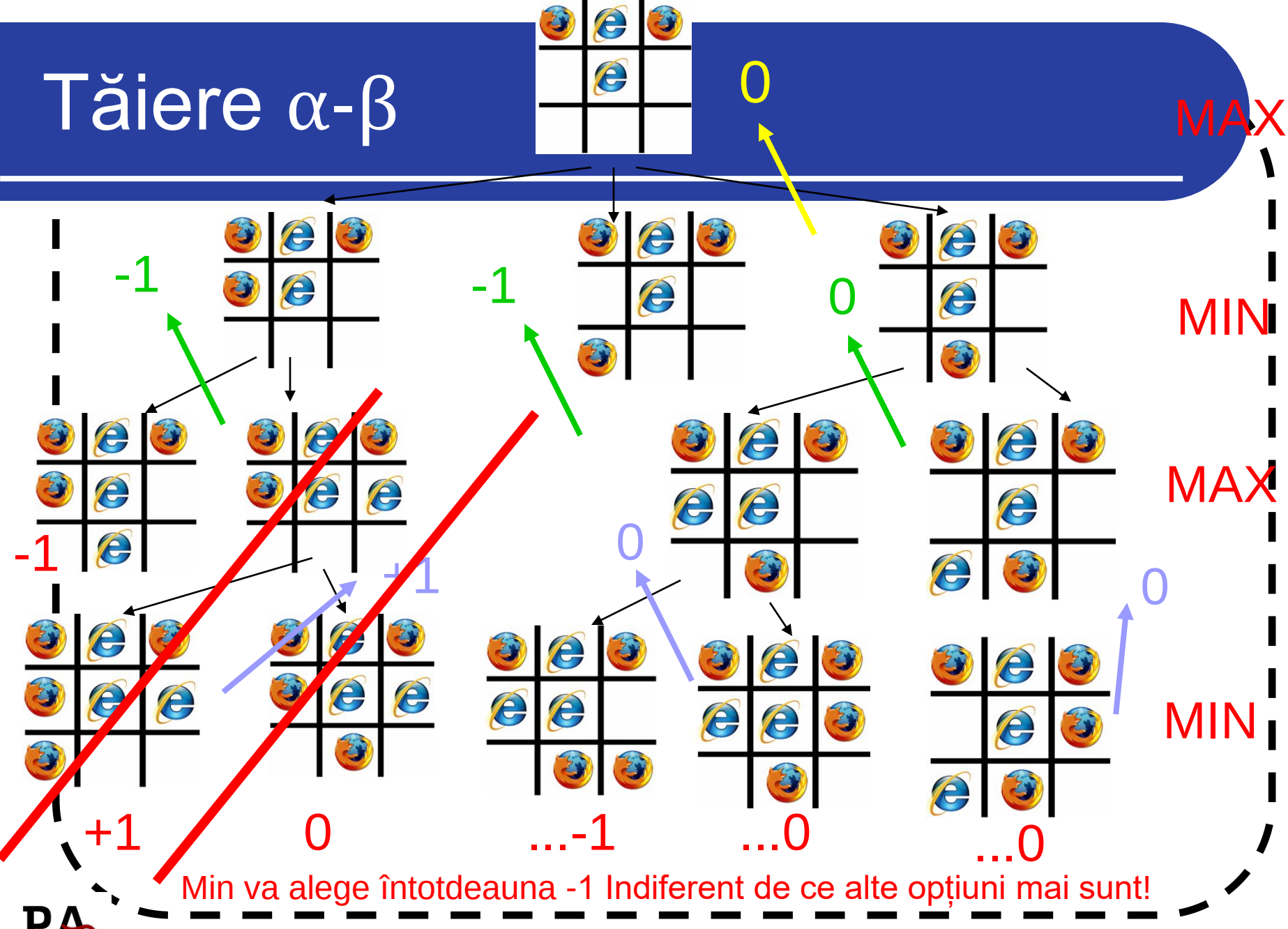
- Idee:
  - Dacă $V_{21}<V_1$ toată ramura V_2 poate fi Ignorată.



Tăiere α-β

- α = max dintre valorile găsite pentru un nod MAX.
- β = min dintre valorile găsite pentru un nod MIN.
- Tăiem o ramură dacă:
  - am găsit un nod pe nivelul MAX cu valoare β <= oricare din valorile α calculate anterior;
  - am găsit un nod pe nivelul MIN cu valoare α >= oricare din valorile β calculate anterior.
- Teorema α-β. Fie J un nod din arborele MINIMAX explorat. Daca $\alpha(J) \geq \beta(J)$, atunci explorarea nodului J nu este necesară.

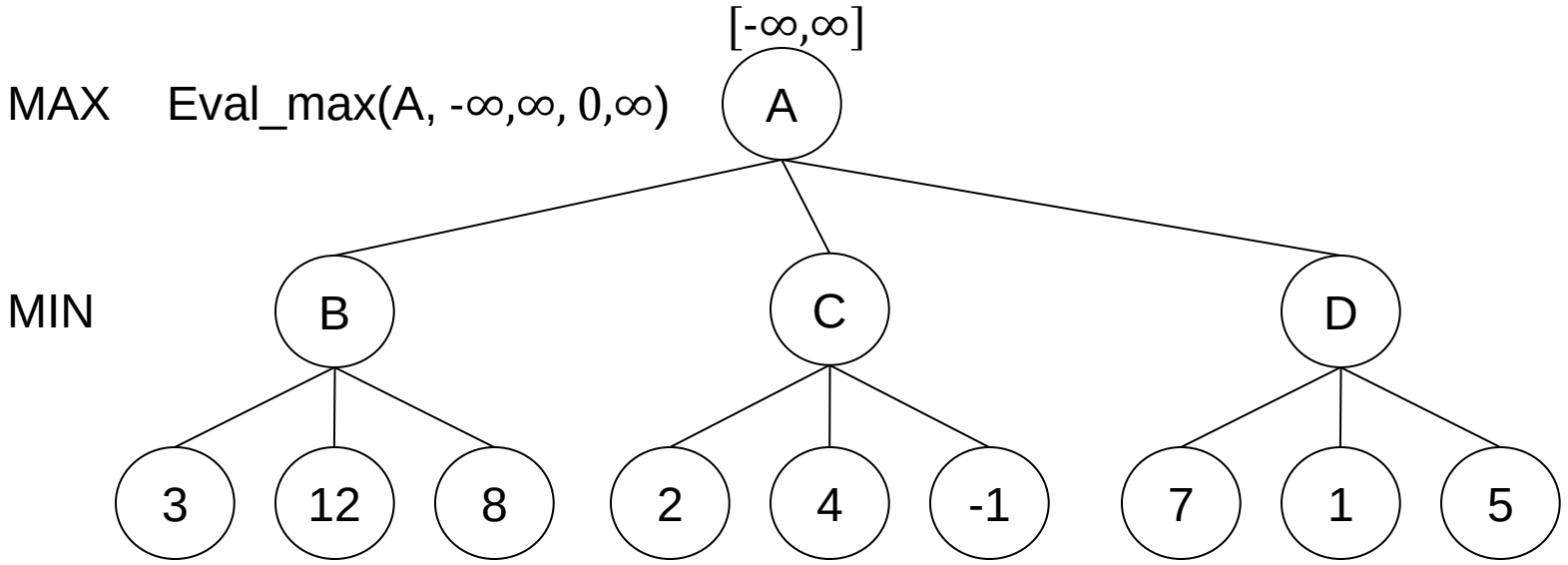
Tăiere α-β


0
MAX
-1
-1
0
MIN
-1
MAX
0
+1
0
0
MIN
+1
0
...-1
...0
...0
Min va alege întotdeauna -1 Indiferent de ce alte opțiuni mai sunt!

Algoritm α-β

- α-β(n, limită)
  - w = eval_max(n, -∞, ∞, 0, limită)
  - **Întoarce** m ∈ mutări(n) a.i. VAL(m) = w
- eval_max(n, α, β, nivel, limită)
  - **Dacă** n este frunză **Întoarce** cost(n)
  - **Dacă** (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat
  - a = -∞ // valoarea curenta a nodului de tip Max
  - **Pentru fiecare** (n' ∈ succs(n))
    - a = max(a, eval_min(n', max(α,a), β, nivel+1, limită)); // propag
    - **Dacă** (a ≥ β) **oprire**;
  - **Întoarce** a
- similar eval_min

Alt exemplu (II)



eval_max(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)

Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat

a = -∞ // valoarea curenta a nodului de tip max

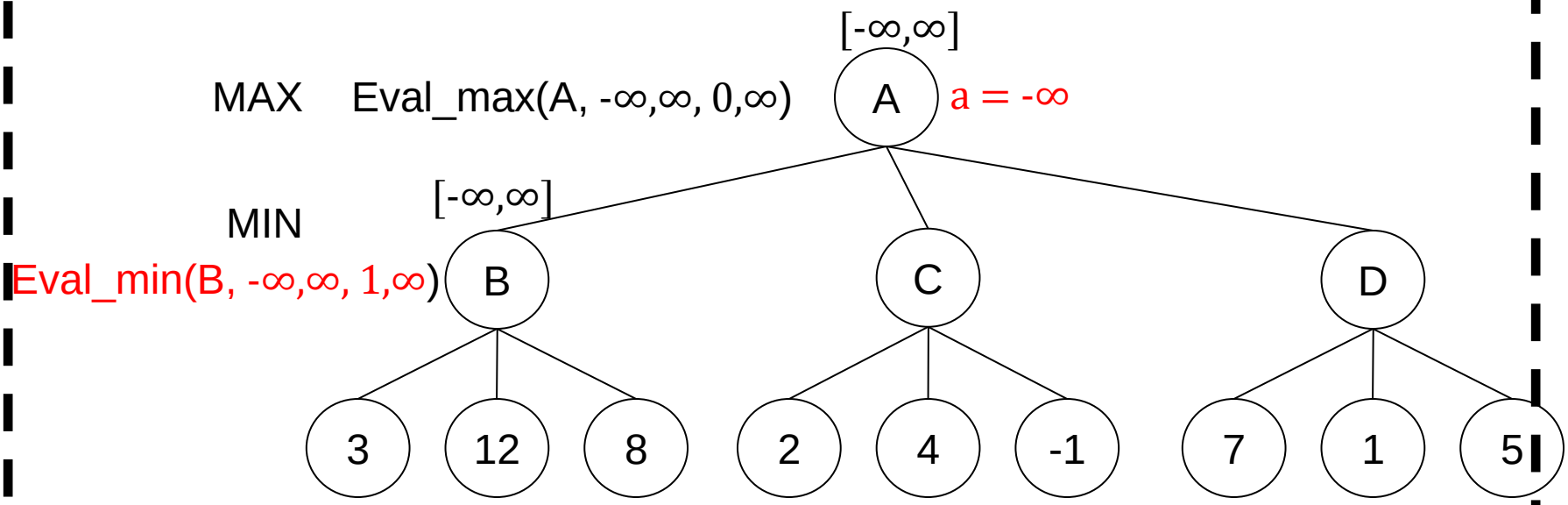
Pentru fiecare (n' ∈ succs(n)) {

a = max(a, eval_min(n', max(α,a), β, nivel+1, limită)); // propag

Dacă (a ≥ β) **oprire**; }

Întoarce a

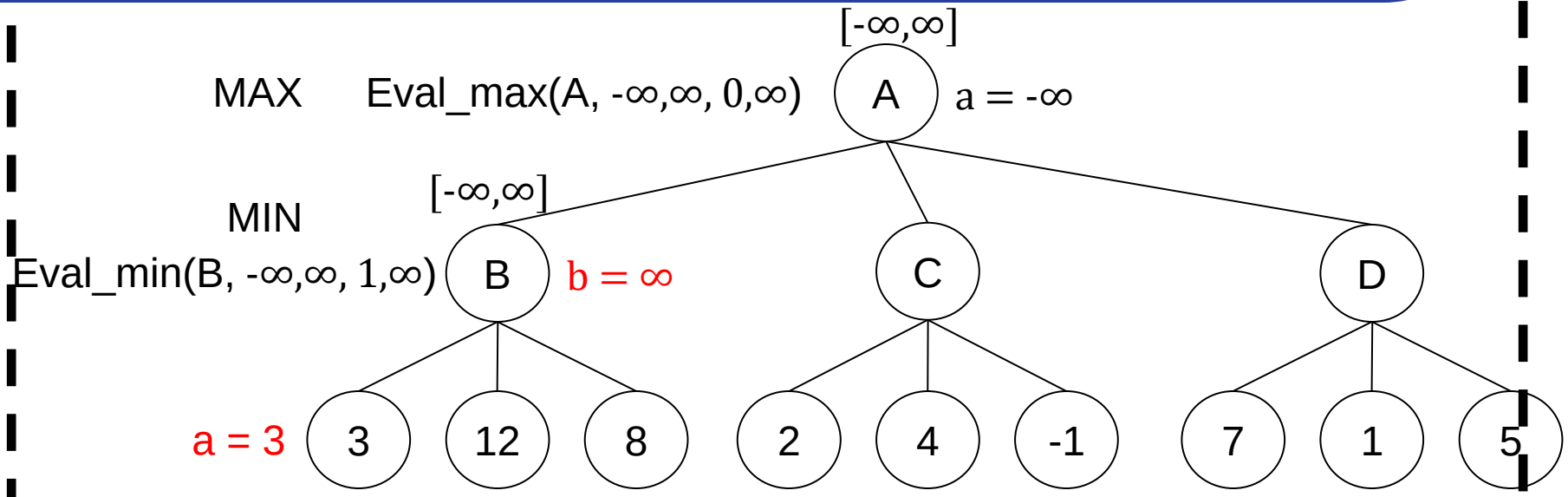
Alt exemplu (III)



eval_max(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)
Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat
a = -∞ // valoarea curenta a nodului de tip max
Pentru fiecare (n' ∈ succs(n)) {
    a = max(a, eval_min(n', max(α,a), β, nivel+1, limită)); // propag
    Dacă (a ≥ β) **oprire**; }
Întoarce a

Alt exemplu (IV)



eval_min(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)

Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat

b = ∞ // valoarea curenta a nodului de tip min

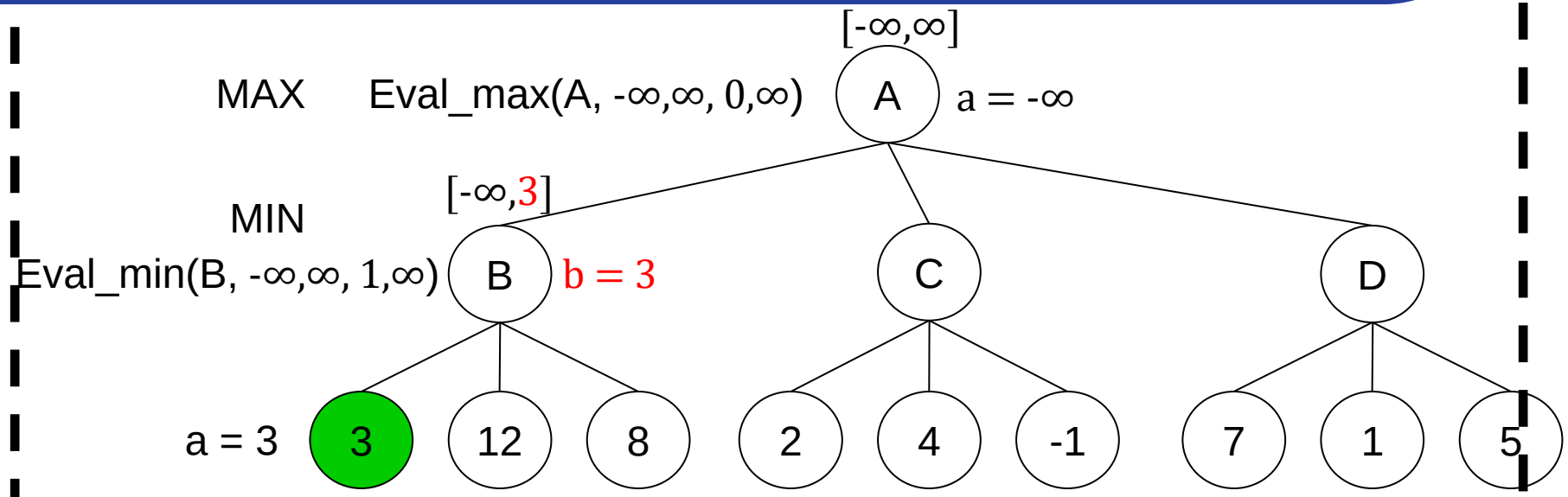
Pentru fiecare (n' ∈ succs(n)) {

b = min(b, eval_max(n', α, min(β,b), nivel+1, limită)); // propag

Dacă (b ≤ α) **oprire**; }

Întoarce b

Alt exemplu (V)



eval_min(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)

Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat

b = ∞ // valoarea curenta a nodului de tip min

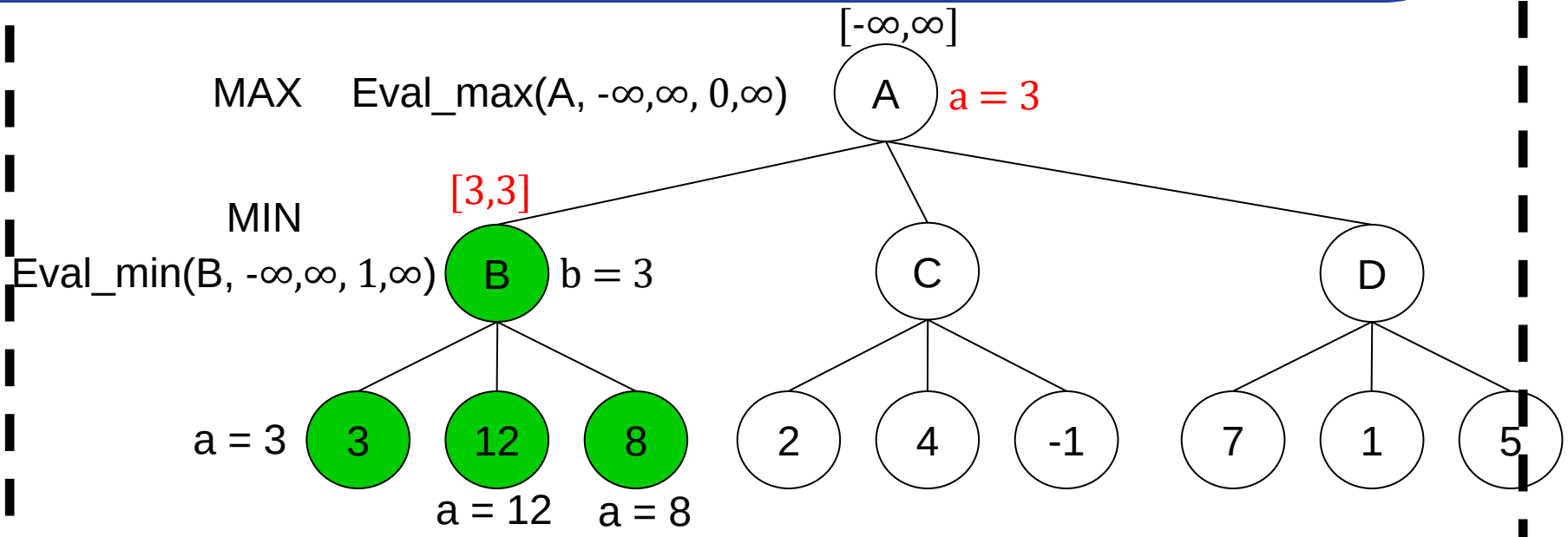
Pentru fiecare (n' ∈ succs(n)) {

b = min(b, eval_max(n', α, min(β,b), nivel+1, limită)); // propag

Dacă (b ≤ α) **oprire**; }

Întoarce b

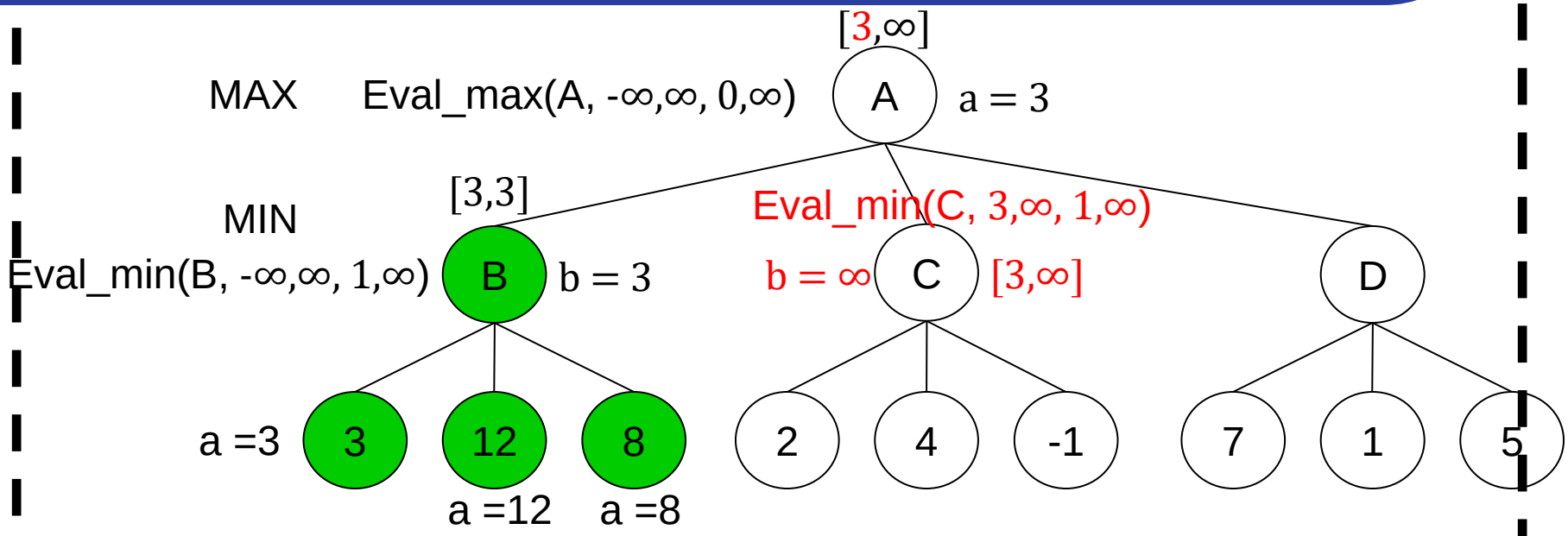
Alt exemplu (VI)



eval_max(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)
Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat
a = -∞ // valoarea curenta a nodului de tip max
Pentru fiecare (n' ∈ succs(n)) {
    a = max(a, eval_min(n', max(α,a), β, nivel+1, limită)); // propag
    Dacă (a ≥ β) **oprire**; }
Întoarce a

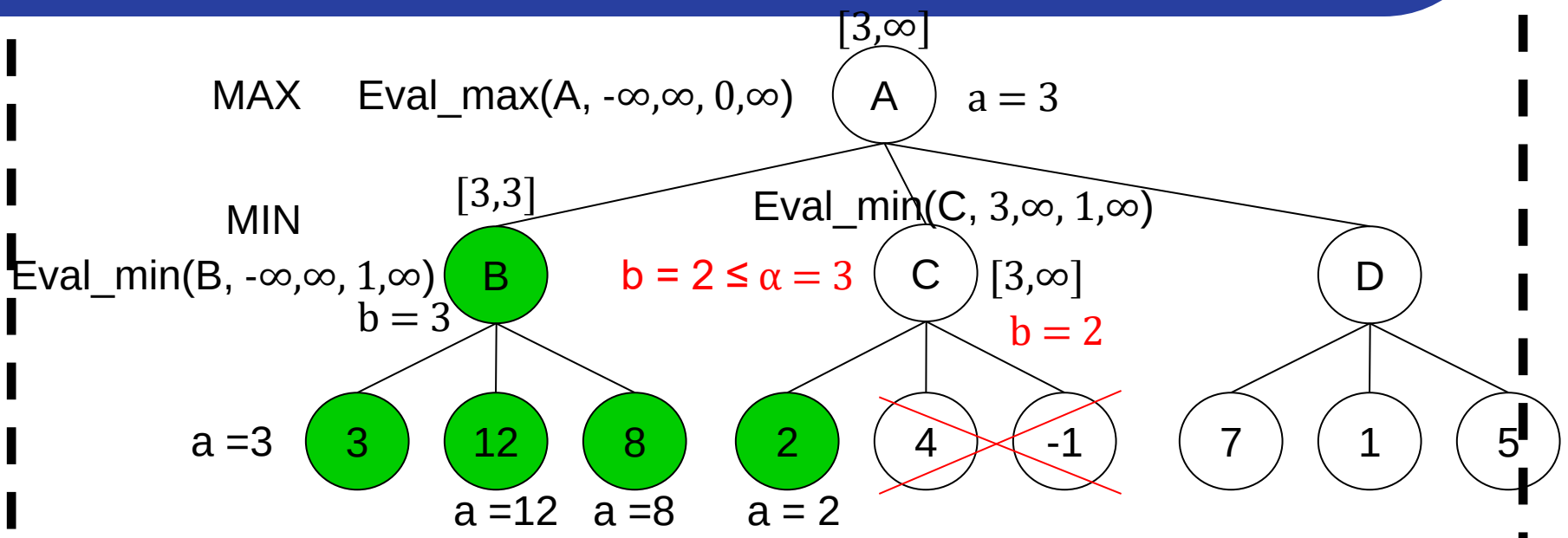
Alt exemplu (VII)



eval_min(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)
Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat
b = ∞ // valoarea curenta a nodului de tip min
Pentru fiecare (n' ∈ succs(n)) {
    b = min(b, eval_max(n', α, min(β,b), nivel+1, limită)); // propag
    Dacă (b ≤ α) **oprire**; }
Întoarce b

Alt exemplu (VIII)



eval_min(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)

Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat

b = ∞ // valoarea curenta a nodului de tip min

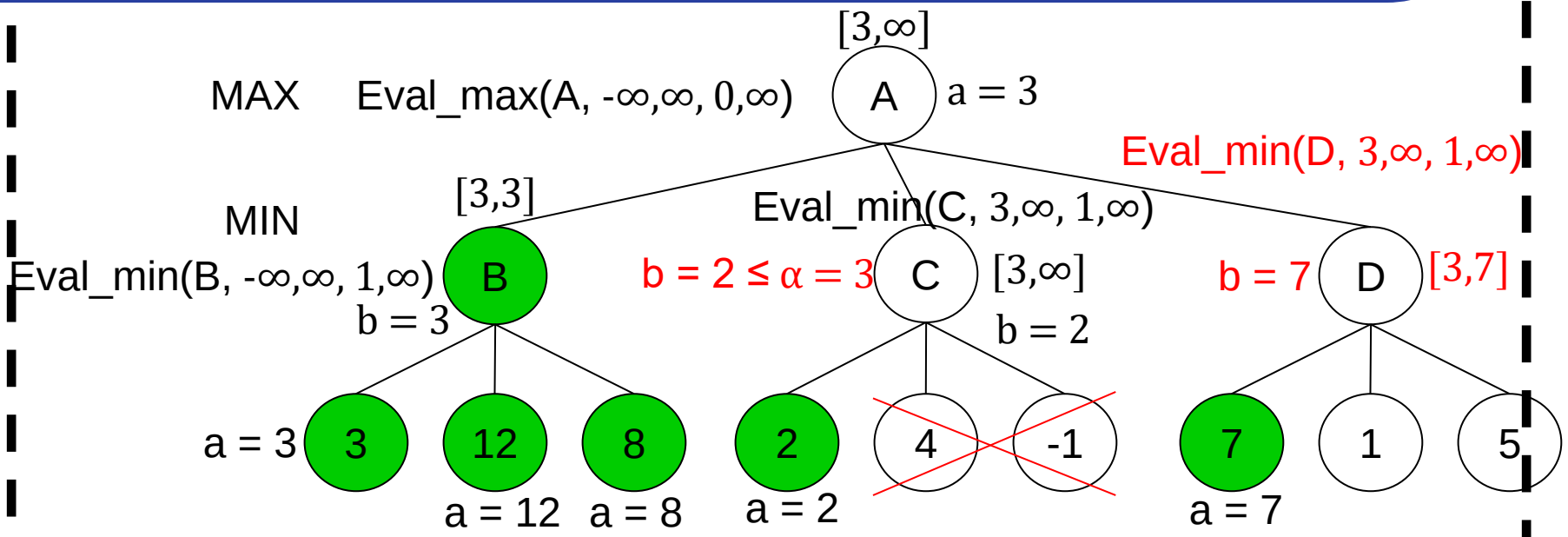
Pentru fiecare (n' ∈ succs(n)) {

b = min(b, eval_max(n', α, min(β,b), nivel+1, limită)); // propag

Dacă (b ≤ α) **oprire**; }

Întoarce b

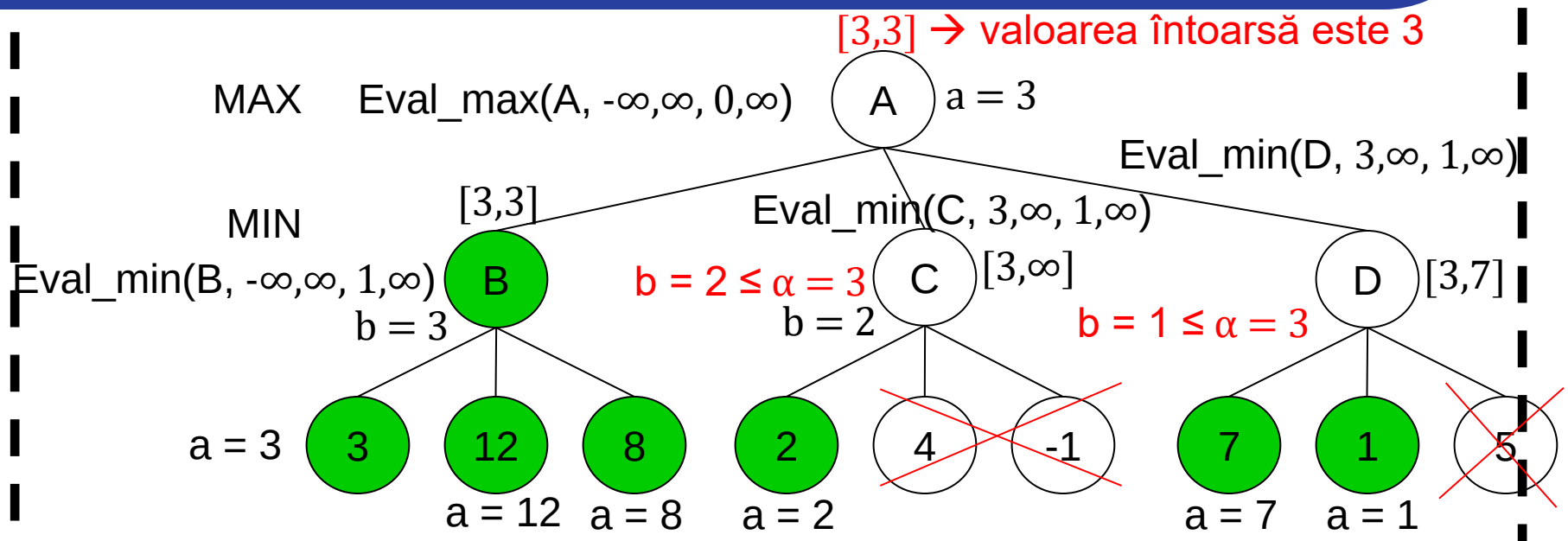
Alt exemplu (IX)



eval_min(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)
Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat
b = ∞ // valoarea curenta a nodului de tip min
Pentru fiecare (n' ∈ succs(n)) {
    b = min(b, eval_max(n', α, min(β,b), nivel+1, limită)); // propag
    Dacă (b ≤ α) **oprire**; }
Întoarce b

Alt exemplu (X)



eval_max(n, α, β, nivel, limită)

Dacă n este frunză **Întoarce** cost(n)
Dacă (nivel ≥ limita) **Întoarce** euristică(n) // sunt limitat
a = -∞ // valoarea curenta a nodului de tip max
Pentru fiecare (n' ∈ succs(n)) {
    a = max(a, eval_min(n', max(α,a), β, nivel+1, limită)); // propag
    Dacă (a ≥ β) **oprire**; }
Întoarce a

Observații α-β

- Reduce complexitatea minimax în cazul ideal de la
  - Număr_ramificări^{număr_nivele} la Număr_ramificări$^{număr_nivele/2}$
- Contează foarte mult ordinea în care analizăm mutările!
  - Sortarea mutărilor după un criteriu dat nu este costisitoare comparativ cu costul exponențial al algoritmului.
- Se folosesc euristici pentru a alege mutările examinate mai întâi:
  - ex: la șah se aleg întâi mutările în care se iau piese;
  - sau se aleg mai întâi mutările cu scor bun în parcurgeri precedente;
  - sau se aleg mutările care au mai generat tăieri.

Observații MINIMAX și α-β

- Algoritmi de căutare în adâncime.
- Pot cauza probleme când avem un timp limită.
- → Soluție posibilă IDDFS (căutare în adâncime mărind iterativ adâncimea maximă până la care căutăm).



Concluzii

- Algoritmi cu complexitate foarte mare.
- Soluții euristice pentru limitarea complexității.
- Recomandabil să se combine cu alte strategii – baze de date cu poziții, pattern-matching.



ÎNTREBĂRI?



Bibliografie Curs 6

- Giumale – Introducere în Analiza Algoritmilor cap 5 și 5.1
- Cormen – Introducere în Algoritmi cap Algoritmi elementari de grafuri (23) – Reprezentări + Căutări
- http://www.h3c.com/portal/res/200706/01/20070601_108959_image001_201240_57_0.gif
- http://ashitani.jp/gv/
- http://en.wikipedia.org/wiki/PageRank



Proiectarea Algoritmilor

Curs 6 – Introducere în grafuri,
Parcurgerea grafurilor



Bibliografie

- Giumale – Introducere în Analiza Algoritmilor cap 5 și 5.1
- Cormen – Introducere în Algoritmi cap Algoritmi elementari de grafuri (23) – Reprezentări + Căutări
- http://static5.depositphotos.com/1030015/514/i/950/depositphotos_5143007-stock-photo-global-computer-network.jpg
- http://ashitani.jp/gv/
- http://www.graphviz.org
- http://en.wikipedia.org/wiki/PageRank

Plan curs

- Introducere
- Modalități de reprezentare
- Exemple de probleme practice
- Algoritmi de parcurgere
  - BFS
  - DFS

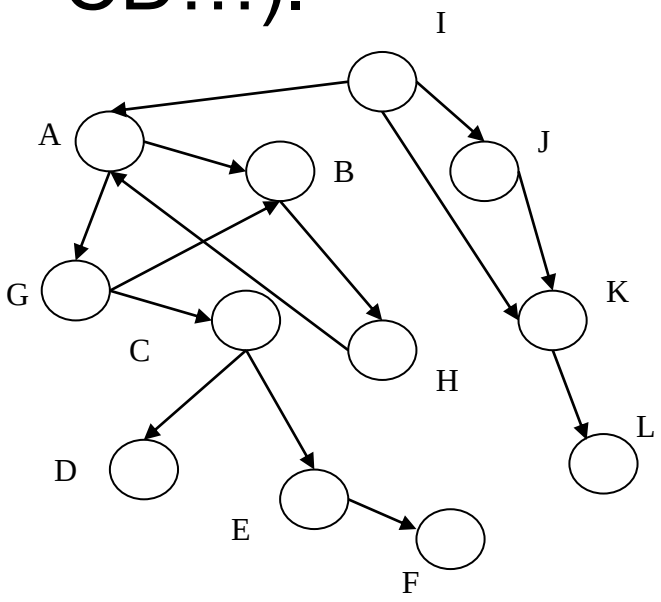


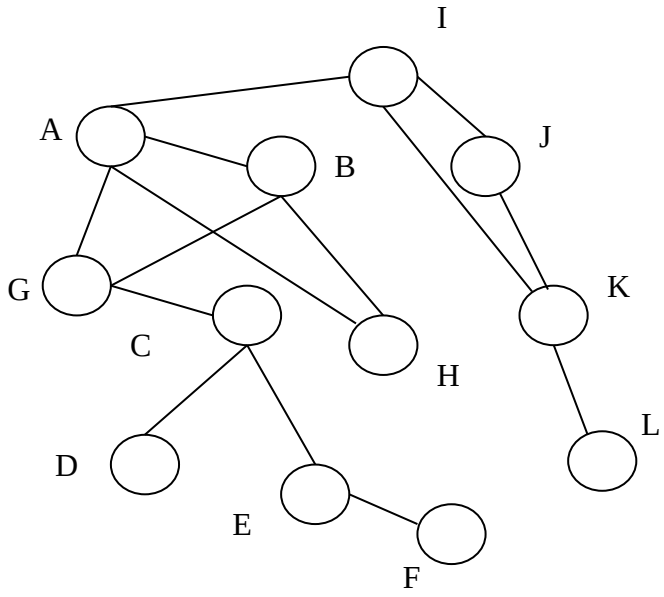
Introducere

- Circa 6 cursuri în care sunt prezentați algoritmii cei mai importanți pentru prelucrarea grafurilor:
  - Parcurgere
  - Sortare topologică
  - Componente tare conexe
  - Puncte de articulație
  - Punți
  - Arbori minimi de acoperire
  - Drumuri de cost minim
  - Fluxuri maxime
- Încercăm să legăm algoritmii de aplicații cât mai practice.

Tipuri de grafuri

- Orientate: noduri (A-L) + arce (AB, BC, CD…).

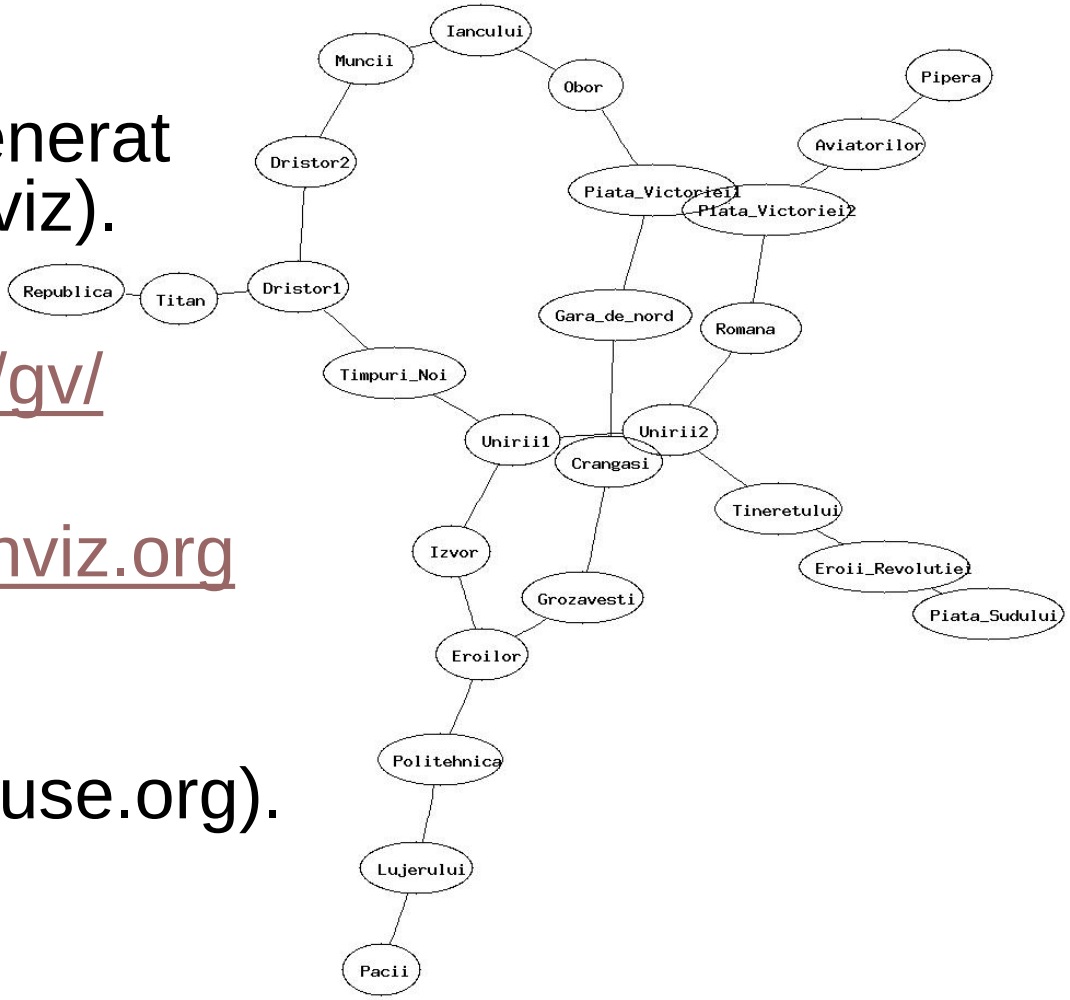




- Neorientate: noduri (A-L) + muchii (AB, BC, CD…).

Exemplu graf neorientat

- Exemplu graf generat cu neato (graphviz).
- http://ashitani.jp/gv/
- http://www.graphviz.org
- Biblioteci pentru vizualizare (Prefuse.org).



Modalități de descriere ale grafurilor

- Reprezentare în memorie:
  - Liste de adiacență;
  - Matrice de adiacență.

- Reprezentarea datelor de intrare:
  - Tupluri (sursă, destinație);
    - Întâlnite mai ales în descrierile folosind baze de date.
  - Limbaje specializate (ex: dot, GraphML, rdf).



Formate de reprezentare

- Listă adiacență :
  - Eroilor: Politehnica, Grozăvești, Izvor
  - Muncii: Dristor2, Ianculului…
- Matrice adiacență:

	Unirii2	Tineretului	Romană	…
Unirii2	-	1	1	
Tineretului	1	-		
Romană	1		-	
…				

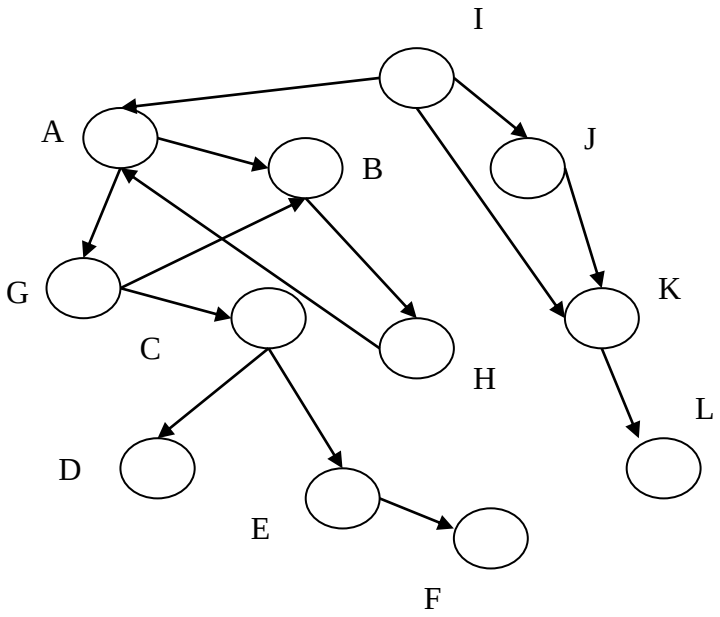
- Tupluri:
  - (Dristor1; Dristor2)
  - (Eroilor; Grozăvești)
  - …
- Dot:
  - graph G {node;
  - Dristor2--Muncii--Ianculului—Obor;
  - Piata_Victoriei1--Gara_de_nord--Crangasi--Grozavesti--Eroilor;
  - Pacii--Lujerului--Politehnica--Eroilor;
  - Republica--Titan--Dristor1--Timpuri_Noi--Unirii1--Izvor--Eroilor;
  - Dristor1--Dristor2;
  - Unirii1--Unirii2;
  - Piata_Victoriei1--Piata_Victoriei2;
  - Piata_Sudului--Eroii_Revolutiei--Tineretului--Unirii2--Romana--Piata_Victoriei2--Aviatorilor--Pipera;
  - }



Formate de reprezentare - GraphML

- GraphML
  - <graphml xmlns="http://graphml.graphdrawing.org/xmlns">
  - <graph edgedefault="undirected">
  - <!-- data schema -->
  - <key id="name" for="node" attr.name="name" attr.type="string"/>
  - <key id="gender" for="node" attr.name="gender" attr.type="string"/>
  - <!-- nodes -->
  - <node id="1">
  - <data key="name">Jeff</data>
  - <data key="gender">M</data>
  - </node>
  - <node id="2">
  - <data key="name">Ed</data>
  - <data key="gender">M</data>
  - </node>
  - <edge source="1" target="2"></edge>
  - </graph>
  - </graphml>

Matrice de adiacență

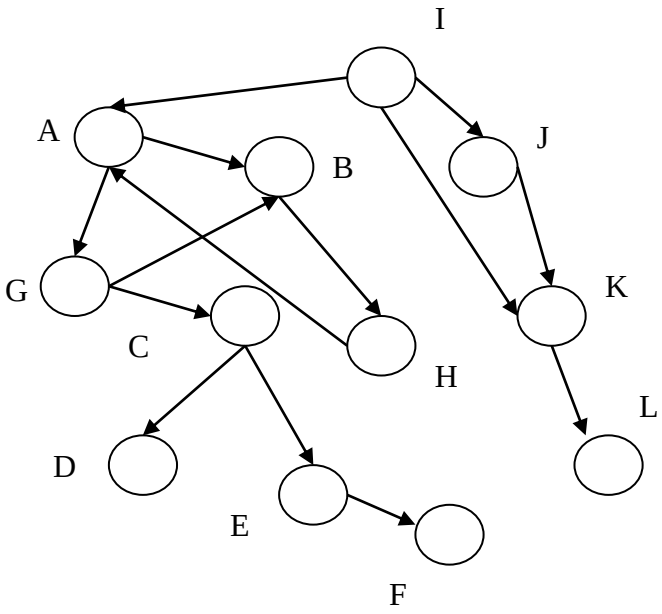


Cum se determină matricea de adiacență?

Matrice de adiacență

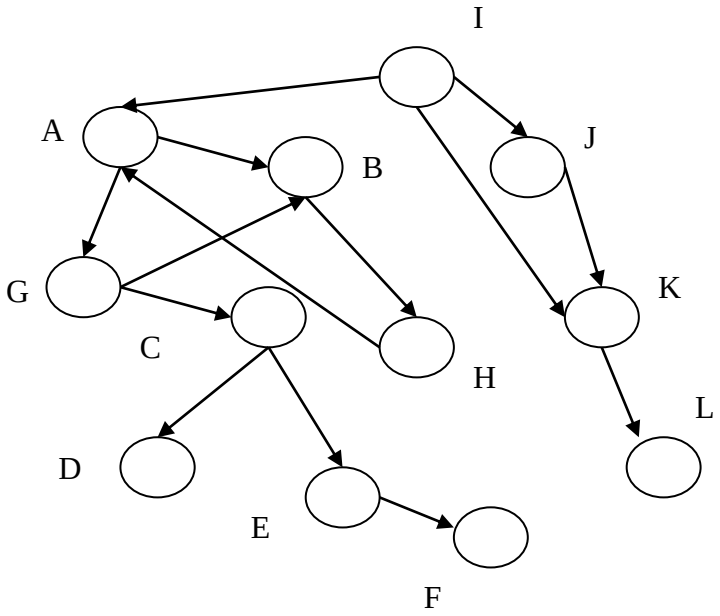
Matricea este rară?

- G – rar
- G – dens



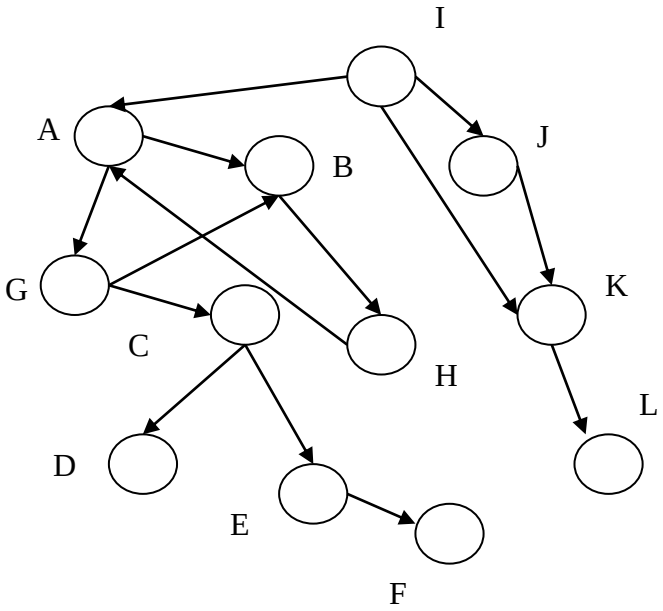
	A	B	C	D	E	F	G	H	I	J	K	L
A		1					1					
B								1				
C				1	1							
D												
E						1						
F												
G		1	1									
H	1											
I	1									1	1	
J											1	
K												1
L												

Vector de adiacență



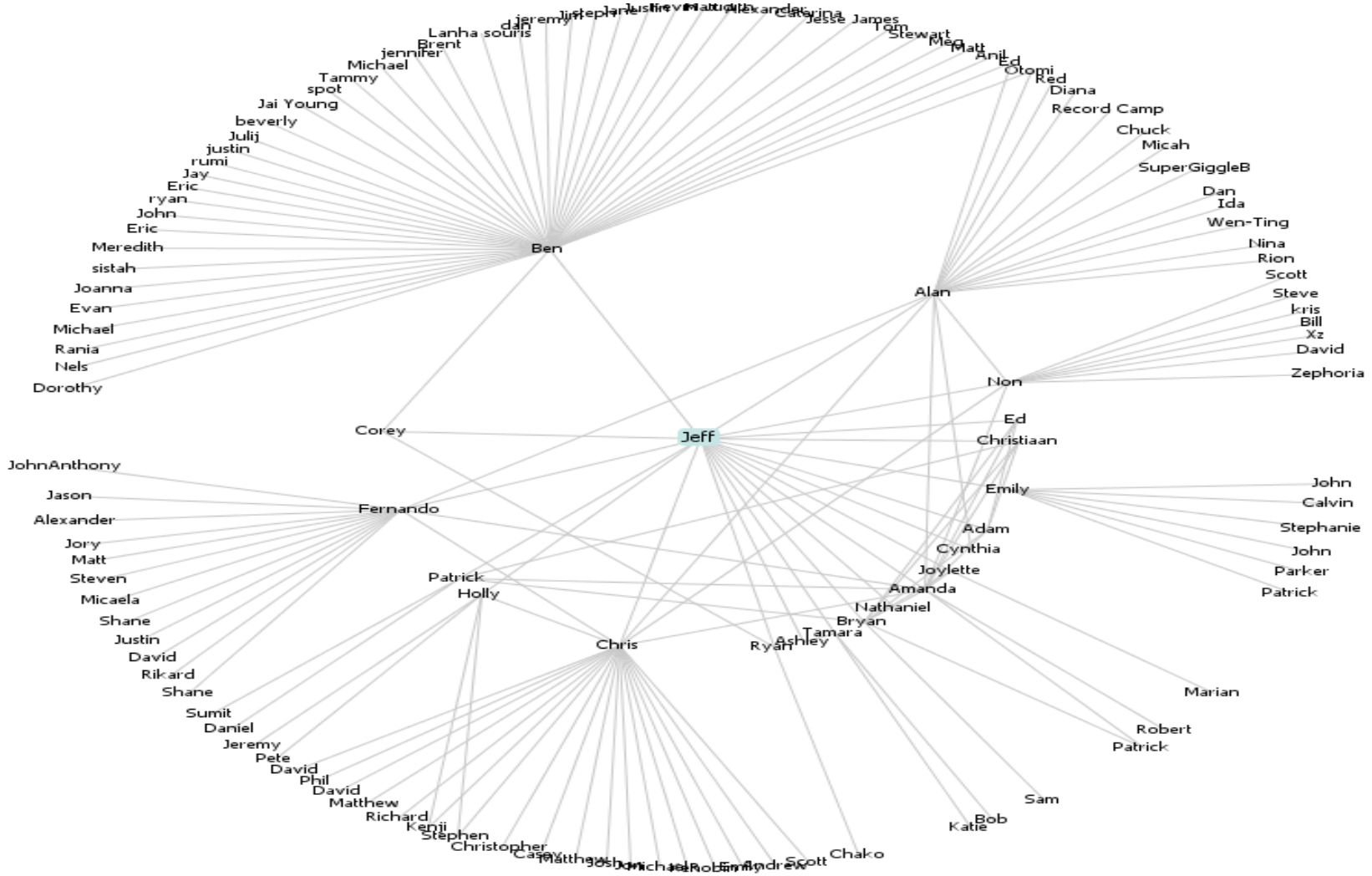
Cum se determină vectorul de adiacență?

Vector de adiacență



A	**B**	**G**	
B	**H**		
C	**D**	**E**	
D			
E	**F**		
F			
G	**B**	**C**	
H	**A**		
I	**A**	**J**	**K**
J	**K**		
K	**L**		
L			

Utilizări practice - Rețele sociale

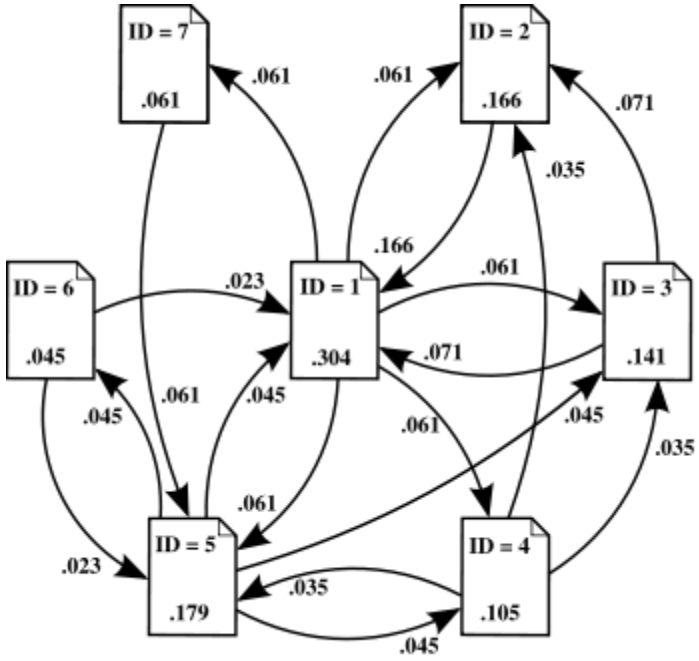


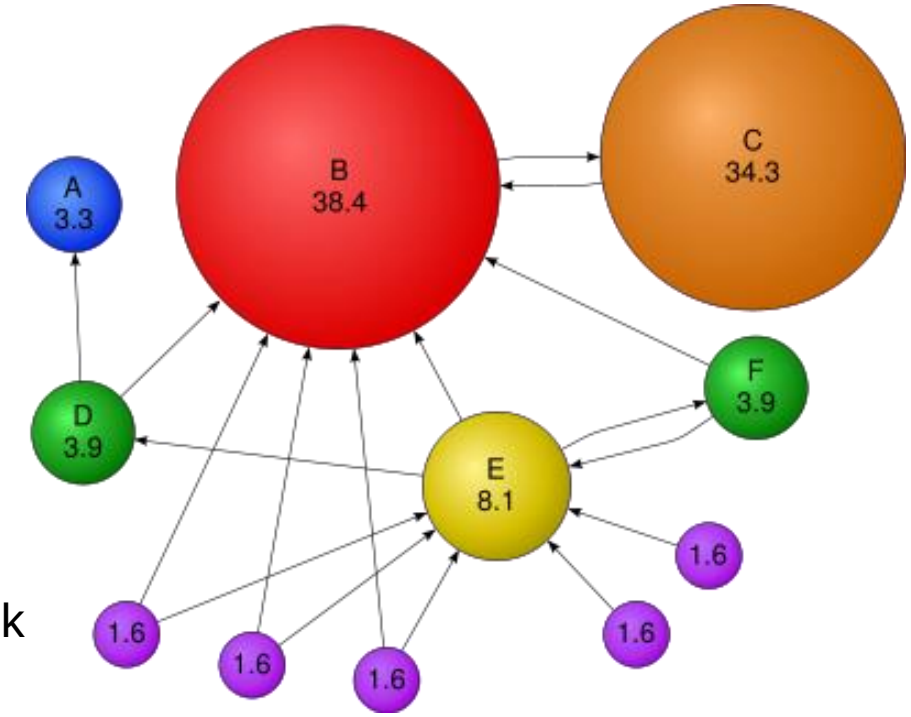
Utilizări practice – Rețele de calculatoare



http://static5.depositphotos.com/1030015/514/i/950/depositphotos_5143007-stock-photo-global-computer-network.jpg

Utilizări practice - Web





http://en.wikipedia.org/wiki/PageRank

Utilizări practice

- Hărți, rețele (calculatoare, instalații, etc.), rețele sociale, analiza fluxurilor (semaforizare, proiectarea dimensiunii țevilor de apă).

- Exemple simple:
  - Cel mai scurt drum între punctele A și B pe o hartă.
  - Radialitate – în rețele sociale: gradul în care rețeaua socială a unui individ se întinde în rețeaua globală pentru a schimba date și influență.
  - Page Rank (Google).



Algoritmi de parcurgere – Notații (1)

- G = (V,E);
- V – mulțimea de noduri;
- E – mulțimea de muchii / arce;
- (u,v) – arcul / muchia u,v;
- u..v – drum de la u la v; dacă există mai multe variante notăm u..x..v, u..y..v;
- R(u) - reachable(u) = mulțimea nodurilor ce pot fi atinse pe căi ce pleacă din u;



Algoritmi de parcurgere – Notații (2)

- succs(u) – mulțimea succesorilor lui u (graf orientat) sau mulțimea nodurilor adiacente lui u (graf neorientat);
- c(u) – culoarea nodului – specifică starea nodului la un anumit moment al parcurgerii:
  - Alb – nedescoperit;
  - Gri – descoperit, în curs de prelucrare;
  - Negru – descoperit și terminat (cu semnificații diferite pentru BFS si DFS).
- p(u) (π(u)) – "părintele lui u" – identificator al nodului din care s-a ajuns în nodul u prima oară.



Parcurgere în lățime (BFS)

- Nod de start (sursă): s.
- Determină numărul minim de arce / muchii între s și $\forall$ u $\in$ V = numărul de pași între sursă și orice alt nod din graf (acesta este cel mai scurt drum în condițiile în care nu există o funcție de cost asociată grafului).
- $\delta(s,u)$ – costul optim al s..u; $\delta(s,u) = \infty <=> u \notin R(s)$.
- Dist(s,u) – costul drumului descoperit s..u.
- Ex: Politehnica → restul stațiilor de autobuz
  - câte stații trebuie să merg?
  - de câte bilete am nevoie?

BFS – Structura de date

- Folosește o coadă (FIFO) pentru a reține nodurile ce trebuie prelucrate.

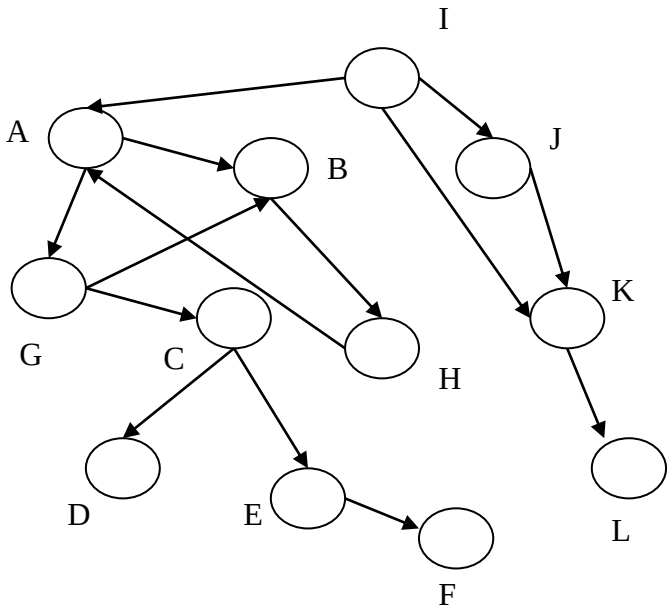
- Pentru fiecare nod se reține:
  - Părintele – $\pi(u)$ (p(u));
  - Dist(s,u) – distanța până la nodul sursă;
  - Culoarea nodului.

BFS – Algoritm

- BFS(s,G)
  - **Pentru fiecare** nod u (u ∈ V)
    - p(u) = null; dist(s,u) = inf; c(u) = alb; // inițializări
  - Q = (); // se folosește o coadă în care reținem nodurile de prelucrat
  - dist(s,s) = 0; // actualizări: distanța de la sursă până la sursă este 0
  - c(s) = gri; // și culoarea lui devine gri
  - Q ← Q + s; // adăugăm sursa în coadă → începem prelucrarea lui s
  - **Cât timp** (!empty(Q)) // cât timp mai am noduri de prelucrat
    - u = top(Q); // se determină nodul din vârful cozii (care va fi prelucrat)
    - **Pentru fiecare** nod v (v ∈ succs(u)) // pentru toți vecinii
      - **Dacă** c(v) este alb // vecinul nu a mai fost găsit până acum
        - **Atunci** { dist(s,v) = dist(s,u) + 1; p(v) = u; c(v) = gri; Q ← Q + v;}
          // actualizăm structura date
    - c(u) = negru; // am terminat de prelucrat nodul curent
    - Q ← Q - u; // nodul este eliminat din coadă

BFS – Exemplu

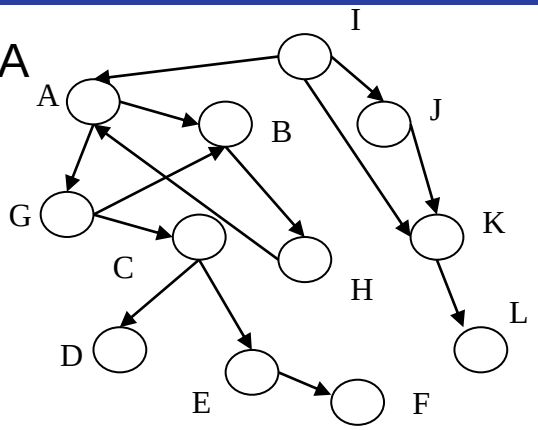
Sursa = A



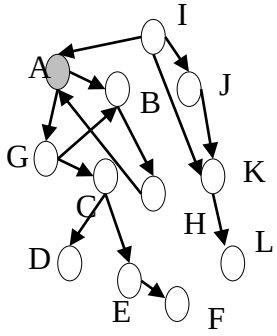
- BFS(s,G)
  - **Pentru fiecare** nod u (u ∈ V)
    - p(u) = null; dist(s,u) = inf; c(u) = alb; // inițializări
  - Q = (); // se folosește o coadă pentru nodurile de prelucrat
  - dist(s,s) = 0; // actualizări: distanța de la s la s e 0
  - c(s) = gri; // și atunci culoarea lui devine gri
  - Q ← Q + s; // adăugăm sursa în coadă → începem cu s
  - **Cât timp** (!empty(Q)) // cât timp am noduri de prelucrat
    - u = top(Q); // se determină nodul din vârful cozii
    - **Pentru fiecare** nod v (v ∈ succs(u)) // pentru toți vecinii
      - **Dacă** c(v) este alb // nodul nu a mai fost găsit, nu e în Q
        Atunci { dist(s,v) = dist(s,u) + 1; p(v) = u; c(v) = gri;
        Q ← Q + v;} // actualizăm structura date
    - c(u) = negru; // am terminat de prelucrat nodul curent
    - Q ← Q - u; // nodul este eliminat din coadă

BFS – Evoluția explorării

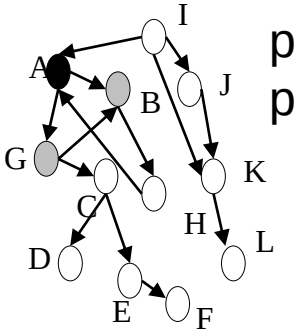
Sursa = A



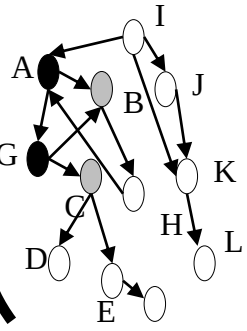
Q = A; d(A) = 0
p(A) = null



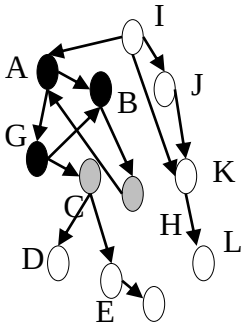
Q = G, B
d(B) = d(G) = 1
p(B) = A
p(G) = A



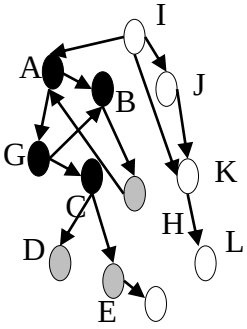
Q = B, C
d(C) = 2



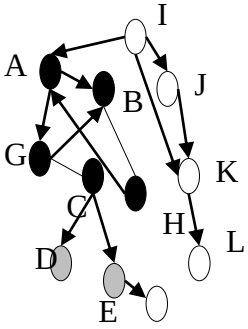
Q = C, H
d(H) = 2



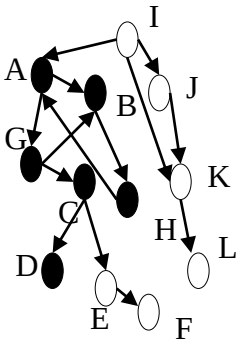
Q = H, D, E
d(D)=d(E)=3



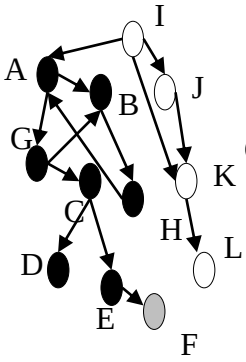
Q = D, E



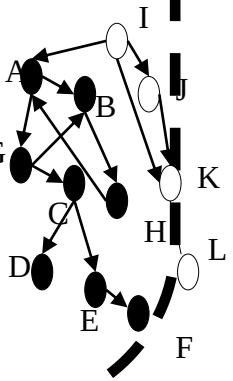
Q = E



Q = F
d(F)=4

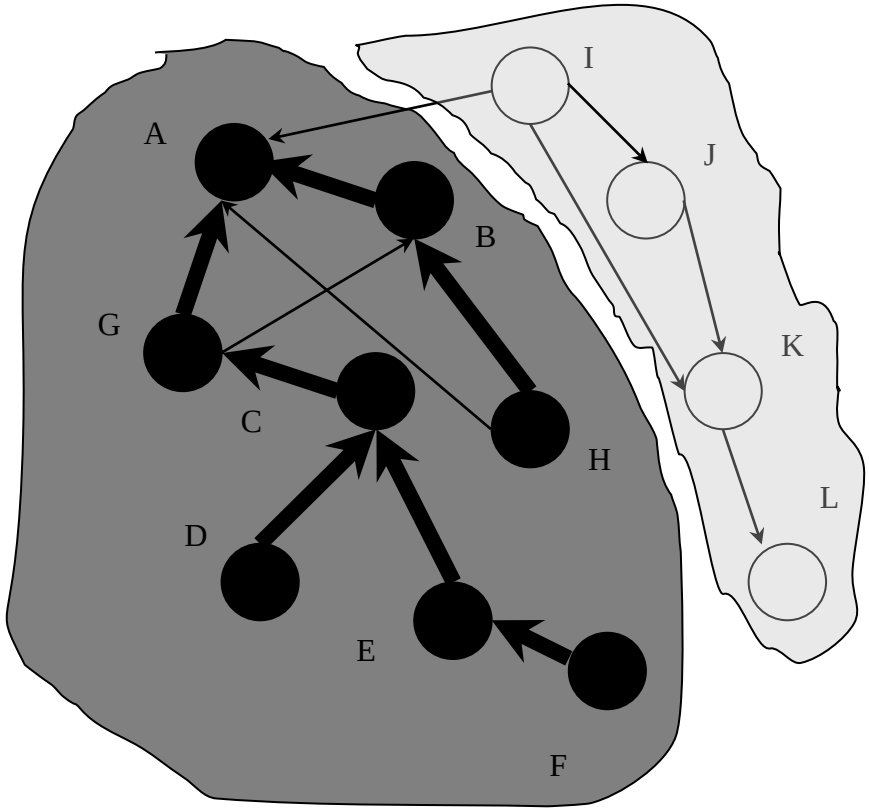


Q = Ø



p(C) = G p(H) = B p(D) = p(E) = C p(F) = E

BFS – Zona de explorare



BFS – Proprietăți (I)

- Lema 5.1. În cursul execuției BFS(s,G) $v \in Q \Leftrightarrow v \in R(s)$.
  - → $v \in Q$ → $v \in R(s)$: Q conține exclusiv noduri din R(s) (BFS parcurge toate nodurile ce pot fi atinse din s);
    - Caz de bază: $s \in R(s)$ Adevărat
    - Pas inducție: Q' conține doar noduri din R(s) → Q ← Q' + v conține doar noduri din R(s) ($v \in R(s)$)
      - Dacă u = top(Q') → $u \in R(s)$ → $v \in succs(u)$ → $(u,v) \in E$ → $v \in R(s)$
  - ← toate nodurile ce pot fi atinse din s vor fi introduse cândva în coadă. (drum $s = v_0 v_1 \ldots v_p = v$)
  - Dem prin inducție! P(i) = $\forall i \in 0..p$, se execută $Q \leftarrow Q + v_i$
    - Caz de bază: i = 0, se execută Q ← Q + s Adevărat
    - Pas inducție: P(i) Adevărat → P(i+1) Adevărat
      - Conform ipotezei inductive, la un moment dat avem $Q \leftarrow Q + v_i$. → cândva, v_i = top(Q) → dacă $c(v_{i+1})$ = alb, atunci este introdus in Q ($Q \leftarrow Q + v_{i+1}$). Altfel, $c(v_{i+1}) \neq$ alb → v_{i+1} este/a fost deja în coadă pt. că un nod își schimbă culoarea numai la inserție.

BFS – Proprietăți (II)

Lema 5.2. $\forall (u,v) \in E, \delta(s,v) \leq \delta(s,u) + 1$

- $\delta(s,v) \leq \delta(s,u) + 1$ în general este egalitate când v este descoperit din u; < când v deja a fost descoperit înainte să se ajungă în u.
- Dem prin reducere prin absurd! Pp. $\delta(s,v) > \delta(s,u) + 1$ → $\delta(s,u) + 1$ este costul unui drum optim $\perp$

- Lema 5.3. La terminarea BFS(s,G) există proprietatea $dist(s,u) \geq \delta(s,u)$.
  - Dem prin inducție folosind Lema 5.1 și Lema 5.2!
    - Caz de bază: $dist(s,s) = 0 = \delta(s,s)$ Adevărat
    - Pas inducție: pt. $\forall$ nod din Q': $dist(s,u) \geq \delta(s,u)$.
      - Cum $dist(s,v) = dist(s,u) + 1$ si $dist(s,u) \geq \delta(s,u)$ → $dist(s,v) \geq \delta(s,u) + 1 \geq \delta(s,v)$
    - Conf. Lema 5.1, $v \in Q \Leftrightarrow v \in R(s)$ și $dist(s,u) \geq \delta(s,u)$.
    - Dacă $v \notin R(s)$, $dist(s,v) = \delta(s,v) = \infty$

BFS – Proprietăți (III)

- Lema 5.4. După orice execuție a ciclului principal al BFS, Q conține $v_1, v_2, \ldots, v_p$ ai:
  - Prop(Q) = $dist(s,v_1) \leq dist(s,v_2) \leq \ldots \leq dist(s,v_p) \leq dist(s,v_1) + 1$
  - => la un moment dat in coadă sunt elemente de pe același nivel din arborele generat de BFS (sau maxim 1 nivel diferență).
  - Dem prin inducție după numărul de elemente din Q! (demonstrăm invariantul Prop(Q) la inserare și eliminare de elemente în/din Q.)
  - Fie $u = v_1 = top(Q)$
  - Inserție: fie v succ lui u și cul(v) = alb → coada devine: $v_1, v_2, \ldots, v_p$, $v_{p+1} = v$. $dist(s,v) = dist(s,v_1) + 1$ → $dist(s,v_1) \leq dist(s,v_2) \leq \ldots \leq dist(s,v_p) \leq dist(s,v_{p+1}) \leq dist(s,v_1) + 1$
  - Eliminare: $dist(s,v_1) \leq dist(s,v_2)$ → $dist(s,v_1) + 1 \leq dist(s,v_2) + 1$ → coada devine: $v_2, \ldots, v_p$, → $dist(s,v_2) \leq \ldots \leq dist(s,v_p) \leq dist(s,v_1) + 1 \leq dist(s,v_2) + 1$
  - Cum relația este respectată, înseamnă că Prop(Q) este adevărată.

BFS – Proprietăți (IV)

- Corolar
  - d(u) = momentul în care nodul u este inserat în coada Q. Atunci:

$$d(u) < d(v) \Rightarrow dist(s,u) \leq dist(s,v).$$

- Teorema 5.1. BFS este corect și după terminare $\delta(s,u) = dist(s,u)$, $\forall$ u din V.
  - Utilizăm notația $V_k = \{ u \in V \mid \delta(s,u) = k \}$
  - Dem prin inducție P(k) = $\forall u \in V_k$ avem proprietatea $\delta(s,u) = dist(s,u)$ && $(k > 0 \Rightarrow \pi(u) \in V_{k-1})$ && $(k = 0 \Rightarrow \pi(u) = null)$
  - Caz de bază: k = 0, $V_0 = \{s\}$, $\delta(s,s) = dist(s,s) = 0$ și $\pi(s) = null$ Adevărat

BFS – Proprietăți (V)

Pas de inducție: k > 0, P(k) adevărat → P(k+1) adevărat

- Alegem la întâmplare un nod v ∈ V_{k+1} → δ(s,v) = k+1 și fie u ∈ V_k predecesorul lui v pe drumul cel mai scurt s..v.
- Caz 1: u este descoperit după v
  - dist(s,v) ≤ dist(s,u) (Corolar 5.1) iar dist(s,u) = δ(s,u) → dist(s,v) ≤ k
  - dist(s,v) ≥ δ(s,v) = k+1 (Lema 5.3) ⊥
- Caz 2: u este descoperit înainte de v și v e descoperit pe arcul (u,v)
  - dist(s,v) = dist(s,u) + 1 și π(v) = u, u ∈ V_k → P(k+1) este adevărat
- Caz 3: u este descoperit înainte de v și v NU e descoperit pe arcul (u,v). Fie z ≠ u , a.î. v e descoperit pe arcul (z,v)
  - a) d(u) < d(z) < d(v) → cul(v) = alb la d(u) → v e descoperit pe arcul (u, v) ⊥
  - b) d(z) < d(u) → dist(s,z) ≤ dist(s,u) = δ(s,u) → dist(s,z) ≤ k
    dist(s,v) = dist(s,z) + 1 și k + 1 = δ(s,v) ≤ dist(s,v) → dist(s,z) ≥ k
    → dist(s,z) = k → dist(s,v) = k + 1 = δ(s,v) → drumul s..z,v e optim
    →z ∈ V_k → π(v) = z ∈ V_k → P(k+1) este adevărat

Complexitate? Optimalitate? Completitudine?

BFS – Complexitate și Optimalitate

Complexitate:

O(n+m)

n = număr noduri

m = număr muchii

Optimalitate: DA

Parcurge tot graful? NU

Parcurgere în adâncime (DFS)

- Nu mai avem nod de start, nodurile fiind parcurse în ordine.

- d(u) = momentul descoperirii nodului (se trece prima oară prin u și e totodată și momentul începerii explorării zonei din graf ce poate fi atinsă din u).
- f(u) = timpul de finalizare al nodului (momentul în care prelucrarea nodului u a luat sfârșit)
  - Tot subarborele de adâncime dominat de u a fost explorat.
  - Alternativ: tot subgraful accesibil din u a fost descoperit și finalizat deja.

DFS – Structura de date

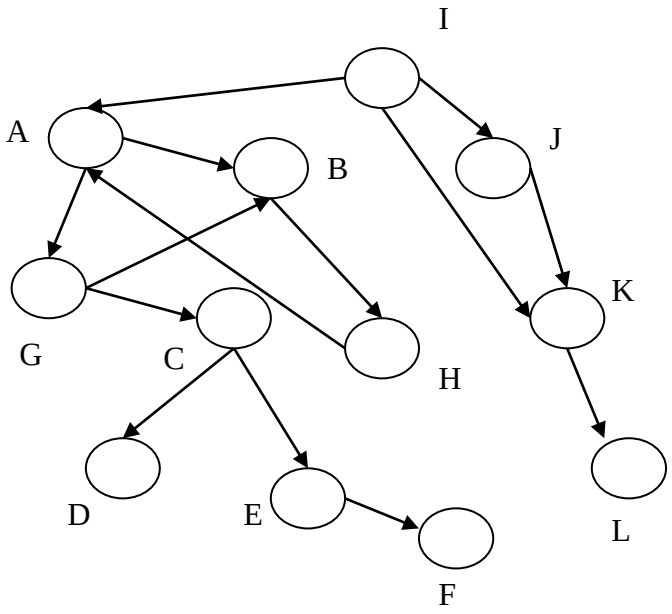
- Folosește o stivă (LIFO) pentru a reține nodurile ce trebuie prelucrate
  - În implementările uzuale, stiva este rareori folosită explicit;
  - Se apelează la recursivitate pentru a simula stiva.
- Folosește o variabilă globală *timp* pe baza căreia se calculează timpii de descoperire și de finalizare ai fiecărui nod.
- Pentru fiecare nod se reține:
  - Părintele – $\pi(u)$ ($p(u)$);
  - Timpul de descoperire – $d(u)$;
  - Timpul de finalizare – $f(u)$;
  - Culoarea nodului.

DFS – Algoritm

- DFS(G)
  - V = noduri(G)
  - **Pentru fiecare** nod u (u ∈ V)
    - c(u) = alb; p(u) = null; // inițializare structură date
  - timp = 0; // reține distanța de la rădăcina arborelui DFS pană la nodul curent
  - **Pentru fiecare** nod u (u ∈ V)
    - **Dacă** c(u) este alb
      - **Atunci** explorare(u); // explorez nodul

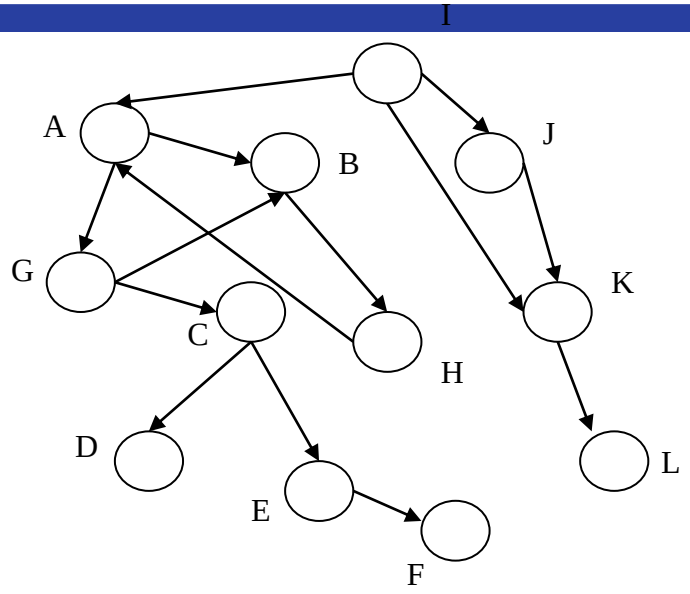
- explorare(u)
  - d(u) = ++ timp; // timpul de descoperire al nodului u
  - c(u) = gri; // nod în curs de explorare
  - **Pentru fiecare** nod v (v ∈ succs(u)) // încerc sa prelucrez vecinii
    - **Dacă** c(v) este alb
      - **Atunci** {p(v) = u; explorare(v);} // dacă nu au fost prelucrați deja
  - c(u) = negru; // am terminat de explorat nodul u
  - f(u) = ++ timp; // timpul de finalizare al nodului u

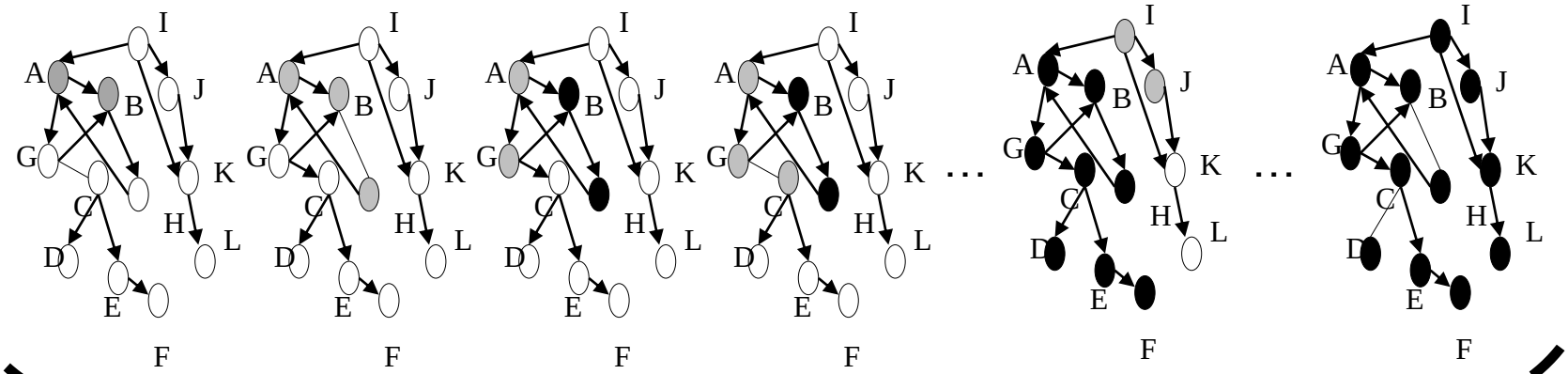
DFS – Exemplu



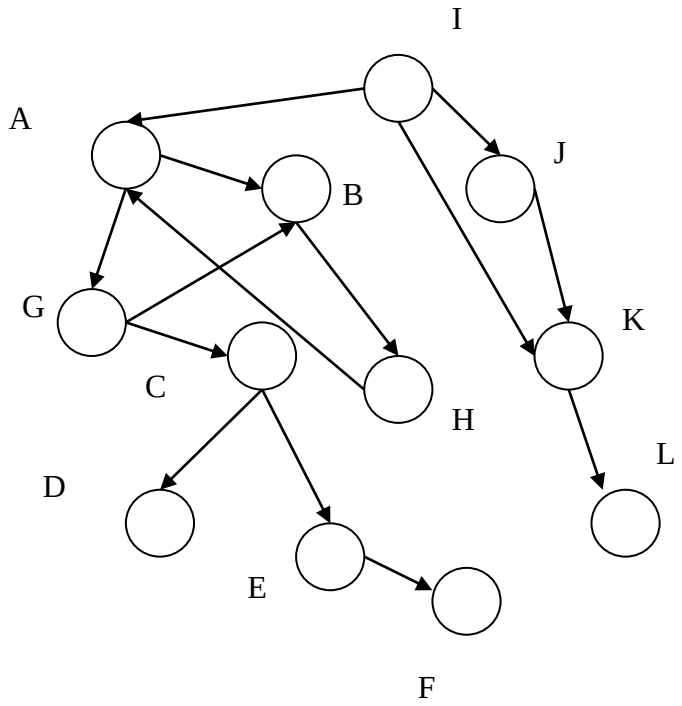
- DFS(G)
  - V = noduri(G)
  - **Pentru fiecare** nod u (u ∈ V)
    - c(u) = alb; p(u) = null; // inițializare structură date
  - timp = 0; // reține distanța de la rădăcina arborelui // DFS până la nodul curent
  - **Pentru fiecare** nod u (u ∈ V)
    - **Dacă** c(u) este alb
      - **Atunci** explorare(u); // explorez nodul
- explorare(u)
  - d(u) = ++ timp; // timpul de descoperire al nodului u
  - c(u) = gri; // nod in curs de explorare
  - **Pentru fiecare** nod v (v ∈ succs(u)) // încerc sa // prelucrez vecinii
    - **Dacă** c(v) este alb
      - **Atunci** {p(v) = u; explorare(v);} // dacă nu au // fost prelucrați deja
  - c(u) = negru; // am terminat de explorat nodul u
  - f(u) = ++ timp; // timpul de finalizare al nodului u

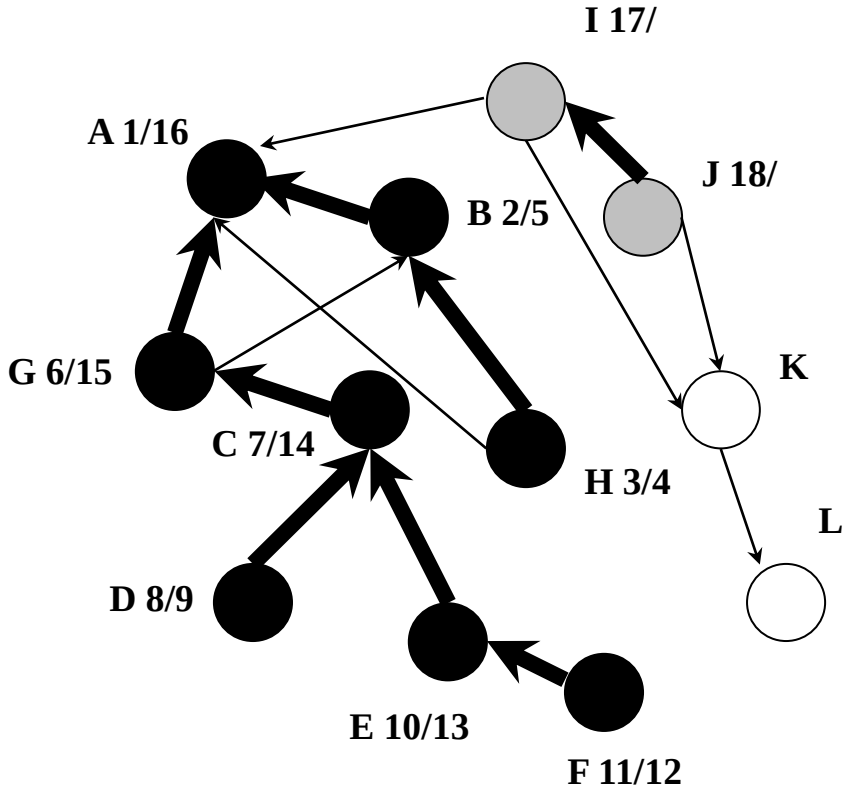
DFS – Evoluția explorării



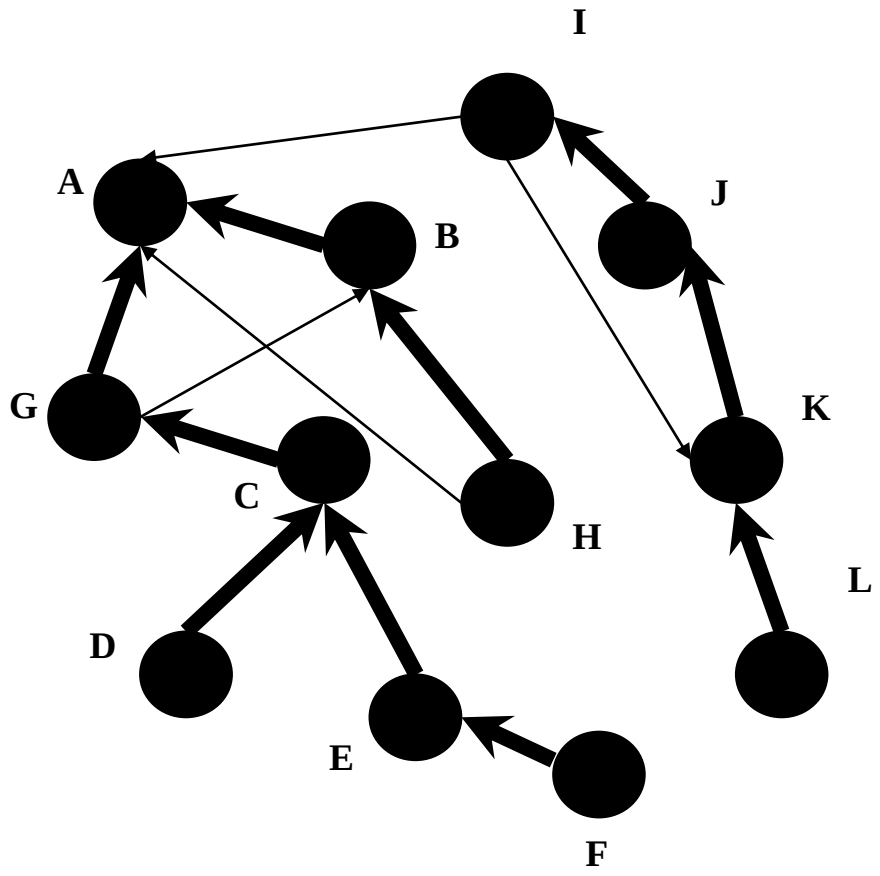


DFS – Calculul timpilor

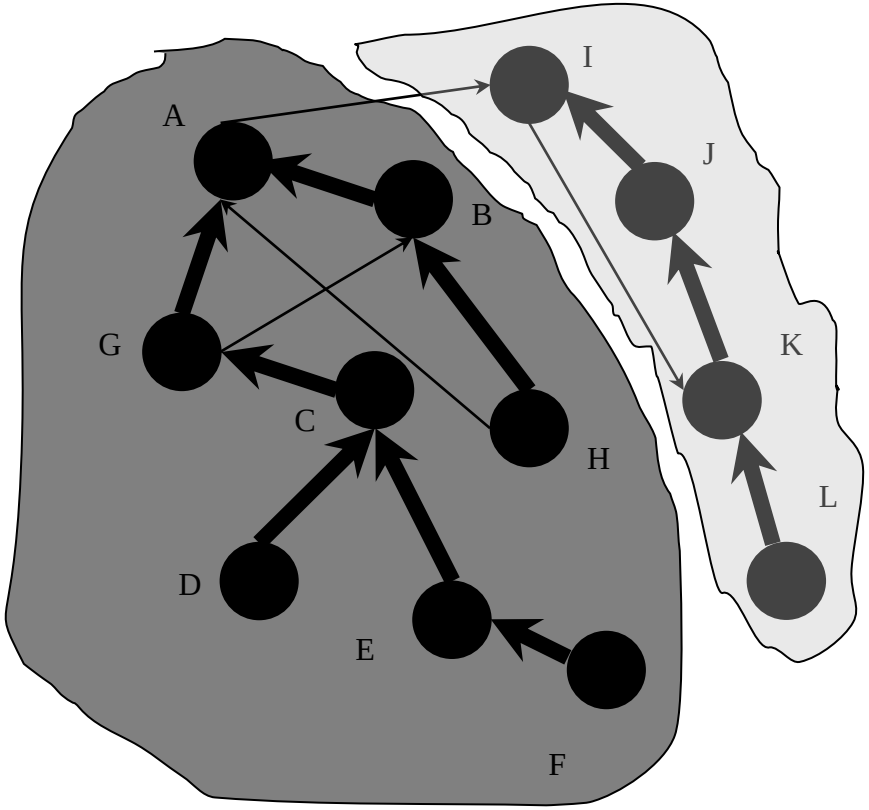




DFS – Pădurea de arbori de parcurgere în adâncime



DFS – Zone de explorare



DFS – Proprietăți (I)

- I(u) = intervalul de prelucrare al nodului (d(u),f(u)).
- Lema 5.5. G = (V,E); u ∈ V; pentru fiecare v descoperit de DFS pornind din u este construită o cale v, p(v), p(p(v)),…, u.
  - Fie calea u = $v_0v_1…v_n$ = v. Dem prin inducție ca $\pi(v_i) = v_{i-1}$!
- Teorema 5.2. G = (V,E); DFS(G) sparge graful G într-o pădure de arbori Arb(G) = { Arb(u); p(u) = null } unde Arb(u) = (V(u),E(u));
  - V(u) = { v | d(u) < d(v) < f(u) } + {u};
  - E(u) = { (v, z) | v, z ∈ V(u) && p(z) = v }.
  - Dem: Conform algoritmului, se pot identifica noduri în ciclul principal sau din funcția de explorare. Dacă u e descoperit în ciclul principal, atunci ∃ o cale către toți succesorii dată de părinți → V(u) = { v | d(u) < d(v) < f(u) } iar arcele sunt chiar cele ce desemnează părinții



DFS – Proprietăți (II)

- Teorema 5.3. Dacă DFS(G) generează 1 singur arbore => G este conex. (Reciproca este adevărată?)
  - Graf orientat conex înseamnă că prin transformarea arcelor în muchii se obține un graf neorientat conex.

- Teorema 5.4. Teorema parantezelor:
  - $\forall$ u, v avem $I(u) \cap I(v) = \emptyset$ sau $I(u) \subset I(v)$ sau $I(v) \subset I(u)$.
  - Dem prin considerarea tuturor combinațiilor posibile!
  - a) $I(u) < I(v)$: $d(u) < f(u) < d(v) < f(v)$: $v \notin R(u) \rightarrow$ v rămâne alb pe durata prelucrării lui u $\rightarrow f(u) < d(v)$
  - b) $I(v) \subset I(u)$: $d(u) < d(v) < f(v) < f(u)$: $v \in R(u) \rightarrow$ v este descoperit din u și devine negru înaintea terminării prelucrării lui u $\rightarrow f(v) < f(u)$
  - c) $I(v) < I(u)$: $d(v) < f(v) < d(u) < f(u)$ Analog a)
  - d) $I(u) \subset I(v)$: $d(v) < f(u) < f(u) < f(v)$ Analog b)

DFS – Proprietăți (III)

- Teorema 5.5. $\forall$ u, v $\in$ V, atunci v $\in$ V(u) $\Leftrightarrow$ I(v) $\subset$ I(u).

- Teorema 5.6. Teorema drumurilor albe:
  - G = (V,E); Arb(u); v este descendent al lui u in Arb(u) $\Leftrightarrow$ la momentul d(u) există o cale numai cu noduri albe u..v.
  - Demonstrație prin inducție!
  - v $\in$ V(u) $\rightarrow$ la momentul d(u) există o cale numai cu noduri albe u..v
    - Dacă v $\in$ V(u) $\rightarrow$ $\exists$ o cale unică v .. α .. u de pointeri π. Fie un nod oarecare z din calea α. $\rightarrow$ (Teorema 5.5) d(u) < d(z) < f(z) < f(u) $\rightarrow$ la d(u), c(z) = alb și cum z a fost ales la întâmplare $\rightarrow$ toate nodurile de pe calea α sunt albe la d(u).
  - la momentul d(u) există o cale numai cu noduri albe u..v $\rightarrow$ v $\in$ V(u)
    - Fie u = $v_0 v_1 \ldots v_p$ = v o cale din G, a.î. La d(u) avem c(v_i) = alb, $\forall$i $\in$ 0,p. Dem prin inducție după i că v_i este descendentul lui u în Arb(u).
    - Caz de bază: d(u) < d(v_1) < f(u) $\rightarrow$ (Teorema 5.5) v_1 descendent al lui u Adevărat
    - Pas inducție: v_i descendent al lui u $\rightarrow$ v_{i+1} descendent al lui u
      - v_i descendent al lui u $\rightarrow$ d(u) < d(v_i) < f(v_i) < f(u) . Cum v_{i+1} este alb la d(u) și este succesorul lui v_i $\rightarrow$ v_{i+1} este descoperit după d(u), dar înainte de f(v_i). $\rightarrow$ d(u) < d(v_{i+1}) < f(v_i) < f(u) $\rightarrow$ v_{i+1} descendent al lui u

Clasificări ale arcelor grafului (I)

- Arc direct (de arbore) (u, v)
  - Ce fel de noduri?
- Arc invers (de ciclu) (u, v)
  - Ce fel de noduri?
- Arc înainte (u, v)
  - Ce fel de noduri?
- Arc transversal (u, v)
  - Ce fel de noduri?

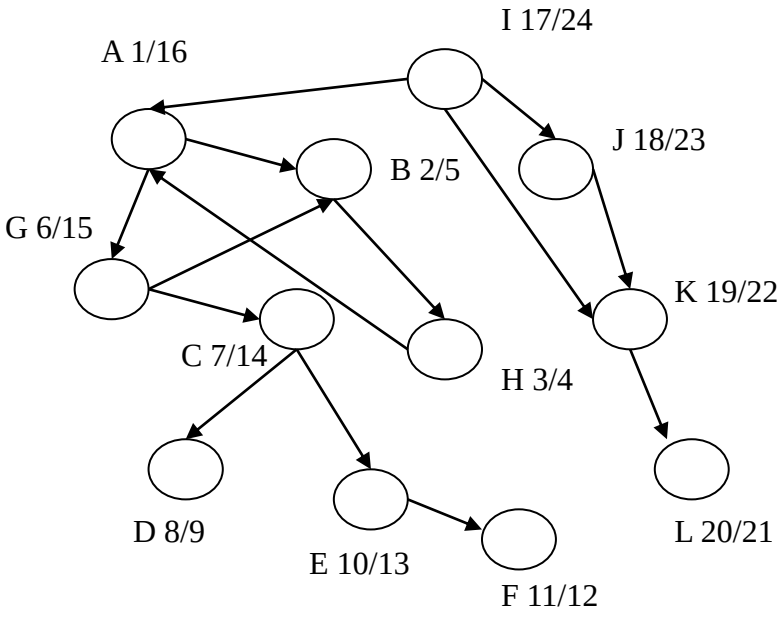


Clasificări ale arcelor grafului (II)

- Arc direct (de arbore) (u, v)
  - între nod gri și nod alb;
- Arc invers (de ciclu) (u, v)
  - între nod gri și nod gri;
- Arc înainte (u, v)
  - nod gri și nod negru și d(u) < d(v);
- Arc transversal (u, v)
  - nod gri și nod negru și d(u) > d(v).



Clasificări ale arcelor grafului (III)



Arc direct (de arbore) (u, v)
între nod gri și nod alb;
Arc invers (de ciclu) (u, v)
între nod gri și nod gri;
Arc înainte (u, v)
nod gri și nod negru și d(u) < d(v);
Arc transversal (u, v)
nod gri și nod negru și d(u) > d(v).

Arc direct (de arbore):
AB, BH, AG, GC, CD, CE, EF, IJ, JK, KL
Arc invers (de ciclu):
HA
Arc înainte:
IK
Arc transversal:
GB, IA

DFS – Proprietăți (IV)

- Teorema 5.7. Într-un graf neorientat, DFS poate descoperi doar muchii directe și inverse.
  - Dem prin considerarea cazurilor posibile!
  - Fie muchia (u,v) ∈ E și pp. d(u) < d(v). Muchia poate fi străbătută din u sau din v:
    - Caz (u,v): c(u) = gri, c(v) = alb → muchie directă
    - Caz (v,u): la d(u) ∃ o cale cu noduri albe u..v → (Teorema drumurilor albe) v este descendent al lui u în Arb(u) → (Teorema 5.5) d(u) < d(v) < f(v) < f(u) → în intervalul (d(v), f(v)) când se investighează (v,u) c(u) = c(v) = gri → muchie inversă

DFS – Proprietăți (V)

- Teorema 5.8. G = graf orientat; G ciclic ⇔ în timpul execuției DFS găsim arce inverse.
  - Dem prin exploatarea proprietăților de ciclu și de arc invers!
  - G ciclic → DFS descoperă arce inverse
    - Fie un ciclu și u primul nod din ciclu descoperit de DFS. Atunci ∃ u..v si (v,u) arcul care închide ciclul. d(u) < d(v) (ipoteză) iar u..v conține doar noduri albe → (Teorema drumurilor albe & Teorema 5.5) d(u) < d(v) < f(v) < f(u) → în intervalul (d(v), f(v)) când se explorează (v,u) c(u) = c(v) = gri → arc invers
  - DFS descoperă arce inverse → G ciclic
    - Fie (v,u) arc invers → d(u) < d(v) < f(v) < f(u) → (Teorema 5.5) → v este descendent al lui u în Arb(u) → ∃ calea u..v care inchide ciclul

DFS – Complexitate și Optimalitate

Complexitate:

O(n+m)

n = număr noduri

m = număr muchii

Optimalitate: NU

Parcurge tot graful? DA

ÎNTREBĂRI?



Bibliografie curs 7

- Giumale – Introducere in Analiza Algoritmilor cap. 5.1 și 5.2
- Cormen – Introducere în Algoritmi cap. Algoritmi elementari de grafuri (23) – Sortare topologică + ComponenteTare Conexe
- http://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm

Proiectarea Algoritmilor

Curs 7 – Parcurgere în adâncime (DFS), Sortare Topologica & Componente Tare Conexe



Bibliografie

- Giumale – Introducere în Analiza Algoritmilor cap. 5.1 și 5.2
- Cormen – Introducere în Algoritmi cap. Algoritmi elementari de grafuri (23) – Sortare topologică + ComponenteTare Conexe
- http://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm

Parcurgere în adâncime (DFS)

- Nu mai avem nod de start, nodurile fiind parcurse în ordine.

- d(u) = momentul descoperirii nodului (se trece prima oară prin u și e totodată și momentul începerii explorării zonei din graf ce poate fi atinsă din u).
- f(u) = timpul de finalizare al nodului (momentul în care prelucrarea nodului u a luat sfârșit)
  - Tot subarborele de adâncime dominat de u a fost explorat.
  - Alternativ: tot subgraful accesibil din u a fost descoperit și finalizat deja.

DFS – Structura de date

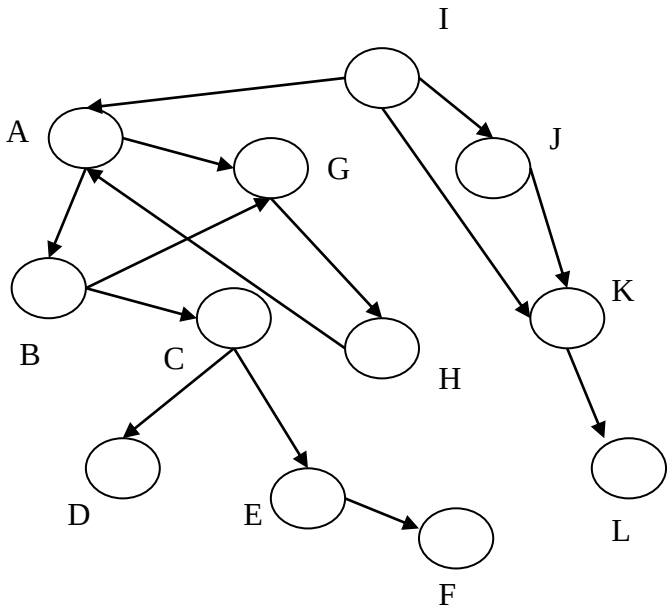
- Folosește o stiva (LIFO) pentru a reține nodurile ce trebuie prelucrate
  - În implementările uzuale, stiva este rareori folosită explicit;
  - Se apelează la recursivitate pentru a simula stiva.
- Folosește o variabilă globală *timp* pe baza căreia se calculează timpii de descoperire și de finalizare ai fiecărui nod.
- Pentru fiecare nod se reține:
  - Părintele – $\pi(u)$ (p(u));
  - Timpul de descoperire – d(u);
  - Timpul de finalizare – f(u);
  - Culoarea nodului.

DFS – Algoritm

- DFS(G)
  - V = noduri(G)
  - **Pentru fiecare** nod u (u ∈ V)
    - c(u) = alb; p(u) = null; // inițializare structură date
  - timp = 0; // reține distanța de la rădăcina arborelui DFS pană la nodul curent
  - **Pentru fiecare** nod u (u ∈ V)
    - **Dacă** c(u) este alb
      - **Atunci** explorare(u); // explorez nodul

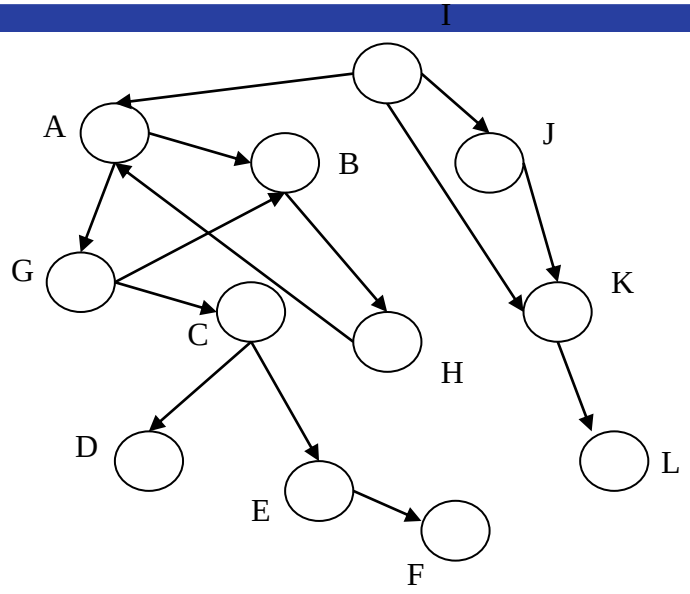
- explorare(u)
  - d(u) = ++ timp; // timpul de descoperire al nodului u
  - c(u) = gri; // nod în curs de explorare
  - **Pentru fiecare** nod v (v ∈ succs(u)) // încerc sa prelucrez vecinii
    - **Dacă** c(v) este alb
      - **Atunci** {p(v) = u; explorare(v);} // dacă nu au fost prelucrați deja
  - c(u) = negru; // am terminat de explorat nodul u
  - f(u) = ++ timp; // timpul de finalizare al nodului u

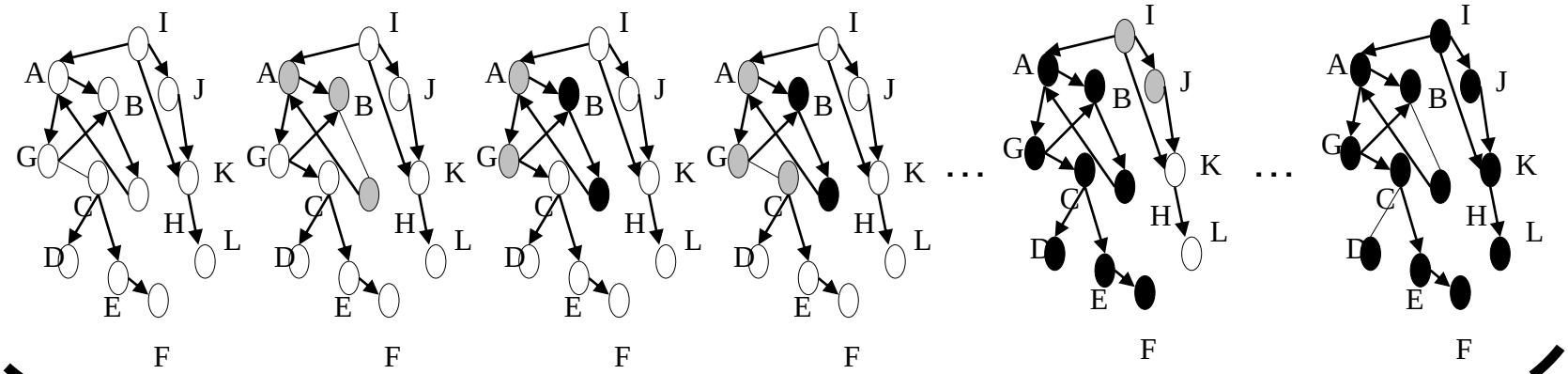
DFS – Exemplu



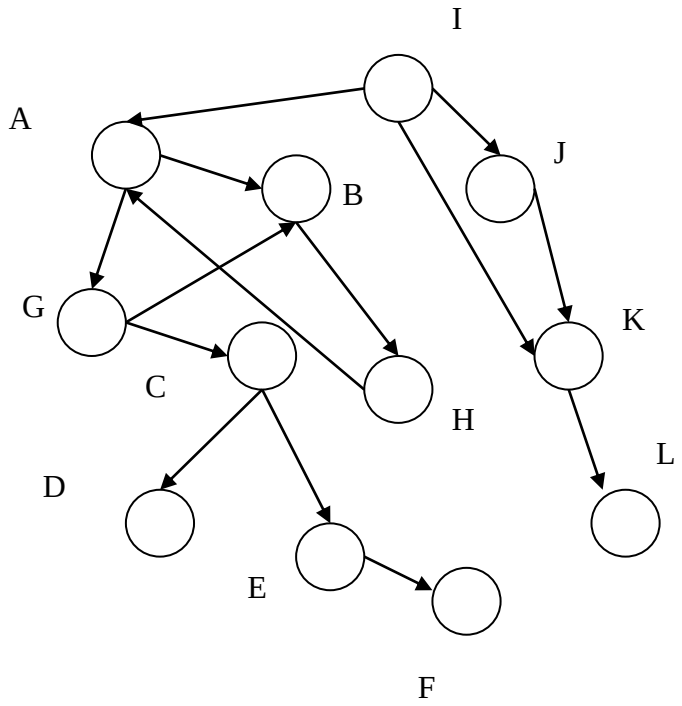
- DFS(G)
  - V = noduri(G)
  - **Pentru fiecare** nod u (u ∈ V)
    - c(u) = alb; p(u) = null; // inițializare structură date
  - timp = 0; // reține distanța de la rădăcina arborelui // DFS până la nodul curent
  - **Pentru fiecare** nod u (u ∈ V)
    - **Dacă** c(u) este alb
      - **Atunci** explorare(u); // explorez nodul
- explorare(u)
  - d(u) = ++ timp; // timpul de descoperire al nodului u
  - c(u) = gri; // nod in curs de explorare
  - **Pentru fiecare** nod v (v ∈ succs(u)) // încerc sa // prelucrez vecinii
    - **Dacă** c(v) este alb
      - **Atunci** {p(v) = u; explorare(v);} // dacă nu au // fost prelucrați deja
  - c(u) = negru; // am terminat de explorat nodul u
  - f(u) = ++ timp; // timpul de finalizare al nodului u

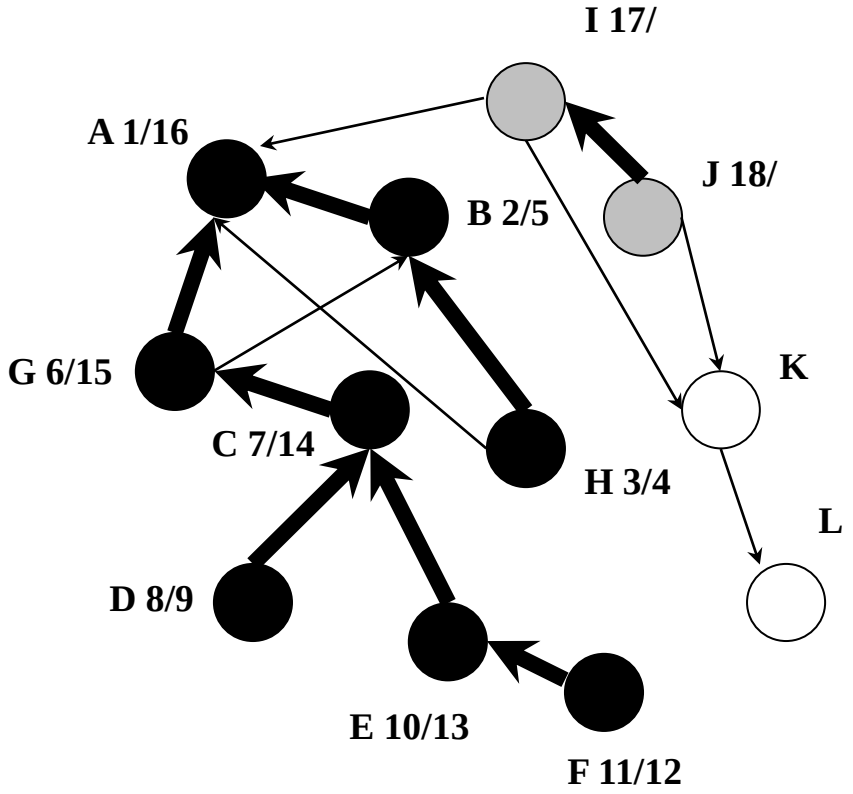
DFS – Evoluția explorării



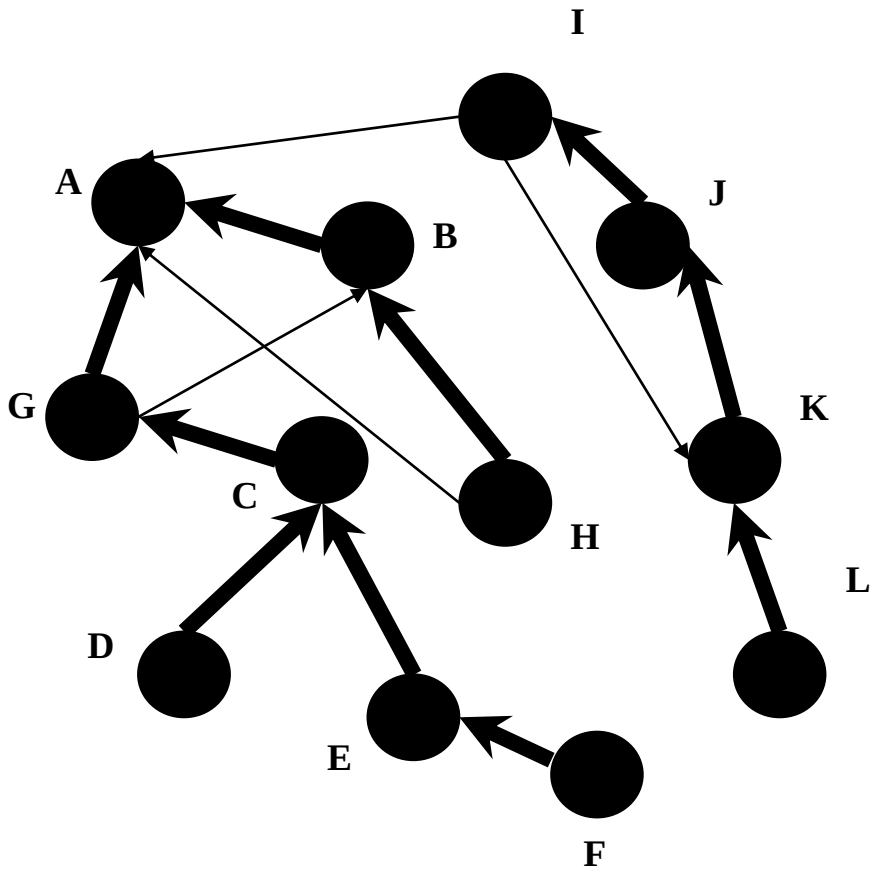


DFS - Calculul timpilor

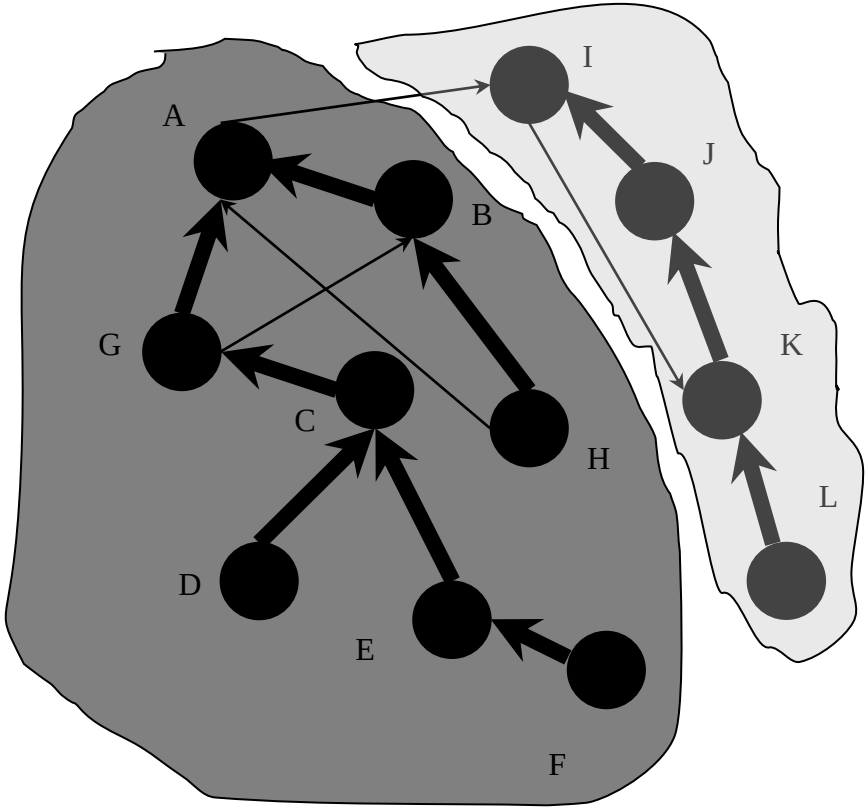




DFS – Pădurea de arbori de parcurgere în adâncime



DFS – Zone de explorare



DFS – Proprietăți (I)

- I(u) = intervalul de prelucrare al nodului (d(u),f(u)).
- Lema 5.5. G = (V,E); u ∈ V; pentru fiecare v descoperit de DFS pornind din u este construită o cale v, p(v), p(p(v)),…, u.
  - Fie calea u = $v_0v_1…v_n$ = v. Dem prin inducție ca $\pi(v_i) = v_{i-1}$!
- Teorema 5.2. G = (V,E); DFS(G) sparge graful G într-o pădure de arbori Arb(G) = { Arb(u); p(u) = null } unde Arb(u) = (V(u),E(u));
  - V(u) = { v | d(u) < d(v) < f(u) } + {u};
  - E(u) = { (v, z) | v, z ∈ V(u) && p(z) = v }.
  - Dem: Conform algoritmului, se pot identifica noduri în ciclul principal sau din funcția de explorare. Dacă u e descoperit în ciclul principal, atunci ∃ o cale către toți succesorii dată de părinți → V(u) = { v | d(u) < d(v) < f(u) } iar arcele sunt chiar cele ce desemnează părinții

DFS – Proprietăți (II)

- Teorema 5.3. Dacă DFS(G) generează 1 singur arbore => G este conex. (Reciproca este adevărată?)
  - Graf orientat conex înseamnă că prin transformarea arcelor în muchii se obține un graf neorientat conex.

- Teorema 5.4. Teorema parantezelor:
  - $\forall$ u, v avem $I(u) \cap I(v) = \emptyset$ sau $I(u) \subset I(v)$ sau $I(v) \subset I(u)$.
  - Dem prin considerarea tuturor combinațiilor posibile!
  - a) $I(u) < I(v)$: $d(u) < f(u) < d(v) < f(v)$: $v \notin R(u) \rightarrow$ v rămâne alb pe durata prelucrării lui u $\rightarrow f(u) < d(v)$
  - b) $I(v) \subset I(u)$: $d(u) < d(v) < f(v) < f(u)$: $v \in R(u) \rightarrow$ v este descoperit din u și devine negru înaintea terminării prelucrării lui u $\rightarrow f(v) < f(u)$
  - c) $I(v) < I(u)$: $d(v) < f(v) < d(u) < f(u)$ Analog a)
  - d) $I(u) \subset I(v)$: $d(v) < f(u) < f(u) < f(v)$ Analog b)

DFS – Proprietăți (III)

- Teorema 5.5. $\forall$ u, v $\in$ V, atunci v $\in$ V(u) $\Leftrightarrow$ I(v) $\subset$ I(u).

- Teorema 5.6. Teorema drumurilor albe:
  - G = (V,E); Arb(u); v este descendent al lui u in Arb(u) $\Leftrightarrow$ la momentul d(u) există o cale numai cu noduri albe u..v.
  - Demonstrație prin inducție!
  - v $\in$ V(u) $\rightarrow$ la momentul d(u) există o cale numai cu noduri albe u..v
    - Dacă v $\in$ V(u) $\rightarrow$ $\exists$ o cale unică v.. α..u de pointeri π. Fie un nod oarecare z din calea α. $\rightarrow$ (Teorema 5.5) d(u) < d(z) < f(z) < f(u) $\rightarrow$ la d(u), c(z) = alb și cum z a fost ales la întâmplare $\rightarrow$ toate nodurile de pe calea α sunt albe la d(u).
  - la momentul d(u) există o cale numai cu noduri albe u..v $\rightarrow$ v $\in$ V(u)
    - Fie u = $v_0 v_1 \ldots v_p$ = v o cale din G, a.î. La d(u) avem c(v_i) = alb, $\forall$i $\in$ 0,p. Dem prin inducție după i că v_i este descendentul lui u în Arb(u).
    - Caz de bază: d(u) < d(v_1) < f(u) $\rightarrow$ (Teorema 5.5) v_1 descendent al lui u Adevărat
    - Pas inducție: v_i descendent al lui u $\rightarrow$ v_{i+1} descendent al lui u
      - v_i descendent al lui u $\rightarrow$ d(u) < d(v_i) < f(v_i) < f(u) . Cum v_{i+1} este alb la d(u) și este succesorul lui v_i $\rightarrow$ v_{i+1} este descoperit după d(u), dar înainte de f(v_i). $\rightarrow$ d(u) < d(v_{i+1}) < f(v_i) < f(u) $\rightarrow$ v_{i+1} descendent al lui u

Clasificări ale arcelor grafului (I)

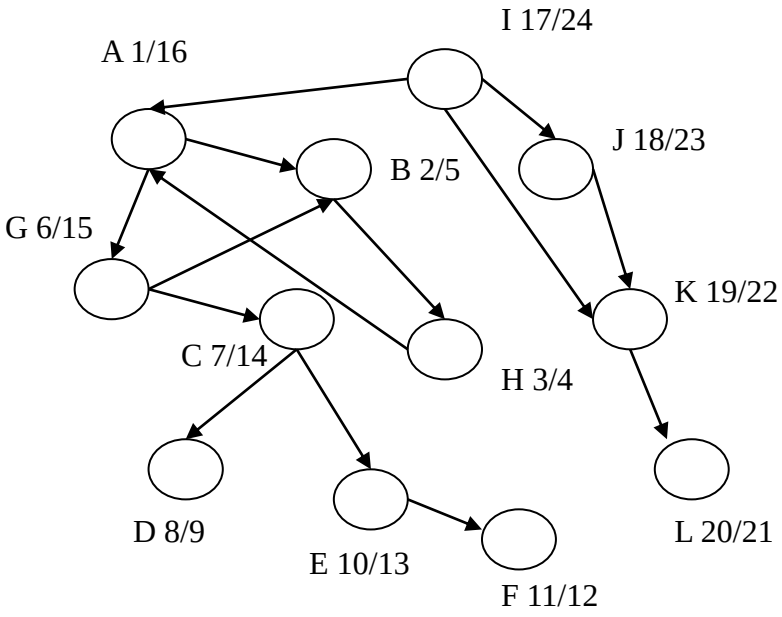
- Arc direct (de arbore) (u, v)
  - Ce fel de noduri?
- Arc invers (de ciclu) (u, v)
  - Ce fel de noduri?
- Arc înainte (u, v)
  - Ce fel de noduri?
- Arc transversal (u, v)
  - Ce fel de noduri?

Clasificări ale arcelor grafului (II)

- Arc direct (de arbore) (u, v)
  - între nod gri și nod alb;
- Arc invers (de ciclu) (u, v)
  - între nod gri și nod gri;
- Arc înainte (u, v)
  - nod gri și nod negru și d(u) < d(v);
- Arc transversal (u, v)
  - nod gri și nod negru și d(u) > d(v).



Clasificări ale arcelor grafului (III)



Arc direct (de arbore) (u, v)
între nod gri și nod alb;
Arc invers (de ciclu) (u, v)
între nod gri și nod gri;
Arc înainte (u, v)
nod gri și nod negru și d(u) < d(v);
Arc transversal (u, v)
nod gri și nod negru și d(u) > d(v).

Arc direct (de arbore):
AB, BH, AG, GC, CD, CE, EF, IJ, JK, KL
Arc invers (de ciclu):
HA
Arc înainte:
IK
Arc transversal:
GB, IA

DFS – Proprietăți (IV)

- Teorema 5.7. Într-un graf neorientat, DFS poate descoperi doar muchii directe și inverse.
  - Dem prin considerarea cazurilor posibile!
  - Fie muchia (u,v) ∈ E și pp. d(u) < d(v). Muchia poate fi străbătută din u sau din v:
    - Caz (u,v): c(u) = gri, c(v) = alb → muchie directă
    - Caz (v,u): la d(u) ∃ o cale cu noduri albe u..v → (Teorema drumurilor albe) v este descendent al lui u în Arb(u) → (Teorema 5.5) d(u) < d(v) < f(v) < f(u) → în intervalul (d(v), f(v)) când se investighează (v,u) c(u) = c(v) = gri → muchie inversă

DFS – Proprietăți (V)

- Teorema 5.8. G = graf orientat; G ciclic ⇔ în timpul execuției DFS găsim arce inverse.
  - Dem prin exploatarea proprietăților de ciclu și de arc invers!
  - G ciclic → DFS descoperă arce inverse
    - Fie un ciclu și u primul nod din ciclu descoperit de DFS. Atunci ∃ u..v si (v,u) arcul care închide ciclul. d(u) < d(v) (ipoteză) iar u..v conține doar noduri albe → (Teorema drumurilor albe & Teorema 5.5) d(u) < d(v) < f(v) < f(u) → în intervalul (d(v), f(v)) când se explorează (v,u) c(u) = c(v) = gri → arc invers
  - DFS descoperă arce inverse → G ciclic
    - Fie (v,u) arc invers → d(u) < d(v) < f(v) < f(u) → (Teorema 5.5) → v este descendent al lui u în Arb(u) → ∃ calea u..v care inchide ciclul



DFS – Complexitate și Optimalitate

Complexitate:

O(n+m)

n = număr noduri

m = număr muchii

Optimalitate: NU

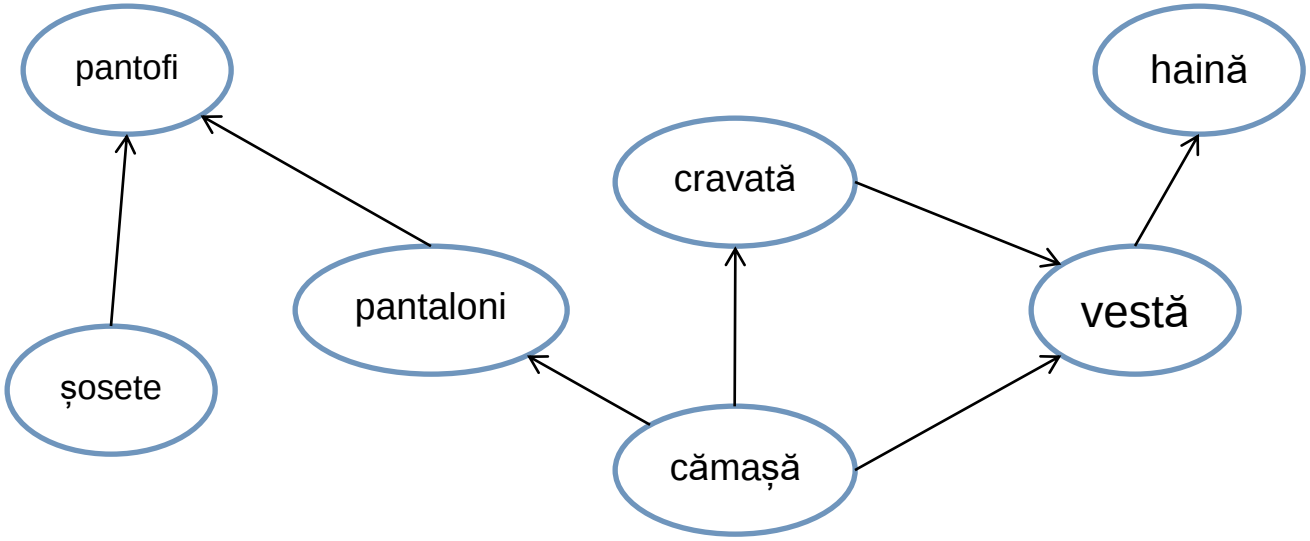
Parcurge tot graful? DA

Sortare topologică

- Se folosește la sortarea unei mulțimi parțial ordonate (nu orice pereche de elemente pot fi comparate).
- Fie A o mulțime parțial ordonată față de o relație de ordine $\propto$ ($\propto \subseteq A*A$) atunci $\exists$ e_1 și e_2 astfel încât e_1, e_2 nu pot fi comparate.
- O sortare topologică a lui A este o listă L = $<e_1,e_2 \ldots e_n>$, cu proprietatea că $\forall$ i, j, dacă $e_i \propto e_j$, atunci i < j.

Sortare topologică - Exemplu

- A = { pantofi, șosete, cravată, haină, vestă, pantaloni, cămașă }.



Sortare topologică

- G = (V,E) orientat, aciclic.
- V_S – secvența de noduri a.î. $\forall(u,v) \in E$, avem index(u) < index(v).
- Scop: Sortare_topologică(G) => V_S.
- Idee bazată pe DFS:
  - G = (V,E) orientat, aciclic; la sfârșitul DFS avem $\forall(u,v) \in E$, f(v) < f(u)
  - => colectăm în V_S vârfurile în ordinea descrescătoare a timpilor f



Algoritm sortare topologică

- Sortare_topologică (G)
  - **Pentru fiecare** nod u (u ∈ V) {c(u) = alb;} // inițializări
  - $V_S = \emptyset$;
  - **Pentru fiecare** nod u (u ∈ V) // pentru fiecare componentă conexă
    - **Dacă** c(u) este alb
      - V_S = Explorează (u, V_S) // prelucrez componenta conexă
  - **Întoarce** V_S

- Explorează (u, V_S)
  - c(u) = gri // prelucrez nodul, deci îi actualizez culoarea
  - **Pentru fiecare** nod v (v ∈ succs(u))
    - **Dacă** c(v) este alb **atunci** V_S = Explorează (v, V_S) // recursivitate
    - **Dacă** c(v) este gri **atunci Întoarce** Eroare: graf ciclic
  - c(u) = negru // am terminat prelucrarea nodului
  - **Întoarce** cons(u, V_S) // inserează nodul u la începutul lui V_S

Sortare topologică – Observație

- Observație: În general există mai multe sortări posibile!

- Ex:
  - cămașă, cravată, vestă, haină, șosete, pantaloni, pantofi
  - cămașă, pantaloni, cravată, vestă, haină, șosete, pantofi
  - șosete, cămașă, cravată, vestă, haină, pantaloni, pantofi
  - șosete, cămașă, pantaloni, cravată, vestă, haină, pantofi
  - șosete, cămașă, pantaloni, pantofi, cravată, vestă, haină

- Care e numărul maxim de sortări topologice?
- Dar numărul minim?
- Când se obțin aceste valori?

Complexitate?

Sortare topologică – Complexitate

Complexitate:

O(n+m)

n = număr noduri

m = număr muchii



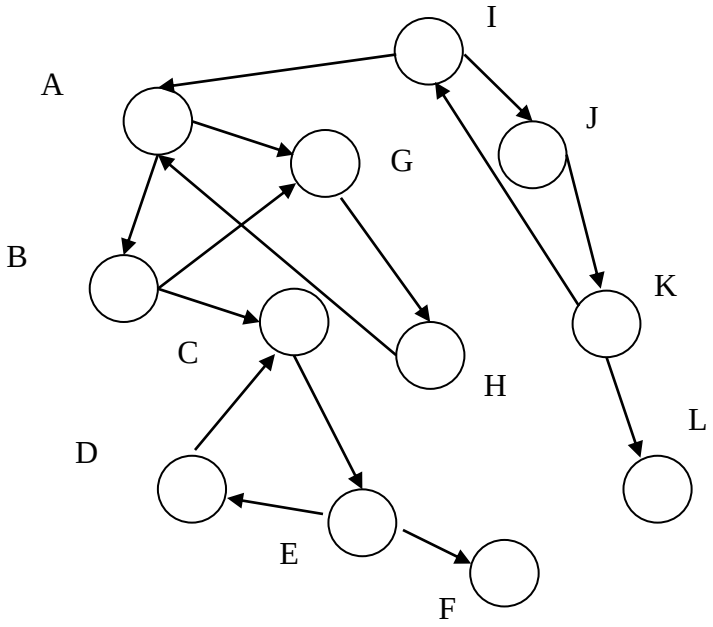
Componente Tare Conexe (CTC)

- Definiție: Fie G = (V,E) un graf orientat. G este tare-conex ⇔ ∀ u,v∈V ∃ o cale u..v și o cale v..u (u∈R(v) și v∈R(u)).

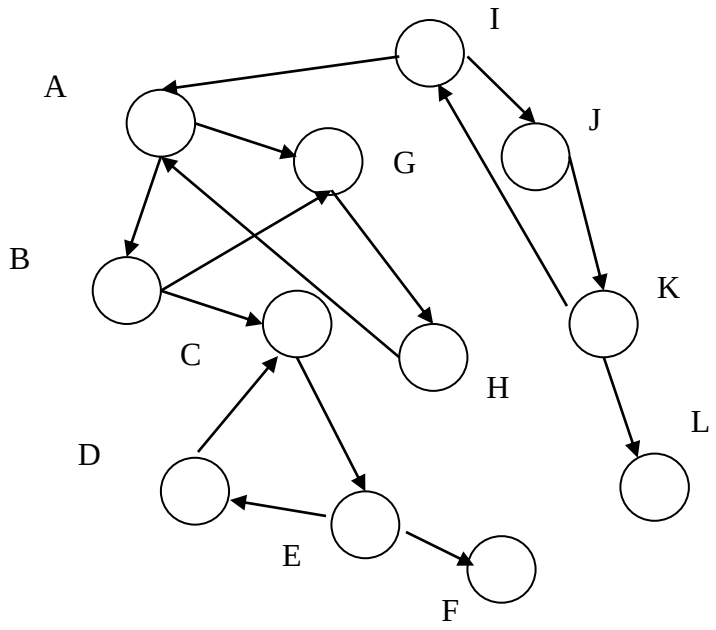
- Definiție: G = (V,E) graf orientat. G' = (V',E'), V'⊆V, E'⊆E. G' este o CTC a lui G ⇔ G' e tare-conex (∀ u,v∈V', u∈R(v) și v∈R(u)) și G' este maximal (ca număr de noduri).

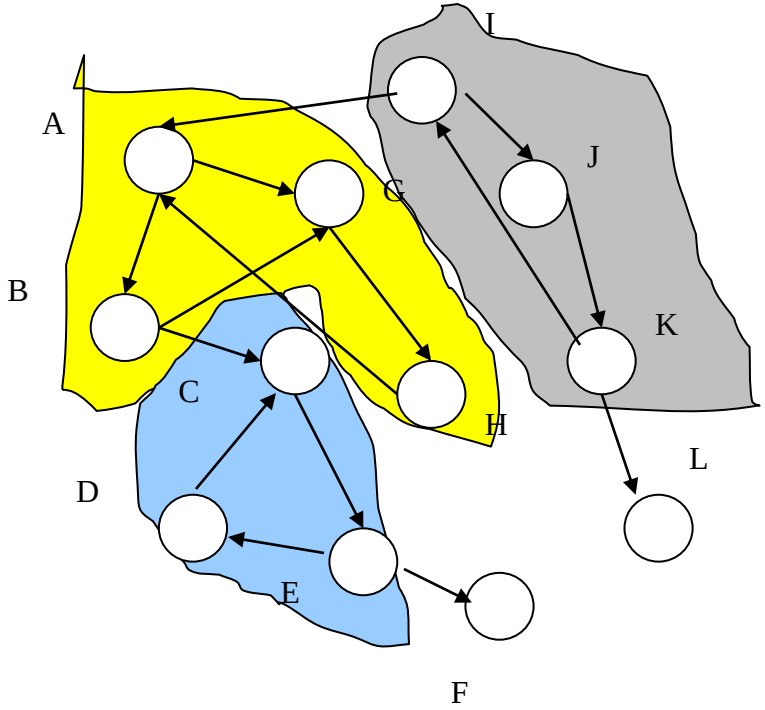
- Lema 5.6: G = (V,E) graf orientat, G' CTC => ∀ u,v∈V' avem ∀ u..v din G are noduri exclusiv in V'
  - Dem: ∀ z a.î. u..z..v => z∈R(u) și v∈R(z). Dar u∈R(v) => z∈R(v) => v și z sunt în aceeași CTC.

Exemplu (I) – determinare CTC



Exemplu (II) – determinare CTC(2)



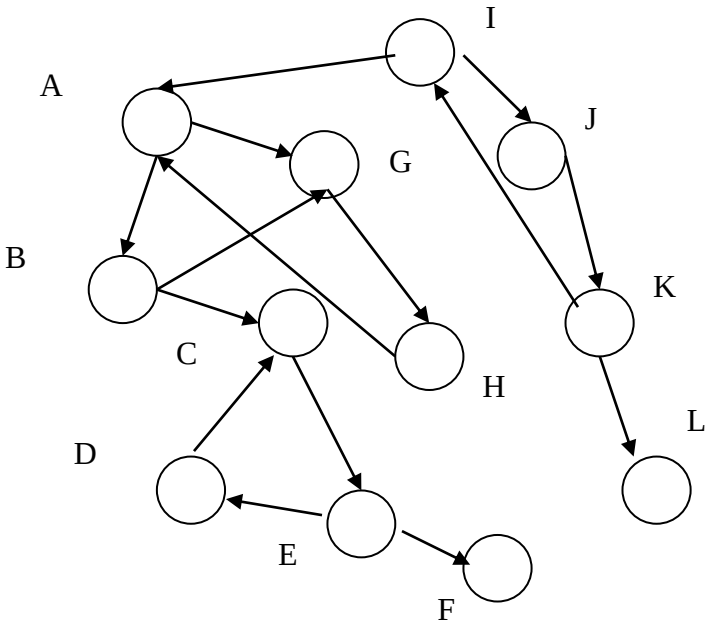


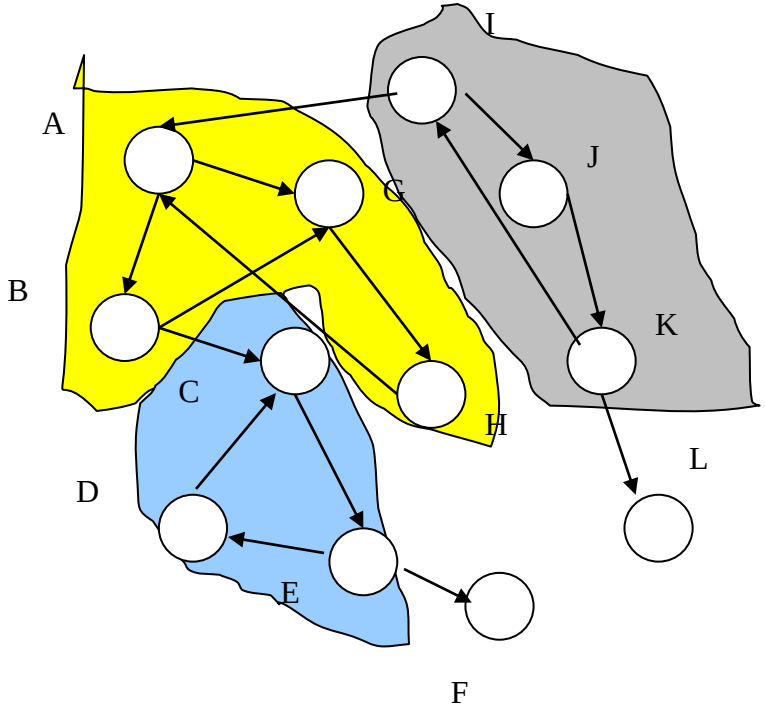
Componente Tare Conexe (CTC)

- Teorema 5.10: G = (V,E) orientat, G' = (V',E') o CTC a lui G. Toate nodurile v∈V' sunt grupate in același Arb(u) construit de DFS(G), unde u este primul nod descoperit al componentei.
  - Dem: ∀ v∈V', v ≠ u, ∃ u..v drum cu noduri albe la momentul descoperirii d(u); toate nodurile drumului sunt în V' (conf. Lema 5.6) => (din Teorema drumurilor albe) v este descendent al lui u în Arb(u)

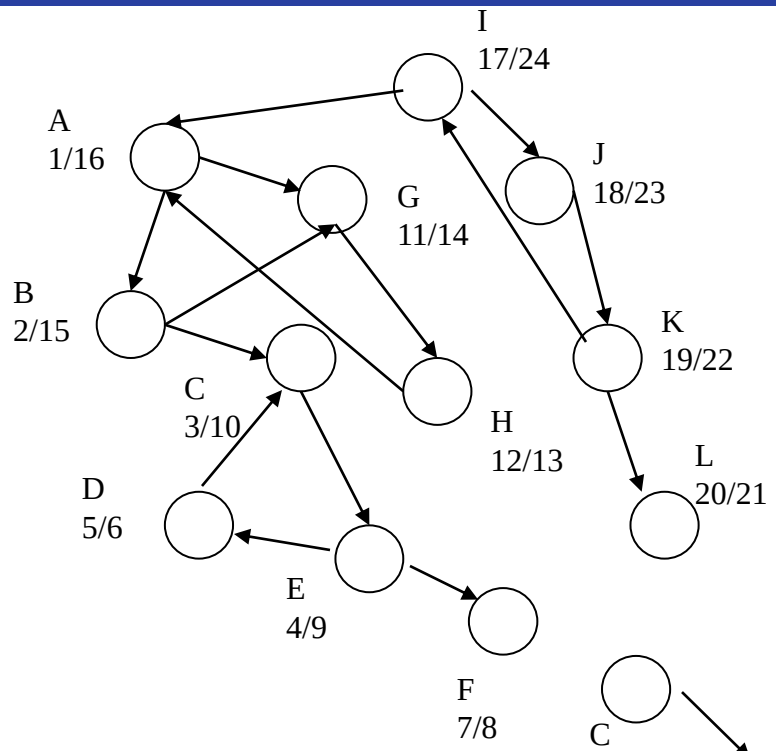
Exemplu (III) – DFS

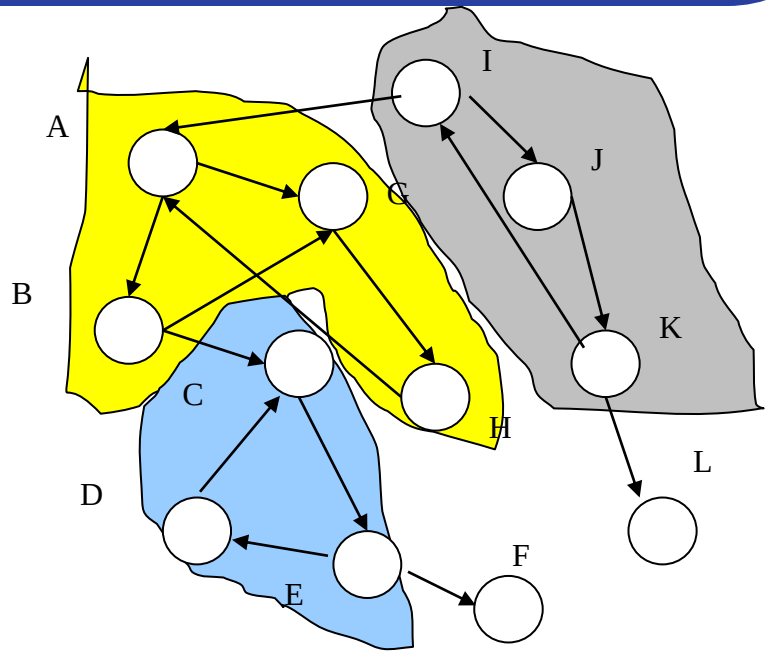
- Aplicare DFS pornind din primul nod al fiecărei CTC

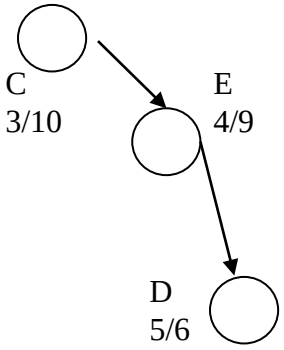


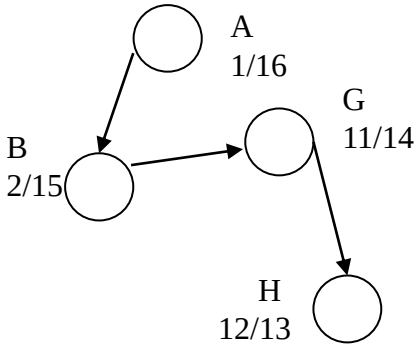


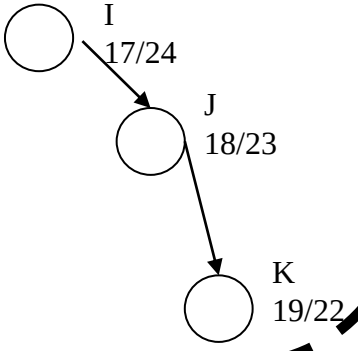
Exemplu (IV) - DFS(2)











Conform Teoremei nodurile din aceeași CTC sunt grupate în același arbore DFS!

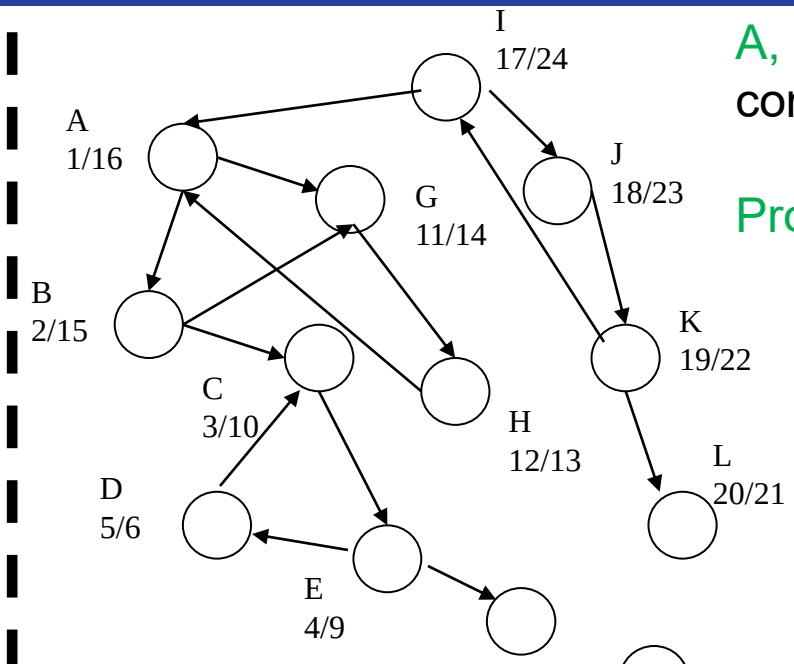
Componente Tare Conexe (CTC)

- Definiție: G = (V,E) orientat, $u \in V$. $\Phi(u)$ = strămoș DFS al lui u determinat în cursul DFS(G) dacă:
  - $\Phi(u) \in R(u)$
  - $f(\Phi(u)) = \max\{f(v) \mid v \in R(u)\}$

- Ce e $\Phi(u)$? $\Phi(u)$ este primul nod din CTC descoperit de DFS(G)

- Teorema 5.11: $\Phi(u)$ satisface următoarele proprietăți:
  1. $f(u) \leq f(\Phi(u))$ când e egalitate? u este primul nod din CTC
  2. $\forall v \in R(u), f(\Phi(v)) \leq f(\Phi(u))$ ce înseamnă ca e egalitate? u si v sunt in aceeași CTC
  3. $\Phi(\Phi(u)) = \Phi(u)$ Dem: $\Phi(u) \in R(u) \xrightarrow{2} f(\Phi(\Phi(u))) \leq f(\Phi(u))$; $\xrightarrow{1} f(\Phi(\Phi(u))) \geq f(\Phi(u)) \rightarrow f(\Phi(\Phi(u))) = f(\Phi(u)) \rightarrow \Phi(\Phi(u)) = \Phi(u)$

Exemplu (V) – stramosi



A, C, I sunt strămoși DFS ai nodurilor din componenta conexă din care fac parte.

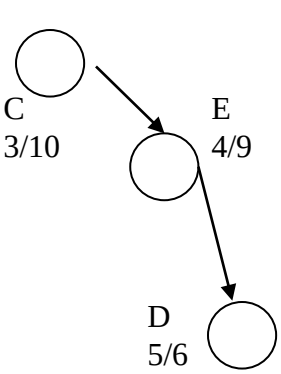
Proprietățile strămoșilor:

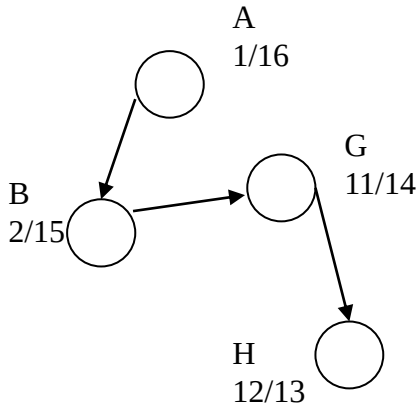
- $f(u) \leq f(\Phi(u))$

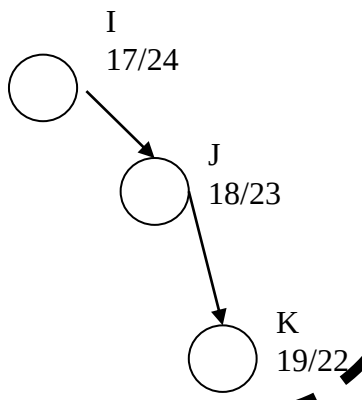
  Ex: $f(B) \leq f(\Phi(B)) = f(A)$
- $\forall\ v \in R(u),\ f(\Phi(v)) \leq f(\Phi(u))$

  Ex: $E \in R(B),\ f(\Phi(E)) = f(C) \leq f(A) = f(\Phi(B))$
- $\Phi(\Phi(u)) = \Phi(u)$

  Ex: $\Phi\ (\Phi(E)) = \Phi(C) = C = \Phi(E)$







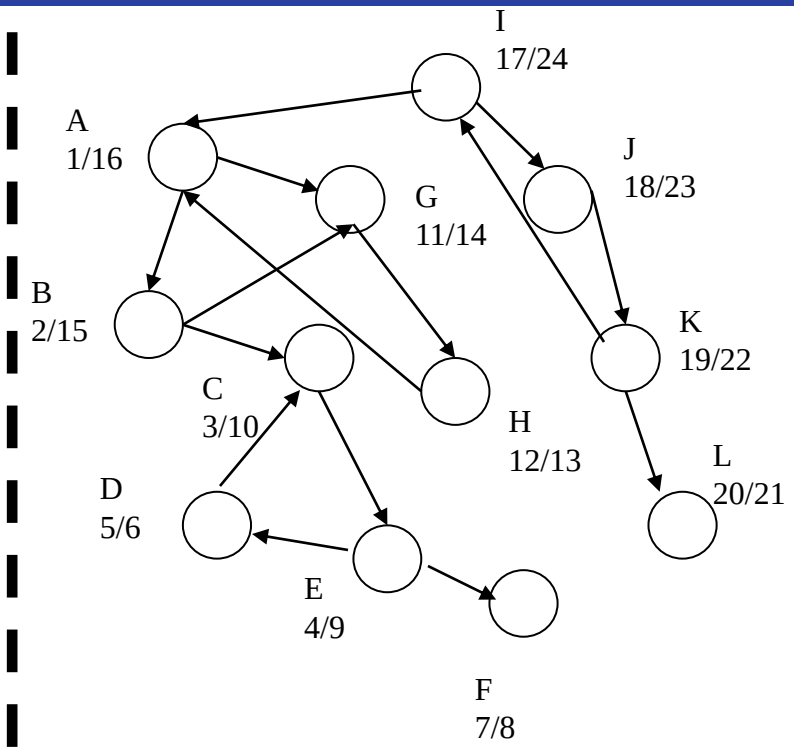
Componente Tare Conexe (CTC)

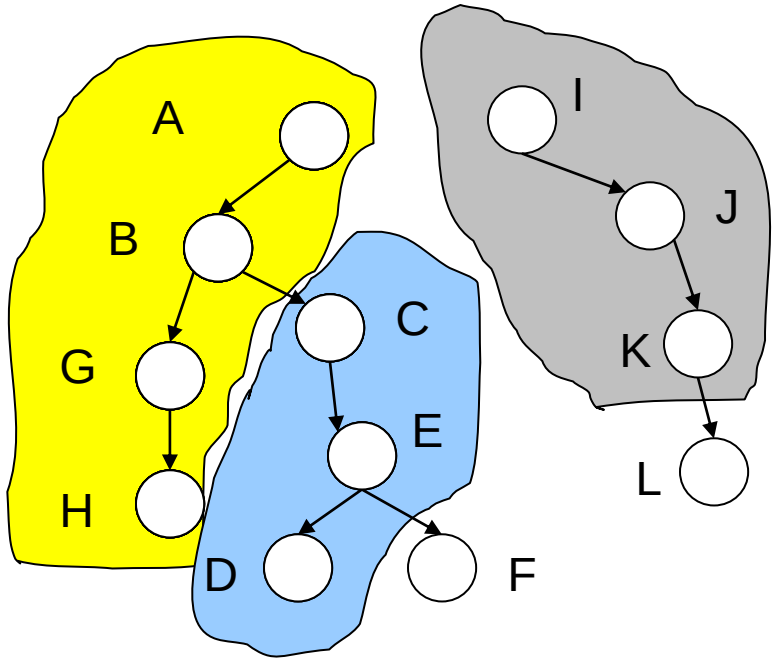
- Din Definiție: $\Phi(u) \in R(u)$ și $f(\Phi(u)) = \max\{f(v) \mid v \in R(u)\}$.

- Teorema 5.12. G = (V,E) orientat, $\forall\ u \in V$, u este descendent al lui $\Phi(u)$ în Arb($\Phi(u)$) construit de DFS.
  - Dem: prin considerarea tuturor culorilor posibile ale lui $\Phi(u)$ la momentul d(u).

- Teorema 5.13. G = (V,E) orientat, $\forall\ u,v \in V$; u și v aparțin aceleiași CTC $\Leftrightarrow \Phi(u) = \Phi(v)$.
  - Dem folosind proprietățile strămoșilor :
    - $\forall\ u,v \in$ aceleiași CTC => $\Phi(u) = \Phi(v)$: $v \in R(u)$, $f(\Phi(v)) \leq f(\Phi(u))$ și $u \in R(v)$, $f(\Phi(u)) \leq f(\Phi(v))$ => $f(\Phi(u)) = f(\Phi(v))$ → $\Phi(u) = \Phi(v)$
    - $\Phi(u) = \Phi(v)$ => $\Phi(u) \in R(u)$ => u și $\Phi(u) \in$ aceleiași CTC
      și $\Phi(u) \in R(v)$ => v și $\Phi(u) \in$ aceleiași CTC
      => u și v $\in$ aceleiași CTC

Exemplu (VI)





Primul nod dintr-o CTC descoperit prin DFS va avea drept succesori în arborele generat de DFS toate elementele componentei conexe!

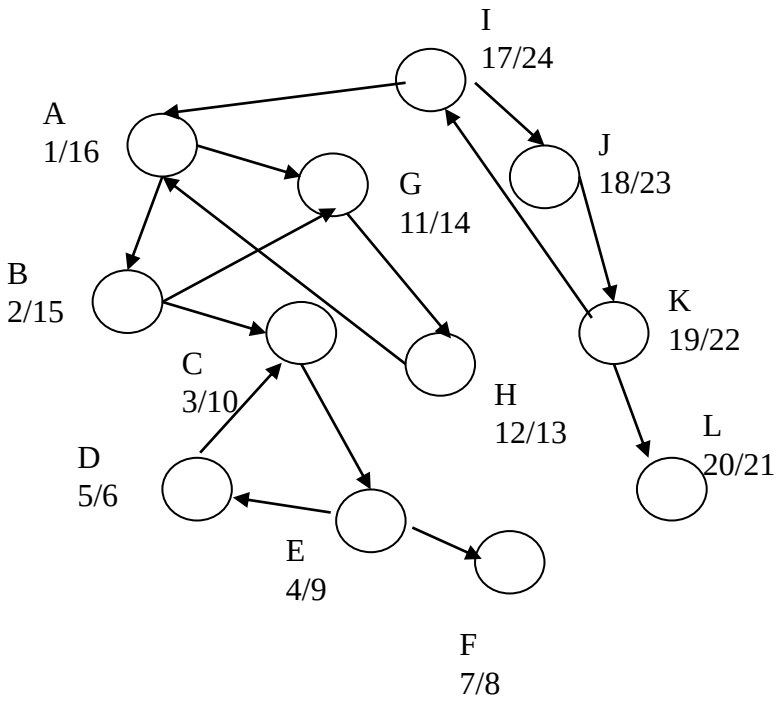
Componente Tare Conexe (CTC)

- Probleme:
  - vrem ca fiecare arbore construit să conțină o CTC.
  - trebuie să eliminăm nodurile care nu sunt în componenta conexă.

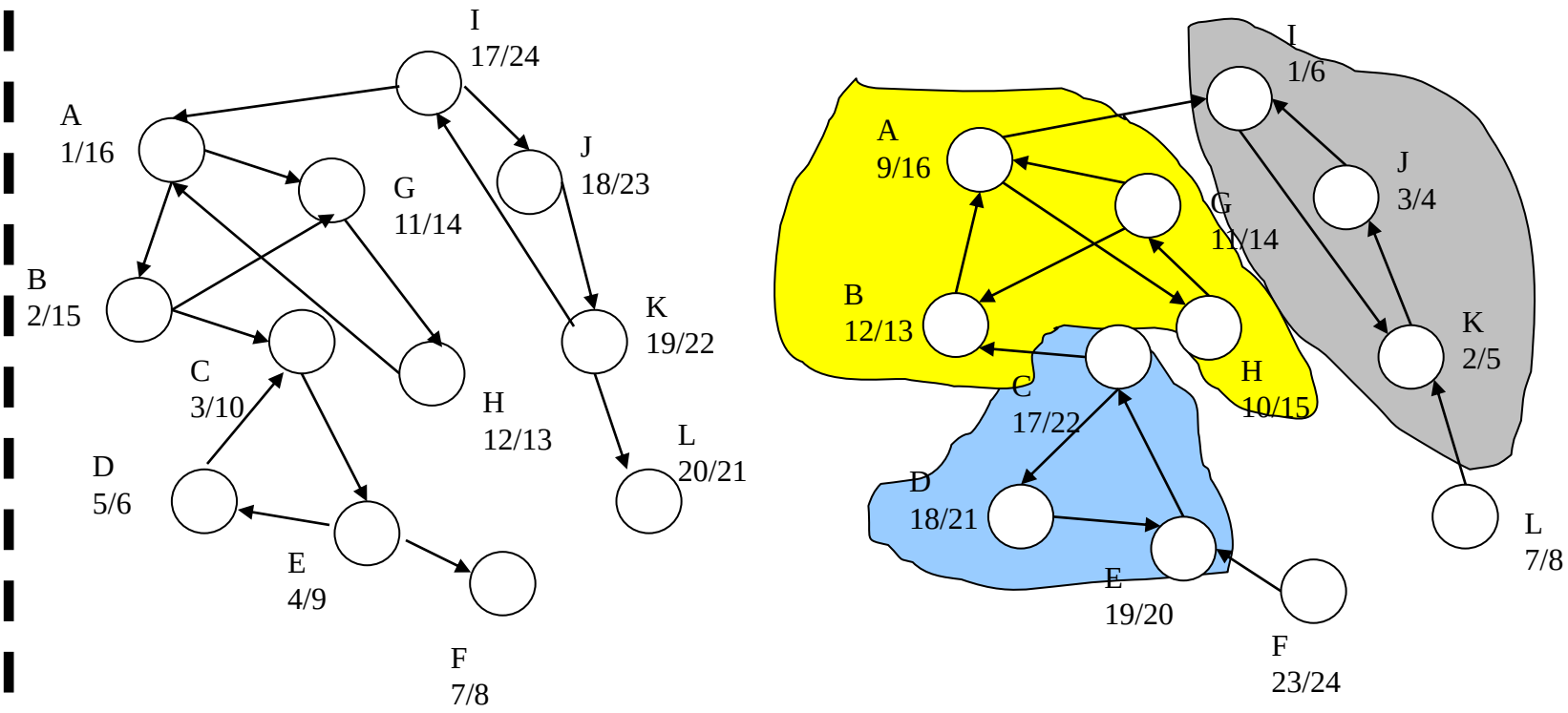
- Idee – eliminăm nodurile ce nu aparțin CTC! Cum?
  - Dacă aparțin Arb(u) și nu CTC => ∃ u..v și ∄ v..u.
  - → DFS pe graful transpus, în ordinea descrescătoare a timpilor de finalizare obținuți din DFS pe graful normal!



Exemplu (VII) – DFS (G^T)

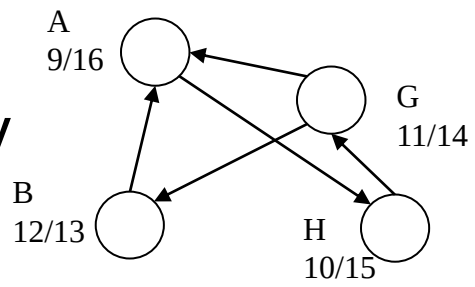


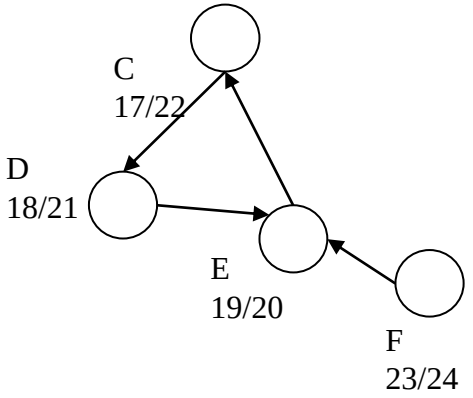
Exemplu (VIII) – DFS (G^T) (2)

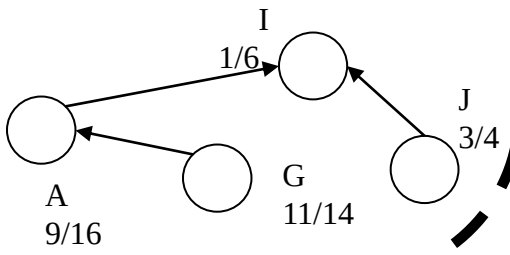


Componente Tare Conexe (CTC)

- Cazuri în DFS(G^T):
  - v este în CTC descoperita din u → v poate fi descoperit din u și în DFS(G^T).
  - v∉CTC dar v∈Arb(u) în DFS(G)→ nu va fi atins în DFS(G^T) din u.
  - v∉CTC dar ∃ v..u în G → f(v) > f(u) → v va fi deja colorat în negru când se explorează u.

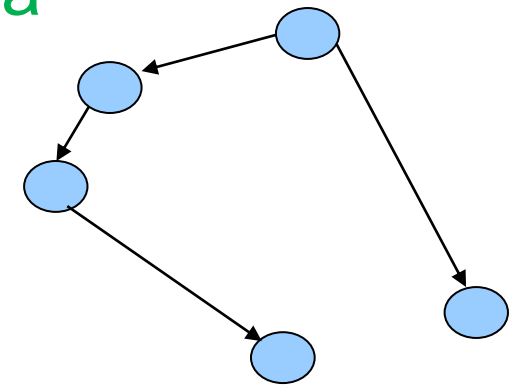






Observatii

- Înlocuind componentele tare conexe cu noduri obținem un graf aciclic. De ce?
- Pentru că altfel am avea o singură CTC!



- Prima parcurgere DFS este o sortare topologică. De ce?
- Pentru că sortează nodurile în ordinea inversă a timpilor de finalizare a strămoșilor fiecărei CTC!

Pseudocod algoritm CTC

- Algoritmul lui Kosaraju:

- CTC(G)
  - DFS(G)
  - G^T = transpune(G)
  - DFS(G^T) (în bucla principală se tratează nodurile în ordinea descrescătoare a timpilor de finalizare de la primul DFS)

- Componentele conexe sunt reprezentate de pădurea de arbori generați de DFS(G^T).

Complexitate?

Algoritmul lui Kosaraju

Complexitate

O(n+m)

n = numar noduri

m = numar muchii

Corectitudine algoritm CTC (1)

- Teoremă: Algoritmul CTC calculează corect componentele tare conexe ale unui graf G = (V,E).
- Dem. prin inducție după nr. de arbori de adâncime găsiți de DFS al G^T că vârfurile din fiecare arbore formează o CTC:
  - Fiecare pas demonstrează că arborele format în acel pas e o CTC, presupunând că toți arborii produși deja sunt CTC.
  - P_1: trivial pentru că ∄ arbori anteriori.
  - $P_n \rightarrow P_{n+1}$: Fie arborele T obținut in pasul curent având rădăcina r. Notăm $C_r = \{v \in V \mid \Phi(v) = r\}$.

Corectitudine algoritm CTC (2)

- Demonstrăm că $u \in T \Leftrightarrow u \in C_r$:
- $u \in C_r \rightarrow u \in T$:
  - $u \in C_r \rightarrow \exists$ r..u → toate nodurile din C_r ajung în același arbore DFS (Arb(r)). Dar $r \in C_r$ și r e rădăcina lui T → $\forall u \in C_r$ => $u \in T$
- $u \in T \rightarrow u \in C_r$: demonstrăm că $\forall$ w a.î. $f(\Phi(w)) > f(r)$ sau $f(\Phi(w)) < f(r)$, $w \notin T$
  - Dacă $f(\Phi(w)) > f(r)$ → la d(r), w e deja pus în CTC cu rădăcina $\Phi(w)$ pt. că nodurile sunt considerate în ordinea inversă a timpilor de finalizare → $w \notin T$
  - Dacă $f(\Phi(w)) < f(r)$ → $w \notin T$ pt. că altfel ($w \in T$) → $\exists$ r..w în G^T → $\exists$ w..r în G → $r \in R(w)$, → $f(\Phi(w)) \geq f(\Phi(r)) = f(r)$ (F)
  - → T conține doar nodurile pt. care $\Phi(w) = r$ → $T = C_r$

ÎNTREBĂRI?



Bibliografie curs 8

[1] Giumale – Introducere în Analiza Algoritmilor cap. 5.3, 5.4, 5.4.1

[2] Cormen – Introducere în Algoritmi cap. Heap-uri binomiale (20), Heap-uri Fibonacci (21), Drumuri minime de sursă unica - primele 2 subcapitole (25.1 și 25.2)

[3] R. Sedgewick, K. Wayne - Algorithms and Data Structures Fall 2007 – Curs Princeton - http://www.cs.princeton.edu/~rs/AlgsDS07/06PriorityQueues.pdf

[4] Heap Fibonacci: http://www.cse.yorku.ca/~aaw/Jason/FibonacciHeapAnimation.html

Proiectarea Algoritmilor

Curs 8 – Puncte de articulație, Punți, Drumuri minime





Bibliografie

[1] Giumale – Introducere în Analiza Algoritmilor cap. 5.3, 5.4, 5.4.1

[2] Cormen – Introducere în Algoritmi cap. Heap-uri binomiale (20), Heap-uri Fibonacci (21), Drumuri minime de sursă unica - primele 2 subcapitole (25.1 și 25.2)

[3] R. Sedgewick, K. Wayne - Algorithms and Data Structures Fall 2007 – Curs Princeton - http://www.cs.princeton.edu/~rs/AlgsDS07/06PriorityQueues.pdf

[4] Heap Fibonacci: https://www.cs.usfca.edu/~galles/JavascriptVisual/FibonacciHeap.html

[5] Fibonacci heap: http://www.growingwiththeweb.com/data-structures/fibonacci-heap/overview/

Obiective

- “Descoperirea” algoritmilor de:
  - Identificare a punctelor de articulație;
  - Identificare a punților;
  - Identificare a drumurilor de cost minim.

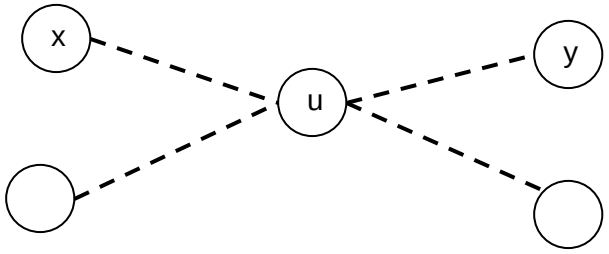
- Identificarea structurilor de date necesare pentru reducerea complexității acestor algoritmi.



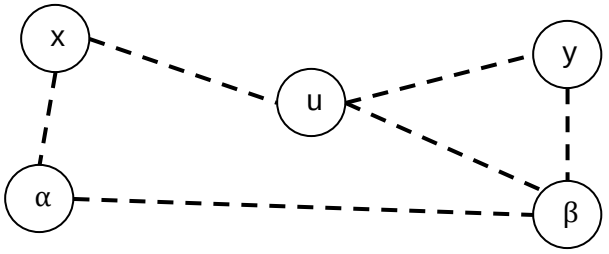


Puncte de articulație. Def. Exemple

- Definiție: G = (V,E) graf neorientat, u∈V. U este punct de articulație dacă ∃ x,y∈V, x ≠ y, x ≠ u, y ≠ u, a.î. ∀ x..y în G trece prin u.



Orice drum x..y trece prin u → u este punct de articulație.



Exista x..α..y care nu trece prin u → u nu mai este punct de articulație!

Algoritm naiv de detectare a punctelor de articulație

- Elimină fiecare nod și verifică conectivitatea grafului rezultat:
  - Graf conex → nodul nu e punct de articulație.
  - Altfel → punct de articulație.

Complexitate?

O(V(V+E))



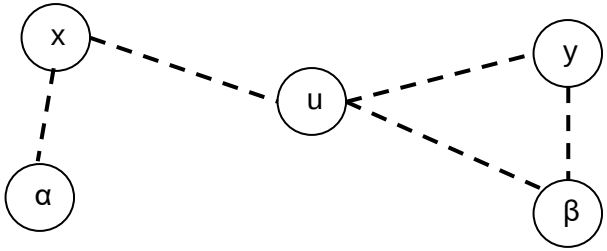


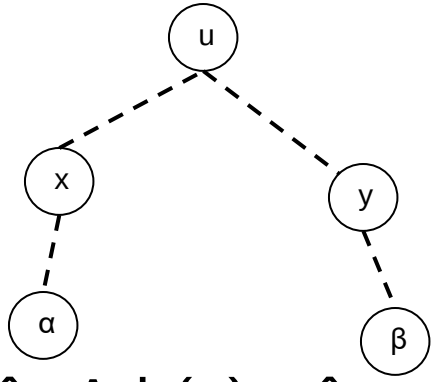
Puncte de articulație. Teoremă

- Teorema 5.15: G = (V,E), graf neorientat și u∈V. U este punct de articulație în G ⇔ în urma DFS în G una din proprietățile de mai jos este satisfăcută:
  - p(u) = null și u domină cel puțin 2 subarbori;
  - p(u) ≠ null și ∃v descendent al lui u în Arb(u) a.î. ∀x∈Arb(v) și ∀(x,z) parcursă de DFS(G) avem d(z) ≥ d(u).

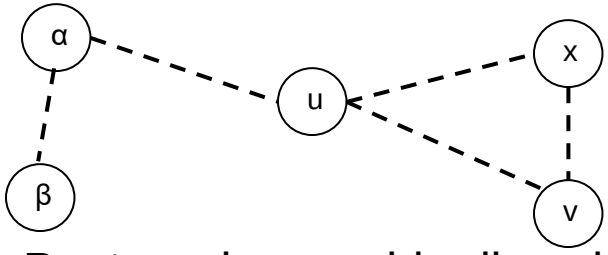
Situații posibile

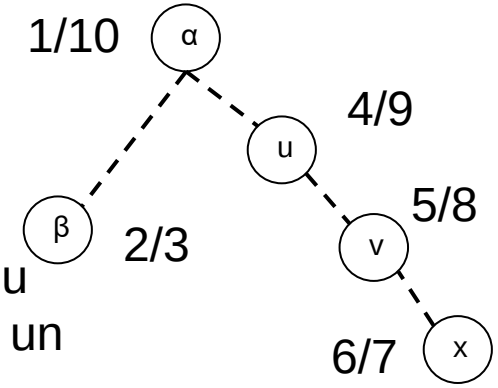
- 1) p(u) = null și u domina cel puțin 2 subarbori:





- 2) p(u) ≠ null și ∃v descendent al lui u în Arb(u) a.î. ∀x∈Arb(v) și ∀(x,z) parcursă de DFS(G) d(z) ≥ d(u):

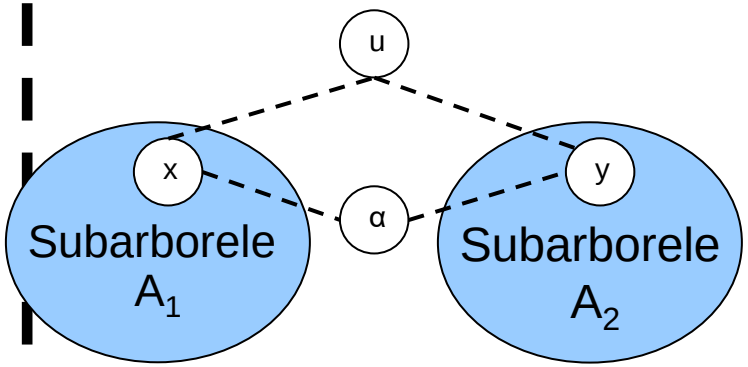




Pentru orice muchie din subarborele lui v nu există nici o muchie înapoi spre u sau spre un nod descoperit înaintea lui u.

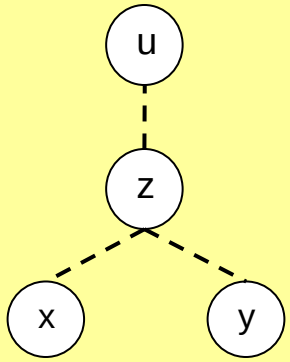
Puncte de articulație. Demonstrație teoremă (Ia)

- p(u) = null și u domină cel puțin 2 subarbori ⟹ u este punct de articulație.
- Dem (Reducere la absurd): Fie A_1 și A_2 cei 2 subarbori, $x \in A_1$, $y \in A_2$. Pp $\exists$ x..α..y și u $\notin$ x..α..y.
- z = primul nod descoperit de DFS din care se poate ajunge la x și la y. Cf. T drumurilor albe x,y ∈ Arb(z).
- Dar x,y∈Arb(u) ➔ 2 cazuri:



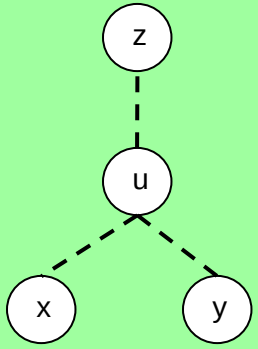
Pp $\exists$ x..α..y și u$\notin$ x..α..y.

Caz 1: d(u)<d(z):



Contradictie (1) x,y nu sunt în subarbori diferiți ai lui Arb(u).

Caz 2: d(z)<d(u):



Contradictie (1), p(u) ≠ null.

Puncte de articulație. Demonstrație teoremă (Ib)

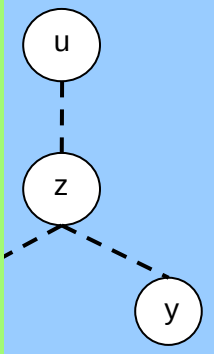
- u este punct de articulație și este descoperit în ciclul principal al DFS ⟹ p(u) = null și u domină cel puțin 2 subarbori.

- Dem (Reducere la absurd): Fie nodurile x și y a.î. u ∈ ∀ x..y. u = primul nod descoperit din cale (altfel u nu mai e descoperit în ciclul principal al DFS) => p(u) = null și x, y ∈ Arb(u).

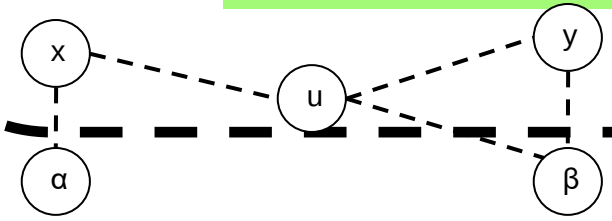
- pp că x, y
  fie z rădă
  x..z..y → u
  se contraz

```
DFS(G)
    V = noduri(G)
    Pentru fiecare nod u (u ∈ V)
        c(u) = alb; p(u) = null; // inițializare structură date
    timp = 0; // reține distanța de la rădăcina arborelui DFS până la nodul curent
    Pentru fiecare nod u (u ∈ V)
        Dacă c(u) este alb
            Atunci explorare(u); // explorez nodul
```

celași subarbore

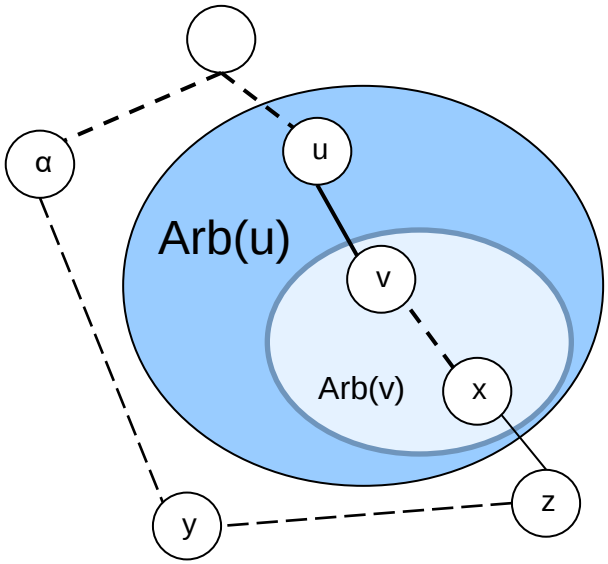


Contradicție ∃ x..z..y =>
u nu este punct de articulație



Puncte de articulație. Demonstrație teoremă (IIa)

- p(u) ≠ null și ∃ v descendent al lui u în Arb(u) a.î. ∀ x ∈ Arb(v) și ∀ (x,z) parcursă de DFS(G) are d(z) ≥ d(u) ⟹ u este punct de articulație.



Dem (Reducere la absurd): Pp. u nu e punct de articulație → ∃ w ∈ Arb(v), y ∉ Arb(u) a.î. y..w. Fie z primul nod din y..w a.î. z ∉ Arb(u) și x ultimul nod din w..y a.î. x ∈ Arb(u) → (x,z) taie frontiera Arb(u).

Dacă d(z) > d(u) → u..x, z alb la d(u) → z ∈ Arb(u) → contradicție (z ∉ Arb(u))

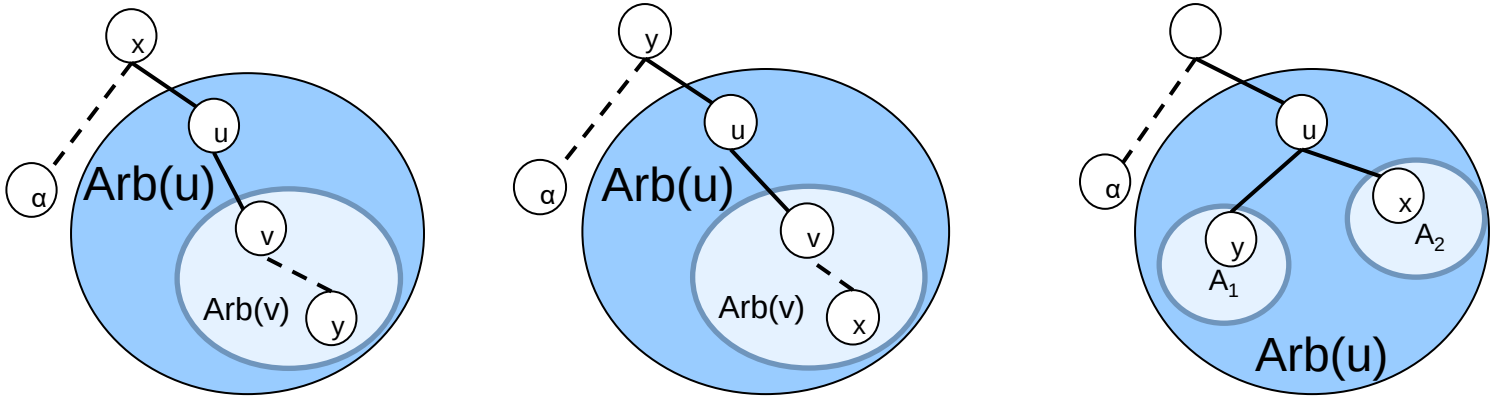
Dacă d(z) < d(u) → contradicție (ipoteza)

→ ∄ y..w → u punct de articulație

Puncte de articulație. Demonstrație teoremă (IIb)

- u este punct de articulație și nu este descoperit în ciclul principal al DFS $\Longrightarrow$ p(u) ≠ null și $\exists$ v descendent al lui u în Arb(u) a.î. $\forall$ x $\in$ Arb(v) și $\forall$ (x,z) parcursă de DFS(G) având d(z) ≥ d(u).

- Dem: Fie nodurile x și y a.î. u $\in$ $\forall$ x..y și p(u) ≠ null. Se pot forma 3 tipuri de structuri:



- Pentru primele 2 structuri, nu trebuie sa existe muchie care sa formeze ciclu de la nici un nod din Arb(v) către vreun predecesor al lui u. Altfel $\exists$ x..y a.î. u $\notin$ x..y.

- Pentru a 3-a structura, trebuie să $\nexists$ muchie care să formeze ciclu către un predecesor al lui u de la niciun nod din cel puțin un subarbore A_1 sau A_2.

Puncte de articulație. Structuri de date.

- Structura de date de la DFS + pentru fiecare nod u ∈ V se reține:
  - Low(u) = min{d(v) | v descoperit pornind din u în cursul DFS și c(v) ≠ alb}
  - Subarb(u) = numărul subarborilor dominați de u (dacă este ≥ 2, atunci avem un punct de articulație).





Idee algoritm

- Se aplică DFS și se salvează pentru fiecare nod până unde merge înapoi (low): low[u] = min {d(u), d(v) pentru toate muchiile înapoi (u,v), low(w) pentru toți fiii w ai lui u}.

- Pentru eficiență, trebuie ca fiii să se parcurgă înaintea părinților → ordinea inversă a d(u).





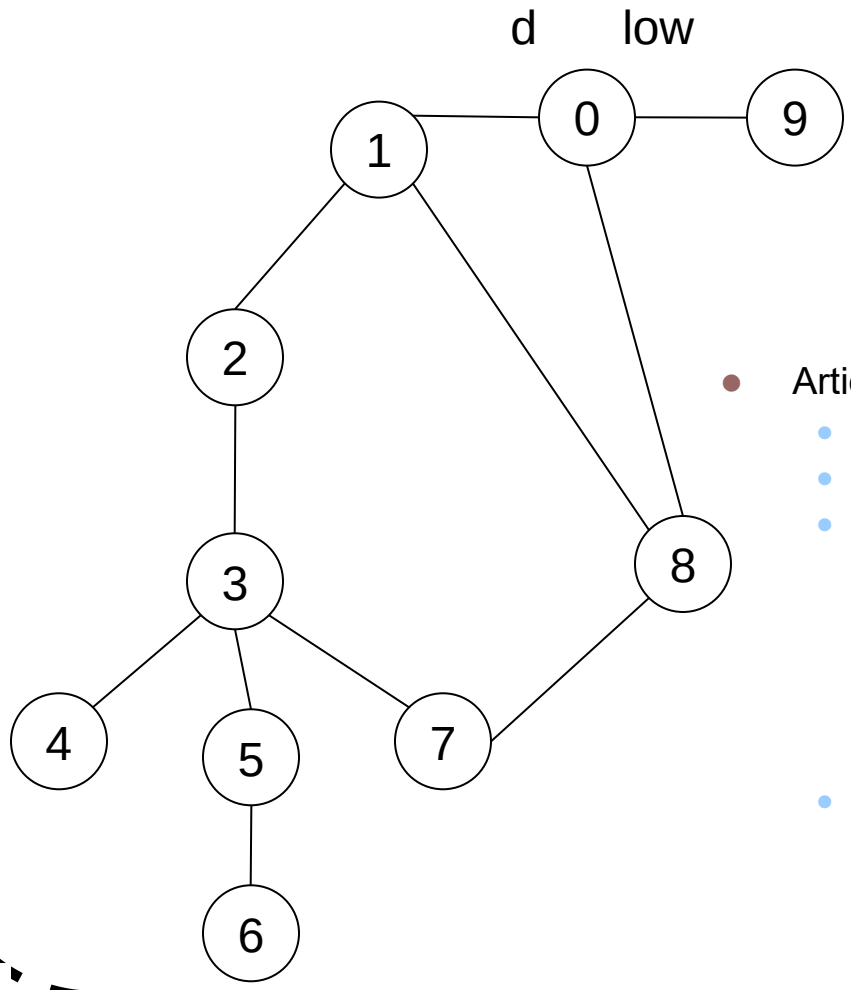
Algoritm Tarjan (I)

- Articulații (G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - subarb(u) = 0; // reține numărul de subarbori dominați de u
    - art(u) = 0; // reține punctele de articulație
  - **Pentru fiecare** (u∈V)
    - **Dacă** c(u) e alb
      - Exploreaza(u);
      - **Dacă** (subarb(u) > 1) // cazul în care u este rădăcina în arborele
        - art(u) = 1 // DFS și are mai mulți subarbori → cazul // 1 al teoremei

Algoritm Tarjan (II)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori
        // dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) ≠ null && low(v) ≥ d(u)) art(u) = 1;
        // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

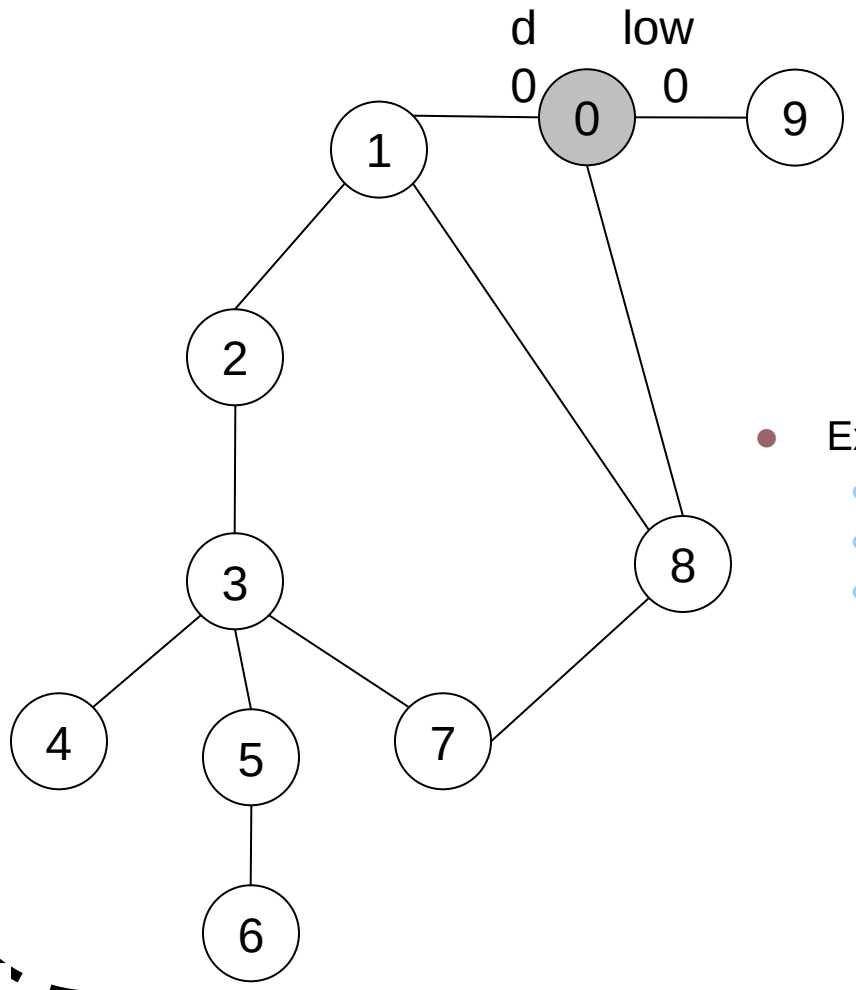
Exemplu rulare (1)



Timp = 0
C(i) = alb
D(i) = 0
Low(i) = 0
P(i) = null
Subarb(i) =0
Art(i) = 0
Exploreaza (0)

- Articulații (G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - subarb(u) = 0; // reține numărul de subarbori dominați de u
    - art(u) = 0; // reține punctele de articulație
  - **Pentru fiecare** (u∈V)
    - **Dacă** c(u) e alb
      - Exploreaza(u);
      - **Dacă** (subarb(u) > 1) // cazul în care u este rădăcina în
        - art(u) = 1 // arborele DFS și are mai mulți // subarbori → cazul 1 al teoremei

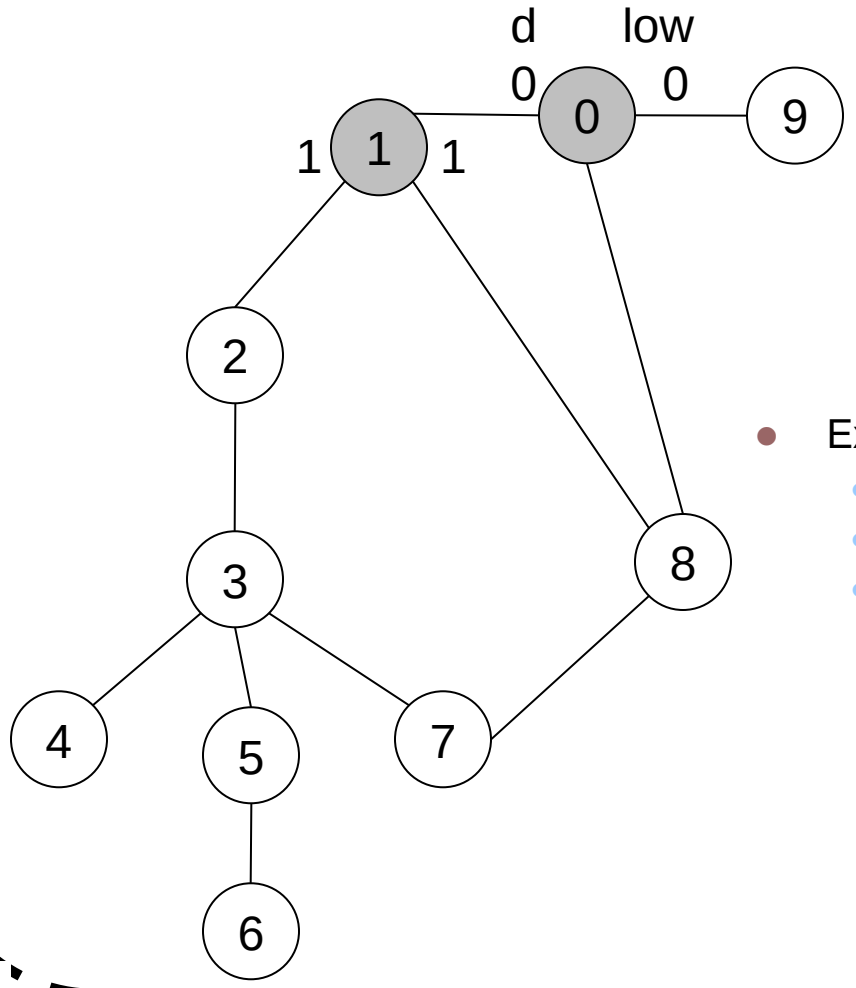
Exemplu rulare (2)



Low(0) = d(0) = 0
Timp = 1
C(0) = gri
P(1) = 0
Subarb(0) = 1
Exploreaza (1)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

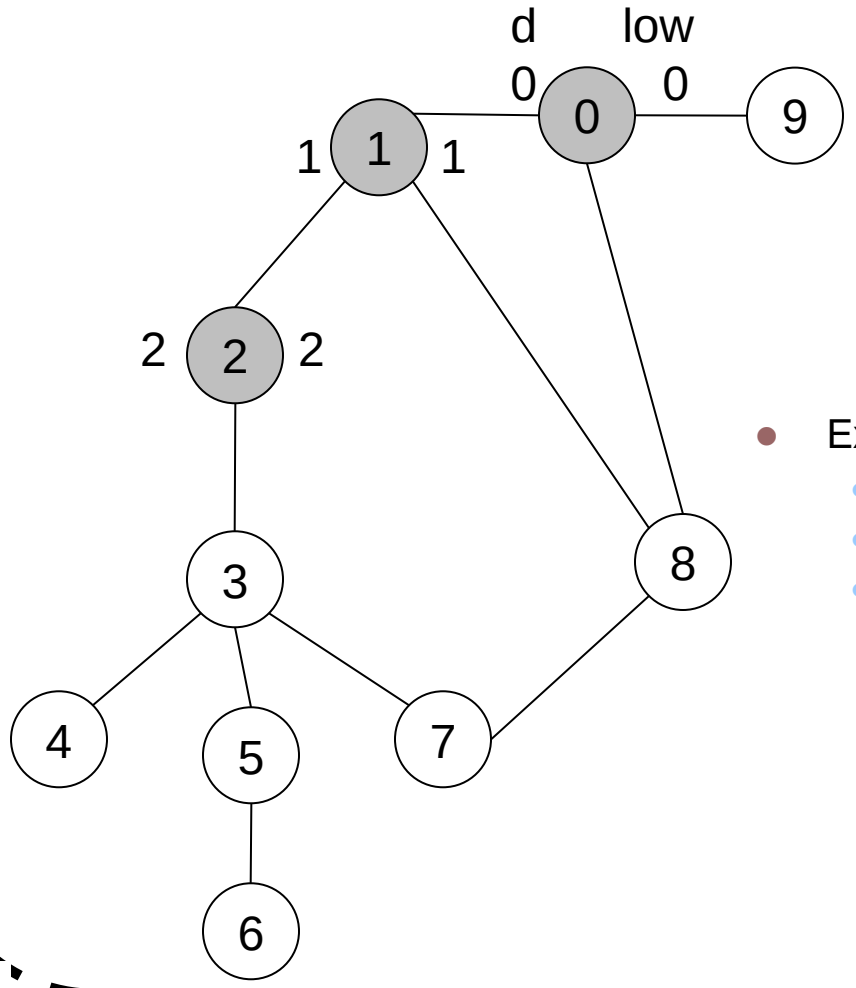
Exemplu rulare (3)



Low(1) = d(1) = 1
Timp = 2
C(1) = gri
P(2) = 1
Subarb(1) = 1
Exploreaza (2)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

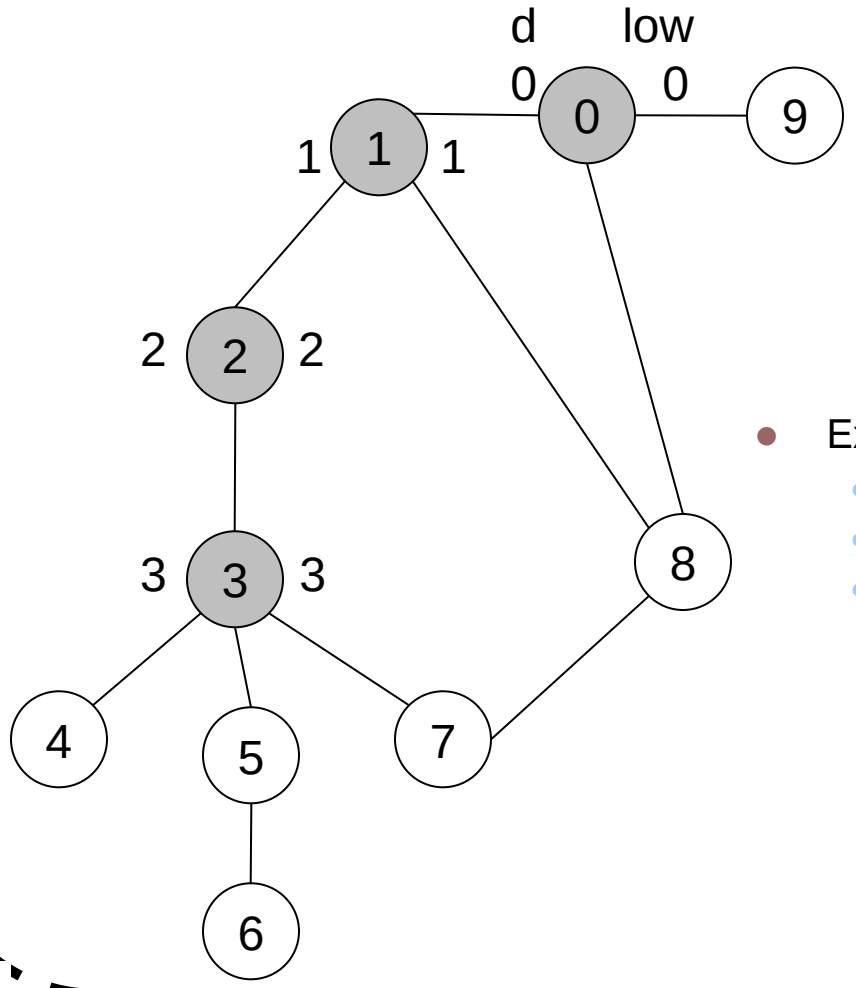
Exemplu rulare (4)



Low(2) = d(2) = 2
Timp = 3
C(2) = gri
P(3) = 2
Subarb(2) = 1
Exploreaza (3)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

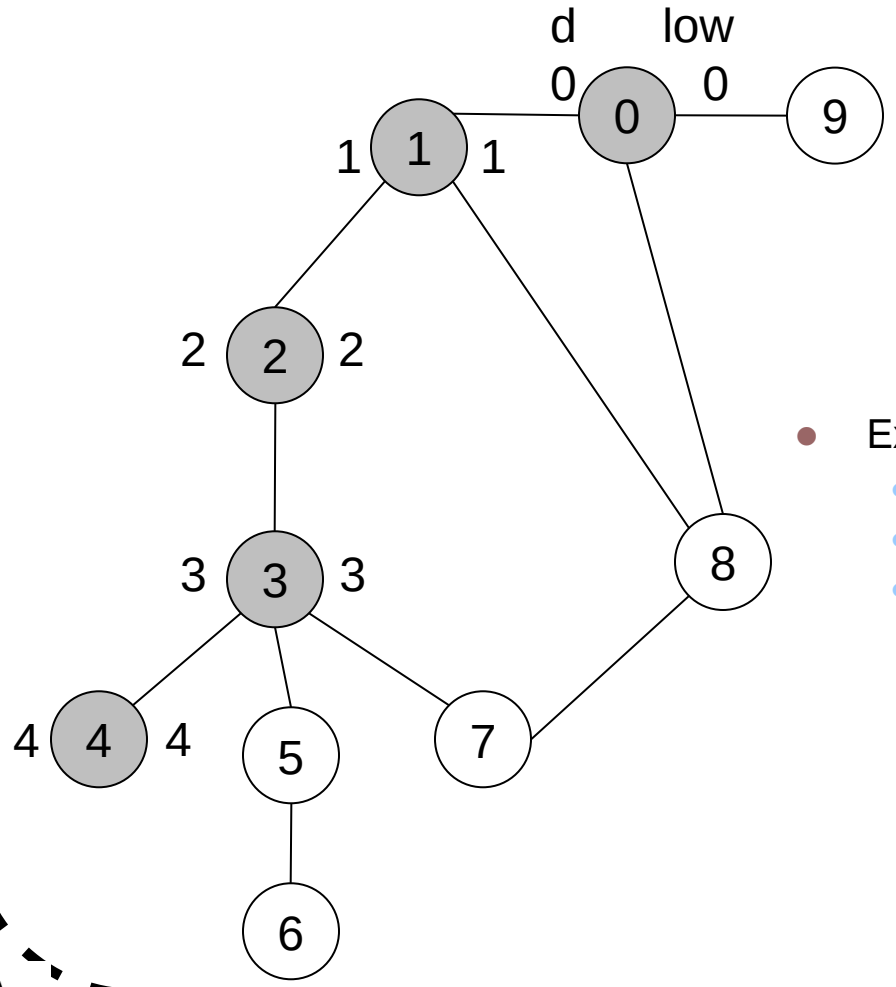
Exemplu rulare (5)



Low(3) = d(3) = 3
Timp = 4
C(3) = gri
P(4) = 3
Subarb(3) = 1
Exploreaza (4)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

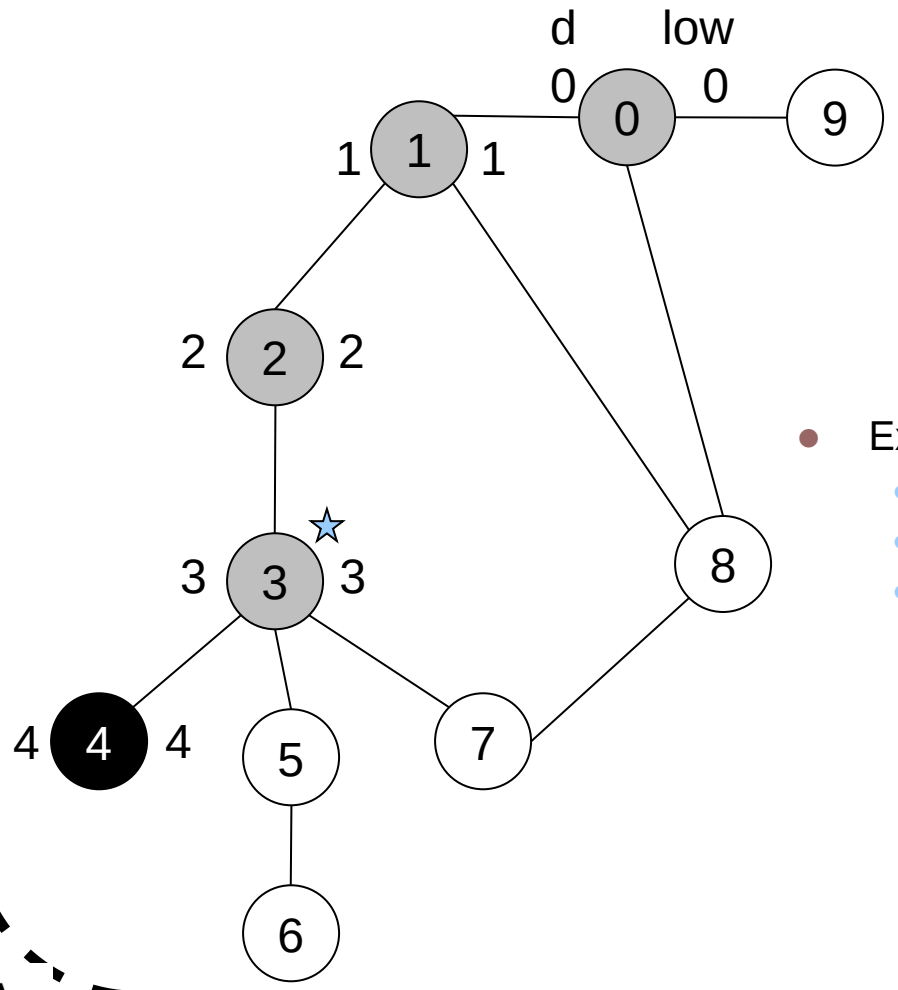
Exemplu rulare (6)



Low(4) = d(4)=4
Timp =5
C(4) =gri
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

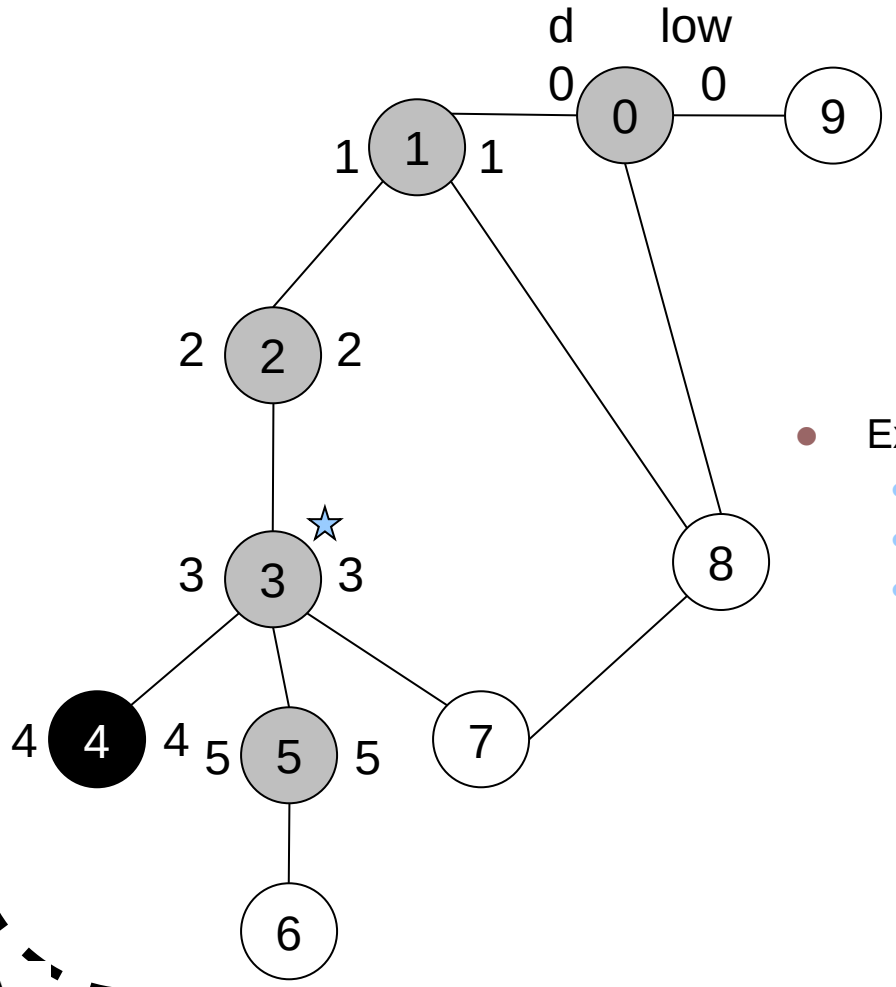
Exemplu rulare (7)



Low(4) = d(4) = 4
Timp = 5
C(4) = gri
revenire
Low (3) = min {low(3), low(4)} = 3
Low(4) > d(3) → art(3) = 1
P(5) = 3
Subarb(3) = 2
Exploreaza (5)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

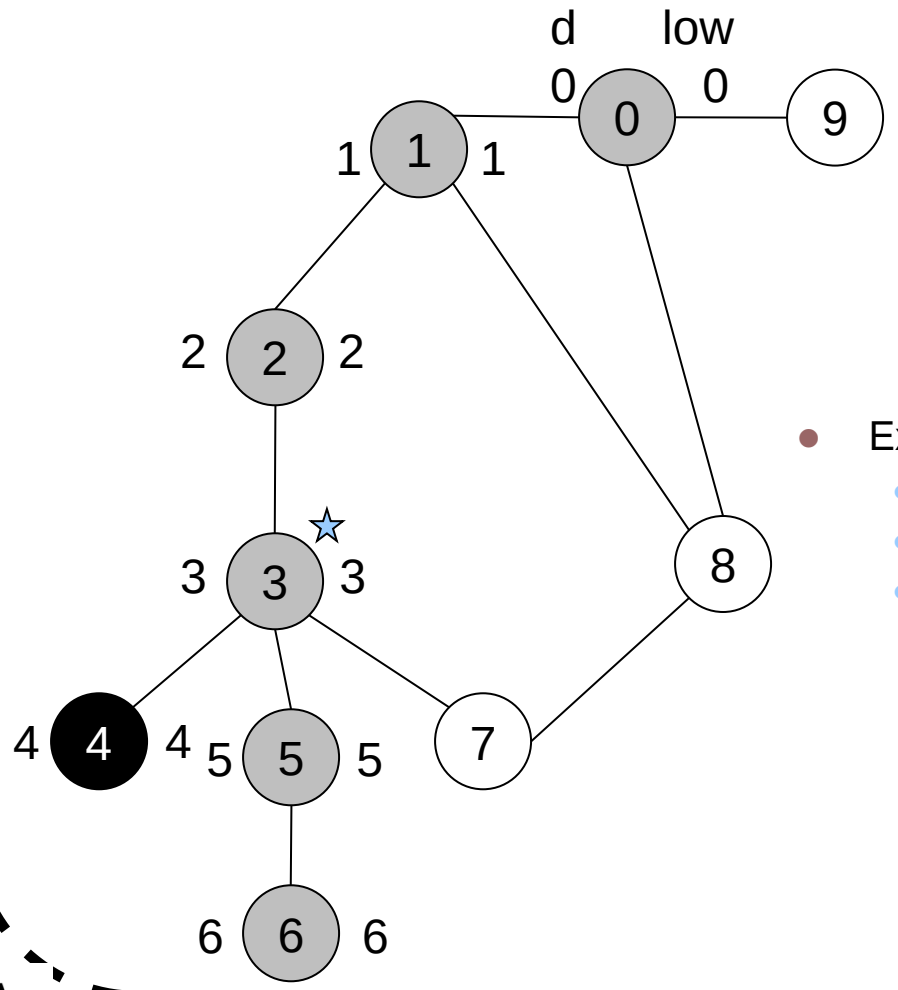
Exemplu rulare (8)



Low(5) = d(5) = 5
Timp = 6
C(5) = gri
P(6) = 5
Subarb(5) = 1
Exploreaza (6)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

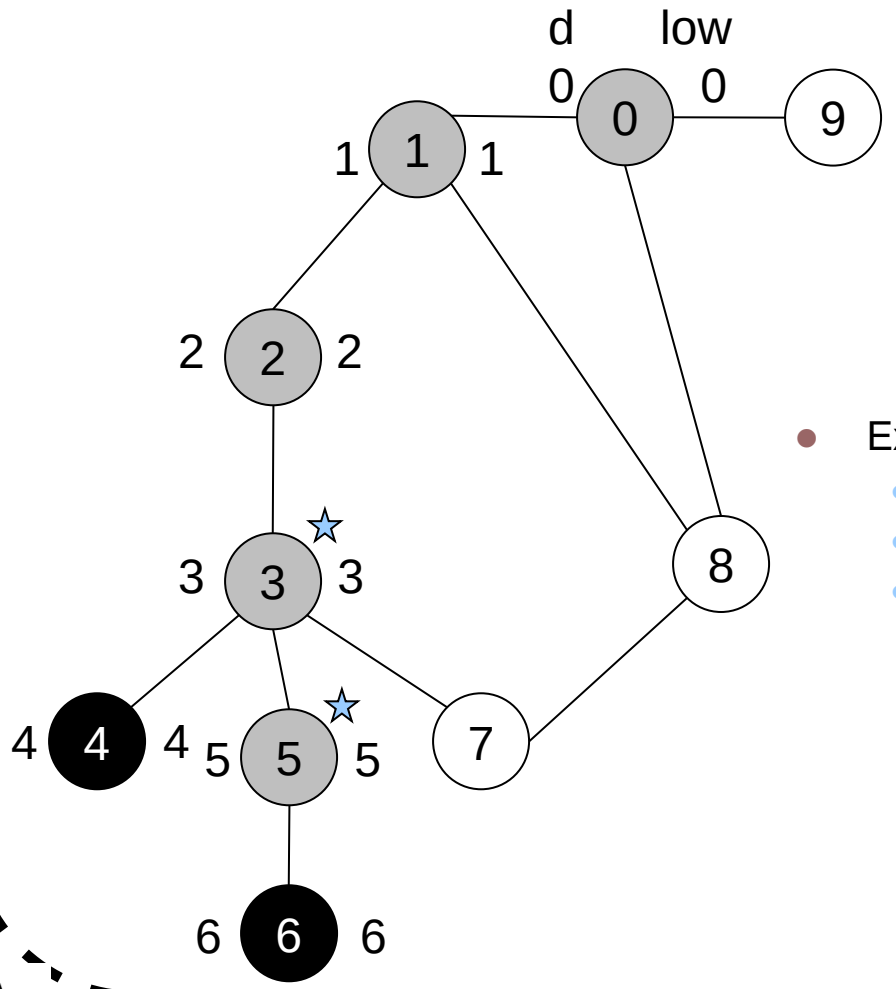
Exemplu rulare (9)



Low(6) = d(6) = 6
Timp = 7
C(6) = gri
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

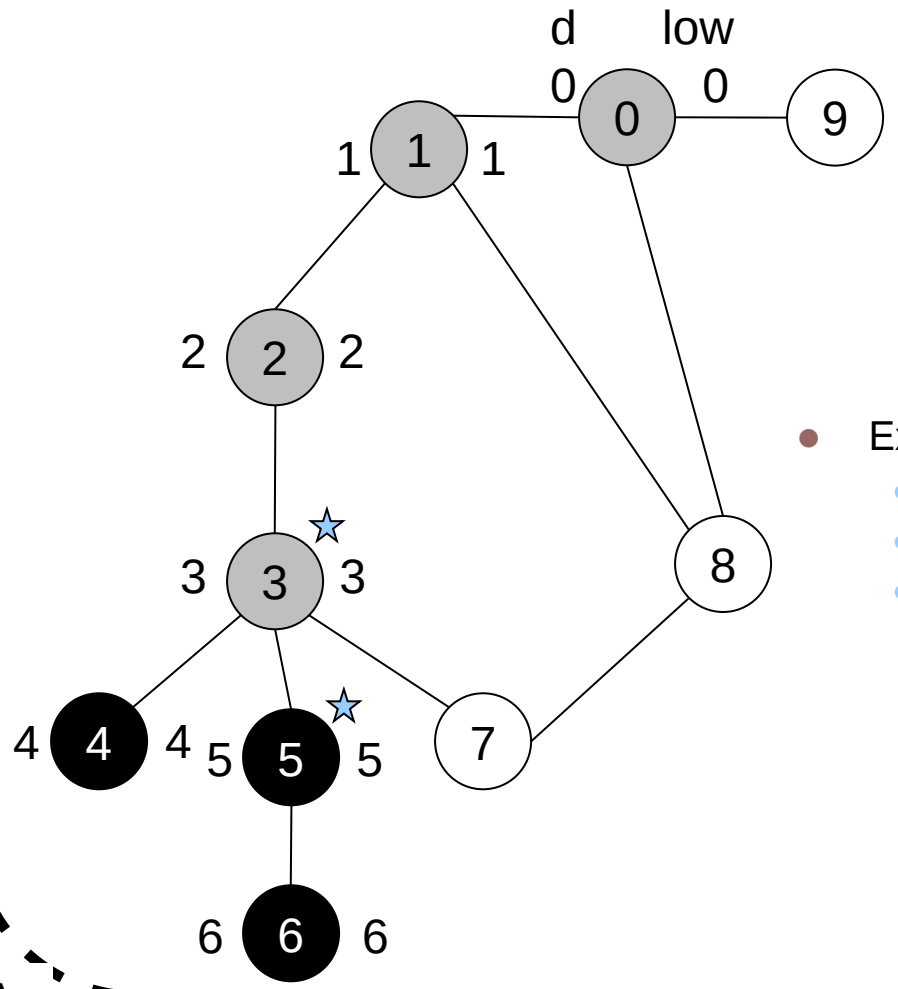
Exemplu rulare (10)



Low(6) = d(6) = 6
Timp = 7
C(6) = gri
revenire
Low (5) = min {low(5), low(6)} = 5
Low(6) > d(5) → art(5) = 1
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

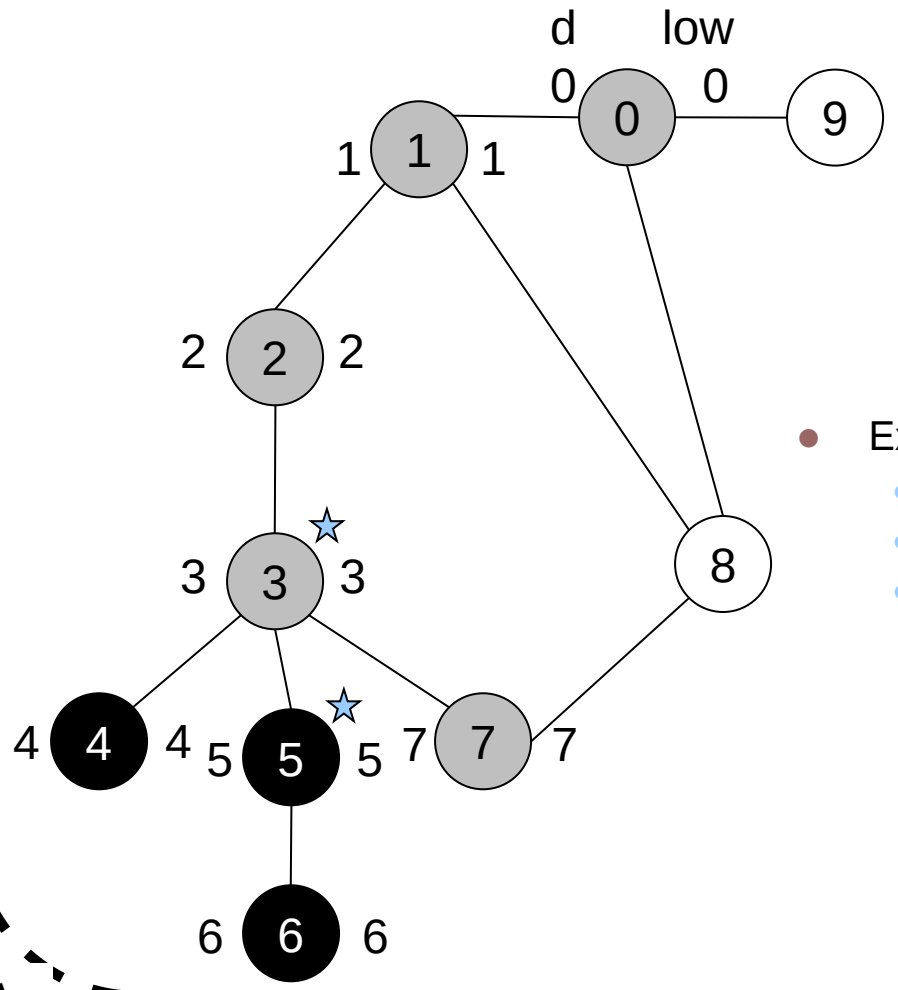
Exemplu rulare (11)



Low(5) = d(5) = 5
Timp = 7
C(5) = gri
revenire
Low (3) = min {low(3), low(5)} = 3
Low(5) > d(3) → art(3) = 1
P(7) = 3
Subarb(3) = 3
Exploreaza (7)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

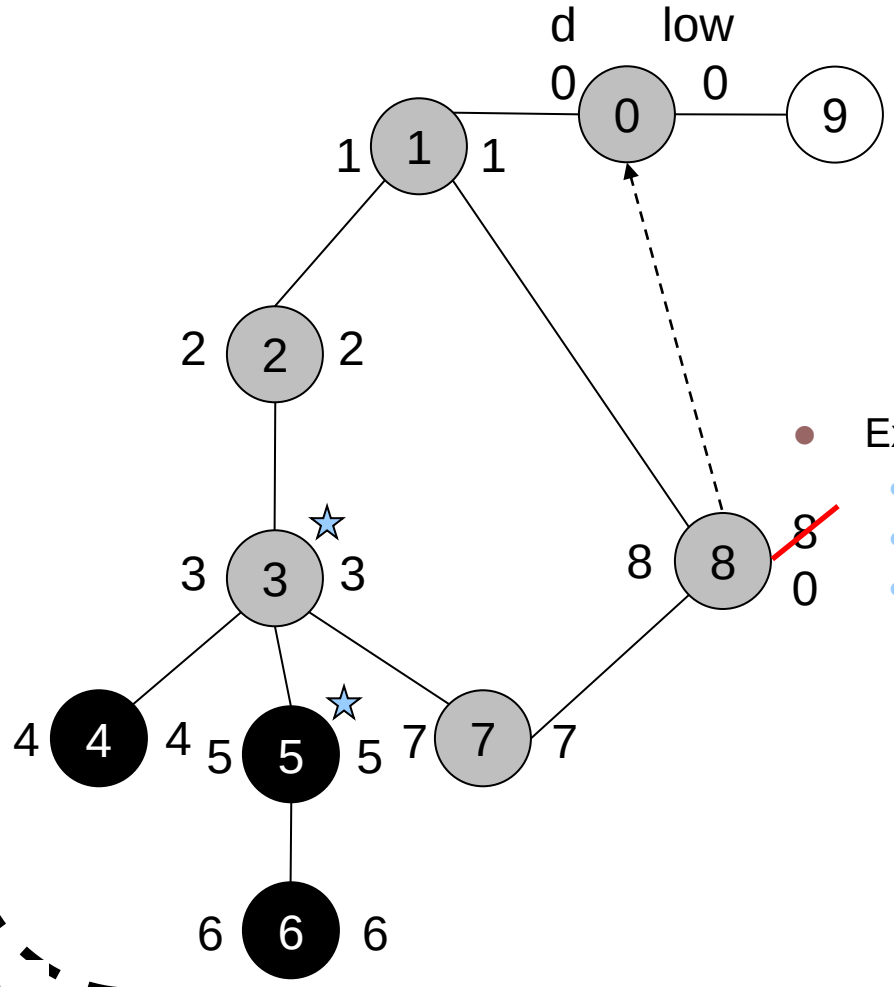
Exemplu rulare (12)



Low(7) = d(7) = 7
Timp = 8
C(7) = gri
P(8) = 7
Subarb(7) = 1
Exploreaza (8)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

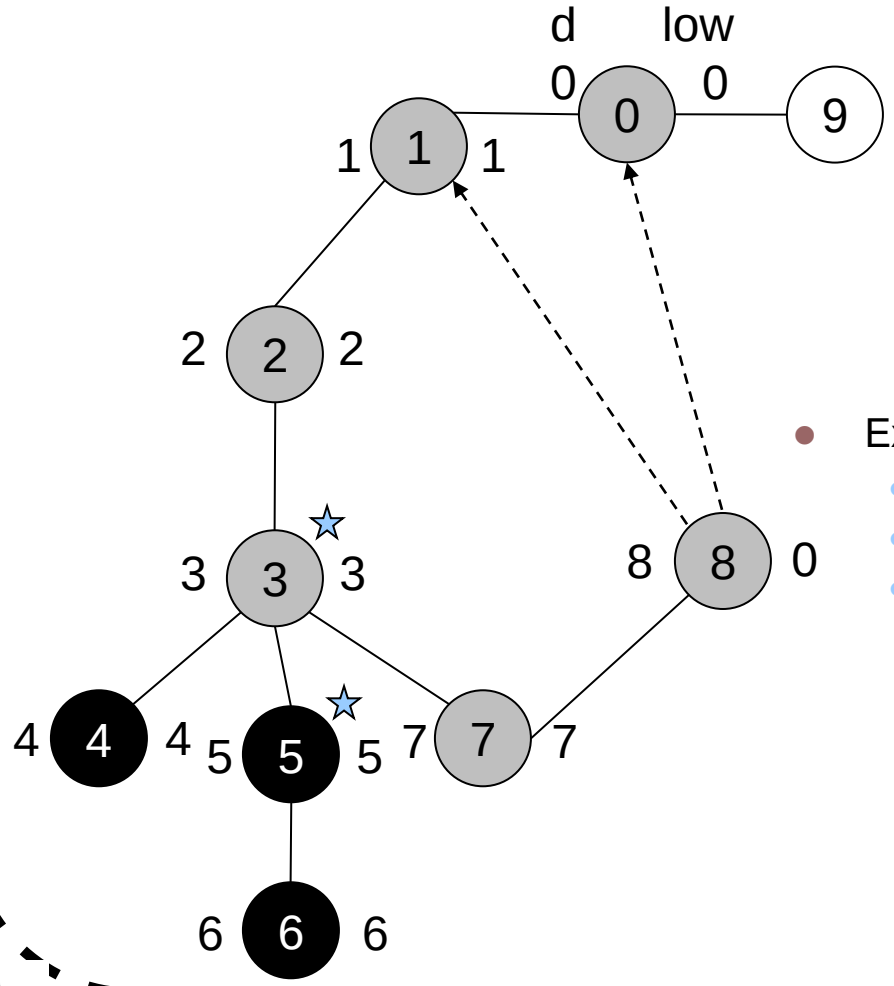
Exemplu rulare (13)



Low(8) = d(8) = 8
Timp = 9
C(8) = gri
Low(8) = min{d(0), low(8)} = 0

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

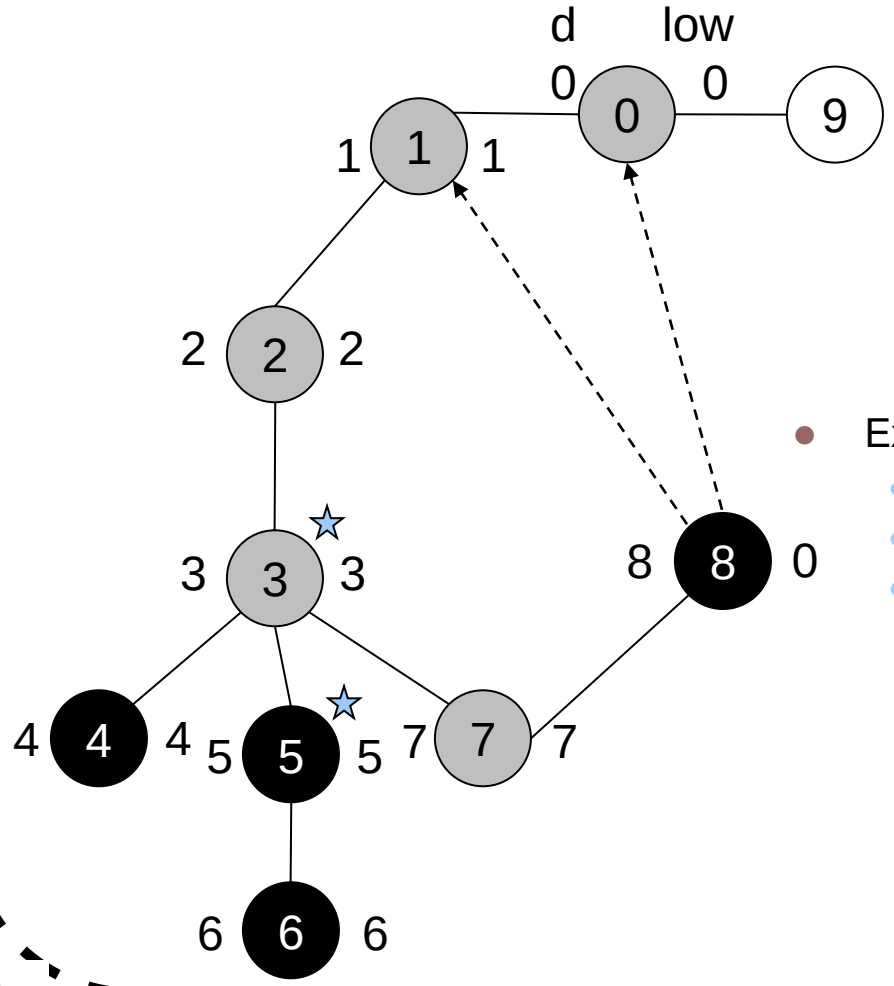
Exemplu rulare (14)



d(8) = 8
Low(8) = 0
Timp = 9
C(8) = gri
Low(8) = min{d(1), low(8)} = 0

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

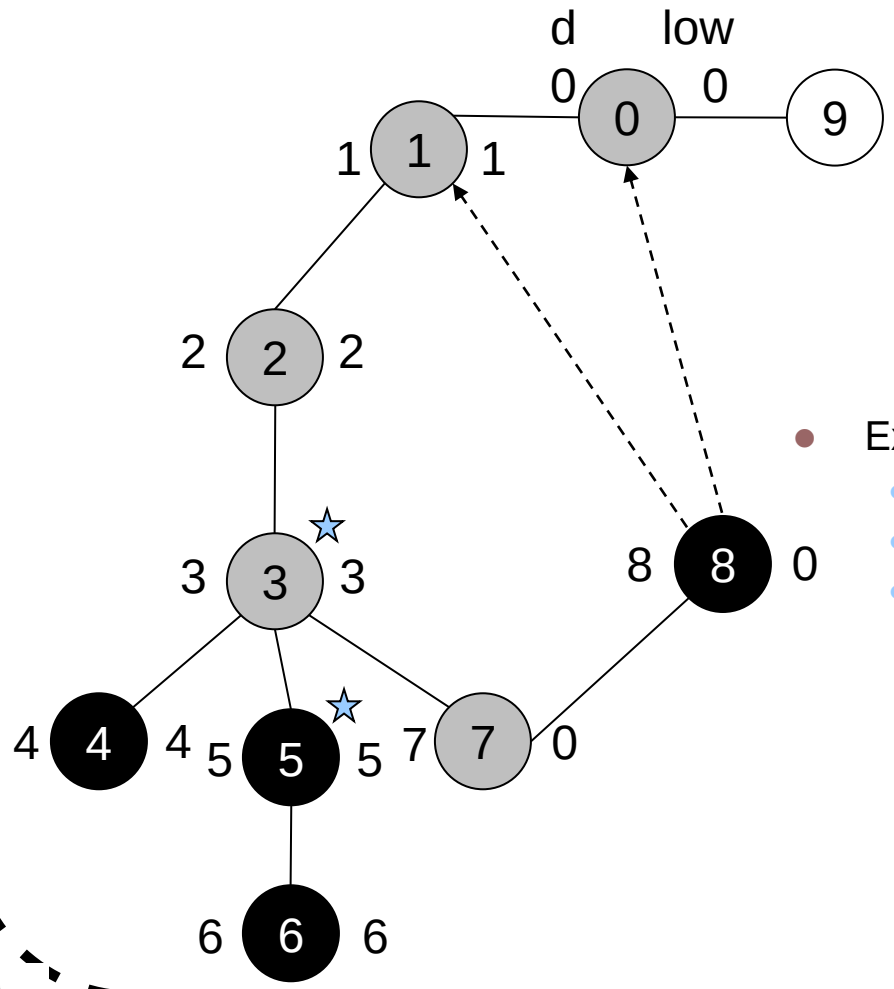
Exemplu rulare (15)



d(8) = 8
Low(8) = 0
Timp = 9
C(8) = gri
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

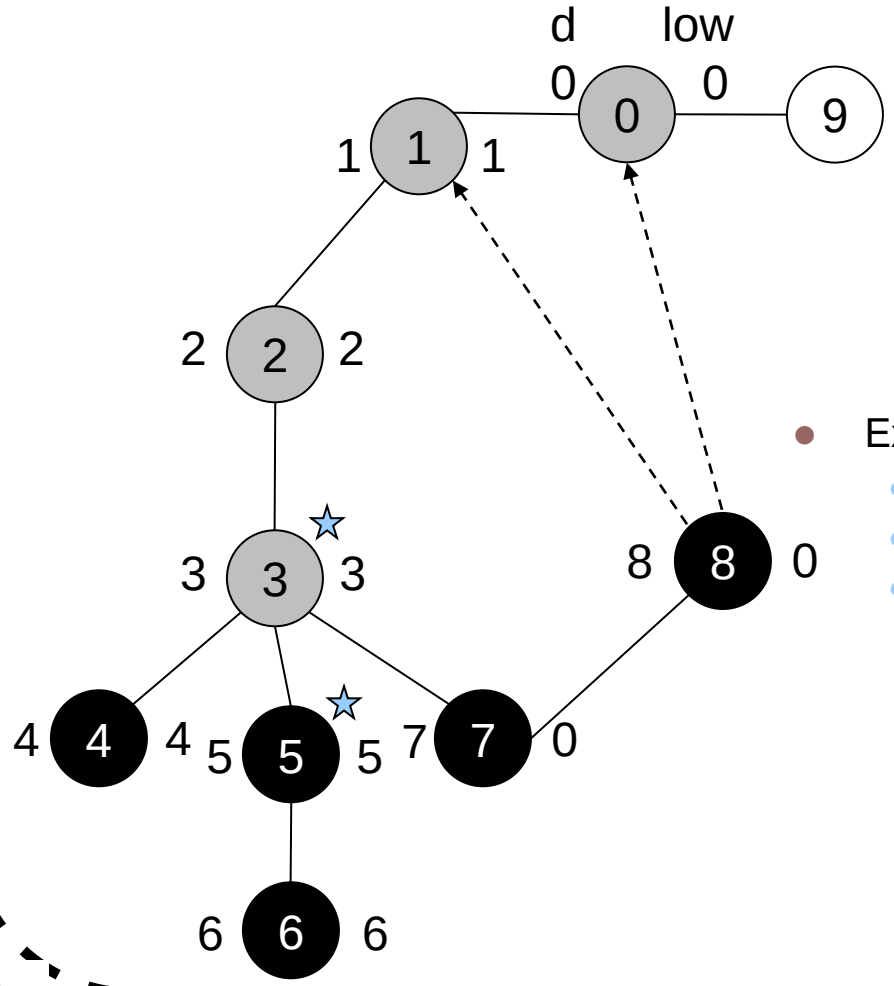
Exemplu rulare (16)



low(7) = min(low(7), low(8)) = 0
low(8) < d(7) → nu se modifică art(7)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

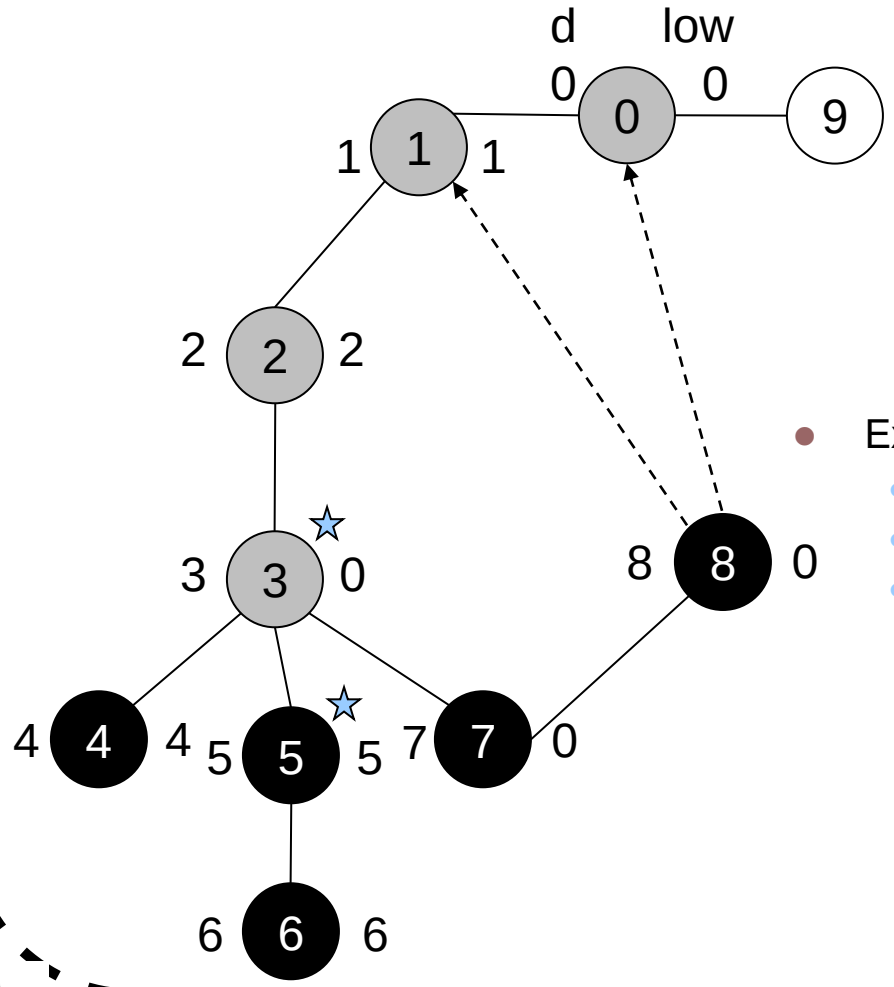
Exemplu rulare (17)



low(7) = min(low(7), low(8)) = 0
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

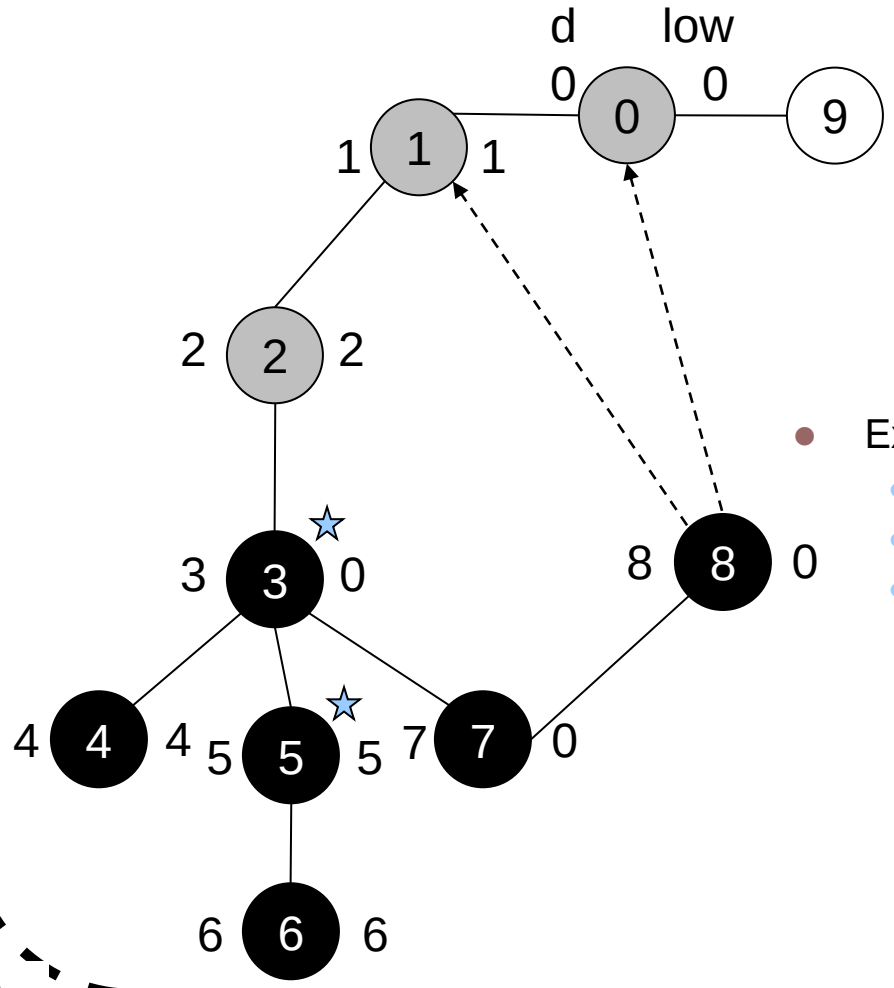
Exemplu rulare (18)



low(3) = min(low(3), low(7)) = 0
low(7) < d(3) → nu se modifică art(3)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

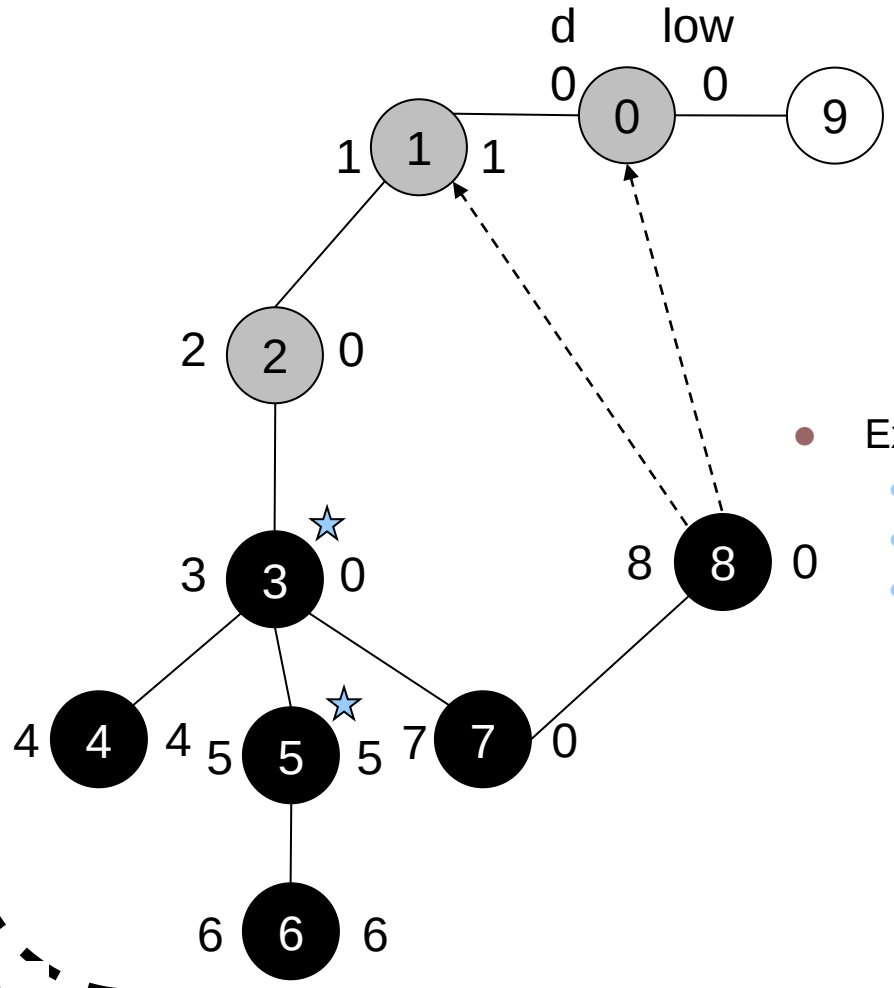
Exemplu rulare (19)



low(3) = min(low(3), low(7)) = 0
low(7) < d(3) → nu se modifică
art(3)
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

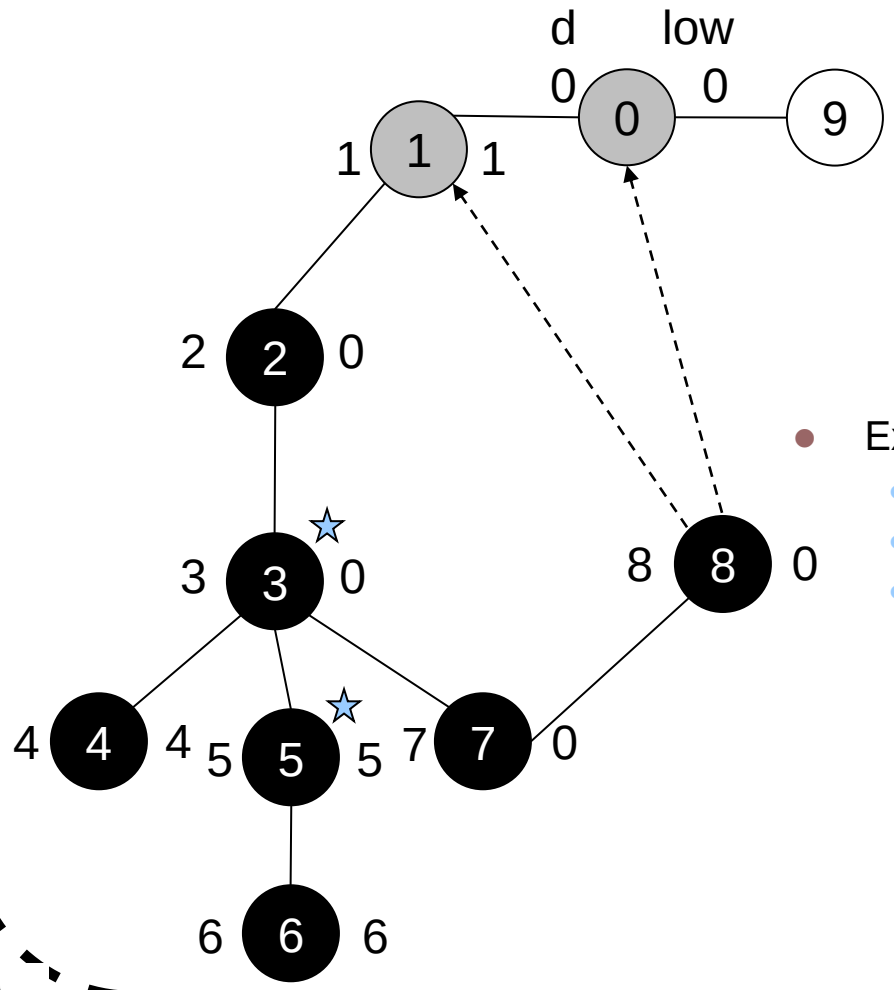
Exemplu rulare (20)



low(2) = min(low(2), low(3)) = 0
low(3) < d(2) → nu se modifică art(2)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

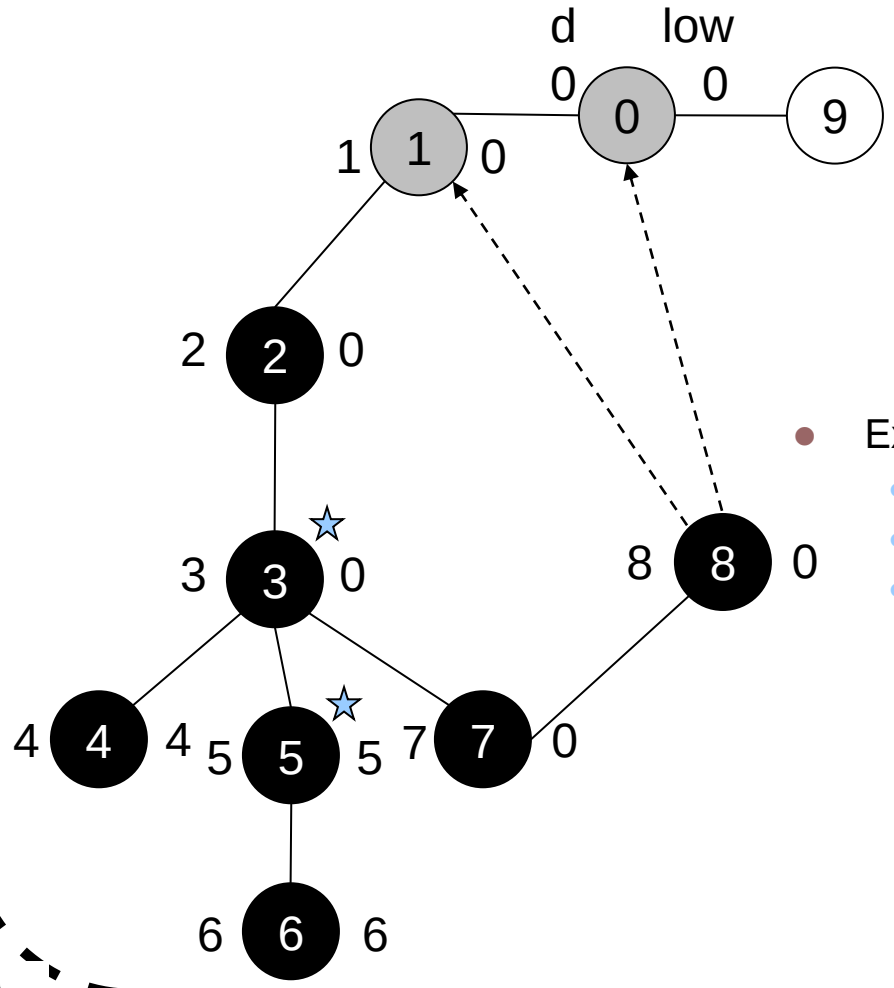
Exemplu rulare (21)



low(2) = min(low(2), low(3)) = 0
low(3) < d(2) → nu se modifică
art(2)
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

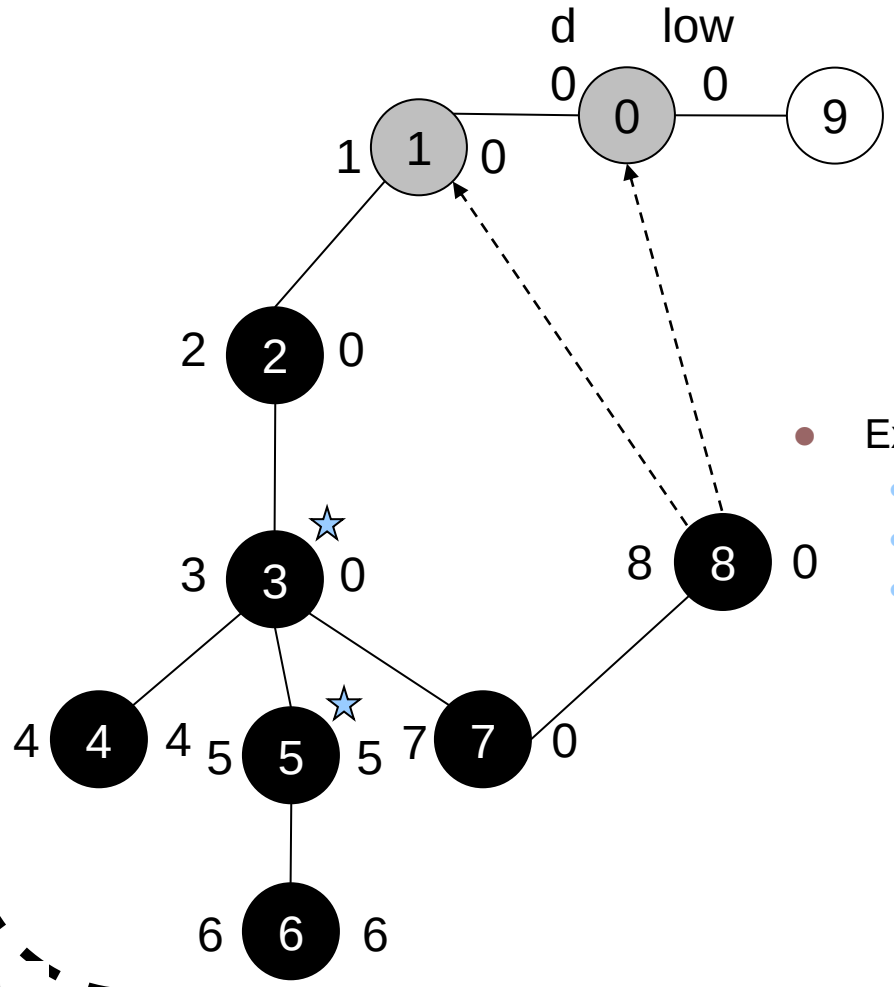
Exemplu rulare (22)



low(1) = min(low(1), low(2)) = 0
low(2) < d(1) → nu se modifică art(1)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

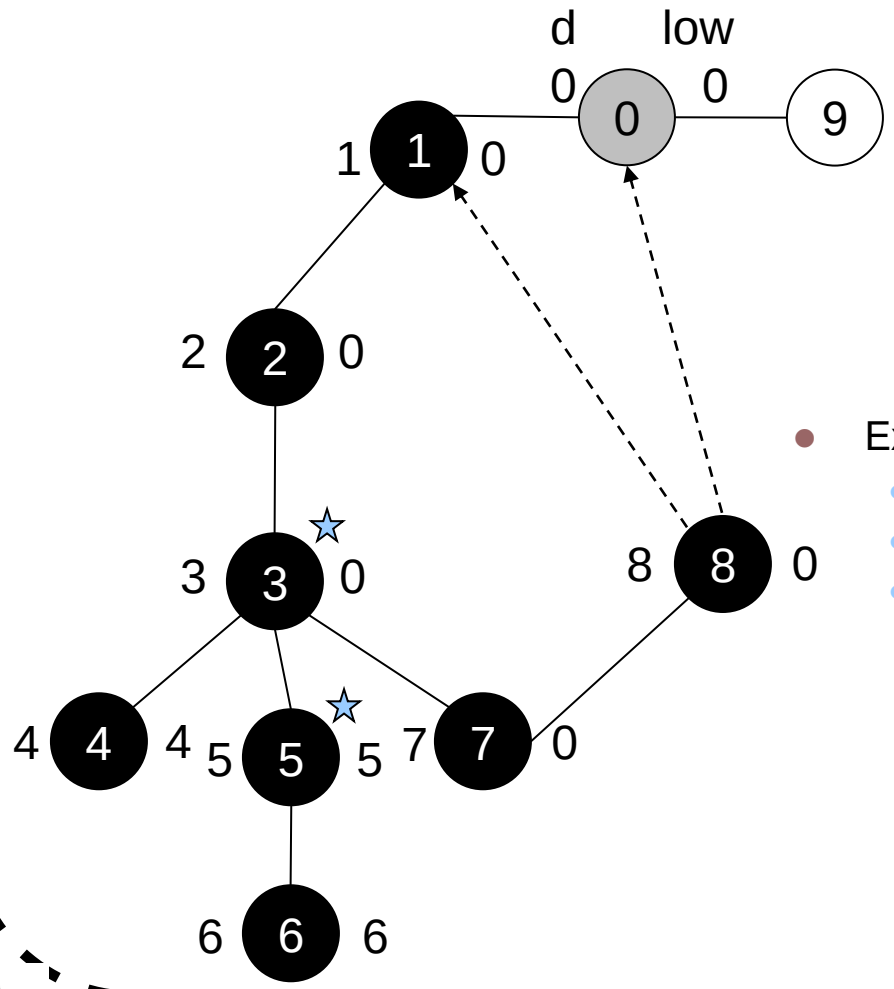
Exemplu rulare (23)



low(1) = 0
low(1) = min(low(1), d(8)) = 0

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

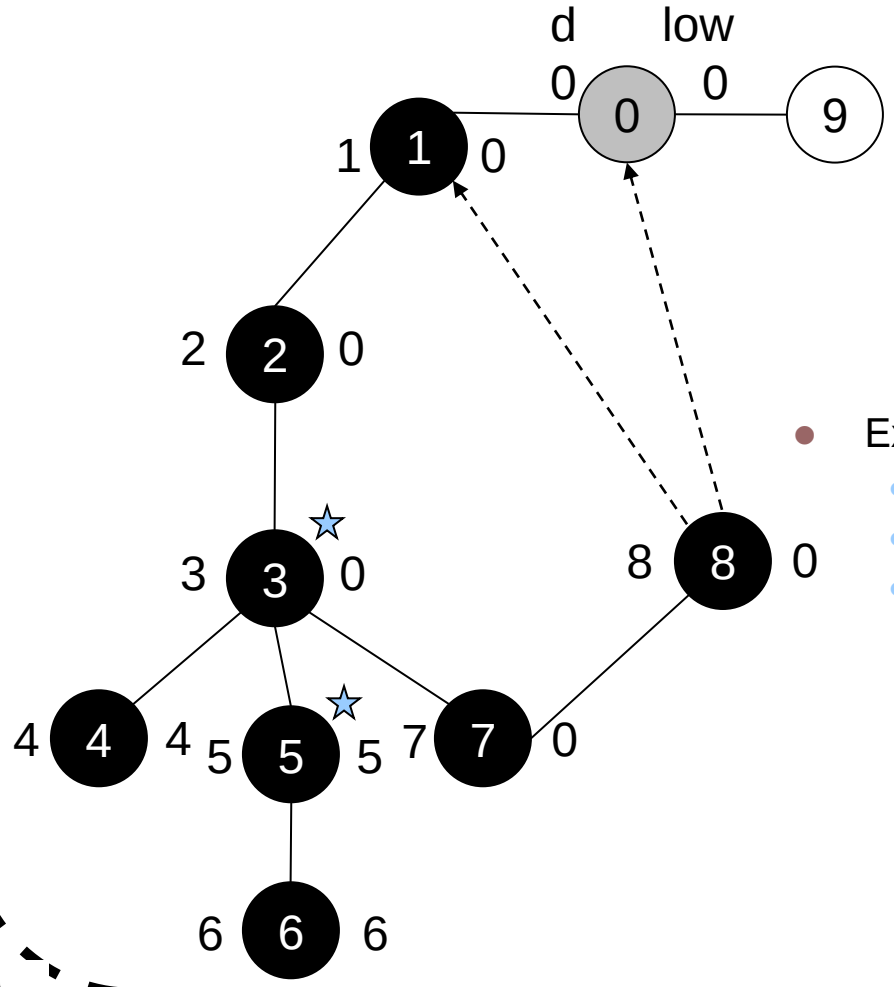
Exemplu rulare (24)



low(1) = 0
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

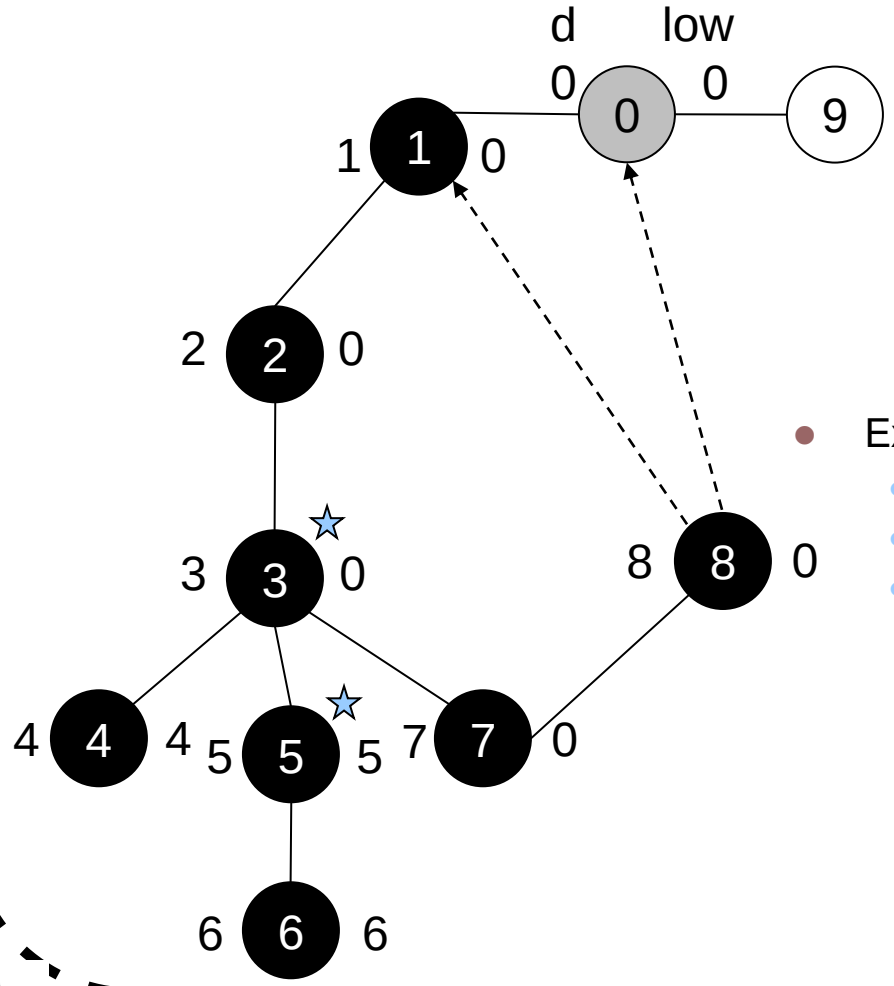
Exemplu rulare (25)



low(0) = min{low(1), low(0)} = 0
P(0) = null → continuă cu următorul copil

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

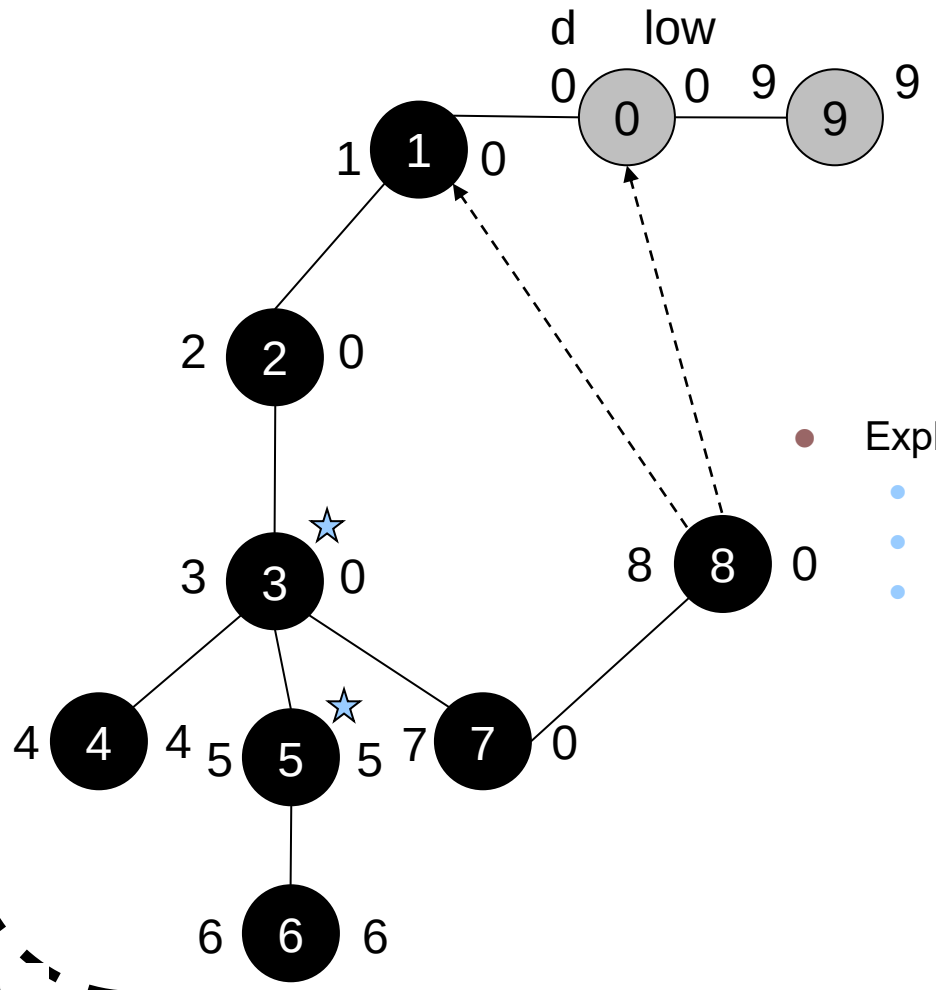
Exemplu rulare (26)



low(0) = min(low(0), d(8)) = 0
P(9) = 0
Subarb(0) = 2
Exploreaza (9)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

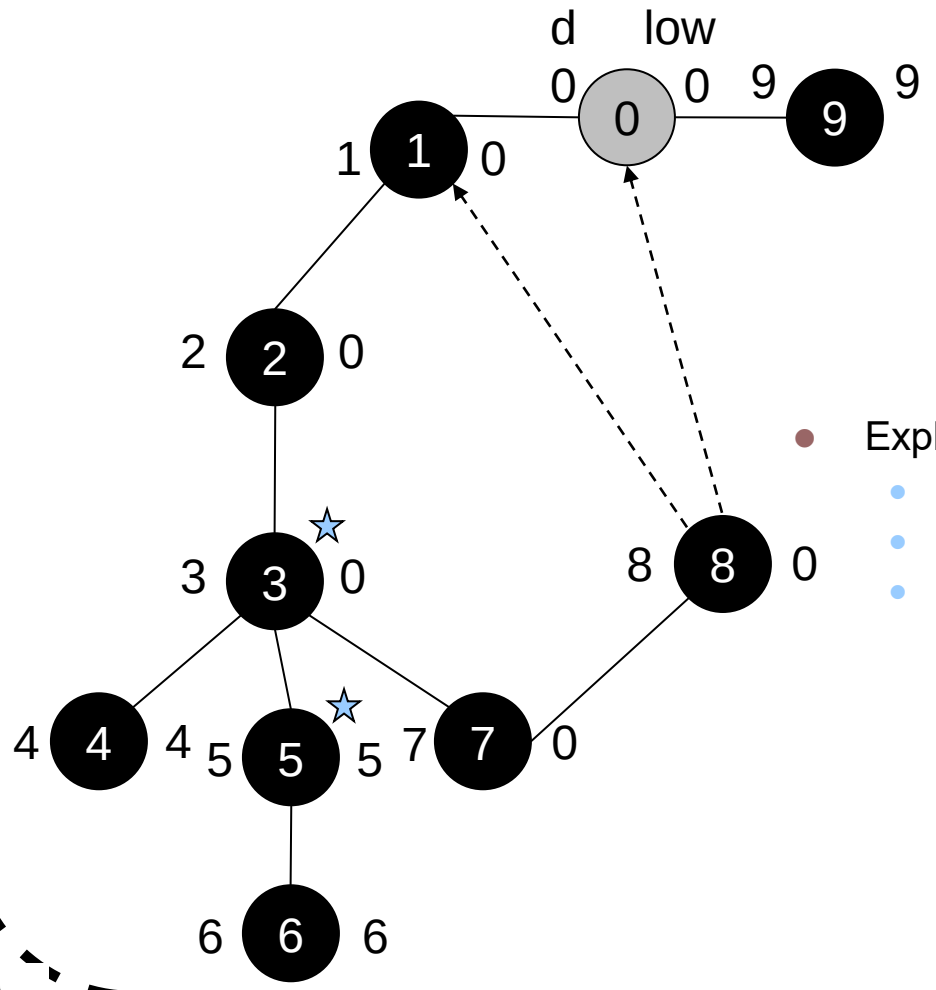
Exemplu rulare (27)



Low(9) = d(9) = 9
Timp = 10
C(9) = gri
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

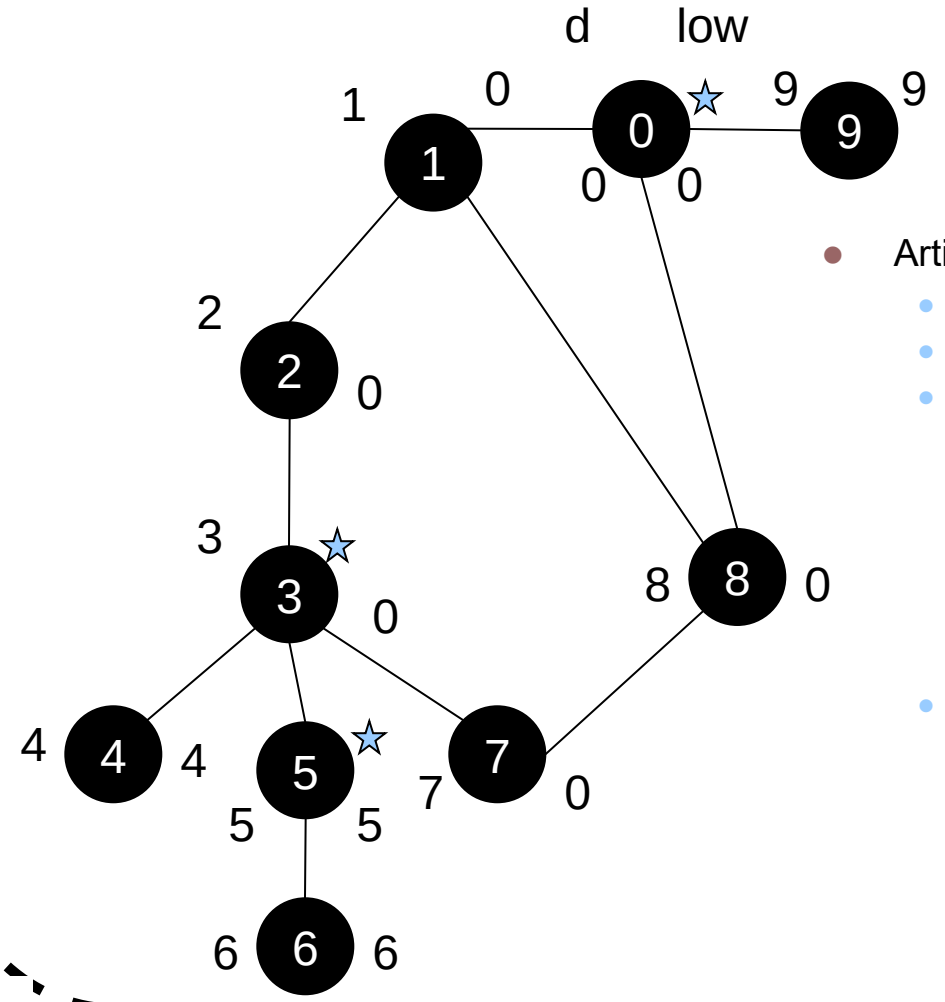
Exemplu rulare (28)



low(0) = min(low(0), low(9)) = 0
P(0) = null → revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

Exemplu rulare (29)



Subarb(0) = 2 > 1 → art(0) = 1

- Articulații (G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - subarb(u) = 0; // reține numărul de subarbori // dominați de u
    - art(u) = 0; // reține punctele de articulație
  - **Pentru fiecare** (u∈V)
    - **Dacă** c(u) e alb
      - Exploreaza(u);
      - **Dacă** (subarb(u) > 1) // cazul în care u este // rădăcina în arborele art(u) = 1 // DFS și are mai mulți subarbori // → cazul 1 al teoremei

Algoritmul lui Tarjan adaptat pentru determinarea CTC

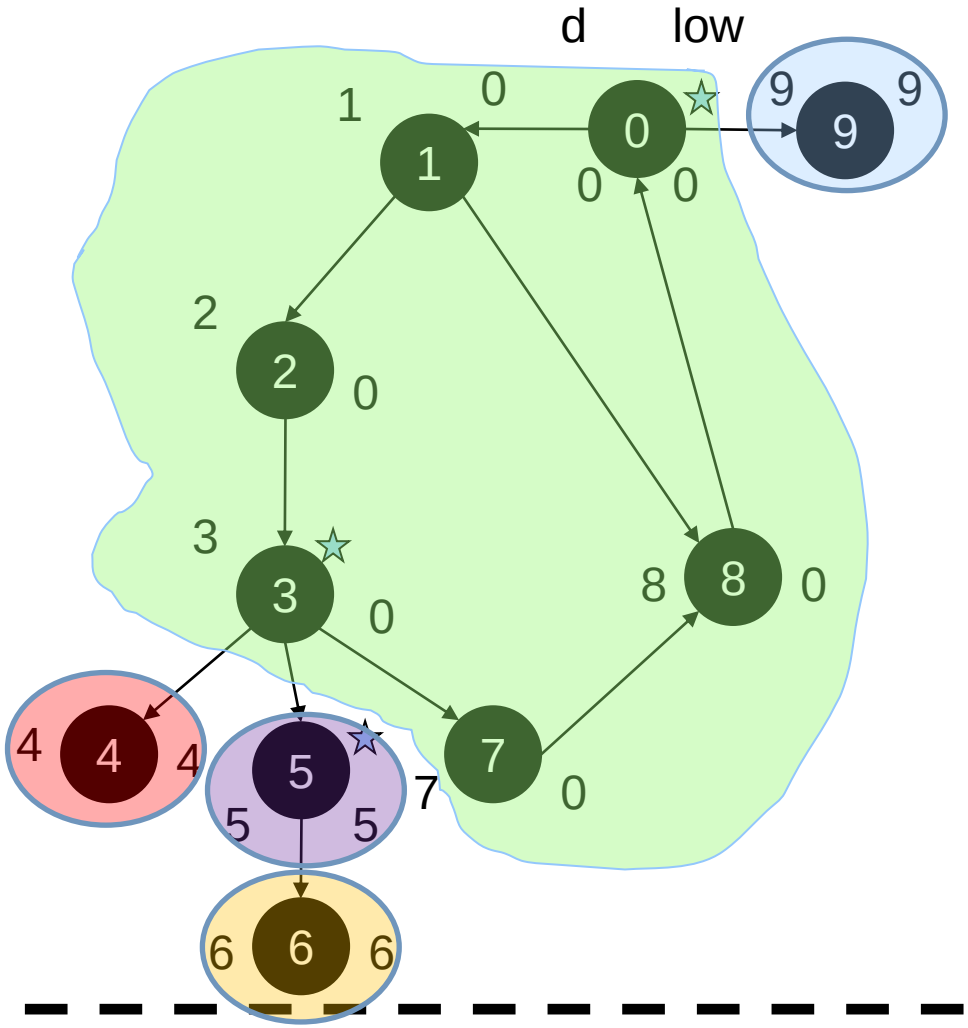
- index = 0 // nivelul pe care este nodul în arborele DFS
- S = empty // se folosește o stivă care se inițializează cu Ø
- **Pentru fiecare** v din V
  - **Dacă** (v.index e nedefinit) **atunci** // se pornește DFS din fiecare nod pe care
    - Tarjan(v) // nu l-am vizitat încă

- Tarjan(v)
  - v.index = index // se setează nivelul nodului v
  - v.lowlink = index // reține strămoșul nodului v
  - index = index + 1 // incrementez nivelul
  - S.push(v) // introduc v în stivă
  - **Pentru fiecare** (v, v') din E // se prelucrează succesorii lui v
    - **Dacă** (v'.index e nedefinit **sau** v' e în S) **atunci** // CTC deja identificate sunt ignorate
      - **Dacă** (v'.index e nedefinit) **atunci** Tarjan(v') // dacă nu a fost vizitat v' intru în recursivitate
      - v.lowlink = min(v.lowlink, v'.lowlink) //actualizez strămoșul
  - **Dacă** (v.lowlink == v.index) **atunci** // printez CTC începând de la coadă spre rădăcină
    - print "CTC:"
    - **Repetă**
      - v' = S.pop // extrag nodul din stiva și îl printez
      - print v'
    - **Până când** (v' == v) // până când extrag rădăcina



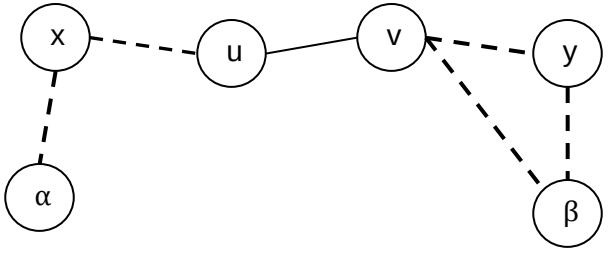


Exemplu rulare (CTC)

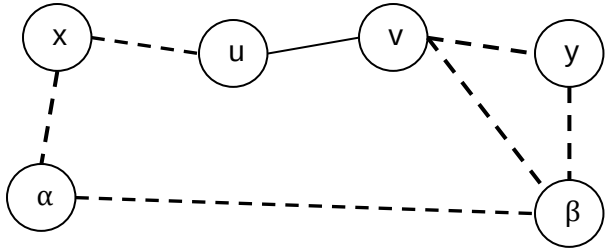


Punți

- Definiție: G = (V,E), graf neorientat și (u,v) ∈ E. (u,v) este punte în G ⇔ ∃ x,y ∈ V, x ≠ y, a.î. ∀ x..y conține muchia (u,v).



Orice drum x..y trece prin (u,v) =>(u,v) este punte



(u,v) nu este punte

Algoritm punți (I)

- Punți(G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** nod u (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - punte(u) =0; // înlocuiește: subarb(u) = 0; art(u) = 0;
  - **Pentru fiecare** nod u (u ∈ V)
    - **Dacă** c(u) e alb
      - Explorează(u)



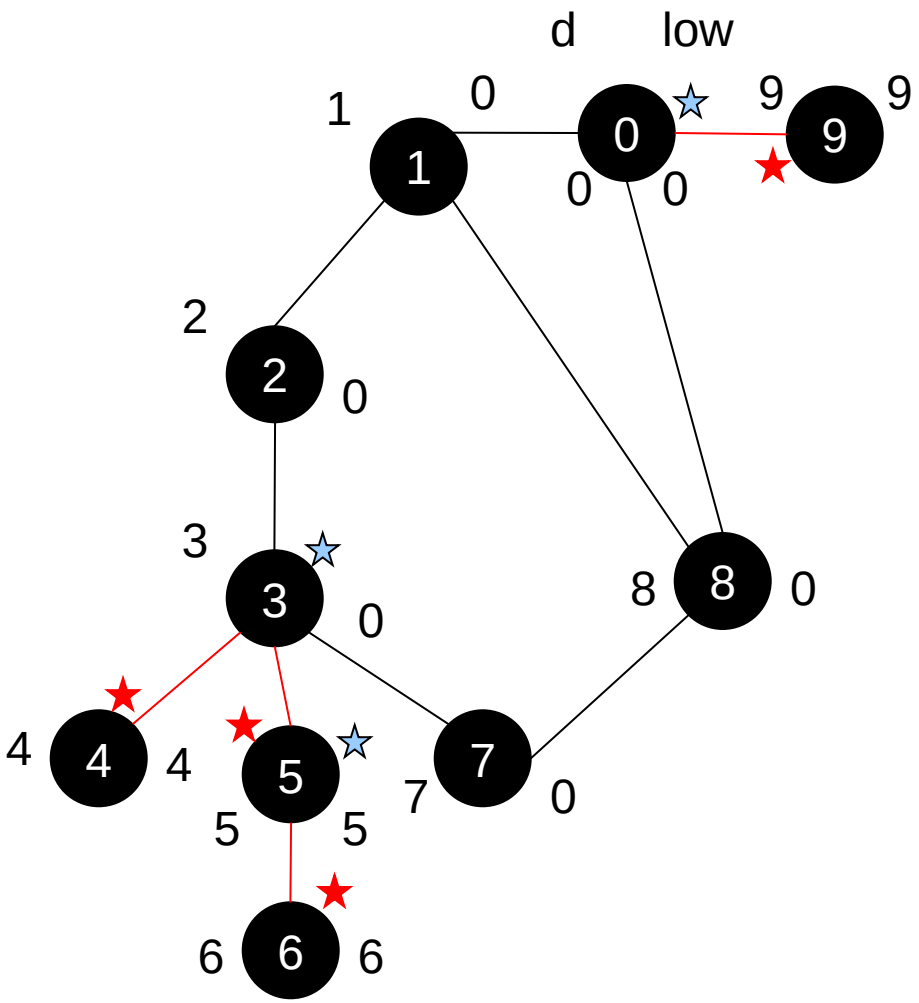


Algoritm punți (II)

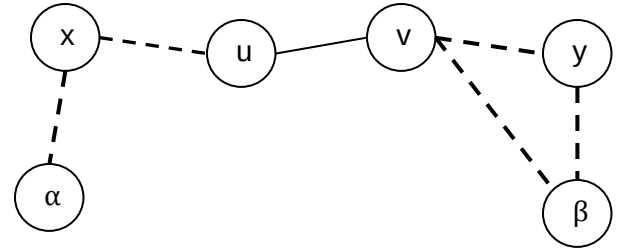
- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v (v ∈ succs(u))
    - **Dacă** c(v) e alb
      - p(v) = u; // se elimină: subarb(u)++;
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (low(v) > d(u)) punte(v) = 1;

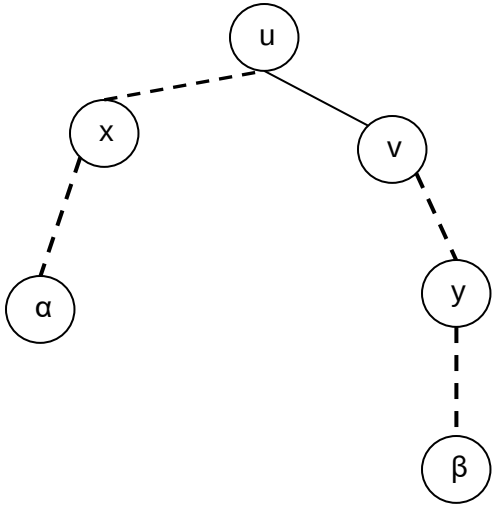
      // în loc de: **Dacă**(p(u) != null && low(v) ≥ d(u))
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

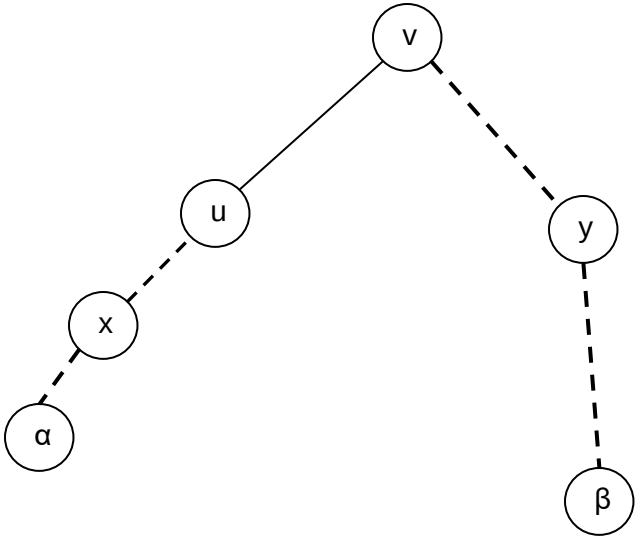
Exemplu rulare (Punți)



Exemplu







DFS din u; puntea este detectată în v

DFS din v; puntea este detectată în u

Drumuri de cost minim

- G = (V,E) un graf, iar w:E → $\Re$ o funcție de cost asociată arcelor grafului (w(u,v) = costul arcului (u,v)).
- Cost(u..v) = costul drumului u..v (este aditiv – costul drumului = suma costurilor arcelor).
- Variante:
  1. Drumuri punct – multipunct: pentru un nod dat s ∈ V, să se găsească un drum de cost minim de la s la ∀u ∈ V; Dijkstra, Bellman-Ford
  2. Drumuri multipunct – punct: pentru un nod dat e ∈ V, să se găsească un drum de cost minim de la ∀u ∈ V la e; G^T și apoi 1
  3. Drumuri punct – punct: pentru două noduri date u și v ∈ V, să se găsească un drum u..v de cost minim; Folosind 1
  4. Drumuri multipunct – multipunct: ∀u, v ∈ V, să se găsească un drum u..v de cost minim. Floyd-Warshall
  5. Drumuri de cost maxim!

Temă de gândire pentru acasă – posibil subiect de examen!

Optimalitatea drumurilor minime (I)

- Lemă 25.1 (Subdrumurile unui drum minim sunt drumuri optimale): G = (V,E), w : E → $\Re$ funcție de cost asociată. Fie p = $v_1v_2...v_k$ un drum optim de la v_1 la v_k. Atunci pentru orice i și j cu $1 \leq i \leq j \leq k$, subdrumul lui p de la v_i la v_j este un drum minim.

- Dem: Fie $p_{ij} = v_i..v_j$ subdrumul din p dintre v_i și v_j. → p = $v_1..v_i..v_j..v_k$ => cost (p) = cost ($v_1..v_i$) + cost ($v_i..v_j$) + cost ($v_j..v_k$).
- Pp. prin absurd că $v_i..v_j$ nu e optim => ∃p' a.î. cost (p') < cost ($v_i..v_j$) => p nu e drum minim → Contrazice ipoteza → p_{ij} este drum minim.

Optimalitatea drumurilor minime (II)

- Corolar 25.2: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. Fie p = s..uv un drum optim de la s la v. Atunci costul optim al acestui drum poate fi scris ca δ(s,v) = δ(s,u) + w(u,v).

- Dem: Conform teoremei anterioare, s..u e un drum optim => cost (s..u) = δ(s,u).

- Lemă 25.3: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. ∀ (u,v) ∈ E avem δ(s,v) ≤ δ(s,u) + w(u,v).

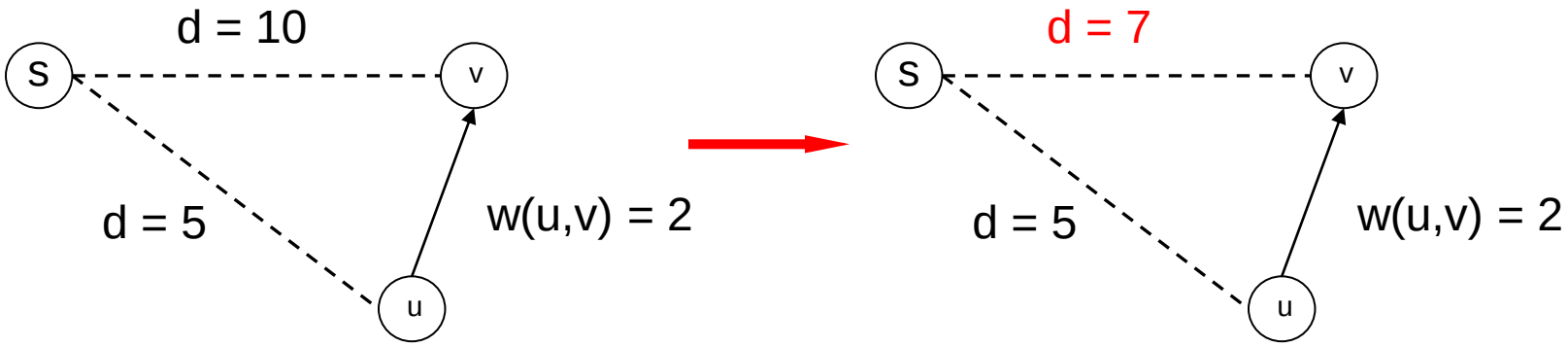
- Dem: Orice drum optim are costul mai mic ca al oricărui alt drum.

Drumuri minime de sursă unică

- Sunt concepuți pentru grafuri orientate.
- Bazați pe algoritmi Greedy.
- Se pornește de la nodul de start și pe baza unui optim local, drumurile sunt extinse și optimizate până la soluția finală.
- Notații:
  - s = nodul sursă
  - d[v] = costul drumului descoperit s..v;
  - δ(u,v) = costul drumului optim u..v; δ(u,v)=∞ dacă v $\notin$ R(u);
  - p(v) = predecesorul lui v pe drumul s..v.

Drumuri minime de sursă unică

- Relaxarea arcelor → dacă d[v] > d[u] + w(u,v), atunci actualizează d[v].



- Exemple: Dijkstra și Bellman–Ford.

Algoritmul lui Dijkstra (I)

- Folosește o coadă de priorități în care se adaugă nodurile în funcție de distanța cunoscută în momentul respectiv de la s până la nod.
- Se folosește **NUMAI** pentru costuri pozitive ($w(u,v) > 0, \forall u,v \in V$).
- Dijkstra_generic (G,s)
  - V = nodurile lui G
  - **Cât timp** (V != ∅)
    - u = nod din V cu d[u] min
    - V = V - {u}
    - **Pentru fiecare** (v ∈ succesorii lui u) relaxare_arc(u,v)

    // optimizare drum s..v pentru v ∈ succesorilor lui u

Relaxarea arcelor (I)

- Lemă 25.5: G = (V,E), w : E → $\Re$ funcție de cost asociată. ∀ v ∈ V, d[v] obținut de algoritmul lui Dijkstra respectă d[v] ≥ δ(s,v). În plus, odată atinsă valoarea δ(s,v), ea nu se mai modifică.

- Dem:
  - ∀ v ∈ V, v ∉ R(s) → d[v] = δ(s,v) = ∞; d[s] = δ(s,s) = 0 (inițializare)
  - Pt v ∈ R(s), inițializare → d[v] = ∞ ≥ δ(s,v). Dem. prin reducere la absurd că după oricâte relaxări, relația se menține. Fie v primul vârf pentru care relaxarea (u,v) determină d[v] < δ(s,v) → după relaxarea (u,v): d[u] + w(u,v) = d[v] < δ(s,v) ≤ δ(s,u) + w(u,v) → d[u] < δ(s,u). Dar relaxarea nu modifică d[u], iar v e primul pentru care d[v] < δ(s,v). Contrazice presupunerea! => d[v] ≥ δ(s,v), ∀ v ∈ V
  - Cum d[v] ≥ δ(s,v) => odată ajuns la d[v] = δ(s,v), ea nu mai scade. Cum relaxarea nu creste valorile => d[v] nu se mai modifică.

Relaxarea arcelor (II)

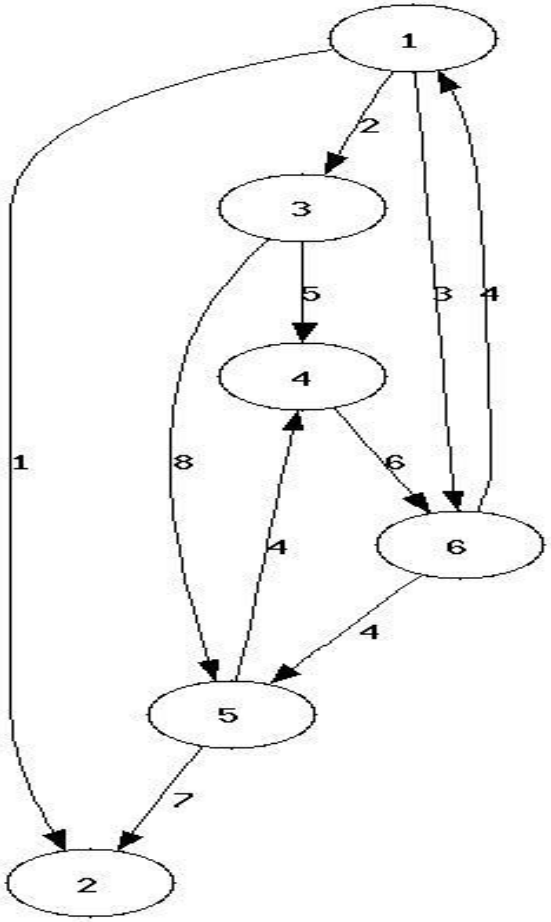
- Lemă 25.7: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. Fie p = s..uv un drum optim de la s la v. Dacă d[u] = δ(s,u) la un moment dat, atunci începând cu momentul imediat următor relaxării arcului (u,v) avem d[v] = δ(s,v)

- Dem:
  - Dacă înainte de relaxare d[v] > d[u] + w(u,v), prin relaxare → d[v] = d[u] + w(u,v). Altfel, d[v] ≤ d[u] + w(u,v) => după relaxare avem d[v] ≤ d[u] + w(u,v).
  - Cum d[u] = δ(s,u) și relaxarea (u,v) nu modifică d[u] => d[v] ≤ d[u] + w(u,v) = δ(s,u) + w(u,v) = δ(s,v) (conf. Corolar 25.2) → d[v] = δ(s,v)

Algoritmul lui Dijkstra (II)

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V) // inițializări
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele

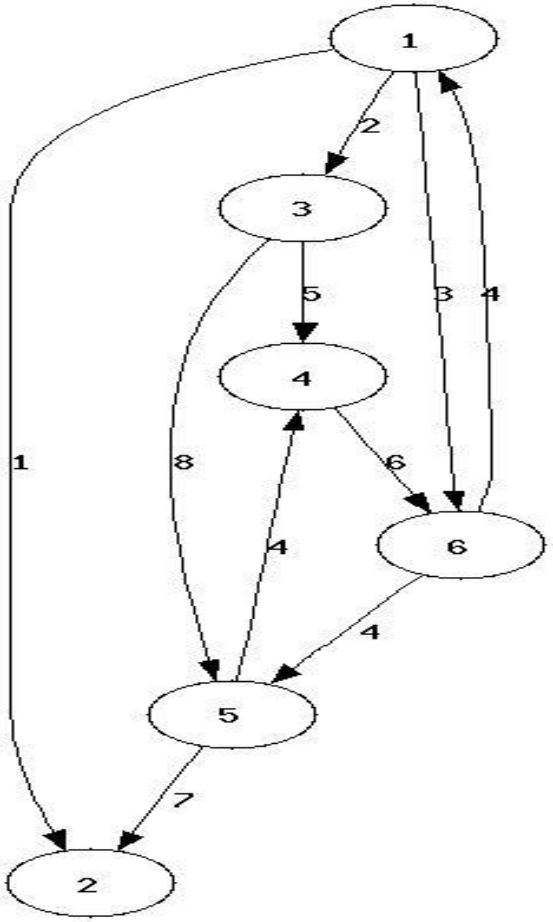
Exemplu (I)



Exemplu (II)

- d[1] = 0;
- (1): d[2] = 1; d[3] = 2; d[6] = 3;
- (2): d[4] = 7; d[5] = 10;
- (3): d[5] = 7;

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] // minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele



Complexitate Dijkstra

- Depinde de ExtrageMin – coadă cu priorități.
- Operații ce trebuie realizate pe coadă + frecvența lor:
  - insert – V;
  - delete – V;
  - conține? – E;
  - micșorează_val – E;
  - este_vidă? – V.

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută in cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q si v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // si părintele

Implementare cu vectori

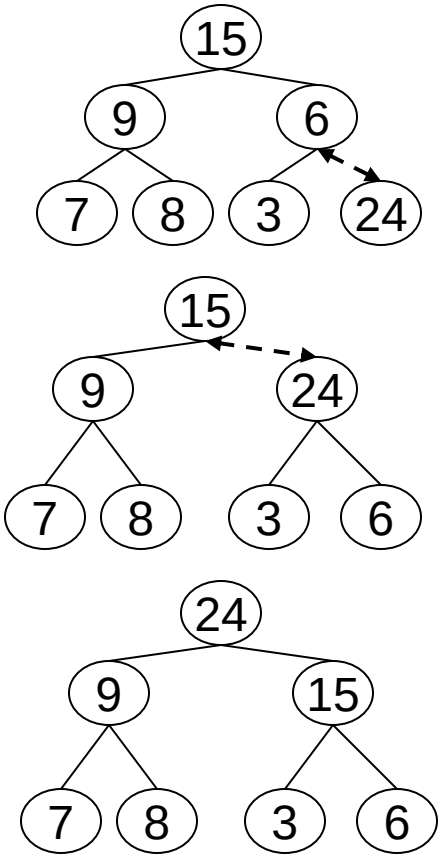
- Costuri:
  - insert – 1 * V = V;
  - delete – V * V = V^2 (necesită căutarea minimului);
  - conține? – 1 * E = E;
  - micșorează_val – 1 * E = E;
  - este_vidă? – 1 * V = V;
- Cea mai bună metodă pentru grafuri "dese" ($E \approx V^2$)!

Implementare cu heap binar

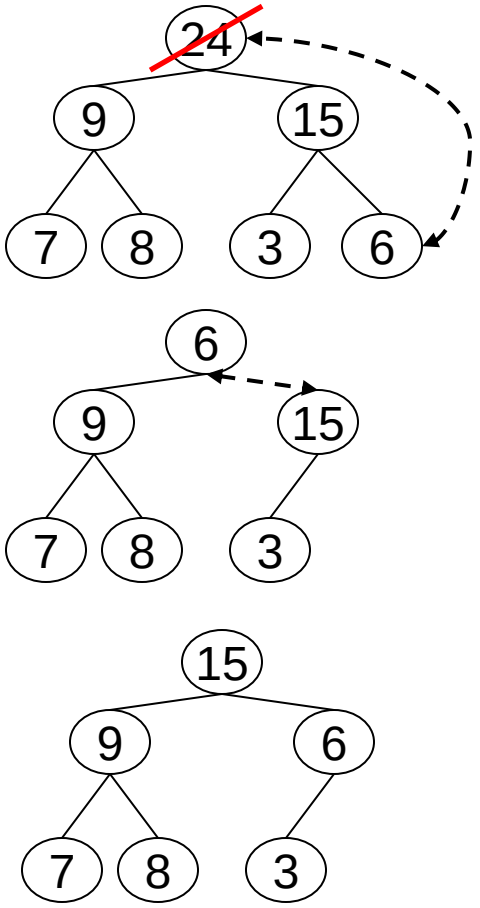
- Heap binar – structură de date de tip arbore binar + 2 constrângeri:
  - Fiecare nivel este complet; ultimul se umple de la stânga la dreapta;
  - $\forall u \in Heap$; $u \geq$ răd(st(u)) și $u \geq$ răd(dr(u)) (u este $\geq$ decât ambii copii ai săi) unde $\geq$ este o relație de ordine pe mulțimea pe care sunt definite elementele heapului.

Operatii pe Heap Binar

insert



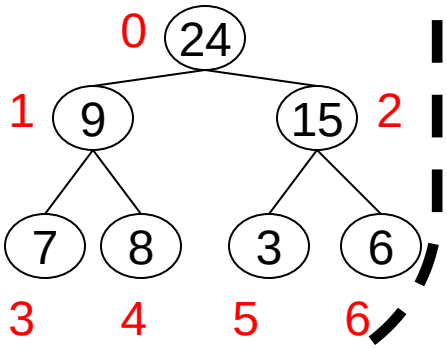
delete



Implementare Heap Binar

- Implementare folosind vectori.
- Poziție[i] = unde se găsește în indexul de valori elementul de pe poziția i din heap.
- Reverse[i] = unde se găsește în heap elementul de pe poziția i din valoare.
- Implementare disponibila la [3].

Index	0	1	2	3	4	5	6
Valoare	7	6	15	8	24	9	3
Poziție							
Reverse							



Heap Binar

- Costuri:
  - insert – logV * V = VlogV;
  - delete – logV * V = VlogV;
  - conține? – 1 * E = E;
  - micșorează_val – logV * E = ElogV;
  - este_vidă? – 1 * V = V.

- Eficient dacă graful are arce puține comparativ cu numărul de noduri.



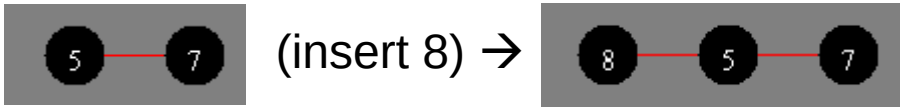
Heap Fibonacci

- Poate fi format din mai mulți arbori.
- Cheia unui părinte ≤ cheia oricărui copil.
- Fiind dat un nod u și un heap H:
  - p(u) – părintele lui u;
  - copil(u) – legătura către unul din copiii lui u;
  - st(u), dr(u) – legătura la frații din stânga și din dreapta (cei de pe primul nivel sunt legați între ei astfel);
  - grad(u) – numărul de copii ai lui u;
  - min(H) – cel mai mic nod din H;
  - n(H) – numărul de noduri din H.



Operatii Heap Fibonacci

- Inserare nod – O(1)
  - construiește un nou arbore cu un singur nod



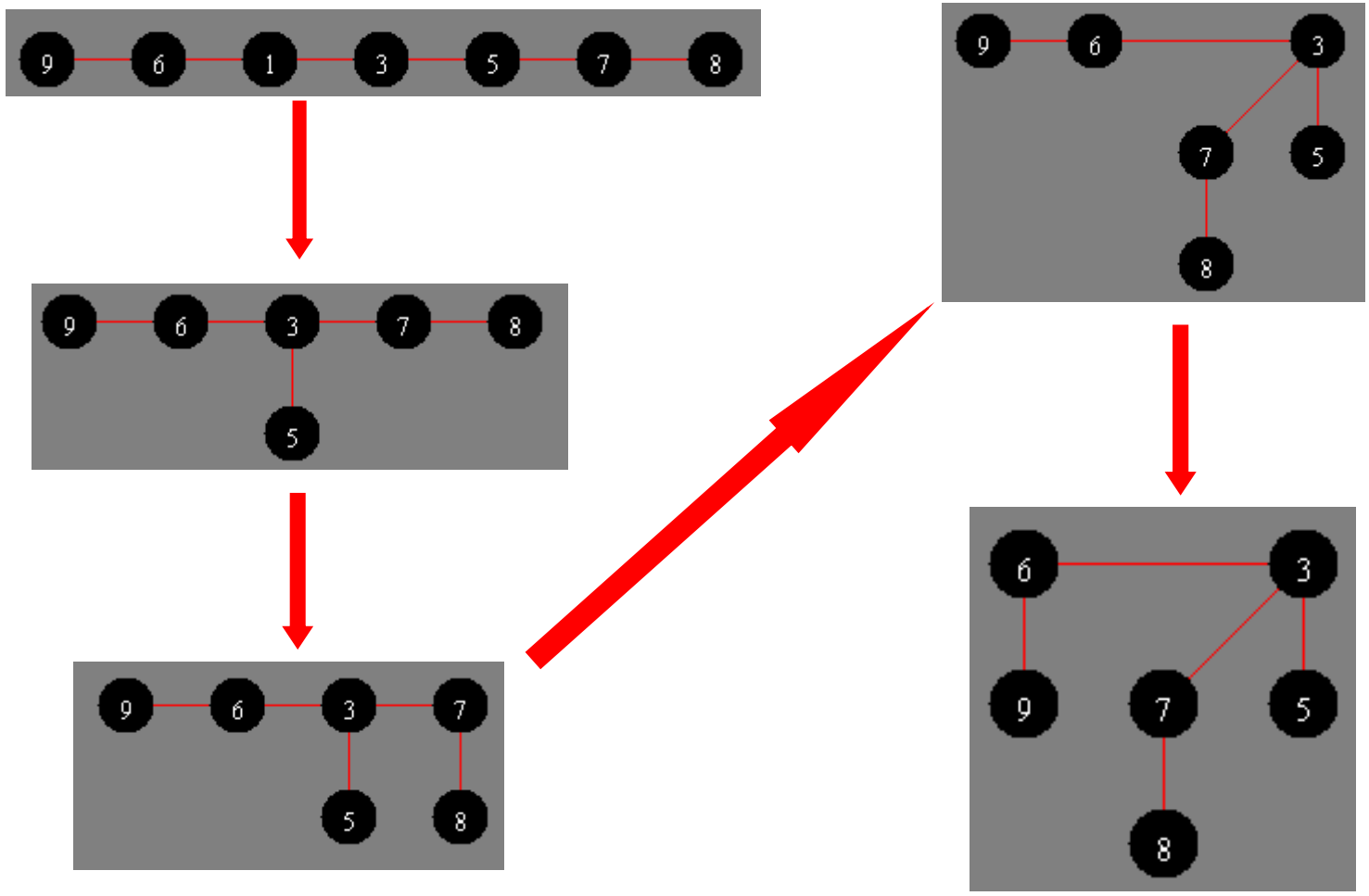
- Min – accesibil direct - min(H) – O(1)
- ExtrageMin O(logn) – cost amortizat!
  - Mută copiii minimului pe primul nivel;
  - **Consolidează** heap-ul.

Operatii Heap Fibonacci

- Consolidare Heap
  - Cât timp există 2 arbori cu grade egale Arb(x) și Arb(y), x < y:
    - Arb(y) transformat în copil al lui x;
    - grad[x] ++;
- Applet și implementare disponibile la [4].
- Gif + implementare puteți găsi la [5]



Consolidare Heap



Costuri Heap Fibonacci

- Costuri:
  - insert – 1 * V = V;
  - delete – logV * V = VlogV(amortizat!);
  - micșorează_val – 1 * E = E;
  - este_vidă? – 1 * V = V.
- Cea mai rapidă structură dpdv teoretic.

Concluzii Dijkstra

- Implementarea trebuie realizată în funcție de tipul grafului pe care lucrăm:
  - vectori pentru grafuri “dese” - $O(V^2)$;
  - heap pentru grafuri “rare”: HB - O(E logV), HF - O(V log V+E)
- Heapul Fibonacci este mai eficient decât heapul binar dar mai dificil de implementat.



ÎNTREBĂRI?



Proiectarea Algoritmilor

Curs 9 – Drumuri de cost minim



Bibliografie

[1] Giumale – Introducere în Analiza Algoritmilor cap. 5.4, 5.4.1

[2] Cormen – Introducere în Algoritmi cap. Heap-uri binomiale (20), Heap-uri Fibonacci (21), Drumuri minime de sursă unica - primele 2 subcapitole (25.1 și 25.2)

[3] R. Sedgewick, K. Wayne - Algorithms and Data Structures Fall 2007 – Curs Princeton - http://www.cs.princeton.edu/~rs/AlgsDS07/06PriorityQueues.pdf

[4] Heap Fibonacci: https://www.cs.usfca.edu/~galles/JavascriptVisual/FibonacciHeap.html

[5] Fibonacci heap: http://www.growingwiththeweb.com/data-structures/fibonacci-heap/overview/

Obiective

- “Descoperirea” algoritmilor de identificare a drumurilor de cost minim.
- Recunoașterea caracteristicilor acestor algoritmi.



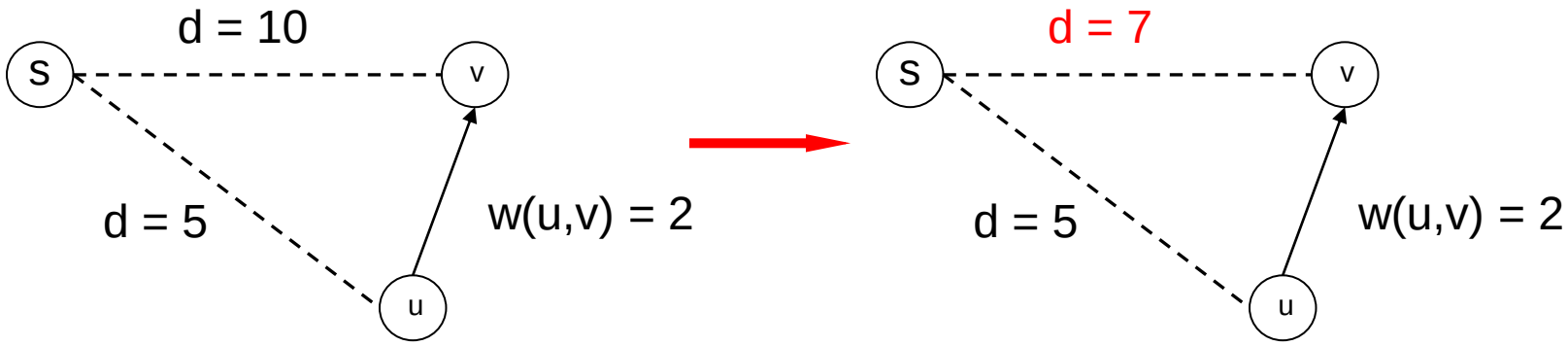
Drumuri minime de sursă unică

- Sunt concepuți pentru grafuri orientate.
- Bazați pe algoritmi Greedy.
- Se pornește de la nodul de start și pe baza unui optim local, drumurile sunt extinse și optimizate până la soluția finală.
- Notații:
  - s = nodul sursă
  - d[v] = costul drumului descoperit s..v;
  - δ(u,v) = costul drumului optim u..v; δ(u,v)=∞ dacă v $\notin$ R(u);
  - p(v) = predecesorul lui v pe drumul s..v.



Relaxarea arcelor (I)

- Relaxarea arcelor:
  - **Dacă** d[v] > d[u] + w(u,v), **atunci**
    - d[v] = d[u] + w(u,v);
    - p[v] = u



- Exemple: Dijkstra și Bellman–Ford.

Algoritmul lui Dijkstra (I)

- Folosește o coadă de priorități în care se adaugă nodurile în funcție de distanța cunoscută în momentul respectiv de la s până la nod.
- Se folosește **NUMAI** pentru costuri pozitive ($w(u,v) > 0, \forall u,v \in V$).
- Dijkstra_generic (G,s)
  - V = nodurile lui G
  - **Cât timp** (V != ∅)
    - u = nod din V cu d[u] min
    - V = V - {u}
    - **Pentru fiecare** (v ∈ succesorii lui u) relaxare_arc(u,v)

    // optimizare drum s..v pentru v ∈ succesorilor lui u

Optimalitatea drumurilor minime – Reminder

- Lemă 25.1 (Subdrumurile unui drum minim sunt drumuri optimale): G = (V,E), w : E → $\Re$ funcție de cost asociată. Fie p = $v_1v_2...v_k$ un drum optim de la v_1 la v_k. Atunci pentru orice i și j cu $1 \leq i \leq j \leq k$, subdrumul lui p de la v_i la v_j este un drum minim.

- Corolar 25.2: G = (V,E), w : E → $\Re$ funcție de cost asociată. Fie p = s..uv un drum optim de la s la v. Atunci costul optim al acestui drum poate fi scris ca $\delta(s,v) = \delta(s,u) + w(u,v)$.

- Lemă 25.3: G = (V,E), w : E → $\Re$ funcție de cost asociată. $\forall (u,v) \in E$ avem $\delta(s,v) \leq \delta(s,u) + w(u,v)$.

Relaxarea arcelor (II)

- Lemă 25.5: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. ∀ v ∈ V, d[v] obținut de algoritmul lui Dijkstra respectă d[v] ≥ δ(s,v). În plus, odată atinsă valoarea δ(s,v), ea nu se mai modifică.

- Dem:
  - ∀ v ∈ V, v ∉ R(s) → d[v] = δ(s,v) = ∞; d[s] = δ(s,s) = 0 (inițializare)
  - Pt v ∈ R(s), inițializare → d[v] = ∞ ≥ δ(s,v). Dem. prin reducere la absurd că după oricâte relaxări, relația se menține.
    - Fie v primul vârf pentru care relaxarea (u,v) determină d[v] < δ(s,v) → după relaxarea (u,v): d[u] + w(u,v) = d[v] < δ(s,v) ≤ δ(s,u) + w(u,v) → d[u] < δ(s,u). Dar relaxarea nu modifică d[u], iar v e primul pentru care d[v] < δ(s,v). Contrazice presupunerea! => d[v] ≥ δ(s,v), ∀ v ∈ V
  - Cum d[v] ≥ δ(s,v) => odată ajuns la d[v] = δ(s,v), ea nu mai scade. Cum relaxarea nu creste valorile => d[v] nu se mai modifică.

Relaxarea arcelor (III)

- Lemă 25.7: G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. Fie p = s..uv un drum optim de la s la v. Dacă d[u] = δ(s,u) la un moment dat, atunci începând cu momentul imediat următor relaxării arcului (u,v) avem d[v] = δ(s,v)

- Dem:
  - Dacă înainte de relaxare d[v] > d[u] + w(u,v), prin relaxare → d[v] = d[u] + w(u,v). Altfel, d[v] ≤ d[u] + w(u,v) => după relaxare avem d[v] ≤ d[u] + w(u,v).
  - Cum d[u] = δ(s,u) și relaxarea (u,v) nu modifică d[u] => d[v] ≤ d[u] + w(u,v) = δ(s,u) + w(u,v) = δ(s,v) (conf. Corolar 25.2) → d[v] = δ(s,v)

Algoritmul lui Dijkstra (II)

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V) // inițializări
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele

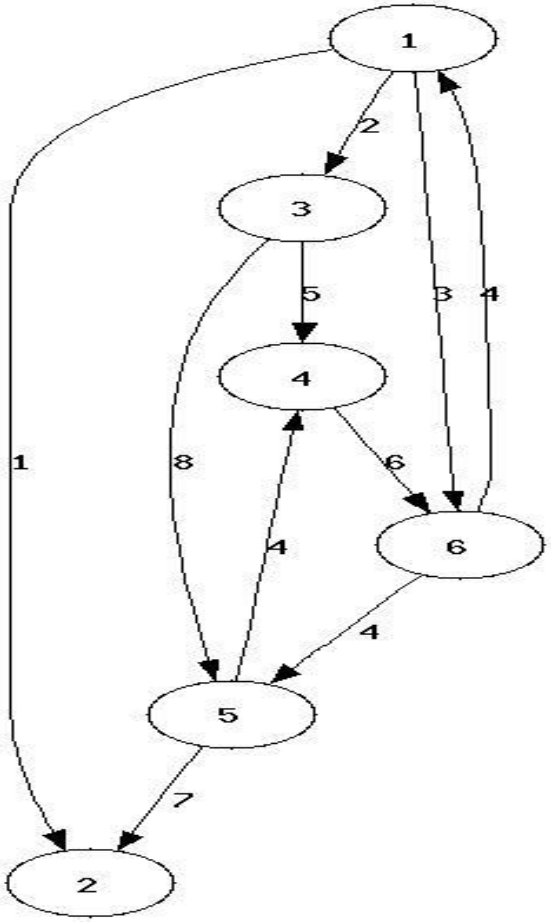
Corectitudine Dijkstra

- Teoremă. G = (V,E), w : E → $\Re$ funcție de cost asociată nenegativă. La terminarea aplicării algoritmului Dijkstra pe acest graf plecând din sursa s vom avea d[v] = δ(s,v) pentru ∀ v ∈ V.

- Dem: prin reducere la absurd se demonstrează că la scoaterea din Q a fiecărui nod v avem d[v] = δ(s,v) și egalitatea se menține și ulterior.
  - Pp. u e primul nod pt. care d[u] ≠ δ(s,u) la scoaterea din Q. u ≠ s pt. că altfel d[u] = δ(s,u) = 0 și u ∈ R(s) pt.că altfel d[u] = δ(s,u) = ∞. => La scoaterea lui u din Q, ∃ drum s..u și fie p drumul optim s..u a.î. p = s..xy..u, unde x ∉ Q iar y ∈ Q.
  - (Dacă nu impunem y ∈ Q, atunci toate nodurile până la u au fost scoase din Q și u a rămas în vârful cozii. Cum u e primul nod pentru care d[u] ≠ δ(s,u), iar calea e optimă, avem situația din Lema 25.7 => La scoaterea lui u din Q, d[u] = δ(s,u).)
  - Cum u e primul nod pt. care d[u] ≠ δ(s,u) => d[x] = δ(s,x) la momentul extragerii lui u din Q → d[y] = δ(s,y) prin relaxarea (x,y) (conf. Lema 25.7).
  - y precede u pe drumul p => d[y] = δ(s,y) ≤ δ(s,u) ≤ d[u] (conf. Lema 25.5).
  - Cum y ∈ Q la momentul scoaterii lui u din Q => d[u] ≤ d[y]
  - => d[y] = δ(s,y) = δ(s,u) = d[u] – Contrazice ipoteza! => d[u] = δ(s,u) și conf. Lema 25.5, egalitatea se menține și ulterior.



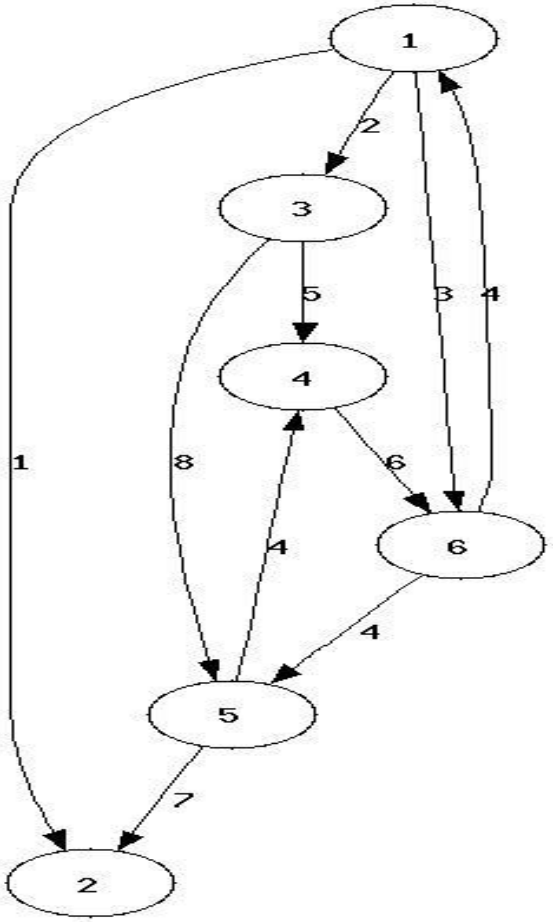
Exemplu (I)



Exemplu (II)

- d[1] = 0;
- (1): d[2] = 1; d[3] = 2; d[6] = 3;
- (2): d[4] = 7; d[5] = 10;
- (3): d[5] = 7;

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] // minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele



Complexitate Dijkstra

- Depinde de ExtrageMin – coadă cu priorități.
- Operații ce trebuie realizate pe coadă + frecvența lor:
  - insert – V;
  - delete – V;
  - conține? – E;
  - micșorează_val – E;
  - este_vidă? – V.

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută in cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q si v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // si părintele

Implementare cu vectori

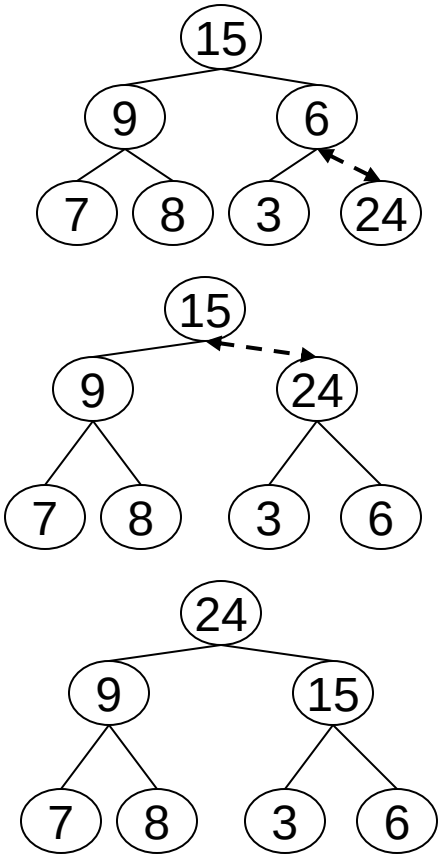
- Costuri:
  - insert – 1 * V = V;
  - delete – V * V = V^2 (necesită căutarea minimului);
  - conține? – 1 * E = E;
  - micșorează_val – 1 * E = E;
  - este_vidă? – 1 * V = V;
- Cea mai bună metodă pentru grafuri "dese" ($E \approx V^2$)!

Implementare cu heap binar

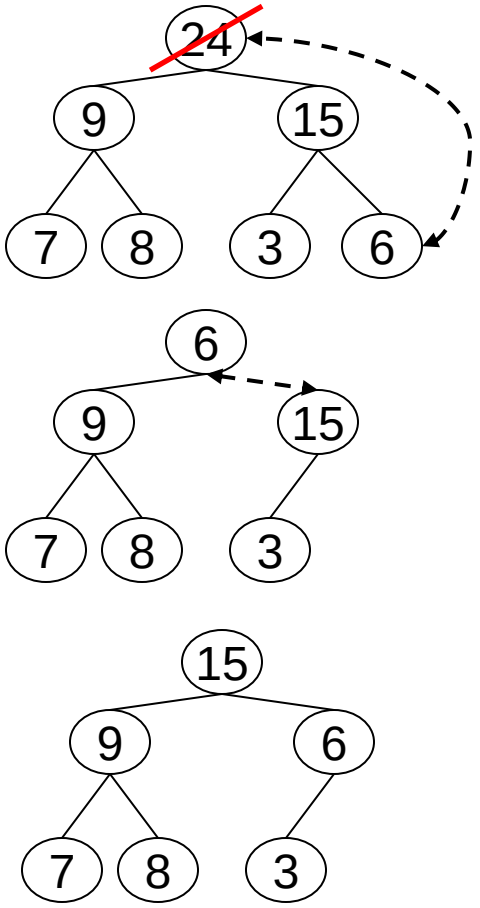
- Heap binar – structură de date de tip arbore binar + 2 constrângeri:
  - Fiecare nivel este complet; ultimul se umple de la stânga la dreapta;
  - $\forall u \in Heap$; $u \geq$ răd(st(u)) și $u \geq$ răd(dr(u)) (u este $\geq$ decât ambii copii ai săi) unde $\geq$ este o relație de ordine pe mulțimea pe care sunt definite elementele heapului.

Operatii pe Heap Binar

insert



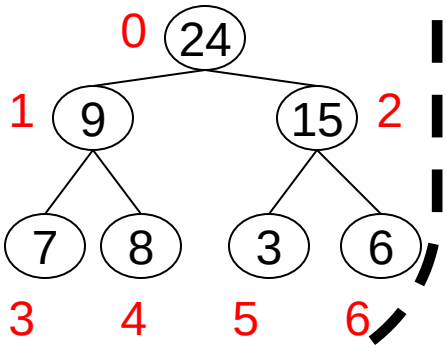
delete



Implementare Heap Binar

- Implementare folosind vectori.
- Poziție[i] = unde se găsește în indexul de valori elementul de pe poziția i din heap.
- Reverse[i] = unde se găsește în heap elementul de pe poziția i din valoare.
- Implementare disponibila la [3].

Index	0	1	2	3	4	5	6
Valoare	7	6	15	8	24	9	3
Poziție							
Reverse							



Heap Binar

- Costuri:
  - insert – logV * V = VlogV;
  - delete – logV * V = VlogV;
  - conține? – 1 * E = E;
  - micșorează_val – logV * E = ElogV;
  - este_vidă? – 1 * V = V.

- Eficient dacă graful are arce puține comparativ cu numărul de noduri.



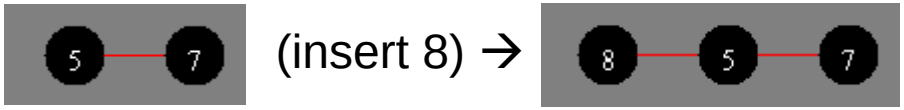
Heap Fibonacci

- Poate fi format din mai mulți arbori.
- Cheia unui părinte ≤ cheia oricărui copil.
- Fiind dat un nod u și un heap H:
  - p(u) – părintele lui u;
  - copil(u) – legătura către unul din copiii lui u;
  - st(u), dr(u) – legătura la frații din stânga și din dreapta (cei de pe primul nivel sunt legați între ei astfel);
  - grad(u) – numărul de copii ai lui u;
  - min(H) – cel mai mic nod din H;
  - n(H) – numărul de noduri din H.



Operatii Heap Fibonacci

- Inserare nod – O(1)
  - construiește un nou arbore cu un singur nod



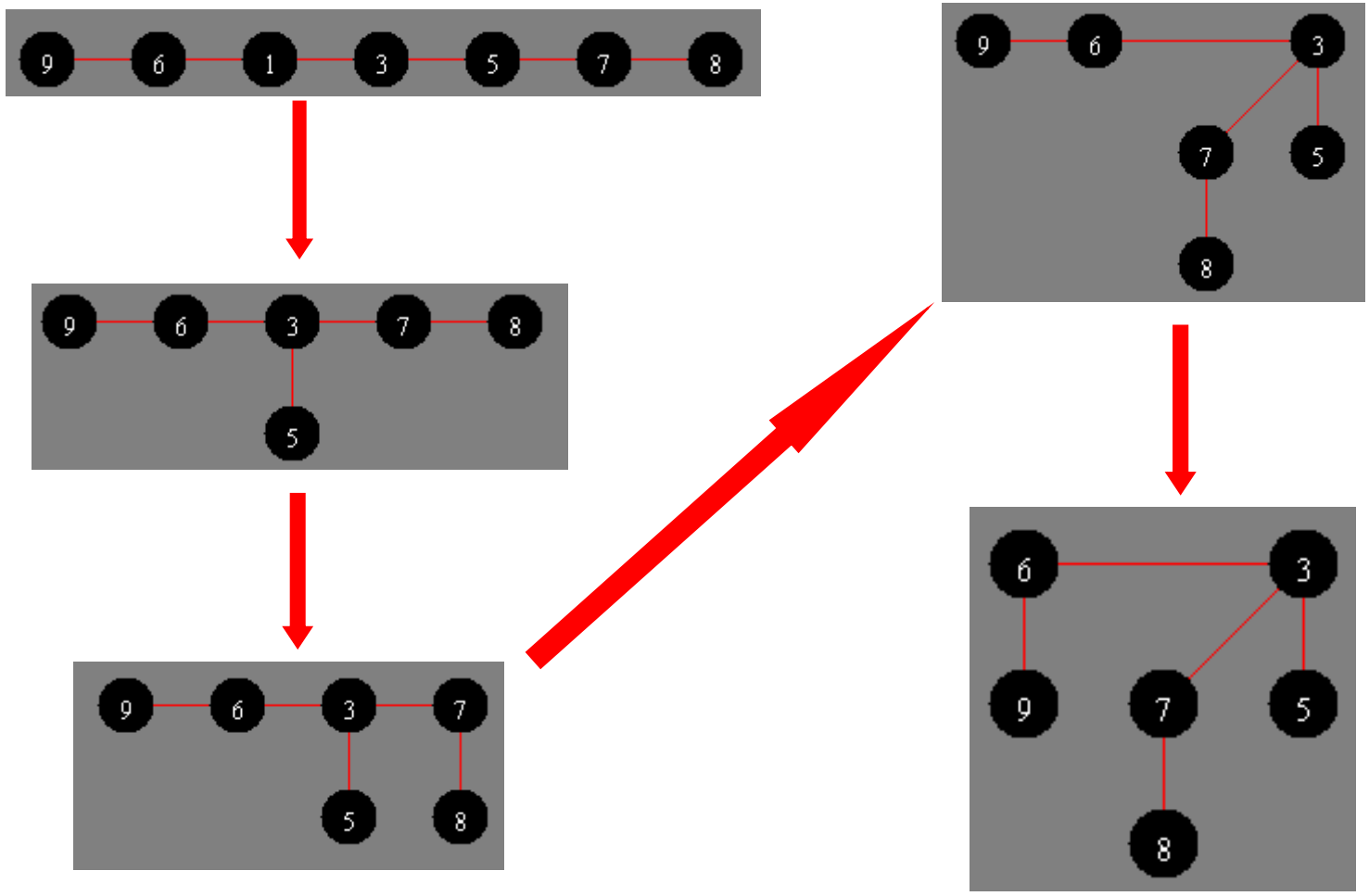
- Min – accesibil direct - min(H) – O(1)
- ExtrageMin O(logn) – cost amortizat!
  - Mută copiii minimului pe primul nivel;
  - **Consolidează** heap-ul.

Operatii Heap Fibonacci

- Consolidare Heap
  - Cât timp există 2 arbori cu grade egale Arb(x) și Arb(y), x < y:
    - Arb(y) transformat în copil al lui x;
    - grad[x] ++;
- Applet și implementare disponibile la [4].
- Gif + implementare puteți găsi la [5]



Consolidare Heap



Costuri Heap Fibonacci

- Costuri:
  - insert – 1 * V = V;
  - delete – logV * V = VlogV(amortizat!);
  - micșorează_val – 1 * E = E;
  - este_vidă? – 1 * V = V.
- Cea mai rapidă structură dpdv teoretic.

Concluzii Dijkstra (I)

- Dijkstra(G,s)
  - **Pentru fiecare nod u** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare nod v** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele

Complexitate?

Vectori - $O(V^2)$

HB - $O(E \log V)$

HF - $O(V \log V + E)$

Concluzii Dijkstra (II)

- Implementarea trebuie realizată în funcție de tipul grafului pe care lucrăm:
  - vectori pentru grafuri “dese”;
  - heap pentru grafuri “rare”.

- Heapul Fibonacci este mai eficient decât heapul binar dar mai dificil de implementat.

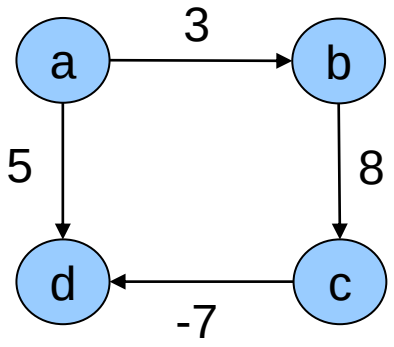


Concluzii Dijkstra (III)

- Implementarea trebuie realizată în funcție de tipul grafului pe care lucrăm:
  - vectori pentru grafuri “dese” - $O(V^2)$;
  - heap pentru grafuri “rare”: HB - $O(E \log V)$, HF - $O(V \log V+E)$
- Heapul Fibonacci este mai eficient decât heapul binar dar mai dificil de implementat.

Problemă Dijkstra

- Exemplu rulare:
  - d[a] = 0; d[b] = d[c] = d[d] = ∞
  - d[b] = 3; d[d] = 5; d[c] = ∞
  - d[d] = 5; d[c] = 11;



  - **d este extras din coadă!** În momentul extragerii din coadă distanța pană la nodul d se consideră a fi calculată și a fi optimă.
  - Se extrage nodul c; d[d] nu va mai fi actualizată – nodul d fiind deja eliminat din coadă.
- → Algoritmul nu funcționează pentru grafuri ce conțin arce de cost negativ!

Exemplu practic – arce de cost negativ (I)

Currency conversion. Given currencies and exchange rates, what is best way to convert one ounce of gold to US dollars?

- 1 oz. gold ⇒ $327.25.
- 1 oz. gold ⇒ £208.10 ⇒ ⇒ $327.00. [208.10 × 1.5714]
- 1 oz. gold ⇒ 455.2 Francs ⇒ 304.39 Euros ⇒ $327.28. [455.2 × .6677 × 1.0752]

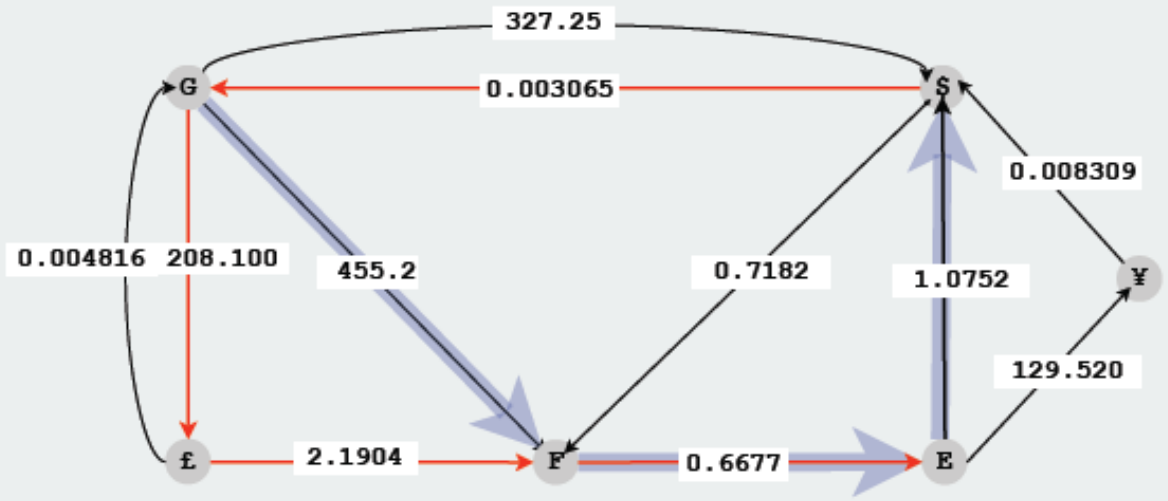
Currency	£	Euro	¥	Franc	$	Gold
UK Pound	1.0000	0.6853	0.005290	0.4569	0.6368	208.100
Euro	1.4599	1.0000	0.007721	0.6677	0.9303	304.028
Japanese Yen	189.050	129.520	1.0000	85.4694	120.400	39346.7
Swiss Franc	2.1904	1.4978	0.011574	1.0000	1.3929	455.200
US Dollar	1.5714	1.0752	0.008309	0.7182	1.0000	327.250
Gold (oz.)	0.004816	0.003295	0.0000255	0.002201	0.003065	1.0000

*slide din cursul de algoritmi de la Princeton – Sedgewick&Wayne[1]

Exemplu practic – arce de cost negativ (II)

Graph formulation.

- Vertex = currency.
- Edge = transaction, with weight equal to exchange rate.
- Find path that maximizes product of weights.



*slide din cursul de algoritmi de la Princeton – Sedgewick&Wayne[1]

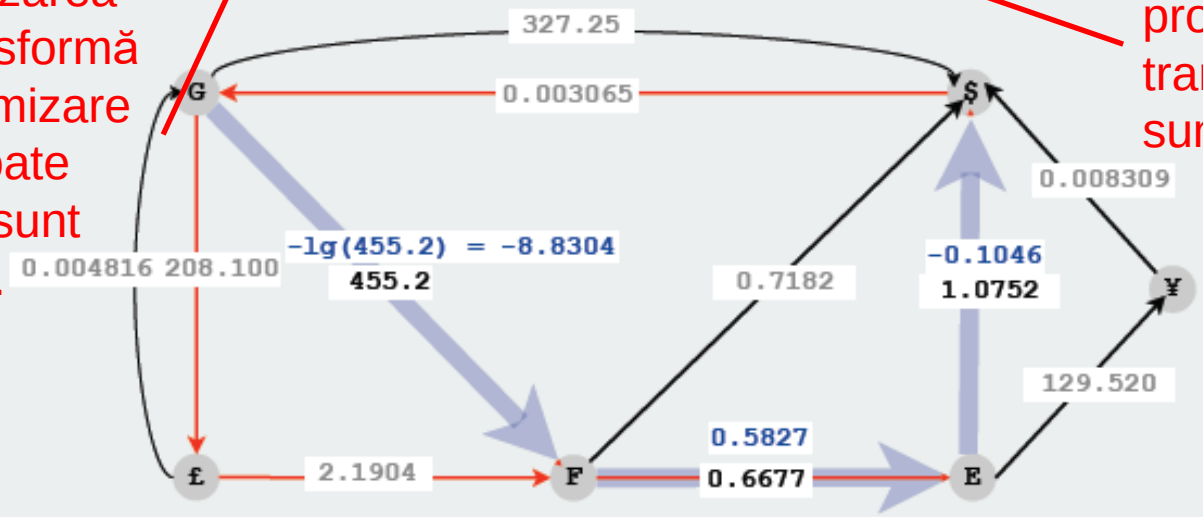
Exemplu practic – arce de cost negativ (III)

Reduce to shortest path problem by taking logs

- Let weight(v-w) = – lg (exchange rate from currency v to w)
- multiplication turns to addition
- Shortest path with costs c corresponds to best exchange sequence.

Maximizarea se transformă în minimizare dacă toate arcele sunt negate.

Prin logaritmare produsul se transformă în sumă.

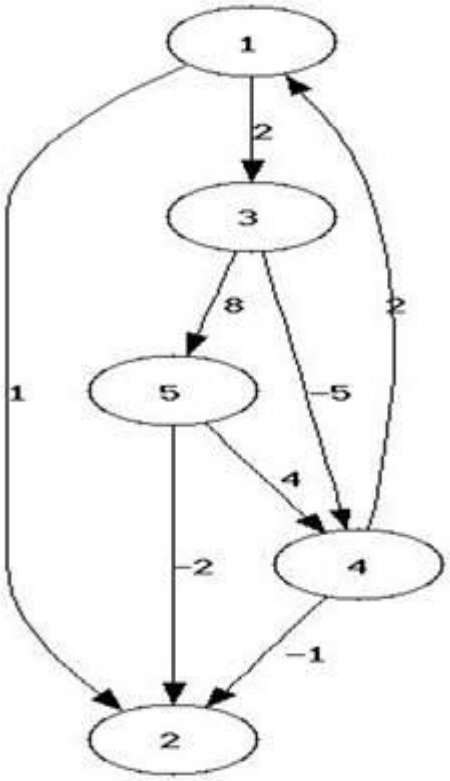


*slide din cursul de algoritmi de la Princeton – Sedgewick&Wayne[1]

Cicluri de cost negativ

$$\delta(u, v) = \begin{cases} \Sigma\, w(x,y),\ (x,y) \in u..v \\ (u..v \text{ fiind drumul optim}); \\ \\ \infty, \text{ dacă nu există drum} \\ u..v. \end{cases}$$

- Dacă există pe drumul u..v un ciclu de cost negativ x..y →
  - δ(u,v) = δ(u,v) + cost(x..y) < δ(u,v)
  - → valoarea lui δ(u,v) va scădea continuu → costul este -∞
  - → δ(u,v) = -∞

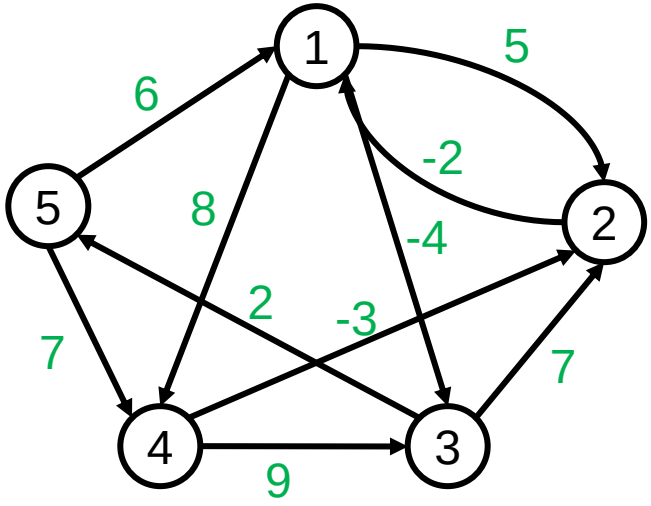


1-3-4 ciclu de cost negativ(-1)→ toate costurile din graf sunt -∞

Algoritmul Bellman-Ford

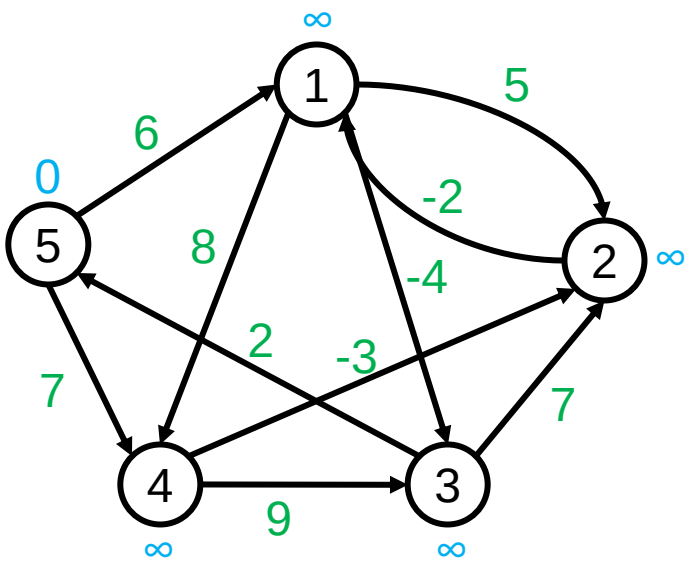
- BellmanFord(G,s) // G=(V,E),s=sursa
  - **Pentru fiecare** nod v (v ∈ V) // inițializări
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță de la s la s
  - **Pentru** *i* **de la** *1* **la** |*V*| *-1* // pentru fiecare pas pornind din s // spre restul nodurilor se încearcă construcția unor drumuri // optime de dimensiune i
    - **Pentru fiecare** (u,v) din E
      // pentru arcele ce pleacă de la nodurile deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

Exemplu Bellman-Ford (I)



- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

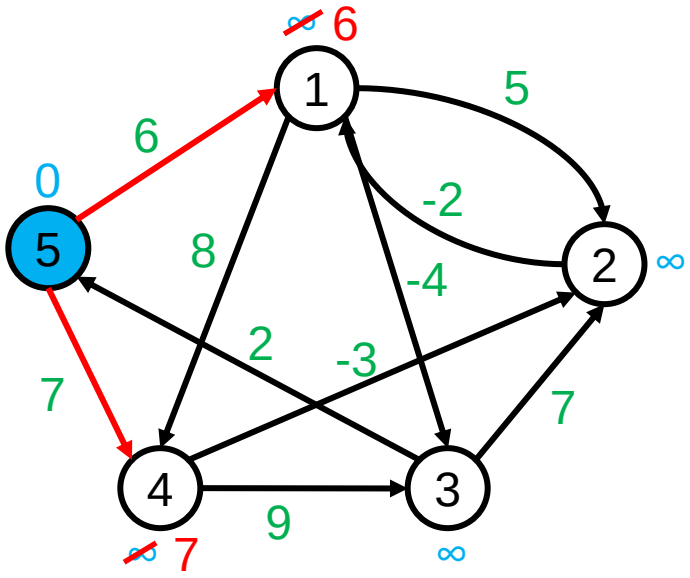
Exemplu Bellman-Ford (II)



d[1] = d[2] = d[3] = d[4] = d[5] = ∞
d[5] = 0

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

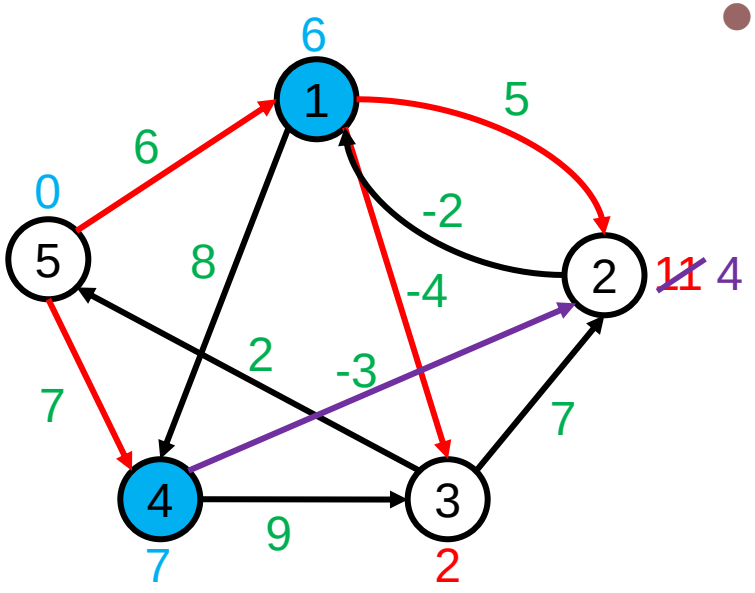
Exemplu Bellman-Ford (III)



Pas 1: relaxare (5,1) și (5,4)
d[1] = 6, p[1] = 5
d[4] = 7, p[4] = 5

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

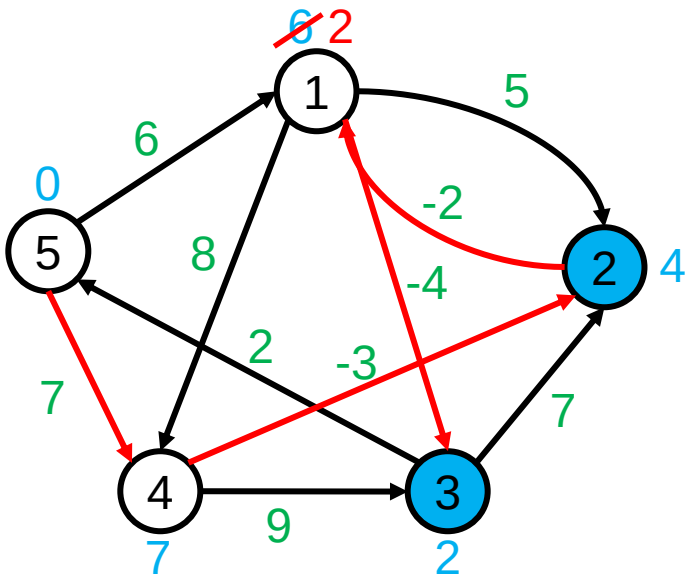
Exemplu Bellman-Ford (IV)



Pas 2: relaxare (1,2) și (1,3)
d[2] = 11, p[2] = 1
d[3] = 2, p[3] = 1
relaxare (4,2)
d[2] = 4, p[2] = 4

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

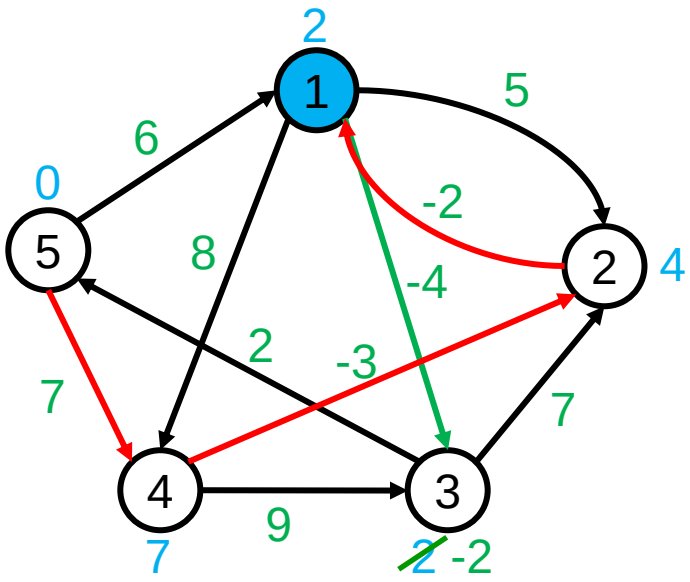
Exemplu Bellman-Ford (V)



Pas 3: relaxare (2,1)
d[1] = 2, p[1] = 2

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

Exemplu Bellman-Ford (VI)



Pas 4: relaxare (1,3)
d[3] = -2, p[3] = 1

- BellmanFord(G,s)
  - **Pentru fiecare** v din V // init
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0; // actualizare distanță până la s
  - **Pentru** i **de la** 1 **la** |V| -1 // pt // fiecare pas de la s spre V-s
    - **Pentru fiecare** (u,v) din E // pt. // arcele ce pleacă de la nodurile // deja considerate
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // se relaxează arcele // corespunzătoare
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
  - **Pentru fiecare** (u,v) din E
    - **Dacă** d[v] > d[u] + w(u,v) **atunci**
      - **Eroare** (”ciclu negativ”);

Complexitate Bellman-Ford

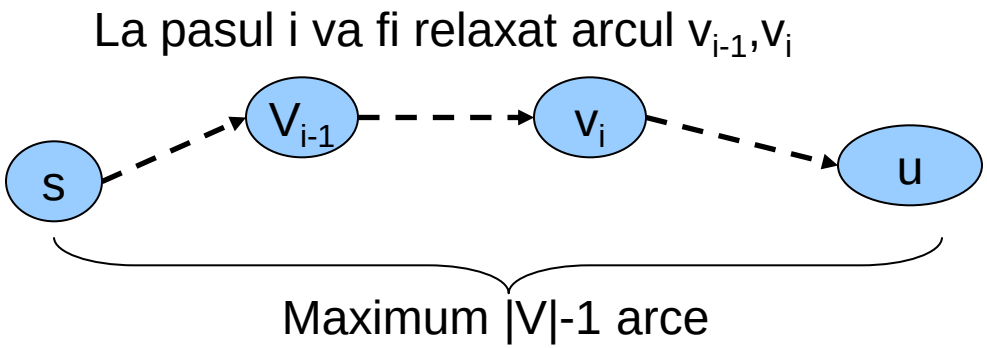
- cazul defavorabil:
  - **Pentru** i **de la** 1 **la** |V| - 1 — V
    - **Pentru fiecare** (u,v) din E — * E
      - **Dacă** d[v] > d[u] + w(u,v) **atunci**
        - d[v] = d[u] + w(u,v);
        - p[v] = u;

O(VE)

Corectitudine Bellman-Ford (I)

- Lemă 25.12: G = (V,E), w : E → $\Re$ funcție de cost asociată; dacă G nu conține ciclu de cost negativ atunci după |V| - 1 iterații ale relaxării fiecărui arc avem d[v] = δ(s,v) pentru $\forall$ v $\in$ R(s).

- Dem prin inducție:
  - Fie s = $v_0, v_1 \ldots v_k$ = u un drum minim în graf cu k ≤ |V| - 1.



Corectitudine Bellman-Ford (II)

- Demonstrăm că în pasul i: $d[v_i] = \delta(s,v_i)$ și se menține până la sfârșit.
- P_0: (inițializare) → $d[s] = d[v_0] = 0 = \delta(s,s) = \delta(s,v_0)$ și conf. Lema 25.5, relația se menține până la sfârșit.
- $P_{i-1} \rightarrow P_i$:
  - P_{i-1}: $d[v_{i-1}] = \delta(s,v_{i-1})$,
  - În pasul i se relaxează arcul (v_{i-1},v_i), => conf. Lema 25.7 => $d[v_i] = d[v_{i-1}] + w(v_{i-1},v_i) = \delta(s,v_{i-1}) + w(v_{i-1},v_i) = \delta(s,v_i)$. Conf. Lema 25.5, relația se menține până la sfârșit.
  - Cum $i \in (1,|V|-1)$ → relația e adevărată pentru toate nodurile accesibile din s → $d[v] = \delta(s,v)$, $\forall\ v \in R(s)$.

Corectitudine Bellman-Ford (III)

- Teorema. G = (V,E), w : E → $\mathfrak{R}$ funcție de cost asociată. Algoritmul Bellman-Ford aplicat acestui graf plecând din sursa s nu returnează EROARE dacă G nu conține cicluri negative, iar la terminare d[v] = δ(s,v) pentru ∀ v ∈ V. Dacă G conține cel puțin un ciclu negativ accesibil din s, atunci algoritmul întoarce EROARE.

- Dem: pe baza Lemei 25.12.
  - Dacă ∄ ciclu negativ:
    - d[v] = δ(s,v) ∀ v ∈ R(s)
    - d[v] = δ(s,v) = ∞, ∀ v ∉ R(s) (inițializare)
    - → d[v] ≤ d[u] + w(u,v) → nu se întoarce eroare (conf. Lema 25.3)
  - Dacă ∃ ciclu negativ → în cei |V| - 1 pași se scad costurile drumurilor, iar în final ciclul se menține → Eroare

Optimizări Bellman-Ford

- Observații!
  - 1) Dacă d[v] nu se modifică la pasul i atunci nu trebuie sa relaxăm niciunul din arcele care pleacă din v la pasul i + 1.
  - 2) => păstrăm o coadă cu vârfurile modificate (o singură copie).

Bellman-Ford optimizat 1

- BellmanFordOpt1(G,s)
  - **Pentru fiecare nod v** (v ∈ V)
    - d[v] = ∞;
    - p[v] = null;
  - d[s] = 0;
  - **Pentru** i **de la** 1 **la** |V| -1
    - **Dacă nu** s-a modificat nicio distanță în iterația anterioară
      - **Break**
    - **Pentru fiecare nod u** (u ∈ V)
      - **Dacă** d[u] a fost modificat în iterația anterioară (i anterior)
        - **Pentru fiecare nod v** (v ∈ succesorilor lui u)
          - Dacă d[v] > d[u] + w(u,v) **atunci** // relaxez arcele ce pleacă din u
            - d[v] = d[u] + w(u,v);
            - p[v] = u;
- Ce se întâmplă cu cicluri negative?

Bellman-Ford optimizat 2

- BellmanFordOpt2(G,s)
  - **Pentru fiecare nod v** (v ∈ V)
    - d[v] = ∞;
    - p[v] = null;
    - marcat[v] = false; // marcăm nodurile pentru care am făcut relaxare
  - Q = ∅; // coadă cu priorități
  - d[s] = 0; marcat[s] = true; Introdu(Q,s);
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); marcat[u] = false; // extrag minimul
    - **Pentru fiecare nod v** (v ∈ succesorilor lui u)
      - **Dacă** d[v] > d[u] + w(u,v) **atunci** // relaxez arcele ce pleacă din u
        - d[v] = d[u] + w(u,v);
        - p[v] = u;
        - **Dacă** (marcat[v] == false) {marcat[v] = true; Introdu(Q,v);}
- Observație: nu mai detectează cicluri negative!

Floyd-Warshall (Roy-Floyd)

- Algoritm prin care se calculează distanțele minime între oricare 2 noduri dintr-un graf (drumuri optime multipunct-multipunct).
- Exemplu clasic de programare dinamică.
- Idee: la pasul k se calculează cel mai bun cost între u și v folosind cel mai bun cost u..k și cel mai bun cost k..v calculat până în momentul respectiv.
- Se aplică pentru grafuri ce nu conțin cicluri de cost negativ.



Notații

- G = (V,E); V = {1, 2, …, n};
- w : V x V → $\Re$; w(i, i) = 0; w(i, j) = ∞ dacă (i,j) $\notin$ E;
- $d^k(i,j)$ = costul drumului i..j construit astfel încât drumul trece doar prin noduri din mulțimea {1, 2, .., k};
- δ(i,j) = costul drumului optim i..j; δ(i,j) = ∞ dacă ∄ i..j;
- $\delta^k(i,j)$ = costul drumului optim i..j ce trece doar prin noduri din mulțimea {1, 2, .., k}; $\delta^k(i,j)$ = ∞ dacă ∄ un astfel de drum i..j;
- $p^k(i,j)$ = predecesorul lui j pe drumul i..j ce trece doar prin noduri din mulțimea {1, 2, .., k}.

Teorema Floyd - Warshall

- Teoremă: Fie formulele de mai jos pentru calculul valorii $d^k(i,j)$, $0 \leq k \leq n$:
  - $d^0(i,j) = w(i,j)$;
  - $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$, pentru $0 < k \leq n$;

  Atunci $d^n(i,j) = \delta(i,j)$, pentru $\forall\ i, j \in V$

- Dem:
  - Prin inducție după k dem. că $d^k(i,j) = \delta^k(i,j)$. (next slide)
  - Pt. k = n, i..j trece prin $\forall\ v \in V$ si avem $d^k(i,j) \leq d^{k-1}(i,j)$, $\forall\ k = 1,n \rightarrow d^n(i,j) \leq d^{k-1}(i,j)$, $\forall\ k = 1,n$
  - Din $d^k(i,j) = \delta^k(i,j) \rightarrow d^n(i,j) = \delta^n(i,j) \leq d^{k-1}(i,j) = \delta^{k-1}(i,j)$, $\forall\ k = 1,n \rightarrow d^n(i,j) = \delta^n(i,j) = \delta(i,j)$

Demonstrație teorema Floyd - Warshall

- $k = 0$: 0 noduri intermediare → i..j = (i,j), la fel ca inițializarea $d^0(i,j) = w(i,j)$;
- $0 < k \leq n$: $d^{k-1}(i,j) = \delta^{k-1}(i,j)$ → $d^k(i,j) = \delta^k(i,j)$
- a) $k \notin$ drumului optim i..j: drumul optim nu se modifică

  $(\delta^{k-1}(i,j) = \delta^k(i,j) \leq \delta^{k-1}(i,k) + \delta^{k-1}(k,j))$
- $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$

  $d^k(i,j) = \min\{\delta^{k-1}(i,j), \delta^{k-1}(i,k) + \delta^{k-1}(k,j)\} = \delta^{k-1}(i,j) = \delta^k(i,j)$
- b) $k \in$ drumului optim i..j: i..j se descompune în i..k și k..j optime $(\delta^{k-1}(i,k) = d^{k-1}(i,k)$ și $\delta^{k-1}(k,j) = d^{k-1}(k,j))$ și

  $\delta^k(i,j) = \delta^{k-1}(i,k) + \delta^{k-1}(k,j)$.
- i..j optim → $\delta^k(i,j) \leq \delta^{k-1}(i,j)$
- $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$
- $d^k(i,j) = \min\{\delta^{k-1}(i,j), \delta^{k-1}(i,k) + \delta^{k-1}(k,j)\} = \delta^{k-1}(i,k) + \delta^{k-1}(k,j) = \delta^k(i,j)$

Algoritm Floyd-Warshall

- **Floyd-Warshall(G)**
- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n // inițializări
    - $d^0(i,j) = w(i,j)$
    - **Dacă** $(w(i,j) == \infty)$
      - $p^0(i,j) = null;$
    - **Altfel** $p^0(i,j) = i;$
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - **Dacă** $(d^{k-1}(i,j) > d^{k-1}(i,k) + d^{k-1}(k,j))$ // determinăm minimul
        - $d^k(i,j) = d^{k-1}(i,k) + d^{k-1}(k,j)$
        - $p^k(i,j) = p^{k-1}(k,j);$ // și actualizăm părintele
      - **Altfel**
        - $d^k(i,j) = d^{k-1}(i,j)$
        - $p^k(i,j) = p^{k-1}(i,j);$

Complexitate?

$O(V^3)$

Complexitate spațială?

$O(V^3)$

Observație

- Putem folosi o singură matrice în loc de n?
- Problemă: în pasul k, pentru k < i și k < j, d(i,k) și d(k,j) folosite la calculul d(i,j) sunt $d^k(k,j)$ și $d^k(i,k)$ în loc de $d^{k-1}(k,j)$ și $d^{k-1}(i,k)$. Dacă dem. că $d^k(k,j)= d^{k-1}(k,j)$ și $d^k(i,k)=d^{k-1}(i,k)$, atunci putem folosi o singură matrice.
- Dar: $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$
  - $d^k(k,j) = \min\{d^{k-1}(k,j), d^{k-1}(k,k) + d^{k-1}(k,j)\} = d^{k-1}(k,j)$
  - $d^k(i,k) = \min\{d^{k-1}(i,k), d^{k-1}(i,k) + d^{k-1}(k,k)\} = d^{k-1}(i,k)$
- → Algoritm modificat pentru a folosi o singura matrice → complexitate spațială: $O(n^2)$.

Algoritm Floyd-Warshall

Floyd-Warshall2(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n // inițializări
    - d(i,j) = w(i,j)
    - **Dacă** (w(i,j) == ∞)
      - p(i,j) = null;
    - **Altfel** p(i,j) = i;
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - **Dacă** (d(i,j) > d(i,k) + d(k,j)) // determinăm minimul
        - d(i,j) = d(i,k) + d(k,j)
        - p(i,j) = p(k,j); // și actualizăm părintele

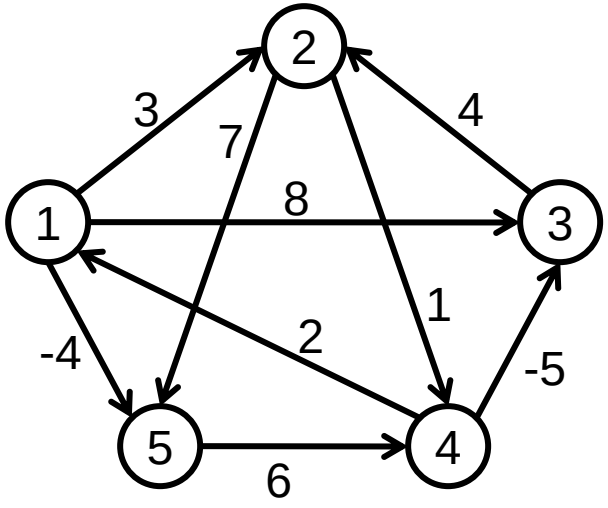
Complexitate?

$O(V^3)$

Complexitate spațială?

$O(V^2)$

Exemplu (I)

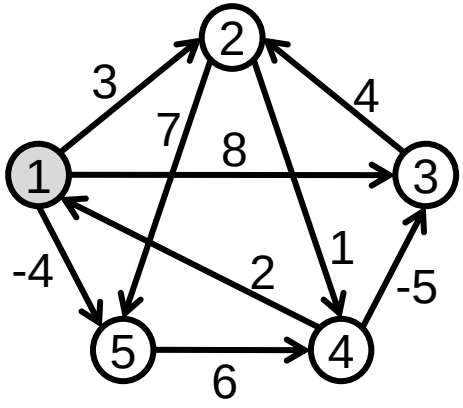


$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & nil & 4 & nil & nil \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (II)

$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$



$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

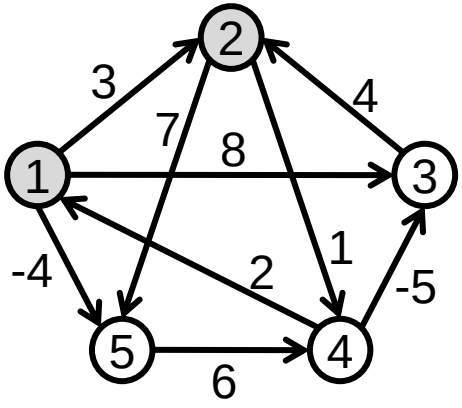
$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & nil & 4 & nil & nil \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (III)

$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$



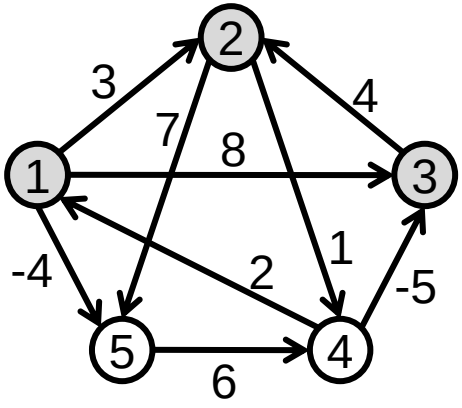
$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (IV)

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$



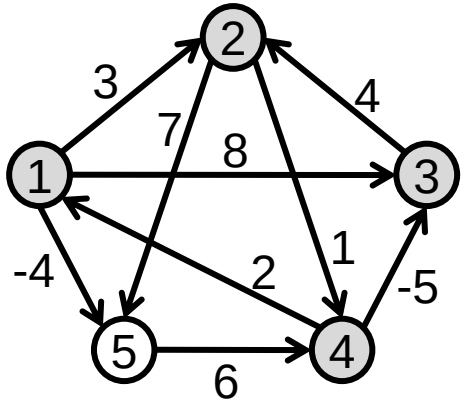
$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 3 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (V)

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

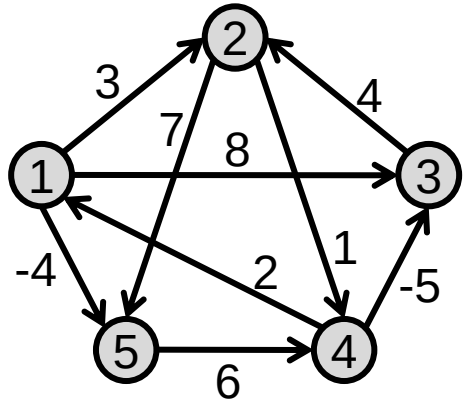


$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 3 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 4 & 2 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

Exemplu (VI)

$$D = \begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$



$$D = \begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 4 & 2 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 3 & 4 & 5 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

Închiderea tranzitivă (I)

- Fie G = (V,E). Închiderea tranzitivă a lui E e un G* = (V,E*), unde

$$E^*(i,j)=\begin{cases}1, \text{ dacă } \exists\ i..j\\ 0, \text{ dacă } \nexists\ i..j\end{cases}$$

- Poate fi determinată prin modificarea algoritmului Floyd-Warshall:
  - min $\Longrightarrow$ operatorul boolean sau ($\vee$)
  - + $\Longrightarrow$ operatorul boolean și ($\wedge$)



Închiderea tranzitivă (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - $E^*(i,j) = (((i,j) \in E) \vee (i = j))$ // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - $E^*(i,j) = E^*(i,j) \vee (E^*(i,k) \wedge E^*(k,j))$

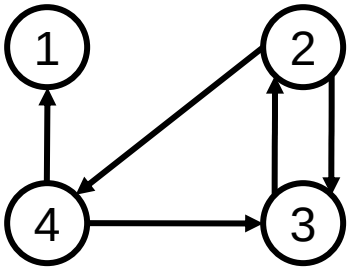
Complexitate? $O(V^3)$

Complexitate spațială? $O(V^2)$

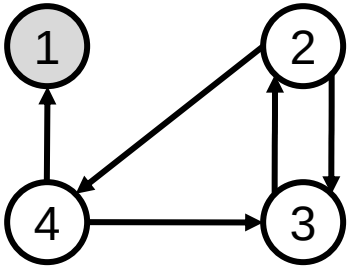
Exemplu (I)

Închidere_tranzitivă(G)

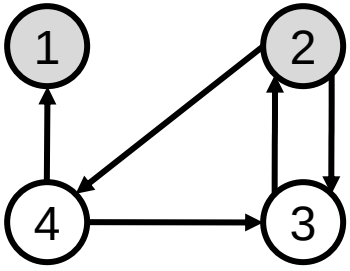
- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(0)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$



$$T^{(1)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

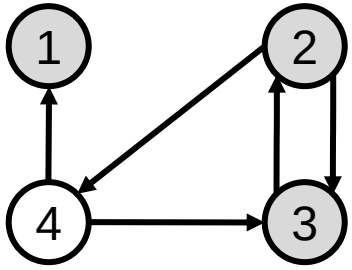


$$T^{(2)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

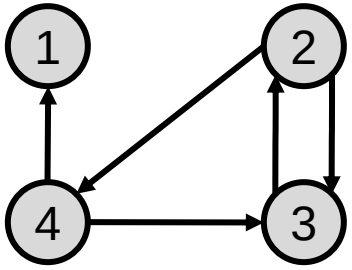
Exemplu (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(3)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

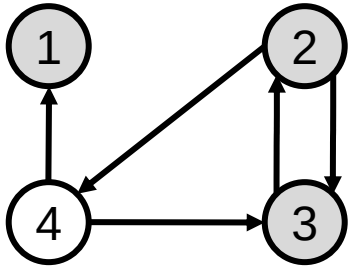


$$T^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

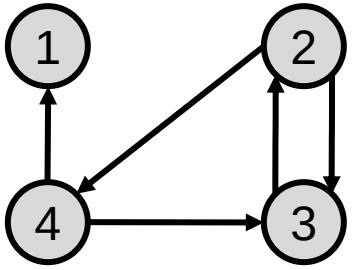
Exemplu (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(3)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$



$$T^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Algoritmul lui Johnson

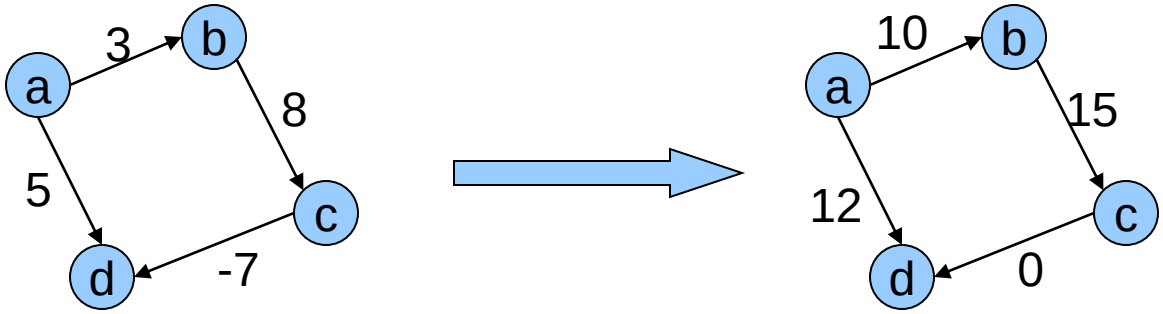
- Pentru grafuri rare.
- Folosește liste de adiacență.
- Bazat pe Dijkstra și Bellman-Ford.
- Complexitate: $O(V^2 \log V + VE)$
  - Mai bună decât Floyd-Warshall pentru grafuri rare.

Idee algoritm Johnson

- Dacă graful are numai arce pozitive:
  - se aplică Dijkstra pentru fiecare nod $\Longrightarrow$ cost $V^2 \log V$.
- Altfel se calculează costuri pozitive pentru fiecare arc menținând proprietățile:
  - $w_1(u,v) \geq 0, \forall (u,v) \in E$;
  - p este drum minim utilizând w $\Leftrightarrow$ p este drum minim utilizând w_1.

Construcție w_1 (I)

- Idee 1: identificare arcul cu cel mai mic cost – c; adunare la costul fiecărui arc valoarea c;



cost(a..b..d) < cost(a,d)

cost(a..b..d) > cost(a,d)

Nu funcționează!!!!

Construcție w_1 (II)

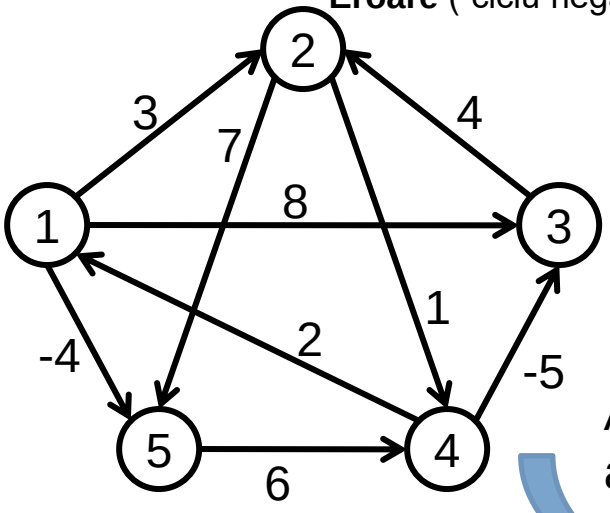
- Idee 2: $w_1(u,v) = w(u,v) + h(u) - h(v)$;
- unde $h : V \rightarrow \mathfrak{R}$;
- se adaugă un nod s;
- se unește s cu toate nodurile grafului prin arce de cost 0;
- se aplică BF pe acest graf => $h(v) = \delta(s,v)$;
- → $w_1(u,v) = w(u,v) + h(u) - h(v)$.

Algoritm Johnson

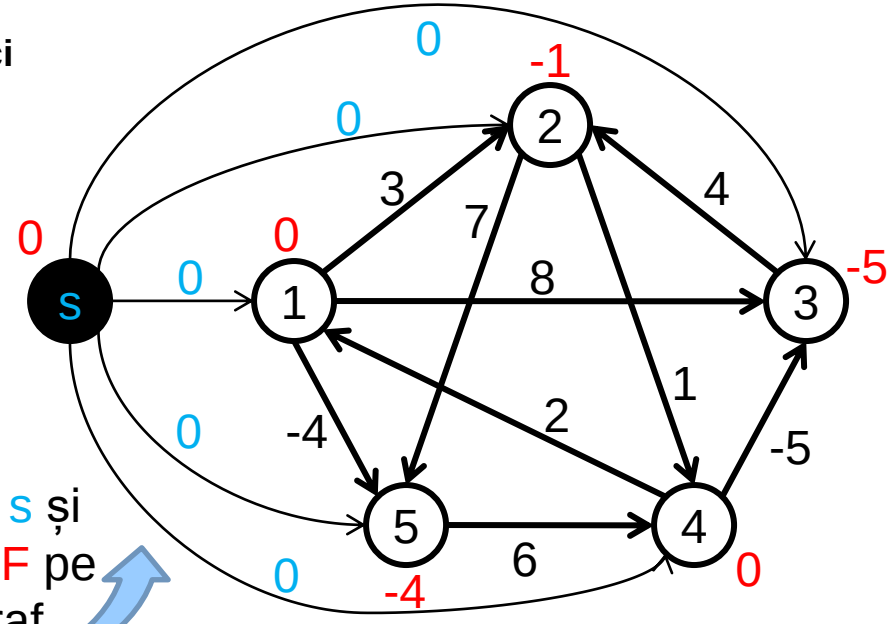
- Johnson(G)
  - // Construim G' = (V',E');
  - V' = V $\cup$ {s}; // adăugăm nodul s
  - E' = E $\cup$ (s,u), $\forall u \in V$; w(s,u) = 0; // și îl legăm de toate nodurile
  - **Dacă** BF(G',s) e fals // aplic BF pe G'
    - **Eroare** "ciclu negativ"
  - **Altfel**
    - **Pentru fiecare** v $\in$ V
      - h(v) = δ(s,v); // calculat prin BF
    - **Pentru fiecare** (u,v) $\in$ E
      - w_1(u,v) = w(u,v) + h(u) - h(v) // calculez noile costuri pozitive
    - **Pentru fiecare** (u $\in$ V)
      - Dijkstra(G,w_1,u) // aplic Dijkstra pentru fiecare nod
      - **Pentru fiecare** (v $\in$ V)
        - d(u,v) = δ_1(u,v) + h(v) - h(u) // calculez costurile pe graful inițial

Exemplu (I)

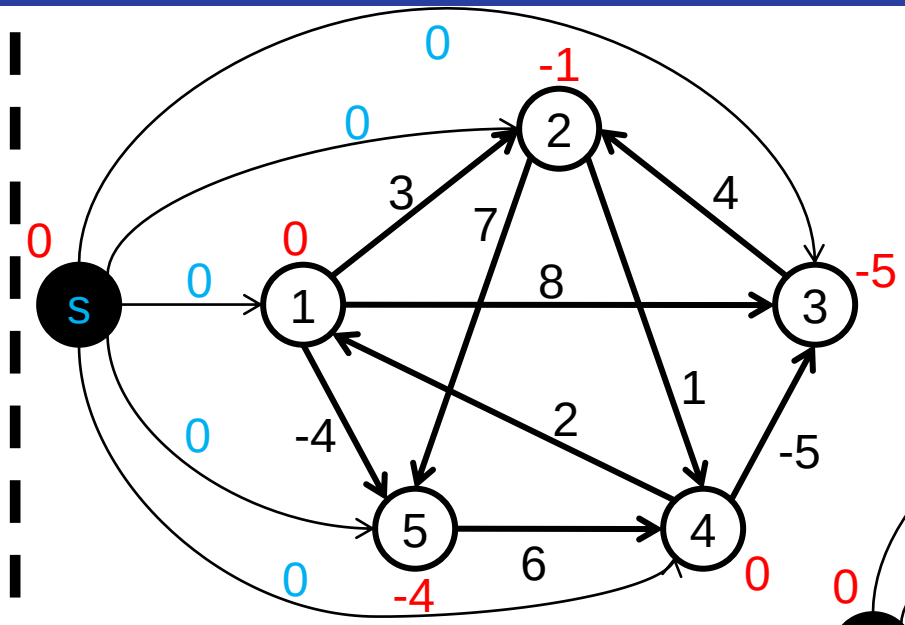
```
BellmanFord(G,s)   // G=(V,E),s=sursa
    Pentru fiecare v din V // inițializări
        d[v] = ∞; p[v] = null;
    d[s] = 0; // actualizare distanță de la s la s
    Pentru i de la 1 la |V| -1 // se construiesc drumuri optime de dimensiune i
        Pentru fiecare (u,v) din E  // pentru arcele ce pleacă de la nodurile deja considerate
            Dacă d[v] > d[u] + w(u,v) atunci // se relaxează arcele corespunzătoare
                d[v] = d[u] + w(u,v);  p[v] = u;
    Pentru fiecare (u,v) din E
        Dacă d[v] > d[u] + w(u,v) atunci
            Eroare ("ciclu negativ");
```



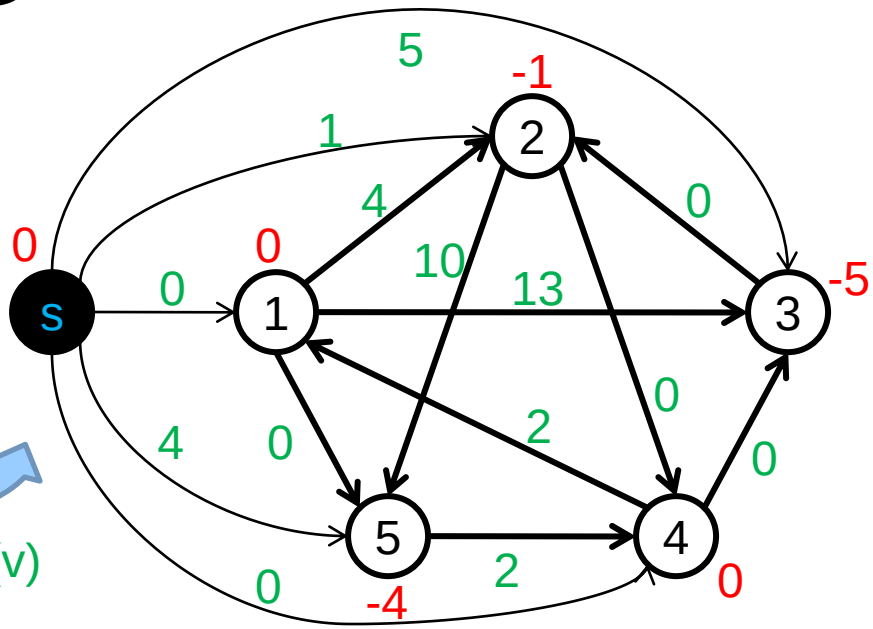
Adaug s și aplic BF pe noul graf.



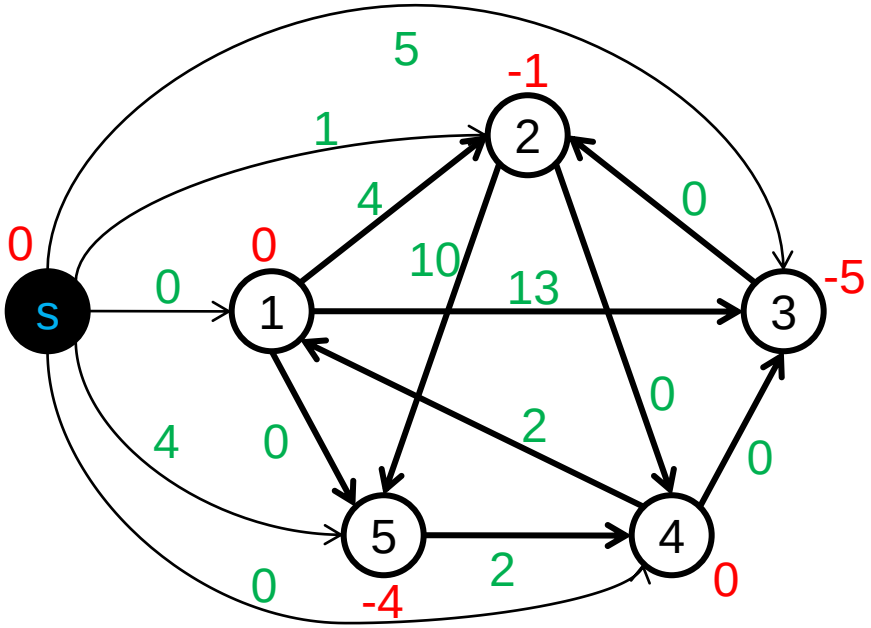
Exemplu (II)

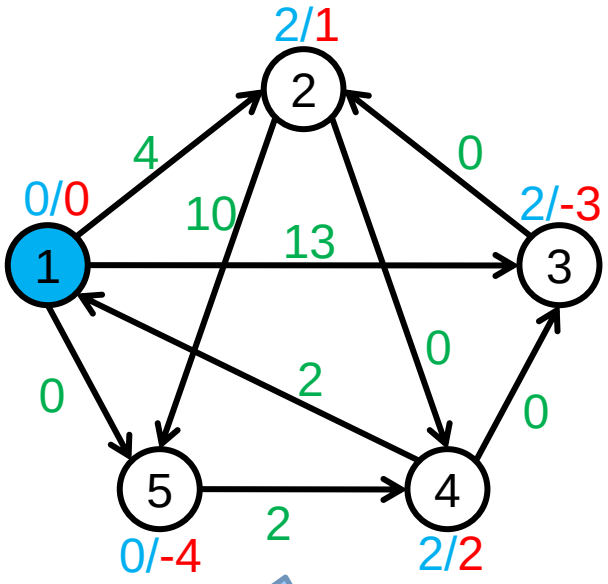


$w_1(u,v) = w(u,v) + h(u) - h(v)$



Exemplu (III)





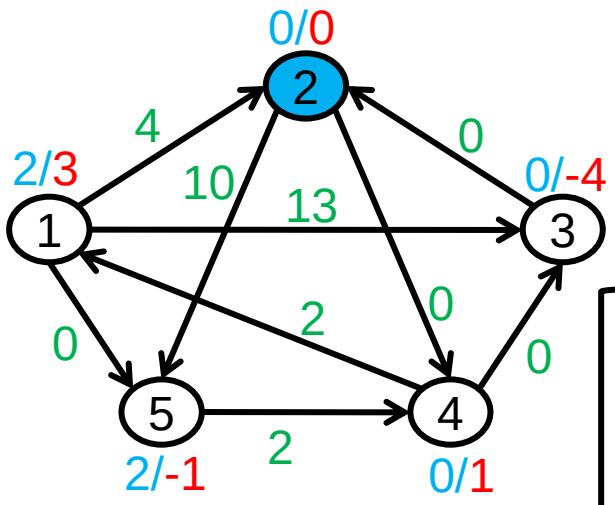
Eliminăm s

Aplicăm Dijkstra din fiecare nod ($\delta_1(u,v)$).

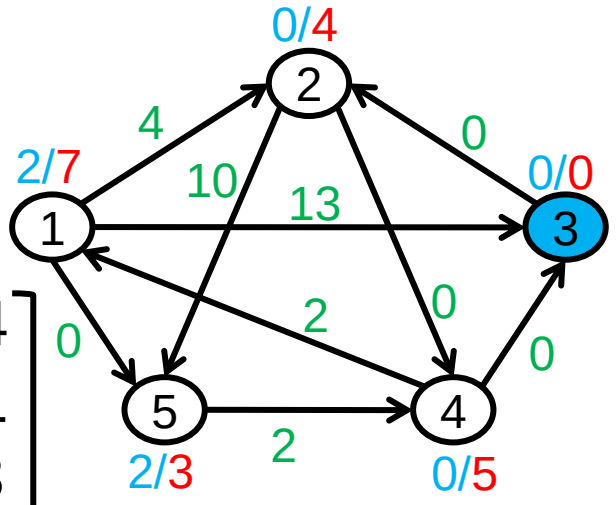
Refacem distanțele:

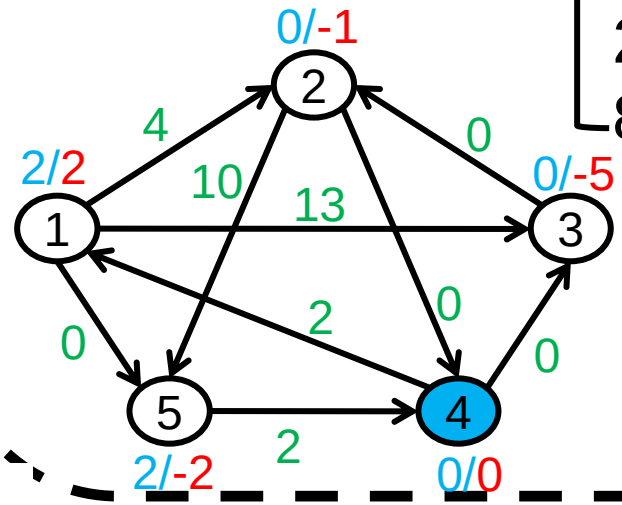
$$d(u,v) = \delta_1(u,v) + h(v) - h(u)$$

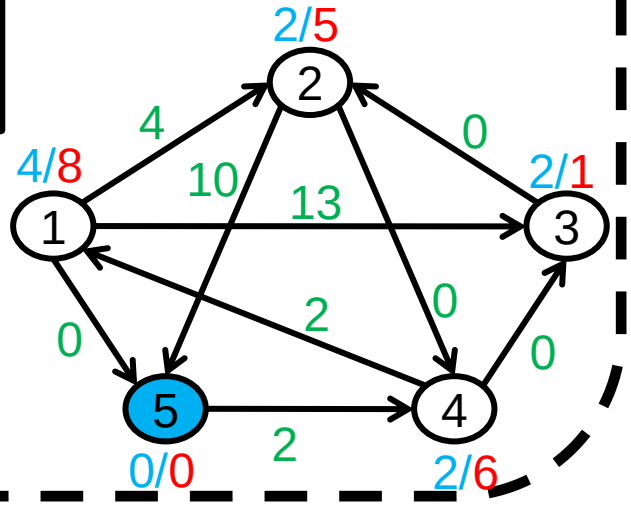
Exemplu (IV)



$$\begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$







Concluzii Floyd-Warshall & Johnson

- Algoritmi ce găsesc drumurile minime între oricare 2 noduri din graf.
- Funcționează pe grafuri cu arce ce au costuri negative (dar care nu au cicluri de cost negativ).
- Floyd-Warshall e optim pentru grafuri dese.
- Johnson e mai bun pentru grafuri rare.



ÎNTREBĂRI?



Bibliografie curs 10

[1] http://monalisa.cacr.caltech.edu/monalisa__Service_Applications__Monitoring_VRVS.html

[2] http://www.cobblestoneconcepts.com/ucgis2summer2002/guo/guo.html

[3] Giumale – Introducere in Analiza Algoritmilor cap. 5.5

[4] R. Sedgewick, K Wayne – curs de algoritmi Princeton 2007 www.cs.princeton.edu/~rs/AlgsDS07/ 01UnionFind si 14MST

[5] http://tagging.pui.ch/automated_tag_clustering

[6] Cormen – Introducere în Algoritmi cap. 24

Proiectarea Algoritmilor

Curs 10 – Drumuri de cost minim multi-punct multi-punct



Bibliografie

- [1] R. Sedgewick, K. Wayne - Algorithms and Data Structures Fall 2007 – Curs Princeton - http://www.cs.princeton.edu/~rs/AlgsDS07/
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*
- [3] Giumale – Introducere în Analiza Algoritmilor cap. 5.4, 5.4.2

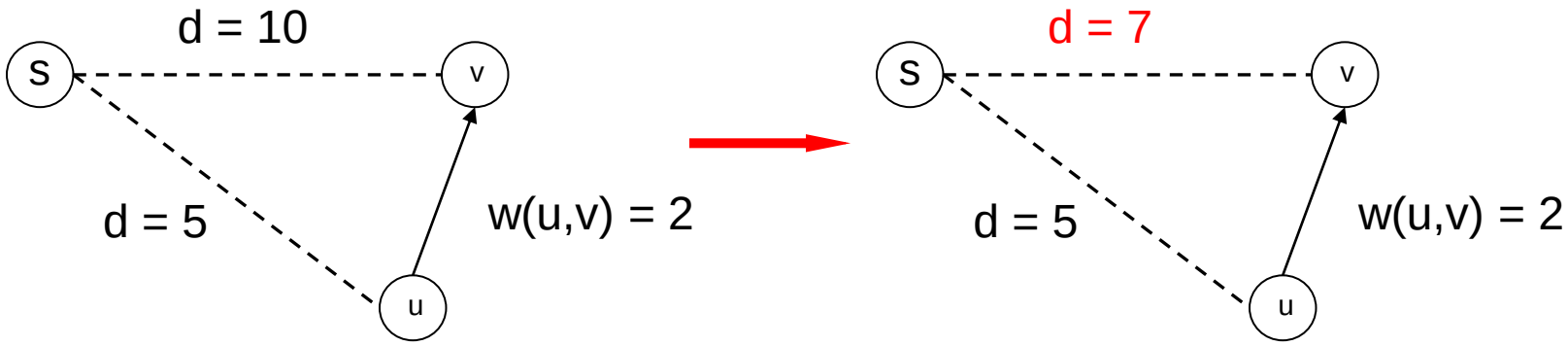
Obiective

- “Descoperirea” algoritmilor de identificare a drumurilor de cost minim.
- Recunoașterea caracteristicilor acestor algoritmi.



Relaxarea arcelor - Reminder

- Relaxarea arcelor:
  - **Dacă** d[v] > d[u] + w(u,v), **atunci**
    - d[v] = d[u] + w(u,v);
    - p[v] = u



Floyd-Warshall (Roy-Floyd)

- Algoritm prin care se calculează distanțele minime între oricare 2 noduri dintr-un graf (drumuri optime multipunct-multipunct).
- Exemplu clasic de programare dinamică.
- Idee: la pasul k se calculează cel mai bun cost între u și v folosind cel mai bun cost u..k și cel mai bun cost k..v calculat până în momentul respectiv.
- Se aplică pentru grafuri ce nu conțin cicluri de cost negativ.



Notații

- G = (V,E); V = {1, 2, …, n};
- w : V x V → $\Re$; w(i, i) = 0; w(i, j) = ∞ dacă (i,j) $\notin$ E;
- $d^k(i,j)$ = costul drumului i..j construit astfel încât drumul trece doar prin noduri din mulțimea {1, 2, .., k};
- δ(i,j) = costul drumului optim i..j; δ(i,j) = ∞ dacă ∄ i..j;
- $\delta^k(i,j)$ = costul drumului optim i..j ce trece doar prin noduri din mulțimea {1, 2, .., k}; $\delta^k(i,j)$ = ∞ dacă ∄ un astfel de drum i..j;
- $p^k(i,j)$ = predecesorul lui j pe drumul i..j ce trece doar prin noduri din mulțimea {1, 2, .., k}.

Teorema Floyd - Warshall

- Teoremă: Fie formulele de mai jos pentru calculul valorii $d^k(i,j)$, $0 \leq k \leq n$:
  - $d^0(i,j) = w(i,j)$;
  - $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$, pentru $0 < k \leq n$;

  Atunci $d^n(i,j) = \delta(i,j)$, pentru $\forall\ i, j \in V$

- Dem:
  - Prin inducție după k dem. că $d^k(i,j) = \delta^k(i,j)$. (next slide)
  - Pt. k = n, i..j trece prin $\forall\ v \in V$ si avem $d^k(i,j) \leq d^{k-1}(i,j)$, $\forall\ k = 1,n \rightarrow d^n(i,j) \leq d^{k-1}(i,j)$, $\forall\ k = 1,n$
  - Din $d^k(i,j) = \delta^k(i,j) \rightarrow d^n(i,j) = \delta^n(i,j) \leq d^{k-1}(i,j) = \delta^{k-1}(i,j)$, $\forall\ k = 1,n \rightarrow d^n(i,j) = \delta^n(i,j) = \delta(i,j)$

Demonstrație teorema Floyd - Warshall

- $k = 0$: 0 noduri intermediare → i..j = (i,j), la fel ca inițializarea $d^0(i,j) = w(i,j)$;
- $0 < k \leq n$: $d^{k-1}(i,j) = \delta^{k-1}(i,j)$ → $d^k(i,j) = \delta^k(i,j)$
- a) $k \notin$ drumului optim i..j: drumul optim nu se modifică

  $(\delta^{k-1}(i,j) = \delta^k(i,j) \leq \delta^{k-1}(i,k) + \delta^{k-1}(k,j))$
- $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$

  $d^k(i,j) = \min\{\delta^{k-1}(i,j), \delta^{k-1}(i,k) + \delta^{k-1}(k,j)\} = \delta^{k-1}(i,j) = \delta^k(i,j)$
- b) $k \in$ drumului optim i..j: i..j se descompune în i..k și k..j optime $(\delta^{k-1}(i,k) = d^{k-1}(i,k)$ și $\delta^{k-1}(k,j) = d^{k-1}(k,j))$ și

  $\delta^k(i,j) = \delta^{k-1}(i,k) + \delta^{k-1}(k,j)$.
- i..j optim → $\delta^k(i,j) \leq \delta^{k-1}(i,j)$
- $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$
- $d^k(i,j) = \min\{\delta^{k-1}(i,j), \delta^{k-1}(i,k) + \delta^{k-1}(k,j)\} = \delta^{k-1}(i,k) + \delta^{k-1}(k,j) = \delta^k(i,j)$

Algoritm Floyd-Warshall

- **Floyd-Warshall(G)**
- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n // inițializări
    - $d^0(i,j) = w(i,j)$
    - **Dacă** $(w(i,j) == \infty)$
      - $p^0(i,j) = null;$
    - **Altfel** $p^0(i,j) = i;$
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - **Dacă** $(d^{k-1}(i,j) > d^{k-1}(i,k) + d^{k-1}(k,j))$ // determinăm minimul
        - $d^k(i,j) = d^{k-1}(i,k) + d^{k-1}(k,j)$
        - $p^k(i,j) = p^{k-1}(k,j);$ // și actualizăm părintele
      - **Altfel**
        - $d^k(i,j) = d^{k-1}(i,j)$
        - $p^k(i,j) = p^{k-1}(i,j);$

Complexitate?

$O(V^3)$

Complexitate spațială?

$O(V^3)$

Observație

- Putem folosi o singură matrice în loc de n?
- Problemă: în pasul k, pentru k < i și k < j, d(i,k) și d(k,j) folosite la calculul d(i,j) sunt $d^k(k,j)$ și $d^k(i,k)$ în loc de $d^{k-1}(k,j)$ și $d^{k-1}(i,k)$. Dacă dem. că $d^k(k,j)= d^{k-1}(k,j)$ și $d^k(i,k)=d^{k-1}(i,k)$, atunci putem folosi o singură matrice.
- Dar: $d^k(i,j) = \min\{d^{k-1}(i,j), d^{k-1}(i,k) + d^{k-1}(k,j)\}$
  - $d^k(k,j) = \min\{d^{k-1}(k,j), d^{k-1}(k,k) + d^{k-1}(k,j)\} = d^{k-1}(k,j)$
  - $d^k(i,k) = \min\{d^{k-1}(i,k), d^{k-1}(i,k) + d^{k-1}(k,k)\} = d^{k-1}(i,k)$
- → Algoritm modificat pentru a folosi o singura matrice → complexitate spațială: $O(n^2)$.

Algoritm Floyd-Warshall

Floyd-Warshall2(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n // inițializări
    - d(i,j) = w(i,j)
    - **Dacă** (w(i,j) == ∞)
      - p(i,j) = null;
    - **Altfel** p(i,j) = i;
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - **Dacă** (d(i,j) > d(i,k) + d(k,j)) // determinăm minimul
        - d(i,j) = d(i,k) + d(k,j)
        - p(i,j) = p(k,j); // și actualizăm părintele

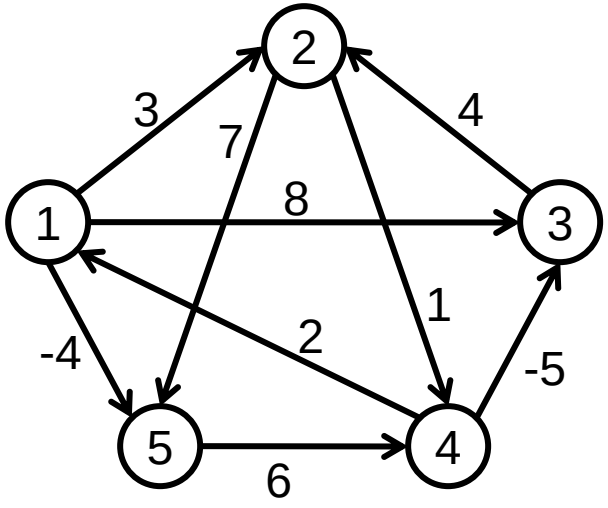
Complexitate?

$O(V^3)$

Complexitate spațială?

$O(V^2)$

Exemplu (I)

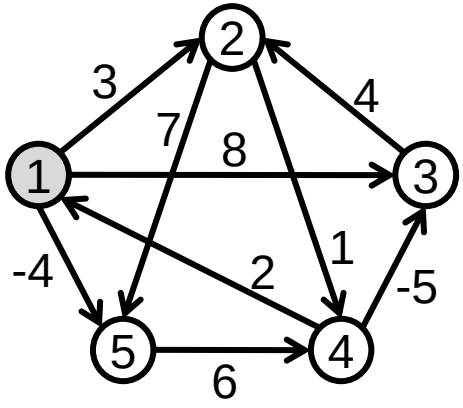


$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & nil & 4 & nil & nil \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (II)

$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$



$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

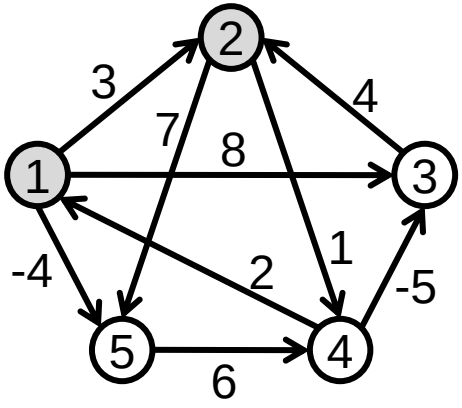
$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & nil & 4 & nil & nil \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (III)

$$D = \begin{bmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$



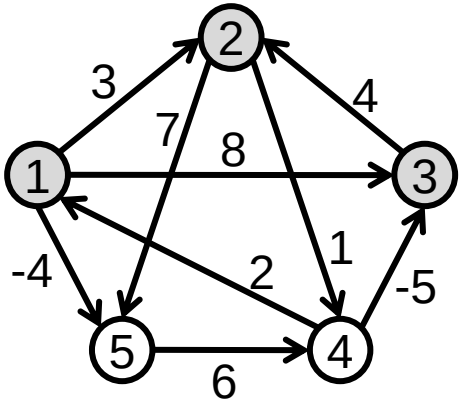
$$p = \begin{bmatrix} nil & 1 & 1 & nil & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & nil & nil \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (IV)

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 1 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$



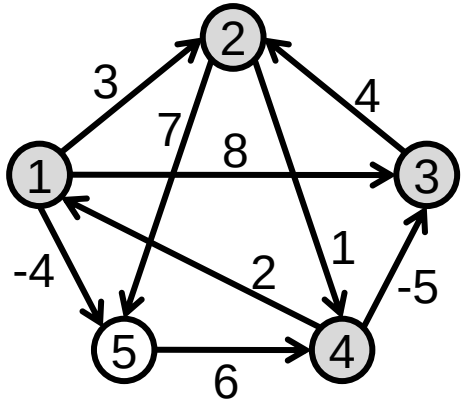
$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 3 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

Exemplu (V)

$$D = \begin{bmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

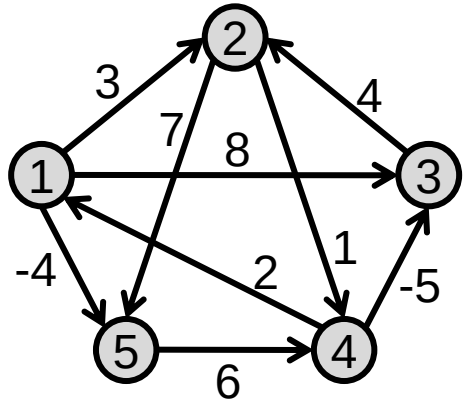


$$p = \begin{bmatrix} nil & 1 & 1 & 2 & 1 \\ nil & nil & nil & 2 & 2 \\ nil & 3 & nil & 2 & 2 \\ 4 & 3 & 4 & nil & 1 \\ nil & nil & nil & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 4 & 2 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

Exemplu (VI)

$$D = \begin{bmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$



$$D = \begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 1 & 4 & 2 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

$$p = \begin{bmatrix} nil & 3 & 4 & 5 & 1 \\ 4 & nil & 4 & 2 & 1 \\ 4 & 3 & nil & 2 & 1 \\ 4 & 3 & 4 & nil & 1 \\ 4 & 3 & 4 & 5 & nil \end{bmatrix}$$

Închiderea tranzitivă (I)

- Fie G = (V,E). Închiderea tranzitivă a lui E e un G* = (V,E*), unde

$$E^*(i,j)=\begin{cases}1, \text{ dacă } \exists\ i..j\\ 0, \text{ dacă } \nexists\ i..j\end{cases}$$

- Poate fi determinată prin modificarea algoritmului Floyd-Warshall:
  - min $\Longrightarrow$ operatorul boolean sau ($\vee$)
  - + $\Longrightarrow$ operatorul boolean și ($\wedge$)



Închiderea tranzitivă (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - $E^*(i,j) = (((i,j) \in E) \vee (i = j))$ // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - $E^*(i,j) = E^*(i,j) \vee (E^*(i,k) \wedge E^*(k,j))$

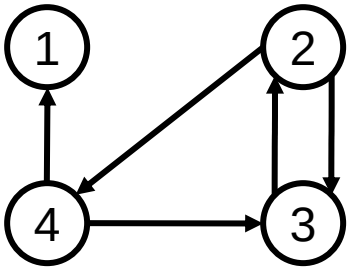
Complexitate? $O(V^3)$

Complexitate spațială? $O(V^2)$

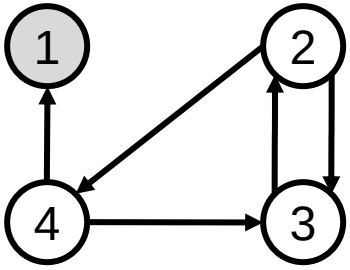
Exemplu (I)

Închidere_tranzitivă(G)

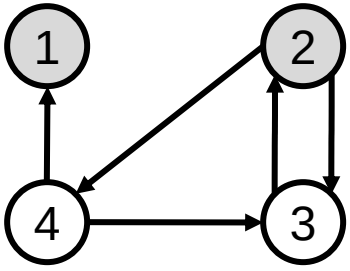
- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(0)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$



$$T^{(1)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

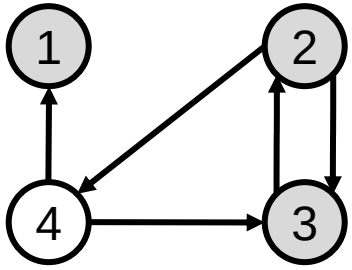


$$T^{(2)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

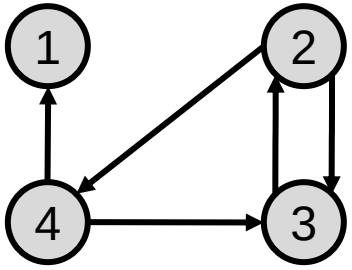
Exemplu (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(3)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

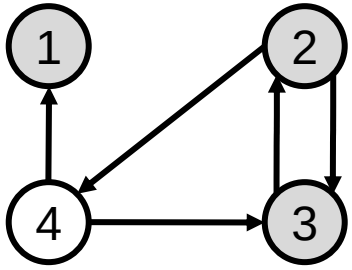


$$T^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

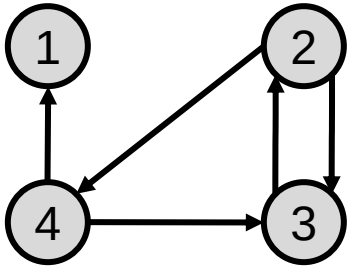
Exemplu (II)

Închidere_tranzitivă(G)

- **Pentru** i **de la** 1 **la** n
  - **Pentru** j **de la** 1 **la** n
    - E* (i,j) = (i,j) ∈ E ∨ i = j
      // inițializări
- **Pentru** k **de la** 1 **la** n
  - **Pentru** i **de la** 1 **la** n
    - **Pentru** j **de la** 1 **la** n
      - E* (i,j) = E* (i,j) ∨ (E* (i,k) ∧ E* (k,j))



$$T^{(3)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$



$$T^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Algoritmul lui Johnson

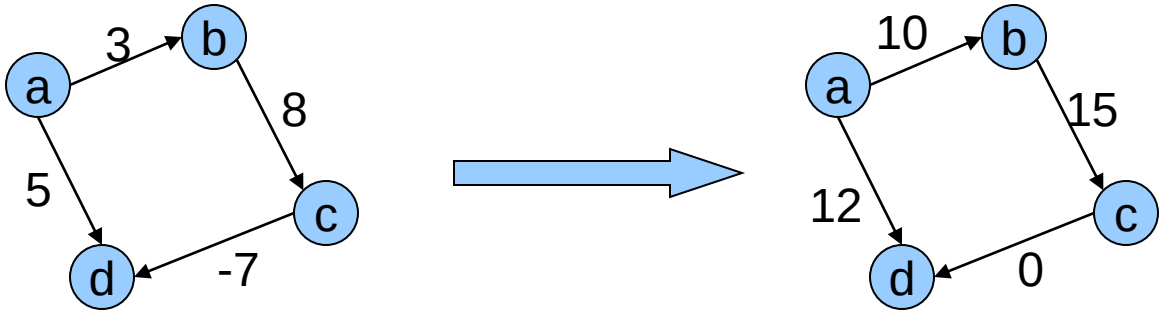
- Pentru grafuri rare.
- Folosește liste de adiacență.
- Bazat pe Dijkstra și Bellman-Ford.
- Complexitate: $O(V^2 \log V + VE)$
  - Mai bună decât Floyd-Warshall pentru grafuri rare.

Idee algoritm Johnson

- Dacă graful are numai arce pozitive:
  - se aplică Dijkstra pentru fiecare nod $\Longrightarrow$ cost $V^2 \log V$.
- Altfel se calculează costuri pozitive pentru fiecare arc menținând proprietățile:
  - $w_1(u,v) \geq 0, \forall (u,v) \in E$;
  - p este drum minim utilizând w $\Leftrightarrow$ p este drum minim utilizând w_1.

Construcție w_1 (I)

- Idee 1: identificare arcul cu cel mai mic cost – c; adunare la costul fiecărui arc valoarea c;



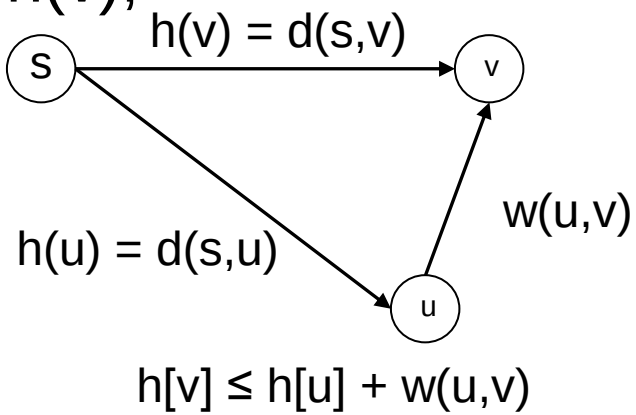
cost(a..b..d) < cost(a,d)

cost(a..b..d) > cost(a,d)

Nu funcționează!!!!

Construcție w_1 (II)

- Idee 2: $w_1(u,v) = w(u,v) + h(u) - h(v)$;
- unde $h : V \rightarrow \mathfrak{R}$;
- se adaugă un nod s;
- se unește s cu toate nodurile grafului prin arce de cost 0;
- se aplică BF pe acest graf => $h(v) = \delta(s,v)$;
- → $w_1(u,v) = w(u,v) + h(u) - h(v)$.

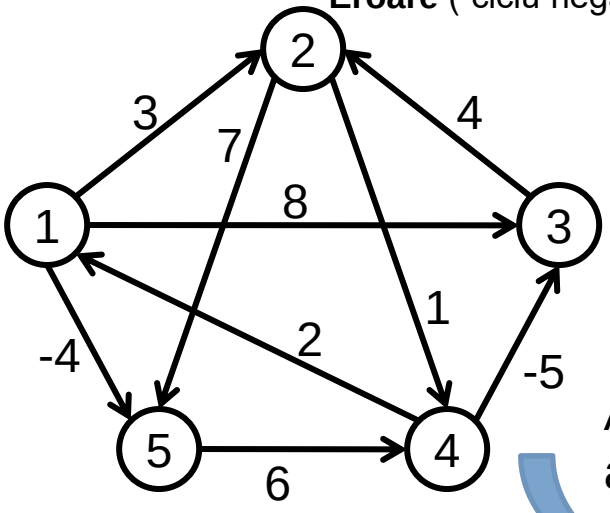


Algoritm Johnson

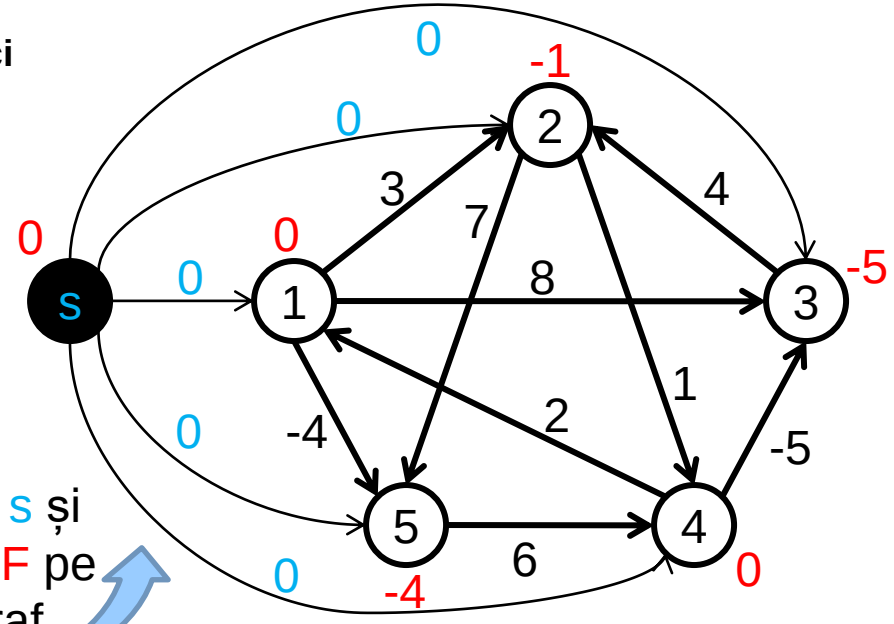
- Johnson(G)
  - // Construim G' = (V',E');
  - V' = V $\cup$ {s}; // adăugăm nodul s
  - E' = E $\cup$ (s,u), $\forall u \in V$; w(s,u) = 0; // și îl legăm de toate nodurile
  - **Dacă** BF(G',s) e fals // aplic BF pe G'
    - **Eroare** "ciclu negativ"
  - **Altfel**
    - **Pentru fiecare** v $\in$ V
      - h(v) = δ(s,v); // calculat prin BF
    - **Pentru fiecare** (u,v) $\in$ E
      - w_1(u,v) = w(u,v) + h(u) - h(v) // calculez noile costuri pozitive
    - **Pentru fiecare** (u $\in$ V)
      - Dijkstra(G,w_1,u) // aplic Dijkstra pentru fiecare nod
      - **Pentru fiecare** (v $\in$ V)
        - d(u,v) = δ_1(u,v) + h(v) - h(u) // calculez costurile pe graful inițial

Exemplu (I)

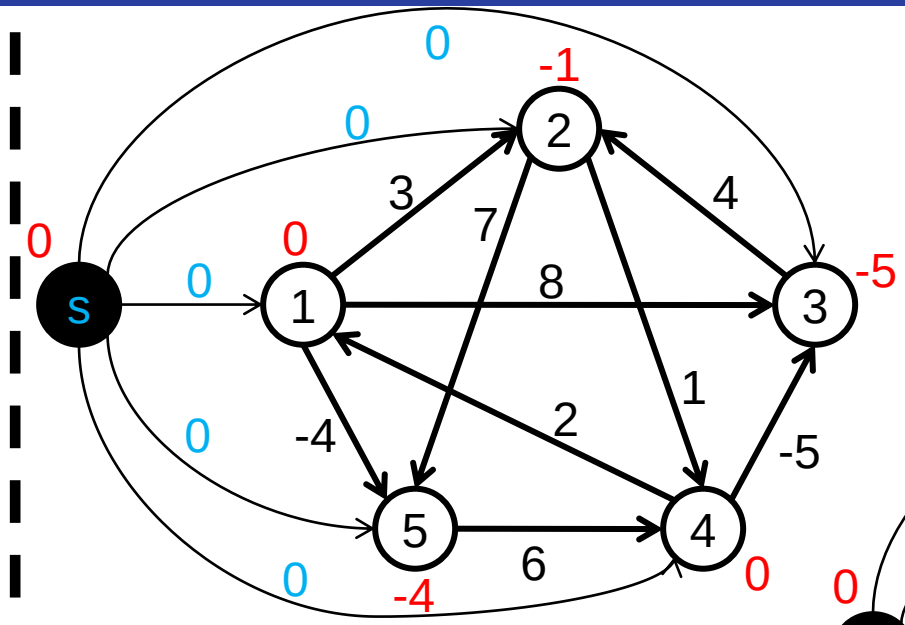
```
BellmanFord(G,s)   // G=(V,E),s=sursa
    Pentru fiecare v din V // inițializări
        d[v] = ∞; p[v] = null;
    d[s] = 0; // actualizare distanță de la s la s
    Pentru i de la 1 la |V| -1 // se construiesc drumuri optime de dimensiune i
        Pentru fiecare (u,v) din E  // pentru arcele ce pleacă de la nodurile deja considerate
            Dacă d[v] > d[u] + w(u,v) atunci // se relaxează arcele corespunzătoare
                d[v] = d[u] + w(u,v);  p[v] = u;
    Pentru fiecare (u,v) din E
        Dacă d[v] > d[u] + w(u,v) atunci
            Eroare ("ciclu negativ");
```



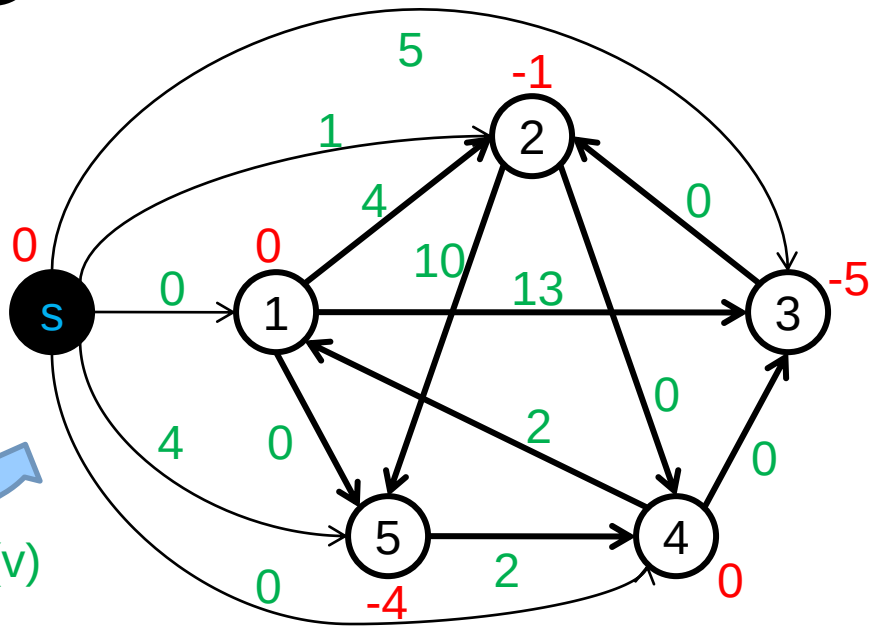
Adaug s și aplic BF pe noul graf.



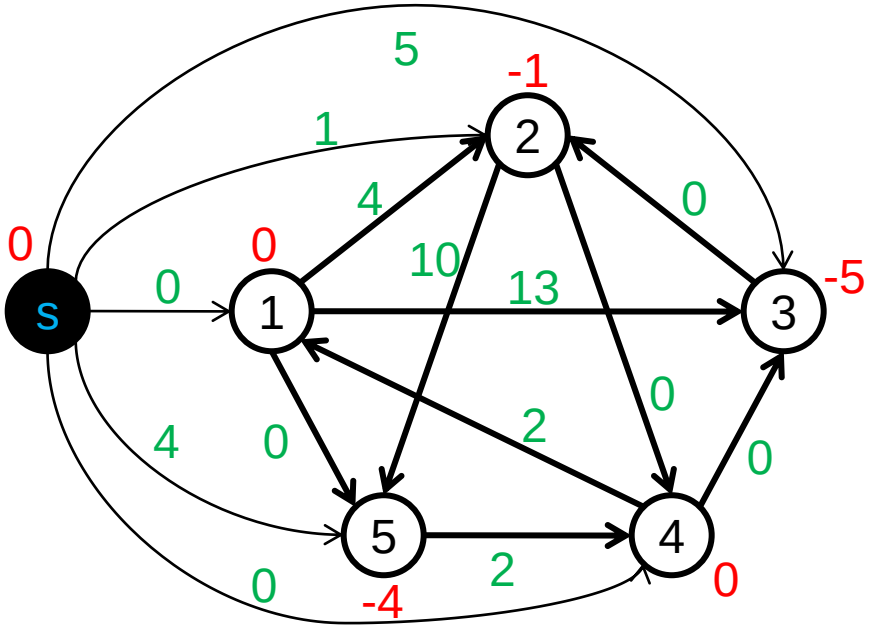
Exemplu (II)

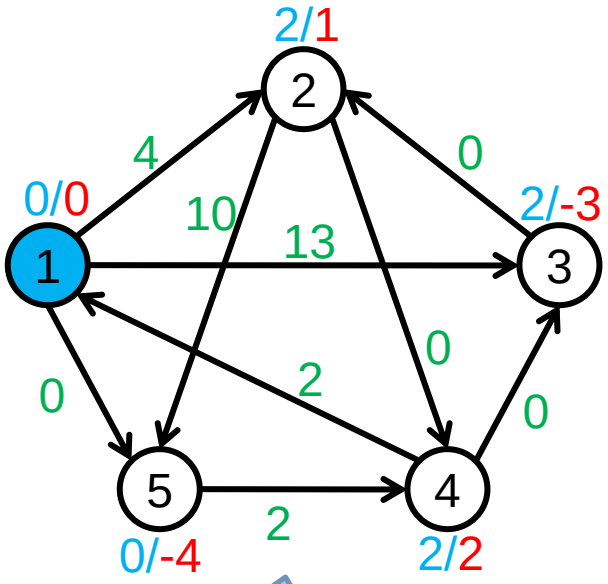


$w_1(u,v) = w(u,v) + h(u) - h(v)$



Exemplu (III)





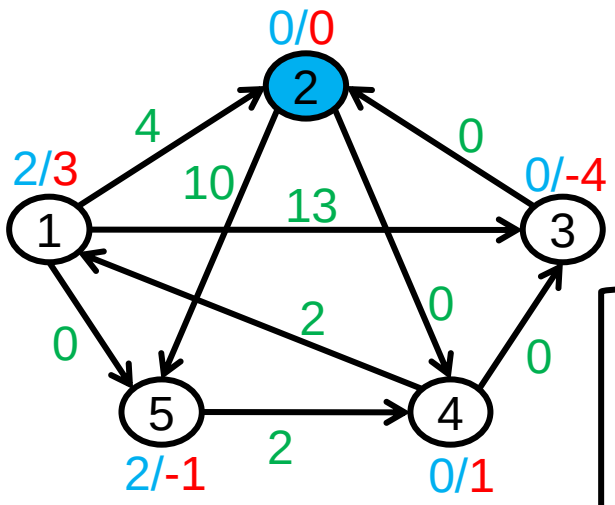
Eliminăm s

Aplicăm Dijkstra din fiecare nod ($\delta_1(u,v)$).

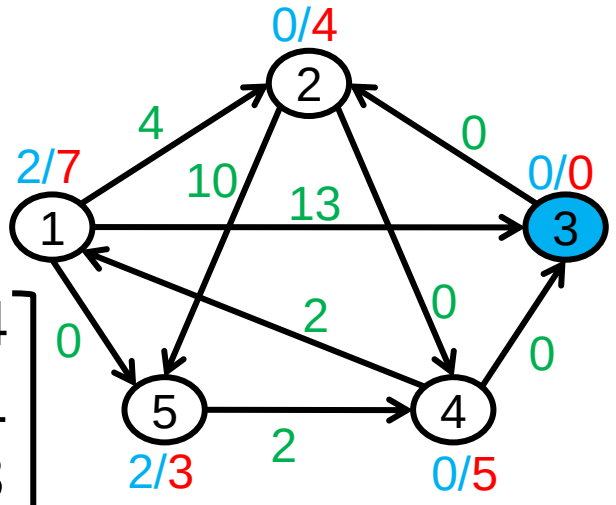
Refacem distanțele:

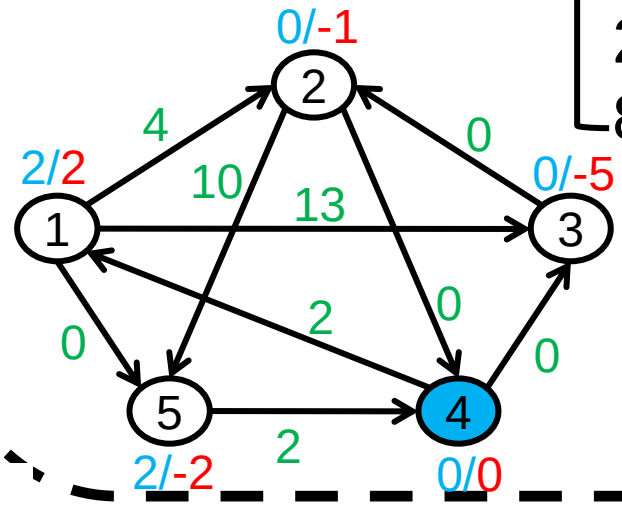
$$d(u,v) = \delta_1(u,v) + h(v) - h(u)$$

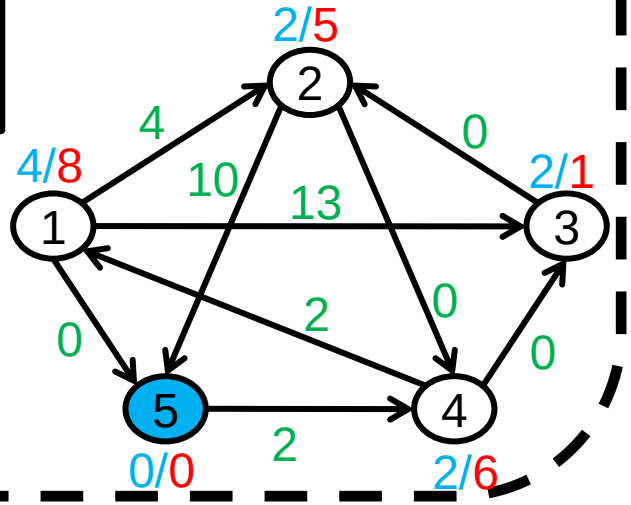
Exemplu (IV)



$$\begin{bmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{bmatrix}$$







Concluzii Floyd-Warshall & Johnson

- Algoritmi ce găsesc drumurile minime între oricare 2 noduri din graf.
- Funcționează pe grafuri cu arce ce au costuri negative (dar care nu au cicluri de cost negativ).
- Floyd-Warshall e optim pentru grafuri dese.
- Johnson e mai bun pentru grafuri rare.



ÎNTREBĂRI?



Bibliografie curs 11

[1] http://monalisa.cacr.caltech.edu/monalisa__Service_Applications__Monitoring_VRVS.html

[2] http://www.cobblestoneconcepts.com/ucgis2summer2002/guo/guo.html

[3] Giumale – Introducere in Analiza Algoritmilor cap. 5.5

[4] R. Sedgewick, K Wayne – curs de algoritmi Princeton 2007 www.cs.princeton.edu/~rs/AlgsDS07/ 01UnionFind si 14MST

[5] http://tagging.pui.ch/automated_tag_clustering

[6] Cormen – Introducere în Algoritmi cap. 24

Proiectarea Algoritmilor

Curs 11 – Arbori minimi de acoperire



Bibliografie

[1] http://monalisa.cern.ch/monalisa__Service_Applications__Monitoring_VRVS.html

[2] https://cdn.images.express.co.uk/img/dynamic/78/590x/secondary/Nuclear-war-USA-899096.jpg

[3] Giumale – Introducere in Analiza Algoritmilor cap. 5.5

[4] R. Sedgewick, K Wayne – curs de algoritmi Princeton 2007 www.cs.princeton.edu/~rs/AlgsDS07/ 01UnionFind.pdf si 14MST.pdf

[5] https://www.pinterest.co.uk/pin/516928863449789442/

[6] Cormen – Introducere în Algoritmi cap. Arbori de acoperire minimi (24)

[7] Tarjan, R. & Van Leeuwen, J. (1984). Worst-Case Analysis of Set Union Algorithms. Journal of the Assoaatton for Computmg Machinery, Vot. 31, No. 2, pp 245--281 https://www.researchgate.net/publication/220430653_Worst-case_Analysis_of_Set_Union_Algorithms

Planul cursului

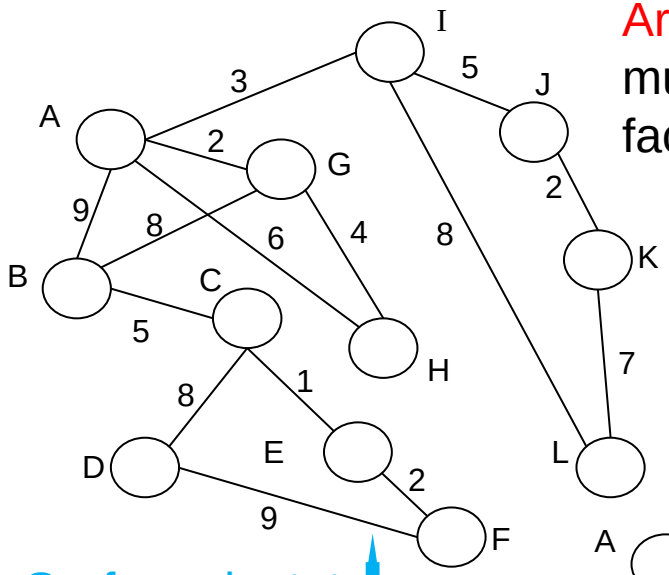
- Arbori minimi de acoperire:
  - Definiție;
  - Utilizare;
  - Algoritmi.
- Operații cu mulțimi disjuncte:
  - Structuri de date pentru reprezentarea mulțimilor disjuncte;
  - Algoritmi pentru reuniune și căutare;
  - Calcul de complexitate.



Arbori minimi de acoperire – Definiții

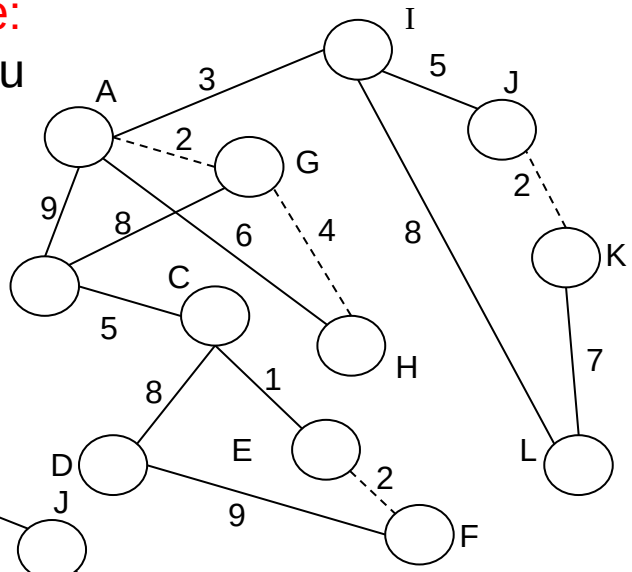
- Fie G = (V,E) graf neorientat și conex, iar w: E → $\mathfrak{R}$ o funcție de cost (w(u,v) = costul muchiei (u,v)).
- Definiție: Un arbore liber al lui G este un graf neorientat conex și aciclic Arb = (V',E'); V' $\subseteq$ V, E' $\subseteq$ E. Costul arborelui este: C(Arb) = Σ w(e), e $\in$ E'.
- Definiție: Un arbore liber se numește arbore de acoperire dacă V' = V.
- Definiție: Un arbore de acoperire (Arb) se numește arbore minim de acoperire (notăm AMA) dacă Arb $\in$ ARB(G) a.î. C(Arb) = min{C(Arb') | Arb' $\in$ ARB(G)}.

Exemple

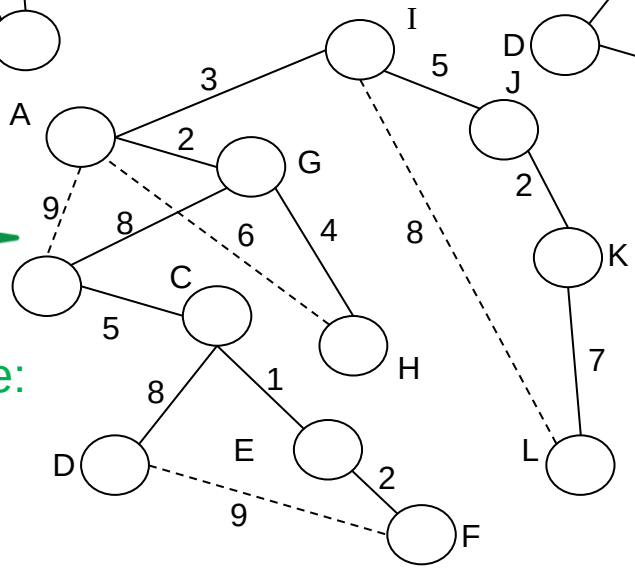


Graf neorientat conex

Arbore de acoperire:
muchiile punctate nu fac parte din arbore



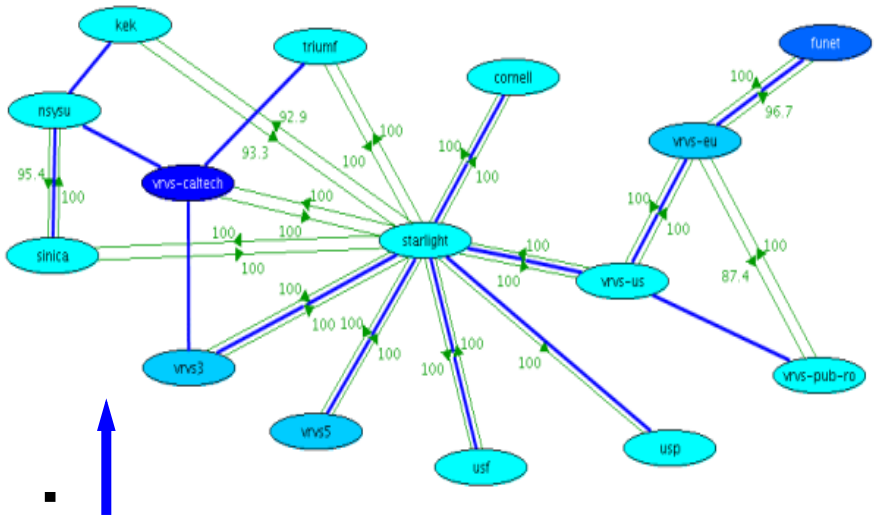
Arbore minim de acoperire:
muchiile punctate au fost eliminate din graf

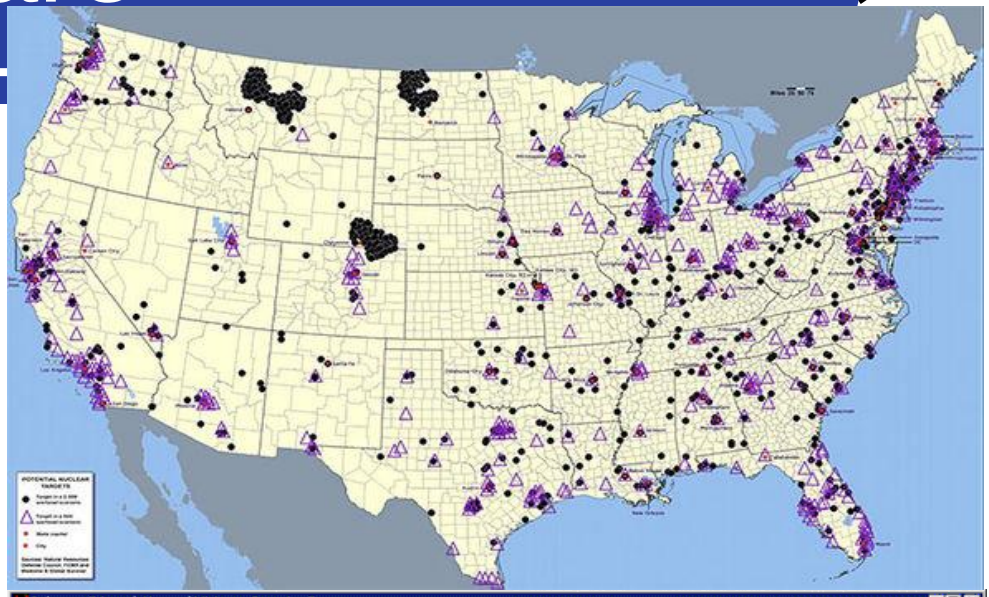


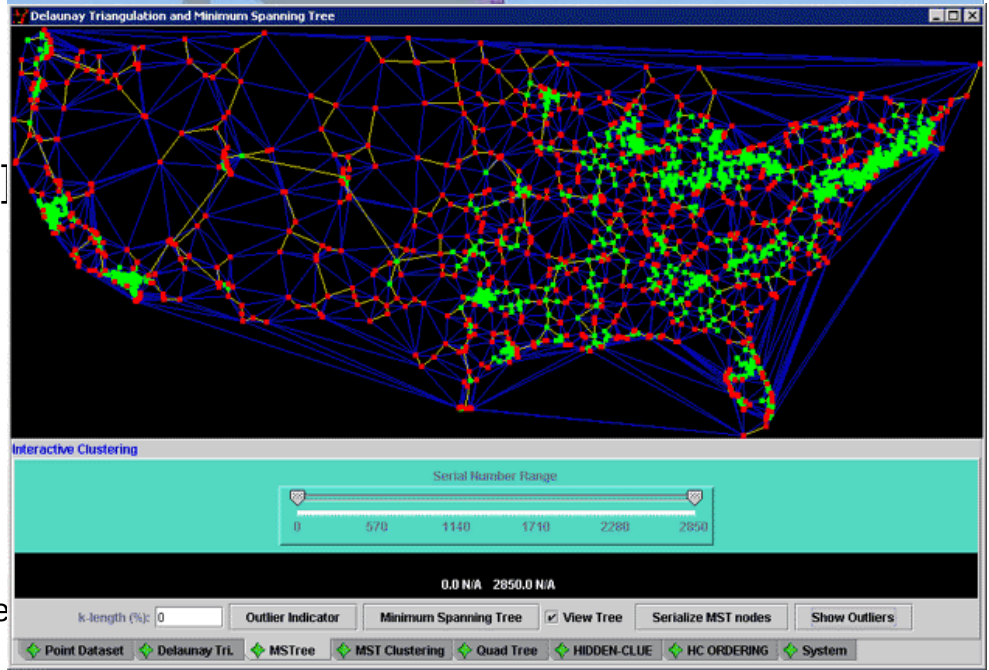
Utilizări

- Proiectarea rețelelor:
  - Electrice, calculatoare, drumuri.
- Clustering.
- Algoritmi de aproximare pentru probleme NP-complete.

Exemple de utilizare







- MonALISA - Arborele minim de acoperire al conexiunilor si calitatea conexiunilor peer-to-peer pentru un set de relee VRVS (caltech)[1]

- Cca 2850 de orașe din USA [2] (sus) împreună cu arbore minim de acoperire ale acestora (http://www.cobblestoneconcepts.com/ucgis2summer2002/guo/guo.html)

AMA – Definiții (II)

- Definiție: Fie $A \subseteq E$ o mulțime de muchii ale unui graf $G = (V,E)$ și $(S,V-S)$ o partiționare a lui V. Partiționarea respectă mulțimea A dacă $\nexists e \in A$ care taie frontiera dintre S și $V-S$ ($\forall (u,v) \in A \rightarrow u,v \in S$ sau $u,v \in V-S$).

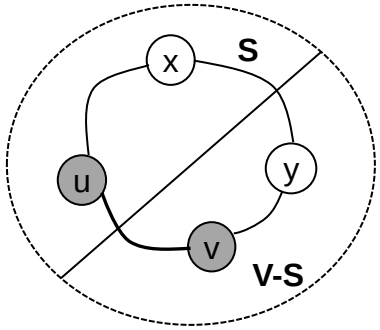
- Definiție: Fie $A \subseteq E'$ o mulțime de muchii ale unui AMA parțial $Arb = (V,E')$ al grafului $G = (V,E)$, iar $e \in E$ o muchie oarecare din G. Muchia e este sigură în raport cu A dacă mulțimea $A \cup \{e\}$ face parte dintr-un AMA al lui G.

AMA – Teoremă

- Teorema 5.23: Fie A o mulțime de muchii ale unui AMA parțial al grafului G = (V,E). Fie (S,V-S) o partiționare care respectă A, iar (u,v) ∈ E o muchie care taie frontiera dintre S și V-S a.î.

w(u,v) = min{w(x,y) | (x,y) ∈ E și (x ∈ S, y ∈ V-S) sau (x ∈ V-S, y ∈ S)}

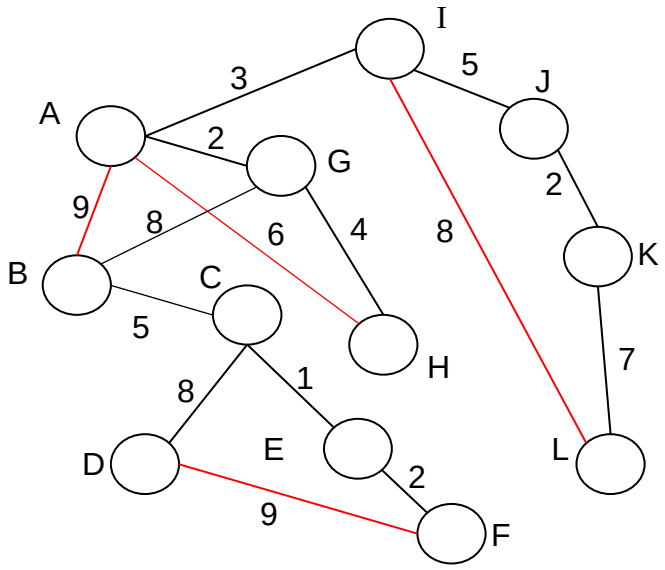
Muchia (u,v) este sigură in raport cu A.



- Dem (Reducere la absurd):
  - pp (u,v) nu e muchie sigură.
  - (I) → ∃ AMA Arb' = (V,E'), a.î. A ⊆ E'. Pp (u,v) ∉ Arb'
  - În Arb' ∃ cale u..v → ∃ (x,y) ∈ u..v care taie partiționarea și (x,y) ∈ Arb'
  - (x,y) ∉A, (u,v) ∉ A pt. că partiționarea respectă A, iar w(u,v) ≤ w(x,y) (I)
  - Dacă în Arb' eliminăm (x,y) și adăugăm (u,v) → Arb" = (V,E"), E" = E' – {(x,y)} + {(u,v)}
  - C(Arb") ≤ C(Arb'), Arb' – AMA → C(Arb') = C(Arb") → Arb" – AMA → (u,v) – muchie sigură.

Proprietăți (I)

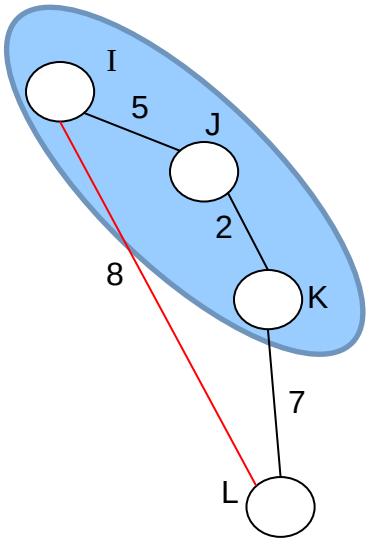
G = (V,E), C = (V',E') – ciclu în G; e ∈ E' a.î. w(e) = max {w(e') | e' ∈ E'} => e ∉ Arb(G) unde Arb(G) = AMA în G.



- Dem (Reducere la absurd): Pp e ∈ Arb(G).
- Eliminând e din Arb(G) → 2 mulțimi de muchii: S_1, S_2.
- e ∈ E' (ciclu) → ∃ e' ∈ E', w(e) > w(e') a.î. un capăt din e' este în S_1 și celalalt în S_2.
- Arb(G) – e + e' = arbore de acoperire.
- Cost(Arb(G) - w(e) + w(e') < Cost(Arb(G)) => Arb(G) nu este arbore minim.

Proprietăți (II)

G = (V,E), S = (V',E') un AMA parțial al lui G, V' $\subset$ V; e = (u,v) a.î. e $\notin$ E' și (u $\in$ V' și v $\notin$ V') sau (u $\notin$ V' și v $\in$ V') cu proprietatea că: w(u,v) = min{w(u',v')| (u'$\in$V' și v'$\notin$V') sau (u'$\notin$V' și v'$\in$V')} => (u,v) $\in$ AMA.



•Dem (Reducere la absurd): Pp e $\notin$ AMA

•Fie AMA Arb(G) și Arb' = Arb(G) – e' + e (unde e' o muchie similară cu e).

•Arb'= arbore de acoperire.

•Cost(Arb') <= Cost(Arb) → Arb(G) este AMA → Arb' este AMA → e ϵ Arb' → e ϵ AMA

AMA

- Bazați pe ideea de muchie sigură – se identifică o muchie sigură și se adaugă în AMA.

- 2 algoritmi de tip greedy:
  - Prim: se pornește cu un nod și se extinde pe rând cu muchiile cele mai ieftine care au un singur capăt în mulțimea de muchii deja formată (Proprietatea 2). Algoritmul este asemănător algoritmului Dijkstra.

  - Kruskal: inițial toate nodurile formează câte o mulțime și la fiecare pas se reunesc 2 mulțimi printr-o muchie. Muchiile sunt considerate în ordinea costurilor și sunt adăugate în arbore doar dacă nu creează ciclu (Proprietatea 1).

Algoritmul lui Prim

- Prim(G,w,s)

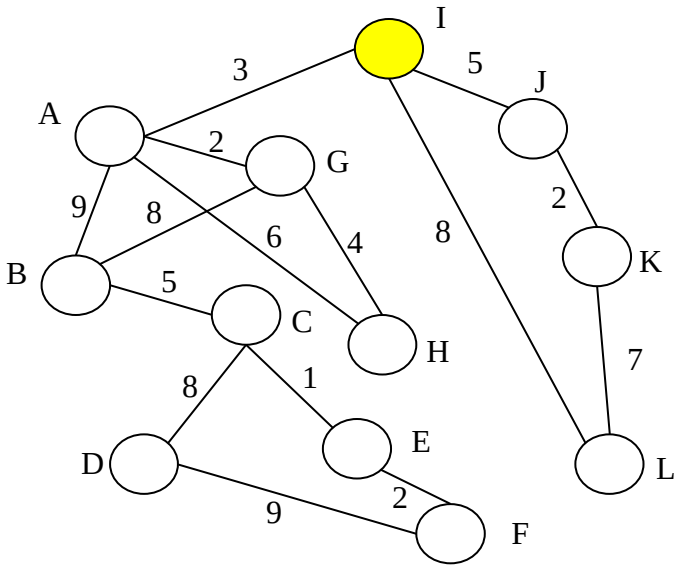
Implementare în Java la [4] !

  - A = ∅ // inițializare AMA
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null // inițializăm distanța și părintele
  - d[s] = 0; // nodul de start are distanța 0
  - Q = constrQ(V, d); // ordonată după costul muchiei
    // care unește nodul de AMA deja creat
  - **Cât timp** (Q != ∅) // cât timp mai sunt noduri neadăugate
    - u = ExtrageMin(Q); // extrag nodul aflat cel mai aproape
    - A = A ∪ {(u,p[u])}; // adaug muchia în AMA
    - **Pentru fiecare** (v ∈ succs(u))
      - **Dacă** d[v] > w(u,v) **atunci**
        - d[v] = w(u,v); //+ d[u] // actualizăm distanțele și părinții nodurilor
        - p[v] = u; // adiacente care nu sunt în AMA încă
  - **Întoarce** A - {(s,p(s))} // prima muchie adăugată



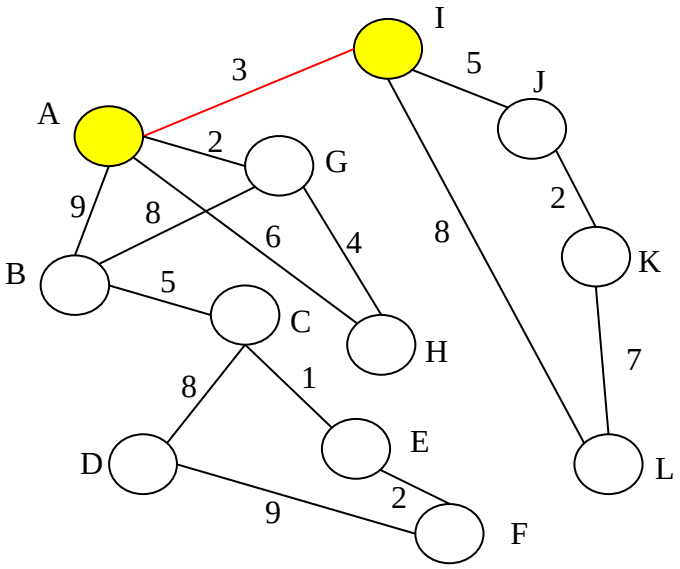
Exemplu (I)

- Pornim din I



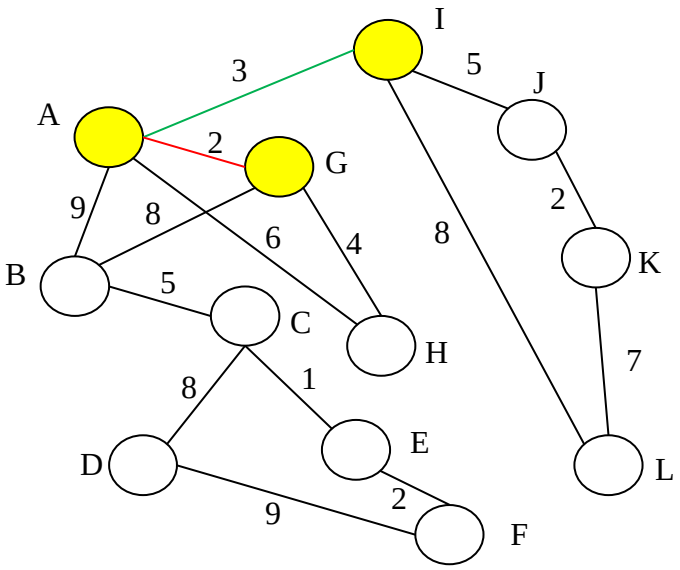
- Q: A(3), J(5), L(8), B(∞), C(∞), D(∞), E(∞), F(∞), G(∞), H(∞), K(∞) → A

Exemplu (II)



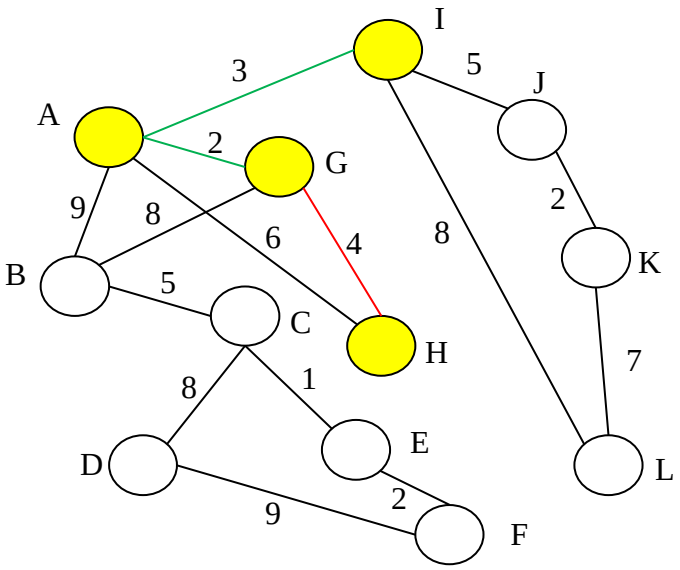
- Q: G(2), J(5), H(6), L(8), B(9), C(∞), D(∞), E(∞), F(∞), K(∞) → G

Exemplu (III)



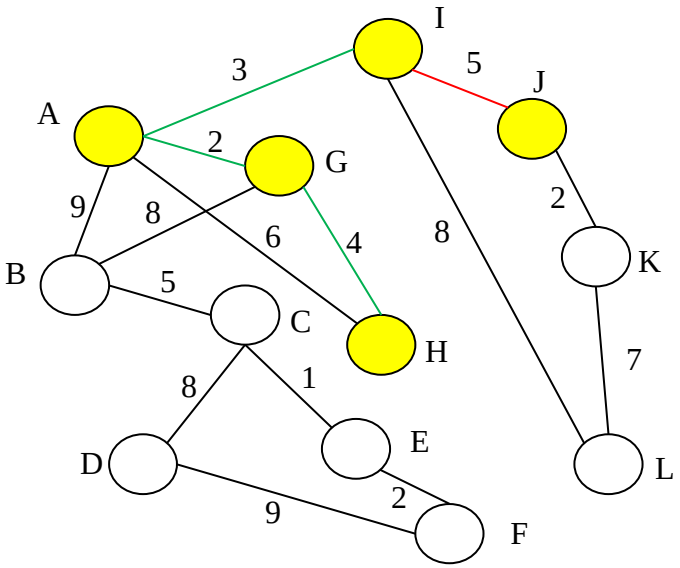
- Q: G(2), J(5), H(6), L(8), B(9), C(∞), D(∞), E(∞), F(∞), K(∞) → G
- Q: H(4), J(5), L(8), B(8), C(∞), D(∞), E(∞), F(∞), K(∞) → H

Exemplu (IV)



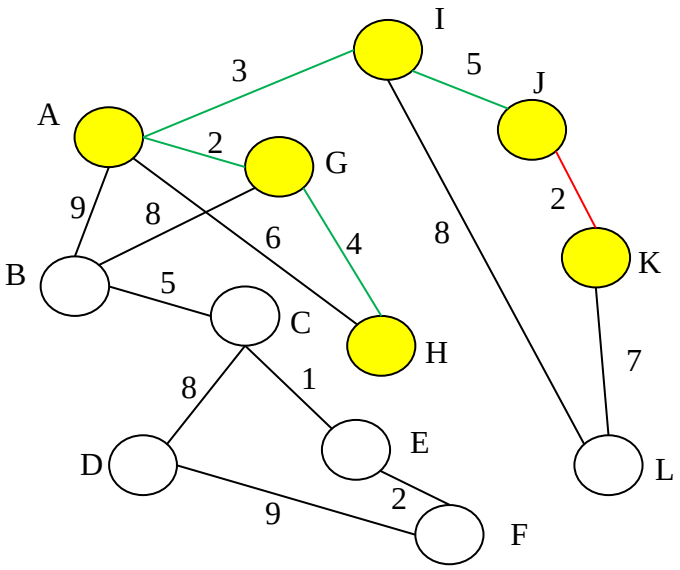
- Q: J(5), L(8), B(8), C(∞), D(∞), E(∞), F(∞), K(∞) → J

Exemplu (V)



- Q: K(2), L(8), B(8), C(∞), D(∞), E(∞), F(∞)
  → K

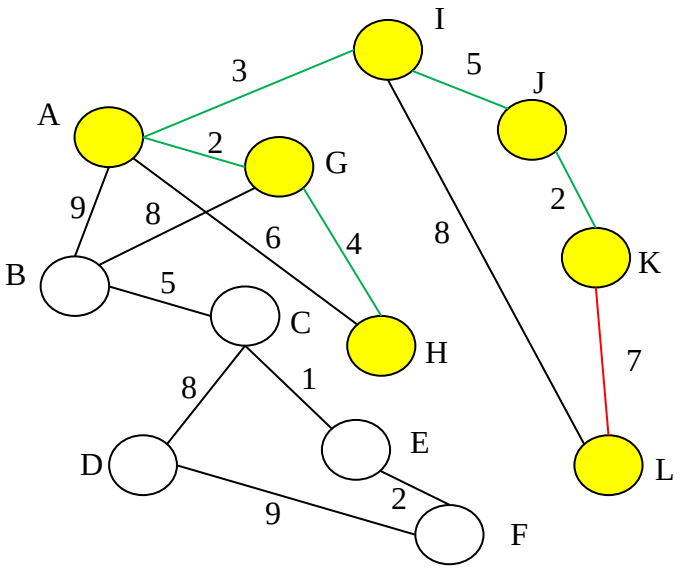
Exemplu (VI)



- Q: K(2), L(8), B(8), C(∞), D(∞), E(∞), F(∞) → K

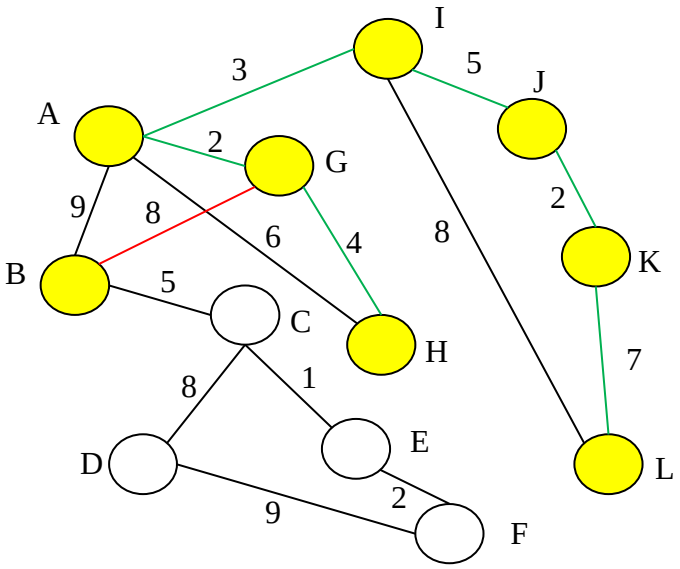
- Q: L(7), B(8), C(∞), D(∞), E(∞), F(∞) → L

Exemplu (VII)



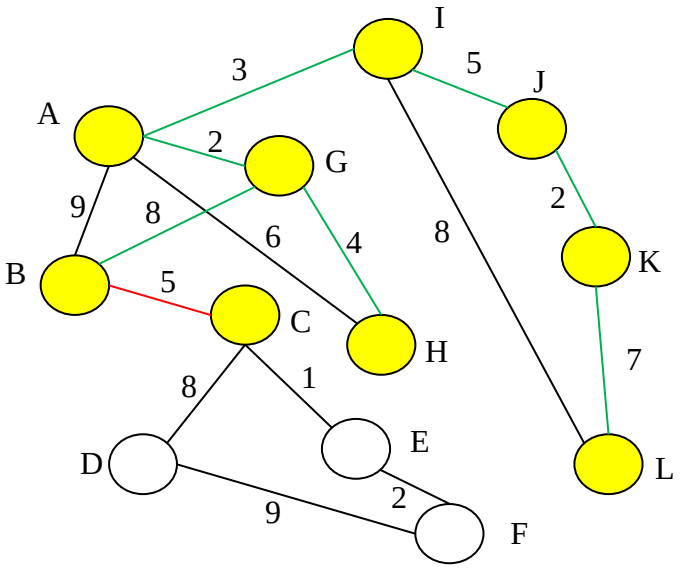
- Q: B(8), C(∞), D(∞), E(∞), F(∞) → B

Exemplu (VIII)



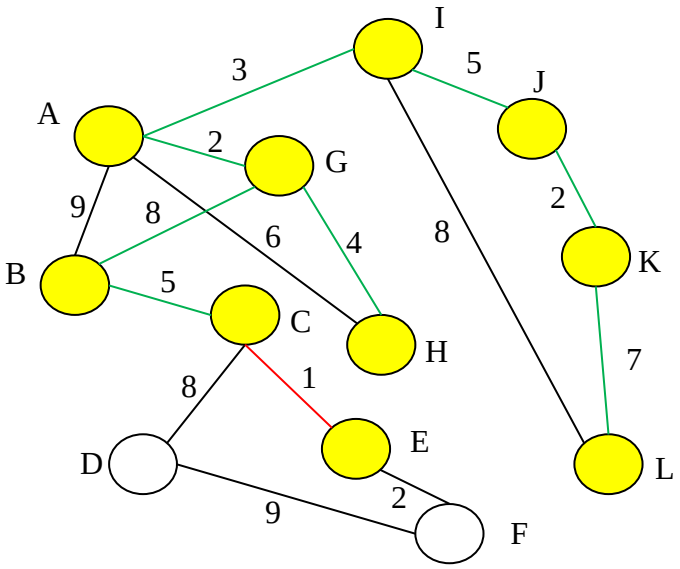
- Q: C(5), D(∞), E(∞), F(∞) → C

Exemplu (IX)



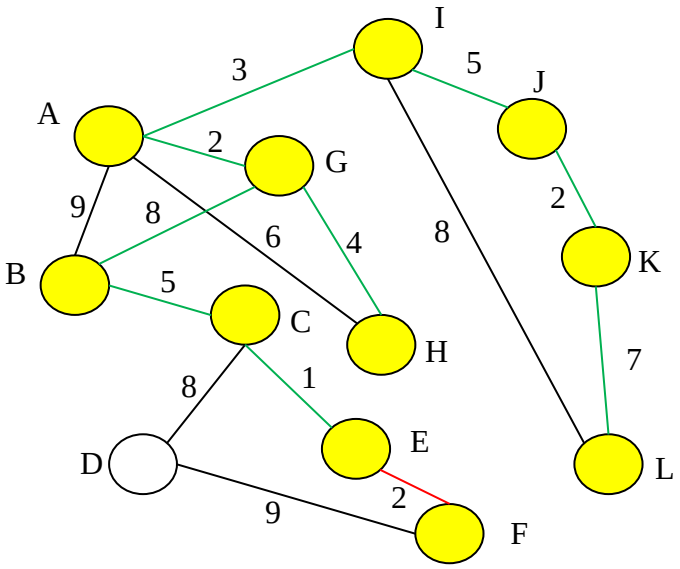
- Q: E(1), D(8), F(∞) → E

Exemplu (X)



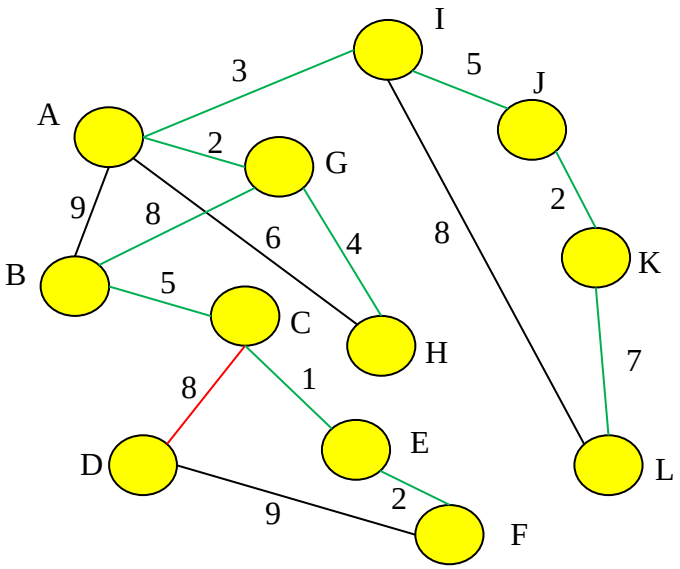
- Q: F(2), D(8) → F

Exemplu (XI)



- Q: D(8) → D

Exemplu (XII)



- Q: Ø

Corectitudine (I)

- 1. Arătăm că muchiile pe care le adăugăm aparțin Arb:
- Dem prin inducție după muchiile adăugate în AMA:
- P_1: avem V' = s, E' = ∅. Adaug muchia (u,s), u = nod adiacent sursei aflat cel mai aproape de aceasta → din Propr. 2 → (u,s) ∈ Arb.
- $P_n \rightarrow P_{n+1}$:
  - S = (V',E') mulțimea vârfurilor și muchiilor adăugate deja în arbore înainte de a adăuga (u,p[u]).
  - p[u] ∈ V', u ∉ V'; (u,p[u]) are cost minim dintre muchiile care au un capăt în S (conform extrage minim)
  - din Propr. 2 → (u,p[u]) ∈ Arb

Corectitudine (II)

- 2. arătăm că muchiile ignorate nu fac parte din Arb:
  - d[v] scade tot timpul de-a lungul algoritmului până când v este adăugat în AMA. În momentul adăugării, s-a găsit muchia de cost minim ce conectează nodul v la AMA;
  - Pp. (u,v) a.î. Arb(u) = Arb(v)
    - → (u,v) creează un ciclu în Arb(u) (arborii sunt aciclici) – fie ciclul format din u..x..v și (u,v).
    - w(u,v) = max {w(u',v') | (u',v') ∈ Arb(u)} DE CE?
      - Nodul u i-a fost adiacent nodului v, dar nu a fost ales la niciunul din momentele ulterioare de timp, când au fost parcurse muchiile din u..x..v → (u,v) are costul maxim din ciclu
    - → din Propr. 1 → (u,v) ∉ Arb

Algoritmul lui Prim

- Prim(G,w,s)
  - A = ∅ // AMA
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null // inițializăm distanța și părintele
  - d[s] = 0; // nodul de start are distanța 0
  - Q = constrQ(V, d); // ordonată după costul muchiei
    // care unește nodul de AMA deja creat
  - **Cât timp** (Q != ∅) // cât timp mai sunt noduri neadăugate
    - u = ExtrageMin(Q); // extrag nodul aflat cel mai aproape
    - A = A ∪ {(u,p[u])}; // adaug muchia în AMA
    - **Pentru fiecare** (v ∈ succs(u))
      - **Dacă** d[v] > w(u,v) **atunci**
        - d[v] = w(u,v); //+ d[u] // actualizăm distanțele și părinții nodurilor
        - p[v] = u; // adiacente care nu sunt în AMA încă
  - **Întoarce** A - {(s,p(s))} // prima muchie adăugată

Complexitate?



Reminder Dijkstra (II)

- Dijkstra(G,s)
  - **Pentru fiecare** (u ∈ V)
    - d[u] = ∞; p[u] = null;
  - d[s] = 0;
  - Q = construiește_coada(V) // coadă cu priorități
  - **Cât timp** (Q != ∅)
    - u = ExtrageMin(Q); // extrage din V elementul cu d[u] minim
    - // Q = Q - {u} – se execută în cadrul lui ExtrageMin
    - **Pentru fiecare** (v ∈ Q și v din succesorii lui u)
      - **Dacă** (d[v] > d[u] + w(u,v))
        - d[v] = d[u] + w(u,v) // actualizez distanța
        - p[v] = u // și părintele

Complexitate Prim

- Depinde de implementare (vezi Dijkstra)
  - Matrice de adiacență $O(V^2)$
  - Heap binar $O(ElogV)$
  - Heap Fibonacci $O(VlogV+E)$

- Concluzii
  - Grafuri dese
    - Matrice de adiacență preferată
  - Grafuri rare
    - Heap binar sau Fibonacci



Algoritmul lui Kruskal

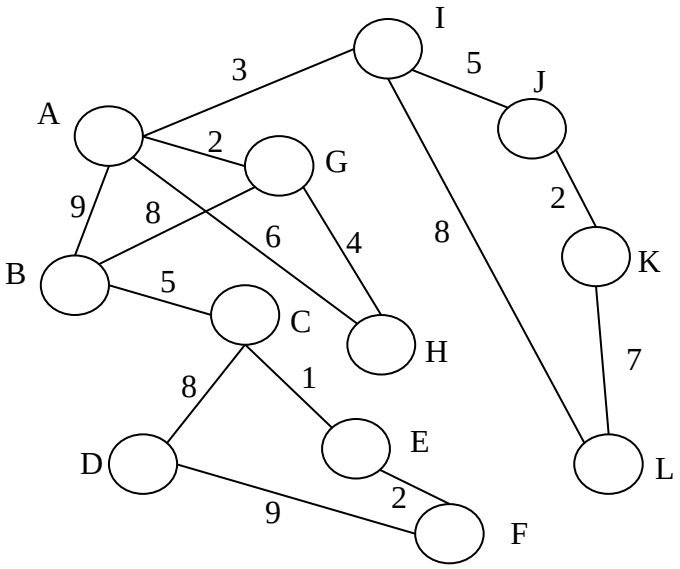
- Kruskal(G,w)

Implementare în Java la [4] !

  - A = ∅; // inițializare AMA
  - **Pentru fiecare** (v ∈ V)
    - Constr_Arb(v) // creează o mulțime formată din nodul respectiv // (un arbore cu un singur nod)
  - Sortează_asc(E,w) // se sortează muchiile în funcție de // costul lor
  - **Pentru fiecare** ((u,v) ∈ E) // muchiile se extrag în ordinea // costului
    - **Dacă** Arb(u) != Arb(v) **atunci** // verificăm dacă se creează ciclu
      - Arb(u) = Arb(u) ∪ Arb(v) // se reunesc mulțimile de noduri (arborii)
      - A = A ∪ {(u,v)} // se adaugă muchia sigură în AMA
  - **Întoarce** A

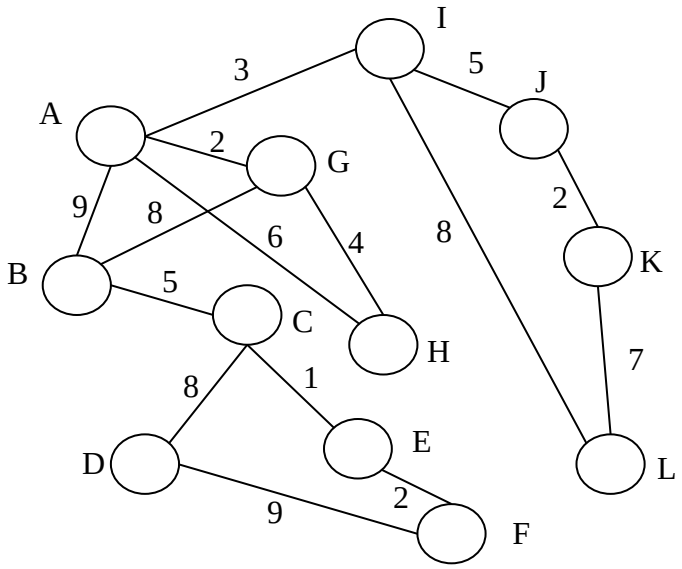


Exemplu (I)

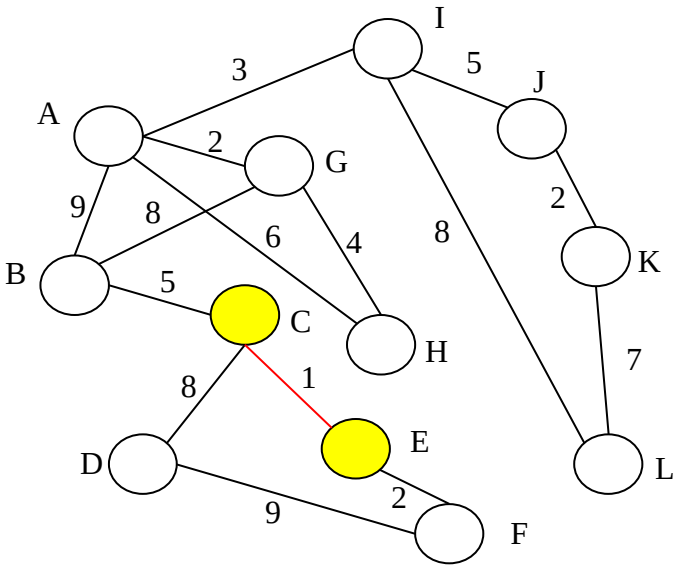


- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9

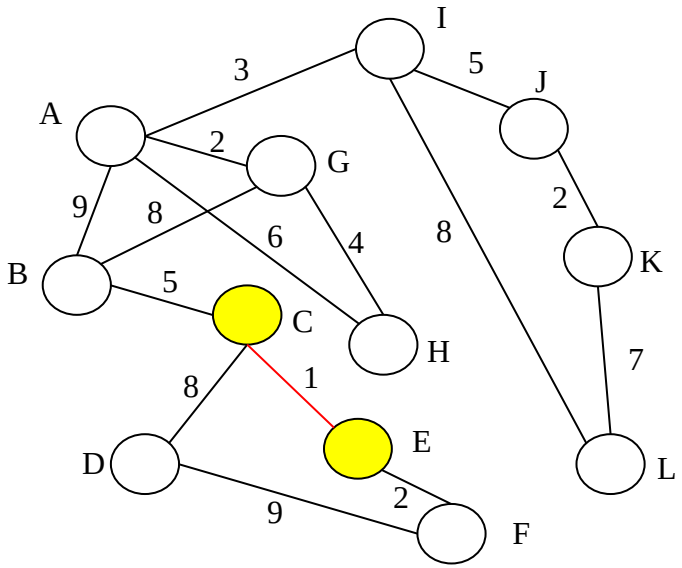
Exemplu (II)



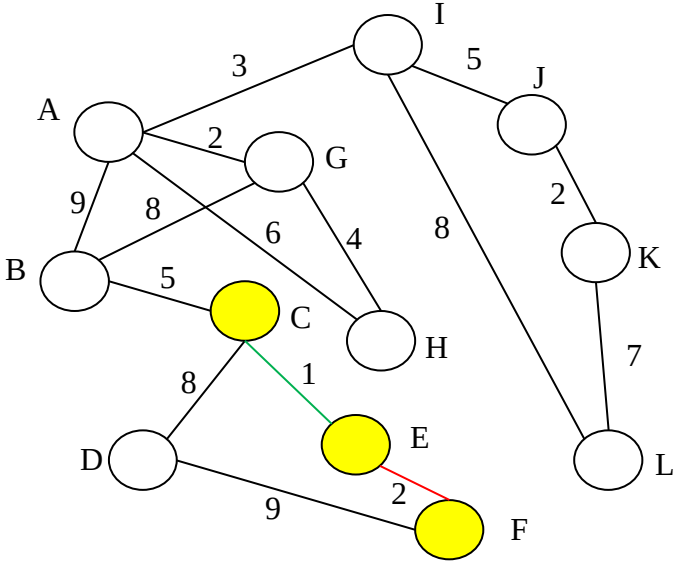
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



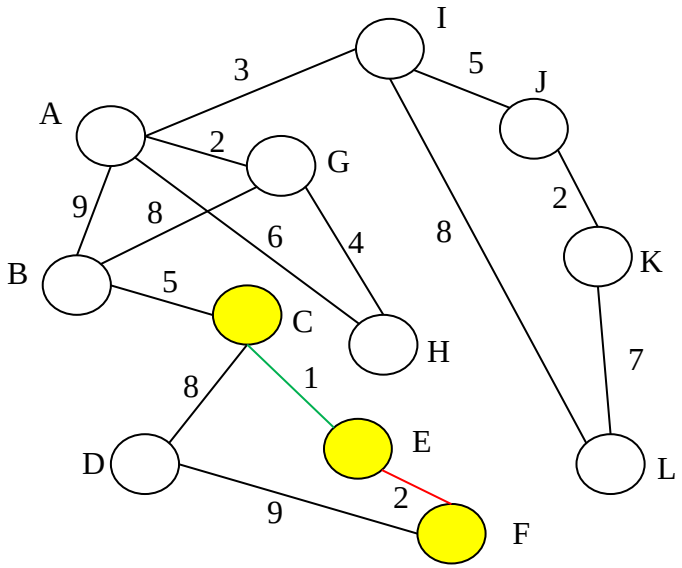
Exemplu (III)



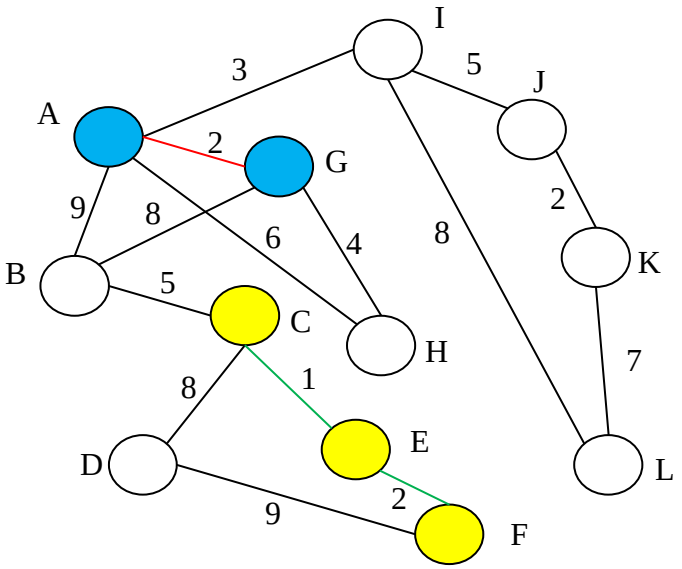
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



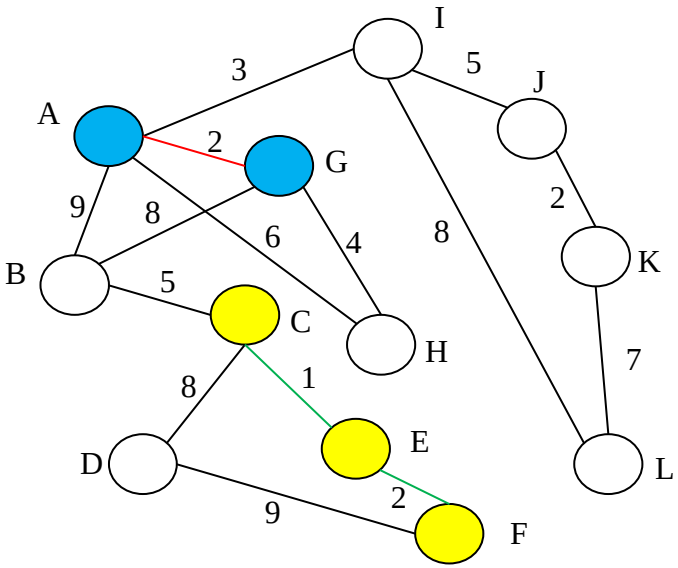
Exemplu (IV)



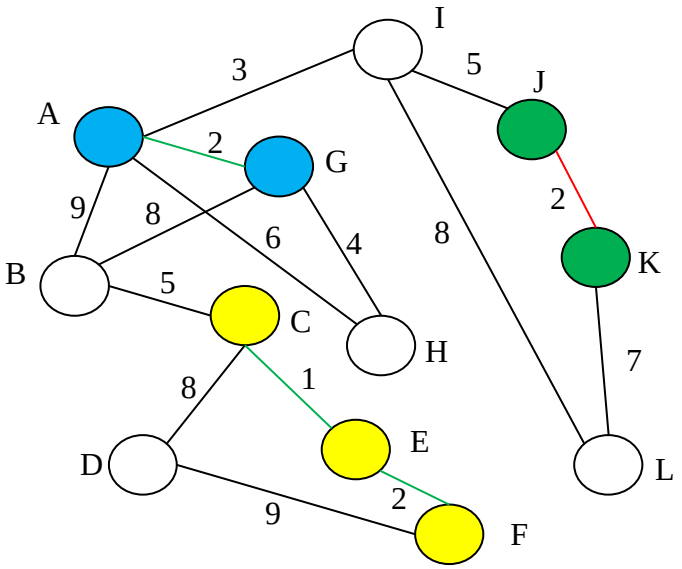
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



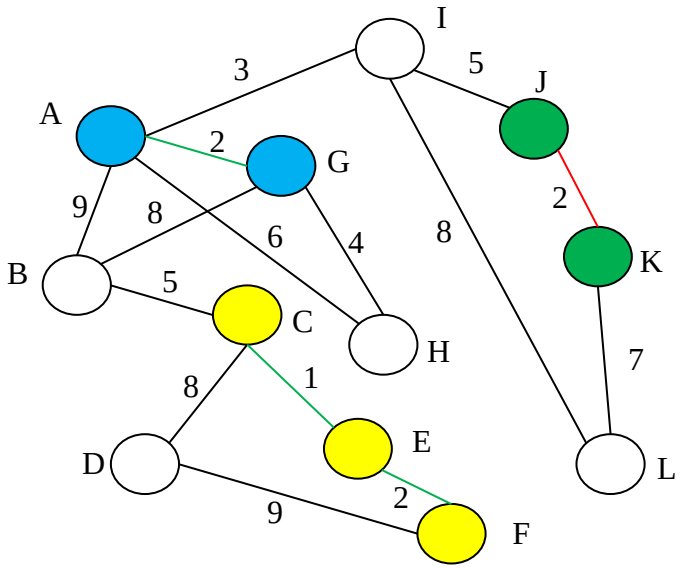
Exemplu (V)



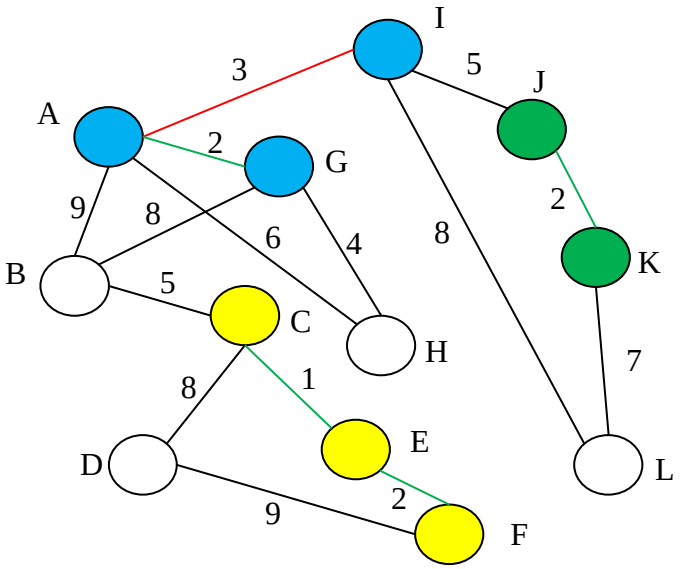
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



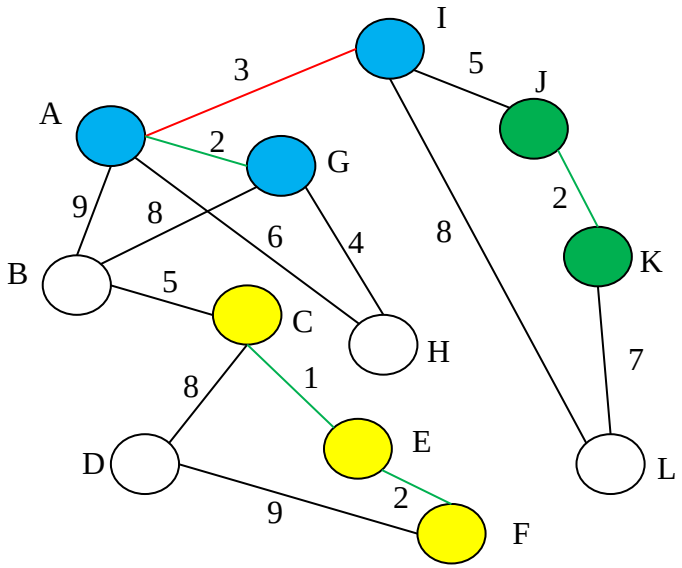
Exemplu (VI)



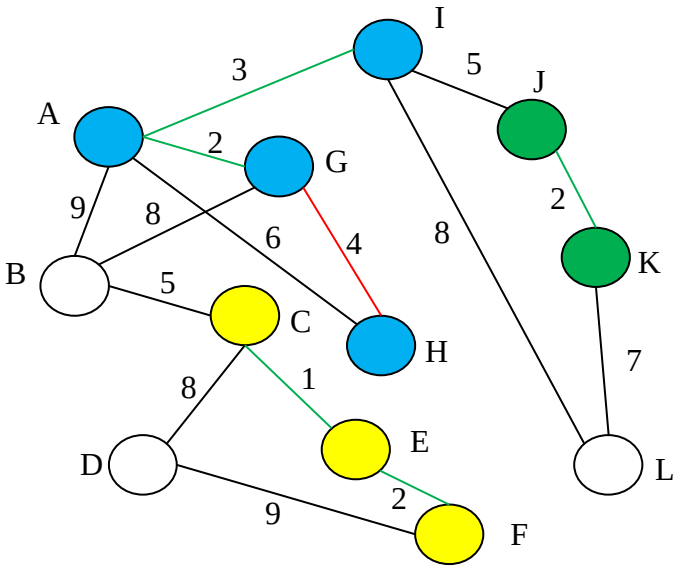
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



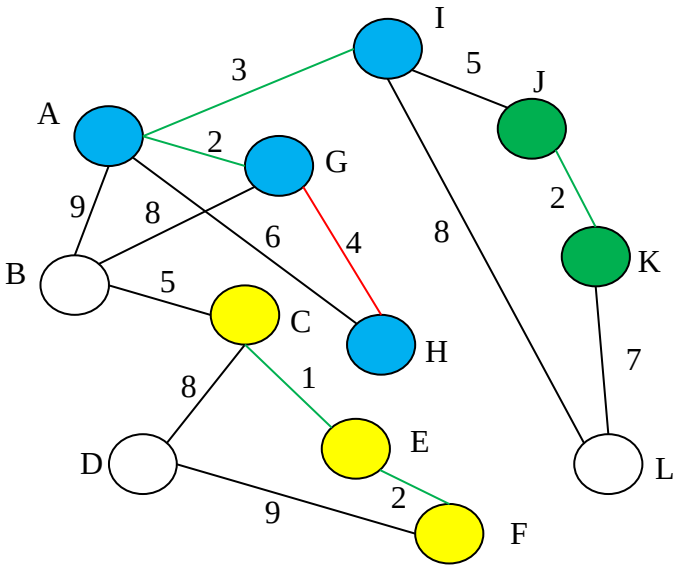
Exemplu (VII)



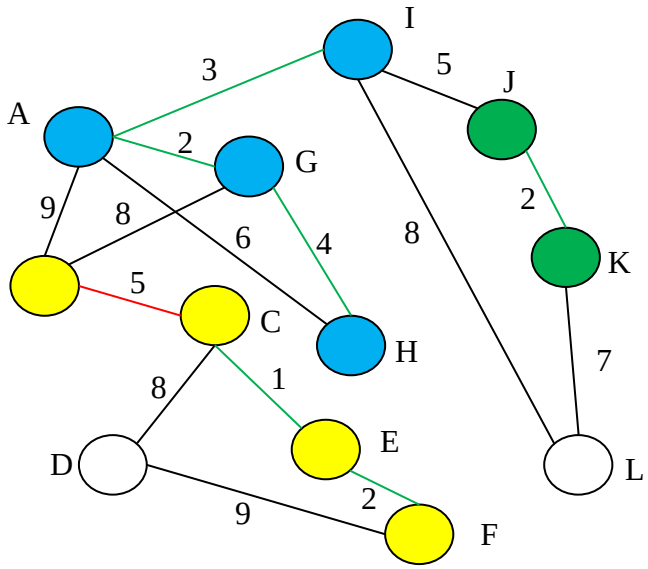
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



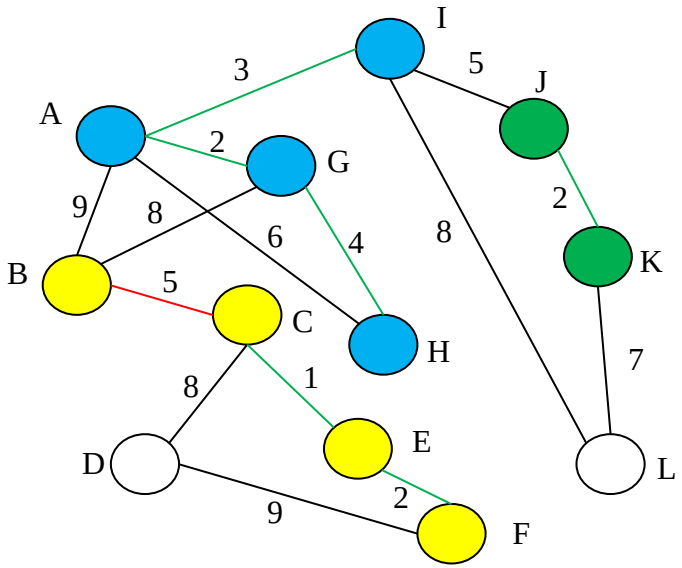
Exemplu (VIII)



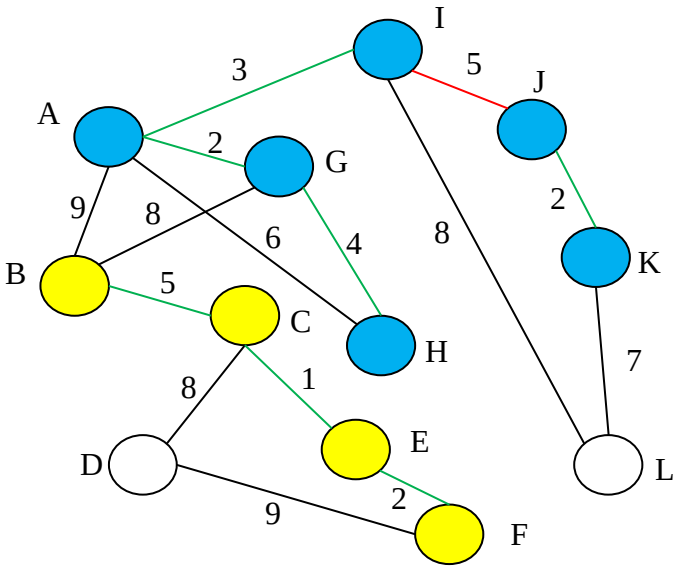
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



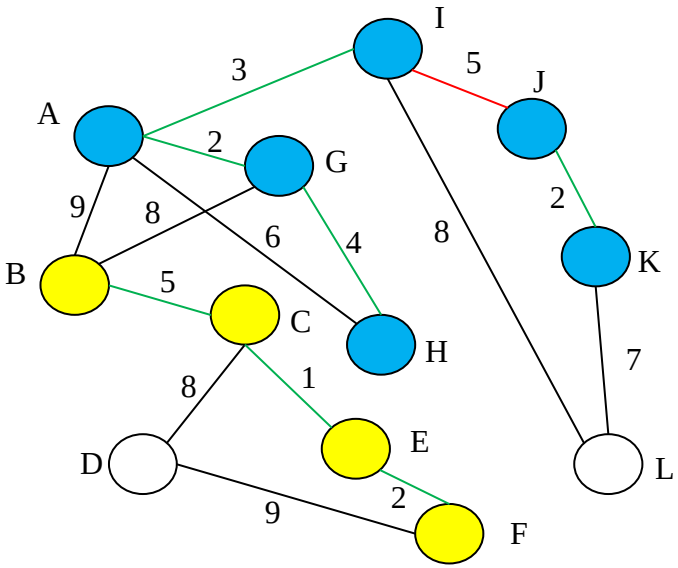
Exemplu (IX)



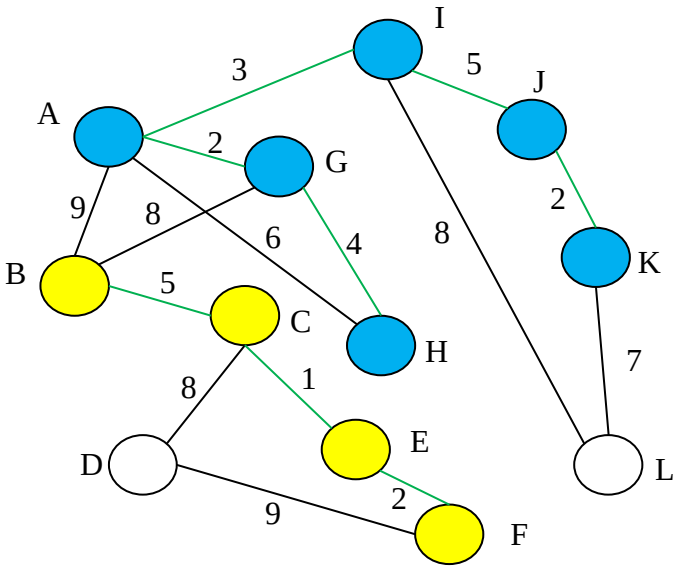
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



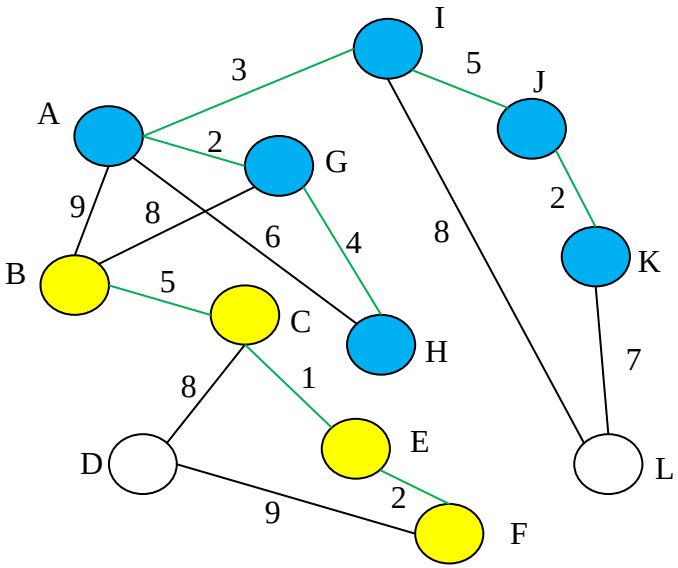
Exemplu (X)



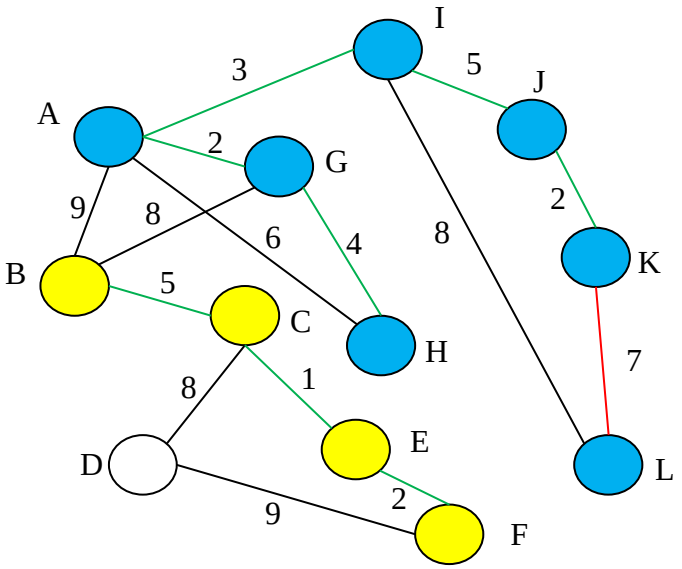
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



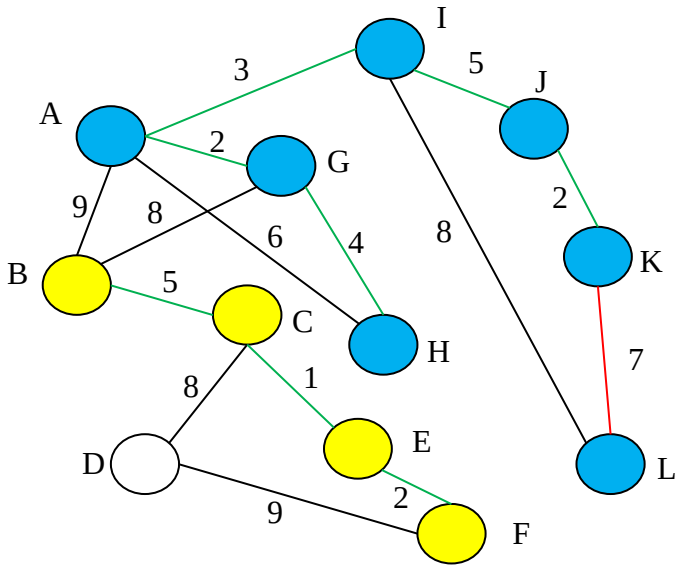
Exemplu (XI)



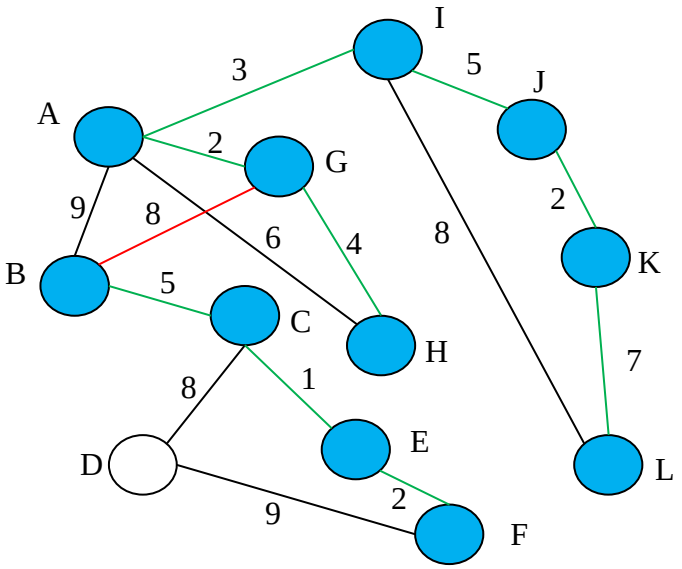
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



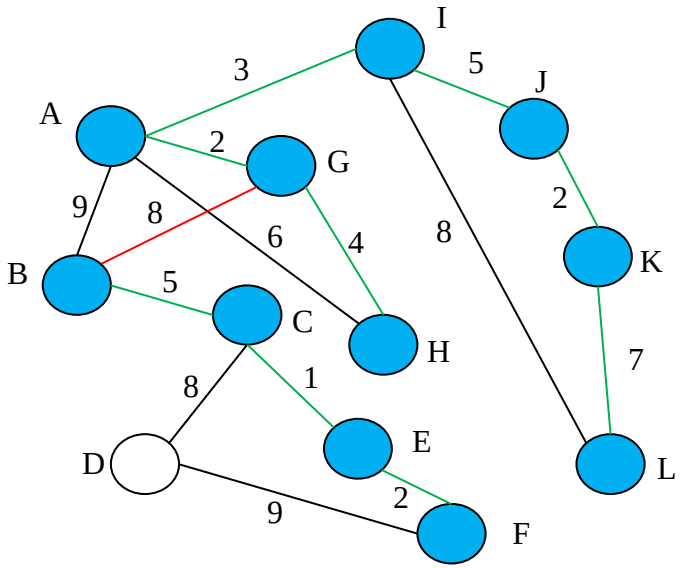
Exemplu (XII)



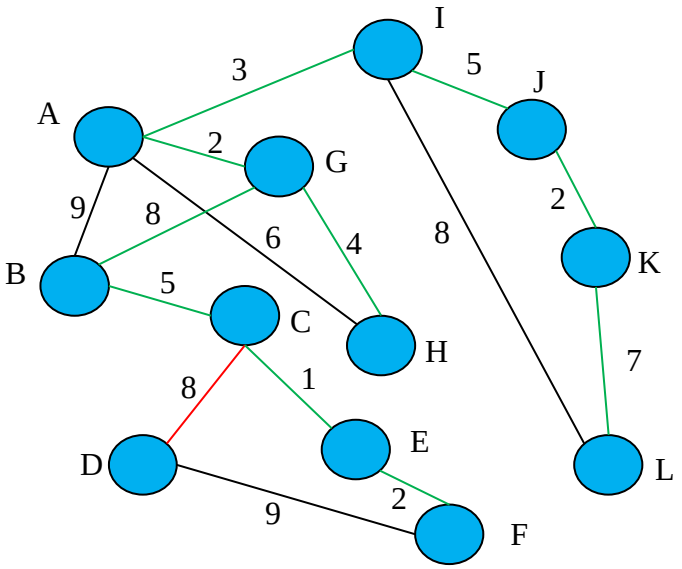
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



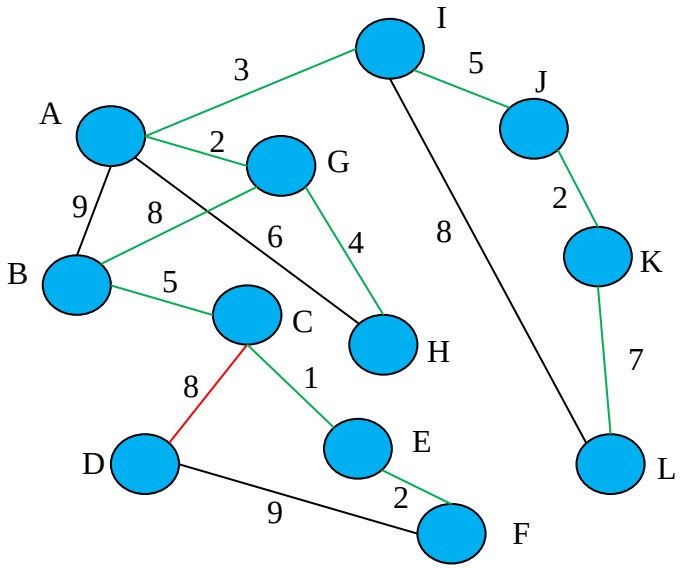
Exemplu (XIII)



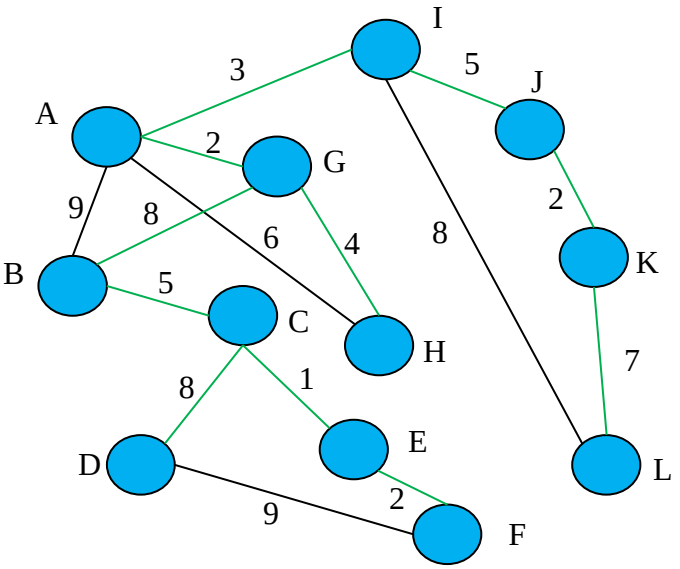
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



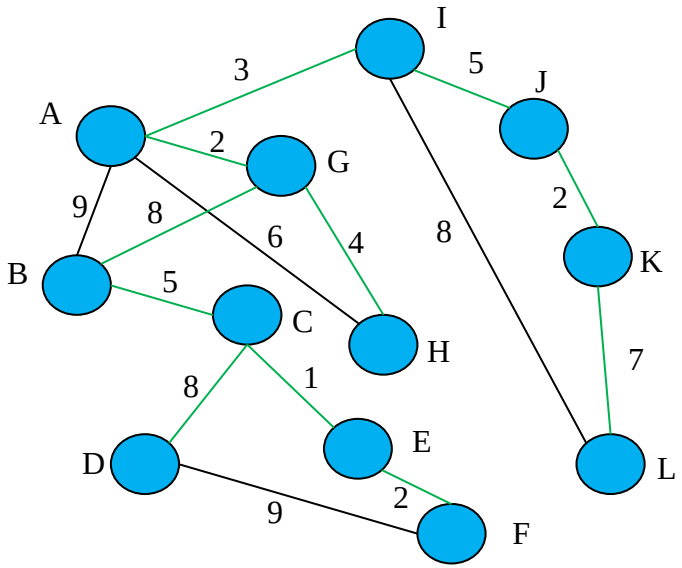
Exemplu (XIV)



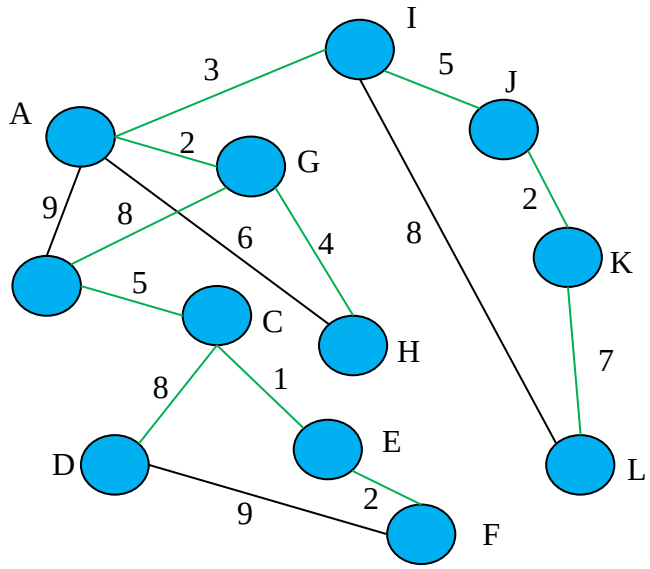
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



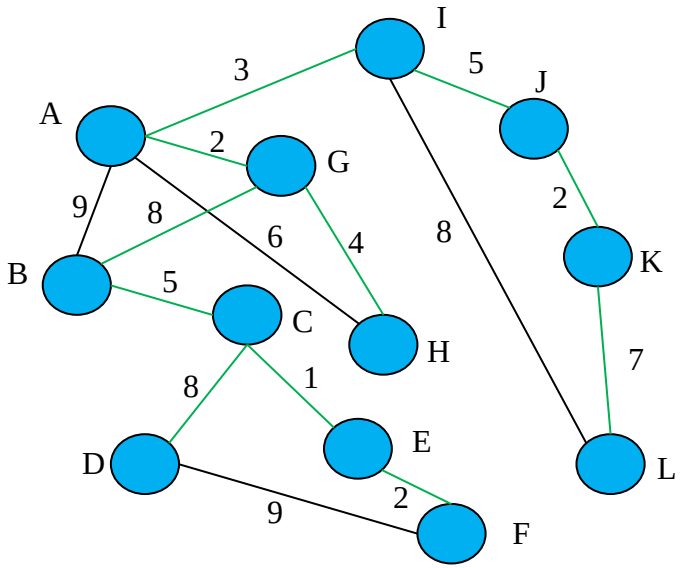
Exemplu (XV)



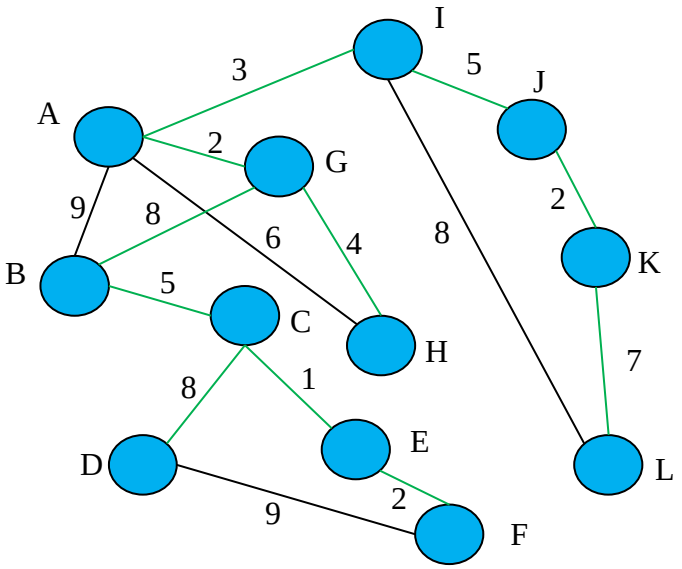
- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



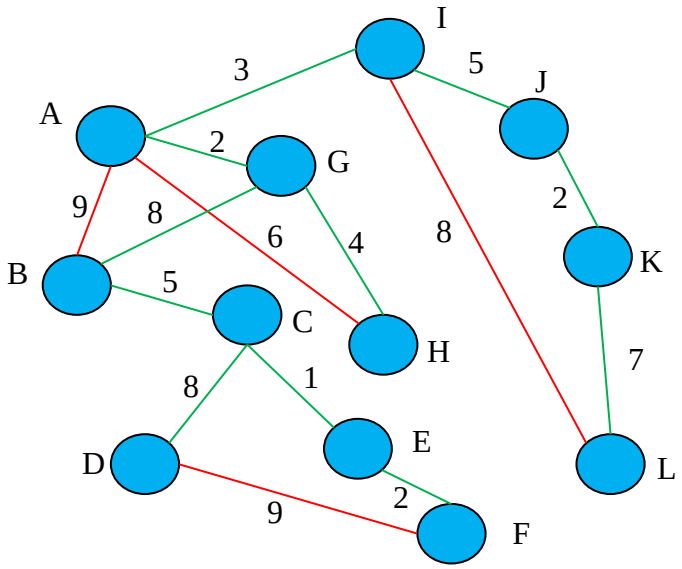
Exemplu (XVI)

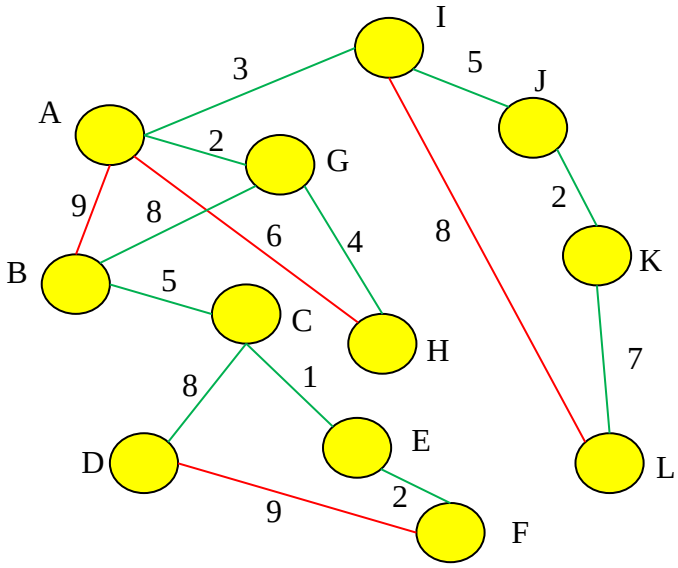


- CE -1
- EF -2
- AG-2
- JK-2
- AI-3
- GH-4
- BC-5
- IJ-5
- AH-6
- KL-7
- BG-8
- CD-8
- IL-8
- AB-9
- DF-9



Comparație Prim - Kruskal





Corectitudine (I)

- 1. arătăm că muchiile ignorate nu fac parte din Arb:
  - Pp. (u,v) a.î. Arb(u) = Arb(v)
    - → (u,v) creează un ciclu în Arb(u) (arborii sunt aciclici)
    - w(u,v) = max {w(u',v') | (u',v') $\in$ Arb(u)} (din faptul că muchiile sunt sortate crescător)
    - → din Propr. 1 → (u,v) $\notin$ Arb

Corectitudine (II)

- 2. arătăm că muchiile pe care le adăugăm aparțin Arb:
- Dem prin inducție după muchiile adăugate în AMA:
- P_1: Avem nodurile u și v, cu muchia (u,v) având proprietatea w(u,v) = min {w(u',v') | (u',v') ∈ E} → din Propr. 2 → (u,v) ∈ Arb.
- $P_n \rightarrow P_{n+1}$:
  - Arb(u) != Arb(v)
  - → (u,v) muchie cu un capăt în Arb(u)
  - (u,v) are cel mai mic cost din muchiile cu un capăt în u (din faptul că muchiile sunt sortate crescător)
  - → din Propr. 2 → (u,v) ∈ Arb

Algoritmul lui Kruskal

- Kruskal(G,w)
  - A = ∅; // AMA
  - **Pentru fiecare** (v ∈ V)
    - Constr_Arb(v) // creează o mulțime formată din nodul respectiv // (un arbore cu un singur nod)
  - Sortează_asc(E,w) // se sortează muchiile în funcție de // costul lor
  - **Pentru fiecare** ((u,v) ∈ E) // muchiile se extrag în ordinea // costului
    - **Dacă** Arb(u) != Arb(v) **atunci** // verificăm dacă se creează ciclu
      - Arb(u) = Arb(u) ∪ Arb(v) // se reunesc mulțimile de noduri (arborii)
      - A = A ∪ {(u,v)} // se adaugă muchia sigură în AMA
  - **Întoarce** A

Complexitate?

Complexitate Kruskal

- Elementele algoritmului:
  - Sortarea muchiilor: O(ElogE) ≈ O(ElogV)
  - Arb(u) = Arb(v) – compararea a 2 mulțimi disjuncte
    {1,2,3} {4,5,6} – mai precis trebuie identificat dacă 2 elemente sunt în aceeași mulțime
  - Arb(u) ∪ Arb(v) – reuniunea a 2 mulțimi disjuncte într-una singură

- → depinde de implementarea mulțimilor disjuncte

Variante de implementare mulțimi disjuncte (Var. 1) – contraexemplu

Mulțimile implementate ca vectori (populară la laborator ☺) – NERECOMANDATĂ ☹

- Comparare (M_1, M_2)
  - **Pentru fiecare** (u ∈ M_1)
    - **Pentru fiecare** (v ∈ M_2)
      - **Dacă** (u = v) **Întoarce true**
  - **Întoarce false**
- Complexitate: V^2

- Reuniune (M_1, M_2)
  - **Pentru** i **de la** length(M_1) **la** length(M_1) + length(M_2)
    - $M_1[i] = M_2[i - length(M_1)]$
  - **Întoarce** M_1
- Complexitate: V

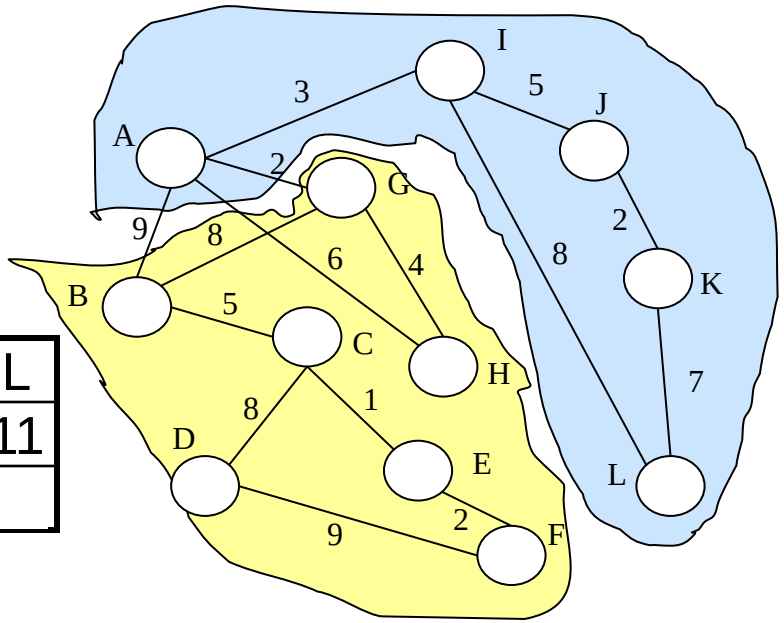
- numărul de apelări – E

- Complexitate totală: $E*V^2$

Variante de implementare mulțimi disjuncte (Var. 2) – Regăsire Rapidă

- Mulțimile - vectori
- Id - vector de id-uri conținând id-ul primului nod din componentă

A	B	C	D	E	F	G	H	I	J	K	L
0	1	2	3	4	5	6	7	8	9	10	11



- Arb(u) != Arb(v)
  - Complexitate?
- Arb(u) = Arb(u) ∪ Arb(v)
  - Complexitate?

Complexitate maximă?

Regăsire rapidă (Complexitate)

- Compararea – O(1) // Căutare în vector și verificare dacă au același id
- Reuniunea – O(V) // trebuie să modifice toate id-urile nodurilor din una din mulțimi

- Complexitate maximă
  - O(V * E) // E = numărul de reuniuni

- Inacceptabil pentru grafuri f mari



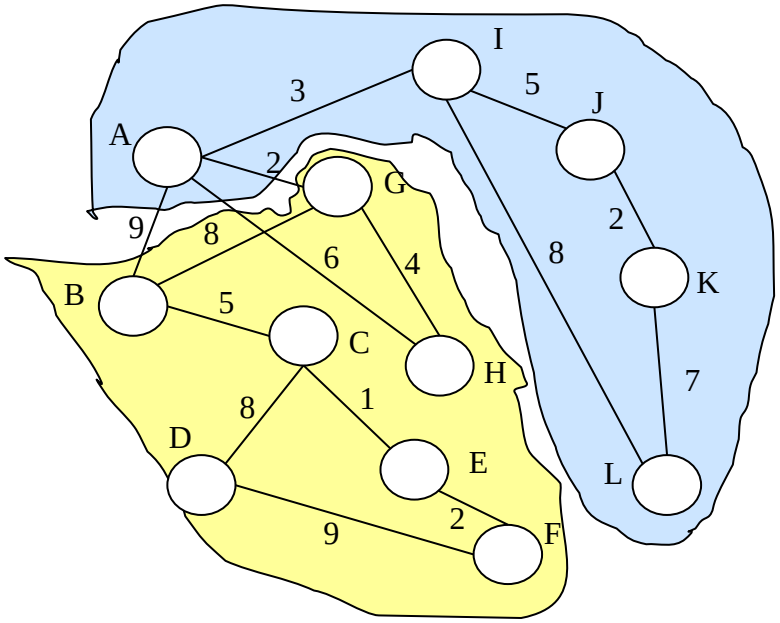
Variante de implementare mulțimi disjuncte (Var. 3) – Reuniune Rapidă

- se folosește tot un vector auxiliar de id-uri

A	B	C	D	E	F	G	H	I	J	K	L
0	1	2	3	4	5	6	7	8	9	10	11

- id[i] reprezintă părintele lui i

- pentru rădăcina arborelui id[i] = i



Variante de implementare mulțimi disjuncte – reuniune rapidă

- Comparare (u, v)
  - Verifică dacă 2 noduri au aceeași rădăcină;
  - Implică identificarea rădăcinii:

- Arb(i) // identificarea rădăcinii unei componente
  - **Cât timp** (i != id[i]) i = id[i];
  - **Întoarce** i

- Comparare (u, v)
  - **Întoarce** Arb(u) != Arb(v)

Complexitate?

- Reuniune (u,v) // implică identificarea rădăcinii
  - v = Arb(v)
  - id[v] = u;

Reuniune rapidă (Complexitate)

Compararea – O(V) // în cel mai rău caz, am o lista și trebuie să trec din părinte în părinte.

Reuniunea – O(V) // implică regăsirea rădăcinii pentru a ști unde se face modificarea

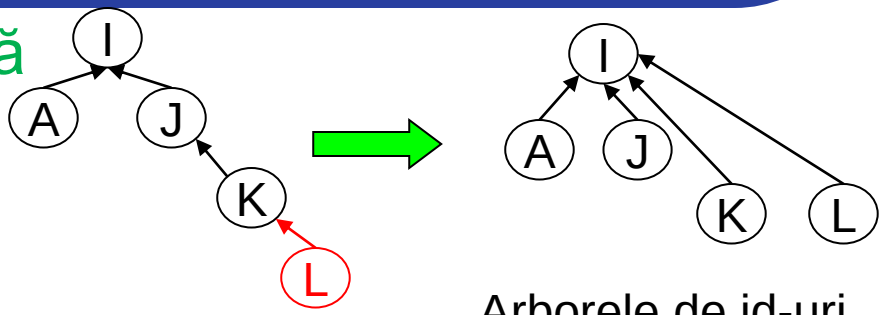


Optimizarea reuniunii rapide (1)

- Reuniune rapidă balansată
- Se menține numărul de noduri din fiecare subarbore.
- Se adaugă arborele mic la cel mare pentru a face mai puține căutări → înălțimea arborelui e mai mică și numărul de căutări scade de la V la lg V.
- Complexitate:
  - Compararea – O(lg V)
  - Reuniune – O(lg V)

Optimizarea reuniunii rapide (2)

- Reuniune rapidă balansată cu înjumătățirea căii:



Arborele de noduri

Arborele de id-uri

- Identificarea rădăcinii:
  - Arb(i)
    - **Cât timp** (i != id[i])
      - id[i] = id[id[i]];
      - i = id[i];
    - **Întoarce** i

K: id[K] = id[J] = I

L: id[L] = id[K] = I

- Menține o înălțime redusă a arborilor.

Implementare în Java și exemplu la [4]

- Pentru căi foarte lungi → reuniunea rapidă balansată cu compresia căii [7] modifică id-urile nodurilor din arbore la rădăcină: Dacă (i != id[i]): id[i] = Arb(id[i])) Întoarce id[i]

Complexitate după optimizări

- Orice secvență de E operații de căutare și reuniune asupra unui graf cu V noduri consumă $O(V + E*\alpha(V,E))$ [4, 7].
- α – de câte ori trebuie aplicat lg pentru a ajunge la 1.
  - În practică este ≤ 5.
- → În practică O(E)

Complexitate Kruskal

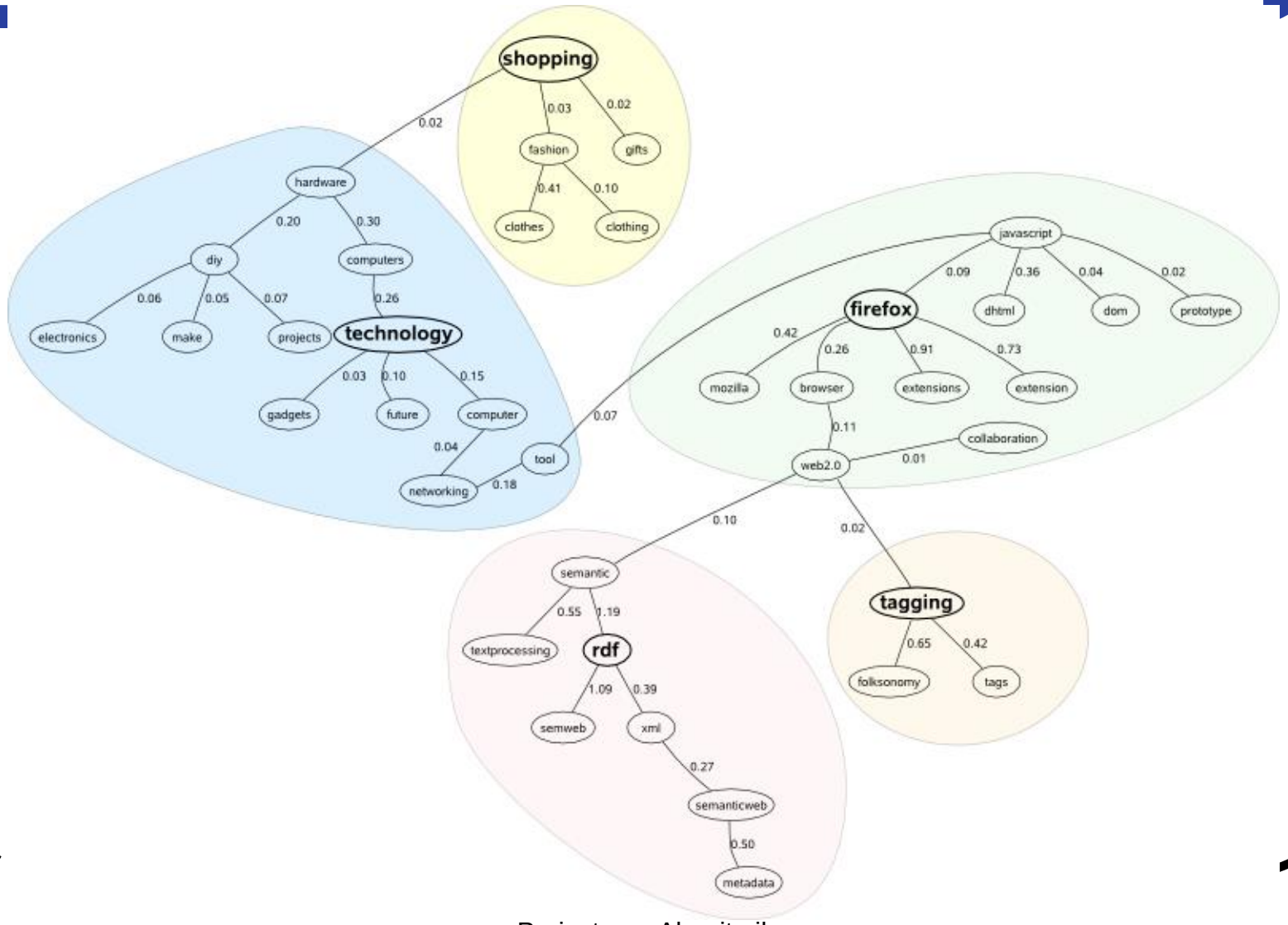
- Max (complexitate sortare, complexitate operații mulțimi) = max(O(ElogV), O(E)) = O(ElogV)

- → Complexitatea algoritmului Kruskal este dată de complexitatea sortării costurilor muchiilor.

Aplicație practică

- K-clustering
  - Împărțirea unui set de obiecte în k grupuri astfel încât obiectele din cadrul unui grup să fie “apropiate” considerând o “distanță” dată.
- Utilizat în clasificare, căutare (web search de exemplu).
- Dându-se un întreg K și un grup de obiecte, se cere să se împartă grupul de obiecte în K grupuri astfel încât spațiul dintre grupuri să fie maximizat.

Exemplu


shopping
0.02
0.03
0.02
fashion
gifts
0.41
0.10
clothes
clothing
hardware
0.20
0.30
diy
computers
0.06
0.05
0.07
0.26
electronics
make
projects
technology
0.03
0.10
0.15
gadgets
future
computer
0.04
tool
networking
0.18
0.07
javascript
0.09
0.36
0.04
0.02
firefox
dhtml
dom
prototype
0.42
0.26
0.91
0.73
mozilla
browser
extensions
extension
0.11
collaboration
web2.0
0.01
0.10
0.02
semantic
tagging
0.55
1.19
0.65
0.42
textprocessing
rdf
folksonomy
tags
1.09
0.39
semweb
xml
0.27
semanticweb
0.50
metadata

Algoritm

- Se formează V clustere (un cluster per obiect).
- Găsește cele mai apropiate 2 obiecte din clustere diferite și unește cele 2 clustere.
- Se oprește când au mai rămas k clustere.
- → chiar algoritmul Kruskal



ÎNTREBĂRI?



Bibliografie curs 12

[1] C. Giumale – Introducere in Analiza Algoritmilor - cap. 5.6

[2] Cormen – Introducere in algoritmi - cap. 27

[3] Wikipedia - http://en.wikipedia.org/wiki/Ford-Fulkerson_algorithm

Proiectarea Algoritmilor

Curs 12 – Rețele de flux. Flux maxim.



Bibliografie

[1] C. Giumale – Introducere în Analiza Algoritmilor - cap. 5.6

[2] Cormen – Introducere în algoritmi - cap. Flux Maxim (27)

[3] Wikipedia - http://en.wikipedia.org/wiki/Ford-Fulkerson_algorithm

[4] R. Sedgewick, K Wayne – curs de algoritmi Princeton 2007 www.cs.princeton.edu/~rs/AlgsDS07/ 01UnionFind si 14MST

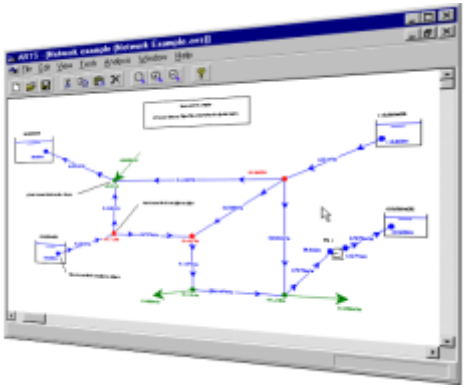
Obiective

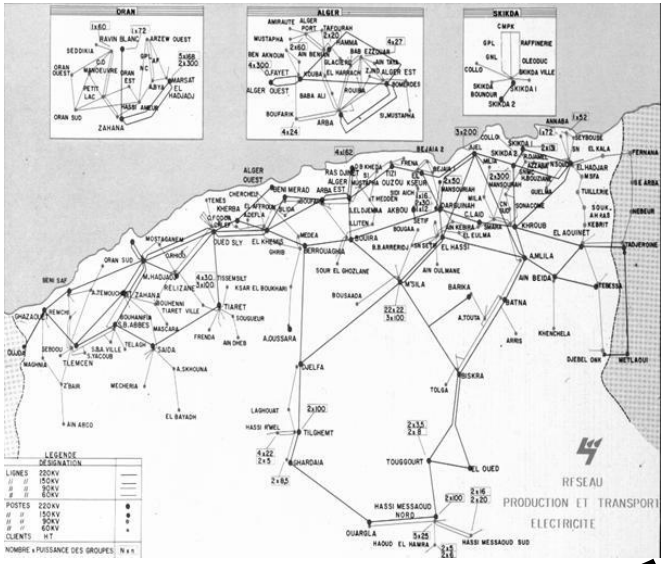
- Definirea conceptului de rețea de flux (sau de transport).
- Identificarea principalilor algoritmi ce calculează fluxul maxim printr-o rețea.



Definirea problemei

- Rețea ce transportă diferite materiale între un producător și o destinație.
- Fiecare arc are o capacitate maximă de transport.
- Trebuie identificat fluxul maxim ce poate fi transportat prin rețea.
- Rețele:
  - Electrice;
  - Apă;
  - Informații;
  - Drumuri.



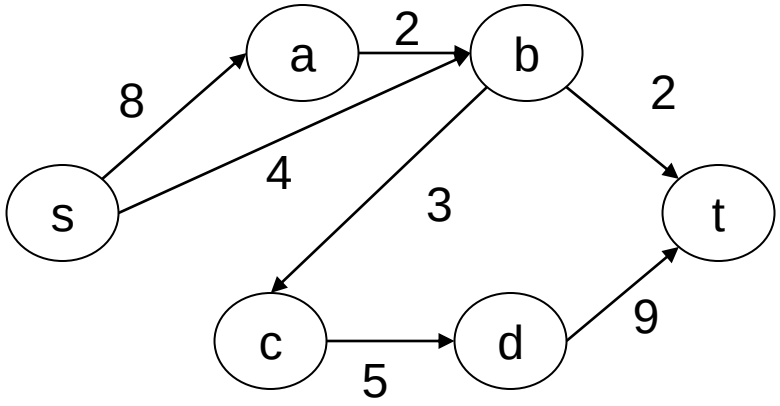


Rețea de flux – Definiție

- G(V,E) graf orientat;
- $c(u,v) \geq 0 \quad \forall (u,v)$ – c = capacitatea arcelor;
- Dacă $(u,v) \notin E \rightarrow c(u,v) = 0$;
- S – sursa traficului;
- T – destinația traficului (drena);
- Presupunem că $\forall u \in V \setminus \{s, t\} \exists$ s..u..t.



Exemplu de rețea de flux

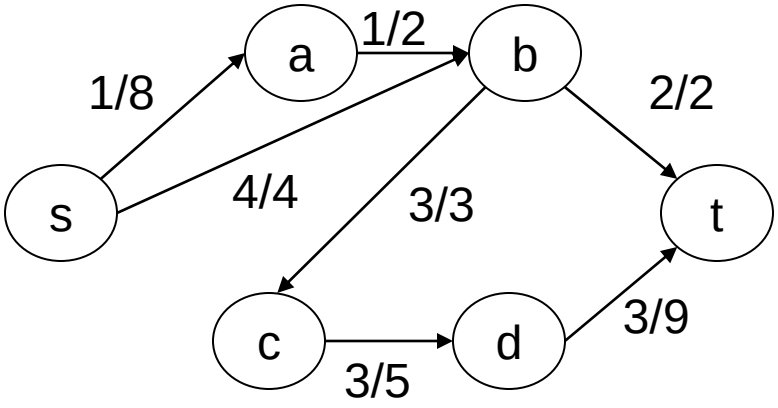


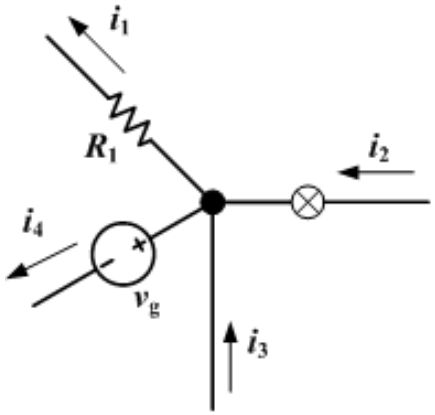
- s – sursa, t – destinația.
- Pe arce este reprezentată capacitatea arcului.

Flux. Definiție. Proprietăți.

- G = (V,E) – rețea de flux;
- c: V x V→$\Re$ - capacitatea rețelei;
- f: V x V→$\Re$ - fluxul prin rețeaua G;
- Proprietăți:
  - $\forall u, v \in V, f(u,v) \leq c(u,v)$ (fluxul printr-un arc este mai mic sau egal cu capacitatea arcului) – respectarea capacității arcelor;
  - $\forall u, v \in V, f(u,v) = -f(v,u)$ – simetria fluxului;
  - $\Sigma f(u,v) = 0$ pentru $\forall u \in V \setminus \{s,t\}$ – conservarea fluxului.

Exemplu de fluxuri





i2+i3-i4-i1=0 (P3)

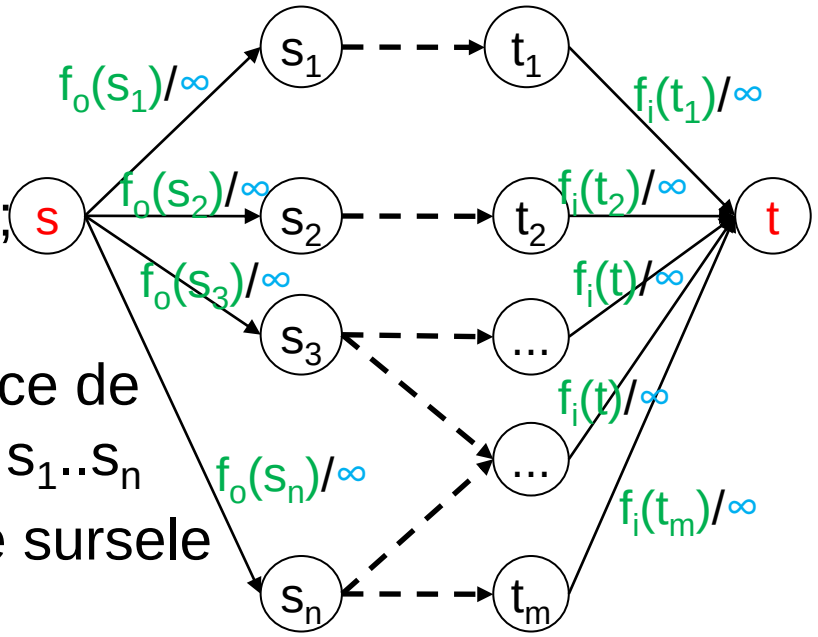
- $\Sigma f(u,v) = 0$ pentru $\forall u \in V \setminus \{s,t\}$ – fluxul se conservă;
- Proprietatea 3 = legea curentului (Kirchoff) ☺ - suma I. curenților ce intră într-un nod = suma I. curenților ce ies din nodul respectiv.

Flux. Notații.

- f(u,v) – fluxul din u spre v;
- $f_i(u) = \Sigma f(v,u)$ – fluxul total care intra în nodul u;
- $f_o(u) = \Sigma f(u,v)$ – fluxul total care iese din nodul u;
- Valoarea totală a fluxului:
  - $|f| = \Sigma f(s,v) = f_o(s)$;
  - |f| = fluxul ce părăsește sursa;
  - Cf. proprietăților P1-P3: $|f| = \Sigma f(s,v) = \Sigma f(v,t) = f_i(t)$.

Surse multiple, destinații multiple

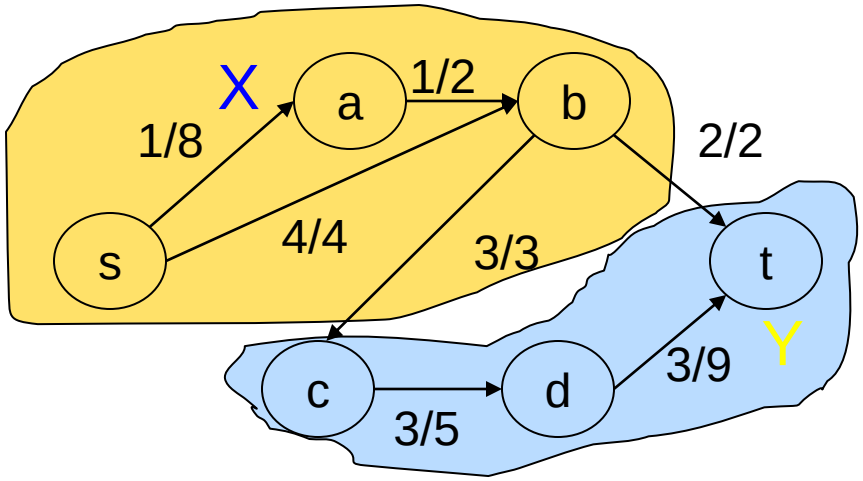
- Surse multiple $\{s_1, s_2, \ldots, s_n\}$;
- Destinații multiple $\{t_1, t_2, \ldots, t_m\}$;
- Se adaugă o sursă unică cu arce de capacitate infinită spre sursele $s_1..s_n$ și flux egal cu fluxul generat de sursele respective;
- Se adaugă o destinație unică t și arce de capacitate infinită între $t_1..t_m$ și t și flux egal cu fluxul ce intră în destinațiile respective.



Operații cu fluxuri

- X,Y – mulțimi de noduri;
- $f(X,Y) = \Sigma_{x \in X} \Sigma_{y \in Y} f(x,y)$ = fluxul între X și Y;
- Operații:
  - $\forall X \in V$: $f(X,X) = 0$;
  - $\forall X,Y \in V$: $f(X,Y) = -f(Y,X)$;
  - $\forall X,Y,Z \in V$ și $Y \subseteq X$:
    - $f(X \setminus Y, Z) = f(X,Z) - f(Y,Z)$;
    - $f(Z, X \setminus Y) = f(Z,X) - f(Z,Y)$;
  - $\forall X,Y,Z \in V$ și $X \cap Y = \emptyset$:
    - $f(X \cup Y, Z) = f(X,Z) + f(Y,Z)$;
    - $f(Z, X \cup Y) = f(Z,X) + f(Z,Y)$
  - $f(s,V) = f(V,t)$

Exemplu operații fluxuri (1)

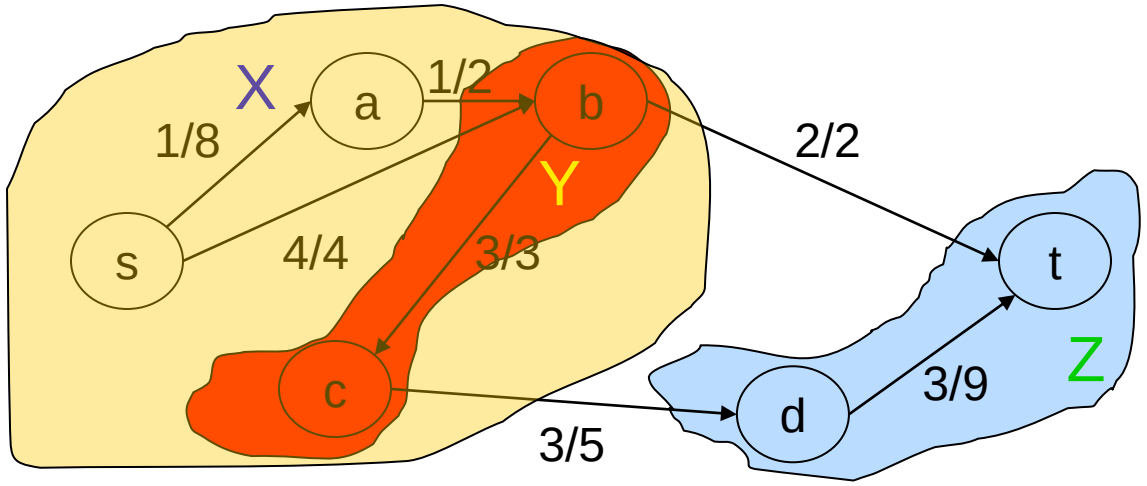


$$f(X,Y) = \Sigma_{x \in X} \Sigma_{y \in Y} f(x,y)$$

$f(X,X) = f(s,a) + f(a,s) + f(s,b) + f(b,s) + f(a,b) + f(b,a) = 0$

$f(X,Y) = f(b,c) + f(b,t) = -f(c,b) - f(t,b) = -f(Y,X)$

Exemplu operații fluxuri (2)



$\forall X, Y, Z \in V$ si $Y \subseteq X$

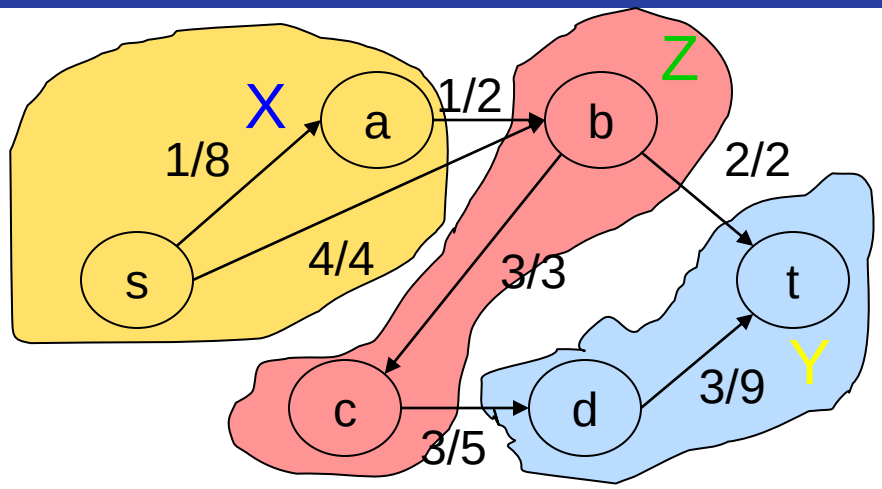
$f(X \setminus Y, Z) = f(X, Z) - f(Y, Z)$

$f(Z, X \setminus Y) = f(Z, X) - f(Z, Y)$

$f(X \setminus Y, Z) = 0 = f(b,t) + f(c,d) - f(b,t) - f(c,d) = f(X,Z) - f(Y,Z)$

$f(Z, X \setminus Y) = 0 = f(t,b) + f(d,c) - f(t,b) - f(d,c) = f(Z,X) - f(Z,Y)$

Exemplu operații fluxuri (3)



$\forall X, Y, Z \in V$ si $X \cap Y = \varnothing$

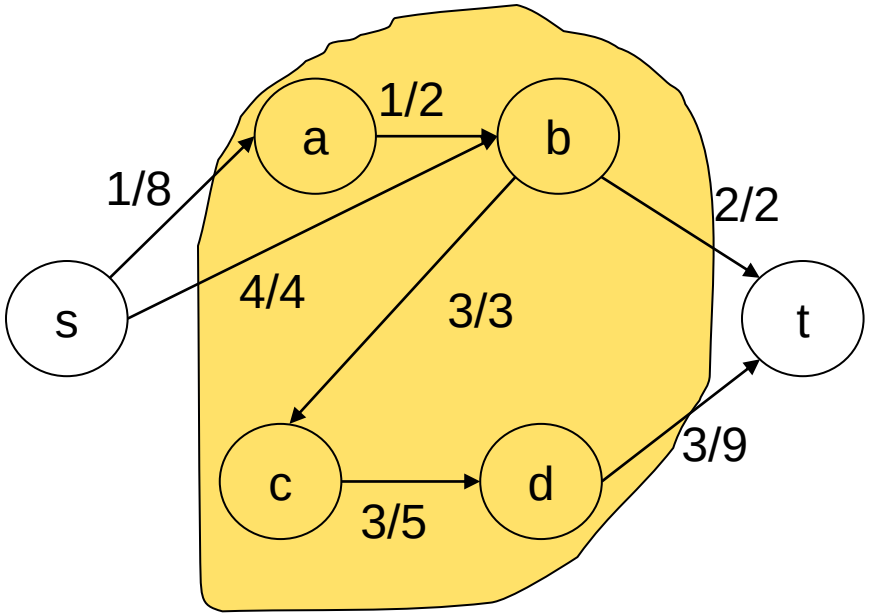
$f(X \cup Y, Z) = f(X,Z) + f(Y,Z)$

$f(Z, X \cup Y) = f(Z,X) + f(Z,Y)$

$f(X \cup Y, Z) = f(s,b) + f(a,b) + f(t,b) + f(d,c) = f(X,Z) + f(Y,Z)$

$f(Z, X \cup Y) = f(b,a) + f(b,s) + f(b,t) + f(c,d) = f(Z,X) + f(Z,Y)$

Exemplu operații fluxuri (4)



f(s, V) = f(V, t)

f(s, V) = f(s,a) + f(s,b) = 5 = f(d,t) + f(b,t) = f(V, t)

Arc rezidual. Capacitate reziduală.

- Definiție: Un arc (u,v) pentru care f(u,v) < c(u,v) se numește arc rezidual.
- ➔Fluxul pe acest arc se poate mări.
- Definiție: Cantitatea cu care se poate mări fluxul pe arcul (u,v) se numește capacitatea reziduală a arcului (u,v) ($c_f(u,v)$):

$$c_f(u,v) = c(u,v) - f(u,v)$$



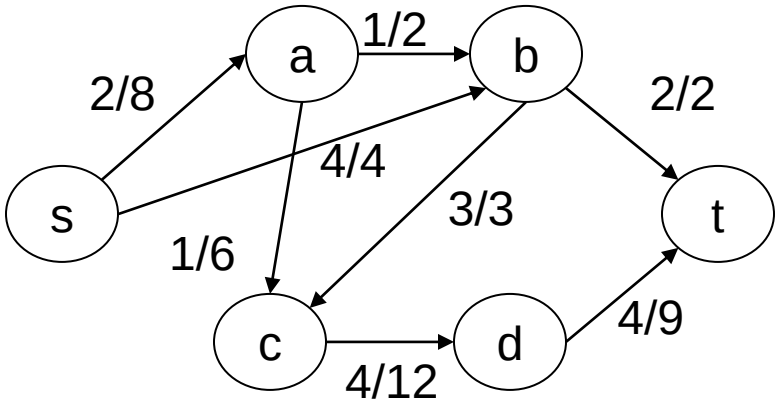
Rețea reziduală. Cale reziduală.

- G = (V,E) rețea de flux cu funcția de capacitate c.
- Definiție: Rețeaua reziduală ($G_f = (V,E_f)$) este o rețea de flux formată din arcele ce admit creșterea fluxului:

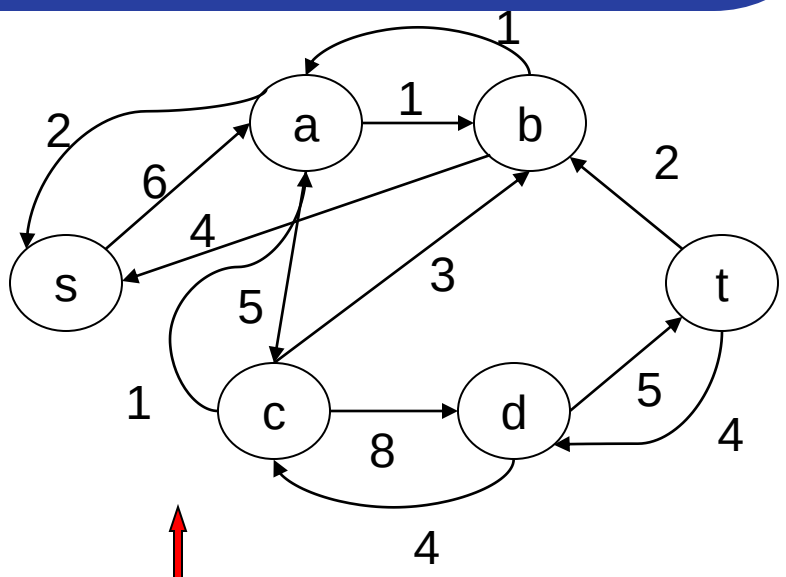
$$E_f = \{(u,v) \in V \times V \mid c_f(u,v) > 0\}.$$

- Observație: $E_f \not\subseteq E$!!!
- Definiție: Cale reziduală e un drum s..t $\subseteq G_f$, unde $c_f(u,v)$ este capacitatea reziduală a arcului (u,v).
- Definiție: Capacitatea reziduală a căii = capacitatea reziduală minimă de pe calea s..t descoperită.

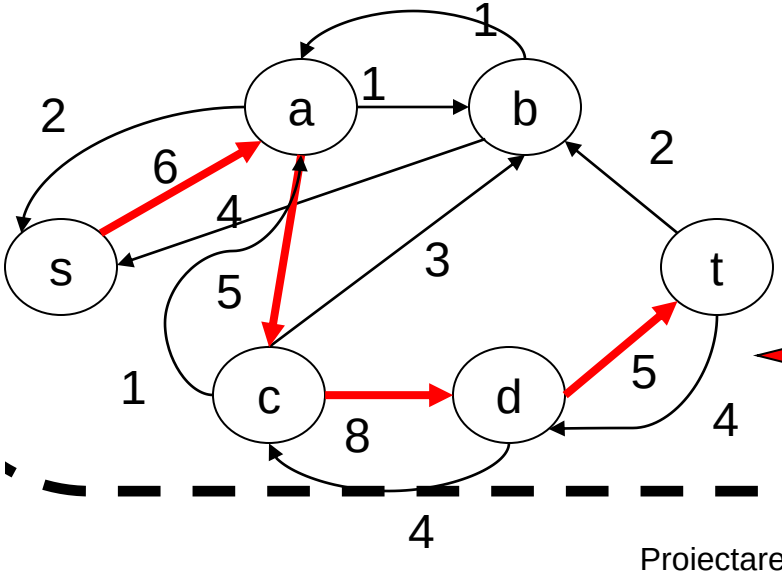
Exemplu rețea reziduală



$c_f(u,v) = c(u,v) - f(u,v)$



Rețeaua reziduală $G_f = (V,E_f)$ unde
$E_f = \{(u,v) \in V \times V \mid c_f(u,v) > 0\}$



Calea reziduală: s → a → c → d → t
Capacitatea reziduală a căii:
$c_f(p) = \min\{6, 5, 8, 5\} = 5$

Rețea reziduală

- Lemă 5.16: Fie G = (V,E) rețea de flux, f fluxul în G, G_f rețeaua reziduală a lui G. Fie f' un flux prin G_f și f+f' o funcție definită astfel:

$$f+f'(u,v) = f(u,v) + f'(u,v).$$

- Atunci f+f' reprezintă un flux în G și

$$|f+f'| = |f| + |f'|$$

- Această Lemă ne spune cum putem mări fluxul printr-o rețea de flux.



Flux în rețeaua reziduală

- Lemă 5.17: G – rețea de flux, f flux în G, p = s..t – cale reziduală în G_f, f_p:V x V->$\Re$ se definește ca fiind:

$$f_p(u,v) = \begin{cases} c_f(p), \text{ dacă } (u,v) \in p \\ -c_f(p), \text{ dacă } (v,u) \in p \\ 0, \text{ dacă } (u,v) \text{ și } (v,u) \notin p \end{cases}$$

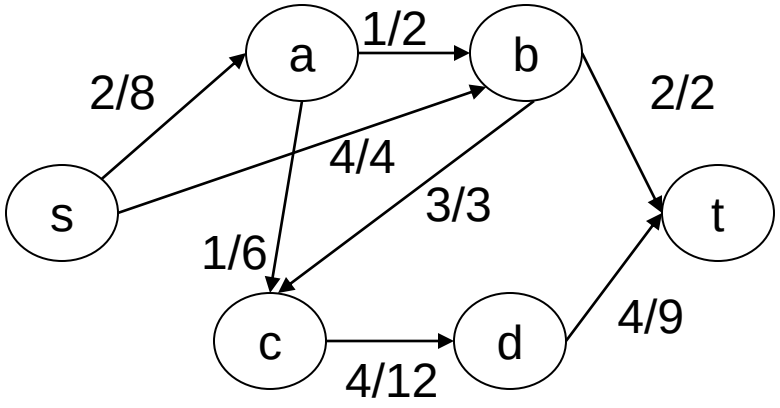
f_p = flux în G_f; $|f_p| = c_f(p)$

- Corolar 5.4: f' = f + f_p = flux în G, astfel încât

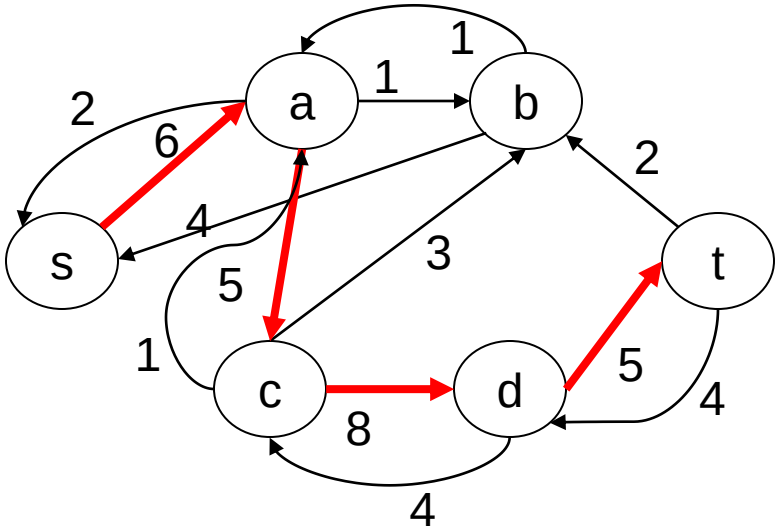
$$|f'| = |f| + |f_p| > |f|$$

- Această Lemă ne spune cum se definește fluxul printr-o rețea reziduală.

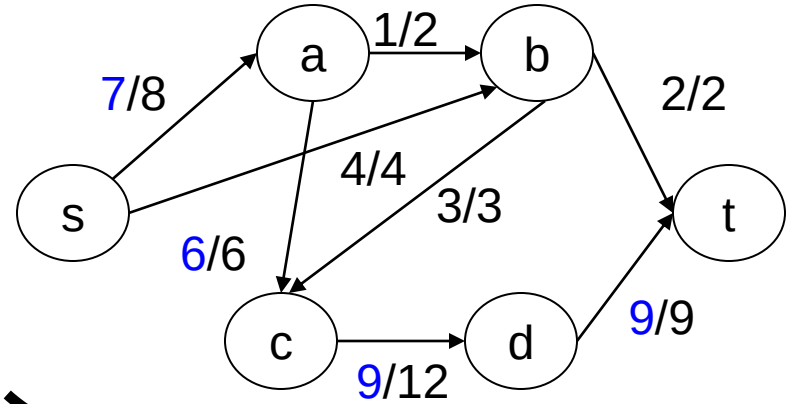
Exemplu maximizare flux



$f(s..t) = 2 + 4 = 6$



$|f_p(s..t)| = c_f(s..t) = 5$



$f'(s..t) = f + f_p = 6 + 5 = 11$

Calculul fluxului maxim

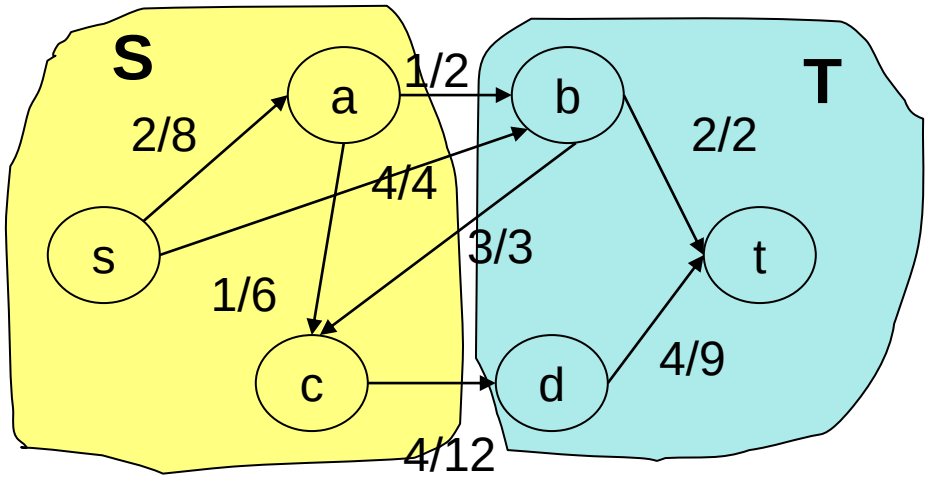
- Metoda Ford-Fulkerson
  - f(u,v) = 0 ∀ (u,v) // inițializarea fluxului
  - **Repetă** // creștere iterativă a fluxului
    - găsește un drum s..p..t pe care se poate mări fluxul (cale reziduală)
    - f = f + flux(s..p..t)
  - **Până când** nu se mai poate găsi nici un drum s..p..t
  - **Întoarce** f

- În funcție de metodele de identificare a căii reziduale există mai mulți algoritmi ce urmează această metodă.

Tăieturi în rețele de flux

- Definiție: O tăietură (S,T) a unei rețele de flux G = partiționare a nodurilor în 2 mulțimi disjuncte S și T = V \ S astfel încât $s \in S$ și $t \in T$.
  - $f(S,T) = \Sigma_{x \in S} \Sigma_{y \in T} f(x,y)$ – fluxul prin tăietura
  - $c(S,T) = \Sigma_{x \in S} \Sigma_{y \in T} c(x,y)$ – capacitatea tăieturii
- Lema 5.18: Fluxul prin tăietură = fluxul prin rețea – $f(S,T) = |f|$
- Corolar 5.5: S, T – tăietură oarecare – fluxul maxim este limitat superior de capacitatea tăieturii $|f| \leq c(S,T)$

Exemplu de tăietură într-o rețea de flux



- f(S,T) = f(s,b) + f(a,b) + f(c,d) + f(c,b) = 4 + 1 + 4 - 3 = 6 = f(s,V)

- c(S,T) = c(a,b) + c(s,b) + c(c,d) = 18

Flux maxim – tăietură minimă

- Teorema 5.25 (Flux maxim – tăietură minimă): G = (V,E) rețea de flux – următoarele afirmații sunt echivalente:
  - f este o funcție de flux în G astfel încât |f| este flux maxim total în G;
  - rețeaua reziduală G_f nu are căi reziduale;
  - există o tăietură (S,T) astfel încât |f| = c(S,T).

Algoritmul Ford – Fulkerson

- Ford – Fulkerson(G,s,t)
  - **Pentru fiecare** (u,v) din E
    - f(u,v) = f(v,u) = 0 // inițializare
  - **Cât timp**
    - Există o cale reziduală p între s..t în G_f
      - $c_f(p) = \min\{c_f(u,v) \mid (u,v) \text{ din } p\}$ // capacitatea reziduală
      - **Pentru fiecare** (u,v) din p
        - $f(u,v) = f(u,v) + c_f(p)$
        - $f(v,u) = -f(u,v)$
  - **Întoarce** |f|

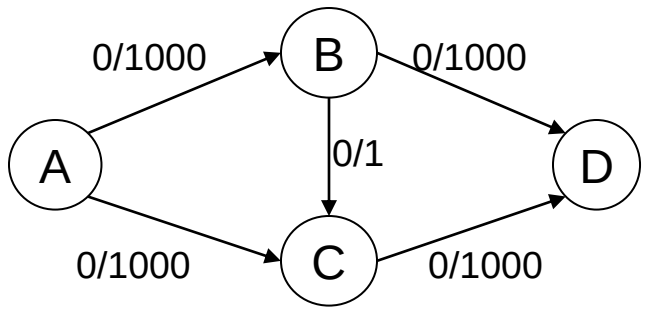
Complexitate?

Algoritmul Ford – Fulkerson (2)

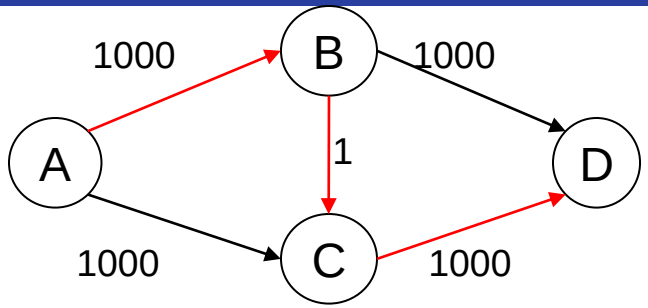
- Ford – Fulkerson(G,s,t)
  - **Pentru fiecare** (u,v) din E
    - f(u,v) = f(v,u) = 0 // O(E)
  - **Cât timp** // O(?)
    - Există o cale reziduală p între s..t în G_f
      - $c_f(p) = \min\{c_f(u,v) \mid (u,v) \text{ din } p\}$ // O(E)
      - **Pentru fiecare** (u,v) din p // O(E)
        - f(u,v) = f(u,v) + $c_f(p)$
        - f(v,u) = -f(u,v)
  - **Întoarce** |f|

Complexitate?

Exemplu Ford – Fulkerson (1)

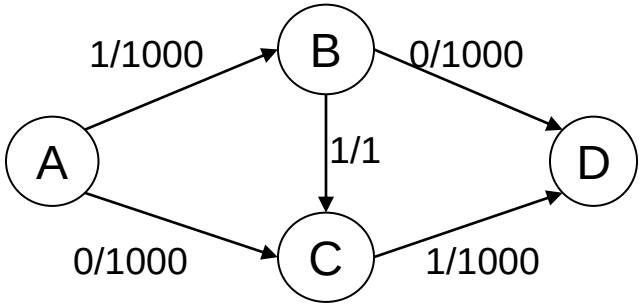


G_f

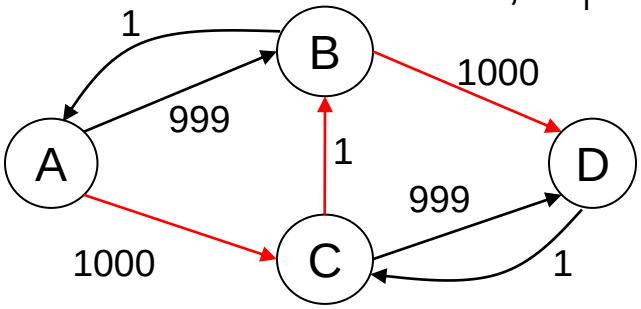


G

Cale reziduală: A-B-C-D; $C_f = 1$

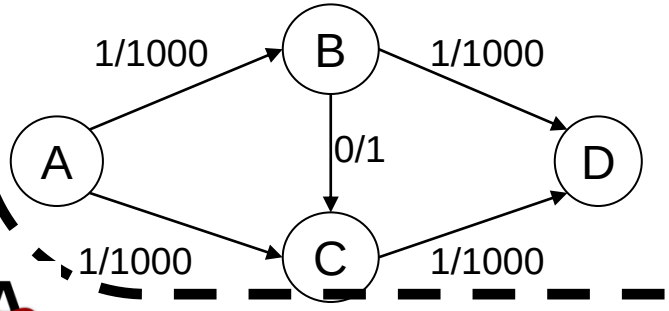


G_f

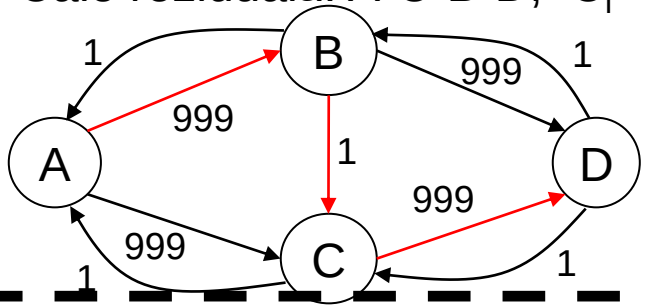


G

Cale reziduală: A-C-B-D; $C_f = 1$



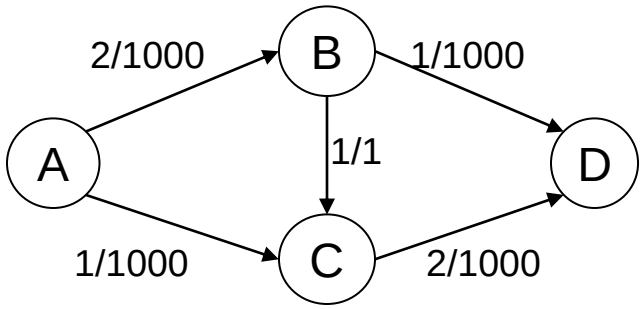
G_f



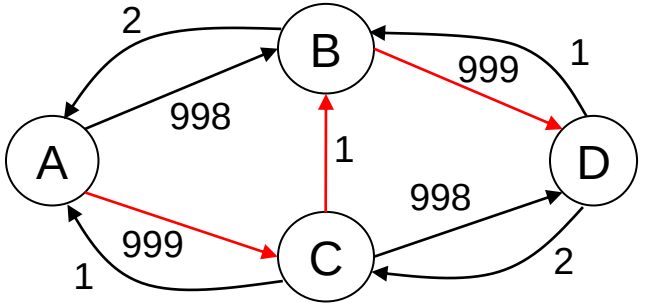
G

Cale reziduală: A-B-C-D; $C_f = 1$

Exemplu Ford – Fulkerson (2)



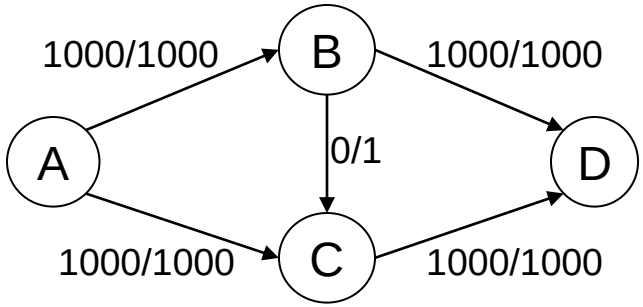
G_f



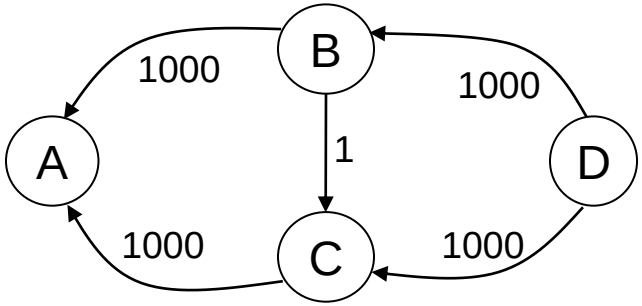
G

Cale reziduală: A-C-B-D; $C_f = 1$

. . .



G_f



Cale reziduală:Ø

După câți pași se ajunge la forma finală?

Complexitate Ford – Fulkerson

- Complexitate $O(E * f_{max})$
- f_{max} = fluxul maxim

Algoritmul Ford – Fulkerson – discuție

- Probleme ce pot să apară:
  - Se folosesc căi cu capacitate mică;
  - Se pun fluxuri pe mai multe arce decât este nevoie.

- Îmbunătățiri:
  - Se aleg căile reziduale cu capacitate maximă – complexitatea va depinde în continuare de f_{max} și de valoarea capacităților;
  - Se aleg căile reziduale cele mai scurte → în acest caz complexitatea nu mai depinde de f_{max} ci numai de numărul de arce (ex. Edmonds-Karp: identificarea căilor reziduale minime prin aplicarea unui BFS)

Algoritmul Edmonds – Karp (1)

- Edmonds – Karp(G, s, t)
  - **Pentru fiecare** (u,v) din E
    - f(u,v) = f(v,u) = 0 // inițializare
  - **Cât timp**
    - Există căi reziduale între s..t în G_f
      - Determină calea reziduală de lungime minimă p aplicând BFS
      - $c_f(p) = \min\{c_f(u,v) \mid (u,v) \text{ din } p\}$ // capacitatea reziduală
      - **Pentru fiecare** (u,v) din p
        - $f(u,v) = f(u,v) + c_f(p)$
        - $f(v,u) = -f(u,v)$
  - **Întoarce** |f|

Complexitate?

Algoritmul Edmonds – Karp (2)

- Edmonds – Karp(G, s, t)
  - **Pentru fiecare** (u,v) din E
    - f(u,v) = f(v,u) = 0 // O(E)
  - **Cât timp** // O(E*V) [vezi Cormen]
    - Există căi reziduale între s..t în G_f
      - Determină calea reziduală de lungime minimă p aplicând BFS // O(E)
      - $c_f(p) = \min\{c_f(u,v) \mid (u,v) \text{ din } p\}$ // O(E)
      - **Pentru fiecare** (u,v) din p // O(E)
        - $f(u,v) = f(u,v) + c_f(p)$
        - $f(v,u) = -f(u,v)$
  - **Întoarce** |f|

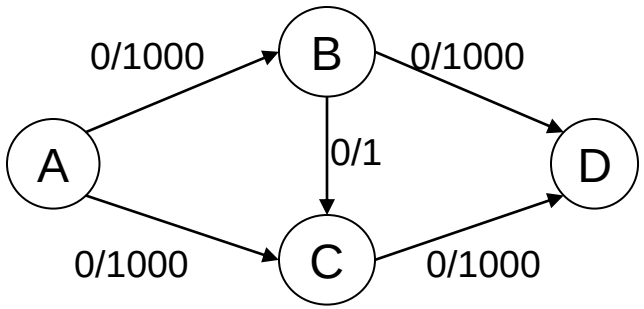
Lema 27.8: folosind Edmonds – Karp, pentru $\forall$ v $\in$ V \ {s, t}, δ (s,v) in reteaua reziduala G_f creste monoton

De câte ori un arc poate fi critic în rețeaua reziduală? O(V)
Câte arce? O(E)

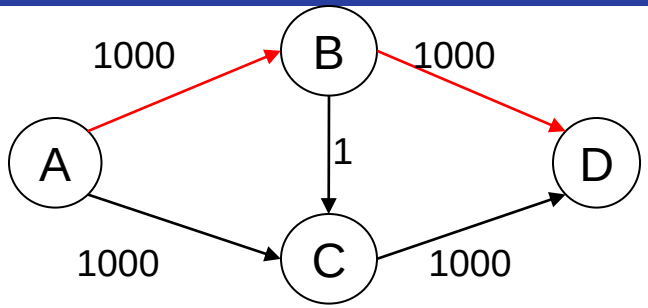
Complexitate?

$O(E^2 * V)$

Exemplu Edmonds-Karp

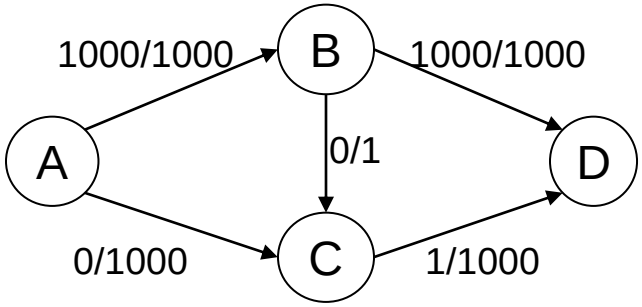


G_f

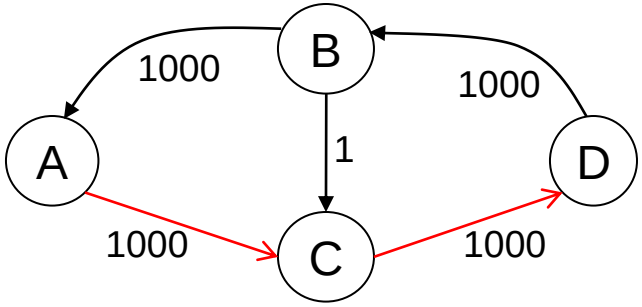


G

Cale reziduală: A-B-D; $C_f = 1000$

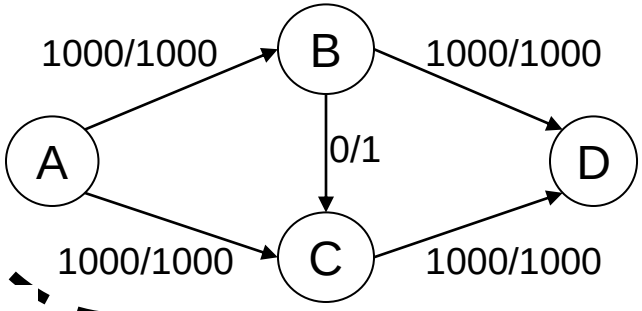


G_f

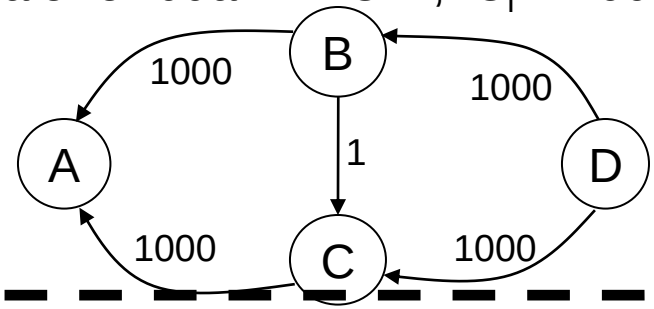


G

Cale reziduală: A-C-D; $C_f = 1000$



G_f



Cale reziduală: Ø

Pompare preflux (1)

- Idee: Simularea curgerii lichidelor într-un sistem de conducte ce leagă noduri aflate la diverse înălțimi;
- Sursa – înălțime |V|;
- Inițial toate nodurile exceptând sursa sunt la înălțime 0;
- Destinația rămâne în permanență la înălțimea 0!

Pompare preflux (2)

- Există un preflux inițial în rețea obținut prin încărcarea la capacitate maximă a tuturor conductelor ce pleacă din s;
- Excesul de flux dintr-un nod poate fi stocat într-un rezervor al nodului (Notat e(u));
- Când un nod u are flux disponibil în rezervor și o conductă spre un alt nod v nu este încărcată complet → înălțimea lui u este crescută pentru a permite curgerea din u în v.

Pompare preflux – Definiții (1)

- G = (V,E) rețea de flux;
- Definiție: Preflux = f: V x V → $\Re$ astfel încât să fie satisfăcute restricțiile:
  - $f(u,v) \leq c(u,v), \forall (u,v) \in E$ – respectarea capacității arcelor;
  - $f(u,v) = -f(v,u), \forall u,v \in V$ – simetria fluxului;
  - $\Sigma_{v \in V} f(v,u) \geq 0, \forall u \in V \setminus \{s\}$ – ~~conservarea fluxului.~~
- Definiție: Supraîncărcare a unui nod:
  - $e(u) = f(V,u) \geq 0 , \forall u \in V \setminus \{s\}$.

Pompare preflux – Definiții (2)

- Definiție: O funcție h: V → N este o funcție de înălțime dacă îndeplinește restricțiile:
  - h(s) = |V| – fixă;
  - h(t) = 0 – fixă;
  - $h(u) \leq h(v) + 1$ pentru orice arc rezidual $(u,v) \in G_f$ – variabilă.

- Lema 5.19: G – rețea de flux, h: V → N este o funcție de înălțime. Dacă $\forall u, v \in V$, $h(u) > h(v) + 1$ atunci arcul (u,v) nu este arc rezidual.

Pompare preflux – Metode folosite

- Pompare(u,v) // pompează fluxul în exces (e(u) > 0)
  // are loc doar dacă diferența de înălțime dintre u și v este 1
  // (h(u) = h(v) + 1), altfel nu e arc rezidual și nu ne interesează
  - d = min(e(u), c_f(u,v)); // cantitatea de flux pompată
  - f(u,v) = f(u,v) + d; // actualizare flux pe arcul (u,v)
  - f(v,u) = -f(u,v); // respectarea simetriei
  - e(u) = e(u) – d; // actualizare supraîncărcare la sursă
  - e(v) = e(v) + d; // actualizare supraîncărcare la destinație

- Înălțare(u) // mărește h(u) dacă u are flux în exces
  // (e(u) > 0) și u $\notin$ {s, t} $\forall$(u,v) $\in$ G_f avem h(u) ≤ h(v)
  - h(u) = 1 + min{h(v) | (u,v) $\in$ G_f}

Complexitate?

Pompare preflux – Inițializare

- Init_preflux(G, s, t)
  - **Pentru fiecare** (u ∈ V)
    - e(u) = 0 // inițializare exces flux în nodul u
    - h(u) = 0 // inițializare înălțime nod u
  - **Pentru fiecare** ((u,v) ∈ E) // inițializare fluxuri
    - f(u,v) = 0
    - f(v,u) = 0
  - h(s) = |V| // inițializare înălțime sursă
  - **Pentru fiecare** (u ∈ succs(s) \ {s})
    - f(s,u) = c(s,u); // actualizare flux
    - f(u,s) = -c(s,u);
    - e(u) = c(s,u) // actualizare exces

Complexitate?

Pompare preflux – Algoritm

- Pompare_preflux(G, s, t)
  - Init_preflux(G, s, t) // inițializarea prefluxului
  - **Cât timp** (1) // cât timp pot face pompări sau înălțări
    - **Dacă** ($\exists u \in V \setminus \{s, t\}, v \in V$ | e(u) > 0 **și** $c_f(u,v) > 0$ **și** h(u) = h(v) + 1) // încerc să pompez
      - Pompare(u,v); **continuă;**
    - **Dacă** ($\exists u \in V \setminus \{s, t\}, v \in V$ | e(u) > 0 **și** $\forall (u,v) \in E_f$, $h(u) \leq h(v)$)
      - Înălțare(u); **continuă;** // încerc să înalț
    - **Întrerupe;** // nu mai pot face nimic → am ajuns la flux max
  - **Întoarce** e(t) // e(t) = |f| = fluxul total în rețea

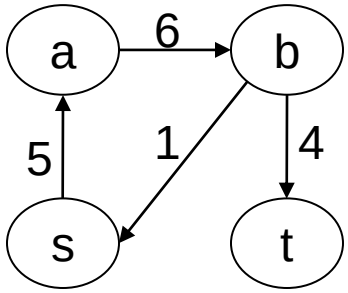
- Complexitate?

Pompare preflux – Complexitate

- Init_preflux: O(E)
- Pompare(u,v): O(1)
- Înălțare(u): O(V) – implică găsirea minimului dintre nodurile succesoare
- Cât timp: [vezi Cormen]
  - Câte înălțări?
    - Care e înălțimea maximă? 2 |V| - 1 – drum rezidual de lungime maximă
    - Care este numărul maxim total de înălțări? (2 |V| - 1) (|V| - 2)
  - Câte pompări?
    - Pompări saturate: 2 |V| |E| - de câte ori un arc poate fi saturat? (în funcție de suma h(u) + h(v))
    - Pompări nesaturate: 4 $|V|^2$ (|V| + |E|) – sumă înălțimi noduri excedentare
- Complexitate totală: $O(V^2 * E)$ [vezi Cormen]

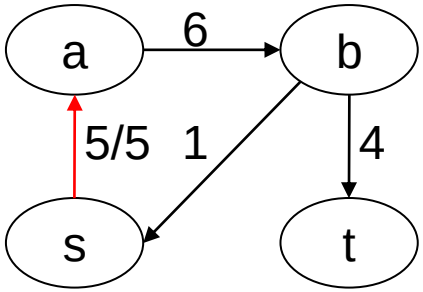
Exemplu Pompare preflux (1)

G

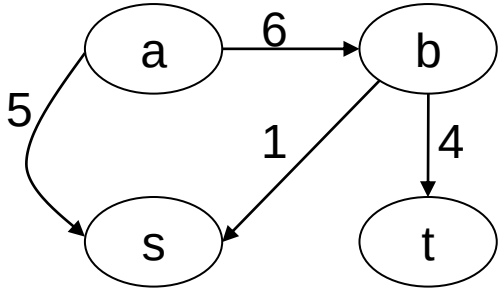


Init_preflux

h(s) = 4
h(a) = h(b) = h(t) = 0
e(a) = 5
e(s) = e(b) = e(t) = 0



G_f



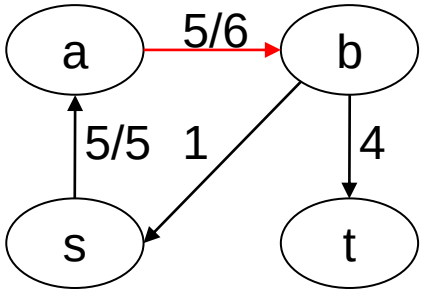
G_f

Înălțare
(a)

h(s) = 4
h(a) = 1
h(b) = h(t) = 0
e(a) = 5
e(s) = e(b) = e(t) = 0

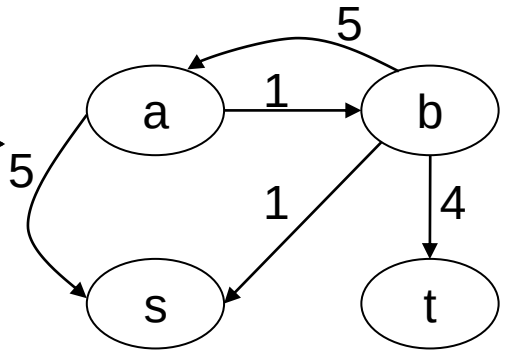
Pompare
(a,b)

Exemplu Pompare preflux (2)



h(s) = 4
h(a) = 1
h(b) = h(t) = 0
e(b) = 5
e(s) = e(a) = e(t) = 0

G_f

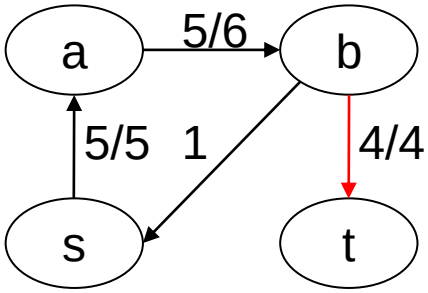


Înălțare
(b)

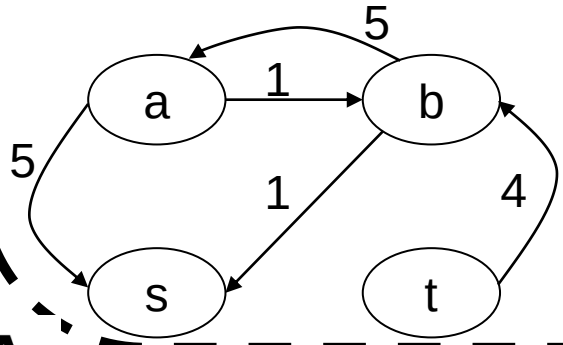
h(s) = 4
h(a) = h(b) = 1
h(t) = 0
e(t) = 4
e(s) = e(a) = 0
e(b) = 1

G_f

Pompare
(b,t)



h(s) = 4
h(a) = h(b) = 1
h(t) = 0
e(b) = 5
e(s) = e(a) = e(t) = 0

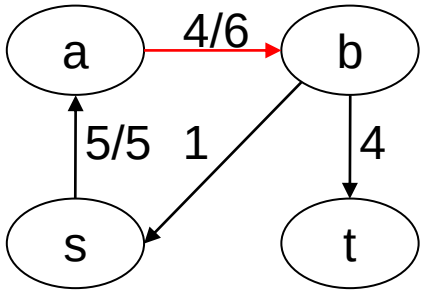


Înălțare
(b)

h(s) = 4 e(s) = 0
h(a) = 1 e(a) = 0
h(b) = 2 e(b) = 1
h(t) = 0 e(t) = 4

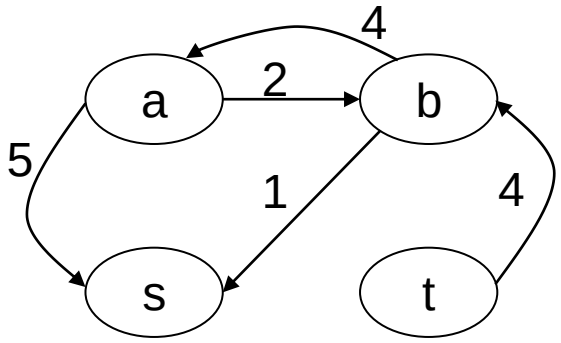
Pompare
(b,a)

Exemplu Pompare preflux (3)



h(s) = 4 e(s) = 0
h(a) = 1 e(a) = 1
h(b) = 2 e(b) = 0
h(t) = 0 e(t) = 4

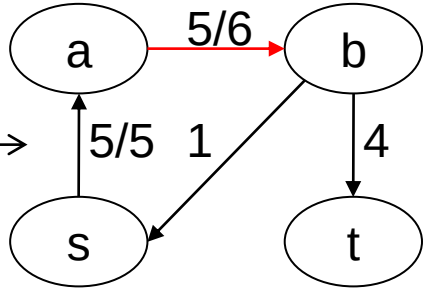
G_f



Înălțare
(a)

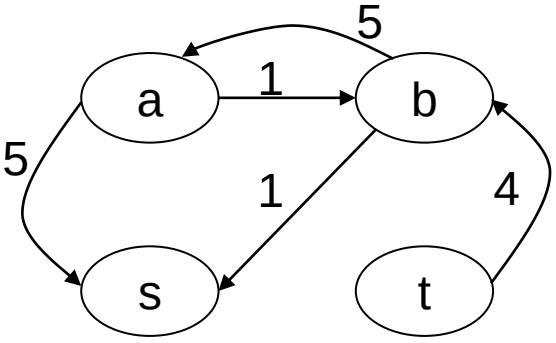
h(s) = 4 e(s) = 0
h(a) = 3 e(a) = 1
h(b) = 2 e(b) = 0
h(t) = 0 e(t) = 4

Pompare
(a,b)



h(s) = 4 e(s) = 0
h(a) = 3 e(a) = 0
h(b) = 2 e(b) = 1
h(t) = 0 e(t) = 4

G_f

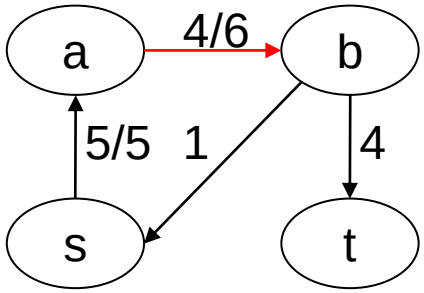


Înălțare
(b)

h(s) = 4 e(s) = 0
h(a) = 3 e(a) = 0
h(b) = 4 e(b) = 1
h(t) = 0 e(t) = 4

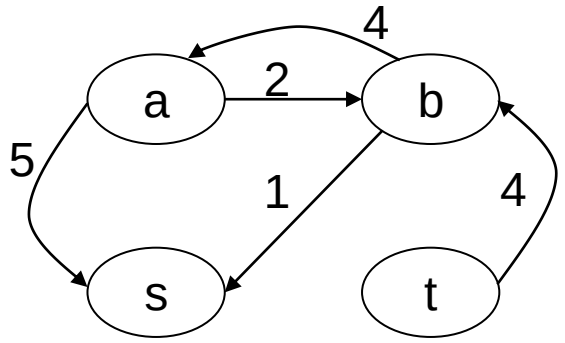
Pompare
(b,a)

Exemplu Pompare preflux (4)



h(s) = 4 e(s) = 0
h(a) = 3 e(a) = 1
h(b) = 4 e(b) = 0
h(t) = 0 e(t) = 4

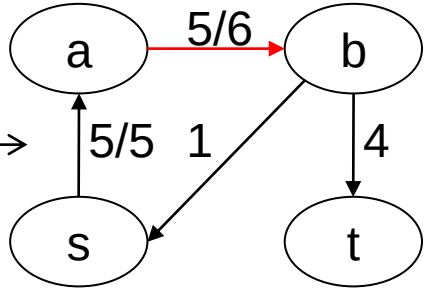
G_f



Înălțare
(a)

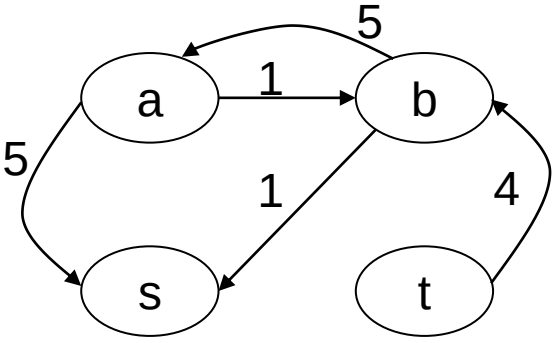
h(s) = 4 e(s) = 0
h(a) = 5 e(a) = 1
h(b) = 4 e(b) = 0
h(t) = 0 e(t) = 4

Pompare
(a,b)



h(s) = 4 e(s) = 0
h(a) = 5 e(a) = 0
h(b) = 4 e(b) = 1
h(t) = 0 e(t) = 4

G_f

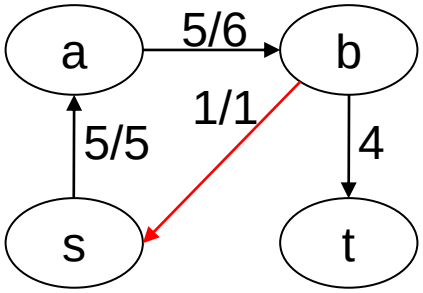


Înălțare
(b)

h(s) = 4 e(s) = 0
h(a) = 5 e(a) = 0
h(b) = 5 e(b) = 1
h(t) = 0 e(t) = 4

Pompare
(b,s)

Exemplu Pompare preflux (5)



h(s) = 4 e(s) = 1
h(a) = 5 e(a) = 0
h(b) = 5 e(b) = 0
h(t) = 0 e(t) = 4

- Care e înălțimea maximă? $5 < 2 * 4 – 1$ $(2 |V| - 1)$
- Care este numărul total de înălțări? $7 < (2 * 4 - 1) * (4 - 2)$ $((2 |V| - 1) (|V| - 2))$
- Pompări saturate: $2 < 2 * 4 * 4$ $(2 |V| |E|)$
- Pompări nesaturate: $5 < 4 * 4^2 * (4 + 4)$ $(4 |V|^2 (|V| + |E|))$
- Număr total de pași: $14 < 4^2 * 4$ $(O(V^2 * E))$

ÎNTREBĂRI?



Bibliografie curs 13

[1] C. Giumale – Introducere in Analiza Algoritmilor - cap. 7

[2] http://frost.ics.uci.edu/cs171/PathDemo.zip

[3] http://www.policyalmanac.org/games/aStarTutorial.htm

[4] http://www.ai.mit.edu/courses/6.034b/searchcomplex.pdf



Proiectarea Algoritmilor

Curs 13 - Algoritmi euristici de explorare



Bibliografie

[1] C. Giumale – Introducere in Analiza Algoritmilor - cap. 7

[2] http://frost.ics.uci.edu/cs171/PathDemo.zip

[3] http://csis.pace.edu/~benjamin/teaching/cs627/webfiles/Astar.pdf

[4] http://www.ai.mit.edu/courses/6.034b/searchcomplex.pdf



Cuprins

- Explorarea spațiului stărilor problemei
- Explorare informată irevocabilă
- Explorări tentative informate
  - Explorare lacomă
  - Explorare tentativă completă
  - Explorare A*

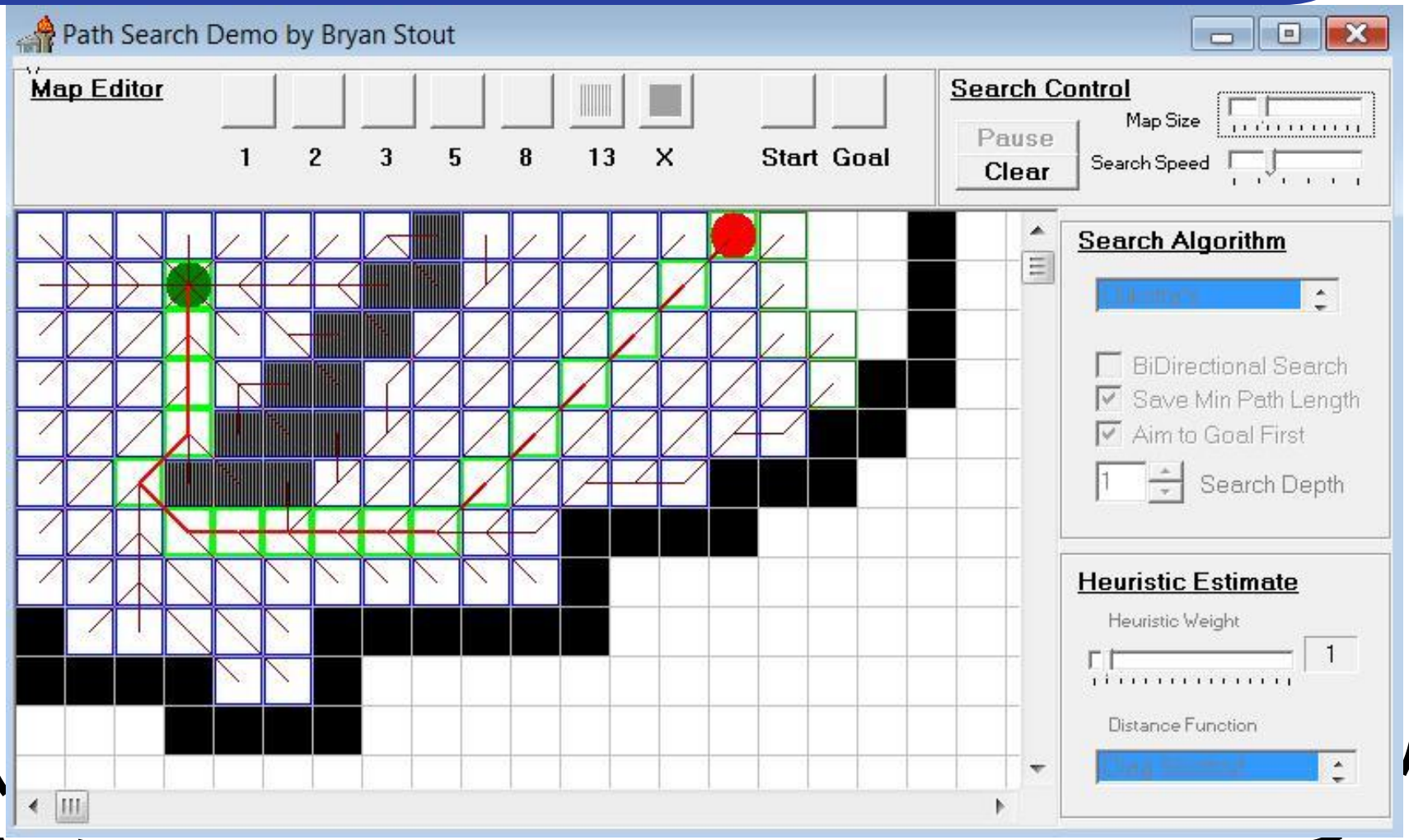
Probleme cu căutările neinformate

- Modelul unor probleme este prea complicat → variantele de rezolvare se bazează pe explorarea spațiului stărilor.

- Probleme:
  - Deseori se calculează prea mult (ex: drumul optim între 2 puncte folosind Dijkstra) - ex: Dijkstra.
  - În cazul grafurilor infinite sau nedescoperite încă, algoritmii clasici fie sunt ineficienți, fie nu garantează găsirea soluției.

- Soluție:
  - Rezolvarea să nu se mai bazeze numai pe calculele exacte ci și pe experiența anterioară (euristici) → direcționarea căutării.

Exemplu Dijkstra



Explorarea spațiului stărilor problemei

Spațiul stărilor unei probleme

- Definiție: Stare a problemei = abstractizare a unei configurații valide a universului problemei, configurație ce determină univoc comportarea locală a fenomenului descris de problemă.

- Definiție: Spațiul stărilor = graf în care nodurile corespund stărilor problemei, iar arcele desemnează tranzițiile valide între stări.
  - Caracteristică importantă: nu este cunoscut apriori, ci este descoperit pe măsura explorării!
  - Descriere
    - Nodul de start (starea inițială);
    - Funcție de expandare a nodurilor (produce lista nodurilor asociate stărilor valide în care se poate ajunge din starea curentă);
    - Predicat de testare dacă un nod corespunde unei stări soluție.

Obiectivele navigării prin spațiul stărilor

- Cartografierea sistematică a spațiului stărilor.

- Asamblarea soluțiilor parțiale care în final conduc la soluția finală. Această soluție finală poate fi:
  - Identificarea stărilor soluție (poziționarea a n regine pe tabla de șah fără să se atace);
  - Drumul străbătut de la starea inițială spre o stare soluție (acoperirea tablei de șah cu un cal);
  - Strategia de rezolvare = arbore multicăi în care rădăcina este starea inițială, iar frunzele sunt stări soluție. În acest arbore, unele noduri corespund unor evenimente neprevăzute care influențează calea de urmat în rezolvare (identificarea monedei false dintr-un grup de 3 monede).

Căutări informate/neinformate; Algoritmi tentativi/irevocabili

- Definiție: Dacă explorarea se bazează pe informația acumulată în cursul explorării, informație prelucrată euristic (costuri)→ algoritm informat.
- Definiție: Dacă explorarea este ‘la întâmplare’ → algoritm neinformat.
- Definiție: Dacă algoritmul de explorare are posibilitatea să abandoneze calea curentă de rezolvare și să revină la o cale anterioară → algoritmi tentativi.
- Definiție: Altfel (algoritmul avansează pe o singură direcție) → algoritmi irevocabili.

Căutări informate vs neinformate

- Căutările informate beneficiază de informații suplimentare pe care le colectează și le utilizează în încercarea de a „ghici” direcția în care trebuie explorat spațiul stărilor pentru a găsi soluția.
- Aceste informații sunt stocate:
  - În nodurile din spațiul stărilor:
    - Starea problemei reprezentată de nod;
    - Părintele nodului curent;
    - Copii nodului curent (obținuți prin expandarea acestuia);
    - Costul asociat nodului curent care estimează calitatea nodului f(n);
    - Adâncimea de explorare.
  - În structuri auxiliare pentru diferențierea nodurilor în raport cu gradul de prelucrare:
    - Expandat (închis) – toți succesorii nodului sunt cunoscuți;
    - Explorat (deschis) – nodul e cunoscut, dar succesorii săi nu;
    - Neexplorat – nodul nu e cunoscut.



Listele CLOSED și OPEN

- OPEN = mulțimea (lista) nodurilor explorate (frontiera dintre zona cunoscută și cea necunoscută).
- CLOSED = mulțimea (lista) nodurilor expandate (regiunea cunoscută în totalitate).
- Explorarea zonelor necunoscute se face prin alegerea și expandarea unui nod din OPEN. După expandare, nodul respectiv e trecut în CLOSED.
- Majoritatea algoritmilor tentativi folosesc lista OPEN, dar doar o parte folosesc lista CLOSED.



Completitudine și optimalitate

- Definiție: Algoritm complet = algoritm de explorare care garantează descoperirea unei soluții, dacă problema acceptă soluție.
  - Algoritmii irevocabili sunt mai rapizi și consumă mai puține resurse decât cei tentativi, dar nu sunt compleți pentru că pierd informație.

- Definiție: Algoritm optimal = algoritm de explorare care descoperă soluția optimă a problemei.

Algoritm generic de explorare

- Explorare(StInit, test_sol)
  - OPEN = {constr_nod(StInit)}; // starea inițială
  - **Cât timp** (OPEN ≠ Ø)
    // mai am noduri de prelucrat
    - nod = selecție_nod(OPEN); // aleg un nod
    - **Dacă** (test_sol(nod)) **Întoarce** nod;
      // am găsit o soluție
    - OPEN = OPEN \ {nod} U expandare{nod};
      // extind căutarea
  - **Întoarce** insucces; // nu s-a găsit nicio soluție

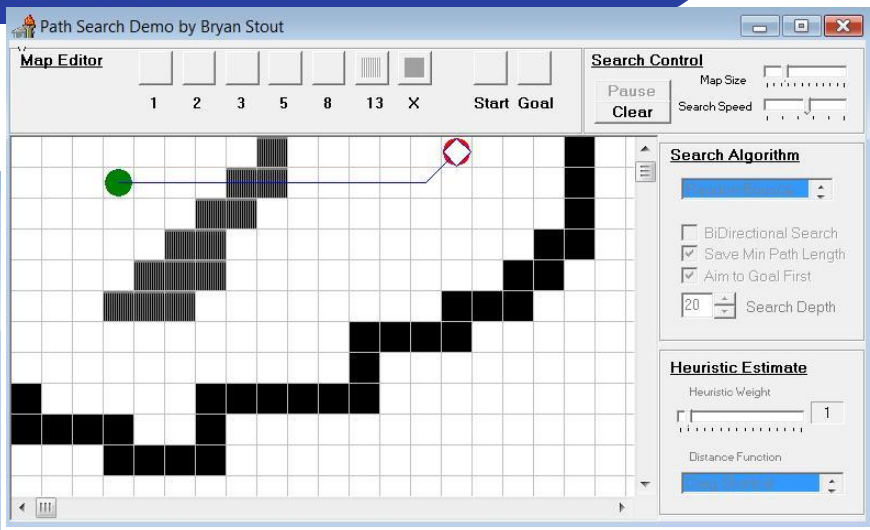
Discuție pe baza algoritmului

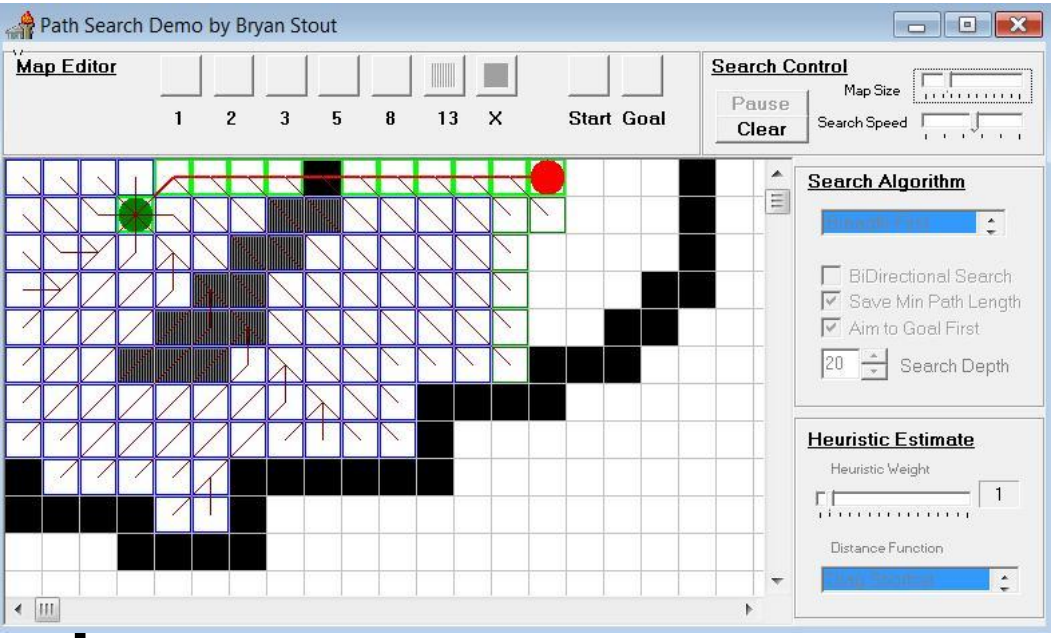
- Dacă **selecție_nod** se realizează independent de costul nodurilor din graful stărilor → căutare neinformată:
  - Dacă e de tip “random” → algoritm aleator - ex: RandomBounce
  - Dacă e de tip “primul venit, primul servit” → OPEN e coadă → BFS – ex: Breadth-first
  - Dacă e de tip “ultimul venit, primul servit” → OPEN e stivă → DFS – ex: Depth-first limitat / IDDFS

- Dacă **selecție_nod** se bazează pe un cost exact sau estimat (euristic) al stărilor problemei → căutare informată:
  - Estimarea costului și folosirea sa în procesul de selecție → esențiale pentru completitudinea, optimalitatea și complexitatea algoritmilor de explorare!

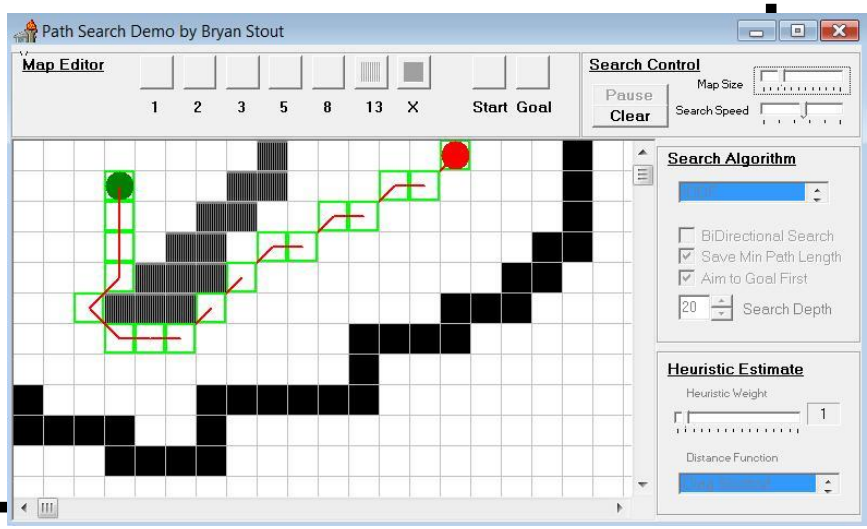
Exemplu de căutări neinformate

RandomBounce →





BFS ↑



IDDFS →

Explorare informată irevocabilă

Algoritm de explorare informată irevocabilă

- Ex: algoritmul alpinistului = algoritmul gradientului maxim.
- Fiecărui nod i se asociază o valoare f(nod) ≥ 0 → calitatea soluției parțiale din care face parte nodul.
- Se păstrează doar cel cu valoare maximă → OPEN are un singur element!



Gradientul Maxim

- Gradient_maxim(StInit, f, test_sol)
  - nod = constr_nod(StInit); // starea inițial
  - π(nod) = null;

  Inițializări

  - **Cât timp** (!test_sol(nod))

    Testez soluția

    - succs = expandare(nod); // nodurile au o valoare estimata
      // prin f
    - **Dacă** (succs = Ø) **Întoarce** insucces;

      Insucces

      // nu mai am noduri de prelucrat
    - succ = selecție_nod(succs); // f(succs) = max {f(n) | n ∈ succs}
    - π(succ) = nod;
    - nod = succ;

      Găsesc calea de continuat
  - **Întoarce** nod; // am ajuns la soluție

    Soluția

Gradientul Maxim

Optimalitate?

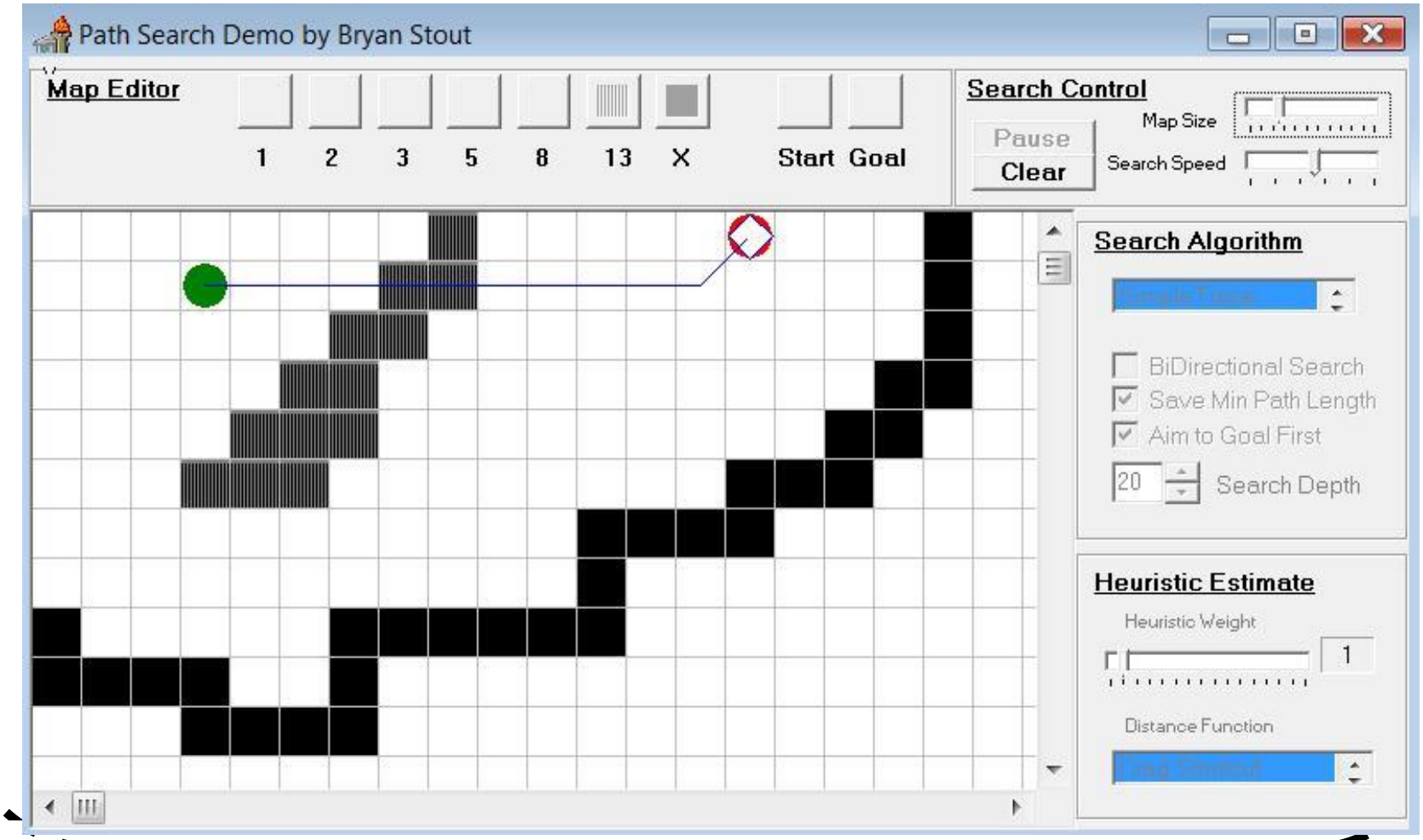
Completitudine?

Complexitate?

Ex: SimpleTrace



Exemplu Gradient Maxim



Discuție algoritmul gradientului maxim

- Algoritmul nu e complet și nu e optimal!
- Complexitate scăzută: O(bd) - b = branching factor, iar d = depth!
- Performanțele algoritmului depind foarte tare de forma teritoriului explorat și de euristica folosită (de dorit să existe puține optime locale și o euristică de evaluare cât mai bună).
- Pseudo-soluție eliminare optim local: se lansează algoritmul de mai multe ori plecând din stări inițiale diferite și se alege cea mai bună soluție obținută.

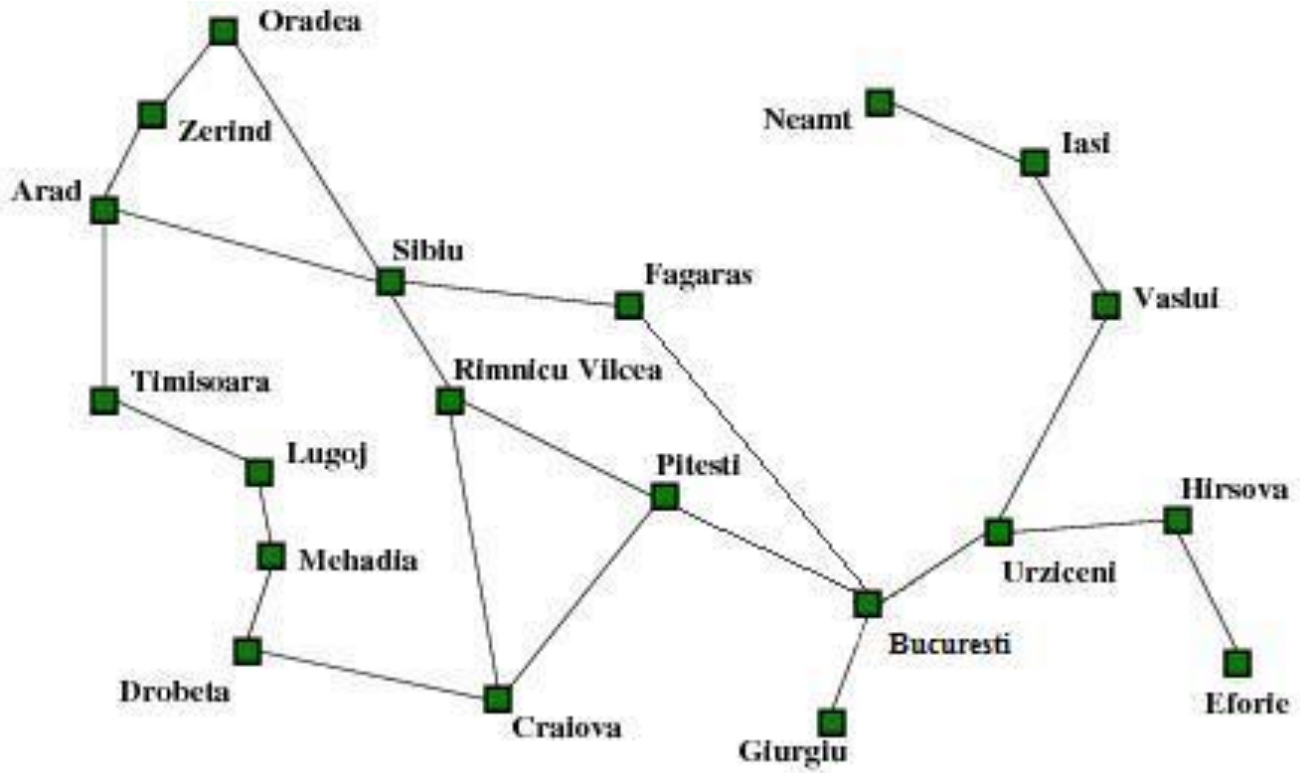


Explorări tentative informate

Detalii generale

- Păstrează toate nodurile de pe frontieră (OPEN), unii păstrând și nodurile expandate (CLOSED).
- Fiecare nod are un cost asociat f(n) ≥ 0 care estimează calitatea nodului (distanța de la nodul respectiv până la un nod soluție).
- Cu cât f(n) este mai mic, cu atât nodul este mai bun.

Prezentarea problemei



- Trebuie să ajungem în București din diverse puncte ale țării pe ruta cea mai scurtă.

Explorare lacomă

- Explorare_lacomă (StInit, f, test_sol)
  - nod = constr_nod(StInit); // starea inițiala
  - π(nod) = null;
  - OPEN = {nod};

  Inițializări

  - **Cât timp** (OPEN ≠ Ø) // mai am noduri de prelucrat
    - nod = selecție_nod (OPEN); // f(nod) = min {f(n) | n ∈ OPEN}
    - **Dacă** (test_sol(nod)) **Întoarce** nod;

    Soluția

    - OPEN = OPEN \ {nod}; // nodul nu e soluție, trebuie expandat
    - succs = expand(nod); // expandare nod
    - **Pentru fiecare** (succ ∈ succs) // actualizare succesori
      - OPEN = OPEN U {succ};
      - π(succ) = nod;

    Continuarea căutării

  - **Întoarce** insucces;

  Insucces

Optimalitate?

Completitudine?

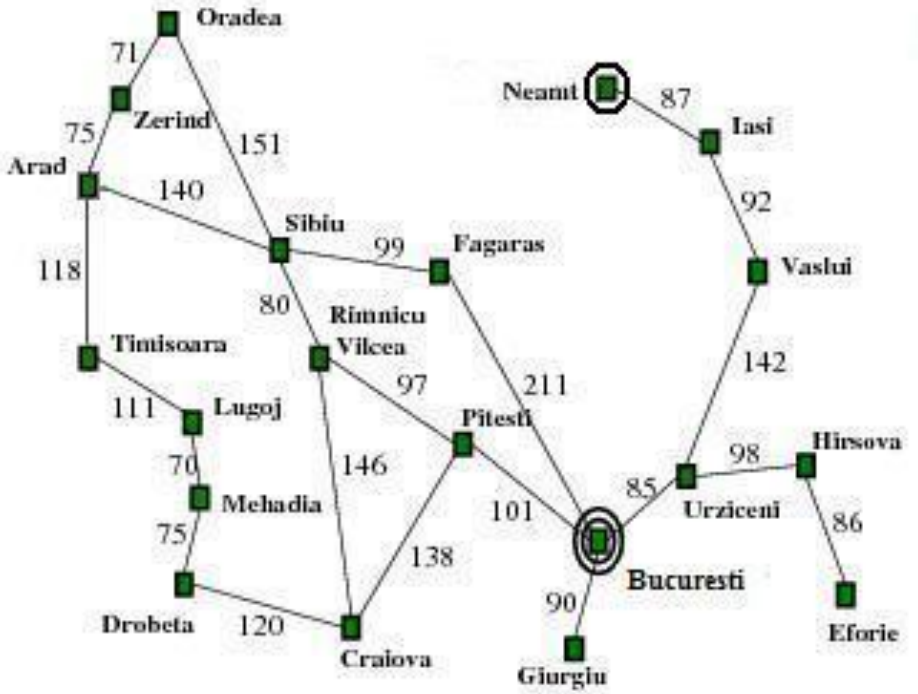
Problema?

f(nod) = distanța de
la nodul curent
până la nodul nod

f(Iași) = 87 → Iași

f(Neamț) = 87

f(Vaslui) = 92 → Neamț



- Drumul Neamț-București? → nu se termină algoritmul!
- → Explorarea lacomă nu e completă → trebuie să se rețină teritoriul deja parcurs ca să se evite ciclurile!

Explorare tentativă completă BF* (BEST FIRST) (1)

- BF*(StInit, f, test_sol)
  - nod = constr_nod(StInit); // starea inițială
  - π(nod) = null;
  - OPEN = {nod}; // noduri explorate dar neexpandate
  - CLOSED = Ø; // noduri expandate

  Inițializări

  - **Cât timp** (OPEN ≠ Ø)
    - nod = selecție nod (OPEN); // f(nod) = min {f(n) | n ∈ OPEN}
    - **Dacă** (test_sol(nod)) **Întoarce** nod;

    Soluția

    - OPEN = OPEN \ {nod};
    - CLOSED = CLOSED U {nod};
    - succs = expand(nod);

    Continuarea căutării

Explorare tentativă completă BF* (BEST FIRST) (2)

- **Pentru fiecare** (succ ∈ succs)
  - **Dacă** (succ ∉ CLOSED U OPEN) **atunci**
    - OPEN = OPEN U {succ}; π(succ) = nod;

Nod nou

  - **Altfel**
    - succ' = apariția lui succ în CLOSED U OPEN
    - **Dacă** (f(succ) < f(succ')) // am găsit o cale mai bună către succ și // redeschidem nodul

π(succ') = nod; // actualizez părintele

f(succ') = f(succ); // și costul nodului

Actualizări

Dacă (succ' ∈ CLOSED) // dacă era considerat expandat, îl redeschid

CLOSED = CLOSED \ {succ'}; OPEN = OPEN U {succ'};

Reprelucrare

- **Întoarce** insucces;

Insucces

ex: Best-first cu diverse euristici

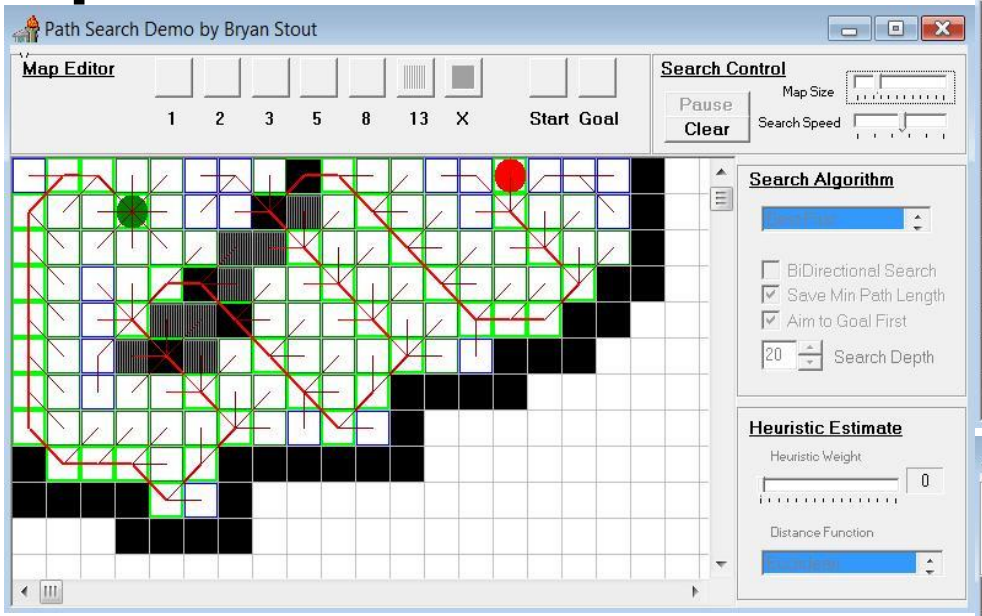
Optimalitate?

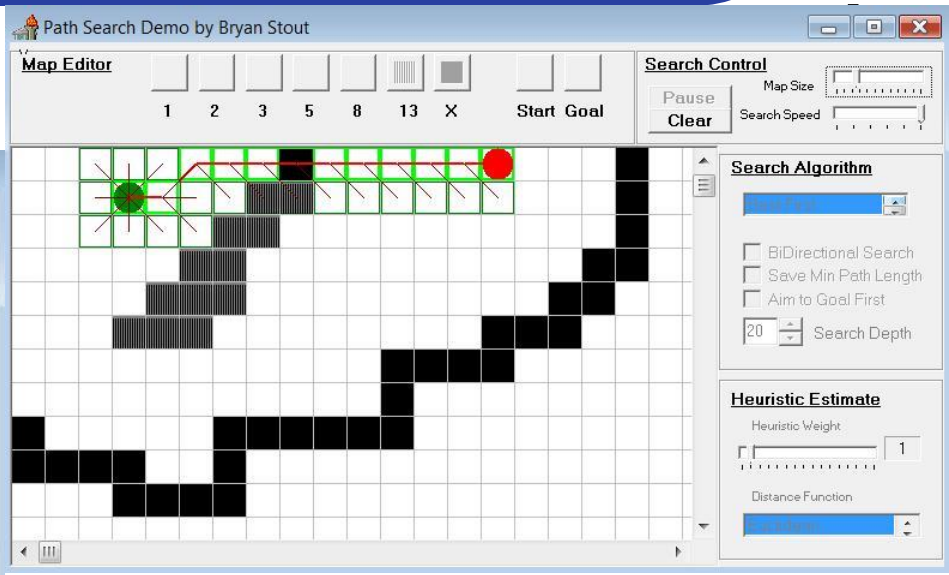
Completitudine?

Complexitate?

Exemple rulare BF* cu diferite euristici

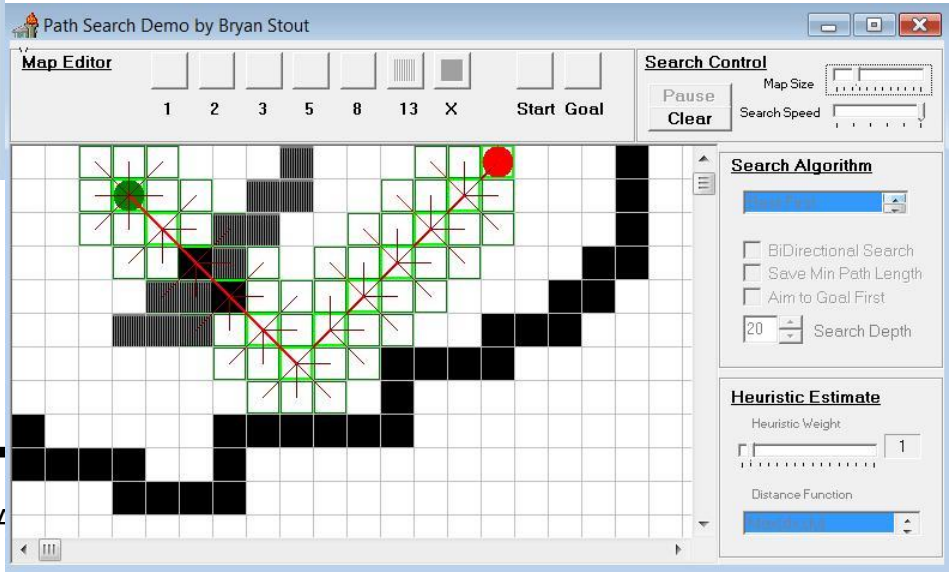
BF* - Euristică Distanta Euclidiană (11 pași, cost 23) →





BF* fără euristici ↑

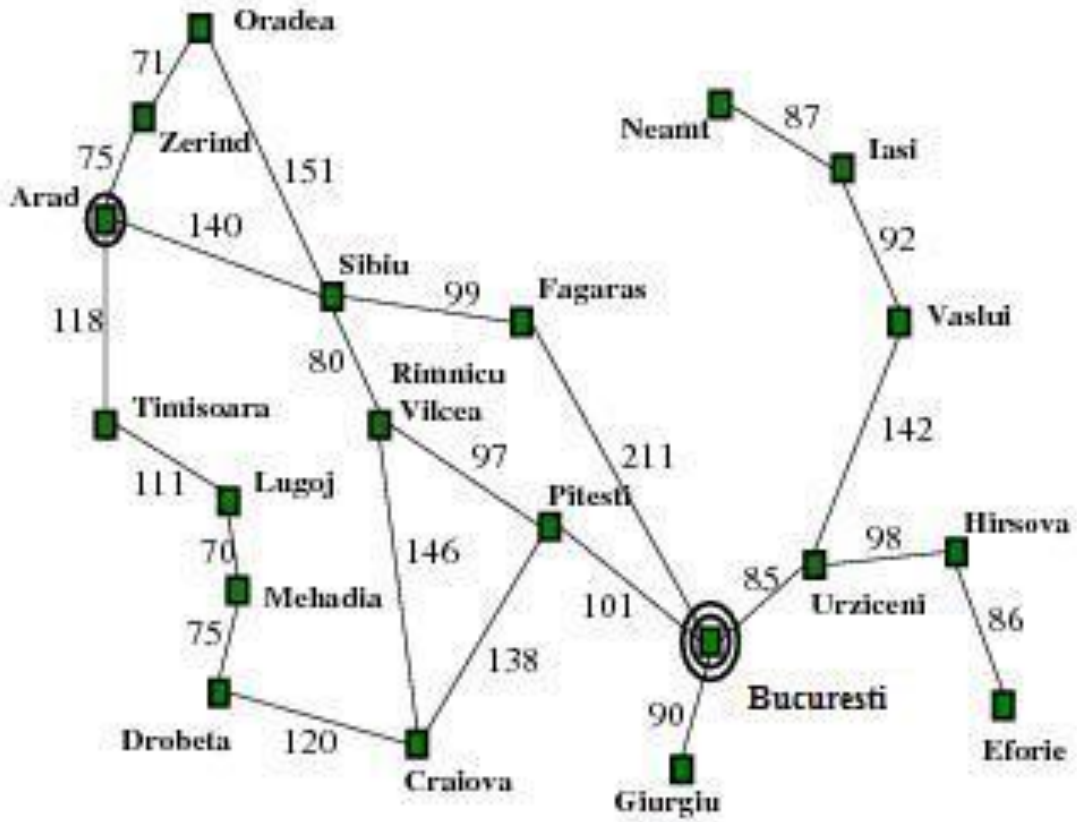
BF* - Euristică maximul distantei pe x și pe y (11 pași, cost 35)→



BF* - completitudine, optimalitate și complexitate

- Păstrează întreg teritoriul explorat:
  - OPEN – nodurile de pe frontieră;
  - CLOSED – nodurile expandate (unele noduri pot fi redeschise) → se evită ciclurile.
- Algoritmul este complet dar nu este optim → optimalitatea depinde de euristica f!
- Complexitate: $O(b^{d+1})$

Aplicație BF*



Distanta in linie dreapta pana la Bucuresti	
Arad	366
Craiova	160
Drobeta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

- Drumul optim Arad-București (f(nod) = distanța în linie dreaptă până la București)

A*

- Variantă a BF*.
- Nu poate fi aplicat mereu → trebuie demonstrat că păstrează ordinea soluțiilor unde soluțiile problemelor sunt drumuri în spațiul stărilor! (vezi Giumale pentru detalii!)
- Costul unui drum este aditiv (= suma costurilor arcelor) și crescător în lungul drumului.
- Folosește două funcții de cost:
  - h(n) - distanța estimată de la nodul curent până la nodul țintă;
  - g(n) - distanța parcursă de la nodul inițial până la nodul curent;
  - f(n) = g(n) + h(n).

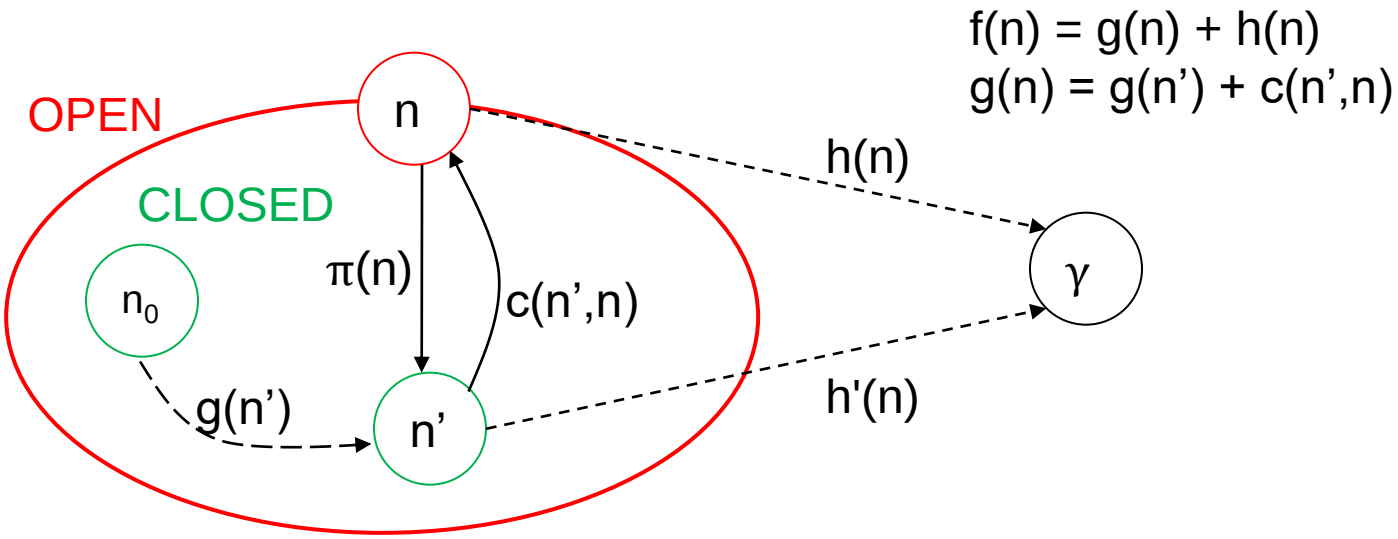
Notații (1)

- **S = (V,E)** – graful asociat spațiului stărilor problemei;
- **n_0** – nodul de start asociat stării inițiale a problemei;
- **$\Gamma \subseteq V$** – mulțimea nodurilor soluție. Un nod soluție se notează γ;
- **c(n,n') > 0** – costul arcului (n,n');
- **π(n)** – părintele lui n;
- **g(n)** – costul drumului n_0..n descoperit de algoritm la momentul curent de timp;
- **g_p(n)** – costul exact al porțiunii n_0..n din lungul unei căi date P;
- **g*(n)** – costul exact al unui drum optim n_0..n;

Notații (2)

- **$h(n) \geq 0$** – costul estimat al drumului optim de la nodul n la cel mai favorabil nod soluție $\gamma \in \Gamma$. În plus $h(\gamma) = 0$, pentru orice $\gamma \in \Gamma$;
- **$h^*(n)$** – costul exact al porțiunii de drum optim $n..\gamma$, pentru cel mai favorabil nod $\gamma \in \Gamma$ ($h^*(n) = \min \{cost(n..\gamma) | \gamma \in \Gamma\}$);
- **$f(n) = g(n) + h(n)$** – costul estimat al întregului drum $n_0..n..\gamma$, pentru cel mai favorabil nod $\gamma \in \Gamma$, unde porțiunea de drum $n_0..n$ este cea descoperită de algoritm la momentul curent de timp în cursul execuției;
- **$f^*(n) = g^*(n) + h^*(n)$** – costul exact al unui drum optim $n_0..n..\gamma$, pentru cel mai favorabil nod $\gamma \in \Gamma$;
- **$C = \min\{f^*(\gamma) | \gamma \in \Gamma\}$** – costul exact al unui drum optim $n_0..\gamma$, $\gamma \in \Gamma$. (C = costul soluției optime);

Funcția de evaluare A*



A* (1)

- A*(StInit, h, test_sol)
  - n_0 = constr_nod(StInit); // starea inițială

Inițializări

  - $f(n_0) = h(n_0)$; $g(n_0) = 0$; $\pi(n_0)$ = null; // euristici
  - OPEN = $\{n_0\}$; CLOSED = Ø; // și cozi
  - **Cât timp** (OPEN ≠ Ø) // mai am noduri de prelucrat
    - nod = selecție nod (OPEN); // f(nod) = min {f(n) | n ∈ OPEN}
    - **Dacă** (test_sol(nod)) **Întoarce** nod;

Soluția

    - OPEN = OPEN \ {nod}; // updatez OPEN
    - CLOSED = CLOSED U {nod}; // și CLOSE
    - succs = expand(nod); // determin nodurile succesoare

Continuarea căutării

A* (2)

- **Pentru fiecare** (succ ∈ succs) { // prelucrare succs
  - g_succ = g(nod) + c(nod, succ); // calculez g
  - f_succ = g_succ + h(succ); // calculez f = g + h

Prelucrare succesori

  - **Dacă** (succ ∉ CLOSED U OPEN) **atunci** // nod nou descoperit →
    - OPEN = OPEN U {succ}; // îl bag în OPEN
      g(succ)= g_succ; f(succ)= f_succ; π(succ) = nod;

Nod nou

  - **altfel** // a mai fost prelucrat
    - **Dacă** (g_succ < g(succ)) // verific dacă noul g este mai mic decât // anteriorul
      g(succ)= g_succ; f(succ)= f_succ; π(succ) = nod; // cale mai bună

Actualizări

Dacă (succ ∈ CLOSED) // dacă era considerat expandat, îl redeschid
CLOSED = CLOSED \ {succ'}; OPEN = OPEN U {succ'};

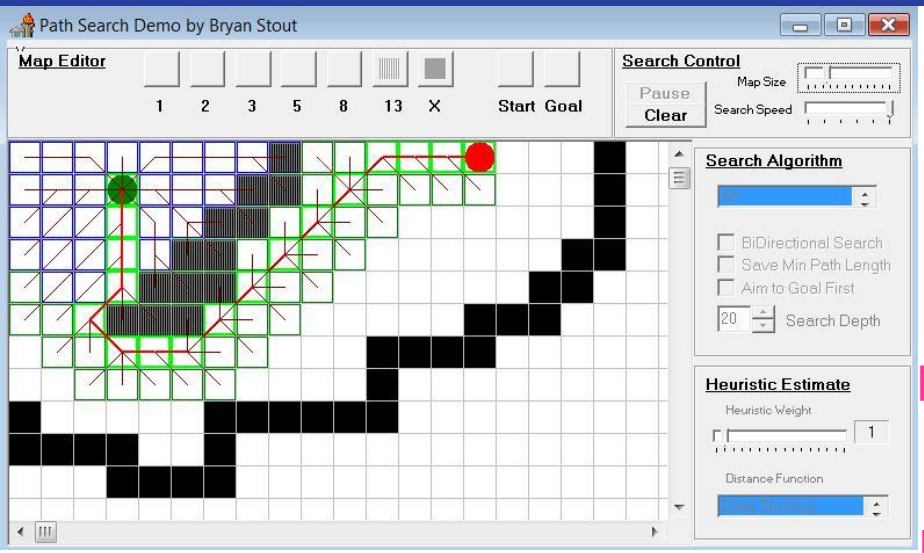
Reprelucrare

- **Întoarce** Insucces;

Insucces

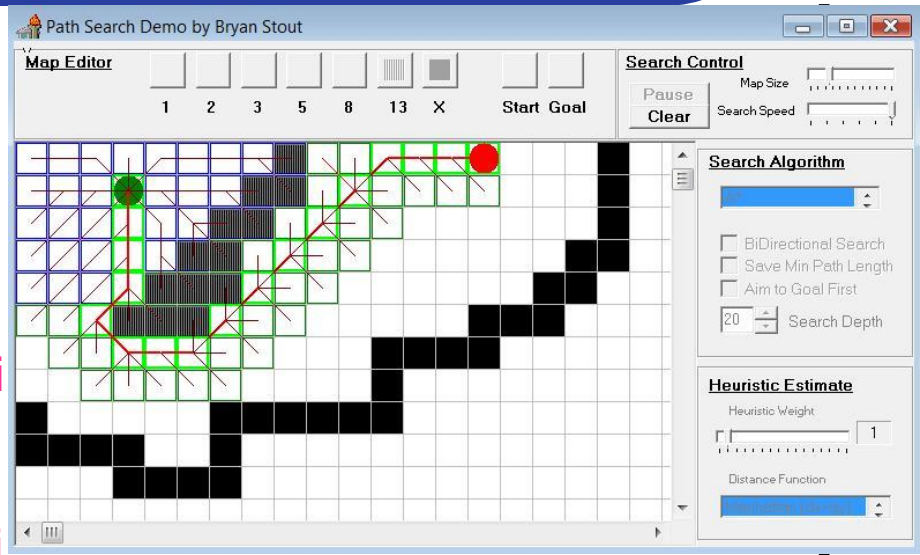
ex: A* cu diverse euristici

Exemple A* cu diverse euristici



A*

16 pași



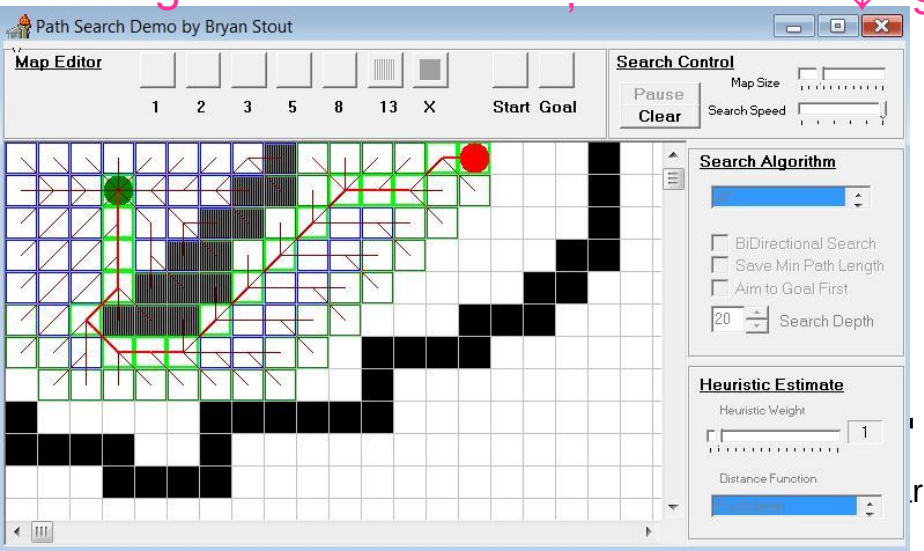
Euristici:

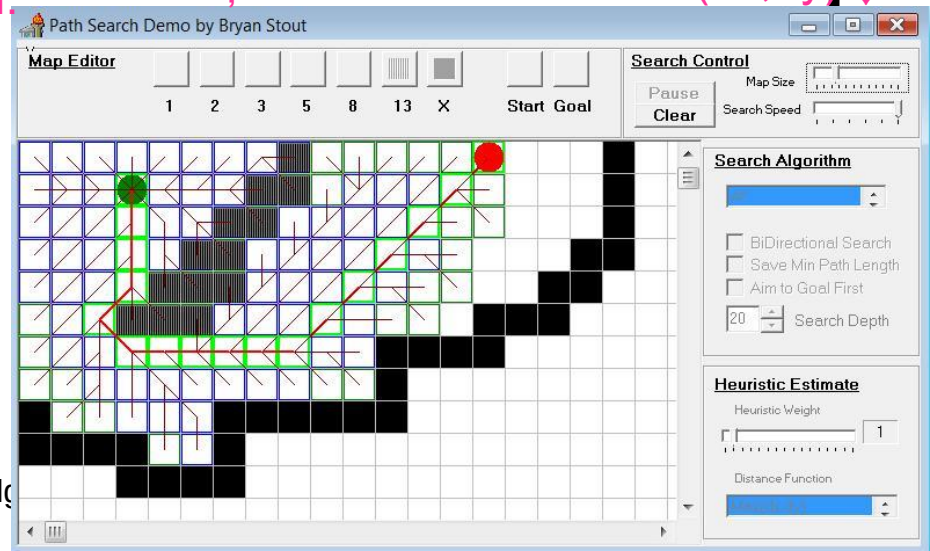
Diagonală↑

Distanța Euclidiană↓

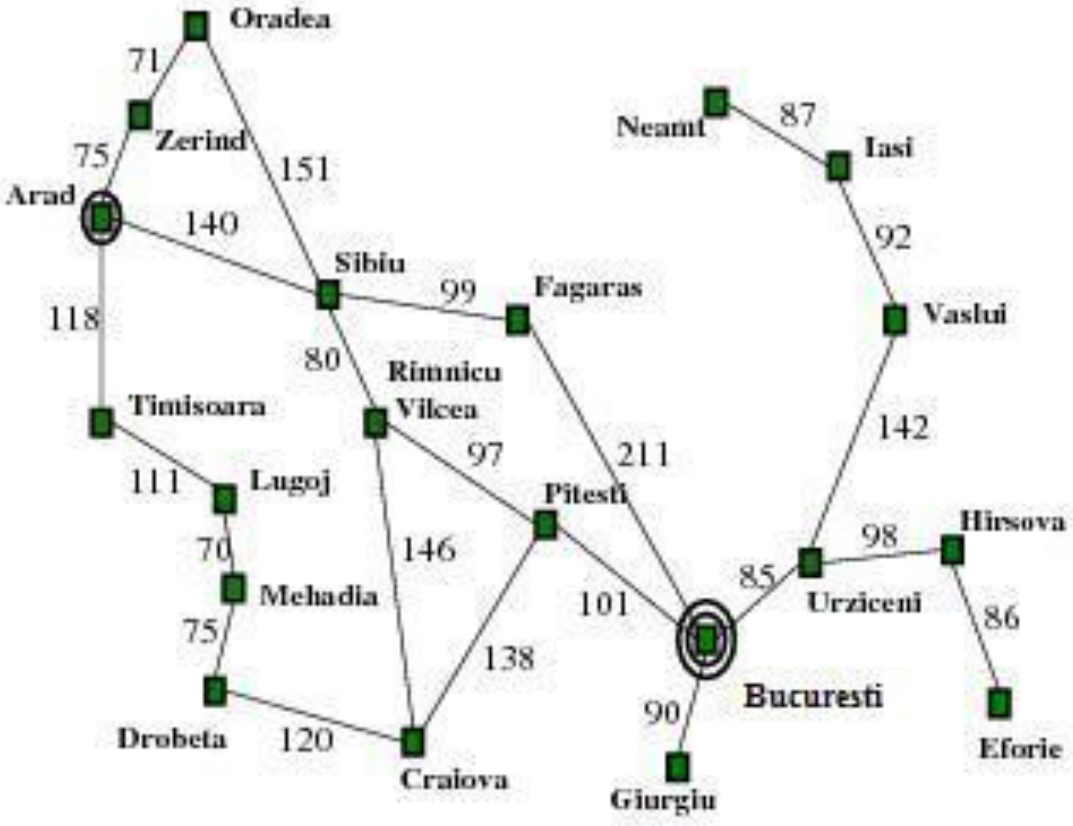
Distanța Manhattan ↑

Max(dx,dy) ↓





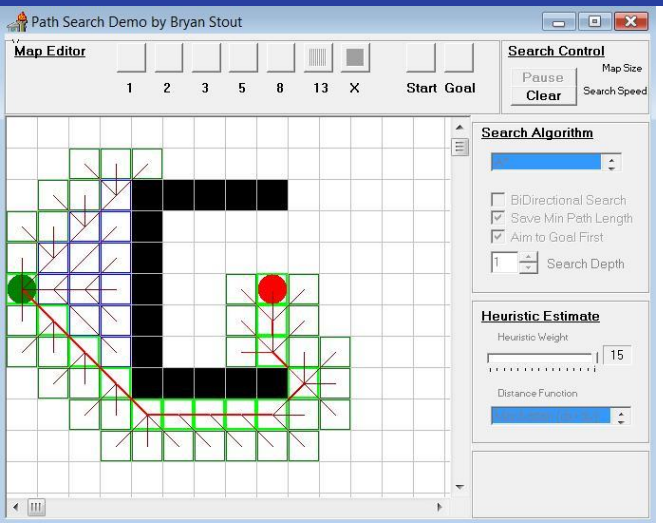
Aplicație A*



Distanta in linie dreapta pana la Bucuresti	
Arad	366
Craiova	160
Drobeta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

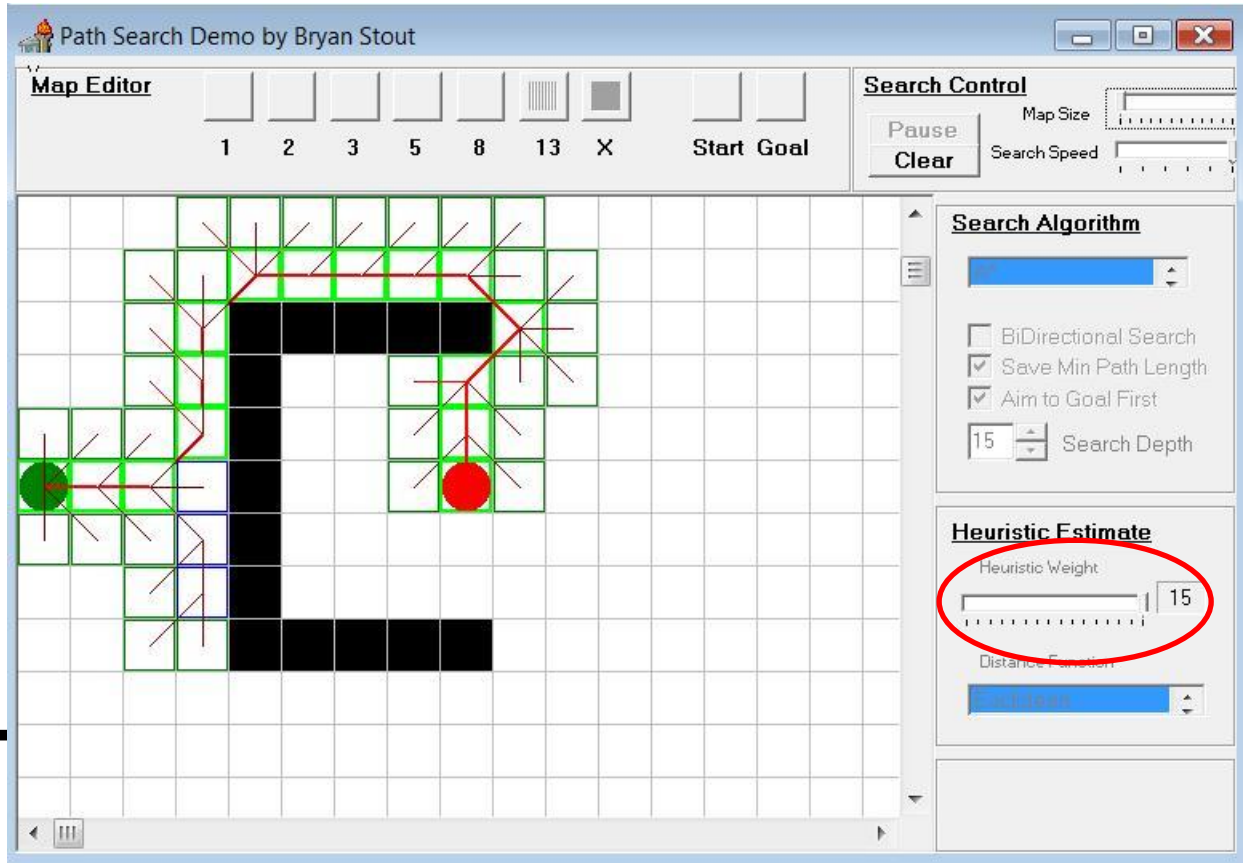
- Drumul optim Arad-București (h(n) = distanța în linie dreaptă până la București, g(n) = distanța parcursă)

Problemă



← A* – Distanța Manhattan – 12 pași

↓ A* – Distanța Euclidiana – 14 pași



Cum se explică???

PA

Algoritmul A* - completitudine și optimalitate (1)

- Teorema 7.1: Algoritmul A* este complet chiar dacă graful explorat nu este finit.

- Lema 7.1: Fie $P = n_0,n_1,...,n_m$ un drum oarecare în graful explorat de A*, astfel încât la un moment T al explorării toate nodurile din P sunt în CLOSED. Atunci, la orice moment de timp egal sau superior lui T, există inegalitatea $g(n_i) \leq g_p(n_i)$, $i = 0,m$:
  - Costul nodurilor din CLOSED poate să scadă, dar de fiecare dată când acest lucru se întâmplă, se pierde timp → scoaterea nodului din CLOSED, punerea în OPEN, prelucrarea acestuia încă o dată → trebuiesc evitate aceste situații → alegerea unei euristici cât mai bune care să minimizeze numărul acestor actualizări!

Algoritmul A* - completitudine și optimalitate (2)

- Definiție 7.2: Funcția euristică h este admisibilă dacă pentru orice nod n din spațiul stărilor $h(n) \leq h^*(n)$. Cu alte cuvinte, o euristică admisibilă h este optimistă și $h(\gamma) = 0$ pentru orice nod $\gamma \in \Gamma$.

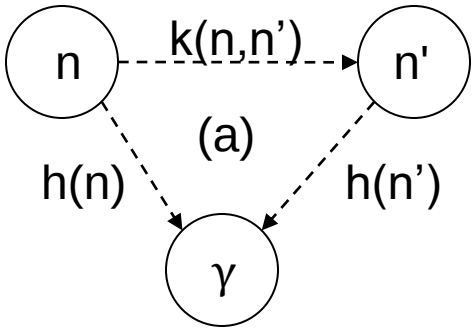
- Teorema 7.2: Algoritmul A* ghidat printr-o euristică admisibilă descoperă soluția optimă dacă există soluții.



Euristici – consistență și monotonie

- Definiție 7.4: O euristică h este consistentă dacă pentru oricare două noduri n și n' ale grafului explorat, astfel încât n' este accesibil din n, există inegalitatea: $h(n) \leq h(n') + k(n,n')$, unde $k(n,n')$ este costul unui drum optim de la n la n'.

- Definiție 7.5: O euristică h este monotonă dacă pentru oricare două noduri n și n' ale grafului explorat, astfel încât n' este succesorul lui n, există inegalitatea $h(n) \leq h(n') + c(n,n')$, unde $c(n,n')$ este costul arcului (n,n').

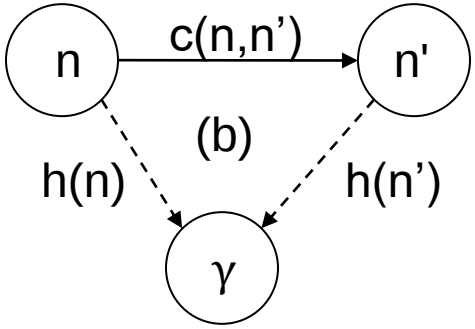


$h(n) \leq h(n') + k(n,n')$

Regula triunghiului pentru euristici:

← Consistență

→ Monotonie



$h(n) \leq h(n') + c(n,n')$

Consistență = monotonie

- Teorema 7.5: O euristică este consistentă ⇔ este monotonă.
  - Demonstrație:
    - h – consistentă → h – monotonă. Alegem n' ∈ succs(n) → k(n,n') = c(n,n') → h(n) ≤ h(n') + c(n,n') → h – monotonă.
    - h – monotonă → h – consistentă. Fie n = $n_1,n_2,\ldots,n_q$ = n', un drum optim n..n' cu cost k(n,n'). → h(n) = h(n_1) ≤ h(n_2) + c(n_1,n_2) ≤ h(n_3) + c(n_1,n_2) + c(n_2,n_3)… ≤ h(n_q) + c(n_1,n_2) + c(n_2,n_3) + …c(n_{q-1},n_q) = h(n_q) + k(n_1,n_q) → h(n) ≤ h(n') + k(n,n') → h – consistentă.

Consistență → admisibilitate

- Teorema 7.6: O euristică consistentă este admisibilă.
  - Demonstrație:
    - Fie h o euristică consistentă → $h(n) \leq h(n') + k(n,n')$, $\forall$ n' accesibil din n. Fie $n' = \gamma \in \Gamma$ → $k(n, \gamma) = \min \{ k(n, \gamma') \mid \gamma' \in \Gamma\} = h^*(n)$ → $h(n) \leq h(\gamma) + h^*(n)$, dar $h(\gamma) = 0$ → $h(n) \leq h^*(n)$ → euristică admisibilă.
- Corolar 7.2: O euristică monotonă este admisibilă.

Dominanță - Definiții și teoremă

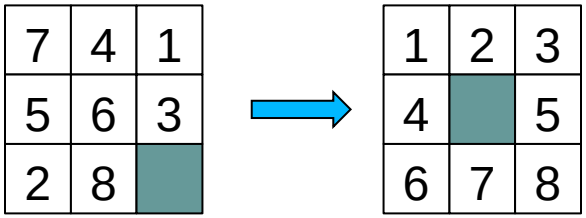
- Definiție 7.6: Fie h_1 și h_2 două euristici admisibile. Spunem că h_1 este mai informată decât h_2 dacă $h_2(n) < h_1(n)$ pentru orice nod $n \notin \Gamma$ din graful spațiului de stare explorat.

- Definiție 7.7: Un algoritm A_1^* domină un algoritm A_2^* dacă orice nod expandat de A_1^* este expandat și de A_2^*. (eventual, A_2^* expandează noduri suplimentare față de A_1^*, deci A_1^* poate fi mai rapid ca A_2^*.)

- Teorema 7.11: Dacă o euristică monotonă h_1 este mai informată decât o euristică monotonă h_2, atunci un algoritm A_1^* condus de h_1 domină un algoritm A_2^* condus de h_2.

Dominanța - Exemplu

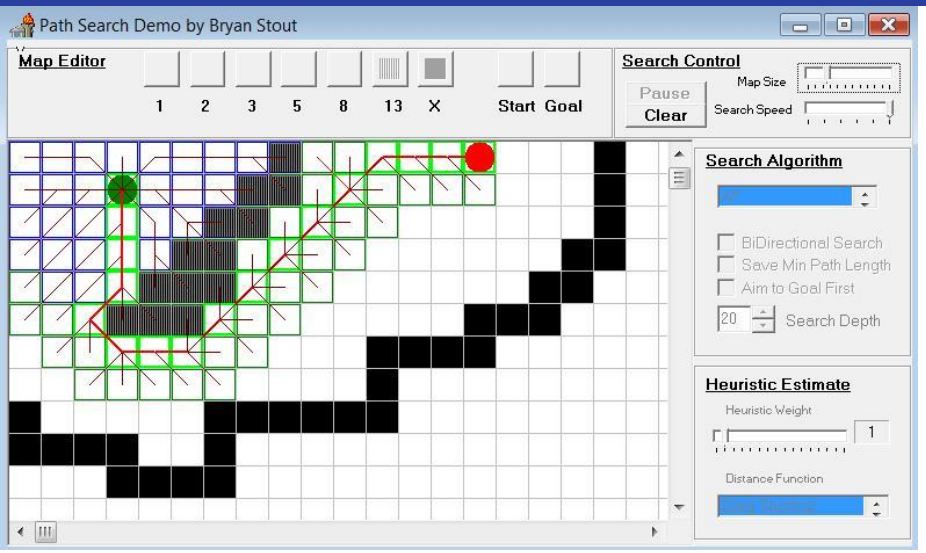
- Considerăm jocul 8-pătrățele care trebuie aranjat pornind de la forma inițială prin mutarea locului ‘liber’ astfel încât să ajungem la forma finală:



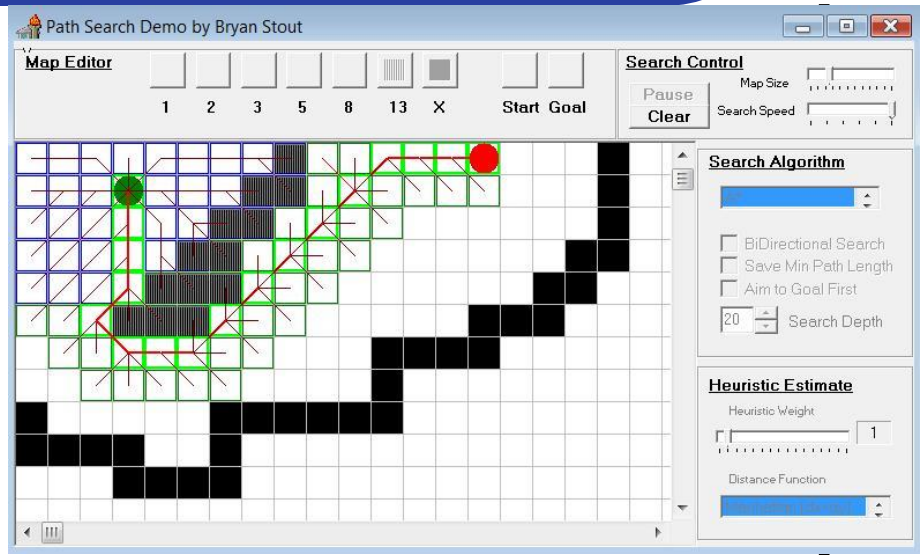
- Două euristici posibile:
  - h_1 = numărul pătrățelelor a căror poziție curentă diferă de poziția finală;
  - $h_1 = \Sigma_{p \in piese}(\delta_p)$, unde $\delta_p = 0$ dacă poziția curentă coincide cu cea finală și $\delta_p = 1$, altfel
  - h_2 = distanța Manhattan = suma distanțelor pe verticală și orizontală între pozițiile curente ale pătrățelelor și pozițiile lor finale
  - $h_2 = \Sigma_{p \in piese}(dist_h_p + dist_v_p)$

Admisibilitate? Monotonie? Dominanță? Care euristică va fi aleasă pentru A*?

Exemple A* cu diverse euristici



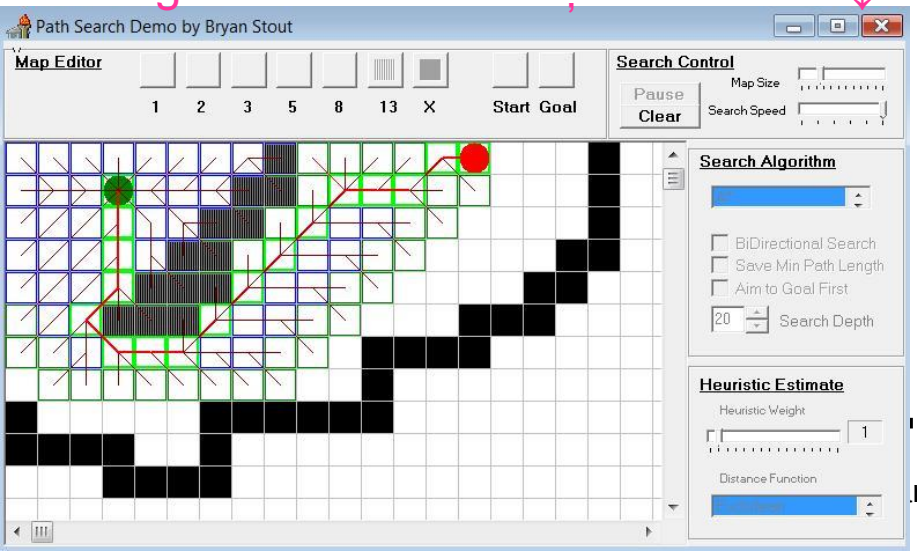
Diagonală↑



Distanța Manhattan ↑

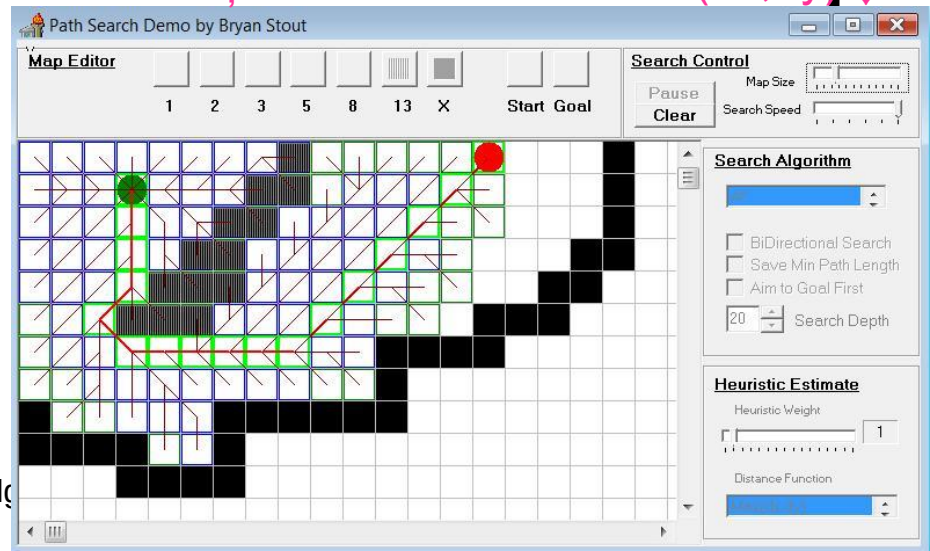
Dominantă

Distanța Euclidiană↓



Max(dx,dy) ↓

euristici



rea Alg

Complexitate A*

- Liniară dacă $|h^*(n) - h(n)| \leq \delta$, unde $\delta \geq 0$ este o constantă.
- Subexponențială, dacă $|h^*(n) - h(n)| \leq O(\log(h^*(n)))$.
- Exponențială, altfel, (dar mult mai bună decât a căutărilor neinformate).
- Mai multe explicații găsiți în Giumale 7.4.4!



ÎNTREBĂRI?



Bibliografie curs 14

[1] C. Giumale – Introducere in Analiza Algoritmilor – cap. 6.1

[2] Cormen – Introducere in algoritmi – cap. 8.3

[3] https://cimec.org.ar/foswiki/pub/Main/Cimec/GeometriaComputacional/TantaloAsymp.pdf

[4] http://www.mersenne.org/

Proiectarea Algoritmilor

Curs 14 - Algoritmi aleatorii



Bibliografie

[1] C. Giumale – Introducere in Analiza Algoritmilor – cap. 6.1

[2] Cormen – Introducere in algoritmi – cap. 8.3

[3] https://cimec.org.ar/foswiki/pub/Main/Cimec/GeometriaComputacional/TantaloAsymp.pdf

[4] http://www.mersenne.org/

Obiective

- Definirea conceptului de algoritm aleator
- Algoritmi Las Vegas
- Algoritmi Monte Carlo
- Analiza algoritmilor aleatori



Algoritmi aleatori

- Micșorăm timpul de rezolvare a problemei relaxând restricțiile impuse soluțiilor.
- Determinarea soluției optime (Las Vegas):
  - Renunțăm la optimalitate (soluția suboptimală are o marjă de eroare garantată prin calcul probabilistic).
- Găsirea unei singure soluții (Monte Carlo):
  - Găsim o soluție ce se apropie cu o probabilitate măsurabilă de soluția exactă.



Algoritmi Las Vegas

- Găsesc soluția corectă a problemei, însă timpul de rezolvare nu poate fi determinat cu exactitate.
- Creșterea timpului de rezolvare → creșterea probabilității de terminare a algoritmului (găsirea soluției optime).
- Timp = ∞ → algoritmul se termină sigur (soluția e optimă).
- Probabilitatea de găsire a soluției crește extrem de repede astfel încât să se determine soluția optimă într-un timp suficient de scurt.



Algoritmi Monte Carlo

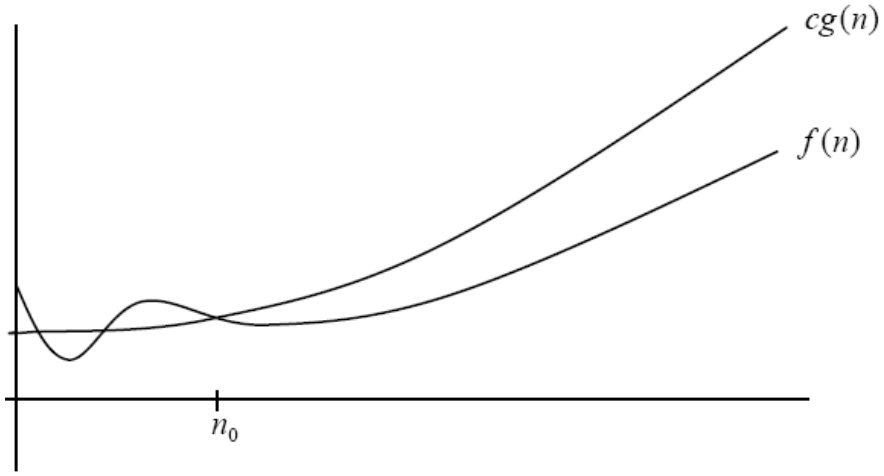
- Găsesc o soluție a problemei care nu e garantat corectă (soluție aproximativă).
- Timp = ∞ → soluția corectă a problemei.
- Probabilitatea ca soluția să fie corectă crește o dată cu timpul de rezolvare.
- Soluția găsită într-un timp acceptabil este aproape sigur corectă.

Reminder – complexitatea algoritmilor

$$O(g(n)) = \{ f(n) \mid \exists c > 0, \exists n_0 > 0, \forall n \geq n_0 : 0 \leq f(n) \leq cg(n) \}$$

- g(n) - limita asimptotică superioară pentru f(n)

$f(n) = O(g(n))$ says:



https://www.scribd.com/document/376943197/TantaloAsymp-pdf

Complexitatea algoritmilor Las Vegas

- Definiția 6.1: Algoritmii Las Vegas au complexitatea $f(n) = O(g(n))$ dacă $\exists\ c > 0$ și $n_0 > 0$ a.î. $\forall\ n \geq n_0$ avem $0 < f(n) < c\ \alpha\ g(n)$ cu o probabilitate de cel puțin $1 - n^{-\alpha}$ pentru $\alpha > 0$ fixat și suficient de mare.

Complexitate algoritmi Monte Carlo

- Definiția 6.1': Algoritmii Monte Carlo au complexitatea $f(n) = O(g(n))$ dacă $\exists\ c > 0$ și $n_0 > 0$ astfel încât:
  - $\forall n \geq n_0,\ 0 < f(n) \leq c\ \alpha\ g(n)$ cu o probabilitate de cel puțin $1 - n^{-\alpha}$ pentru $\alpha > 0$ fixat si suficient de mare;
  - Probabilitatea ca soluția determinată de algoritm să fie corectă este cel puțin $1 - n^{-\alpha}$.

Exemplu algoritm Las Vegas (1)

- Problemă:
  - Capitolele unei cărți sunt stocate într-un fișier text sub forma unei secvențe nevide de linii;
  - Fiecare secvență este precedată de o linie contor ce indică numărul de linii din secvență;
  - Fiecare linie din fișier este terminată prin CR, LF;
  - Toate liniile din secvență au aceeași lungime;
  - Fiecare secvență de linii conține o linie (titlul capitolului) ce se repetă și care apare în cel puțin 10% din numărul de linii al secvenței.
  - Secvențele sunt lungi.☺
- Cerință:
  - Pentru fiecare secvență de linii să se tipărească titlul capitolului (linia care se repetă).

Exemplu algoritm Las Vegas (2)

Users Profiling Using Creativity-based Heuristics

After that, we will try to relate our work to other approaches from the field of creativity research in order to place it in context. Next, we will present the corpus of chat conversations and we will describe the developed application. The most important part of the paper follows, as we present

Users Profiling Using Creativity-based Heuristics

appropriate to the situation in which it occurs, and it must actually be put to some use" (Martindale, 1989). The same idea was proposed by Mayer (1999) who considered that any creative work should be valuable, appropriate, useful, significant, and adaptive (meeting task constraints).



Rezolvare “clasică”

Detectează_linii(fișier)

- **Pentru fiecare** Secv ∈ fișier
  - **Pentru** i **de la** 0 **la** dim(Secv)
    - **Pentru** j **de la** i + 1 **la** dim(Secv)
      - **Dacă** linie(i,Secv) = linie(j,Secv) **atunci**
        - **Întoarce** (linie(i,Secv));

prelucrare secvență

Complexitate – $O(dim(Secv)^2)$

Algoritm Las Vegas pentru rezolvarea problemei

- Secțiunea “prelucrare secvență” se înlocuiește cu următoarea funcție:

- Selecție_linii(n,secv) // n = dim secv
  - **Cât timp**(1) // mereu
    - i = random(0,n-1) // selectez o linie
    - j = random(0,n-1) // și încă una
    - **Dacă** i != j **și** linie(i,Secv) = linie(j,Secv) **atunci** // le compar
      - **Întoarce** linie(i,Secv) // am găsit linia

Analiza algoritmului Las Vegas (I)

- Notații:
  - n = numărul de linii din secvența curentă;
  - q = ponderea liniei repetate în secvență;
  - r = numărul de apariții al liniei repetate: r = n * q / 100;
  - m = numărul de pași necesari terminării algoritmului;
  - P_k = probabilitatea ca la pasul k să fie satisfăcută condiția de terminare a algoritmului;
  - P(m) = probabilitatea ca algoritmul să se termine după m pași.



Analiza algoritmului Las Vegas (II)

- Probabilitatea ca la pasul k linia i să fie una din liniile repetate este r / n.
- Probabilitatea ca la pasul k linia j să fie una din liniile repetate (diferită de i) este (r - 1) / n.
- Condiția de terminare: cele 2 evenimente trebuie să se producă simultan:

$$P_k = r / n * (r-1) / n = q / 100 * (q / 100 – 1 / n)$$

Analiza algoritmului Las Vegas (III)

- Probabilitatea ca algoritmul să NU se termine după m pași:
  - $\Pi_{k = 1 \rightarrow m}(1 - P_k) = \Pi_{k = 1 \rightarrow m} [1 – q / 100 * (q / 100 – 1 / n)] = [1 – q / 100 * (q / 100 – 1 / n)]^m$
- → $P(m) = 1 - [1 – q / 100 * (q / 100 – 1 / n)]^m$
- Pp: $n > 100$; $q > 10\%$
- → $P(m) \geq 1 – [1 – q * (q – 1) / 10.000]^m$

Comparație timp de rulare

- q = 10%:
  - 3500 pași P = 1;
  - 1000 pași – P = 0,9988.
- q = 20%:
  - 1000 pași P = 1.
- q = 30%:
  - 500 pași P = 1.
- Varianta clasică: cazul cel mai defavorabil ~ 5000 pași.

Complexitate algoritm Las Vegas

- Algoritmii Las Vegas au complexitatea $f(n) = O(g(n))$ dacă $\exists$ $c > 0$ si $n_0 > 0$ a.i. $\forall$ $n \geq n_0$ avem $0 < f(n) < c \, \alpha \, g(n)$ cu o probabilitate de cel puțin $1 - n^{-\alpha}$ pentru $\alpha > 0$ fixat si suficient de mare.

- Arătăm că $f(n) = O(\lg(n))$:
  - Notăm: $a = 1 - q * (q - 1) / 10.000$; → $P(m) \geq 1 - a^m$
  - $P(c \, \alpha \, \lg(n))$ = probabilitatea ca algoritmul să se termine în $c \, \alpha \, \lg(n)$ pași;
  - $P(c \, \alpha \, \lg(n)) \geq 1 - a^{c \, \alpha \, \lg(n)} = 1 - n^{c \, \alpha \, \lg(a)} = 1 - n^{-c \, \alpha \, \lg(1/a)}$ pentru că $0 < a < 1$;
  - Dacă alegem $c \geq \lg^{-1}(1/a)$ → $P(c \, \alpha \, \lg(n)) \geq 1 - n^{-\alpha}$ → algoritmul se termină în $\lg^{-1}(1/a) \, \alpha \, \lg(n)$ pași cu o probabilitate $\geq 1 - n^{-\alpha}$ → (definiție) $f(n) = O(\lg(n))$.

Exemplu algoritm Monte Carlo

- Problemă: testarea dacă un număr n dat este prim.
- Rezolvare “clasică”:
- Prim-clasic(n)
  - **Pentru** i **de la** 2 **la** sqrt(n)
    - **Dacă** (n mod i == 0) **întoarce** fals;
  - **Întoarce** adevărat

Complexitate:

O(sqrt(n))

Determinarea numerelor prime - complexitate

- Observație: pentru numere mari – operațiile nu mai durează O(1)!
- → Estimăm numărul de operații în funcție de numărul de biți pe care este exprimat numărul.
- → Prim_clasic – $O(2^{k/2})$ unde k = nr. de biți ocupat de n.



Complexitate nesatisfăcătoare!

- "On September 4, 2006 […] discovering the 44th known Mersenne prime, $2^{32,582,657}$-1. The new prime at 9,808,358 digits is 650,000 digits larger than their previous record prime found last December."
- "On April 12th (**2009**), the 46th known Mersenne prime, $2^{42,643,801}$-1, a 12,837,064 digit number was found" → **February 22, 2018: With no smaller primes found, M(42643801) is officially the 46th Mersenne prime**
- As of October **2009**, 47th Mersenne primes are known. The largest known prime number ($2^{43,112,609} - 1$) is a Mersenne prime. [Wikipedia] → **April 8, 2018: With no smaller primes found, M(43112609) is officially the 47th Mersenne prime**.
- As of October 2014, 48th Mersenne primes are known. The largest known prime number $2^{57,885,161} - 1$ is a Mersenne prime.
- January 7, 2016: the discovery of the largest known prime number, $2^{74,207,281}$-1. The prime number has 22,338,618 digits -- 49th Mersenne prime
- January 3, 2018: 50th Mersenne prime, $2^{77,232,917}$-1, having 23,249,425 digits
- December 21, 2018: 51st Mersenne prime - $2^{82,589,933}$-1, having 24,862,048 digits.

Algoritm aleator (I)

- Teorema 6.1 (mica teoremă a lui Fermat): Dacă n este prim → $\forall\ 0 < x < n$, $x^{n-1} \bmod n = 1$.

- Prim1(n,α) // detectează dacă n e număr prim
  - **Dacă** (n ≤ 1 **sau** n mod 2 = 0) **Întoarce** fals
  - Limit = limită_calcul(n,α) // numărul minim de pași pentru // soluția corectă cu P = 1 - $n^{-\alpha}$
  - **Pentru** i **de la** 0 **la** limit
    - x = random(1, n-1) // aleg un număr oarecare
    - **Dacă** (pow_mod(x,n) ! = 1) **Întoarce** fals // testez teorema // Fermat
  - **Întoarce** adevărat

Complexitate?

Algoritm aleator (II)

- Pow_mod(x,n) // calculează $x^{n-1} \bmod n$
  - r = 1 // restul
  - **Pentru** m **de la** n-1 **la** 0
    - **Dacă** (m mod 2 ≠ 0) // testez dacă puterea e pară
      // sau nu
      - r = x * r mod n
    - x = (x * x) mod n // calculez $x^2 \bmod n$
    - m = m div 2 // înjumătățesc puterea
  - **Întoarce** r

Complexitate:

O(lg(n))

Algoritm aleator (III)

- Problemă: nu putem stabili cu exactitate care este limita de calcul:
  - Nu se poate estima pentru un număr compus n numărul de numere x, 2 < x < n pentru care nu se verifică ecuația;
  - Există numere compuse pentru care orice număr x < n și prim în raport cu n satisface ecuația lui Fermat (ex: nr. Carmichael → 561).
- → Nu știm cu exactitate câte numere sunt!
- → Nu putem calcula probabilitatea!

Altă variantă de algoritm aleator

- Teorema 6.2: Pentru orice număr prim n ecuația $x^2 \bmod n = 1$ are exact 2 soluții:

  $x_1 = 1$ ȘI $x_2 = n - 1$.

- Definiție 6.2: Fie $n > 1$ și $0 < x < n$ două numere astfel încât $x^{n-1} \bmod n \neq 1$ sau $x^2 \bmod n = 1$, $x \neq 1$ și $x \neq n - 1$. X se numește martor al divizibilității lui n.

Algoritmul Miller-Rabin

- Prim2(n,α)
  - **Dacă** (n ≤ 1 **sau** n mod 2 = 0) **Întoarce** fals
  - limit = limita_calcul(n,α)
  - **Pentru** i **de la** 0 **la** limit
    - x = random(1,n-1)
    - **Dacă** (martor_div(x,n)) **Întoarce** fals
  - **Întoarce** adevărat

Complexitate?

Algoritmul Miller-Rabin (II)

- martor_div(x,n) // determină dacă x e
  // martor al divizibilității lui n
  - r = 1; y = x;
  - **Pentru** m **de la** n-1 **la** 0 // puterea
    - **Dacă** (m mod 2 ≠ 0) // putere impară
      - r = y * r mod n
    - z = y // salvez valoarea lui x
    - y = y * y mod n // calculez y^2 mod n
    - **Dacă** (y = 1 **și** z ≠ 1 **și** z ≠ n-1) // verific teorema 6.2
      - **Întoarce** 1
    - m = m div 2 // înjumătățesc puterea
  - **Întoarce** r ≠ 1 // mica teoremă Fermat (x^{n-1} mod n ≠ 1)

Complexitate:

O(lg(n))

Calcularea numărului de pași

- Teorema 6.3: Pentru orice număr n, impar și compus există cel puțin (n-1) / 2 martori ai divizibilității lui n.
- Caz neinteresant: număr prim pentru că oricum algoritmul întoarce adevărat ($P_{corect}(n) = 1$)!
- Caz interesant: număr compus (impar) ($P_{corect}(n) = ?$):
- x = element generat la un pas al algoritmului ($0 < x < n$);
- P(x) = probabilitatea ca numărul **x** generat din cele n-1 posibilități să fie martor al divizibilității;
- $P(x) \geq (n-1) / 2 * 1 / (n-1) = 0.5$;
- $P_{incorect}(n) = \Pi_{1->limit}(1 - P(x)) \leq 1/2^{limit}$;
- → $P_{corect}(n) \geq 1-2^{-limit} = 1 - n^{-\alpha}$ → $limit = \alpha \lg(n)$; →după $\alpha \lg(n)$ pași $P_{corect}(n) \geq 1 - n^{-\alpha}$;
- → Complexitate: $O(\lg^2(n))$ → în funcție de numărul de biți k → Complexitate: $O(k^2)$

Exemplu de utilizare practică

- Quicksort(A, st, dr)
  - **Dacă** st < dr
    - q ← Partiție(A, st, dr)
    - Quicksort(A, st, q - 1)
    - Quicksort(A, q + 1, dr)

- Partiție(A, st, dr)
  - x ← A[st]
  - i ← st - 1
  - **Pentru** j **de la** st **la** dr – 1
    - **Dacă** A[j] ≤ x
      - i ← i + 1
      - **Interschimbă** A[i] ↔ A[j]
  - **Interschimbă** A[i + 1] ↔ A[dr]
  - **Întoarce** i + 1

Cazul defavorabil?

Complexitate ?

Exemplu de utilizare practică (II)

- Problema Quicksort – cazul defavorabil – datele de intrare sunt sortate în ordine inversă.
- Complexitate Quicksort: $O(n^2)$.
- Folosind algoritmi aleatori eliminăm acest caz.



Quicksort-aleator

- Quicksort-Randomizat(A, st, dr)
  - **Dacă** st < dr
    - q ← Partiție-Randomizată(A, st, dr)
    - Quicksort-Randomizat(A, st, q - 1)
    - Quicksort-Randomizat(A, q + 1, dr)

- Partiție-Randomizată(A, st, dr)
  - i ← Random(st, dr)
  - **Interschimbă** A[dr] ↔ A[i]
  - **Întoarce** Partiție(A, st, dr)

INTREBĂRI?

